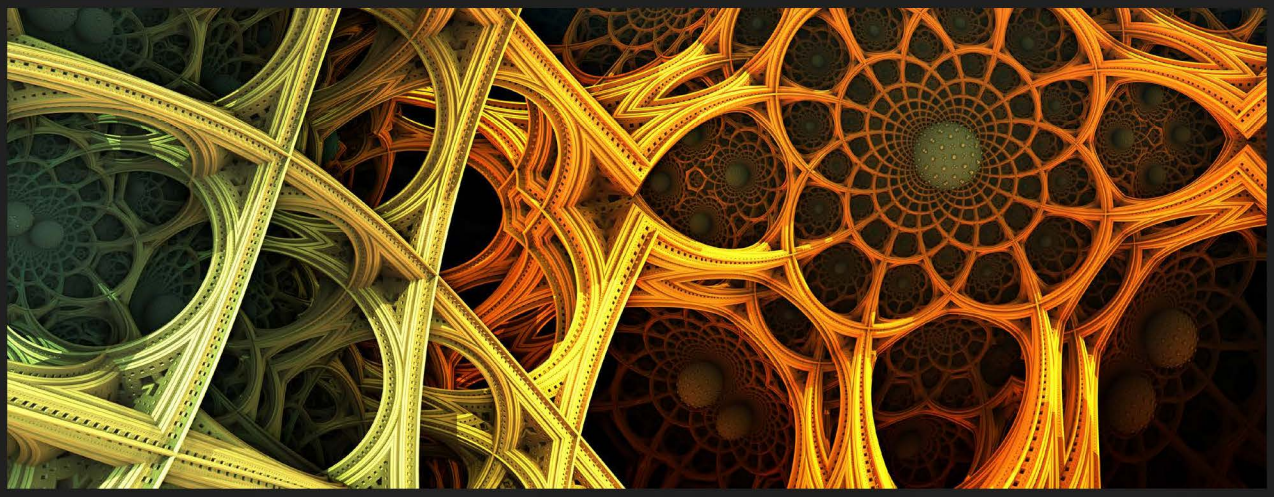


2ND EDITION

Machine Learning Engineering on AWS

Build, deploy, and operationalize LLMs, AI agents, and generative AI systems on AWS

JOSHUA ARVIN LAT

Machine Learning Engineering on AWS

Second Edition

Build, deploy, and operationalize LLMs, AI agents, and generative AI systems on AWS

Joshua Arvin Lat

‹packt›

Machine Learning Engineering on AWS

Second Edition

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.
The author acknowledges the use of cutting-edge AI, such as ChatGPT, with the sole aim of enhancing the language and clarity within the book, thereby ensuring a smooth reading experience for readers. It's important to note that the content itself has been crafted by the author and edited by a professional publishing team.
Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.
Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Sunith Shetty
Relationship Lead: Sanjana Gupta
Project Manager: Shashank Desai
Content Engineer: Nathanya Dias
Technical Editor: Seemanjay Ameriya
Indexer: Manju Arasan
Production Designer: Salma Patel
Growth Lead: Abhishek Kaushik

First published: September 2022
Second edition: May 2026

Production reference: 1270526

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK.

ISBN 978-1-83588-108-8
www.packtpub.com

Contributors

About the author

Joshua Arvin Lat serves as the Vice President of Cybersecurity and AI for Axos. He previously held Chief Technology Officer and Director roles across SaaS platforms, AI automation companies, e-commerce startups, and digital agencies. Because of his proven track record in leading digital transformation within organizations, he has been recognized as one of the winners of the prestigious Orange Boomerang: Digital Leader of the Year 2023 award. Years ago, he led a team that won first place in a global cybersecurity competition for their published research. He is also an AWS AI Hero and has spoken at several international conferences on practical applications of generative AI, software engineering, cybersecurity, and management.

About the reviewers

Sophie Soliven has over 10 years of experience in e-commerce, fintech, and retail operations. She brings extensive APAC operations experience and currently oversees multiple markets across the region, driving performance, scalability, and cost efficiency. Her expertise includes process optimization, risk management, cross-functional leadership, and building data-driven operating models. She holds AWS certifications in AI and machine learning, with a focus on leveraging AI to drive operational excellence and cost optimization. She regularly shares insights on operations, technology, and AI at both local and international forums.

Ike Gabriel "Iggy" Yuson is a technical manager at Elevate Innovations Corp. His expertise lies in implementing cloud-native and serverless applications on AWS. He possesses extensive skills in developing full-stack solutions for both web and mobile platforms. Outside of work, Iggy also actively volunteers in the local community as the User Group Leader of AWS User Group Davao, where he hosts meetups focused on education and networking among AWS enthusiasts and professionals.

Table of Contents

Preface **xi**

Free benefits with your book **xviii**

Chapter 1: A Gentle Introduction to Generative AI and AI Agents on AWS **1**

Technical requirements **2**

Generative AI for the modern machine learning engineer **3**

Exploring foundation models in Amazon Bedrock **7**

Setting up and configuring your SageMaker Studio environment **14**

Configuring IAM permissions for your SageMaker Studio space **26**

Introduction to AI agents with Amazon Bedrock and Strands Agents **31**

Summary **45**

Further reading **46**

Chapter 2: Building AI Agents with SageMaker AI and Bedrock AgentCore **47**

Technical requirements **48**

Deploying a pretrained LLM with SageMaker AI **49**

Building AI agents with Amazon SageMaker AI and Strands Agents **61**

Building AI agents with Amazon Bedrock AgentCore **67**

Getting started with Amazon Bedrock AgentCore • 69

Customizing your agent to interact with a SageMaker AI endpoint • 73

Deploying production-ready agents with Bedrock AgentCore Runtime **80**

Deploying an agent with Bedrock AgentCore Runtime • 80

Cleaning up • 85

Setting up an Amazon Bedrock Knowledge Base **86**

Building a RAG-powered AI agent with Strands Agents **100**

Building a RAG-powered AI agent that interacts with a SageMaker AI inference endpoint 109

Setting up an Inference Endpoint from an Endpoint Configuration • 111

Building the RAG-powered agent • 113

Cleaning up • 119

Summary 120

Further reading 121

Chapter 3: Machine Learning Engineering with Amazon SageMaker AI 123

Technical requirements 124

Setting up and preparing your JupyterLab notebook 125

Preparing a synthetic dataset for binary classification 135

Training an XGBoost binary classifier 147

Deploying an XGBoost model to a real-time inference endpoint 153

Setting up BERT fine-tuning with SageMaker JumpStart 165

Using a smaller dataset for fine-tuning 176

Running the BERT model fine-tuning job 181

Deploying the fine-tuned model to a real-time inference endpoint 186

Summary 190

Further reading 190

Chapter 4: Modernizing Analytics with a Managed Transactional Data Lake 193

Technical requirements 194

Preparing and processing the synthetic data 195

Downloading and preparing the synthetic tabular data • 196

Inspecting the CSV and Parquet files • 202

Uploading the CSV and Parquet files to an S3 bucket • 205

Creating an Amazon S3 table bucket 206

Launching an Amazon EMR cluster with Apache Iceberg installed 209

Setting up an Amazon EMR cluster with Apache Iceberg installed • 211

Updating the EC2 instance profile permissions • 215

Connecting to the EMR cluster • 216
Performing Apache Iceberg queries on S3 tables with Apache Spark 219
Accessing the Spark shell • 219
Creating a namespace • 220
Loading data from a CSV file to a table • 221
Loading data from a Parquet file to a table • 225
Performing time travel queries on S3 tables 228
Summary 233
Further reading 233

Chapter 5: Practical Data Management on AWS 235

Technical requirements 236
Working with AWS Lake Formation permissions 237
Running SQL queries in Amazon Athena 241
Using the AWS CLI to run SQL queries with Amazon Athena • 242
Using Boto3 to run SQL queries with Amazon Athena • 246
Ingesting data into a SageMaker feature store 255
Setting up a SageMaker notebook instance • 257
Preparing the data to be ingested into the feature store • 262
Ingesting data into a feature store • 265
Adding searchable metadata to features 269
Retrieving data from the online and offline feature stores 275
Summary 284
Further reading 284

Chapter 6: Pragmatic Data Processing on AWS 287

Technical requirements 288
Getting started with SageMaker Processing jobs 289
Running your first SageMaker Processing job 293
Preparing the input data and processing script for the back translation job 302
Automating back translation workflows with SageMaker Processing jobs 307
Summary 319

Further reading **320**

Chapter 7: SageMaker AI Model Training and Tuning Capabilities 321

Technical requirements **322**
Setting up a serverless MLflow app **322**
Fine-tuning an LLM on Amazon SageMaker AI **330**
Deploying the Fine-Tuned Model **351**
Performing Hyperparameter Tuning with Amazon SageMaker AI **360**
Deploying the Best-Performing Model from Hyperparameter Tuning **378**
Summary **388**
Further reading **388**

Chapter 8: SageMaker AI Model Deployment Options and Strategies 391

Technical requirements **392**
Preparing your JupyterLab notebook for model deployment **393**
Deploying your model to a real-time inference endpoint **401**
Deploying your model to a serverless inference endpoint **407**
Running batch inference with batch transform **410**
Deploying your model to an asynchronous inference endpoint **413**
Setting up a shadow test with a SageMaker inference endpoint **418**
Creating a shadow test • 420
Inspecting the captured data • 431
Promoting a shadow variant • 435
Cleaning up • 438
Using canary traffic shifting when performing Blue/Green deployments **439**
Configuring a canary deployment in SageMaker AI • 440
Cleaning up • 451
Summary **453**
Further reading **453**

Chapter 9: Automating LLMOps Workflows with SageMaker Pipelines 455

Technical requirements 456
Setting up the project environment and dependencies..... 458
Building and running the Single-Step Fine-Tuning pipeline 469
Building and running the pipeline • 469
Inspecting pipeline execution logs • 480
Building and running the Single-Step evaluation pipeline 484
Setting up and running LLM-as-a-Judge model evaluation • 485
Building and running the pipeline • 495
Inspecting pipeline execution logs • 501
Configuring and running a Two-Step Fine-Tuning and evaluation pipeline..... 504
Building and running the pipeline • 505
Inspecting pipeline execution logs • 517
Preparing the Lambda functions for deployment of a model to an endpoint 522
Setting up a Lambda function that reads a JSON file in S3 • 524
Setting up the Lambda function for checking if an endpoint exists • 528
Setting up the Lambda function for deploying a model to a new endpoint • 531
Setting up the Lambda function for deploying a model to an existing endpoint • 533
Completing the LLMOps pipeline..... 536
Best practices and key considerations for building automated ML workflows..... 548
Operational best practices • 549
Security best practices • 552
Summary 554
Further reading 554

Other Books You May Enjoy 558

Index 561

Preface

A lot has changed since I wrote the first edition of this book. Back then, **generative AI** was still emerging, and most organizations were just starting to explore what **large language models** (**LLMs**) could do and how they would shift the way we build and think about machine learning workflows and pipelines compared to what we were used to. Since then, things have moved quickly. Today, generative AI has moved into production across many industries, with systems powered by **LLMs**, **retrieval-augmented generation**, and **AI agents** becoming an integral part of real-world applications. As you work with these systems, you will quickly realize that building them is no longer just about training and fine-tuning models. It now requires building and operating production-grade systems, automating LLMOps workflows, securing AI applications, and evaluating model, retrieval, and agent performance in real-world environments.

This shift has also created a clear demand for engineers who can combine machine learning expertise with practical cloud engineering skills to build and run these systems in production. **Amazon Web Services** (**AWS**) has also evolved significantly in this space. Using services such as **Amazon Bedrock** and **Amazon SageMaker AI**, you can now manage the end-to-end machine learning lifecycle more effectively, supported by AWS SDKs such as **Boto3** and the **SageMaker Python SDK V3**.

In this second edition, I want to highlight the new approaches, solutions, and capabilities that are shaping how modern AI systems are built on AWS. As you go through the book, you will not just learn concepts. You will see how these ideas come together in practice through extensive hands-on work to build production-ready AI systems. We will start with the foundations of generative AI, LLMs, retrieval-augmented generation, and AI agents on AWS. From there, I will walk you through how to build AI agents using **Bedrock AgentCore** and **Strands Agents**, and explore machine learning engineering practices and workflows using Amazon SageMaker AI. As you move further into the book, you will work with modern data engineering and analytics patterns and practices on AWS, including managed transactional data lakes, practical data management, scalable data processing workflows, and feature store design. We will then go deeper into topics such as model training and fine-tuning, deployment strategies, model evaluation using LLM-as-a-judge techniques, and end-to-end LLMOps automation using **SageMaker Pipelines** and **AWS Lambda**. While you progress through the chapters, I will also highlight best practices and key considerations to help you design systems that are scalable, secure, and reliable in real-world environments.

When I wrote this second edition, my goal was simple. I wanted to help you bridge the gap between theory and real-world implementation. By the end of this book, you will have the understanding and hands-on experience needed to design, build, deploy, and operate modern AI systems on AWS with confidence.

Who this book is for

This book is for AI engineers, data scientists, machine learning engineers, and technology leaders who want to deepen their understanding of machine learning engineering, generative AI, large language models, retrieval-augmented generation, AI agents, and MLOps on AWS. A foundational understanding of artificial intelligence, machine learning, generative AI, and cloud engineering concepts is recommended.

What this book covers

Chapter 1, *A Gentle Introduction to Generative AI and AI Agents on AWS*, explores the fundamentals of generative AI on AWS and shows how to leverage various services and solutions to build AI agents. You will get started by working with foundation models provided through **Amazon Bedrock**, while also covering key concepts and terminology along the way. After setting up a **SageMaker Studio** space, you will build your first AI agent using **Strands Agents**, which interacts with a Bedrock model and is configured with tools to enhance its reasoning and problem-solving capabilities.

Chapter 2, *Building AI Agents with SageMaker AI and Bedrock AgentCore*, shows you how to build agents that interact with a SageMaker AI real-time inference endpoint. You will also learn how to use **Amazon Bedrock Knowledge Bases** and **Amazon S3 Vectors** to build retrieval-augmented generation powered agents that ground responses in external data. By using **Strands Agents** and **Bedrock AgentCore**, you will build agents that integrate model inference with external tools and knowledge retrieval capabilities to deliver more accurate and context-aware applications. You will explore how these building blocks work together to support flexible and production-ready agent-based systems.

Chapter 3, *Machine Learning Engineering with Amazon SageMaker AI*, covers the fundamentals of machine learning engineering on AWS, and shows you how to use Amazon SageMaker AI to implement end-to-end machine learning engineering workflows. You'll start by training and deploying an **XGBoost** model and then move on to fine-tuning a **BERT** model. Through these examples, you will learn how Amazon SageMaker AI simplifies the complexity of building and deploying different types of machine learning models by providing managed capabilities for training, inference, and model lifecycle management.

Chapter 4, *Modernizing Analytics with a Managed Transactional Data Lake*, shows you how to build and work with a transactional data lake using **Amazon S3 tables**. More specifically, you

will learn how to create an Amazon S3 table bucket, launch an **Amazon EMR** cluster with **Apache Iceberg**, and run queries using **Apache Spark**. You will also practice running time travel queries on S3 tables to retrieve previous versions of your data and explore how datasets evolve over time.

Chapter 5, *Practical Data Management on AWS*, explores key AWS services and capabilities for supporting data management in modern analytics and machine learning workloads. You will learn how to work with **AWS Lake Formation** permissions, use **Amazon Athena** to query and process data stored in Amazon S3 table buckets, and ingest data into **Amazon SageMaker Feature Store**. You will also learn how to add searchable metadata to features and retrieve data from both the online and offline feature stores.

Chapter 6, *Pragmatic Data Processing on AWS*, shows you how to use **SageMaker Processing** jobs for running resource-intensive data processing scripts. You will start by running a simplified example to see how a SageMaker Processing job works. You will then download and prepare the input data and the `processing.py` script for the back-translation job to ensure that all prerequisites are in place. You will automate the back-translation process using SageMaker Processing by running your script on a machine learning instance managed by Amazon SageMaker AI. Along the way, you will also learn best practices for designing, managing, and scaling your data processing workflows efficiently and securely.

Chapter 7, *SageMaker AI Model Training and Tuning Capabilities*, explores how to fine-tune a LLM using Amazon SageMaker AI as part of an end-to-end machine learning workflow. You will set up a serverless **MLflow** application to track and manage training experiments, execute a supervised fine-tuning job, and observe how SageMaker AI abstracts the underlying infrastructure while automatically orchestrating the required compute resources. You will then perform hyperparameter tuning to systematically evaluate multiple configurations in parallel and identify the best-performing model based on a defined objective metric. Finally, you will deploy the best-performing model to a real-time inference endpoint for serving predictions.

Chapter 8, *SageMaker AI Model Deployment Options and Strategies*, explores different options and strategies for deploying models in Amazon SageMaker AI. You will work with a pretrained model and deploy it using a real-time endpoint, a serverless endpoint, an asynchronous inference endpoint, and a batch transform job. You will also practice advanced techniques such as **shadow testing** to validate a new model without affecting live traffic, and **canary traffic shifting** to gradually route a portion of production requests to a new model. Along the way, you will learn how to configure data capture to record inference requests and responses for monitoring and evaluation.

Chapter 9, *Automating LLMOps Workflows with SageMaker Pipelines*, shows you how to design and operationalize LLMOps pipelines using **SageMaker Pipelines**. You will begin with single-step pipelines for fine-tuning and evaluation and then extend this foundation into a multi-step

pipeline, which will allow you to explore how individual stages can be composed into structured, automated workflows. As the pipeline evolves, you will introduce additional Lambda-based steps to load model artifacts, extract pipeline outputs, and orchestrate deployment logic. Finally, you will review key design principles and best practices for building production-grade machine learning workflows, with an emphasis on scalability, maintainability, security, and cost efficiency.

To get the most out of this book

You will need an AWS account and a stable internet connection to complete the hands-on exercises in this book. If you are using the digital version of the book, we recommend that you type the code yourself or access it from the book's GitHub repository. This will help you avoid potential errors caused by copying and pasting code.

Download the example code files

This book includes a complete downloadable code bundle containing all the example projects and files used throughout the chapters. We recommend downloading the bundle so you can follow along smoothly and experiment with the examples.

Use the bundle as a practical starting point. Modify it, extend it, and apply what you learn by creating your own variations as you progress through the chapters.

Get the code bundle

If you bought the book directly from Packt:

1. Go to **packtpub.com**
2. Click your **profile picture** and select **Your Orders**
3. Find this book and click **Download Code**

If you bought this book from Amazon or any other channel partner:

1. Go to `packtpub.com/unlock` or scan the following QR code:

2. Search for this book

3. Sign up or log in to your free Packt account
4. Upload your proof of purchase and download the code bundle locally

Usage note: You're free to use and modify this code for personal learning and non-commercial projects.

Download the color images

Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to the *Free benefits with your book section* at the end of the *Preface* to unlock your PDF copy.

Conventions used

There are a number of text conventions used throughout this book.

`CodeInText`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "Define the `wait_for_endpoint()` function that polls the endpoint status until it's in the `InService` state."

A block of code is set as follows:

```
trainer = SFTTrainer(
    model=model,
    training_type=TrainingType.LORA,
    model_package_group=model_package_group,
    training_dataset=dataset,
    s3_output_path=output_path,
    accept_eula=True,
    mlflow_experiment_name=mlflow_experiment_name,
    mlflow_run_name=mlflow_run_name
)
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
from pipeline_wrapper import PipelineExecution
pipeline_execution = PipelineExecution(
    execution=execution
)

# Inspect the final step output(s)
pipeline_execution.outputs()
```

```
# Retrieve the CloudWatch logs for the first step
pipeline_execution.step(1).logs()
```

Any command-line input or output is written as follows:

```
python processing.py
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: "Open a new **Launcher** window by selecting **File** | **New Launcher** from the menu."

> **Note**
>
> Warnings or important notes appear like this.

> **Tip**
>
> Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at `customercare@packt.com` and mention the book's title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit `http://www.packt.com/submit-errata`, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `http://authors.packt.com/`.

Share your thoughts

Once you've read *Machine Learning Engineering on AWS, Second Edition*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.

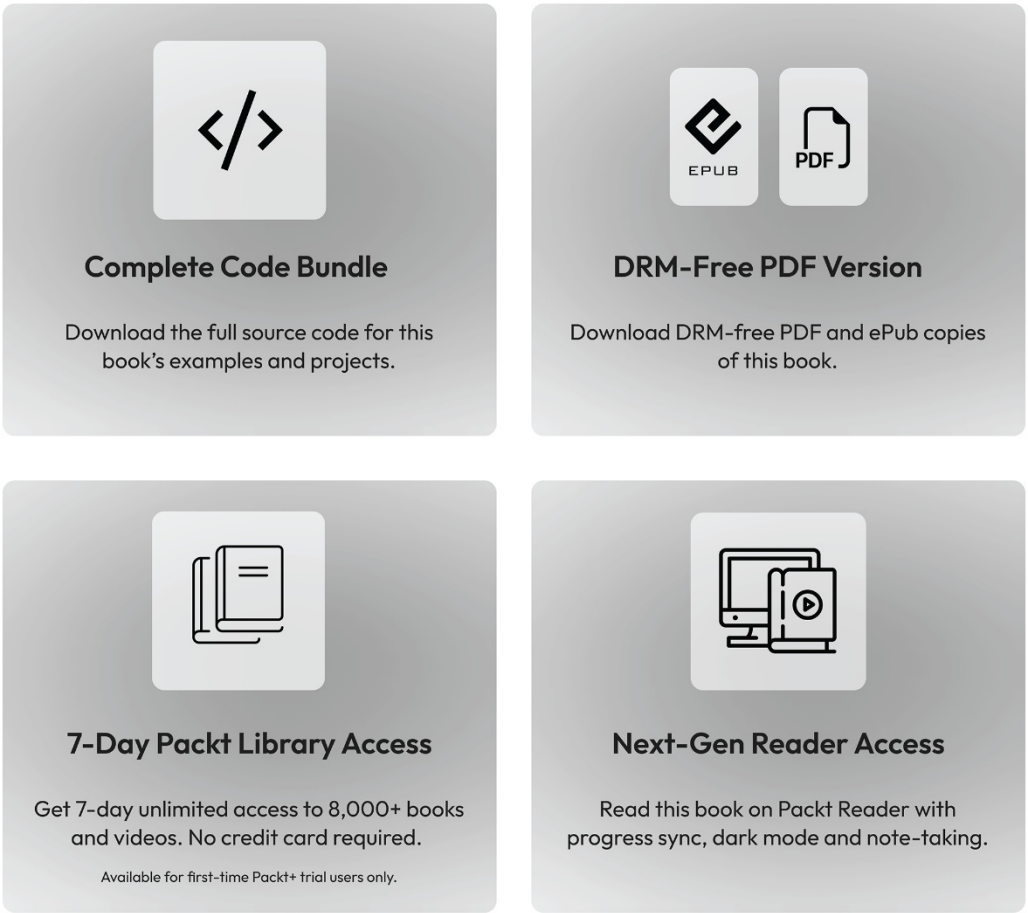
`https://packt.link/r/1835881092`

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Free benefits with your book

This book includes free benefits designed to support your learning and help you apply what you learn effectively. Activate them now for instant access (see the *How to unlock* section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



How to unlock

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

1

A Gentle Introduction to Generative AI and AI Agents on AWS

Imagine that you are playing a video game where you can create and customize your own set of characters. You can think of **generative AI** as a creative assistant that automatically suggests unique abilities, costumes, and names for your custom characters. Given that generative AI is a subset of machine learning, we can then think of **machine learning** (**ML**) as the game engine that powers not only the character customization feature but also every aspect of the game's mechanics, learning, and evolution based on the player interaction data. Continuing this analogy, **machine learning engineering** (**ML engineering**) is like designing, building, and maintaining the game engine to ensure it continuously learns from player data to improve the gaming experience. **AI agents** takes this a step further by acting like autonomous players in the game who can make decisions, perform tasks, and interact with the environment based on what they have learned.

Before we dive deeper into machine learning engineering, it is essential that we have a good grasp of what generative AI is, its use cases, and the concepts relevant to AI agents. This introductory chapter will help us learn several concepts that will be important for the succeeding chapters of this book. One of the fastest ways to learn new concepts is to *learn by doing*. In this chapter, we will explore various examples using a managed service called **Amazon Bedrock** that will help us understand generative AI and the principles behind AI agents. We will also build an AI agent using **Strands Agents** to help you better understand how these concepts come together in practice, demonstrating how agents can reason, interact with models, and perform tasks autonomously.

We will cover the following topics in this chapter:

- Generative AI for the modern machine learning engineer
- Exploring foundation models in Amazon Bedrock
- Setting up and configuring your SageMaker Studio environment
- Configuring IAM permissions for your SageMaker Studio Space
- Introduction to AI agents with Amazon Bedrock and Strands Agents

Do not worry if this is your first time encountering some of these terms, concepts, and solutions, as we will go through each one in more detail later in this chapter. With this in mind, let's get started!

> **Note**
>
> **Your purchase includes a free PDF copy + code bundle**
>
> Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

Technical requirements

We must have the following ready before we jump into the hands-on examples of this chapter:

- **A code editor installed on your local machine** (such as Visual Studio Code or Sublime Text): You'll need this when working with the code and configuration files used throughout the hands-on exercises and examples in this book.
- **A new AWS account**: Creating and using a new AWS account is highly recommended for the hands-on exercises and solutions in this book. This will ensure that your work-related production (or staging) environment resources remain separate and secure. Feel free to create this AWS account by going to `https://aws.amazon.com/free/`. On this page, locate and click on the **Create an AWS Account** button to proceed with the creation of the account. Make sure to read the available AWS documentation along with the relevant FAQ pages to have a better awareness of what is free (and what is not free) when using various services in AWS.

> **Note**
>
> In this book, you will work with various resources, including spaces and inference endpoints used throughout the examples. To better understand the potential costs involved, take a moment to review the pricing information for **Amazon Bedrock**

(`https://aws.amazon.com/bedrock/pricing/`) and **Amazon SageMaker AI** (`https://aws.amazon.com/sagemaker/ai/pricing/`) before proceeding.

Confirm that your currently selected AWS Region is set to the region you want to use for the examples in this chapter. You can change the region at any time by using the region selector in the upper-right corner of the AWS Management Console and choosing the appropriate region (for example, `N. Virginia | us-east-1`) from the dropdown list. While you have many regions to choose from, **us-east-1** is a good starting choice for most examples and exercises.

Generative AI for the modern machine learning engineer

Over the past few years, **generative AI** has advanced significantly. Today, you can generate highly realistic images and text in seconds, extract information from documents, automate content moderation, generate code, and even reason through complex problems across different domains. For the modern **machine learning engineer** (**ML engineer**), a solid understanding of generative AI is no longer optional. In this section, we'll introduce the key concepts and terminology used throughout this book.

Let's start with foundation models. **Foundation models** are large-scale machine learning models trained on massive and diverse datasets, often sourced from multiple domains and internet-scale data. They serve as a versatile starting point for building AI applications without the need to train models from scratch, which can require millions of dollars in compute and data resources. Many foundation models can follow natural language instructions to perform a wide range of tasks, including text generation, summarization, and image creation. Examples of foundation models include **GPT**, **Claude**, **Stable Diffusion**, and earlier models such as **BERT**.

A key reason foundation models are transformative is that they enable **zero-shot** and **few-shot learning**, where models can perform tasks with little or no task-specific training. This represents a major shift from traditional machine learning workflows, which typically require large labeled datasets and training for each new task. As a result, foundation models significantly reduce the effort required to build new AI capabilities and accelerate the development of applications across domains.

> **Note**
>
> Foundation models are typically trained once on large-scale datasets during a computationally expensive training phase. After training, they are primarily used through a process called **inference**, where the model generates outputs in real time based on user inputs. This separation between training and inference is a key reason why managed services can efficiently serve foundation models at scale without requiring users to handle the training process themselves.

Large language models (**LLMs**) are a subset of foundation models trained on large-scale text data and adapted for language-related tasks such as text generation, question answering, translation, and summarization. Continuing the game engine analogy from the start of this chapter, LLMs can be thought of as specialized components within the engine that handle textual content, such as dialog, narratives, and character interactions, by interpreting input and generating meaningful outputs that drive the story forward.

Prompt is an input given to a foundation model, notably LLMs, intended to generate a specific response or output from the model. An example of a prompt would be:

```
Summarize the first chapter of the book "Building and Automating Penetration
Testing Labs in the Cloud".
```

Note that these prompts may vary from simple questions to complex instructions that guide the model to produce or generate the desired response. Modern LLMs often distinguish between different levels of prompts. In addition to the user prompt, there is typically a **system prompt** that defines the model's behavior, tone, and constraints before any user input is processed. This layered structure helps guide the model's responses more reliably and is commonly used in production applications to enforce safety, consistency, and task-specific behavior.

Using an image generator model, prompts can also be used to generate an image similar to what we have in *Figure 1.1*:

Figure 1.1 — An AI-generated image of a cat reading a book

By using the prompt "`cat reading a book`", an image generator model can automatically generate a somewhat realistic image of a cat reading a book. If you are a dog lover, you may try using the prompt "`dog reading a book`" instead!

When a model reads a user's input prompt, it breaks the text down into smaller units called tokens. For example, the tokenized version of the prompt "`Describe a world where gravity does not exist.`" may be composed of around 9 tokens, depending on the tokenizer used by the model. These tokens may include words and punctuation such as "`Describe`", "`a`", "`world`", "`where`", "`gravity`", "`does`", "`not`", "`exist`", and "`.`".

Note

A **token** simply refers to a single atomic unit of text that can be understood and processed by a LLM. A token can be as short as 1 character (for example, a period "`.`") or it could be as long as one word (for example, "`engineer`"). While a token may be associated with an entire word, it could also represent a subword or part of a word. For

> example, the word "`unbelievable`" could be tokenized into "`un`", "`believ`", and "`able`". It is important to note that the resulting set of tokens depends on the tokenizer used, as the final number of tokens is influenced by the specific tokenization rules and algorithms implemented in the tokenizer. Some tokenizers may tokenize "`unbelievable`" into "`un`", "`believ`", and "`able`", while other tokenizers may consider it as a single token (that is, "`unbelievable`"). Behind the scenes, models often convert tokens into numerical representations called **embeddings**. These embeddings capture the semantic meaning of text in a high-dimensional vector space, allowing the model to understand relationships between words and concepts beyond simple token matching. Embeddings are a foundational building block for many GenAI applications, including semantic search, **retrieval-augmented generation** (**RAG**), and recommendation systems.

It's time we take a closer look at prompt engineering. **Prompt engineering** is the practice of carefully designing and refining prompts to guide a model toward generating accurate, relevant, and contextually appropriate responses. For example, when using **Claude**, a foundation model, prompts are often structured in a conversational format that clearly distinguishes between the user and the model, such as:

```
Human: Summarize the first chapter of the book "Building and Automating
Penetration Testing Labs in the Cloud"
Assistant:
```

Such a prompt can help guide the model more effectively toward the intended output. Keep in mind that prompt engineering goes beyond formatting alone. It also involves selecting the right level of detail, providing appropriate context, and adapting the prompt based on the capabilities and limitations of the model being used.

When generating responses, models expose parameters that influence how outputs are produced. For example, settings such as **temperature**, **top-p**, and **maximum token length** control the randomness, diversity, and length of the generated response. These parameters allow developers to fine-tune model behavior depending on whether the use case requires deterministic outputs, creative generation, or concise answers.

Beyond generating text and images, foundation models serve as the core building blocks for more advanced AI applications. By combining prompting techniques, external data sources, and orchestration logic, developers can build systems such as RAG pipelines and AI agents.

These systems extend the capabilities of standalone models by enabling them to interact with external knowledge and tools, making them more useful in real-world applications.

Now that we've covered the core concepts, it's time to take a closer look at machine learning engineering. **Machine learning engineering** involves applying both machine learning and software engineering principles to design, build, and manage production-level ML systems and pipelines. In teams building ML-powered applications, ML engineers are generally responsible for developing and operating the infrastructure used to train, deploy, and monitor models. With the rapid evolution of AI, their role increasingly includes supporting the fine-tuning, orchestration, and integration of LLMs and other models for generative AI applications, while still ensuring ML systems remain robust, scalable, and reliable. In practice, this often involves using services such as **SageMaker AI** to train, deploy, and manage models at scale.

For generative AI projects to succeed, an ML engineer needs to think not only about the *performance* of the deployed model, but also about the *stability*, *scalability*, *security*, *cost-effectiveness*, and *maintainability* of the ML infrastructure, along with *monitoring*, *reliability*, and *compliance*. There is definitely a lot more to consider, and you'll explore these topics in greater depth throughout the book, including how they are implemented using different AWS services for building and operating ML systems at scale. To keep things focused in this introductory chapter, we will use **Amazon Bedrock**, a managed service that abstracts much of the complexity involved in building generative AI applications. As we progress through the chapters, we'll also work with SageMaker AI to explore more advanced use cases and gain hands-on experience with deploying and managing models.

Exploring foundation models in Amazon Bedrock

Amazon Bedrock is a fully managed service that allows you to access, customize, and integrate a wide range of AI models. It simplifies the process of building AI-powered applications and agents by providing seamless access to foundation models and their capabilities, while abstracting much of the underlying infrastructure management. With Bedrock, you can focus on building and delivering your application without worrying about provisioning, scaling, or maintaining the infrastructure. Instead of spending weeks or months developing AI-powered solutions from scratch, you can potentially build production-ready applications in just days or even hours. This significantly accelerates your development cycle and helps you deliver value much faster.

In this section, we will walk through the steps to access and interact with foundation models in Bedrock through the Amazon Bedrock console. More specifically, we'll locate and select **Nova 2 Lite** from the Model catalog, open it in the Chat/Text playground, and submit a few prompts to see how it responds.

Let's go over a few essential services, terms, and concepts that are relevant to this section:

- **Model catalog** — A collection of pre-built AI models that users can choose from for various applications, some of which may offer customization options like fine-tuning for specific use cases. Here, you can explore available foundation models and quickly test them in the playground.
- **Amazon Bedrock Marketplace** — A capability in Amazon Bedrock where users can access and subscribe to AI models from different providers, which allows them to easily deploy and integrate these models into their applications using Bedrock's unified APIs.
- **Modality** — The type of data input or output processed by an AI model, such as text, images, audio, or video. Different models support different modalities, so you'll need to choose one that aligns with your use case. Some models are multimodal, meaning they can handle multiple types of data (for example, combining text and images) within a single interaction, though their supported inputs and outputs may vary.

Having a clear understanding of these concepts, services, and capabilities will make it easier for you to evaluate model capabilities and choose the right model for your specific application requirements.

Note

Before proceeding, make sure that you are signed in to the AWS Management Console.

Let's proceed with exploring foundation models in Amazon Bedrock:

1. Navigate to the Amazon Bedrock console using the search bar, as shown in *Figure 1.2*:

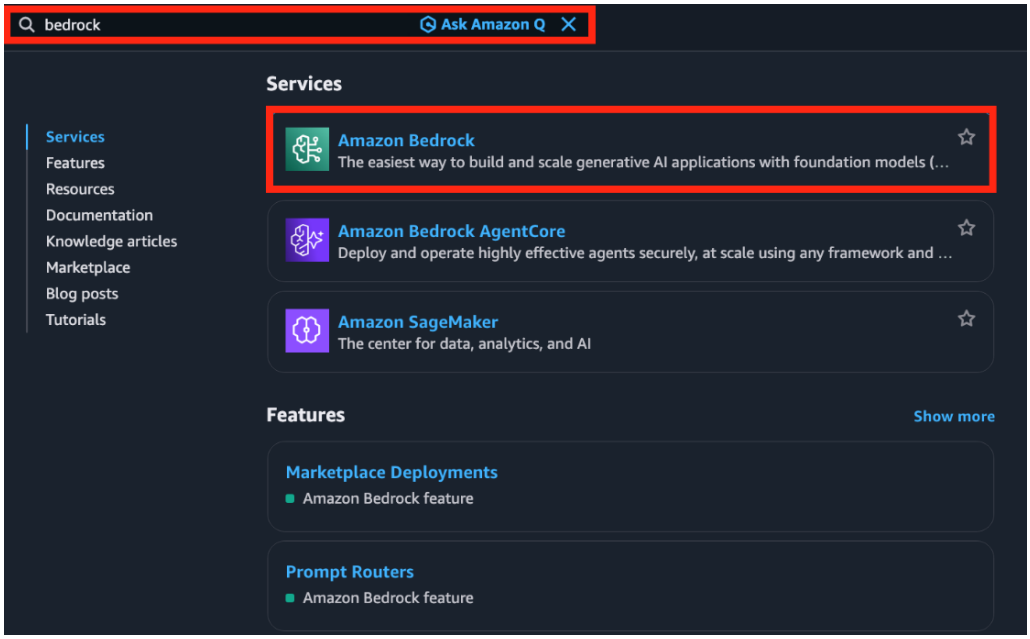


Figure 1.2 — Navigating to the Bedrock console

After typing *bedrock* in the search bar, select the **Amazon Bedrock** service from the list of search results.

2. Next, navigate to the region of choice (**us-east-1**) using the region dropdown in the menu bar. In this chapter, we will create and manage our resources in the **US East (N. Virginia) | us-east-1** region. Make sure to change this to where you want the resources to be created.
3. If the navigation pane (left sidebar) is collapsed, expand it by clicking the hamburger icon (☰) in the top-left corner.
4. In the navigation pane (left sidebar), click **Model catalog** under **Discover**.

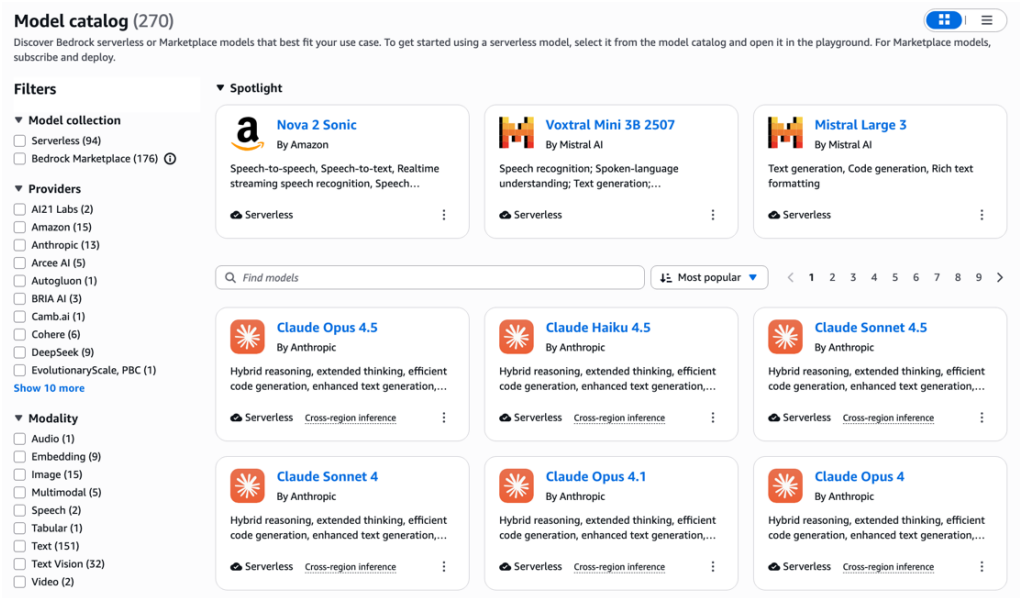


Figure 1.3 — Model catalog

This opens a page displaying the available Amazon Bedrock Serverless and Marketplace models.

5. In the **Filters** panel, locate **Providers** and select **Amazon** to filter the models to only those provided by Amazon. Choose **Nova 2 Lite** from the list of available models.
6. Click **Open in Playground**. You will be redirected to the **Chat/Text Playground** page, where you can interact with the model.

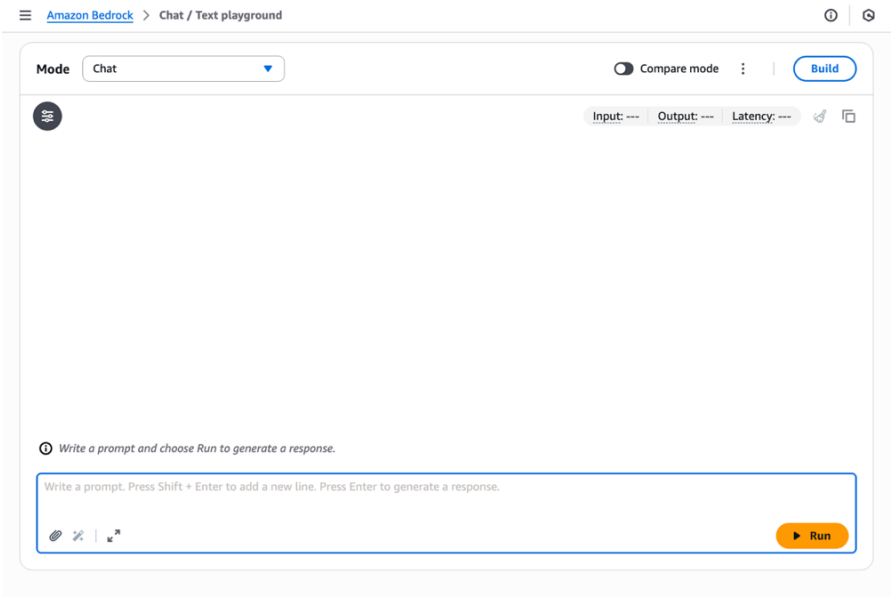


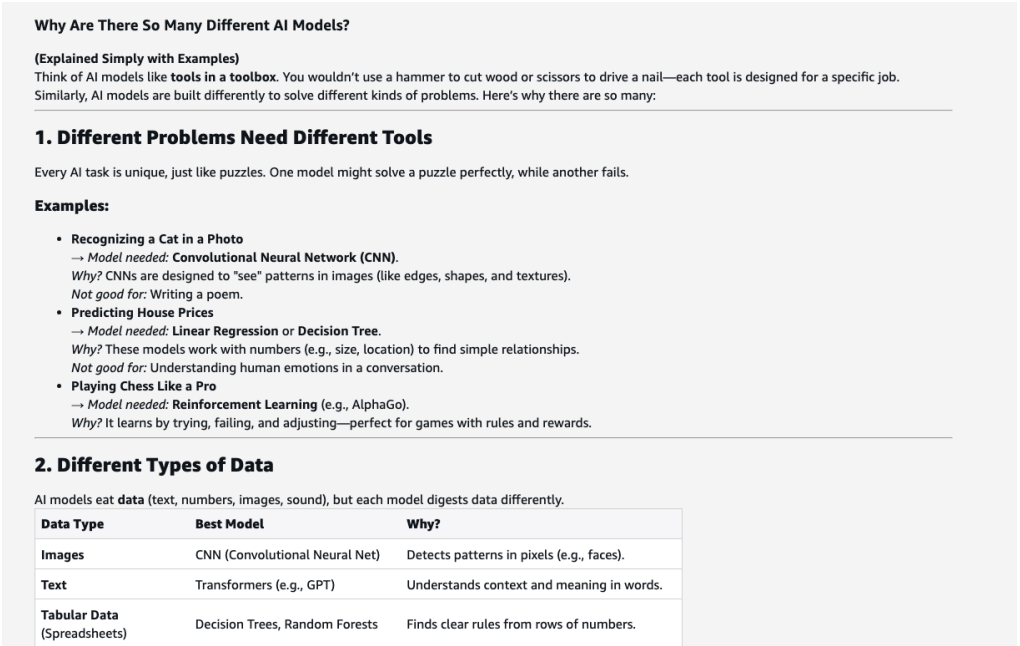
Figure 1.4 — Chat / Text playground

Here, you can test different prompts and see how a model responds to various types of input. You should find a text area where you can enter your prompts, along with a Run button to send them to the model.

7. In the Chat/Text Playground's text area, enter the following prompt:

```
Why are there so many different AI models?
Explain in simple terms, with a few clear examples.
```

8. Click **Run** to submit the prompt and generate the model's response.



Why Are There So Many Different AI Models?

(Explained Simply with Examples)
Think of AI models like **tools in a toolbox**. You wouldn't use a hammer to cut wood or scissors to drive a nail—each tool is designed for a specific job. Similarly, AI models are built differently to solve different kinds of problems. Here's why there are so many:

---

1. Different Problems Need Different Tools

Every AI task is unique, just like puzzles. One model might solve a puzzle perfectly, while another fails.

Examples:

- **Recognizing a Cat in a Photo**
  → *Model needed:* **Convolutional Neural Network (CNN)**.
  Why? CNNs are designed to "see" patterns in images (like edges, shapes, and textures).
  Not good for: Writing a poem.
- **Predicting House Prices**
  → *Model needed:* **Linear Regression** or **Decision Tree**.
  Why? These models work with numbers (e.g., size, location) to find simple relationships.
  Not good for: Understanding human emotions in a conversation.
- **Playing Chess Like a Pro**
  → *Model needed:* **Reinforcement Learning** (e.g., AlphaGo).
  Why? It learns by trying, failing, and adjusting—perfect for games with rules and rewards.

---

2. Different Types of Data

AI models eat **data** (text, numbers, images, sound), but each model digests data differently.

Data Type	Best Model	Why?
Images	CNN (Convolutional Neural Net)	Detects patterns in pixels (e.g., faces).
Text	Transformers (e.g., GPT)	Understands context and meaning in words.
Tabular Data (Spreadsheets)	Decision Trees, Random Forests	Finds clear rules from rows of numbers.

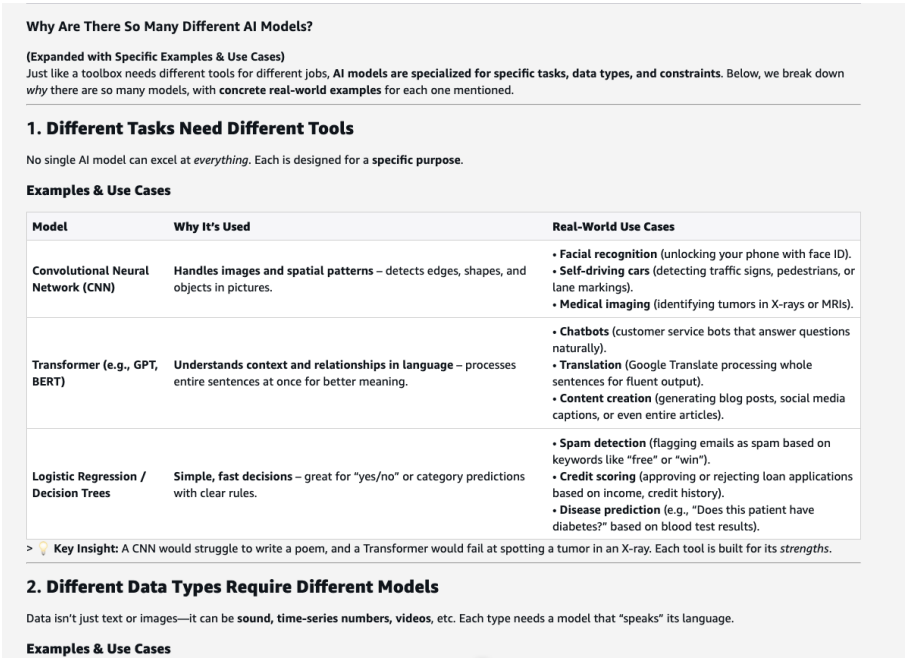
Figure 1.5 — Model's response

Since we asked the model to provide a clear, simple explanation with examples, it responds by explaining that different AI models are designed to tackle different types of problems. For instance, some models excel at processing text, while others are better suited for images or tabular data. Each model is optimized to handle specific types of data, such as natural language, visual information, or sound, making them more effective for their respective tasks.

9. In the text area of the Chat/Text Playground, specify the following prompt:

```
Expand on the previous answer by providing specific examples and use cases
for each model mentioned.
```

10. Click **Run** to submit the prompt and generate the model's response.



Why Are There So Many Different AI Models?

(Expanded with Specific Examples & Use Cases)
Just like a toolbox needs different tools for different jobs, **AI models are specialized for specific tasks, data types, and constraints**. Below, we break down *why* there are so many models, with **concrete real-world examples** for each one mentioned.

1. Different Tasks Need Different Tools

No single AI model can excel at *everything*. Each is designed for a **specific purpose**.

Examples & Use Cases

Model	Why It's Used	Real-World Use Cases
Convolutional Neural Network (CNN)	**Handles images and spatial patterns** – detects edges, shapes, and objects in pictures.	• **Facial recognition** (unlocking your phone with face ID). • **Self-driving cars** (detecting traffic signs, pedestrians, or lane markings). • **Medical imaging** (identifying tumors in X-rays or MRIs).
Transformer (e.g., GPT, BERT)	**Understands context and relationships in language** – processes entire sentences at once for better meaning.	• **Chatbots** (customer service bots that answer questions naturally). • **Translation** (Google Translate processing whole sentences for fluent output). • **Content creation** (generating blog posts, social media captions, or even entire articles).
Logistic Regression / Decision Trees	**Simple, fast decisions** – great for "yes/no" or category predictions with clear rules.	• **Spam detection** (flagging emails as spam based on keywords like "free" or "win"). • **Credit scoring** (approving or rejecting loan applications based on income, credit history). • **Disease prediction** (e.g., "Does this patient have diabetes?" based on blood test results).

> 💡 **Key Insight:** A CNN would struggle to write a poem, and a Transformer would fail at spotting a tumor in an X-ray. Each tool is built for its *strengths*.

2. Different Data Types Require Different Models

Data isn't just text or images—it can be **sound, time-series numbers, videos**, etc. Each type needs a model that "speaks" its language.

Examples & Use Cases

Figure 1.6 — Model's response

The model expands on the previous answer by highlighting real-world use cases for each type of AI model. For example, facial recognition models are used for unlocking your phone with Face ID, while natural language processing models power chatbots for customer service. Translation models are essential for language translation tools, and spam detection models help filter out unwanted emails. Additionally, machine learning models are widely used in credit scoring systems to assess loan eligibility based on financial data.

Note

While the console offers a user-friendly way to interact with AI models, you can also use the API to interact with them programmatically. With the API, you can automate model interactions, seamlessly integrate AI into your applications, and leverage other powerful Bedrock capabilities.

After exploring Amazon Bedrock and using the playground, you can see how this managed service accelerates generative AI development by offering seamless access to powerful models and simplifying experimentation. In addition to providing access to a wide range of foundation

models, Amazon Bedrock offers capabilities that support model customization, agent development, and the use of guardrails to ensure safe and responsible AI usage.

Keep in mind that Amazon Bedrock is just *one* of the many AWS services that allow us to work on AI requirements. The ability of engineers to design, build, and manage AI-powered systems improves as these professionals become more familiar with the services and solutions available in the AWS platform. They may start with AI services that abstract the machine learning engineering work to quickly build AI-powered applications on AWS. Over time, these engineers will make use of the different services, tools, and solutions available on AWS as they become more comfortable dealing with intermediate machine learning engineering requirements. We will cover these in the succeeding chapters of this book!

Setting up and configuring your SageMaker Studio environment

One way to run the examples in this chapter is to set up your local machine, though it requires careful configuration to ensure all notebooks and scripts run correctly. This can work if you have the time and the right hardware, but it often means juggling software versions, installing dependencies, and managing storage. Imagine skipping all that setup and diving straight into exploring and experimenting with various machine learning models. AWS gives you a variety of options for creating cloud-based workspaces specifically designed for AI and ML engineering. One of the easiest and most convenient options is Amazon SageMaker Studio, a fully managed environment that provides everything you need to run code, launch notebooks, and experiment with ML models right away.

In this section, you'll set up and configure your Amazon SageMaker Studio environment which you will use to work on the examples and exercises in the following sections. You will create a SageMaker domain, launch SageMaker Studio, and configure a space with the compute and storage resources necessary to run the notebooks and scripts used throughout this chapter. By the end of this section, you will have a fully functional environment that you can start and stop as needed, while also gaining a better understanding of the key capabilities and concepts introduced along the way.

Before you begin setting up the environment, let's define and discuss a few key services, terms, and concepts relevant to what you'll work with in this section:

- **Amazon SageMaker AI** — A fully managed service that provides a comprehensive set of capabilities to build, train, deploy, and monitor machine learning models at scale. It offers a variety of tools, including interactive notebooks, pre-built algorithms, and managed environments for training, data processing, and inference. With SageMaker AI, you can prepare and process data, experiment with different models, deploy them,

and monitor their performance, all without needing to provision or maintain the underlying compute and storage infrastructure required for ML workflows. By using various capabilities of SageMaker AI, you're able to focus more on model development, data processing, evaluation, and deployment, which significantly shortens the time required to get models into production.

At this point, you might be wondering what some of the capabilities of SageMaker AI are. You can choose which capabilities to use based on your specific use case and workflow. For example, you can use **SageMaker Processing** for large-scale data preprocessing and transformation, **SageMaker Training** jobs for training machine learning models, and **SageMaker HyperPod** to provision resilient clusters for running ML workloads. In addition, you can use features such as **SageMaker Pipelines** for workflow orchestration, **SageMaker Experiments** for tracking and managing experiments, and **SageMaker Model Monitor** for monitoring model performance in production. These are just a few examples, as SageMaker AI offers many more features and capabilities that you can use depending on your needs. You can also use these SageMaker features alongside other AWS services to build end-to-end ML solutions.

- **Amazon Virtual Private Cloud** — A service that allows you to create and configure a private, isolated network in the AWS cloud where you can launch and manage resources. You have full control over networking configurations, including IP address ranges, subnets, and route tables, giving you the flexibility to run your applications and resources in a secure, scalable, and fully private environment. Inside a VPC, you can have **public subnets**, which allow resources to communicate with the internet, and **private subnets**, which keep resources isolated from external access. This lets you design secure, multi-tier architectures with public-facing components separated from internal resources.
- **SageMaker AI Domain** — A logical grouping of resources for organizing user accounts, applications, and their associated components and configurations. A domain consists of an associated storage volume, a list of authorized users, as well as various security, application, policy, and network configurations.
- **SageMaker Studio** — A web-based development environment in SageMaker AI that let's you prepare data, build models, and manage training and deployment workflows.

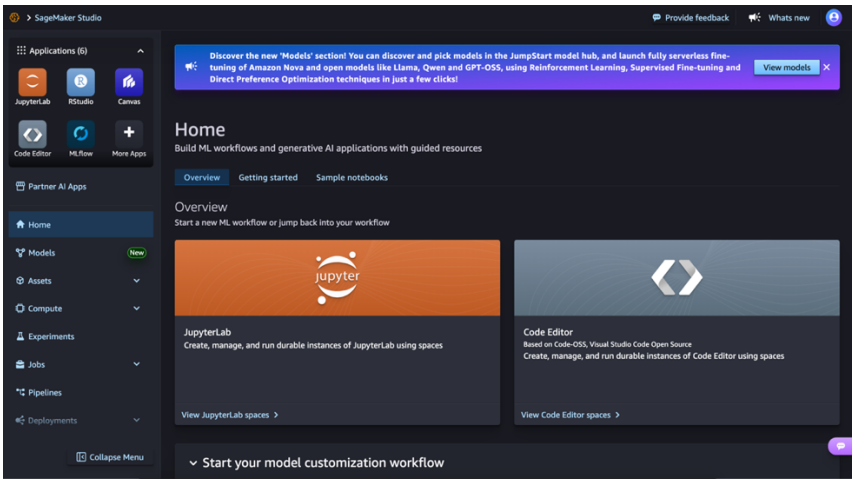


Figure 1.7 — SageMaker Studio

In this interface, you can configure and run spaces, which are managed workspaces that enable end-to-end ML model development, team collaboration, and deployment workflows. The sidebar on the left lets you quickly access different tools, project assets, data files, and the SageMaker AI resources and features you need. It includes **Applications**, **Partner AI Apps**, **Home**, **Models**, **Assets**, **Compute**, **Experiments**, **Jobs**, **Pipelines**, **Deployments**, and **More**. While you can query resources via the API or SDKs, the SageMaker Studio interface makes it easy to browse and access them directly.

- **SageMaker Studio space** — A managed environment designed to handle the storage and resource needs of SageMaker Studio applications. Each space includes a storage volume, an application type, and a runtime image. Essentially, a space acts as a managed container or instance where the application operates with its storage, configuration, and other associated resources. Spaces can be started and stopped as needed, similar to starting and stopping EC2 instances. For example, you can configure a space with 50 GB of storage, an `ml.t3.xlarge` instance type, and a `SageMaker Distribution 3.7.0` image for the runtime environment. This space can be turned on whenever you need to launch new Jupyter notebooks and execute code inside the notebook cells.

- **JupyterLab** — A web-based interactive development environment for working with notebooks, code, and data. It allows you to open and manage multiple notebooks, terminals, and files in separate tabs.

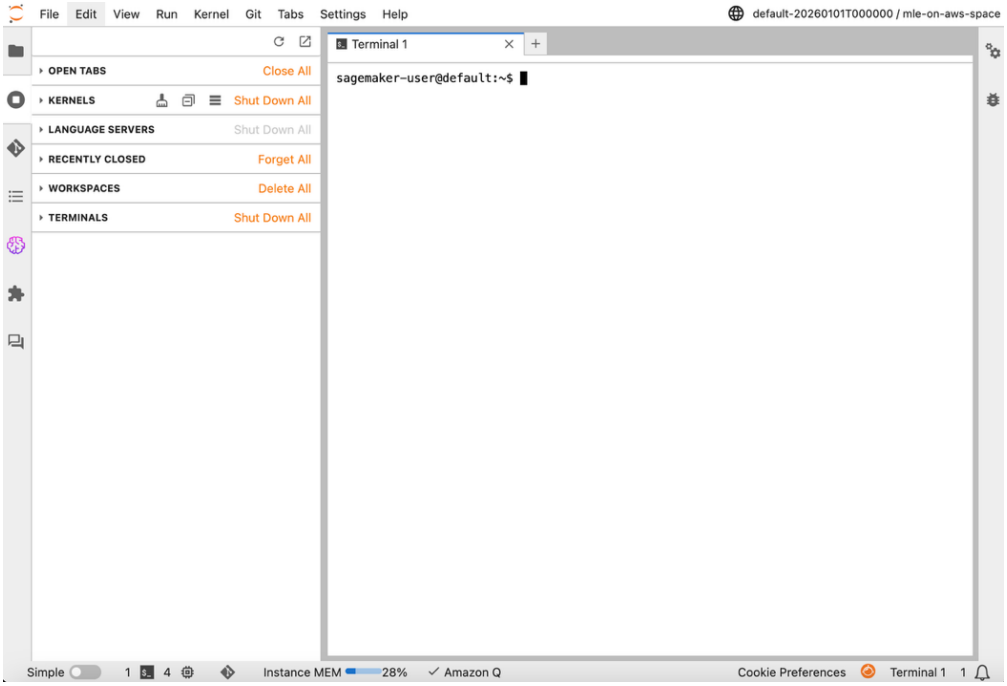


Figure 1.8 — JupyterLab

JupyterLab provides built-in features for file browsing, code editing, and terminal access, which enable you to manage your project resources from a single interface. It also supports interactive data exploration, visualization, and debugging directly within the notebook environment.

- **JupyterLab App Launcher** — A built-in launch page that allows you to quickly create new notebooks, open terminals, and access various development tools.

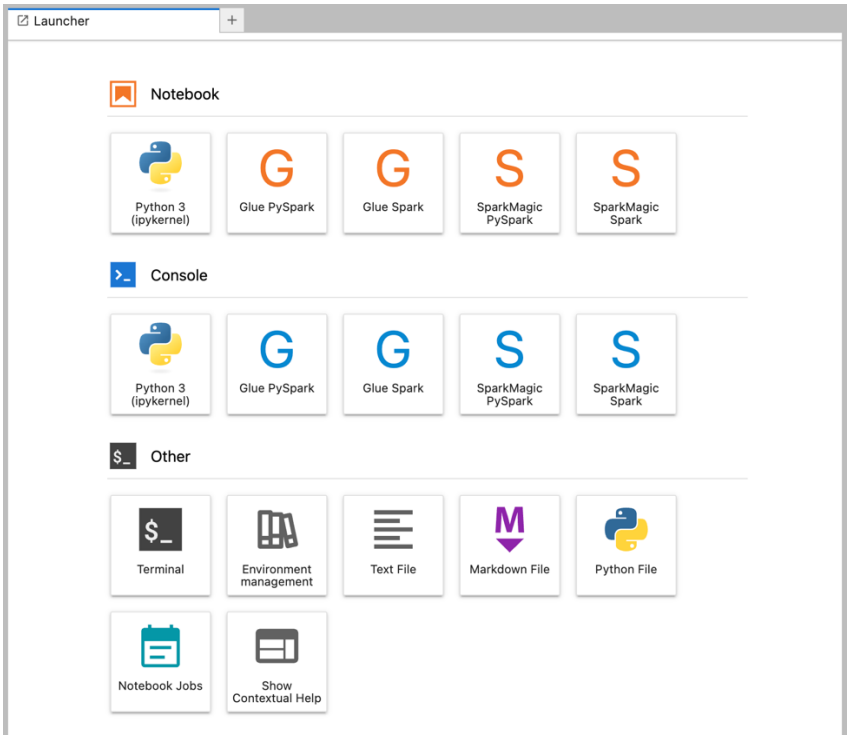


Figure 1.9 — JupyterLab App Launcher

Let's say you want to launch a notebook with a selected kernel or open a terminal session. The JupyterLab App Launcher provides quick access to both options. In addition to this, you can create new text or Markdown files, track and manage notebook jobs, and manage automatic Conda package installations for your environment directly from the launcher.

- **JupyterLab notebook** — An interactive document file (typically with a `.ipynb` extension) that contains executable code cells, Markdown-formatted text, and outputs such as tables and visualizations.

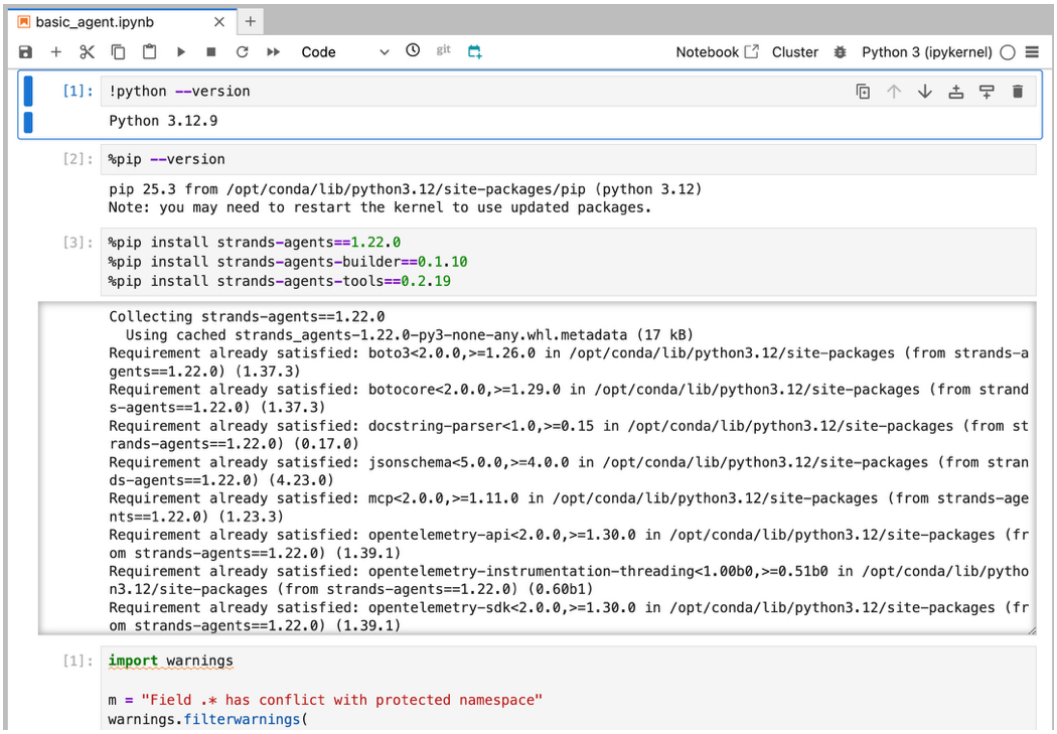


Figure 1.10 — JupyterLab notebook

Inside these notebooks, you can write and execute Python code, run model training jobs, and deploy machine learning models. You can also import and analyze datasets, visualize results, and test different algorithms interactively. For example, let's say you create a new cell in the notebook and write a few lines of Python code to load and preview a dataset:

```
import pandas as pd

# Load a dataset
df = pd.read_csv("transactions.csv")

# Display the first 5 rows
df.head()
```

When you run the cell, the notebook sends the code to the kernel for execution and displays the output directly below the cell.

- **Kernel** — A language-specific process that executes notebook code, manages the runtime environment including variables and memory, and interacts with the JupyterLab interface to handle code execution requests. For example, if you use the **Python 3 (ipykernel)** kernel in your notebook, you should be able to execute Python code in cells and see results interactively within the notebook.

> **Note**
>
> For a detailed overview of Amazon SageMaker Studio, feel free to watch the video *AWS re:Invent 2024 - Accelerate ML workflows with Amazon SageMaker Studio (AIM355)*: `https://www.youtube.com/watch?v=SAeZMA0KaFA`.

Now that you have a better understanding of these key terms and concepts, let's proceed with setting up the SageMaker notebook instance:

1. Navigate to the Amazon SageMaker AI console by typing `sagemaker ai` in the search bar and choosing **Amazon SageMaker AI** from the list of results.

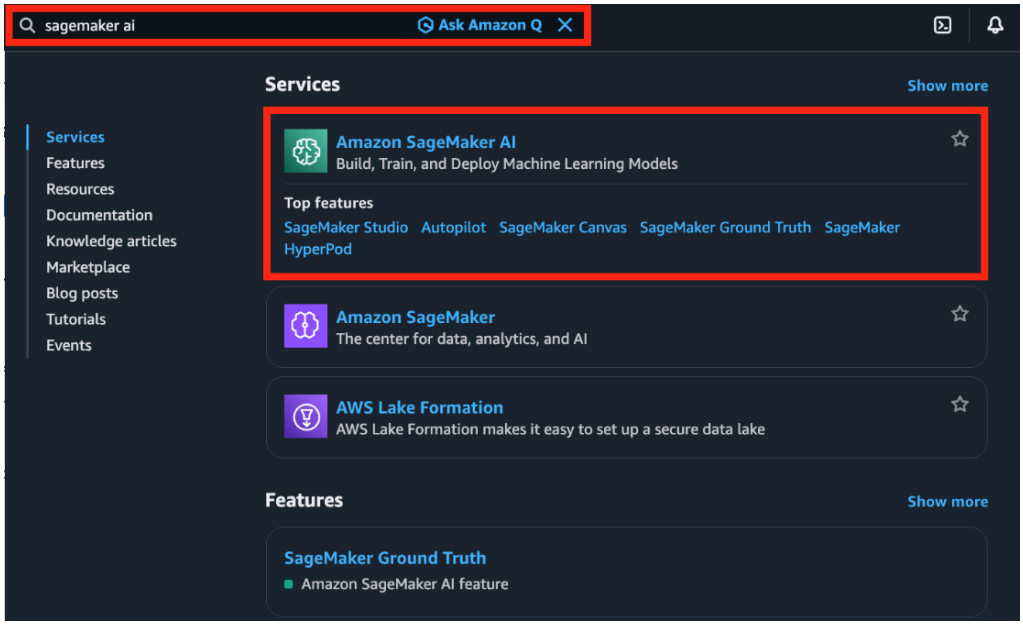


Figure 1.11 — Navigating to the Amazon SageMaker AI console

If you see both **Amazon SageMaker** and **Amazon SageMaker AI** in the search results, make sure to select **Amazon SageMaker AI**. This will redirect you to the Amazon SageMaker AI console, where you can create and manage various machine learning resources such as notebooks, training jobs, and processing jobs.

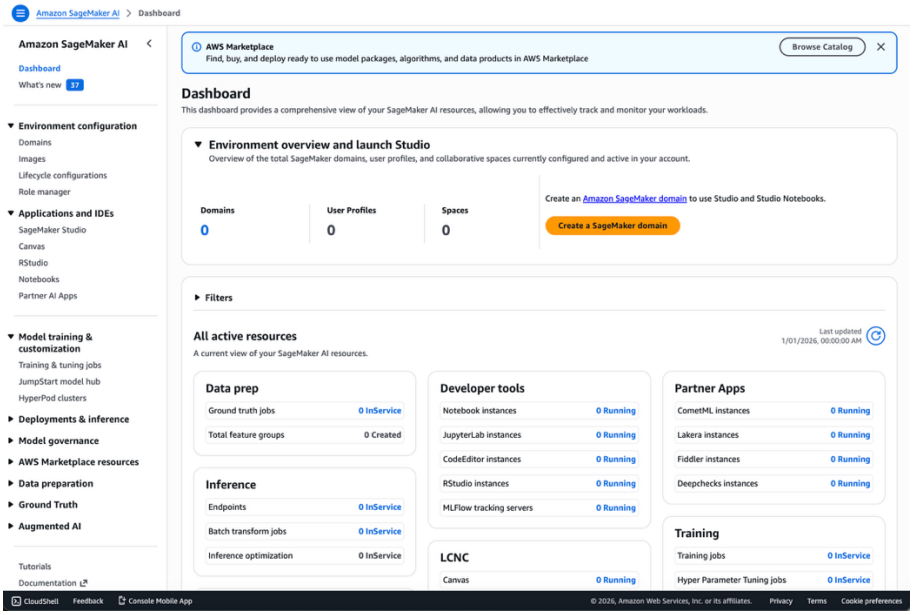


Figure 1.12 — Amazon SageMaker AI console

In the Amazon SageMaker AI console, the navigation pane (left sidebar) includes categories such as **Environment configuration**, **Applications and IDEs**, **Model training and customization**, **Deployment & inference**, **Model governance**, **AWS Marketplace resources**, **Data preparation**, **Ground Truth**, and **Augmented AI**. You should find various features and capabilities of SageMaker AI grouped together under these categories.

2. In the navigation pane, click **Domains** under **Environment configuration**. This should open a page that shows all created domains (if any).
3. Click **Create domain**.
4. Select **Set up for single user (Quick setup)**, then click **Set up** to proceed.

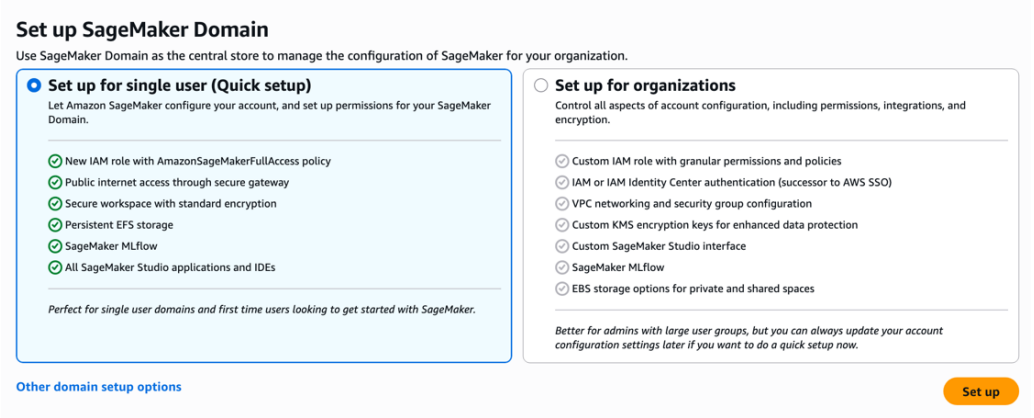


Figure 1.13 — Setting up a SageMaker Domain

When you choose the **Set up for single user (Quick setup)** option, SageMaker automatically provisions several components required to use SageMaker Studio. This includes creating a new IAM role with the necessary permissions, a persistent Amazon EFS storage volume for storing notebooks and files, and a SageMaker domain that manages user access and resources. SageMaker also prepares the required SageMaker Studio applications and IDEs, along with other supporting resources needed to run notebooks, training jobs, and related ML workflows.

Note

If you receive the message "**There is no VPC with available subnets in this region. Create a VPC with subnets in this region before trying again.**", it means that you will first need to create a VPC before you can proceed with the SageMaker Domain setup. To do this, open a new browser tab and access the AWS Management Console. Navigate to the VPC console by typing `vpc` in the search bar and selecting **VPC** from the list of results. In the **VPC dashboard** or **Your VPCs** page, click **Create VPC**. Under **VPC settings**, choose **VPC and more**. Set **Number of public subnets** to `2`, set **Number of private subnets** to `0`, and under **NAT gateways ($)** choose `None`. Finally, click **Create VPC** to create the virtual network required for the SageMaker domain setup. Once the VPC has been created, return to the **Set up SageMaker domain** page and resume the creation of your domain.

5. Setting up your SageMaker domain may take approximately 5 to 10 minutes to complete. Once the domain setup is complete, click **Open Studio**.

> **Note**
>
> When prompted to take a quick tour, click **Watch video** or **Take a quick tour**. Watch the video or complete the tour to familiarize yourself with SageMaker's key features as well as the user interface. To help you get started, you may watch this short 2-minute video titled *Introduction to SageMaker AI | Amazon Web Services*: `https://www.youtube.com/watch?v=1bt11sLo5Pw`.

Clicking **Open Studio** should open the SageMaker Studio interface, where you can access the applications, tools, and resources needed to build, evaluate, and deploy ML models (*Figure 1.14*):

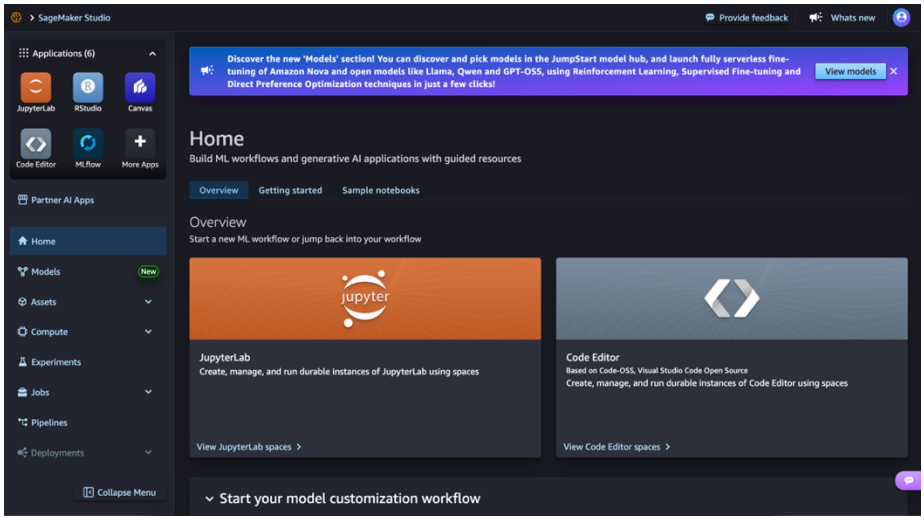


Figure 1.14 — SageMaker Studio

You can think of SageMaker Studio as the control center for your data science, machine learning, and ML engineering work. Here, you can configure and run spaces, which are workspaces where you can launch applications, run notebooks, and work with various ML resources.

6. Select **JupyterLab** from the left sidebar (under **Applications**). You can find this in the upper-left corner of the SageMaker Studio interface.

7. Click the **Create JupyterLab space** button located in the upper-right corner of the page. On the **Create JupyterLab space** page (or window), enter `mle-on-aws-space` as the space name, and then click **Create space**.

> **Note**
>
> Leave the **Sharing** configuration set to **Private**. With this configuration, only you can access and use the space.

8. Configure the space by selecting the **ml.t3.xlarge** instance type and setting the storage size to **50** GB.

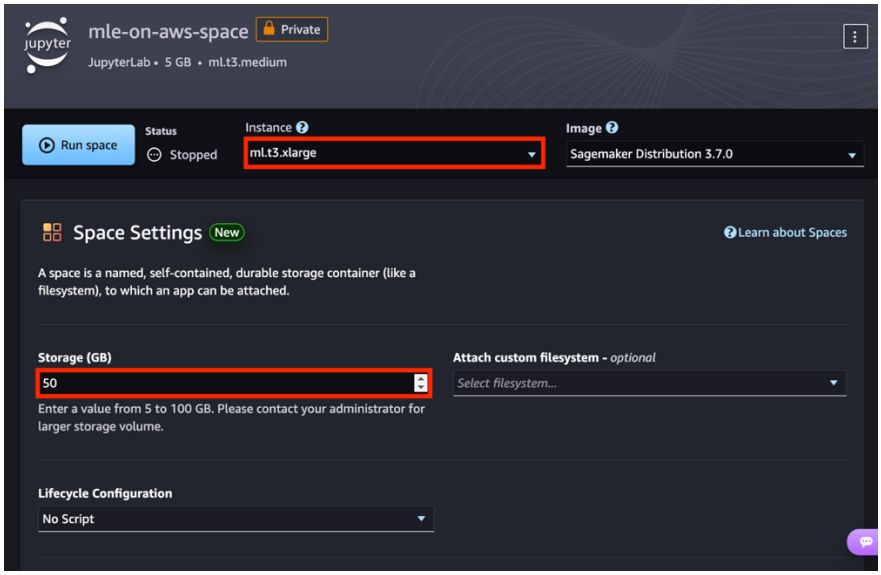


Figure 1.15 — Configuring the space

By specifying `ml.t3.xlarge` for the instance type and `50` GB of storage, you should have enough compute and storage resources to run the examples in this chapter. Compared to the default `ml.t3.medium` instance (2 vCPU, 4 GiB), the `ml.t3.xlarge` instance (4 vCPU, 16 GiB) provides more vCPUs and memory, which helps you run notebooks and data processing tasks more smoothly, especially when working with larger datasets or more resource-intensive operations. Similarly, you increased the storage volume configuration from the default `5` GB to `50` GB. This additional storage ensures that you have enough space for datasets, notebooks, and intermediate files generated while running the examples.

9. Click **Run space** to start the JupyterLab space. It may take approximately 5 to 10 minutes for the space to start. Once the space is running, you can proceed with launching the JupyterLab application.

> **Note**
>
> Make sure to stop the space once you have finished working on the examples in this chapter or whenever you are not actively using it. Stopping the space shuts down the compute resources and helps you manage costs.

10. Once the space is running, click **Open JupyterLab** to launch the JupyterLab interface (*Figure 1.16*).

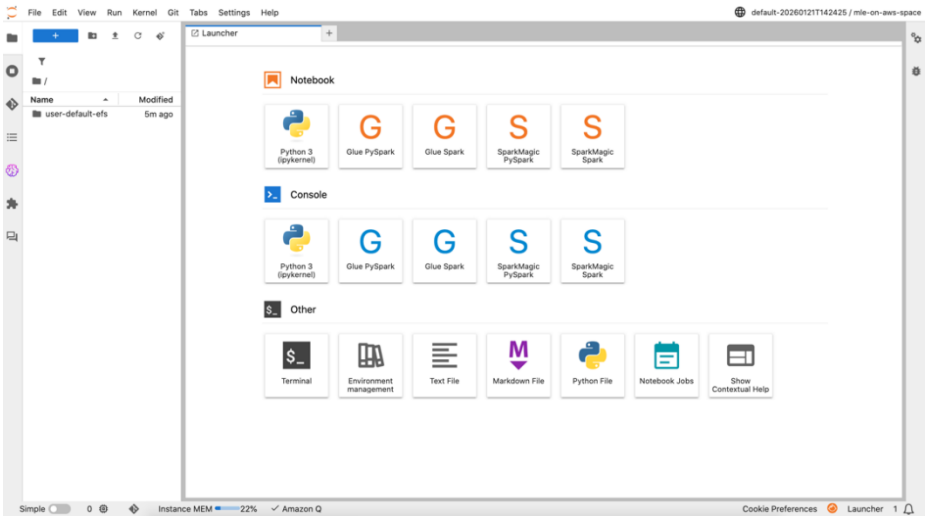


Figure 1.16 — JupyterLab Interface

Here, you can start working with notebooks, manage files, and access the development tools available in the environment. The Launcher should already be open, allowing you to quickly create a new notebook, open a terminal, or manage other resources.

With your SageMaker Studio space set up and running, you now have a managed workspace ready for AI and ML engineering. You can launch notebooks, experiment with models, and explore datasets interactively without worrying about infrastructure management. In the upcoming section, you'll see how to configure IAM permissions for your space that define what you can do within this environment.

Note

Don't forget to turn off your SageMaker Studio space once you're done with the examples or whenever you're not actively using it. You can always restart the space later when you're ready to continue working without losing any of your notebooks or configurations.

Configuring IAM permissions for your SageMaker Studio space

In this section, you will set up the IAM permissions needed for your SageMaker Studio space. This will allow your environment to access and interact with different AWS services. First, you will go to the Amazon SageMaker AI console to locate the space execution role. Then, you'll copy the role name and go to the IAM console to attach the required permissions. By attaching these permissions, you enable your SageMaker Studio space to interact with resources from Amazon SageMaker AI, Amazon S3, and Amazon Bedrock. Once the permissions are applied, your SageMaker Studio space will be fully configured to interact with the relevant resources for this chapter.

Before we configure the IAM permissions of our Studio space, let's define and discuss a few key services, terms, and concepts:

- **AWS Identity and Access Management (IAM)** — A service that enables you to manage access to AWS services and resources securely. With IAM, you can create users, assign roles, and specify permissions for those users to control who can access which resources and what actions they can perform. IAM provides fine-grained control over AWS resources, helping you to secure and manage access to your cloud infrastructure effectively.
- **IAM role** — An identity within your AWS account that has specific permissions attached to it. These roles can be assumed by AWS services, users, or other trusted entities, enabling them to perform actions on your behalf. IAM roles allow you to grant permissions for specific tasks without directly linking those permissions to a particular user or service, which is especially useful for cloud resources that need temporary access to various AWS services and resources. In addition to IAM roles, there are other entities like IAM users (specific individuals or systems with permanent credentials) and IAM groups (collections of users with shared permissions), both of which are used to manage access in a more granular or collective manner.

- **Permission** — Defines what actions can be performed on resources within your AWS environment. Permissions determine whether a particular action, such as reading from or writing to an S3 bucket, is allowed. These permissions are attached to IAM roles, users, or groups and help ensure that only authorized entities can access and interact with your resources. Permissions can be as broad or as granular as necessary, allowing you to fine-tune security and enforce the principle of least privilege.
- **Policy** — Defines permissions and is associated with IAM roles, users, or groups. Policies describe the specific actions allowed or denied on AWS resources and can specify conditions under which those actions are permitted. Policies are essential for managing security in AWS, as they control what users or services can do with the resources in your account. Policies are written in JSON format and can be either managed policies (AWS-predefined) or inline policies (custom, attached directly to a user, group, or role).

When working with IAM, it is essential to understand who or what has access to your AWS resources, what actions they can perform, and under what conditions. This level of control is crucial for maintaining the security of your AWS environment. Properly configuring IAM roles, permissions, and policies ensures that only authorized users and services can access and modify your resources, preventing unauthorized actions. Whether it's granting temporary access to a service like SageMaker Studio or controlling who can read or write to an S3 bucket, this fine-grained access control is fundamental for managing and securing your cloud infrastructure.

With these in mind, let's proceed with configuring the IAM permissions for our SageMaker Studio space:

1. Switch to the browser tab where the AWS Management Console is open. If you are not already in the Amazon SageMaker AI console, type `sagemaker ai` in the search bar and select **Amazon SageMaker AI** from the list of results.
2. In the navigation pane (left sidebar), click **Domains** under **Environment configuration**. This should open a page that shows all created domains (if any). You should find the domain you created in the previous section listed on this page.
3. From the list of domains, select the domain you created in this chapter and click **View** to open the domain details page.

> **Note**
>
> The domain name should start with `QuickSetupDomain` because you selected **Quick Setup** when creating the SageMaker domain.

4. Scroll down to **Authentication and permissions** and locate the **Space execution role**. Copy the Space execution role name (for example, `AmazonSageMaker-ExecutionRole-...`) and keep it in a code editor on your local machine for use in upcoming steps.

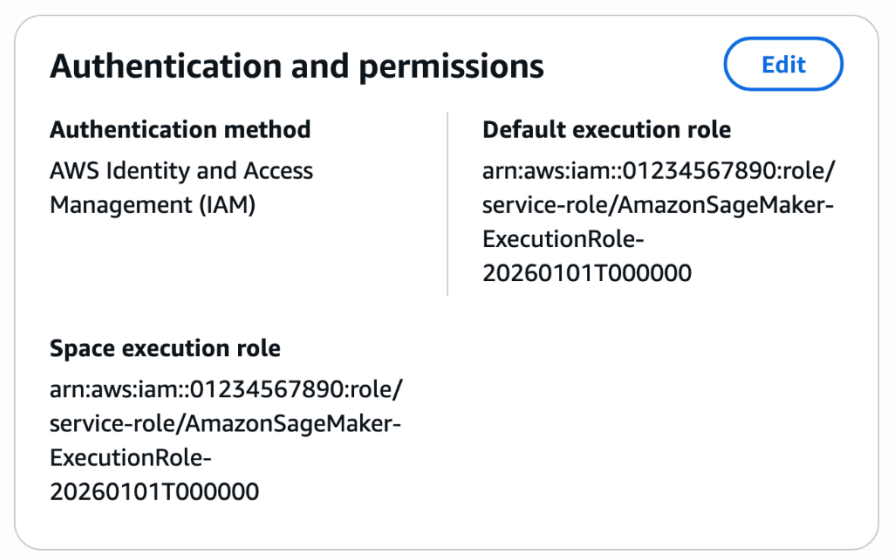


Figure 1.17 — Locating the space execution role

The space execution role is an IAM role that SageMaker AI uses to grant your space permissions to access various AWS resources and other services. The permissions granted by the role depend on how it is configured, which can allow full access or limit actions to specific resources and services.

Note

An IAM role is identified by an **Amazon Resource Name** (**ARN**), which is composed of several parts, including the partition (`arn:aws`), the service (`iam`), the account ID, and the resource path that contains the role information. For example, the ARN `arn:aws:iam::01234567890:role/service-role/AmazonSageMaker-ExecutionRole-20260101T000000` includes the resource type (`role`), an optional path (`service-role/`), and the actual role name (`AmazonSageMaker-ExecutionRole-20260101T000000`). When following the upcoming steps, make sure that you copy only the role name (`AmazonSageMaker-ExecutionRole-20260101T000000`) and not the full ARN.

5. Open a new browser tab and navigate to the AWS Management Console. In the AWS Management Console, type `IAM` in the search bar, and then select **IAM** from the list of results.

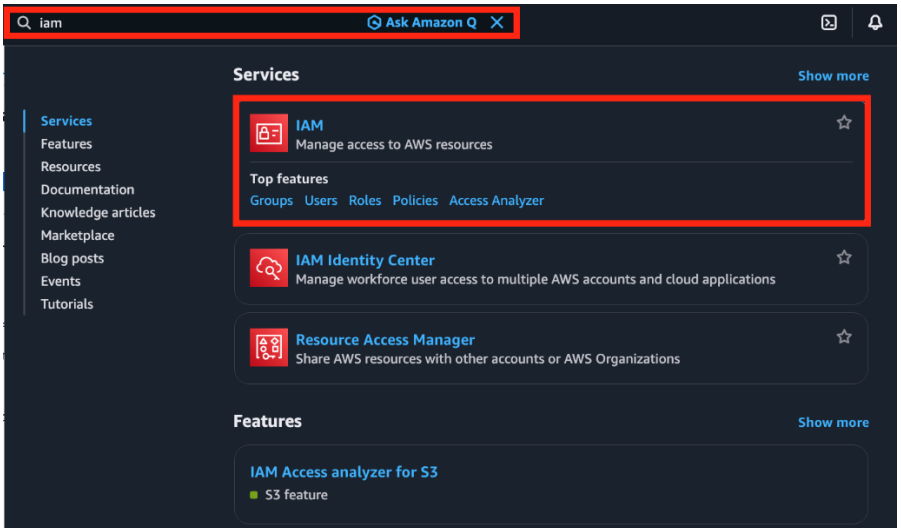


Figure 1.18 — Navigating to the IAM console

In the IAM console, you can manage users, roles, and permissions to securely control access to AWS resources.

6. In the IAM console, select **Roles** (under **Access Management**) from the navigation pane (left sidebar).
7. Use the search box to locate the space execution role you copied earlier to your code editor and select it from the list.

> **Note**
>
> When searching in the IAM console, you do not need to enter the full ARN. Instead, search using the role name portion, which typically begins with `AmazonSageMaker-ExecutionRole--`.

8. Click the role name (under the **Role name** column) to open its details page.
9. Under **Permissions | Permission policies**, click **Add permissions**, and choose **Attach policies** from the dropdown menu. Under **Other permissions policies**, locate and attach the following policies: `AmazonBedrockFullAccess`, `AmazonS3FullAccess`,

`BedrockAgentCoreFullAccess`, and `IAMFullAccess` policies. Click **Add permissions** to apply the changes.

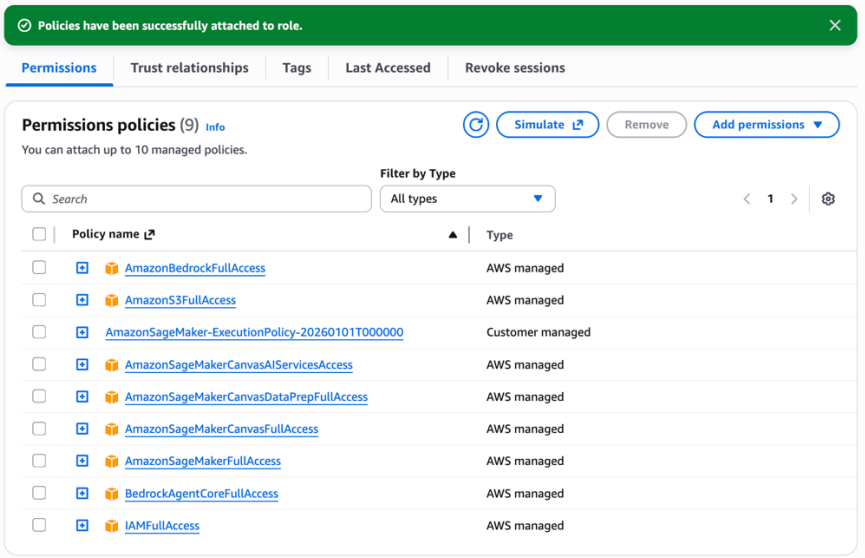


Figure 1.19 — Policies have been successfully attached to the role

You should see the message **Policies have been successfully attached to role.** confirming that the selected permission policies have been attached to the space execution role.

Note

Keep in mind that these policies are overly permissive and grant broad access to resources. In a production environment, you should follow the principle of least privilege and attach only the permissions necessary for your specific tasks.

With the IAM permissions successfully attached to your SageMaker Studio space execution role, your environment now has the required access to interact with AWS services like Amazon S3, Amazon Bedrock, and Amazon SageMaker AI. Configuring these permissions ensures that your notebooks, tools, and agents can securely read, write, and manage resources without running into access errors. Going forward, you can focus on building and deploying models confidently, knowing that your workspace has the proper privileges to perform essential ML workflows.

Introduction to AI agents with Amazon Bedrock and Strands Agents

In this section, you'll build a basic AI agent using Strands Agents and Amazon Bedrock. You'll configure the agent to use a supported Bedrock model and invoke it to answer queries so you can see how the agent performs reasoning and calculations in real time.

To help you understand what an agent is, you can think of it like a smart assistant that sees what's going on, decides on the best action, carries it out, and adjusts based on feedback. For example, you could ask the smart assistant, *"What's the current time?"*, *"How many days until my next birthday?"*, or *"Send a birthday greeting to my friend via email"*, and it would use its tools and integrations to answer, calculate, and take action. More formally, an **AI agent** is a system or application that can observe its environment, make decisions, and act autonomously to achieve specific goals. Many modern agents can also interact with models, tools, and APIs to gather information, perform tasks, and provide responses. If you're wondering what makes these agents especially powerful, it is their ability to leverage external tools and connect to APIs, databases, or services to extend their capabilities beyond their built-in reasoning.

One of the several solutions available to help you build and run AI agents is **Strands Agents**. It let's you build, manage, and deploy AI-powered agents that integrate models, prompts, and various tools with just a few lines of code. You can think of Strands Agents like a game engine for AI. Just as a game engine provides prebuilt components, physics, and controls so developers can focus on creating gameplay, Strands Agents gives you the ready-made building blocks so you can focus on designing your agent's behavior. With everything already connected and configurable, you can quickly assemble, test, and deploy agents without starting from scratch.

> **Note**
>
> Strands Agents is one of several options available for building and running AI agents. Depending on your use case, you might choose other frameworks, libraries, or managed services that support agent-based architectures. In this chapter, you'll use Strands Agents because it provides a straightforward way to build agents, integrate tools, and interact with models available through Amazon Bedrock and Amazon SageMaker AI.

In LLM-powered agents, a LLM acts as the reasoning engine, helping the agent understand inputs, plan actions, and generate intelligent responses. By combining this reasoning ability with autonomous actions, these agents can think on the fly, adapt to changing situations, and

handle complex tasks without constant guidance. To help you better understand what they can do, here are some examples of AI agents:

- **Scheduling Assistant Agent** – Monitors calendars, plans meetings, sends invites, and follows up on updates. You can use this agent to automatically find open time slots for your team or reschedule meetings when conflicts arise, so you never have to manually coordinate schedules. It can even remind participants about upcoming appointments, helping you stay organized effortlessly.
- **Customer Support Agent** – Answers customer queries, retrieves knowledge base articles, and escalates tickets when necessary. You can rely on this agent to handle common questions from your users, freeing up your support team to focus on more complex issues. It can also provide personalized responses, so your customers get accurate help quickly.
- **Data Analysis Agent** – Collects data from multiple sources, runs analysis, and generates reports or summaries. You can ask this agent to pull data from spreadsheets, databases, or APIs and get insights without writing any code yourself. It can highlight trends, spot anomalies, and even create charts so you can make informed decisions faster.
- **Automation Agent** – Executes multi-step workflows, invoking APIs or tools to perform repetitive tasks automatically. You can set up this agent to handle routine tasks like sending emails, updating CRM records, or syncing files across systems, saving you hours of manual work. It can chain multiple actions together, so complex processes run smoothly with minimal effort from you.
- **Monitoring and Alerting Agent** – Observes system metrics or logs and notifies stakeholders or takes corrective actions based on conditions. You can use this agent to keep an eye on servers, applications, or business metrics, so you're alerted immediately if something goes wrong. It can even trigger automated fixes or escalate issues, helping you stay on top of problems before they impact users.

While you might be tempted to jump straight into building complex agents, in this section, we'll start with a simple one that uses just two tools: a calculator tool (`calculator()`) and a current time tool (`current_time()`). With this agent, you can ask questions like, "*I was born on June 1, 1998, and I am 10 years older than my brother. How old is my brother?*" Without tools like the `calculator()` and `current_time()`, even a powerful LLM would most likely make a mistake, since it does not have access to the current date and time and cannot perform exact calculations reliably. Once you've seen this agent in action, you'll be ready to create more advanced agents that integrate multiple tools and handle complex tasks.

With these in mind, let's proceed with building our first AI agent:

1. Switch back to the browser tab where the JupyterLab interface is open. In the **Launcher**, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). This will open a new notebook similar to what is shown in *Figure 1.20*.

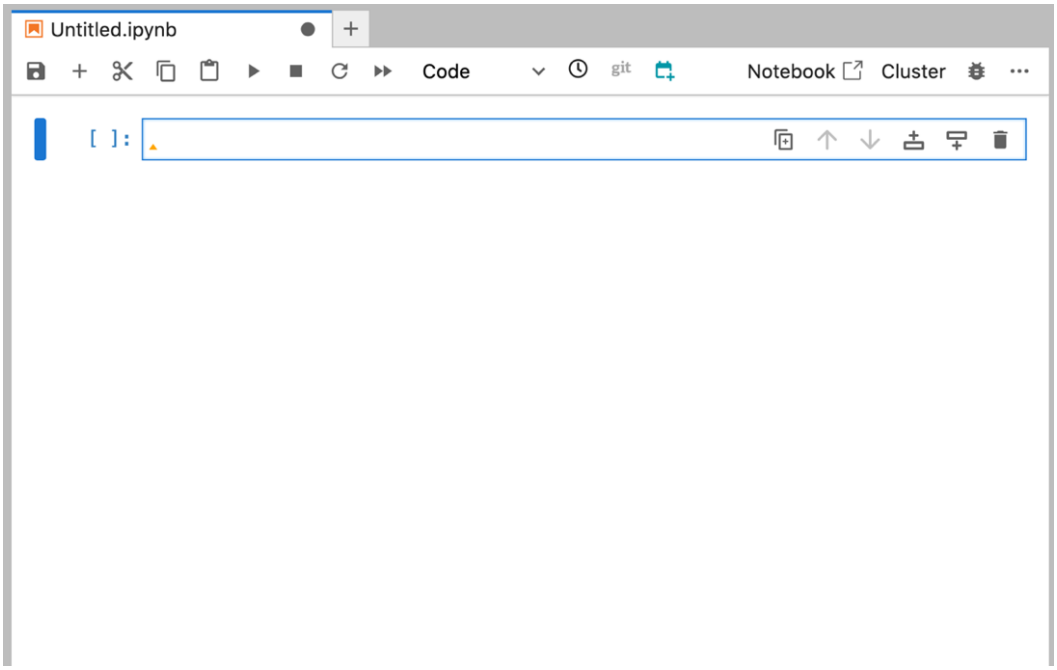


Figure 1.20 — A new Jupyter Notebook

   Here, you can start writing and running your code in an interactive notebook environment. You can think of this notebook like a sketchpad for a designer, where you can draft, test, and refine your ideas before finalizing them, making it convenient to experiment and see results immediately.

2. Rename the notebook to `basic_agent.ipynb` before proceeding (*Figure 1.21*).

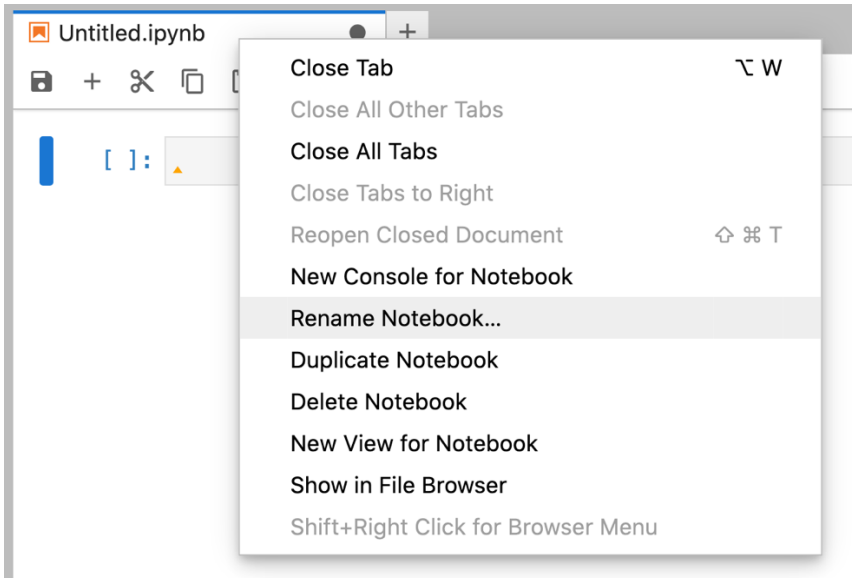


Figure 1.21 — Renaming the notebook

To do this, right-click on the notebook tab and select **Rename Notebook...** from the context menu. Then, enter `basic_agent.ipynb` as the new name and click **Rename**.

> **Note**
>
> We will be entering and executing code for each step in a separate cell within this notebook. After entering the code for each step, you can execute the cell by pressing *Shift + Enter*, which will not only run the current cell but also automatically create a new one below it. For more information on using Jupyter notebooks, visit `https://jupyter-notebook.readthedocs.io/en/latest/notebook.html`.

3. Check the versions of Python and pip installed in your environment. To check the Python version, run the following command:

```
!python --version
```

Next, verify the installed version of pip with this command:

```
%pip --version
```

Running these commands should display the current versions of Python and `pip` in your notebook to ensure you are using the appropriate versions for the examples in this chapter.

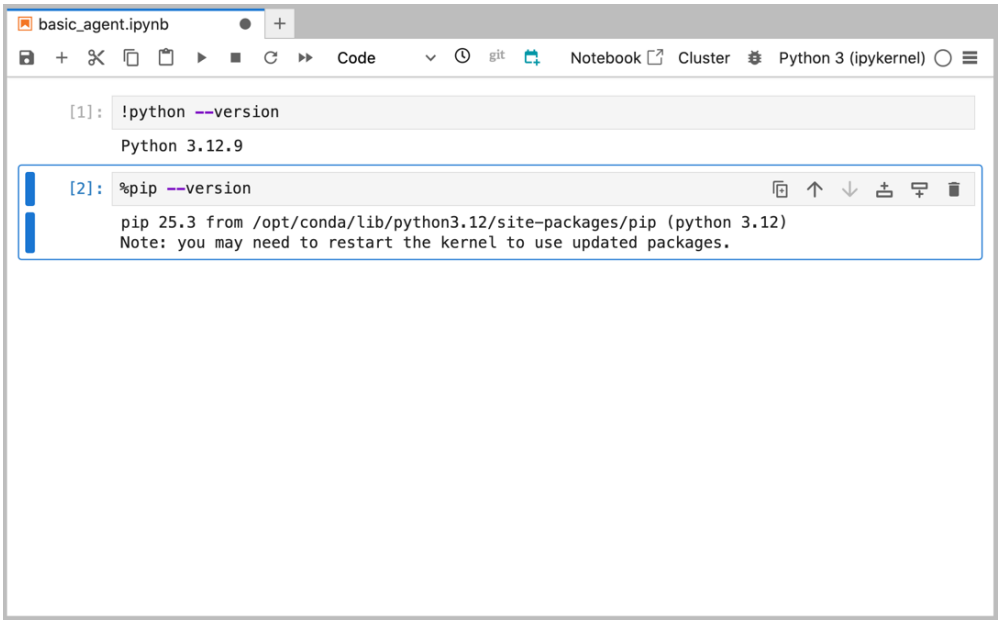


Figure 1.22 — Checking the Python and pip versions

As shown in *Figure 1.22*, the examples in this chapter were tested using Python version `3.12.9` and pip `25.3`. It's recommended to use versions close to these to avoid compatibility issues.

Note

Here, the exclamation point (`!`) at the beginning of the command tells the Jupyter notebook to run the command as a shell command instead of Python code. This allows you to execute terminal commands (like checking Python or `pip` versions) directly from the notebook.

4. In a new code cell of the Jupyter Notebook, run the following to install the required packages:

```
%pip install strands-agents==1.22.0
%pip install strands-agents-builder==0.1.10
%pip install strands-agents-tools==0.2.19
```

These commands will install specific versions of the `strands-agents`, `strands-agents-builder`, and `strands-agents-tools` packages, which are required for the examples used in this chapter.

5. Right-click on the output area and select **Enable Scrolling for Outputs** from the context menu.

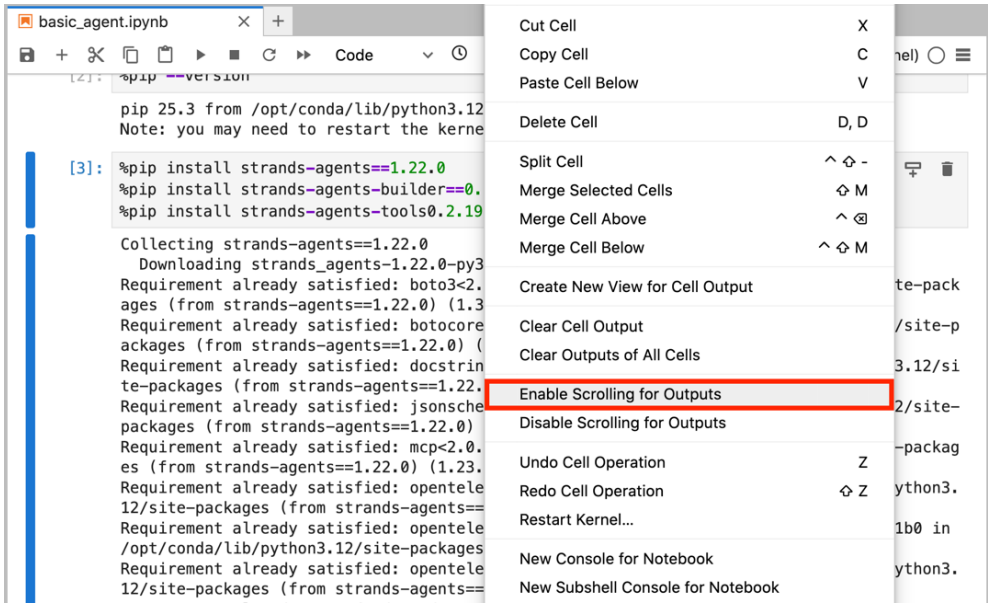


Figure 1.23 — Enabling scrolling for outputs

This will make it more manageable to view long outputs, as it prevents the output from expanding and occupying too much space in the notebook interface.

6. Restart the kernel by selecting **Kernel | Restart Kernel...** from the menu.

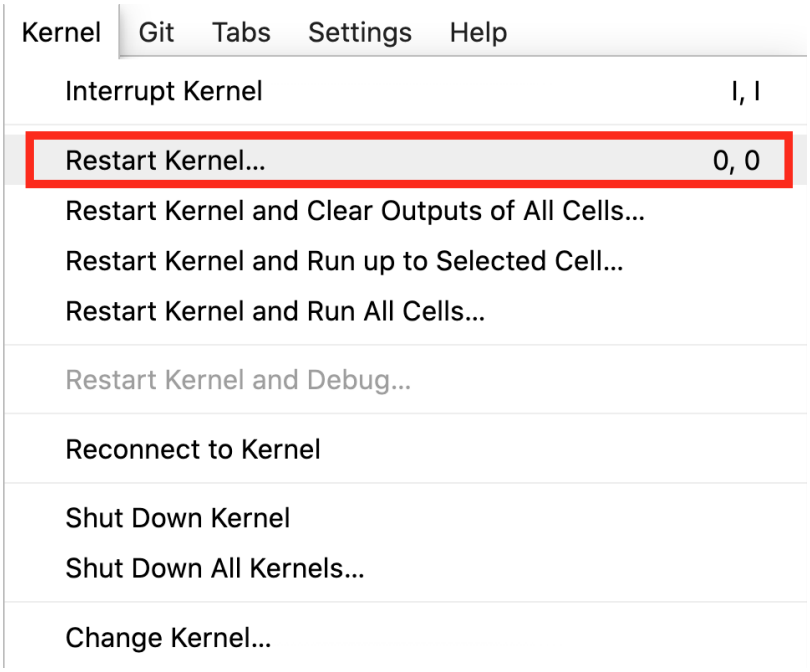


Figure 1.24 — Restarting the kernel

This will ensure the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for 2–3 minutes for the kernel to fully restart.

7. Run the following block of code so that any warnings related to protected namespace conflicts are filtered out:

```
import warnings

m = "Field .* has conflict with protected namespace"
warnings.filterwarnings(
    "ignore",
    message=m
)
```

8. Run the following code to create an `Agent` instance and use it to process the prompt:

```
from strands import Agent
agent = Agent()
```

```
prompt = """What are AI agents? Provide a concise definition (1–2
sentences) that explains what they are and what they do."""

agent(prompt)
```

This will invoke the agent, which responds with a concise definition of AI agents:

```
AI agents are autonomous software systems...
AgentResult(stop_reason='end_turn', ...)
```

Here, we are deliberately testing the agent's basic functionality without invoking external tools, to ensure that the installation and setup are working as intended. In case you encounter the following error:

```
ValidationException: An error occurred (ValidationException) when calling
the ConverseStream operation: The provided model identifier is invalid.
└ Bedrock region: ...
└ Model id: ...
```

This usually means that the model used is not available or the model ID is incorrect. To resolve this, explicitly specify a valid model ID supported in your region, as shown below:

```
model_id = "us.anthropic.claude-sonnet-4-6"

from strands import Agent

agent = Agent(model=model_id)
prompt = """What are AI agents? Provide a concise definition (1–2
sentences) that explains what they are and what they do."""

agent(prompt)
```

After running the updated code, the agent should successfully generate a response similar to the following:

```
## AI Agents

**AI agents are autonomous software systems that perceive their
environment, make decisions, and take actions to achieve specific goals —
```

```
often with minimal human intervention.** ...
AgentResult(stop_reason='end_turn', ...)
```

This confirms that your agent is correctly configured and able to interact with a valid model in Amazon Bedrock. If you encounter a `ValidationException` related to an invalid model identifier in subsequent examples, make sure to explicitly specify a valid `model_id` when initializing the `Agent()` instance. Model availability can vary by region, so always use a model that is supported in your current AWS region to avoid errors.

Note

If you are unsure which `model_id` to use, navigate to the Amazon Bedrock console and open **Model catalog** (under **Discover**). Search for and select a model (for example, **Claude Sonnet 4.6 by Anthropic**), then copy the **Model ID** value (for example, `anthropic.claude-sonnet-4-6`) from the model details page. Using the navigation pane (left sidebar), you can also navigate to **Inference profiles** (under **Infer**) to view available inference profiles along with their **Inference profile ID** values. You can use the **Inference profile ID** value as the `model_id` when initializing an `Agent` instance. If you haven't worked with inference profiles before, they act as an abstraction layer that unifies model ARNs from different regions under one identifier. For more information, feel free to check the following AWS blog post: `https://aws.amazon.com/blogs/machine-learning/getting-started-with-cross-region-inference-in-amazon-bedrock/`.

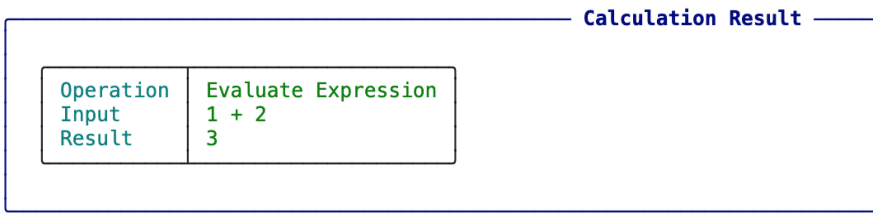
9. Now, let's enhance the agent's capabilities with useful tools. Import the necessary tools by importing `calculator` and `current_time` from the `strands_tools` module:

```
from strands_tools import calculator, current_time
```

10. Before using the `calculator` tool within the agent, let's first test it independently by evaluating a simple mathematical expression:

```
calculator.calculator("1 + 2")
```

This yields the following output:




```
{'status': 'success', 'content': [{'text': 'Result: 3'}]}
```

Figure 1.25 — Using the calculator tool directly to evaluate a mathematical expression

Here, you are directly calling the calculator tool with a simple expression to verify that it works before integrating it into the agent. Keep in mind that you are only testing the tool independently and are not using the agent yet.

11. Similarly, get the current time by invoking the `current_time` function:

```
current_time.current_time()
```

This gives you an output similar to the following:

```
'2026-01-01T00:00:00.000000+00:00'
```

12. Now, let's configure the agent to use the tools we just imported. Create an `Agent` instance and pass the `calculator` and `current_time` tools to the agent's tools parameter:

```
agent = Agent(
    tools=[calculator, current_time]
)
```

If you encounter the following warning:

```
================== WARNING ==================

    This region ... does not support
    our default inference endpoint: ....
    Update the agent to pass in a 'model_id' like so:
    ```
    Agent(..., model='valid_model_id', ...)
    ````
    Documentation: ...

=============================================
```

This means that the default model is not supported in your current region. To resolve this, explicitly specify a valid `model_id` as shown below:

```
model_id = "us.anthropic.claude-sonnet-4-6"
from strands import Agent

agent = Agent(
    model=model_id,
    tools=[calculator, current_time]
)
```

This ensures that the agent uses a model that is supported in your region and can successfully execute with the configured tools.

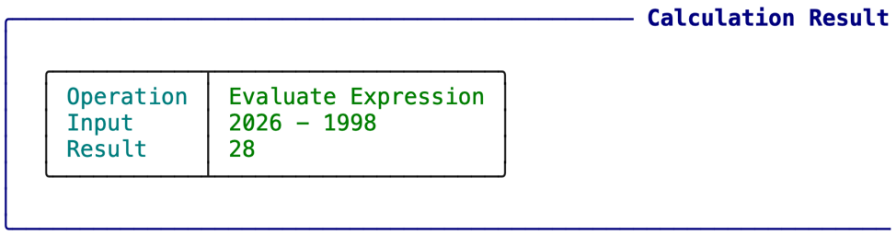
13. With the agent configured, we can now pass a query to test and invoke the agent:

```
prompt = """I was born on June 1, 1998, and I am 10 years older than my
brother. How old is my brother?"""

agent(prompt)
```

This will trigger the agent to calculate the age based on the provided information, utilizing the tools for computations and time-based queries.

```
I'll help you calculate your brother's age. First, let me get the current date
to determine your current age, then I can find your brother's age.
Tool #1: current_time
Now let me calculate the ages:
Tool #2: calculator
```




```
Tool #3: calculator
```

*Figure 1.26 — Output after running agent(prompt)*

Here, the agent processes the prompt and uses the `current_time()` and `calculator()` tools to determine the correct age based on the details provided.

When the agent finishes processing the prompt, you should see a similar output explaining the calculated ages:

```
...
Based on the current date (January 21, 2026), you are currently 27 years
old (turning 28 on June 1, 2026). Since you are 10 years older than your
brother, your brother is currently **18 years old** (he will turn 18 before
June 1, 2026, or has already turned 18 if his birthday has passed this
year). To be more precise: if your brother's birthday has already occurred
in 2026, he is 18. If his birthday hasn't occurred yet this year, he is
still 17 and will turn 18 later in 2026.
```

**Note**

Your output may look slightly different from the output shown in this example. This is expected because the agent relies on the current date and time returned by the `current_time()` tool, and the model may also vary slightly in how it reasons through the prompt and presents the final answer.

14. Now, assign a specific model ID to the variable `mid`:

```
mid = "us.anthropic.claude-sonnet-4-6"
```

15. Initialize the model and agent by importing `BedrockModel` and `Agent`, then creating an instance of the model and initializing the agent with the model:

```
from strands.models.bedrock import BedrockModel
from strands import Agent

model = BedrockModel(model_id=mid)
agent = Agent(model)
```

**Note**

You may use `help(Agent)` to view the documentation and get more information about the Agent class.

16. Provide the prompt to the agent by defining the prompt as a string and passing it to the agent for processing:

```
prompt = """What are AI agents? Provide a concise definition (1–2
sentences) that explains what they are and what they do."""

agent(prompt)
```

This generates a response similar to:

```
AI agents are software programs designed to perceive their environment,
make decisions ...
AgentResult(stop_reason='end_turn', ...)
```

**Note**

If you encounter a `ResourceNotFoundException`, it usually means the model you are trying to use has been marked as Legacy or is no longer active. To fix this, update your agent to use a currently supported model on Amazon Bedrock.

17. Initialize the agent with tools by creating an Agent instance and passing the model along with the list of tools to be used (`calculator` and `current_time`):

```
agent = Agent(
    model=model,
    tools=[calculator, current_time]
)
```

18. Define the prompt by preparing a string that presents a question for the agent to answer:

```
prompt = """I was born on June 1, 1998, and I am 10 years older than my
brother. How old is my brother?"""
```

19. Invoke the agent with the defined prompt to generate a response:

```
agent(prompt)
```

This yields an output similar to the following:

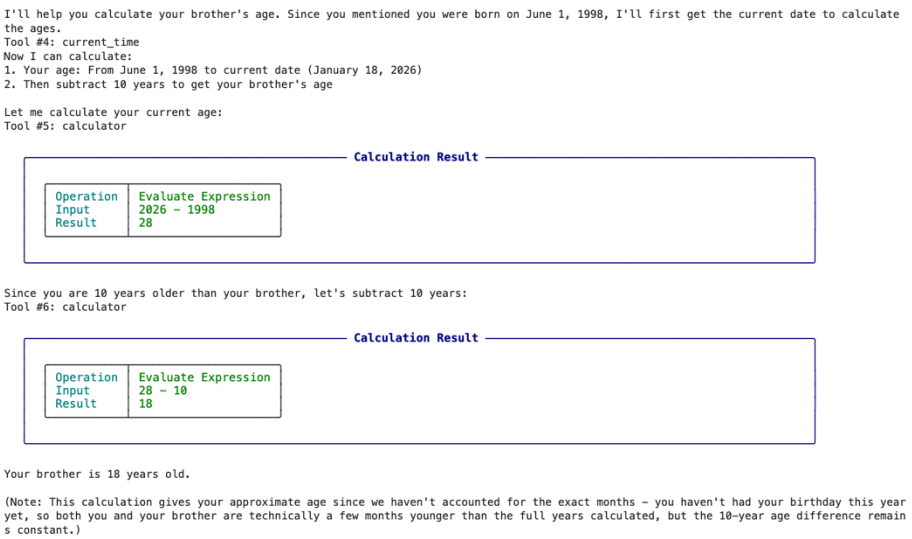



*Figure 1.27 — Output after running agent(prompt)*

You should see how the agent processes the prompt and leverages the `current_time()` and `calculator()` tools to compute the answer based on the provided information.

Since we're using Amazon Bedrock, the agent can simply interact with the model through the API without you having to worry about infrastructure management.

**Note**

As you'll see in the next chapter, it's also possible to configure the agent to interact with a model deployed using SageMaker AI, where your model runs on a managed inference endpoint, allowing the agent to use models outside of Bedrock.

Congratulations! Your agent is now up and running, and can work with a Bedrock model to process input prompts, perform calculations, and reference the current time. You've explored how to configure tools, select a supported model, and execute queries that return meaningful responses. Keep in mind that outputs may vary based on your region or the current date, but this setup provides a strong starting point for getting started with AI agents before moving on to more complex implementations.

**Note**

If you plan to work on the hands-on examples in the next chapter at a later time, you can turn off your SageMaker Studio space to manage costs. To navigate to the space details page, click the JupyterLab icon under **Applications** in the left sidebar (SageMaker Studio). You'll then see a list of running and stopped spaces, where you can click on **mle-on-aws-space** to open the space details page. From here, you can click **Stop space** to stop the running space, which is using a `ml.t3.xlarge` instance.

## Summary

In this chapter, we explored the fundamentals of generative AI on AWS and learned how to leverage various services and solutions to build AI agents. We got our feet wet by working with foundation models provided through Amazon Bedrock, while also covering key concepts and terminology along the way. After setting up a SageMaker Studio space, we built our first AI agent using Strands Agents, which interacts with a Bedrock model and is configured with several tools to enhance its reasoning and problem-solving capabilities.

In the next chapter, you'll build on what you learned here by using Strands Agents to create AI agents that interact with a SageMaker AI real-time inference endpoint and deploying them using **Bedrock AgentCore**. This will give you hands-on experience taking agents from development to a fully managed, production-ready runtime environment. You'll also develop a RAG-powered agent that can incorporate external data into its responses, allowing you to build more dynamic, context-aware applications.

# Further reading

To deepen your understanding of the concepts discussed in this chapter, explore the following resources:

- *Amazon Bedrock FAQs* (`https://aws.amazon.com/bedrock/faqs/`)
- *Amazon Bedrock Pricing* (`https://aws.amazon.com/bedrock/pricing/`)
- *Amazon Bedrock — Prompt engineering guidelines* (`https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-engineering-guidelines.html`)
- *AWS Machine Learning Blog — Welcome to a New Era of Building in the Cloud with Generative AI on AWS* (`https://aws.amazon.com/blogs/machine-learning/welcome-to-a-new-era-of-building-in-the-cloud-with-generative-ai-on-aws/`)
- *AWS Machine Learning Blog — Best practices to build generative AI applications on AWS* (`https://aws.amazon.com/blogs/machine-learning/best-practices-to-build-generative-ai-applications-on-aws/`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 2

# Building AI Agents with SageMaker AI and Bedrock AgentCore

In the previous chapter, you built a simple AI agent using Strands Agents and learned how to set up an agent, configure tools, and interact with a Bedrock model. In this chapter, you'll build on top of what you've learned so far by using a SageMaker AI real-time inference endpoint and incorporating **Bedrock AgentCore** to support building and running production-ready AI agents. You will also extend this by building **retrieval-augmented generation** (**RAG**)-powered AI agents, where external information is retrieved and combined with model responses during execution. You can think of this as a shift from a plug-and-play service to a more customizable setup. Bedrock provides ready-to-use models, while SageMaker AI gives you control over how models are deployed through inference endpoints. As you go through the rest of the book, you'll see how this added flexibility and configurability with SageMaker AI becomes useful when building more complex applications and workflows.

To help you gain hands-on experience running AI agents with SageMaker AI, Strands Agents, and Bedrock AgentCore, we will cover the following topics in this chapter:

- Deploying a pretrained LLM with SageMaker AI
- Building AI agents with Amazon SageMaker AI and Strands Agents
- Building AI agents with Amazon Bedrock AgentCore
- Deploying production-ready agents with Bedrock AgentCore Runtime
- Setting up an Amazon Bedrock Knowledge Base

- Building a RAG-powered AI agent with Strands Agents
- Building a RAG-powered AI agent that interacts with a SageMaker AI inference endpoint

By the end of this chapter, you will have a clear understanding of how to build agents that interact with SageMaker AI inference endpoints. You will also learn how to use Amazon Bedrock Knowledge Bases to build RAG-powered agents that can retrieve and ground responses in external data, enabling more accurate and context-aware applications.

# Technical requirements

Before proceeding with the hands-on examples in this chapter, ensure that the following prerequisites and setup requirements are in place:

- **Sufficient account-level quota for selected ML instance types**: When deploying models, ensure that the applied account-level quota value for the selected ML instance types (for example, **ml.g5.24xlarge for endpoint usage**) is set to at least `1`. If the quota value is `0`, SageMaker AI will not be able to provision the required compute resources, and the job or endpoint creation will fail. You can review and request quota increases through the **Service Quotas** console (within the AWS Management Console).
- **An existing SageMaker Studio space**: You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1* to follow along with the examples in this book.
- **A code editor installed on your local machine** (such as Visual Studio Code or Sublime Text): You'll need this when working with the code and configuration files used throughout the hands-on exercises and examples in this book.
- **A new IAM user with AdministratorAccess permissions**: You can create this IAM user through the IAM Console, by typing `IAM` in the search bar, and then selecting **IAM** from the list of results. Once in the IAM console, select **Users** from the left-hand menu and then click the **Create user** button. Enter the desired user name (`superadmin`), ensure that the checkbox for **Provide user access to the AWS Management Console** is checked, choose **I want to create an IAM user**, and click **Next**. On the **Set permissions** page, choose **Attach policies directly**, then search for and select the `AdministratorAccess` policy. Continue through the remaining steps, and finally clicking the **Create user** button. After completion, you'll be able to use the new IAM user to log in to the AWS Management Console with full administrative permissions.

> **Note**
>
> You will need to sign in using this IAM user account (`superadmin`) when setting up the Amazon Bedrock knowledge base. Keep in mind that this IAM user has overly permissive permissions (`AdministratorAccess`), which grants full access to all AWS services and resources. It's recommended to create more restricted IAM users in production environments to follow the principle of least privilege. For now, this should do the trick, so that you can work through the hands-on examples without running into permission issues.

The Jupyter notebooks, source code, and other files used for each chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

## Deploying a pretrained LLM with SageMaker AI

SageMaker AI inference endpoints allow you to host ML models and make them available for real-time predictions through an API. When you deploy a model to an endpoint, SageMaker AI manages the underlying infrastructure so you can focus on sending requests and receiving responses from your model. This makes it easier for you to deploy and work with a wide range of models, and integrate them into applications, services, or AI agents that require real-time inference.

In this section, you will deploy a pretrained LLM to an inference endpoint using Amazon SageMaker AI. You will begin by launching a new notebook, installing the required libraries, and verifying that the correct prerequisites have been installed. Next, you will use the **SageMaker Python SDK** to deploy a pretrained LLM to a real-time inference endpoint. Finally, you will test the deployed endpoint by sending a sample request and reviewing the generated response to confirm that the LLM is serving predictions successfully.

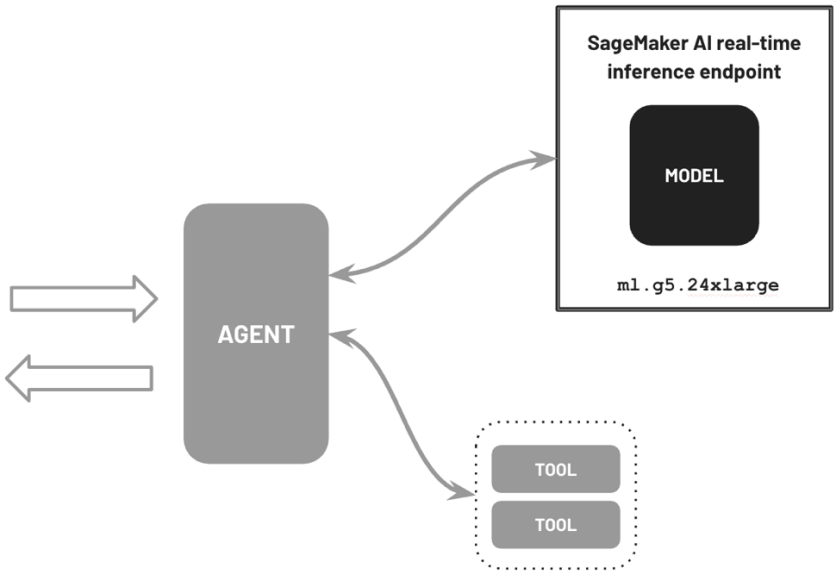



*Figure 2.1 — Where the SageMaker AI inference endpoint fits in the Agent Architecture*

The endpoint you will deploy in this section will be used in the upcoming sections as the inference layer for your AI agent built with Strands Agents and Bedrock AgentCore. It allows the agent to send prompts to the hosted model and receive real-time responses during execution. This provides the foundation for integrating model inference into an agent-based workflow that combines reasoning and tool use.

**Note**

When deploying endpoints, ensure that the applied account-level quota value for the selected ML instance types (for example, **ml.g5.24xlarge for endpoint usage**) is set to at least `1`. If the quota value is `0`, SageMaker AI will not be able to provision the required compute resources, and the deployment will fail. You can review and request quota increases through the Service Quotas console in the AWS Management Console. You can navigate to the Service Quotas console by typing `service quotas` in the search bar, selecting **Service Quotas** from the list of search results, choosing `Amazon SageMaker` from the dropdown list under **Manage quotas** and clicking **View quotas**. From here, you can locate the quota for the selected instance type (for example, **ml.g5.24xlarge for endpoint usage**), check the current applied value, and request a quota increase if necessary. When requesting or verifying the quota, make sure that you are in the correct region and select the correct quota category for your use case. For

example, choose **ml.g5.24xlarge for endpoint usage**. In this book, you will work with various resources, including spaces and inference endpoints used throughout the examples. To better understand the potential costs involved, take a moment to review the pricing information for **Amazon SageMaker AI** (`https://aws.amazon.com/sagemaker/ai/pricing/`) before proceeding.

Now, let's move forward with deploying a pretrained LLM to a SageMaker AI inference endpoint:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**).
2. Rename the notebook to `first_sagemaker_endpoint.ipynb` before proceeding. We will run the code in the following steps within this notebook.
3. Install the `strands-agents` package along with its SageMaker dependencies by running the following in a notebook cell:

```
%pip install 'strands-agents[sagemaker]'
```

4. Uninstall any existing versions of `sagemaker` and `sagemaker-serve` to avoid conflicts, then install the required versions by running the following in a notebook cell:

```
%pip uninstall -y sagemaker sagemaker-serve
%pip install "sagemaker==3.2.0"
%pip install "sagemaker-serve==1.2.0"
```

**Note**

Restart the kernel by selecting **Kernel | Restart Kernel** from the menu. This will ensure the newly installed packages are properly loaded and available for use.

5. Verify that the correct version of the `sagemaker` library is installed

```
%pip show sagemaker
```

You should see output similar to the following, confirming that SageMaker `3.2.0` is installed:

```
Name: sagemaker
Version: 3.2.0
Summary: Open source library for training and deploying models on Amazon
SageMaker.
...
```

6. Verify that the correct version of the sagemaker-serve library is installed:

```
%pip show sagemaker-serve
```

This should give you the following output:

```
Name: sagemaker-serve
Version: 1.2.0
Summary: SageMaker Serve package for model serving and deployment
```

7. Import the SageMaker Python SDK in your notebook:

```
import sagemaker
```

8. Check the location of the installed SageMaker package by running the following:

```
sagemaker.__file__
```

This should return a path similar to:

```
'/opt/conda/lib/python3.12/site-packages/sagemaker/__init__.py'
```

9. Open a new browser tab and open the book's GitHub repository. From the `chapter02` folder, locate the *import_helper.py* file, and then open it to review the code. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

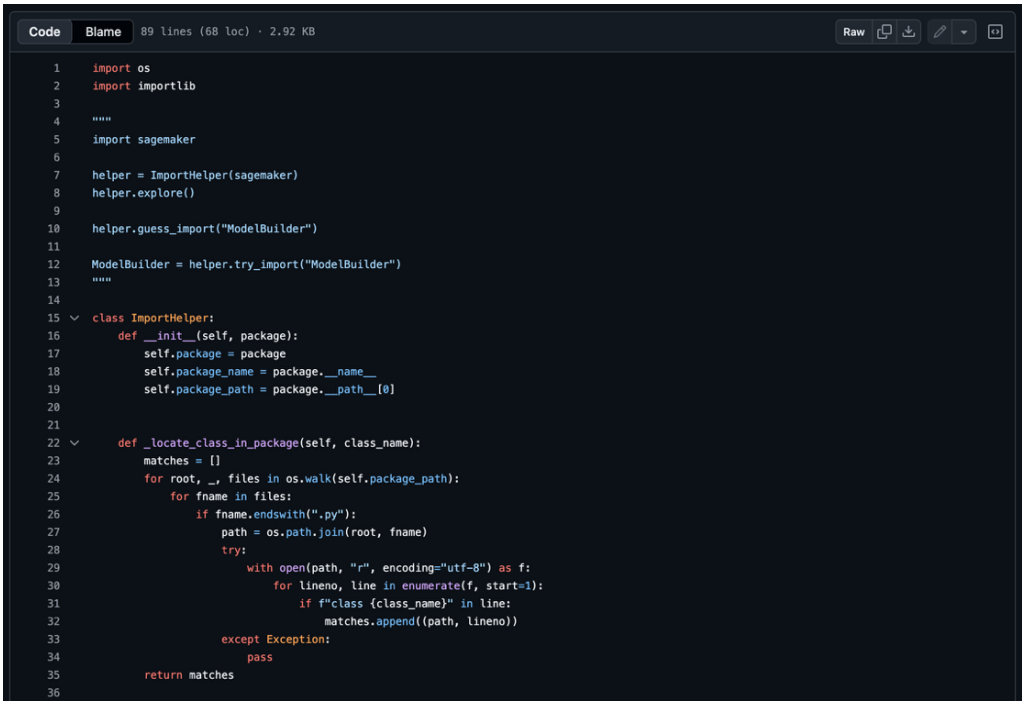



*Figure 2.2 — Locating the import_helper.py file*

The *import_helper.py* file provides an easy way to locate classes within a package and suggest valid import paths. Specifically, it defines the `ImportHelper` class, which contains several methods for exploring and dynamically importing classes. The `guess_import()` method searches for a specified class within the package and prints the likely import path, while the `try_import()` method attempts to import the class dynamically using the constructed path. Additionally, the `explore()` method prints the directory structure of the package to help users navigate its contents.

10. Click the **Download raw file** button to save *import_helper.py* to your local machine, then upload it to your JupyterLab environment in SageMaker Studio.

> **Note**
>
> Make sure that it is placed in the *same* directory as the Jupyter notebook `first_sagemaker_endpoint.ipynb` to avoid import errors.

11. In the browser window where your JupyterLab notebook is open, run the following to import the `ImportHelper` class:

```
from import_helper import ImportHelper
```

12. Initialize the `ImportHelper` and use the `explore()` method to view the structure of the SageMaker package:

```
helper = ImportHelper(sagemaker)
helper.explore()
```

This should yield the following output:

```
+-- sagemaker/
   +-- core/
      +-- apiutils/
      +-- clarify/
      +-- compute_resource_requirements/
      +-- config/
...
```

13. Let's use the `guess_import()` method to locate where a class is defined and see its likely import statement(s). For example, to guess the import statement for the `ModelBuilder` class, run:

```
helper.guess_import("ModelBuilder")
```

This gives you the following output:

```
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/serve/
model_builder.py:142
Likely import: from sagemaker.serve.model_builder import ModelBuilder
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/serve/utils/
exceptions.py:6
Likely import: from sagemaker.serve.utils.exceptions import ModelBuilder
```

14. Import the `ModelBuilder` and `SchemaBuilder` classes:

```
import sagemaker.serve.model_builder as MB
import sagemaker.serve.builder.schema_builder as SB
```

```
ModelBuilder = MB.ModelBuilder
SchemaBuilder = SB.SchemaBuilder
```

15. Initialize a SageMaker `Session` instance and retrieve the default bucket and role ARN:

```
from sagemaker.core.helper.session_helper import (
    Session
)

session = Session()
bucket_name = session.default_bucket()
role_arn = session.get_caller_identity_arn()
```

16. Define the model ID and instance type:

```
model_id = "huggingface-reasoning-qwen3-32b"
instance_type = "ml.g5.24xlarge"
```

Here, you specify the model ID (`model_id`) of the pretrained model SageMaker AI should load for serving predictions. You're also storing in a variable (`instance_type`) the instance type that specifies the compute environment for the endpoint, including GPU, CPU, and memory resources. Choosing `ml.g5.24xlarge` provisions a powerful GPU-backed instance capable of efficiently serving requests to LLMs in real time.

**Note**

SageMaker AI provides a variety of instance types to choose from, each offering different combinations of CPU, GPU, and memory resources. Depending on the size of your model, expected traffic, and performance requirements, you can select smaller instances for lightweight models or larger GPU-backed instances for high-performance inference of LLMs. Keep in mind that larger instances with more GPUs or memory incur higher costs, so choose an instance type that balances performance and budget for your workload. Before proceeding with the next steps and deploying inference endpoints, make sure to review the Amazon SageMaker AI pricing details, including **On-demand pricing** (**Real-Time Inference**) on the official pricing page: `https://aws.amazon.com/sagemaker/ai/pricing/#Instance_details`. This will help you understand the potential charges for the compute instances and resources you'll be using as you configure and test your endpoints, so you can plan your usage accordingly.

17. Define the input and parameters for the model:

```
example_input = {
    "inputs": "AI agents are systems that can",
    "parameters": {
        "max_new_tokens": 256,
        "top_p": 0.95,
        "temperature": 0.7
    }
}
```

The `inputs` field specifies the prompt that is sent to the model for inference. The parameters define generation controls, including `max_new_tokens` for output length and sampling parameters that influence token selection during decoding.

18. Define the expected output for the model:

```
example_output = [
    {"generated_text": """AI agents are systems that can perceive their
environment, reason about goals, and take actions autonomously."""}
]
```

19. Using the defined example input and expected output from the previous steps, initialize a `SchemaBuilder` instance with input and output examples:

```
schema_builder = SB.SchemaBuilder(
    example_input,
    example_output
)
```

20. Set the following environment variables to enable tool calling and automatic tool selection:

```
env_vars = {
    "OPTION_TOOL_CALL_PARSER": "hermes",
    "OPTION_ENABLE_AUTO_TOOL_CHOICE": "true",
}
```

21. Initialize a `ModelBuilder` instance with the specified model ID, schema, session, role, logging level, and instance type.

```
import logging
log_level = logging.ERROR

model_builder = MB.ModelBuilder(
    model=model_id,
    schema_builder=schema_builder,
    sagemaker_session=session,
    role_arn=role_arn,
    log_level=log_level,
    instance_type=instance_type,
    env_vars=env_vars
)
```

22. Build the model using the model builder:

```
model = model_builder.build()
```

23. Import `ModelAccessConfig` and define the access configuration for the model:

```
from sagemaker.core.shapes.shapes import (
    ModelAccessConfig
)

model_access_configs = {
    model_id: ModelAccessConfig(
        accept_eula=True
    )
}
```

Some models require explicit acceptance of the provider's end-user license agreement before they can be deployed. Setting `accept_eula=True` indicates that you agree to these terms.

24. Deploy the model to a SageMaker inference endpoint:

```
%%time

predictor = model_builder.deploy(
```

```
    endpoint_name="sagemaker-endpoint-00",
    model_access_configs=model_access_configs,
    accept_eula=True
)
```

During deployment, SageMaker AI provisions the required infrastructure (using the specified ML instance type `ml.g5.24xlarge`) and sets up the endpoint that will serve prediction requests. The `%%time` magic command measures how long the entire cell takes to run, which will display the time taken to deploy the model. In most cases, this process takes around 10 to 20 minutes to complete, so feel free to grab a cup of coffee or tea while waiting.

**Note**

In case you encounter a `ResourceLimitExceeded` error, this means that your account has reached the maximum allowed instances for the requested type in the current AWS region. You can resolve this by requesting a quota increase or using a different instance type that is available. You can review and request quota increases through the **Service Quotas** console (within the AWS Management Console). To view the applied quota values and perform the required quota increase request for the **ml.g5.24xlarge for endpoint usage** applied account-level quota value (**Increase quota value** = 2), simply navigate to the Service Quotas console by typing `service quotas` in the search bar, selecting **Service Quotas** from the list of search results, choosing `Amazon SageMaker` from the dropdown list under **Manage quotas** and clicking **View quotas**, and locating the quota value to be increased. When requesting or verifying the quota, make sure that you are in the correct region and select the correct quota category for your use case. For example, choose **"ml.g5.24xlarge for endpoint usage"**. SageMaker provides several instance-level quota categories, including cluster usage, cluster spot instance usage, endpoint usage, notebook instance usage, processing job usage, spot training usage, and training job usage. Since this chapter focuses on SageMaker AI inference endpoints, ensure that you modify the quota specifically under the endpoint usage category for the appropriate instance type, rather than the other categories. After your quota increase request is approved, retry deploying the endpoint. You may also need to delete the existing endpoint configuration before retrying to avoid conflicts with the previous failed deployment.

Behind the scenes, SageMaker AI creates an **endpoint configuration** using the parameters you specify, such as the instance type, model container image, and other settings. This configuration defines the resources required for the endpoint. Once the configuration is created, SageMaker AI provisions the necessary infrastructure (for example, the managed inference instance(s)), deploys the model to the endpoint, and makes it available for real-time inference.

**Note**

If you encounter a `ClientError` / `ValidationException` stating "`Cannot create already existing endpoint configuration`" it means that an endpoint configuration with the same name already exists in your account. For example, a previous failed endpoint deployment attempt may have created the endpoint configuration without successfully creating the actual endpoint. To resolve this, navigate to the Amazon SageMaker AI console, go to **Endpoint configurations** (under **Deployments & inference**), delete the existing configuration with that name, and then retry deploying the endpoint. If the model deployment process is taking longer than expected (for example, more than 30 minutes), navigate to the endpoint details page in the SageMaker AI console and check the model container logs. This can help you identify any issues or errors that may be preventing the deployment from completing successfully. You can also check the **Errors** tab, if available, for additional context. In some cases, deployments fail because the required ML instances are not available in the selected region.

25. Let's inspect the sample input data and parameters that will be sent to the endpoint for inference. Run the following code to review the structure and values before making a prediction:

```
print(example_input)
```

This should give you the following:

```
{'inputs': 'AI agents are systems that can', 'parameters':
{'max_new_tokens': 256, 'top_p': 0.95, 'temperature': 0.7}}
```

26. Using the example input from the previous step, make a prediction by invoking the deployed model endpoint:

```
import json

resp = predictor.invoke(
    body=json.dumps(example_input),
    content_type="application/json"
)

print(resp.body.read())
```

You should get an output similar to the following:

```
b'{"generated_text": " perceive their environment and act upon it to
achieve specific goals. These agents are a cornerstone of artificial
intelligence, representing a significant leap from traditional rule-based
programs to dynamic, adaptive systems. To truly understand AI agents, ...
"}\n'
```

**Note**

Remember to delete the SageMaker AI inference endpoint after completing the examples in this chapter. It is recommended to complete all the hands-on examples until the end of this chapter so that you can delete the inference endpoint right away, to ensure you avoid unnecessary charges or resource usage. If you decide to continue the other sections at a later time, it's best to delete the endpoint now and redeploy the model when you're ready to continue.

At this point, your SageMaker AI inference endpoint is up and running, hosting a pretrained LLM that can generate responses to your input prompts. You have completed all the steps from environment setup to model deployment and endpoint testing, confirming that the deployed model can successfully generate outputs for the inputs you provide. With the inference endpoint ready, you can now integrate it into applications such as AI agents, chatbots, or systems that provide insights and support decision-making.

**Note**

You should know when to use Amazon Bedrock and when to use Amazon SageMaker based on your use case. Bedrock is typically the better choice when you want quick access to fully managed foundation models without worrying about infrastructure setup or model deployment. SageMaker AI, on the other hand, is more suitable when you need deeper control over model training, deployment, and hosting configurations. Understanding this distinction helps you choose the right service based on your performance, customization, and operational requirements. If you want a more in-depth comparison, you can review the following resource: `https://docs.aws.amazon.com/decision-guides/latest/bedrock-or-sagemaker/bedrock-or-sagemaker.html`.

## Building AI agents with Amazon SageMaker AI and Strands Agents

In this section, you will build an AI agent that interacts with a model deployed in a SageMaker AI inference endpoint using **Strands Agents**, an open source SDK for building and running AI agents. You will start by setting up a new Jupyter notebook and installing the required packages. After verifying the environment, you will initialize and configure the agent to use the SageMaker AI endpoint, along with tools and prompts that define its behavior. Finally, you will test the agent and observe how it processes a request, calls tools when needed, and generates a response using the deployed endpoint.

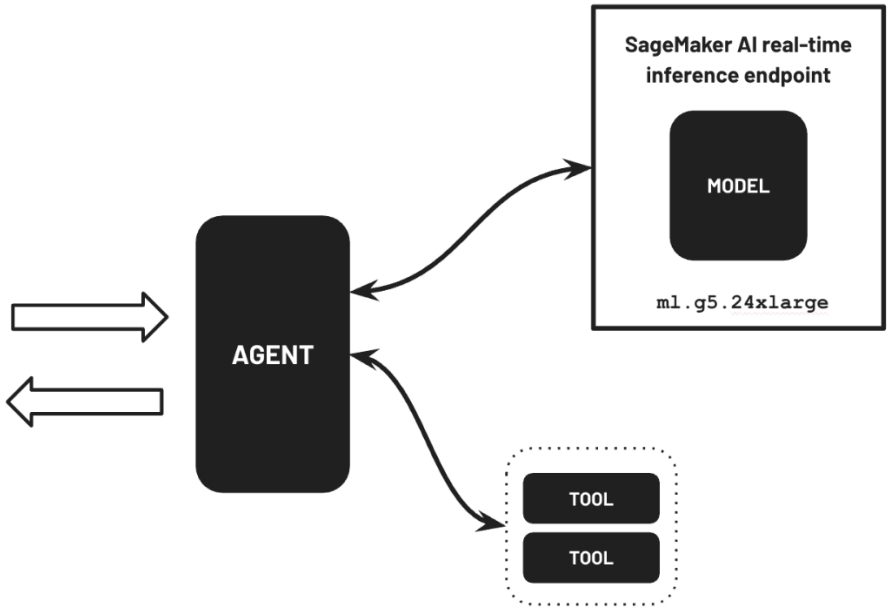



*Figure 2.3 — Building an AI agent that interacts with a SageMaker AI inference endpoint*

The agent you build here will rely on the SageMaker AI inference endpoint set up in the previous section for generating responses during execution. Each request from the agent is sent to the deployed endpoint, which returns real-time responses from the hosted model. As you work through the hands-on examples in this chapter, you will see how SageMaker AI endpoints can be integrated into various agent workflows to support production-ready inference.

**Note**

Before proceeding with the next steps, make sure that your SageMaker AI real-time inference endpoint (`sagemaker-endpoint-00`) is still running and available. The AI agent will send requests to this endpoint when generating responses, so the endpoint must be active for the examples in this section to work correctly. If the endpoint has been deleted, set up the endpoint again before continuing.

Let's go through the steps to set up and run the AI agent that interacts with your SageMaker AI real-time inference endpoint:

1. In the Launcher, create a new Jupyter notebook by choosing **conda_python3** (under **Notebook**). Be sure to rename the notebook to `sagemaker_agent.ipynb` before proceeding. We will run the code in the following steps within this notebook.
2. Install the required packages for this notebook by running the following commands in a JupyterLab cell:

```
%pip install strands-agents==1.22.0
%pip install strands-agents-builder==0.1.10
%pip install 'strands-agents[sagemaker]'
%pip install strands-agents-tools==0.2.19
```

3. After the installation in the previous cell is complete, restart the kernel to ensure the newly installed packages are properly loaded and available for use.
4. Verify that the `strands-agents` package is installed and check its version by running the following in a notebook cell:

```
%pip show strands-agents
```

   You should see that the version is `1.22.0`.

5. Similarly, check that the `strands-agents-builder` package is installed and note its version:

```
%pip show strands-agents-builder
```

   The version should be `0.1.10`.

6. Finally, verify the installation and version of strands-agents-tools by running:

```
%pip show strands-agents-tools
```

   The version should be `0.2.19`.

7. Use `ImportHelper` to locate the module for `SageMakerAIModel` by running the following in a notebook cell:

```
import strands
from import_helper import ImportHelper
```

```
helper = ImportHelper(strands)
helper.guess_import("SageMakerAIModel")
```

This will display the likely module path(s) where `SageMakerAIModel` is defined to help you determine the correct import statement for use in your code.

```
Found in: /opt/conda/lib/python3.12/site-packages/strands/models/
sagemaker.py:90
Likely import: from strands.models.sagemaker import SageMakerAIModel
```

8. Define a unique name for the SageMaker endpoint:

```
endpoint_name="sagemaker-endpoint-00"
```

9. Define the endpoint and payload configurations:

```
from strands.models.sagemaker import SageMakerAIModel

EPConfig = SageMakerAIModel.SageMakerAIEndpointConfig
PLConfig = SageMakerAIModel.SageMakerAIPayloadSchema

endpoint_config = EPConfig(
    endpoint_name=endpoint_name,
    region_name="us-east-1"
)

payload_config = PLConfig(
    max_tokens=4096,
    stream=True
)
```

**Note**

Make sure to replace the `region_name` value with the AWS Region where your SageMaker AI real-time inference endpoint is deployed. For example, if your model was deployed to an endpoint in a different region (such as `ap-southeast-1` or another region you are using), update the value accordingly so the agent can connect to the correct endpoint.

10. Initialize a `SageMakerAIModel` instance by passing in the previously defined endpoint configuration (`endpoint_config`) and payload configuration (`payload_config`):

```
model = SageMakerAIModel(
    endpoint_config=endpoint_config,
    payload_config=payload_config
)
```

11. Initialize a new agent with the specified model (SageMaker endpoint: `sagemaker-endpoint-00`), tools (`calculator` and `current_time`), and system prompt.

```
from strands import Agent
from strands_tools import calculator, current_time

system_prompt = (
    "You MUST use at least one tool to answer the user's question."
    "Do not answer without calling a tool first."
)

agent = Agent(
    model=model,
    tools=[calculator, current_time],
    system_prompt=system_prompt
)
```

12. Finally, run the following to get the agent's response to a prompt:

```
%%time

prompt = """I was born on June 1, 1998, and I am 10 years older than my
brother. How old is my brother?"""

response = agent(prompt)
```

This should generate an output similar to the following:

```
Okay, let's see. The user was born on June 1, 1998, and they are 10 years
older than their brother. The question is asking how old the brother is.
...
Once I have today's date, I can compute the brother's age. Let's outline
```

```
the steps:

1. Call current_time() to get the current date and time in ISO format.
2. Parse the current date to extract the year, month, and day.
3. Calculate the brother's birthdate as June 1, 2008 (since 1998 + 10 years
= 2008).
4. Compare today's date with the brother's birthdate to determine his
current age, considering if his birthday has already occurred this year.
...
Tool #1: current_time

User's birthdate: June 1, 1998.
Current date: January 18, 2026.

So from June 1, 1998, to June 1, 2025, is 27 years. From June 1, 2025, to
January 18, 2026, is 7 months and 17 days. So the user's age is 27 years
plus that time, but since we usually count age in full years, they are
still 27 until their next birthday. Therefore, their brother is 27 - 10 =
17 years old.

So the answer should be 17.
...
```

Here, the agent calculated the user's age by using the `current_time()` tool to retrieve today's date. It then subtracted 10 years to determine the brother's age and considered the time of year to ensure the brother's age was correctly calculated based on whether his birthday had passed.

**Note**

You may get a different response depending on how the agent calculates the age difference and the steps it follows in the process.

At this point, you have successfully built an AI agent using Strands Agents that interacts with a SageMaker AI real-time inference endpoint. You may find that using Strands Agents with Bedrock and configuring it to work with a model deployed with SageMaker AI follows a similar setup and workflow, with only minor differences in how the endpoint is integrated. This is because Strands Agents provides a consistent interface for working with different model providers, making it easier for you to switch between them or use models from multiple

sources within the same application. With the agent working as expected, you are ready to deploy it using Bedrock AgentCore to a production-ready serverless runtime environment.

**Note**

You can find additional information about Strands Agents model providers here: `https://strandsagents.com/docs/user-guide/concepts/model-providers/`.

# Building AI agents with Amazon Bedrock AgentCore

Building and deploying AI agents in real-world environments can introduce several challenges. You often need to manage orchestration, tool integration, scaling, monitoring, and secure access to different services and models. In many cases, setting up and maintaining the infrastructure and workflows required for agents to run reliably can add significant complexity to the development process. **Amazon Bedrock AgentCore** helps address these challenges by providing a platform designed to simplify how you deploy, manage, and operate AI agents in a more structured and scalable way.

In this section, you'll learn how to build and deploy AI agents using Amazon Bedrock AgentCore. You'll start by getting your feet wet with Bedrock AgentCore by running the agent generated by the scaffold in development mode to confirm that everything is set up and working correctly. You'll then customize your agent to interact with a SageMaker AI endpoint for model inference.

Let's first go over a few essential services, terms, and concepts that are relevant to this section:

- **Bedrock AgentCore** — An agentic platform that allows you to build and run AI agents capable of interacting with models, tools, and external systems to perform tasks. The platform simplifies the development and deployment process by handling much of the underlying infrastructure and operational complexity. This helps you move from experimentation to production faster while focusing on designing and improving your agents.
- **Bedrock AgentCore Starter Toolkit** — A **command line interface** (**CLI**) toolkit that allows you to quickly get started with building and deploying agents using Bedrock AgentCore. For example, you can use this CLI tool to scaffold a new project with a single command:

  ```
  agentcore create --project-name helloSageMaker
  ```

  This command generates a fully structured project for your AI agent, including sample code, configuration files, and scripts, so you can start customizing and running your

agent immediately without having to set up the project from scratch. With just a few seconds (or at most a few minutes), you already have a working project directory ready to develop, test, and deploy your agent. Once your project is created, you can run your agent locally in development mode using:

```
agentcore dev
```

This then starts the agent in a local environment so you can test its behavior and ensure everything is working as expected. You can also invoke the agent directly with `agent invoke`, similar to the following:

```
agentcore invoke --dev "Hello Agent!"
```

Running this command sends a prompt to your locally running agent and returns its response, allowing you to quickly iterate and validate your agent's reasoning and tool usage before deploying it to a serverless runtime environment. Once you are satisfied with your agent's behavior and implementation in development mode, you can deploy it to a serverless runtime environment using the following:

```
agentcore deploy
```

The `agentcore deploy` command packages your agent code (along with the other files) and deploys it to a fully managed, serverless runtime environment, which gives it the ability to handle real requests outside your local development setup.

- **Tom's Obvious Minimal Language (TOML)** — A minimal, human-readable configuration format used to define project settings, dependencies, and environment configurations. Files written in this format typically use the *.toml* extension. For example, a basic TOML configuration file might look like the following:

```
[project]
name = "hello-agent"
version = "0.1.0"
description = "Sample AI agent project"
```

In this example, `[project]` defines a section that contains general metadata about the project. The `name`, `version`, and `description` fields specify basic information that development tools can use when building or managing the project. In practice, TOML files can vary depending on the tool or framework using them, and they may include additional sections for dependencies, build settings, scripts, or environment-specific

configurations. Different projects often structure their TOML files slightly differently based on their requirements.

- **uv** — A fast package manager and tool that allows you to quickly install packages, manage virtual environments, and run Python code efficiently. In the hands-on portion of this section, you'll be using `uv sync` to install and synchronize the dependencies defined in the project configuration (such as those listed in the *pyproject.toml* file), to ensure that your local development environment is set up correctly when configuring the AI agent to interact with the SageMaker AI inference endpoint.

One of the biggest advantages of Bedrock AgentCore is how quickly it lets you deploy AI agents. Instead of manually setting up infrastructure, managing servers, and handling scaling, you can get an agent up and running in just a few minutes. Bedrock AgentCore takes care of the heavy lifting so you can focus on building and customizing your agent's behavior. This makes it far easier to move from development to a fully operational agent. With this in mind, let's proceed with setting up a new Bedrock AgentCore project.

## Getting started with Amazon Bedrock AgentCore

In this section, you will set up a new Bedrock AgentCore project and verify that your local development environment is ready. You will start by installing the Bedrock AgentCore Starter Toolkit in a terminal session. Next, you will create a new project, explore its folder structure, and run the agent in development mode to confirm that everything is working as expected. Finally, you will test the agent by invoking it with a sample prompt and verifying that it responds as expected.

Let's proceed with creating our first Bedrock AgentCore project:

1. Open a new Launcher by selecting **File** | **New Launcher** from the menu. In the Launcher, start a new terminal session by choosing **Terminal** (under **Other**). Alternatively, you can start a new terminal session through the **File** menu by selecting **File** | **New**, then choosing **Terminal**.
2. In the terminal (**TERMINAL 1**), verify if the `uv` command is installed and accessible from your terminal by running the following command:

   ```
   which uv
   ```

   If installed, you should see the path where `uv` is located (for example, `/opt/conda/bin/uv`). If no path is returned, it means `uv` is not installed or not in the system's `PATH`.

> **Note**
>
> Make sure that `uv` is installed and accessible from your terminal before continuing. If the command `which uv` does not return a valid path, you will need to install `uv` before proceeding with the next steps.

3. Install the Bedrock AgentCore Starter Toolkit:

```
pip install bedrock-agentcore-starter-toolkit==0.2.7
```

   In this chapter, take note that we are using version `0.2.7` of the Bedrock AgentCore Starter Toolkit.

4. Use `agentcore create` to create a new AgentCore project named `helloSageMaker`:

```
agentcore create --project-name helloSageMaker
```

   This command will create a new directory called `helloSageMaker` containing the initial project files for your AgentCore project.

5. Navigate into the `helloSageMaker` project directory:

```
cd helloSageMaker/
```

6. Activate the virtual environment for the project:

```
source .venv/bin/activate
```

   This virtual environment isolates your project's dependencies, so you can safely install, upgrade, or remove libraries without impacting other projects or the system environment.

7. Install the `tree` command:

```
sudo apt install -y tree
```

8. Use the tree command to show the current directory contents in a tree-like format:

```
tree
```

You should see a tree structure that displays the files and directories within your `helloSageMaker` project directory (*Figure 2.4*).

```
.
├── README.md
├── pyproject.toml
├── src
│   ├── helloSageMaker.egg-info
│   │   ├── PKG-INFO
│   │   ├── SOURCES.txt
│   │   ├── dependency_links.txt
│   │   ├── requires.txt
│   │   └── top_level.txt
│   ├── main.py
│   ├── mcp_client
│   │   └── client.py
│   └── model
│       └── load.py
├── test
│   ├── __init__.py
│   └── test_main.py
└── uv.lock
```

*Figure 2.4 — Output after running the tree command*

The `helloSageMaker` project directory includes a *pyproject.toml* file, which serves as the central configuration for managing the project's dependencies and settings. This file helps ensure that all necessary libraries and frameworks are properly tracked and configured for the project. Within the *src/* directory, you'll find the *main.py* file, which contains the core logic of the agent. This file is where the agent's primary functionality is implemented, including interactions with any deployed models, data handling, and response generation.

9. Run the AgentCore agent locally in development mode:

```
agentcore dev
```

Since no files have been changed or modified yet, this command will run the initial scaffold version of the agent in local mode, allowing you to test the basic setup and functionality of your AgentCore project. Running the `agentcore dev` command should start the development server, similar to what is shown in *Figure 2.5*.

```
🚀Starting development server with hot reloading
Agent: helloSageMaker_Agent
Language: Python
Module: src.main:app
💡Test your agent with: agentcore invoke --dev "Hello" in a new terminal
window
ℹ️  This terminal window will be used to run the dev server
Press Ctrl+C to stop the server

Server will be available at:
  • Localhost: http://localhost:8080/invocations
  • 127.0.0.1: http://127.0.0.1:8080/invocations
  • Local network: http://169.255.255.2:8080/invocations

INFO:     Will watch for changes in these directories: ['/home/sagemaker-
user/helloSageMaker']
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO:     Started reloader process [3617] using StatReload
INFO:     Started server process [3619]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

*Figure 2.5 — Log output after running agentcore dev*

By running `agentcore dev`, the agent starts in local development mode. Since we haven't made any modifications yet, this gives you the chance to check the agent's basic functionality and confirm the initial setup is functioning properly.

10. Open a new terminal tab. Open a new Launcher by selecting **File** | **New Launcher** from the menu. In the Launcher, start a new terminal session by choosing **Terminal** (under **Other**).
11. In the terminal (**TERMINAL 2**), use the `agentcore invoke` command to invoke the AgentCore agent running locally:

```
agentcore invoke --dev "Hello Agent!"
```

You should get a response similar to the following:

```
✓ Response from dev server:
Hello! 👋 How can I help you today? I have access to several tools
including:

- **Code execution** - I can run Python, JavaScript, and TypeScript code in
a secure sandbox environment
- **Web search** - I can search the internet for current information
- **Code context search** - I can find documentation and examples for
programming libraries, APIs, and SDKs
```

```
- **Basic math** - I can perform calculations

What would you like to do?
```

12. Switch back to the first terminal tab (**TERMINAL 1**). Press *CTRL-C* to stop the current process.

At this point, your Bedrock AgentCore project is up and running locally, and you have verified that the setup is functioning as expected. With the initial setup working, you are ready to move on to customizing the agent to interact with a model hosted on a SageMaker AI inference endpoint.

**Note**

While it's tempting to immediately dive into modifying the *main.py* file and begin implementing your custom agent, it helps if the agent prepared by the scaffold is fully functional first. This will ensure that the core setup is working as expected, making it easier to troubleshoot any errors that may arise during the customization process.

## Customizing your agent to interact with a SageMaker AI endpoint

You will now configure and customize your AI agent to interact with a SageMaker AI endpoint. You will start by updating the project's *pyproject.toml* file to include the necessary dependencies and syncing the changes with `uv sync`. Next, you will edit the main application file (*src/main.py*) to import required modules, initialize the Bedrock Agent Core app and configure the agent to use the SageMaker AI inference endpoint. Finally, you will run the agent in development mode and invoke it with a sample prompt to observe its reasoning and tool usage.

**Note**

Before proceeding with this section, make sure that your SageMaker AI real-time inference endpoint (`sagemaker-endpoint-00`) is running and available. The AI agent will interact with this endpoint when generating responses, so the endpoint must be active for the examples in this section to work correctly. If the endpoint has been deleted, set up the endpoint again before continuing.

Let's go through the steps to customize our AI agent to interact with a SageMaker AI inference endpoint:

1. Using the File Browser, locate and open the file *helloSageMaker/pyproject.toml* file in your JupyterLab workspace:

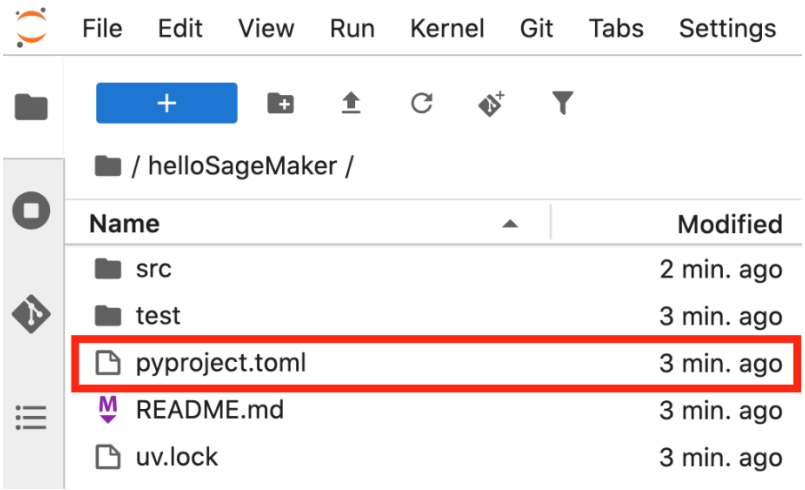



*Figure 2.6 — Locating the helloSageMaker/pyproject.toml file*

   To open the file, you can either double-click it or right-click and select **Open With | Editor**. This will open the *pyproject.toml* file in the JupyterLab editor so you can make your changes.

2. Add `strands-agents[sagemaker]` and `mypy-boto3-sagemaker-runtime` to the list of dependencies in your *pyproject.toml* file:

```
...

dependencies = [
    ...
    "strands-agents-tools >= 0.2.16",
    "strands-agents[sagemaker]",
    "mypy-boto3-sagemaker-runtime",
]
```

> **Note**
>
> Make sure there is a comma after `"strands-agents-tools >= 0.2.16"`.
>
> Omitting the comma will cause a syntax error in your *pyproject.toml* file.

3. Switch back to the terminal (**TERMINAL 1**) and run the following command to sync the changes:

```
uv sync
```

This should give you the following output:

```
(.venv) sagemaker-user@default:~/helloSageMaker$ uv sync
Resolved 162 packages in 249ms
      Built hellosagemaker @ file:///home/sagemaker-user/helloSageMaker
Prepared 11 packages in 979ms
Uninstalled 1 package in 0.40ms
Installed 11 packages in 22ms
 + boto3-stubs==1.42.31
 + botocore-stubs==1.42.31
 + distro==1.9.0
 ~ hellosagemaker==0.1.0 (from file:///home/sagemaker-user/helloSageMaker)
 + jiter==0.12.0
 + mypy-boto3-sagemaker-runtime==1.42.3
 + openai==1.109.1
 + sniffio==1.3.1
 + tqdm==4.67.1
 + types-awscrt==0.31.1
 + types-s3transfer==0.16.0
```

*Figure 2.7 — Log output after running uv sync*

The `uv sync` command ensures that your project's virtual environment matches the dependencies listed in *pyproject.toml* by installing or updating packages as needed. This keeps your development environment up to date with your project configuration.

4. Locate and open the file *helloSageMaker/src/main.py* in your JupyterLab workspace to view and edit the main application code.

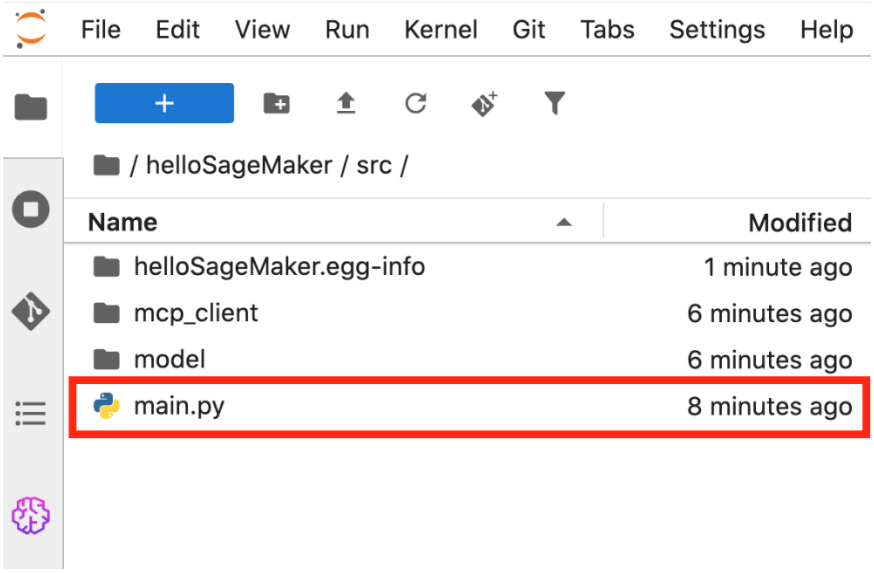



*Figure 2.8 — Locating the helloSageMaker/src/main.py file*

The *src/main.py* file contains the primary logic for your project, and you will make any necessary modifications to this file in the upcoming steps.

5. Delete all the existing code inside the *src/main.py* file.

> **Note**
>
> Since we will be adding our own custom code for this project, you should delete all the existing code in *src/main.py* to start with a clean slate.

6. Add the following code at the top of *src/main.py* to import the required packages and modules:

```
import os
from strands import Agent
from strands_tools import calculator, current_time
from strands.models.sagemaker import SageMakerAIModel

from bedrock_agentcore.runtime import (
    BedrockAgentCoreApp
)
from mcp_client.client import (
```

```
    get_streamable_http_mcp_client
)
```

7. Initialize the Bedrock Agent Core app, logger, and MCP client:

```
app = BedrockAgentCoreApp()
log = app.logger

REGION = os.getenv("AWS_REGION")
mcp_client = get_streamable_http_mcp_client()

SAIModel = SageMakerAIModel
EndpointConfig = SAIModel.SageMakerAIEndpointConfig
PayloadConfig = SAIModel.SageMakerAIPayloadSchema
```

8. Define the system prompt that instructs the agent to always use a tool when responding to user queries. Add the following after the imports and initialization:

```
SYSTEM_PROMPT=(
    "You MUST use at least one tool to answer the user's question."
    "Do not answer without calling a tool first."
)
```

9. Define a function to load and configure the SageMaker AI model with the endpoint and payload settings:

```
def load_sagemaker_model(endpoint_name="sagemaker-endpoint-00"):
    endpoint_config = EndpointConfig(
        endpoint_name=endpoint_name,
        region_name="us-east-1"
    )

    payload_config = PayloadConfig(
        max_tokens=4096, stream=True
    )

    model = SageMakerAIModel(
        endpoint_config=endpoint_config,
        payload_config=payload_config
    )
```

```
    return model
```

**Note**

Make sure to replace `"us-east-1"` with the AWS region where your SageMaker AI inference endpoint is deployed. Specifying the wrong region will prevent the agent from connecting to the endpoint.

10. Define the agent's entry point function to handle incoming prompts and stream responses:

```
@app.entrypoint
async def invoke(payload, context):
    with mcp_client as client:
        agent = Agent(
            model=load_sagemaker_model(),
            tools=[calculator, current_time],
            system_prompt=SYSTEM_PROMPT
        )

        stream = agent.stream_async(
            payload.get("prompt")
        )

        async for event in stream:
            if "data" in event and isinstance(
                event["data"], str
            ):
                yield event["data"]
```

11. Add the following code at the end of *src/main.py* to start the application when the script is executed directly:

```
if __name__ == "__main__":
    app.run()
```

> **Note**
>
> Don't forget to save *src/main.py* or the changes won't take effect when the application runs.

12. Switch back to the first terminal (**TERMINAL 1**) and run the following command to start the Bedrock AgentCore agent in development mode:

```
agentcore dev
```

13. Switch to the second terminal (**TERMINAL 2**). Run the following command to invoke the agent running in development mode with a query:

```
agentcore
invoke --dev "I was born on June 1, 1998, and I am 10 years older than my
brother. How old is my brother?"
```

This command invokes the agent to calculate the brother's age using a SageMaker endpoint (`sagemaker-endpoint-00`). The agent leverages tools (such as `calculator` and `current_time`) to compute the exact ages based on the current date. You should expect a response similar to:

```
✓ Response from dev server:
...
Your brother is 17 years old.

**Calculation:**
- Your age: 2026 (current year) - 1998 (birth year) - 1 (since June 1
hasn't occurred yet in 2026) = **27 years**
- Brother's age: 27 - 10 = **17 years**
```

If you are running these examples in 2027 or a future year, the calculated ages will differ based on the updated current year. The agent will adjust its calculations accordingly to reflect the new year.

At this point, you have successfully customized your AI agent to interact with a SageMaker AI endpoint and verified that it can correctly process queries and instructions using the configured tools. With your agent now fully functional, you are ready to proceed to deploying it in a production-ready runtime environment.

# Deploying production-ready agents with Bedrock AgentCore Runtime

In this section, you'll learn how to deploy production-ready AI agents using Bedrock AgentCore Runtime. You'll take the agent built in earlier sections and host it in a serverless runtime environment. This allows the agent to run at scale without requiring you to manage the underlying infrastructure. By the end of this section, you'll have a good idea of how to deploy AI agents with Bedrock AgentCore Runtime.

## Deploying an agent with Bedrock AgentCore Runtime

Deploying an agent with Bedrock AgentCore Runtime is straightforward. By using the `agentcore deploy` command, you can deploy the agent to the cloud in a single step. Since the agent relies on a model hosted on a SageMaker AI real-time inference endpoint, you'll configure the agent's execution role to include the necessary permissions to invoke that endpoint. After setting up the required permissions, you'll test the deployed agent by sending a sample prompt and verifying that it responds as expected.

**Note**

Behind the scenes, the `agentcore deploy` command packages your agent and deploys it to the **Bedrock AgentCore Runtime**, a fully managed serverless environment that handles scaling, execution, and request routing. This runtime provides a secure and reliable way for running AI agents in production, automatically managing the resources needed to handle multiple requests concurrently.

Now, let's walk through the steps to deploy your AI agent:

1. Switch to the first terminal tab (**TERMINAL 1**) and press *CTRL-C* to stop the current process.
2. Install the `zip` package, a prerequisite for `agentcore deploy`, using the following command:

```
sudo apt install -y zip
```

3. Deploy the agent using the following command:

```
agentcore deploy
```

You should see the following log output:

```
Launching Bedrock AgentCore (cloud mode - RECOMMENDED)...
   • Deploy Python code directly to runtime
   • No Docker required (DEFAULT behavior)
   • Production-ready deployment
...
Launching with direct_code_deploy deployment for agent
'helloSageMaker_Agent'
...
⠸ Launching Bedrock AgentCore...Role doesn't exist, creating new execution
role: AmazonBedrockAgentCoreSDKRuntime-us-east-1-...
Starting execution role creation process for agent: helloSageMaker_Agent
...
Deployment completed successfully – Agent: ...
...
```

Since the required execution role does not exist yet, the deployment process will automatically create a new IAM role. The Bedrock AgentCore resource(s) will use this role, which must have the necessary permissions to invoke the SageMaker AI real-time inference endpoint, as the endpoint is called within the `src/main.py` file.

4. Run the following command (in **TERMINAL 1**) to invoke the deployed agent with a query:

```
agentcore invoke "I was born on June 1, 1998, and I am 10 years older than
my brother. How old is my brother?"
```

Since the execution role does not have the required permissions to invoke the SageMaker AI real-time inference endpoint, you will encounter an `AccessDeniedException`:

```
{"error": "An error occurred (AccessDeniedException) when calling the
InvokeEndpointWithResponseStream operation: User:
arn:aws:sts::...:assumed-role/AmazonBedrockAgentCoreSDKRuntime-us-east-1-
abcde12345/BedrockAgentCore-... is not authorized to
perform: sagemaker:InvokeEndpoint on resource: arn:aws:sagemaker:us-
east-1:...:endpoint/sagemaker-endpoint-00 because no identity-based policy
allows the
sagemaker:InvokeEndpoint action", "error_type": "ClientError", "message":
"An error occurred during streaming"}
```

In case you're wondering why you didn't encounter this error earlier, it's because the space execution role used in development mode has the `SageMakerFullAccess` policy attached, which includes the necessary permissions to invoke the SageMaker AI inference endpoint.

> **Note**
>
> After you deployed the agent using `agentcore deploy`, the required permissions are not granted by default upon creation of the execution role, leading to the `AccessDeniedException`. With this, you'll need to manually attach the necessary SageMaker AI permissions to the agent's execution role before it can successfully invoke the deployed endpoint. Don't worry, as I'll guide you through adding the relevant permissions in the next steps!

5. Switch to the browser tab where the AWS Management Console is already open. Alternatively, you can open a new browser tab and navigate to the AWS Management Console. In the AWS Management Console, type `IAM` in the search bar, and then select **IAM** from the list of results. In the navigation pane (left sidebar), click **Roles** under **Access Management**.
6. In the search bar (highlighted in *Figure 2.9*), type `AmazonBedrockAgentCoreSDKRuntime` to locate the specific IAM role associated with the agent.

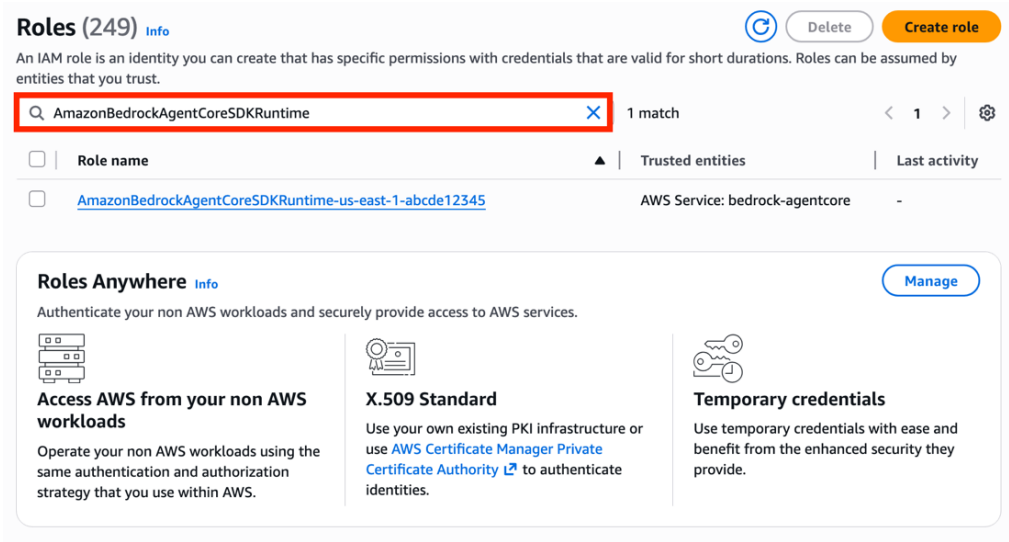



*Figure 2.9 — Searching for the IAM role*

Click the resulting role to open its details page. On the role's details page, click **Add permissions**. Then click **Attach policies**.

7. Under **Other permissions policies**, search for `AmazonSageMakerFullAccess`, select the checkbox next to it, and then click **Add permissions**. Allow a minute or two for the permissions to take effect before proceeding to the next step.

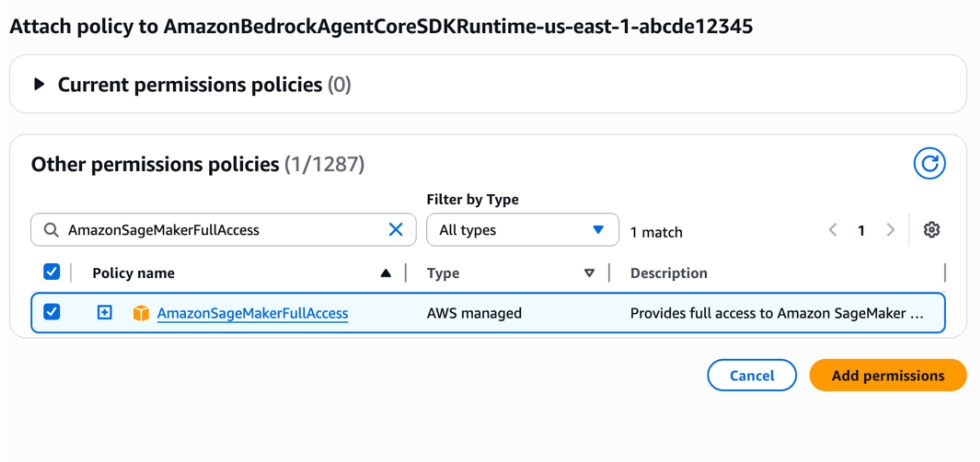



*Figure 2.10 — Attaching the AmazonSageMakerFullAccess policy*

The `AmazonSageMakerFullAccess` policy grants full access to all Amazon SageMaker AI resources, including the ability to create, update, and delete various SageMaker resources. With this policy attached, your deployed agent will now have the necessary permissions to invoke the SageMaker AI real-time inference endpoint without encountering the `AccessDeniedException` error.

**Note**

While the `AmazonSageMakerFullAccess` policy simplifies access management for the examples in this chapter, keep in mind that this policy is considered overly permissive and does not adhere to the principle of least privilege. For

> production workloads, it is best to define and attach more restrictive, custom IAM policies that grant only the specific permissions required.

8. Switch to the browser window with the JupyterLab terminal open. In the terminal, run the following command to invoke the deployed agent:

```
agentcore invoke "I was born on June 1, 1998, and I am 10 years older than
my brother. How old is my brother?"
```

This time, the agent will be successfully invoked without encountering an `AccessDeniedException`. After a few seconds, you should get a response similar to the following:

```
Your brother is 17 years old.

**Calculation:**
1. **Your current age:**
   Born on June 1, 1998. Since today is January 18, 2026 (before June 1),
you are **27 years old** (2025 - 1998 = 27).

2. **Brother's age:**
   You are 10 years older, so:
   $ 27 - 10 = 17 $.

**Brother's birthdate:** June 1, 2008.
```

Note that it may take 1 to 2 minutes for the agent to process the request and display the final output.

> **Note**
>
> Running `agentcore invoke` also returns the commands you can use to check the logs generated by the agent. These commands follow the format `aws logs tail ... --since 1h` and allow you to monitor the agent's activity and debug any issues. You should regularly check these logs, as they provide visibility into errors, execution flow, and unexpected behavior, helping you quickly identify issues and speed up troubleshooting.

*Wasn't that easy?!* You have successfully deployed your AI agent in a serverless runtime environment and tested it to ensure it functions as expected. As you can see, Bedrock AgentCore significantly speeds up the entire development and deployment process by automating project scaffolding, local testing, and serverless deployment. This allows you to focus on building and customizing your AI agent's logic instead of managing infrastructure. With the agent up and running, you can now proceed to clean up the resources created in this chapter to avoid unnecessary charges.

> **Note**
>
> This chapter only scratches the surface of what you can do with Bedrock AgentCore. In real-world scenarios, you can extend your agents with capabilities such as **AgentCore Memory**, **AgentCore Evaluations**, **AgentCore Gateway**, **AgentCore Browser Tool**, **AgentCore Code Interpreter**, **AgentCore Identity**, and **AgentCore Observability** to build more advanced, production-grade agent systems. You can visit `https://aws.amazon.com/bedrock/agentcore/resources/` for more information about this topic.

## Cleaning up

In this section, you will clean up the AWS resources created and used during the hands-on examples of this chapter (so far). More specifically, you will delete the Bedrock AgentCore resources and delete the SageMaker AI endpoint (`ml.g5.24xlarge`). These cleanup steps will help you avoid unnecessary charges and better manage your AWS costs.

> **Note**
>
> While not all resources will be deleted, performing these cleanup steps ensures that the most cost-impacting services, such as SageMaker AI endpoints, are properly stopped or removed.

Let's proceed with cleaning up the resources:

1. In the terminal, use the `agentcore destroy` command to delete the AWS resources that were created using the toolkit:

   ```
   agentcore destroy
   ```

   When prompted, enter `y` to confirm and proceed with the deletion of the resources.

2. Switch to the browser tab where the AWS Management Console is already open. Alternatively, you can open a new browser tab and navigate to the AWS Management Console. Navigate to the Amazon SageMaker AI console by typing `sagemaker ai` in the search bar and choosing **Amazon SageMaker AI** from the list of results.
3. In the navigation pane (left sidebar), click **Endpoints** under **Deployments & inference**. Select `sagemaker-endpoint-00`. Click **Actions** to open the dropdown menu, then choose **Delete** from the options.

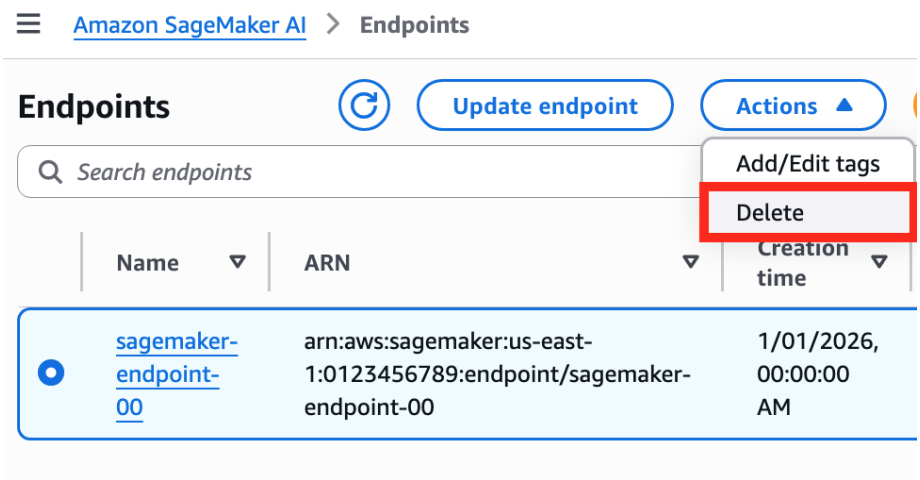



*Figure 2.11 — Deleting the SageMaker AI endpoint*

When prompted that this action will permanently delete your endpoint and cannot be undone, click **Delete**. Keep in mind that once the endpoint is deleted, any agents or applications that were interacting with it will no longer function and their requests to the endpoint will fail.

> **Note**
>
> We will set up this endpoint again later in the chapter during the hands-on walkthrough as part of building the RAG-powered agent.

## Setting up an Amazon Bedrock Knowledge Base

RAG-powered applications and agents use external data sources to improve response accuracy and context awareness. In this section, you will set up an Amazon Bedrock Knowledge Base, which provides the external data store that your application or agent can query at runtime. This enables the system to retrieve relevant information when additional context is needed to produce more accurate responses. To help you understand how retrieval-augmented

generation works in practice, the knowledge base used in this chapter includes sample entries with fictional scientific concepts such as `Xironal Flux Displacement` and `Subatomic Event Polarization`, along with imaginary timelines and experimental details. When queried, the agent can retrieve this information and use it as grounding context for answering questions or completing tasks. Without this retrieval capability, the system would rely only on the model's built-in training data. It would not be able to access external or domain-specific information, which can limit how accurate and up to date its responses are. As a result, it may struggle to produce grounded, context-aware answers for specialized or rapidly changing topics.

Setting up the Amazon Bedrock knowledge base is straightforward because the data is already stored in an Amazon S3 bucket. You simply connect the S3 bucket as the data source and allow Amazon Bedrock to handle ingestion and indexing. Once configured, the knowledge base organizes the content into a structure that supports efficient semantic search, enabling any RAG-powered application or agent to retrieve the most relevant context during execution.

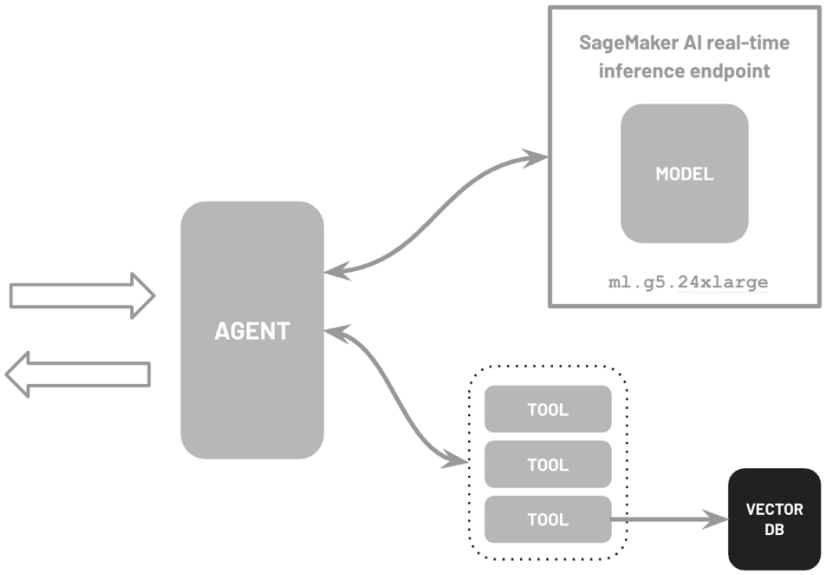



*Figure 2.12 — Where the Knowledge Base fits in the Agent Architecture*

As we proceed through the remaining sections of this chapter, you will see how the knowledge base provides the retrieval layer of the RAG workflow. You will build a RAG-powered AI agent that retrieves relevant context from the knowledge base at runtime to augment its responses. The agent orchestrates tool usage to process the specified prompt and incorporate retrieved information into its reasoning flow. In later sections, you will build an agent that retrieves information from the knowledge base and uses a model hosted on a SageMaker AI real-time inference endpoint to generate accurate, context-aware outputs.

**Note**

Make sure you are signed in using the `superadmin` IAM user before proceeding. This is required for setting up the Amazon Bedrock knowledge base and other related resources, as this IAM user has the necessary permissions to create and manage them. Keep in mind that this account has overly permissive access (`AdministratorAccess`), which is suitable for the hands-on examples in this chapter but is not recommended for production environments, where you should follow the principle of least privilege.

Let's proceed with setting up the Bedrock knowledge base:

1. Navigate to the S3 console by typing `s3` in the search bar and choosing **S3** from the list of results (*Figure 2.13*).

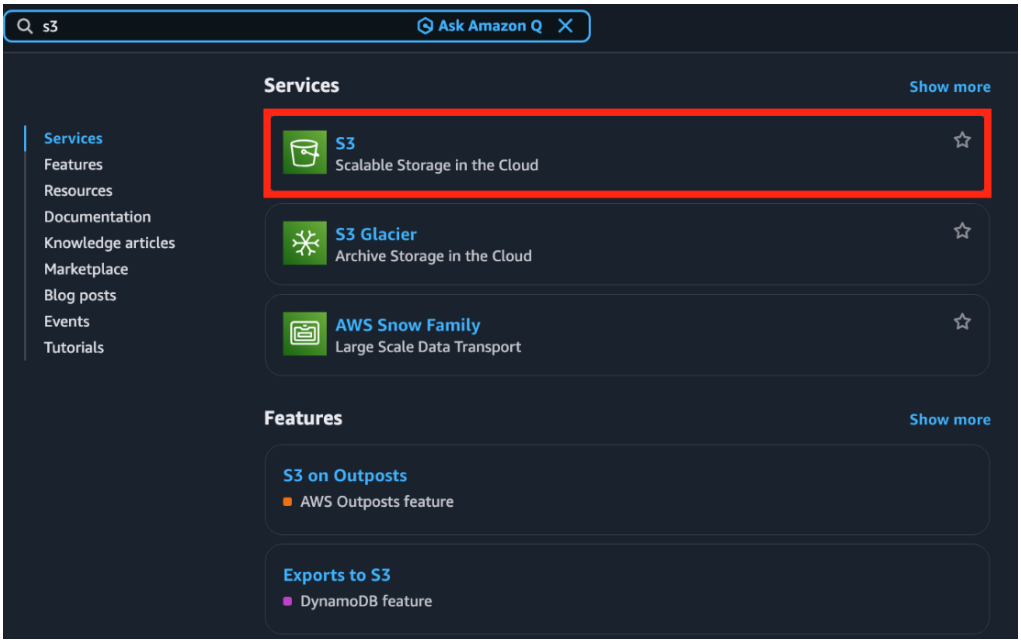



*Figure 2.13 — Navigating to the S3 Console*

   This should redirect you to the Amazon S3 console, where you can view and manage your buckets, objects, and configurations.

2. In the navigation pane (left sidebar), click **General purpose buckets** under **Buckets**. Click **Create bucket**. Under **Bucket name**, enter a unique name for the bucket (for example, `kb-data-source-abcdef12345`).

> **Note**
>
> Bucket names in Amazon S3 must be globally unique. If the name you select is already taken, you'll need to come up with a different one.

3. Click **Create bucket** to proceed with the bucket creation process.
4. Now, open a new browser tab, open the book's GitHub repository, and locate the `chapter02` folder. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.
5. Locate the *data.txt* file, and then open it to review the contents. You should see the following sample text:

```
In the latest study of quasi-differential fields, researchers have
uncovered an unusual phenomenon known as Xironal Flux Displacement (XFD).
This term refers to a unique form of energy distortion that occurs when
Fermathic Particles (FPs) interact with Tertial Waves (TWs). On January 8,
2025, the study, conducted by the Institute of Temporal Modulation (ITM),
posits that the XFD effect is responsible for the previously unexplained
discrepancies in Chronotemporal Synchronization (CTS) seen in deep space
missions.
...
```

   You will use this file as the source data when setting up your RAG knowledge base. Keep in mind that the information it contains is *entirely made up*, as this data will be used to test whether your RAG setup is working as intended.

6. Click the **Download raw file** button to save the *data.txt* file to your local machine. Then, switch back to the browser tab where the S3 console is already open and navigate to your S3 bucket.

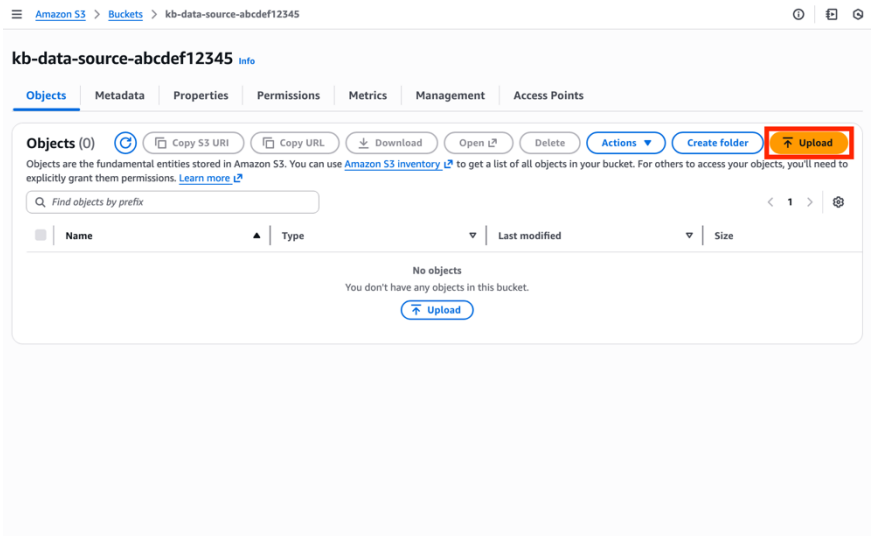



*Figure 2.14 — Uploading the data.txt file*

Click **Upload** to upload the *data.txt* file to your bucket (*Figure 2.14*). On the **Upload** page, click **Add files** and select *data.txt* from your local machine. Then click **Upload** to complete the process.

7. Navigate to the Amazon Bedrock console by typing `bedrock` in the search bar, and then selecting the **Amazon Bedrock** service from the list of search results.

> **Note**
>
> If you are not already signed in as the `superadmin` IAM user, you must sign in with those credentials before proceeding.

8. In the navigation pane (left sidebar), click **Knowledge Bases** under **Build**.
9. Click **Create**, then select **Knowledge Base with vector store** from the available options (*Figure 2.15*).

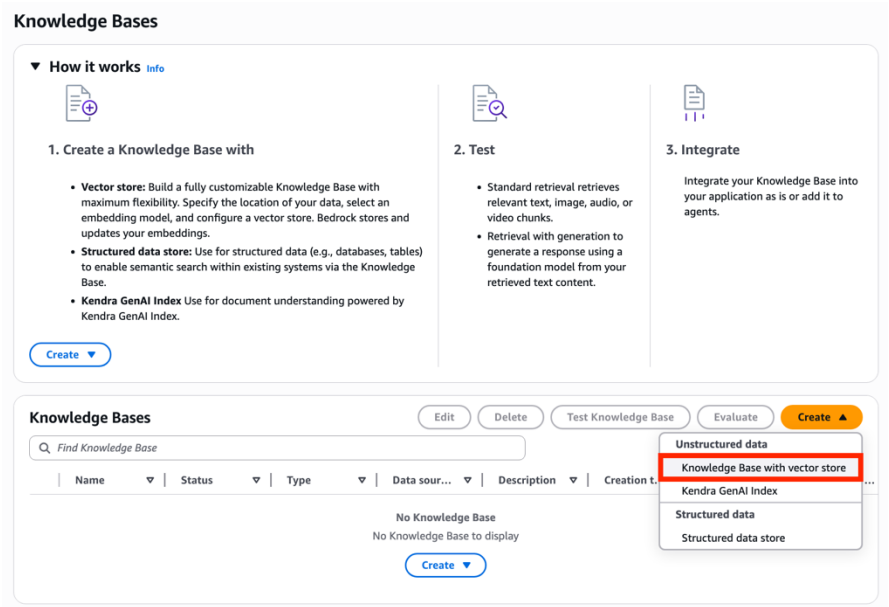



*Figure 2.15 — Creating a knowledge base with vector store*

This option allows you to create a knowledge base that breaks your source data into chunks, generates embeddings for each chunk, and stores them in a vector database. When a query is submitted, it is also converted into an embedding, and the system retrieves the most relevant chunks based on vector similarity rather than keyword matching.

10. In **Step 1: Provide Knowledge Base details**, under **Choose data source type**, select **Amazon S3** from the available options.

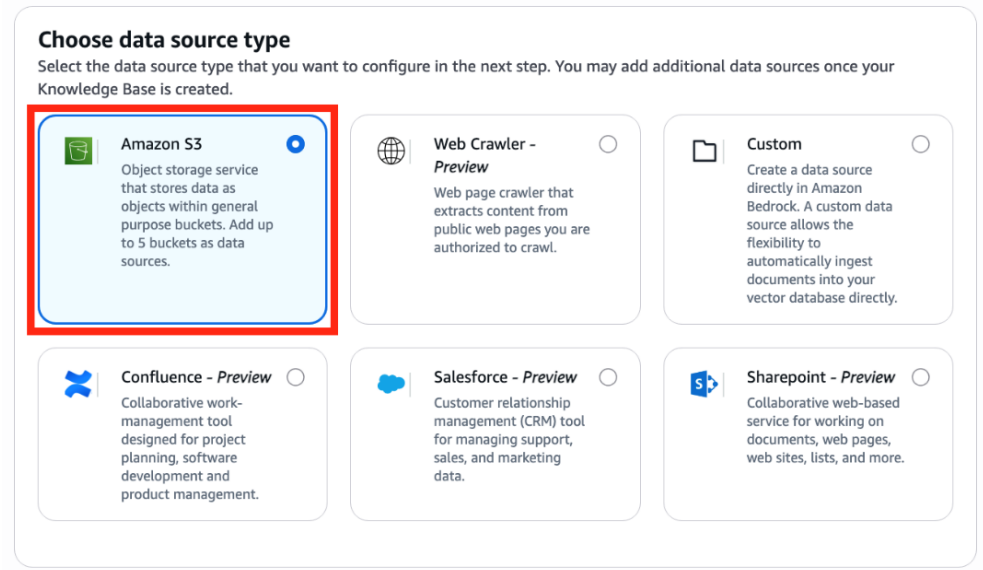



*Figure 2.16 — Choosing a data source type*

This option allows you to use an Amazon S3 bucket as the source of documents that will be ingested into the knowledge base. Click **Next** to proceed to the next step.

11. In **Step 2: Configure data source**, under **S3 URI**, click **Browse S3**. Choose the bucket you created earlier (for example, `kb-data-source-abcdef12345`).

> **Note**
>
> The name of your S3 bucket will be different from the example provided (`kb-data-source-abcdef12345`). Make sure to select the correct bucket you created earlier.

12. For **Parsing strategy**, choose **Amazon Bedrock default parser** (*Figure 2.17*).

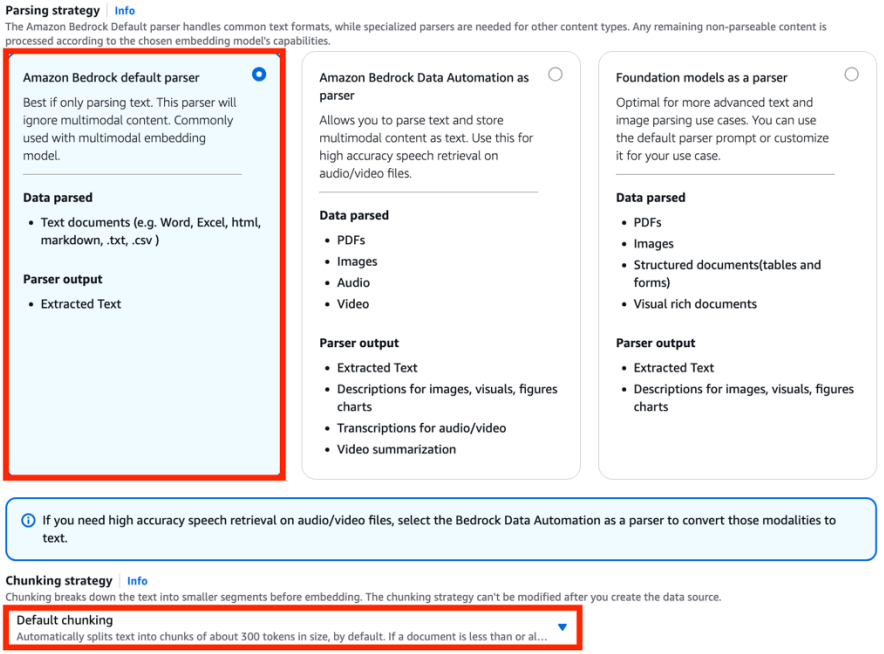



*Figure 2.17 — Selecting a parsing strategy*

For **Chunking strategy**, choose **Default chunking**. Click **Next** to proceed to the next step.

13. In **Step 3: Configure data storage and processing**, under **Embeddings model**, click **Select model**.

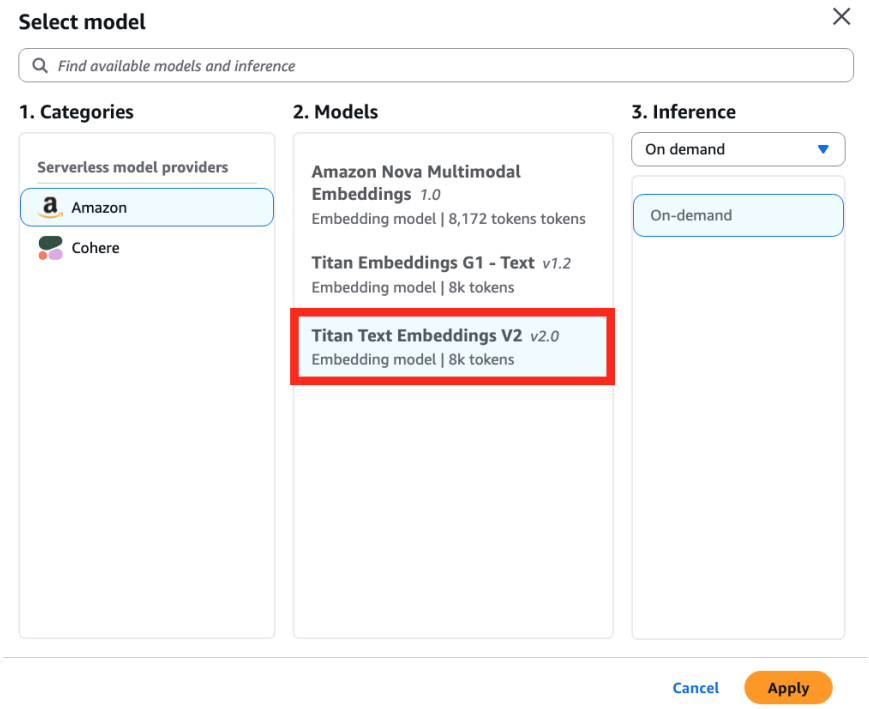



*Figure 2.18 — Selecting the embeddings model*

Choose **Titan Text Embeddings V2** and click **Apply** (*Figure 2.18*).

**Note**

You may choose a different embedding model if a newer version is available in your account or region, as long as it is compatible with Amazon Bedrock Knowledge Bases.

14. Under **Vector Store | Vector store type**, select **Amazon S3 Vectors** (*Figure 2.19*):

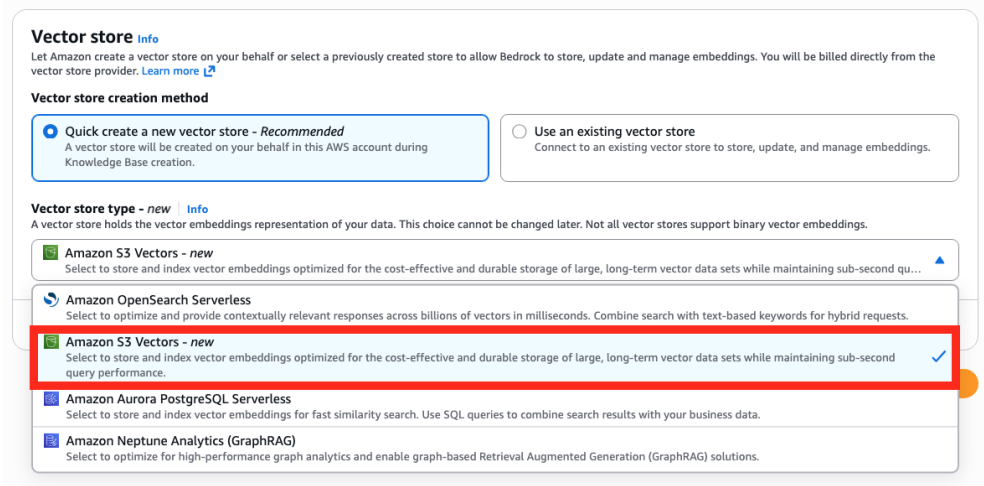



*Figure 2.19 — Choosing the vector store type*

Here, we're selecting Amazon S3 Vectors as the vector store type. This option provides a managed vector storage capability that allows embeddings to be stored in Amazon S3 and queried using similarity search, enabling efficient retrieval for RAG-powered workloads.

15. Click **Next** to proceed to the next step.
16. In **Step 4: Review and Create**, click **Create Knowledge Base**.

> **Note**
>
> You will see the notification that the knowledge base has been created successfully once the process is complete.

17. Under **Data source**, select the only available data source, then click **Sync** to load the uploaded *data.txt* file from S3 into the knowledge base.

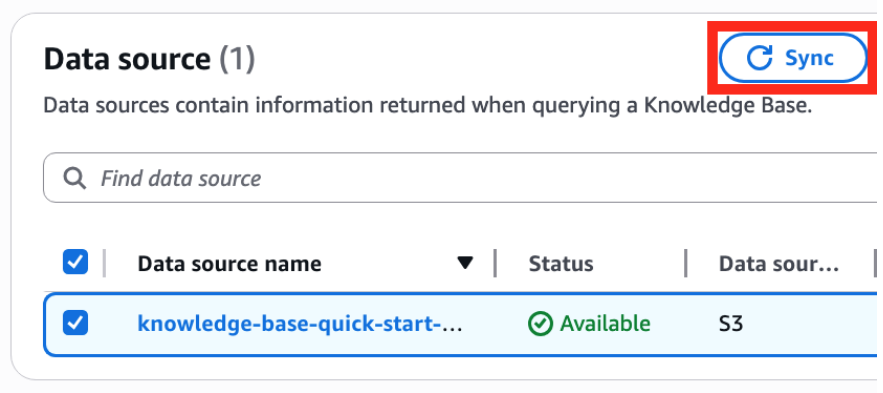



*Figure 2.20 — Syncing your data with your Amazon Bedrock knowledge base*

Here, the selected data source is being synchronized with the Amazon Bedrock knowledge base. During this process, the data stored in the S3 bucket (*data.txt*) is ingested, chunked, and converted into embeddings, which are then stored in the vector store to enable semantic search during retrieval.

**Note**

This step may take a minute or two to complete.

18. Copy the **Knowledge Base ID** value and paste it into a code editor on your local machine. You'll need this ID when using the knowledge base in your code later.

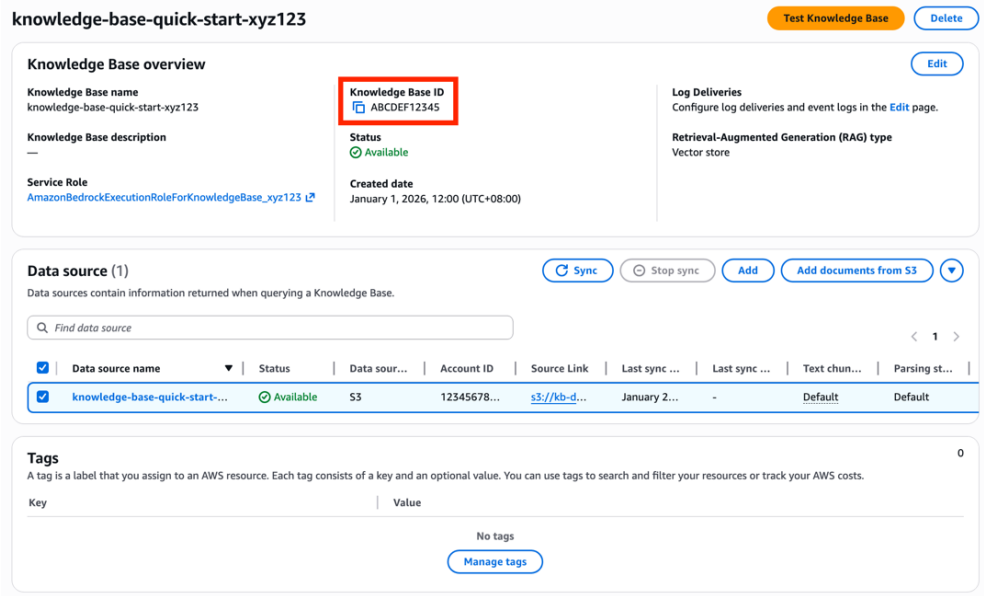



*Figure 2.21 — Locating the Knowledge Base ID*

You will find the **Knowledge Base ID** under the **Knowledge Base overview** section (*Figure 2.21*).

19. Click **Test Knowledge Base**. This will redirect you to a test interface where you can submit queries and verify that the knowledge base is correctly retrieving relevant information from your data.
20. Under **Retrieval and response generation**, choose **Retrieval only: data sources**.
21. In the **Preview** text area, enter the following:

```
Xironal Flux Displacement
```

Click the arrow (**Send message**) to proceed. This will generate an output similar to the following:

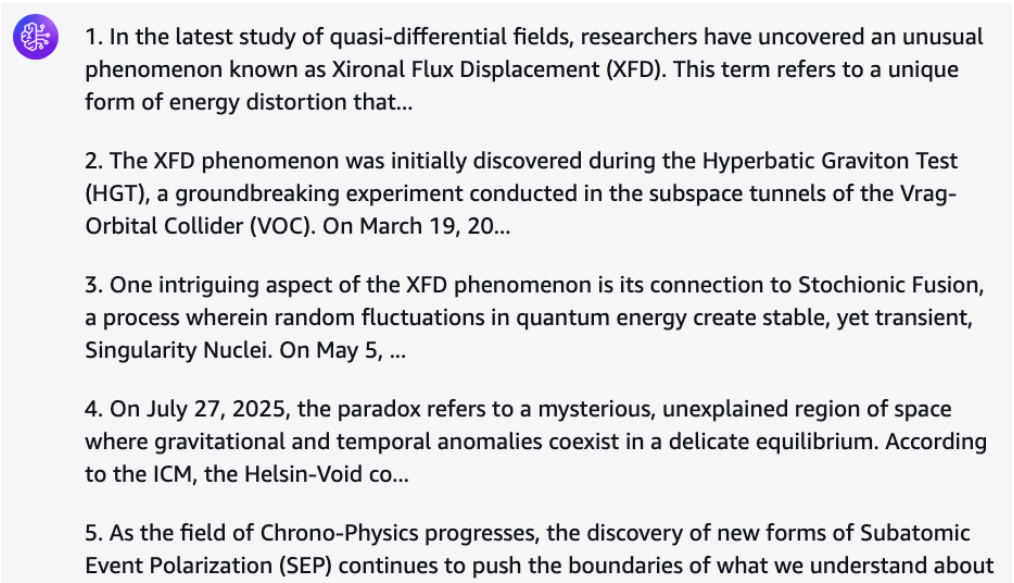



*Figure 2.22 — Retrieval only: data sources*

Here, a retrieval operation is performed against the knowledge base using the provided query (`Xironal Flux Displacement`). The most relevant content is identified and returned based on semantic similarity from the indexed data sources.

22. Under **Retrieval and response generation**, select **Retrieval and response generation: data sources and model**. Under **Categories**, select **Amazon**. Click **Select model**, choose **Nova Lite 1.0**, and then click **Apply**.
23. In the **Preview** text area, enter the following:

```
When was the Xironal Flux Displacement phenomenon discovered?
```

You should see an output similar to the following:

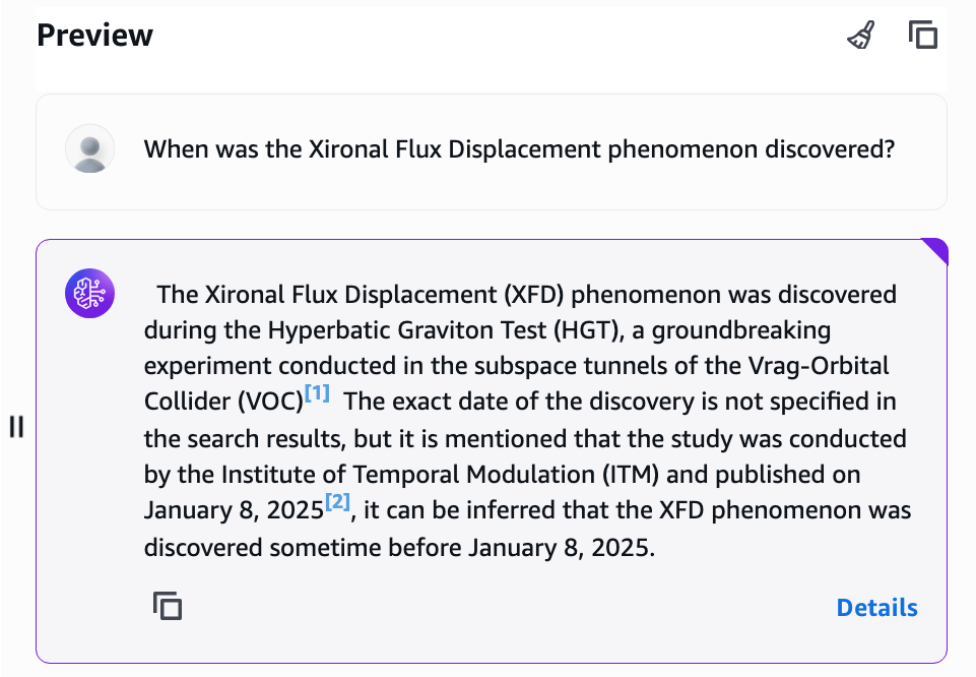



*Figure 2.23 — Retrieval and response generation: data sources and model*

Here, a combined retrieval, and generation process is executed using both the selected data source and the chosen model. Relevant context is first retrieved from the knowledge base, then passed to the model to generate a natural language response.

**Note**

Unlike the previous example, where only matching information was retrieved, you can now ask a question in natural language. The process uses the knowledge base to retrieve relevant context, which is then passed to the model to generate a complete answer.

24. Click **Details** to see the source details, where each container displays a chunk of data from the source related to your query.

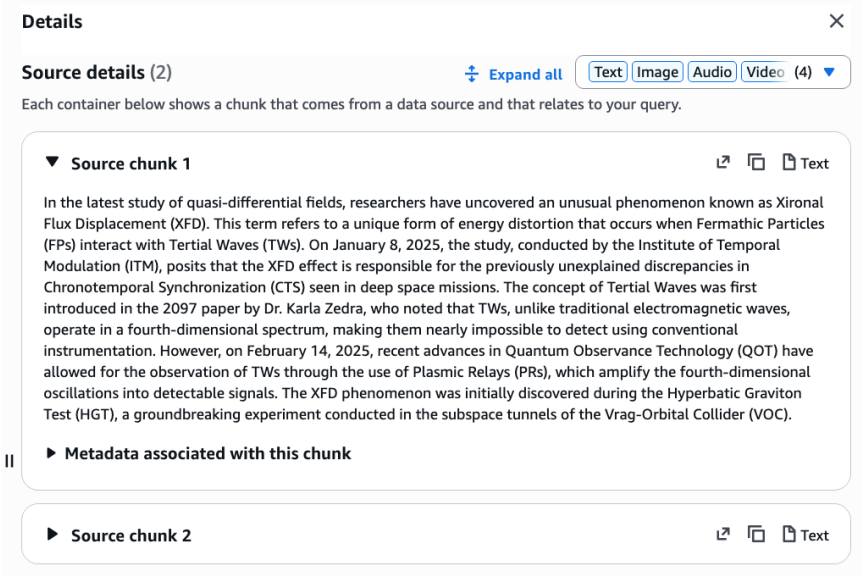



*Figure 2.24 — Source details*

Here, **Source details** helps you explore the individual text chunks that were retrieved from the knowledge base. Each container represents a specific segment of the original data that matched the query based on semantic similarity. This allows you to trace the generated response back to its underlying source content for better transparency and explainability.

*Excellent work so far!* Now that your knowledge base is ready and working as expected, we can proceed with building a simple RAG-powered AI agent in the next section. This will help you explore how retrieval and model reasoning work together in practice.

# Building a RAG-powered AI agent with Strands Agents

With the Bedrock knowledge base ready, you can now proceed to build the RAG-powered AI agent using Strands Agents. In this section, you'll explore how to interact directly with a Bedrock knowledge base, process retrieval results, and turn this into a reusable tool that the agent can call when needed, alongside other available tools. This allows the agent to pull in relevant and up-to-date information at runtime instead of relying only on what it already knows, leading to more accurate and context-aware responses while also supporting more dynamic, multi-step tasks.

To integrate a vector database with Strands Agents, you typically expose it as a tool that the agent can invoke during execution. Strands does not depend on any specific vector database, so the integration pattern remains consistent where you wrap the retrieval logic in a tool function and pass it to the agent.

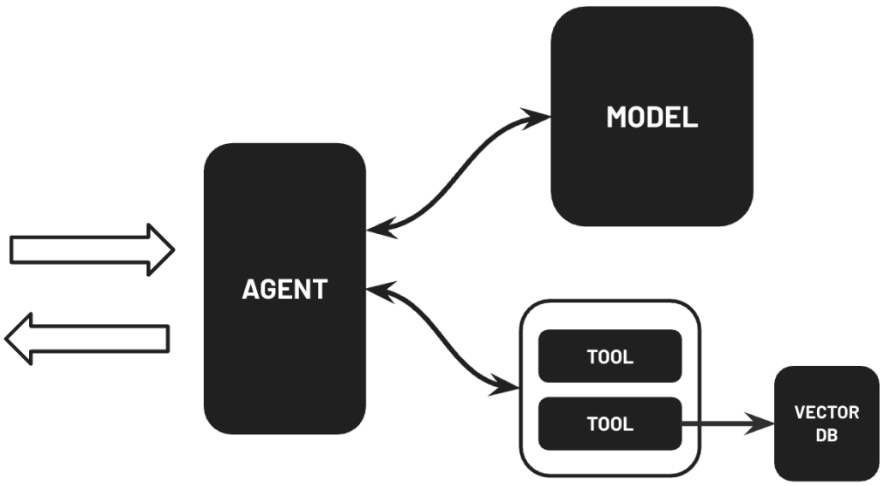



*Figure 2.25 — Building an RAG-powered AI agent with Strands Agents*

In practice, your vector search or knowledge base query becomes a callable function that returns relevant context at runtime. This approach also makes it easier to switch between different retrieval backends or extend the system without changing the agent's core logic. Strands Agents simplifies this process by using the `@tool` decorator to easily turn functions into agent-accessible tools. In this section, you will define a `search_vector_database()` function that the agent can use as a reusable tool for retrieving information from the knowledge base. This function will then be included when you set up and configure the agent, so it can use the knowledge base whenever your AI agent needs additional context during execution.

**Note**

Keep in mind that it is also possible to build RAG-powered applications without AI agents. You can implement RAG using a simple retrieval pipeline where a model directly consumes retrieved context without any agent-based orchestration. AI agents become useful when you need additional capabilities such as tool use, multi-step reasoning, or dynamic decision-making beyond straightforward retrieval and generation.

With these in mind, let's go through the steps to build the RAG-powered AI agent using Strands Agents:

1. Switch back to the browser tab where your Amazon SageMaker Studio space (`mle-on-aws-space`) is open.
2. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**).
3. Rename the notebook to `rag.ipynb` before proceeding. We will run the code in the following steps within this notebook.
4. In a new code cell of the Jupyter Notebook, run the following to install the required packages:

```
%pip install strands-agents==1.22.0
%pip install strands-agents-builder==0.1.10
%pip install strands-agents-tools==0.2.19
%pip install 'strands-agents[sagemaker]'
%pip install strands-agents-tools
```

5. Restart the kernel by selecting **Kernel | Restart Kernel** from the menu. This will ensure the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**.

> **Note**
>
> Wait for 2 to 3 minutes for the kernel to fully restart.

6. Set the following variables to specify the AWS region and Knowledge Base ID.

```
REGION='<SPECIFY REGION>'
KNOWLEDGEBASE_ID='<SPECIFY KNOWLEDGE BASE ID>'
```

Make sure to replace `<SPECIFY REGION>` with the AWS region where your resources are deployed (for example, `us-east-1`), and `<SPECIFY KNOWLEDGE BASE ID>` with the Amazon Bedrock Knowledge Base ID you obtained earlier before running the command.

7. Initialize the Agents for Amazon Bedrock Runtime client using the region you specified earlier:

```
import boto3
runtime = boto3.client(
    "bedrock-agent-runtime",
    region_name=REGION
)
```

8. Perform a retrieval query to search for relevant information from the knowledge base:

```
query = "Xironal Flux Displacement"

retrieval_config = {
    "vectorSearchConfiguration": {
        "numberOfResults": 3
    }
}

response = runtime.retrieve(
    knowledgeBaseId=KNOWLEDGEBASE_ID,
    retrievalQuery={"text": query},
    retrievalConfiguration=retrieval_config
)
```

Here, you submit a query to the knowledge base and retrieve the top 3 matching results using the specified configuration.

9. Inspect the data returned in the response:

```
response['retrievalResults']
```

This should give you the following set of results:

```
[
  {
    'content': {'text': 'In the latest study of quasi-differential fields,
researchers have uncovered an unusual phenomenon known as Xironal Flux
Displacement (XFD). This term refers to a unique form of energy distortion
that occurs when Fermathic Particles (FPs) interact with Tertial Waves
(TWs) ...',
```

```
    ...,
    'score': 0.5297976639182721
  },
  {

'content': {'text': 'The XFD phenomenon was initially discovered during the
Hyperbatic Graviton Test (HGT), a groundbreaking experiment conducted in
the subspace tunnels of the Vrag-Orbital Collider (VOC). On March 19, 2025,
during this test, researchers were shocked to find that the interaction of
FPs with the Gravizon Matrix—a dense gravitational construct theorized to
exist at the quantum level—created a cascading loop of nonlinear time
feedback ...',
    ...,
    'score': 0.4044778450396061},
  {

'content': {'text': 'One intriguing aspect of the XFD phenomenon is its
connection to Stochionic Fusion, a process wherein random fluctuations in
quantum energy create stable, yet transient, Singularity Nuclei. ...',
    ...,
    'score': 0.403453655418125
  }
]
```

Here, you have the top 3 most relevant text entries along with their associated metadata and relevance scores.

10. Process the response to extract the text content from the results:

```
m = map(
    lambda r: r.get(
        'content', {}
    ).get('text', ''),
    response.get('retrievalResults', [])
)

results = list(filter(None, m))
```

11. Display the filtered results:

```
results
```

You should see a list of the top 3 retrieved text entries that are most relevant to the query "`Xironal Flux Displacement`".

```
['In the latest study of quasi-differential fields, researchers have
uncovered an unusual phenomenon known as Xironal Flux Displacement
(XFD)...',
 'The XFD phenomenon was initially discovered during the Hyperbatic
Graviton Test (HGT), a groundbreaking experiment conducted in the subspace
tunnels of the Vrag-Orbital Collider (VOC)...',
 'One intriguing aspect of the XFD phenomenon is its connection to
Stochionic Fusion, a process wherein random fluctuations in quantum energy
create stable, yet transient, Singularity Nuclei...']
```

12. Define a tool function (`search_vector_database`) to search the knowledge base for relevant context based on the user's query:

```
from strands import tool

@tool
def search_vector_database(query: str) -> str:
    """
    Use the knowledge base to find a specific topic by searching the
knowledge base for relevant context to answer questions.
    Args:
        query: A question about concepts or terminology
    Returns:
        A formatted string response generated from the knowledge base
    """

    runtime = boto3.client("bedrock-agent-runtime",
                           region_name=REGION)

    try:
        response = runtime.retrieve(
            knowledgeBaseId=KNOWLEDGEBASE_ID,
            retrievalQuery={"text": query},
            retrievalConfiguration={
                "vectorSearchConfiguration": {
                    "numberOfResults": 3
                }
            }
```

```
        )

        m = map(
            lambda r: r.get(
                'content', {}
            ).get('text', ''),
            response.get('retrievalResults', [])
        )

        results = list(filter(None, m))

        return "\n\n".join(results) or "No results."
    except Exception as e:
        return f"Error: {str(e)}"
```

Here, you reuse the retrieval logic used in previous steps and wrap it into a reusable tool function that the agent can call.

13. Verify that the `search_vector_database()` function is working as expected by calling it with a sample query to retrieve relevant results from the knowledge base:

```
search_vector_database("Xironal Flux Displacement")
```

You should get the following output:

```
'In the latest study of quasi-differential fields, researchers have
uncovered an unusual phenomenon known as Xironal Flux Displacement (XFD).
This term refers to a unique form of energy distortion that occurs when
Fermathic Particles (FPs) interact with Tertial Waves (TWs). On January 8,
2025, the study, conducted by the Institute of Temporal Modulation (ITM),
posits that the XFD effect is responsible for the previously unexplained
discrepancies in Chronotemporal Synchronization (CTS) seen in deep space
missions.

...

On July 27, 2025, the paradox refers to a mysterious, unexplained region of
space where gravitational and temporal anomalies coexist in a delicate
equilibrium.'
```

14. Now that you have verified that the `search_vector_database()` function is working as expected, initialize a `BedrockModel` instance by passing in the model ID for **Claude Sonnet 4.6 by Anthropic**:

```
from strands.models.bedrock import BedrockModel

mid = "us.anthropic.claude-sonnet-4-6"
model = BedrockModel(model_id=mid)
```

> **Note**
>
> If you are unsure which `model_id` to use, navigate to the Amazon Bedrock console and open **Model catalog** (under **Discover**). Search for and select a model (for example, **Claude Sonnet 4.6 by Anthropic**), then copy the **Model ID** value (for example, `anthropic.claude-sonnet-4-6`) from the model details page. Using the navigation pane (left sidebar), you can also navigate to **Inference profiles** (under **Infer**) to view available inference profiles along with their **Inference profile ID** values. You can use the **Inference profile ID** value as the `model_id` when initializing an `Agent` or `BedrockModel` instance. If you haven't worked with inference profiles before, they act as an abstraction layer that unifies model ARNs from different regions under one identifier. For more information, feel free to check the following AWS blog post: `https://aws.amazon.com/blogs/machine-learning/getting-started-with-cross-region-inference-in-amazon-bedrock/`.

15. Configure a new Agent that uses the specified Bedrock model (**Claude Sonnet 4.6 by Anthropic**) and tools (`search_vector_database`, `current_time`) to answer user queries:

```
from strands import Agent
from strands_tools import current_time

SYSTEM_PROMPT= (
    "You are a RAG agent. When a user asks you a question you will first
     check it in your knowledge base (if you can't answer it
     from the current conversation memory)."
    "You MUST use at least one tool to answer the user's question."
    "Do not answer without calling a tool first."
```

```
)

agent = Agent(
    model=model,
    tools=[search_vector_database, current_time],
    system_prompt=SYSTEM_PROMPT
)
```

16. Use the agent to process a prompt and retrieve relevant information from the knowledge base:

```
prompt = """Get the current month and then tell me more about the Xironal
Flux Displacement and Subatomic Event Polarization stored in the knowledge
base which happened during the same quarter the last few years"""
result = agent(prompt)
```

You should get the following output:

```
Sure! Let me start by fetching the current time and searching the knowledge
base for both topics simultaneously!
Tool #1: current_time

Tool #2: search_vector_database

...

## 🗓️ Current Date & Quarter
The current date is **April 15, 2026**, which places us in **Q2 (April–
June)**. So I'll focus on events from the knowledge base that occurred
during **Q2 (April, May, June)** in recent years.

---

## ⚡ Xironal Flux Displacement (XFD) — Q2 Events

**Xironal Flux Displacement (XFD)** is a unique form of energy distortion
that occurs when **Fermathic Particles (FPs)** interact with **Tertial
Waves (TWs)** — fourth-dimensional waves first theorized by Dr. Karla Zedra
in 2097. Here are the key Q2 developments:

...
```

Here, the agent executes the prompt by calling the appropriate tools to retrieve the current time and relevant knowledge base information. It then uses these results to generate a structured response.

**Note**

Keep in mind that the agent's output may vary between runs. This is expected behavior, as the agent may choose different tool execution paths or interpret the prompt differently depending on the context and intermediate results. At the same time, since the result of the `current_time` tool depends on the exact moment of execution, the final output may also change based on when the agent runs. Feel free to modify the prompt to better suit your own use case or experiment with different variations to observe how the agent behaves.

At this point, you should have a good understanding of how to build RAG-powered AI agents. The core idea is to combine a model with external retrieval and expose useful capabilities as tools the agent can use at runtime. Once you are familiar with this pattern, you can apply it to a wide range of use cases, from simple question answering systems to more advanced autonomous workflows.

## Building a RAG-powered AI agent that interacts with a SageMaker AI inference endpoint

In this section, you will build an AI agent that interacts with a model deployed in a SageMaker AI real-time inference endpoint and retrieves relevant information from a knowledge base. You will start by creating an inference endpoint from an existing endpoint configuration that runs on a dedicated managed infrastructure using the `ml.g5.24xlarge` instance type. You will then use Strands Agents to build and run a RAG-powered AI agent to generate a structured set of directories and files.

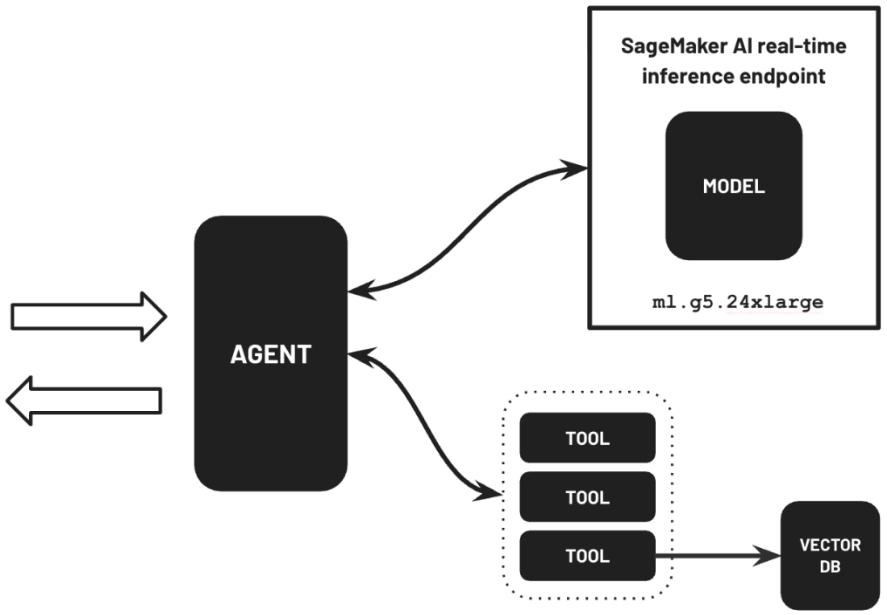



*Figure 2.26 — Building a RAG-powered AI agent that interacts with a SageMaker AI endpoint*

Similar to the previous section, the vector database will be integrated through a tool that the agent can call during execution. This tool-based approach allows the agent to retrieve relevant context from the knowledge base whenever it needs additional information. It also keeps the retrieval logic modular and separate from the agent's core reasoning process.

You'll work with a Python script that makes use of Strands Agents to bring together all the building blocks required for the RAG-powered agent. This includes the SageMaker AI real-time inference endpoint for model interaction, the Bedrock knowledge base for information retrieval, as well as tools for handling time (`current_time`) and performing shell-based operations (`shell`). This script will have a `main()` function that initializes the agent and executes it using the configured model, tools, and system prompt.

```
def main():
    agent = Agent(
        model=get_model(),
        tools=[
            search_vector_database,
            current_time,
            shell
        ],
```

```
        system_prompt=SYSTEM_PROMPT
    )

    result = agent(prompt)
    print(result)
```

When executed, the script runs the RAG-powered AI agent, retrieving relevant information from the knowledge base, invoking tools when needed, and generating a final response that is printed to the console. Once you have a good idea of how to build a basic RAG-powered AI agent, it becomes much easier to extend the same pattern to more complex systems. The core concepts of combining model reasoning, external retrieval, and tool use remain consistent, which allows you to scale from simple workflows to more advanced multi-step agentic applications.

**Note**

Since this section uses an `ml.g5.24xlarge` instance, it is recommended that you complete the entire hands-on walkthrough in one session. This helps avoid leaving the real-time inference endpoint running longer than necessary. Once you have finished the exercises, make sure to delete the endpoint to prevent any unnecessary resource usage and cost. To better understand the potential costs involved when running SageMaker AI real-time inference endpoints, take a moment to review the pricing information before proceeding (`https://aws.amazon.com/sagemaker/ai/pricing/`).

## Setting up an Inference Endpoint from an Endpoint Configuration

Before your RAG-powered agent can interact with a SageMaker AI endpoint, you must have an inference endpoint running. In SageMaker AI, this endpoint is based on an endpoint configuration that specifies the model and infrastructure settings. While you might be used to creating endpoints through the SageMaker Python SDK, you can also create one directly from the AWS Management Console using an existing endpoint configuration.

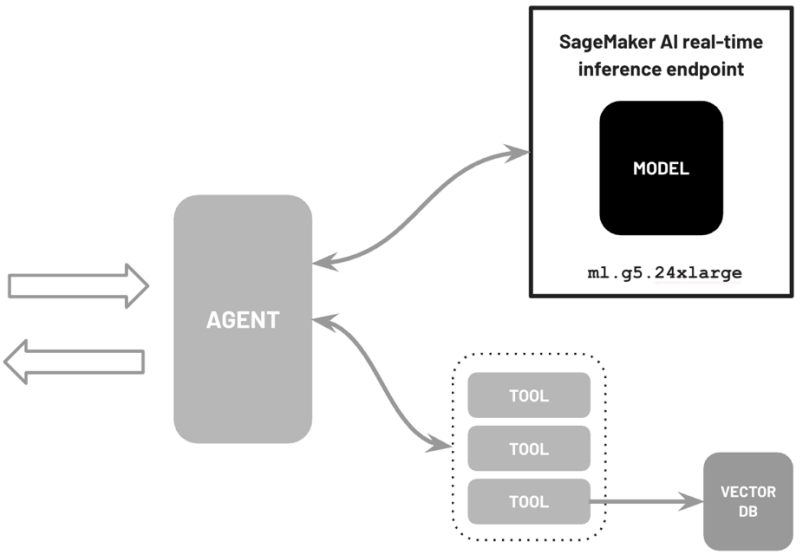



*Figure 2.27 — Where the SageMaker AI inference endpoint fits in the Agent Architecture*

In this section, you'll create a SageMaker AI real-time inference endpoint from an existing endpoint configuration. Once the endpoint is running, it becomes part of a broader RAG-powered agent workflow that also uses a knowledge base and additional tools. The agent uses this real-time endpoint for model inference and combines its outputs with context retrieved from the knowledge base. This integration enables the system to produce more accurate, grounded, and context-aware responses.

Let's proceed with creating the SageMaker AI real-time inference endpoint:

1. Switch back to the browser tab where SageMaker AI console is open. In the navigation pane (left sidebar), select **Endpoints** under **Deployments & inference**.
2. Click **Create endpoint**.
3. Under **Endpoint name**, specify `sagemaker-endpoint-00`.
4. Choose **Use an existing endpoint configuration**. Select `sagemaker-endpoint-00` under **Endpoint configuration**, then click **Select endpoint configuration**.
5. Click **Create endpoint** to start the endpoint creation process.

**Note**

This step may take anywhere from 10 to 15 minutes. You may want to grab a cup of coffee or tea while you wait.

*Wasn't that easy?!* By leveraging an existing endpoint configuration, you can quickly and easily create a new inference endpoint. While the endpoint is being created, you can proceed to the next section and continue building the RAG-powered agent.

## Building the RAG-powered agent

In this section, you will create a Python script that defines and runs the RAG-powered agent. The script integrates tools, a system prompt, a SageMaker AI inference endpoint, and a knowledge base into a single workflow. It uses Strands Agents to orchestrate the agent's reasoning process and tool execution.

Let's proceed with building the RAG-powered agent using Strands Agents:

1. Switch back to the browser tab where your Amazon SageMaker Studio space (`mle-on-aws-space`) is open.
2. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new file by choosing **Python File** (under **Other**). Rename the file to `main.py` before proceeding.

   **Note**

   We will add the code blocks in the following steps to this file. The complete source code is also available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

3. In the `main.py` file, import the necessary libraries, classes, and components for setting up your agent with the required tools and model.

```
import boto3
from strands import Agent, tool
from strands_tools import (
    current_time,
    shell
)
from strands.models.sagemaker import SageMakerAIModel
```

4. Define the necessary variables for your configuration, such as the region, knowledge base ID, and endpoint name:

```
REGION = 'us-east-1'
KNOWLEDGEBASE_ID = '<SPECIFY KNOWLEDGEBASE ID>'
ENDPOINT_NAME = 'sagemaker-endpoint-00'
```

Make sure to replace `<SPECIFY KNOWLEDGEBASE ID>` with your Bedrock Knowledge Base ID. If you are working in a different region, also update the `REGION` value accordingly to match where your resources are deployed.

> **Note**
>
> The code in *main.py* assumes that the SageMaker AI real-time inference endpoint name is `sagemaker-endpoint-00`. If your endpoint has a different name, update the `ENDPOINT_NAME` variable accordingly to match your deployed endpoint.

5. Define the system prompt that configures the behavior of your agent:

```
SYSTEM_PROMPT = (
    "You are a RAG agent. When a user asks you a question you will first
     check it in your knowledge base (if you can't
     answer it from the current conversation memory)."
    "You MUST use as many tools as possible to answer the user's question."
    "You MUST use the shell tool when working with files and directories."
    "Do not answer without calling a tool first."
)
```

Here, we have a sample system prompt used to configure the agent's behavior and tool usage.

> **Note**
>
> Once you've completed the hands-on examples in this chapter, feel free to modify this to suit your specific use case and better control how the agent responds and interacts with tools.

6. Define the prompt for the agent to gather information and create directories based on data from the knowledge base:

```
prompt = """
Get the current month and then tell me more about the Xironal Flux
Displacement and Subatomic Event Polarization stored in the knowledge base
which happened during the same quarter the last few years. Then, using the
generated output, create 1 directory for each month of the said quarter
(for example, JAN_2025) and create a text file (for example, info.txt)
inside each directory that contains 1-2 sentences relevant to what happened
that month. Make sure that the number of directories created is equal to
the number of months in the quarter (for example, only 3 months in a
quarter).

I am expecting the following file and folder structure to look like this:

JAN_2024/
    info.txt

FEB_2024/
    info.txt
"""
```

This prompt instructs the agent to retrieve relevant knowledge base information, determine the correct quarter based on the current month, and then organize the output into a structured set of directories and files.

7. Define a `search_vector_database()` function as a tool to query the knowledge base:

```
@tool
def search_vector_database(query: str) -> str:
    runtime = boto3.client(
        "bedrock-agent-runtime",
        region_name=REGION
    )

    try:
        response = runtime.retrieve(
            knowledgeBaseId=KNOWLEDGEBASE_ID,
            retrievalQuery={"text": query},
            retrievalConfiguration={
```

```
                "vectorSearchConfiguration": {
                    "numberOfResults": 3
                }
            }
        )

        m = map(
            lambda r: r.get(
                'content', {}
            ).get('text', ''),
            response.get('retrievalResults', [])
        )

        results = list(filter(None, m))

        return "\n\n".join(results) or "No results."
    except Exception as e:
        return f"Error: {str(e)}"
```

Here, the `@tool` decorator registers the `search_vector_database()` function as an executable tool that the agent can invoke at runtime to access external capabilities such as querying a knowledge base.

8. Next, define the `get_model()` function to configure and return an instance of the `SageMakerAIModel`.

```
def get_model():
    SM = SageMakerAIModel
    EPConfig = SM.SageMakerAIEndpointConfig
    PLConfig = SM.SageMakerAIPayloadSchema

    endpoint_config = EPConfig(
        endpoint_name=ENDPOINT_NAME,
        region_name=REGION
    )

    payload_config = PLConfig(
        max_tokens=4096,
        stream=True
    )
```

```
    model = SageMakerAIModel(
        endpoint_config=endpoint_config,
        payload_config=payload_config
    )

    return model
```

9. Define the `main()` function to initialize the agent and execute the prompt:

```
def main():
    agent = Agent(
        model=get_model(),
        tools=[
            search_vector_database,
            current_time,
            shell
        ],
        system_prompt=SYSTEM_PROMPT
    )

    result = agent(prompt)
    print(result)
```

Here, the `main()` function initializes the agent with a SageMaker AI real-time endpoint model, the configured tools (`search_vector_database`, `current_time`, and `shell`), and the system prompt. It then executes the prompt through the agent's runtime and prints the final response generated by the model.

10. Add the following code to ensure the `main()` function is executed when the script is run directly:

```
if __name__ == "__main__":
    main()
```

**Note**

Make sure to save the *main.py* file before proceeding to the next step. At the same time, verify that the SageMaker AI real-time inference endpoint (`sagemaker-endpoint-00`) is up and running to ensure the agent can successfully connect and execute requests.

11. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new terminal by choosing **Terminal** (under **Other**). After opening the terminal, navigate to the directory that contains the *main.py* file.
12. With everything ready, run the `main.py` script to execute the agent, which will retrieve information from the knowledge base, process it, and create the directory structure and files.

```
python main.py
```

You should see output similar to the following:

```
<think>
Okay, let's tackle this user's request step by step. First, the user wants
the current month. So I need to use the current_time function to get that.
Then, they mentioned the Xironal Flux Displacement and Subatomic Event
Polarization in the knowledge base during the same quarter of the last few
years. I should use the search_vector_database function to look up those
events.

...

1. Get current month and year using current_time.
2. Determine which quarter the current month is in.
3. Search the knowledge base for the events in that quarter of the last few
years.
4. Create directories for each month in that quarter (three months).
5. For each directory, create an info.txt file with relevant info.

...
```

Here, the agent breaks down the request into smaller steps it can follow and figures out which tools it needs to use to get the information and produce the output. It then starts carrying out those steps in order. Before running any shell commands, it asks for your permission to make sure it is safe to proceed. When prompted with "`Do you want to proceed with execution? [y/*]`", enter `y` to allow the execution to continue.

**Note**

After running the RAG-powered AI agent, you should see several new directories and files created in the same directory where you created *main.py*. Note that the agent's

> execution may vary between runs, and it may also make mistakes depending on how it interprets and completes the requested steps.

At this point, you should have a good understanding of how to build an agent that interacts with a model deployed in a SageMaker AI real-time inference endpoint and retrieves relevant data from a knowledge base using a RAG approach. Keep in mind that the exact steps the agent takes may vary depending on how it interprets the task and the available information.

## Cleaning up

In this section, you will clean up the AWS resources created and used during the hands-on examples of this chapter. More specifically, you will delete the SageMaker AI endpoint (`ml.g5.24xlarge`) as well as stop the SageMaker Studio space (`mle-on-aws-space`), which is using an `ml.t3.xlarge` instance. These cleanup steps will help you avoid unnecessary charges and better manage your AWS costs.

> **Note**
>
> While not all resources will be deleted, performing these cleanup steps ensures that the most cost-impacting services, such as SageMaker AI endpoints and SageMaker Studio spaces, are properly stopped or removed.

Let's proceed with cleaning up the resources:

1. Switch to the browser tab where the AWS Management Console is already open. Alternatively, you can open a new browser tab and navigate to the AWS Management Console. Navigate to the Amazon SageMaker AI console by typing `sagemaker ai` in the search bar and choosing **Amazon SageMaker AI** from the list of results.
2. In the navigation pane (left sidebar), click **Endpoints** under **Deployments & inference**. Select `sagemaker-endpoint-00`. Click **Actions** to open the dropdown menu, then choose **Delete** from the options. When prompted that this action will permanently delete your endpoint and cannot be undone, click **Delete**.
3. Finally, switch to the browser tab where SageMaker Studio is open. Navigate to the page showing the space details for `mle-on-aws-space`, then click **Stop Space**. When prompted that any active resources will also be stopped, click **Stop space**.

> **Note**
>
> To navigate to the space details page, click the JupyterLab icon under **Applications** in the left sidebar. You'll then see a list of running and stopped spaces, where you can click

on `mle-on-aws-space` to open the space details page. From here, you can click **Stop space** to stop the running space, which is using a `ml.t3.xlarge` instance.

You have now completed the cleanup process to ensure that all resources created for the hands-on examples are safely stopped or deleted. With the cleanup steps completed, it is a good time to reflect on what you built. In this section, you built an AI agent that interacts with a model deployed in a SageMaker AI real-time inference endpoint and retrieves relevant information from a Bedrock knowledge base. You used Strands Agents to implement and run a RAG-powered workflow that generates a structured set of directories and files, along with other tools for task execution and automation. From here, you can extend the same core pattern of combining model reasoning, external retrieval, and tool use to build more advanced multi-step agentic applications.

**Note**

Feel free to check all other AWS resources you set up and provisioned in this chapter to ensure nothing is inadvertently left running, such as the knowledge base or any related data sources.

## Summary

In this chapter, you learned how to build agents that interact with a SageMaker AI real-time inference endpoint. You also learned how to use Amazon Bedrock Knowledge Bases to build RAG-powered agents that retrieve and ground responses in external data. By using Strands Agents and Bedrock AgentCore, you were able to build agents that integrate model inference with external tools and knowledge retrieval capabilities to deliver more accurate and context-aware applications. You explored how these building blocks work together to support more flexible and production-ready agent-based systems.

In the next chapter, you'll dive deeper into SageMaker AI and explore how to use it for training and deploying models. As you work through the examples, you'll see how SageMaker AI simplifies the complexity of building, training, and deploying models by providing fully managed infrastructure so that you can focus on developing and improving your models rather than managing the underlying infrastructure.

# Further reading

To deepen your understanding of the concepts discussed in this chapter, explore the following resources:

- *Amazon SageMaker AI | Developer Guide | What is Amazon SageMaker AI? (*`https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html`*)*
- *Amazon SageMaker AI | Developer Guide | Deploy models for inference* (`https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model.html`)
- *Strands Agents | Get started* (`https://strandsagents.com/docs/user-guide/quickstart/overview/`)
- *Strands Agents | Tools overview (*`https://strandsagents.com/docs/user-guide/concepts/tools/`*)*
- *Knowledge bases for Amazon Bedrock — How it works* (`https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-it-works.html`)
- *Retrieval-Augmented Generation for Large Language Models: A Survey* (`https://arxiv.org/abs/2312.10997`)
- *Amazon Bedrock AgentCore | Developer Guide | Get started with Amazon Bedrock AgentCore* (`https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agentcore-get-started-cli.html`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 3

# Machine Learning Engineering with Amazon SageMaker AI

The practice of **machine learning** (**ML**) engineering has evolved significantly over the past decade. Traditional ML workflows placed strong emphasis on feature engineering, structured datasets, and algorithm-level optimization. With the emergence of foundation models and large language models, the workflow has shifted toward prompt engineering, model adaptation, and managing extremely large pretrained models. While traditional models were often trained from scratch, modern approaches increasingly leverage pretrained models and customize them through fine-tuning and alignment techniques.

ML engineering continues to evolve, requiring ML engineers to adapt to the changing needs of modern AI applications and systems. Rather than focusing solely on building models from scratch, engineers now concentrate on updating, adapting, and optimizing large models while integrating emerging technologies into existing stacks. As you build modern AI-powered solutions on AWS, you should be familiar with how Amazon SageMaker AI supports end-to-end ML engineering workflows, since this fully managed service allows you to focus on model development, alignment, and deployment instead of managing underlying infrastructure.

To help you explore how you can use Amazon SageMaker AI to implement end-to-end ML engineering workflows involving both traditional ML models and foundation models, this chapter covers the following sections:

- Setting up and preparing your JupyterLab notebook
- Preparing a synthetic dataset for binary classification
- Training an XGBoost binary classifier
- Deploying an XGBoost model to a real-time inference endpoint
- Setting up BERT fine-tuning with SageMaker JumpStart

- Using a smaller dataset for fine-tuning
- Running the BERT model fine-tuning job
- Deploying the fine-tuned model to a real-time inference endpoint

By working through the examples in this chapter, you will see how SageMaker AI helps simplify the complexity of building and deploying various types of ML models by providing managed infrastructure for training, inference, and model lifecycle operations. This will prepare you for the subsequent chapters of this book, where you will explore more advanced ML engineering concepts and practical techniques.

# Technical requirements

Before proceeding with the hands-on examples in this chapter, confirm that the following prerequisites and setup requirements have been addressed:

- When launching training jobs or deploying models, ensure that the applied account-level quota value for the selected ML instance types (for example, ml.p3.2xlarge for training job usage) is set to at least `1`. If the quota value is `0`, SageMaker AI will not be able to provision the required compute resources, and the job or endpoint creation will fail. You can review and request quota increases through the **Service Quotas** console (within the AWS Management Console).
- To enable live logging while using the SageMaker Python SDK for model deployments, ensure that the execution role (for example, `AmazonSageMaker-ExecutionRole-...`) has the appropriate permissions. You can attach the `CloudWatchFullAccessV2` policy (or a more granular alternative) to the role.

> **Note**
>
> Since the role can have a maximum of 10 policies, you may need to remove `AmazonSageMakerCanvasSMDataScienceAssistantAccess` if it is not needed.

The Jupyter notebooks, source code, and other files used for each chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

> **Note**
>
> It is recommended to use an AWS **Identity and Access Management** (**IAM**) user with limited permissions instead of the root account when running the examples in this book. We will discuss this, along with other security best practices, in detail toward the

> end of this book. If you are just starting to use AWS, you may proceed with using the root account in the meantime.

## Setting up and preparing your JupyterLab notebook

In this section, you will prepare and configure your JupyterLab notebook, which you will use throughout the first half of this chapter to train and deploy an XGBoost model. We will go through the steps of launching your SageMaker Studio space, creating and configuring your notebook, installing the required version of the SageMaker Python SDK, and exploring its internal package structure to gain a better understanding of how the SDK is organized.

Let's define a few key terms, concepts, services, and resources relevant to what you'll work with in this section:

- **SageMaker AI domain**: A logical group of resources used to organize user profiles, applications, and their associated resources.
- **SageMaker Studio**: A browser-based development environment in Amazon SageMaker AI that enables you to prepare data, build models, and manage model training and deployment workflows.

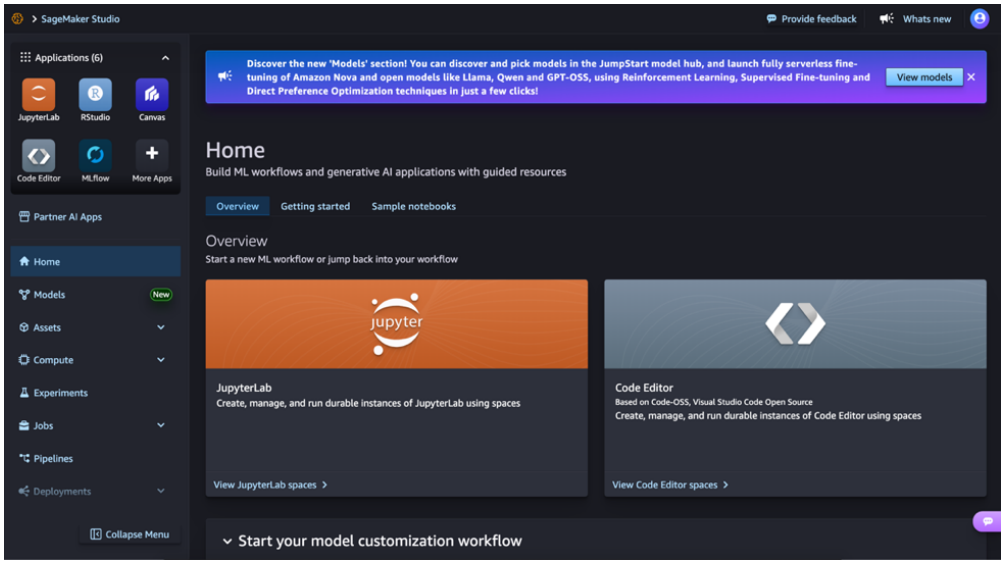



*Figure 3.1 — SageMaker Studio*

Here, you can create and run spaces, which are managed working environments that support end-to-end ML model development, collaboration, and deployment

workflows. The sidebar on the left side of the interface allows you to access various tools, project resources, and data files, as well as ML resources and features available in the SageMaker AI platform.

- **SageMaker Studio space**: A managed environment used to isolate, configure, and manage the storage and resource needs of Amazon SageMaker Studio applications. It is composed of a storage volume, an application type, and the image on which the application runtime is based. You can think of a space as a managed logical container or instance where the application runs and accesses the storage, runtime configuration, and other resources associated with the space.
- **SageMaker Python SDK**: An open source Python library that allows you to interact programmatically with Amazon SageMaker AI. You can use the SageMaker Python SDK to manage model training jobs, deploy models to endpoints, manage experiments, and automate ML workflows directly from your notebook.

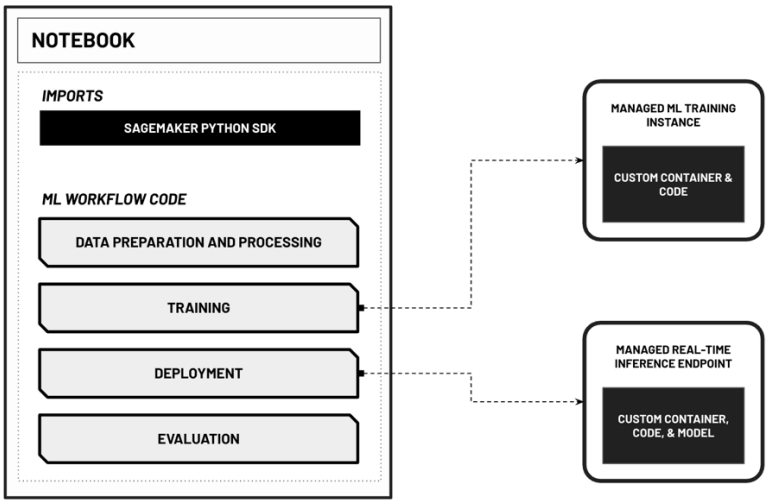



*Figure 3.2 — Using the SageMaker Python SDK for training and deploying ML models*

When using the SageMaker Python SDK, model training and deployment processes typically run on dedicated compute resources (ML instances) provisioned specifically for these workloads. ML instances used for training jobs are automatically terminated after the job completes. For real-time inference, deployed models run on managed endpoint instances that remain available (depending on your configuration) to serve prediction requests until the endpoint is deleted. Keep in mind that these ML instances are separate from the compute environment where your notebook code is executed.

- **JupyterLab**: A web-based interactive development environment for working with notebooks, code, and data. It allows you to open and manage multiple notebooks, terminals, and files in separate tabs.
- **JupyterLab notebook**: An interactive document file (typically with a `.ipynb` extension) that contains executable code cells, Markdown-formatted text, and outputs such as tables and visualizations. Inside these notebooks, you can write and execute Python code, run model training jobs, and deploy ML models.
- **Kernel**: A standalone language-specific process that runs notebook code, manages the runtime environment such as variables and memory, and communicates with the JupyterLab interface to process code execution requests.
- **XGBoost (eXtreme Gradient Boosting)**: A high-performance, scalable implementation of gradient-boosted decision trees commonly used for classification and regression tasks. The decision to use XGBoost or another ML algorithm depends on factors such as dataset characteristics, problem type, model complexity, and performance requirements. Different algorithms are designed to address different use cases, so selecting the appropriate algorithm is an important part when building ML models.

> **Note**
>
> This section assumes that you are running the hands-on examples inside a space in Amazon SageMaker Studio. You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1*.

Now that you have a good understanding of these key concepts, let's proceed with setting up your JupyterLab notebook:

1. In the browser tab where the AWS Management Console is open, type `sagemaker ai` in the search bar, and select **Amazon SageMaker AI** from the list of results.
2. In the navigation pane (left sidebar), click **Domains** under **Environment configuration**. This should open a page that shows all created domains (if any).
3. From the list of domains, locate the domain you created in this chapter, and click **Open Studio** to open the domain details page. When prompted to select a user profile, select your user profile and click **Open Studio**.
4. In the left sidebar, choose **Home**, then locate the **mle-on-aws-space** space under **Recent spaces**. Click **Run** then **Open** (or **Run and open**) to open the space.

5. In the left sidebar of JupyterLab, right-click and select **New Folder**. Name the folder `part1`, then double-click it to navigate into the directory.
6. Click the **+** button to open the Launcher, then select **Python 3** (under **Notebook**). This will create a new `Untitled.ipynb` notebook inside the `part1` directory.
7. Let's rename the notebook to `mle_sagemaker.ipynb`. Right-click on the notebook tab and select **Rename notebook** from the context menu. Then, enter `mle_sagemaker.ipynb` as the new name and click **Rename**.

   **Note**

   We will be entering and executing code for each step in a separate cell within this notebook. After entering the code for each step, you can execute the cell by pressing *Shift* + *Enter*, which will not only run the current cell but also automatically create a new one below it.

8. Uninstall the current versions of `sagemaker` and `sagemaker-serve`, and then install the specified versions (`3.2.0` for `sagemaker` and `1.2.0` for `sagemaker-serve`):

   ```
   %pip uninstall -y sagemaker sagemaker-serve
   %pip install "sagemaker==3.2.0"
   %pip install "sagemaker-serve==1.2.0"
   ```

   **Note**

   In this chapter, you will use version 3.2.0 of the SageMaker Python SDK. Installing the specified version of the SageMaker Python SDK is critical because substantial changes were introduced between major releases, particularly between v2 and v3. Ensuring that you are using the correct SDK version helps maintain compatibility with the code samples used in this chapter.

9. Install the `conda-pack` package as well:

   ```
   %pip install conda-pack
   ```

10. Restart the kernel by selecting **Kernel** | **Restart Kernel...** from the menu.

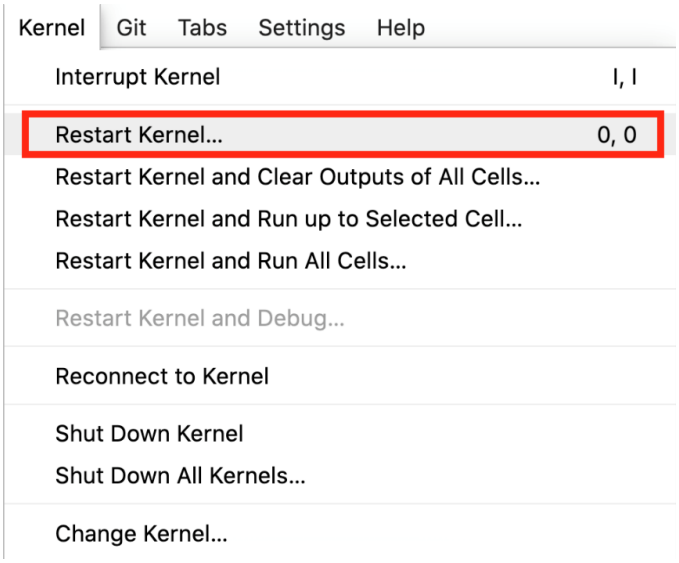



*Figure 3.3 — Restarting the kernel*

This will ensure that the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

11. After restarting the kernel, run the following in a new cell to suppress specific namespace conflict warnings using Python's `warnings` module before continuing with the rest of the notebook:

```
import warnings

m="Field .* has conflict with protected namespace"
warnings.filterwarnings(
    "ignore",
    message=m
)
```

This suppresses specific warnings that may appear during imports, so you can focus on the relevant output.

12. Import the `boto3` library and retrieve the current AWS region associated with your session:

```
import boto3

region = boto3.Session().region_name
print(region)
```

You should get `us-east-1` as the region name value.

> **Note**
>
> If you are running this notebook in a different region, or if `boto3` was configured with a specific region in your environment, you may get a different region value.

13. Run the following block of code to download the `import_helper.py` script from GitHub:

```
import urllib.request
from pathlib import Path

URL_PARTS = [
    "https://raw.githubusercontent.com/",
    "PacktPublishing/",
    "Machine-Learning-Engineering-on-",
    "AWS-Second-Edition/",
    "refs/heads/main/",
    "chapter01/import_helper.py",
]

url = "".join(URL_PARTS)
output_file = Path("import_helper.py")
urllib.request.urlretrieve(url, output_file)
```

14. Use the `ImportHelper` class to explore the structure of SageMaker, including its packages and modules:

```
import sagemaker
from import_helper import ImportHelper
```

```
helper = ImportHelper(sagemaker)
helper.explore()
```

After running `helper.explore()`, you should see the following folder structure for SageMaker:

```
+-- sagemaker/
   +-- core/
      +-- apiutils/
      +-- clarify/
      ...
   +-- train/
      +-- aws_batch/
      +-- common_utils/
      ...
   +-- mlops/
      +-- local/
      +-- workflow/
   +-- serve/
      +-- async_inference/
      +-- batch_inference/
      ...
```

Under `sagemaker`, you'll find the full implementation of the Amazon SageMaker Python SDK, organized by functional areas of the ML lifecycle. The `core` directory contains foundational components such as configuration, serialization and deserialization, debugging tools, experiments, lineage tracking, JumpStart integration, model monitoring, training utilities, workflow orchestration, and various helpers and utilities.

Beyond `core`, the structure is grouped by major capabilities: `train` provides training logic (including local and remote execution, container drivers, evaluation, and recipes), `serve` handles inference features (async and batch inference, model servers such as TorchServe and Triton, serverless options, validation, and model formats), and `mlops`, `lineage`, and `ai_registry` support MLOps workflows, metadata tracking, and model registry schemas. Together, these modules reflect the SDK's end-to-end support for building, training, deploying, and managing various types of ML models.

> **Note**
>
> If the folder structure appears significantly different, try reinstalling SageMaker 3.2.0 and SageMaker Serve 1.2.0, then restart the kernel to ensure that the correct versions are used.

15. Use the `guess_import()` method from `ImportHelper` to locate the class and get the likely `import` statement for `SchemaBuilder`:

```
helper.guess_import("SchemaBuilder")
```

You should see output similar to the following:

```
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/serve/builder/
schema_builder.py:91
Likely import: from sagemaker.serve.builder.schema_builder import
SchemaBuilder
```

16. Iterate over a list of class names (`InferenceSpec`, `Session`, `ModelBuilder`, and `ModelTrainer`) and use `helper.guess_import()` to guess their likely `import` statements:

```
im = ["InferenceSpec",
      "Session",
      "ModelBuilder",
      "ModelTrainer"]

for i in im:
    print("-"*20)
    print(f"Running helper.guess_import() for {i}")
    helper.guess_import(i)
```

After running the loop, you should see output similar to the following, showing the file locations and potential `import` statements for each class:

```
--------------------
Running helper.guess_import() for InferenceSpec
...
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/serve/spec/
```

```
inference_spec.py:7
Likely import: from sagemaker.serve.spec.inference_spec import
InferenceSpec
--------------------
Running helper.guess_import() for Session
...
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/core/helper/
session_helper.py:119
Likely import: from sagemaker.core.helper.session_helper import Session
...
--------------------
Running helper.guess_import() for ModelBuilder
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/serve/
model_builder.py:142
Likely import: from sagemaker.serve.model_builder import ModelBuilder
...
--------------------
Running helper.guess_import() for ModelTrainer
Found in: /opt/conda/lib/python3.12/site-packages/sagemaker/train/
model_trainer.py:130
Likely import: from sagemaker.train.model_trainer import ModelTrainer
```

While the results may not always be fully correct or accurate, `ImportHelper` provides a useful way to identify where a class or function is defined and suggests the likely `import` statement you would use.

> **Note**
>
> For the most up-to-date usage examples, correct import paths, and detailed API reference, see the SageMaker Python SDK documentation at `https://sagemaker.readthedocs.io/en/stable/`.

17. Import the necessary classes from SageMaker, including `SchemaBuilder`, `InferenceSpec`, `Session`, `ModelBuilder`, and `ModelTrainer`:

```
from sagemaker.serve.builder.schema_builder import (
    SchemaBuilder
)
from sagemaker.serve.spec.inference_spec import (
    InferenceSpec
```

```
)
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)
from sagemaker.serve import ModelBuilder
from sagemaker.train import ModelTrainer
```

18. Define the `copy_model_server_file()` function, which copies the `model.py` file from the `triton` folder in the SageMaker `serve` directory to the root of the `serve` directory:

```
import os
import shutil

def copy_model_server_file():
    sagemaker_path = os.path.dirname(
        sagemaker.__file__)

    source_path = os.path.join(
        sagemaker_path, 'serve',
        'model_server', 'triton', 'model.py')

    destination_path = os.path.join(
        sagemaker_path, 'serve', 'model.py')

    if os.path.exists(source_path):
        shutil.copy(source_path, destination_path)
        print(f"Successfully copied "
              f"model.py from {source_path} "
              f"to {destination_path}")
    else:
        print(f"Source file not found "
              f"at {source_path}")
```

19. Use the `copy_model_server_file()` function defined in the previous step to copy the `model.py` file to the `serve` directory:

```
copy_model_server_file()
```

**Note**

This should address a possible issue that may have existed in earlier versions of the SageMaker Python SDK when building and deploying ML models.

At this point, you have successfully prepared your working environment, including installing the specified version (3.2.0) of the SageMaker Python SDK, restarting the kernel so that the newly installed packages are properly loaded, and exploring the SDK structure. With this, you can now proceed with preparing the synthetic dataset that will be used for training and deploying the XGBoost model in the first half of this chapter.

## Preparing a synthetic dataset for binary classification

In this section, you will generate a synthetic dataset for binary classification, split it into training, validation, and test sets, and upload the resulting CSV files to an Amazon S3 bucket for use in subsequent training, deployment and evaluation steps. This dataset will have the following columns:

- `target`: The dependent variable (what the model will try to predict); in this case, it can be either `0` (*LABEL 0*) or `1` (*LABEL 1*)
- `a` and `b`; The independent variables or features (inputs used to predict the `target` value)

Using a portion of this dataset, a model will be trained so that future values of `a` and `b` can be used to automatically predict the corresponding `target` value.

**Note**

For example, `a` could be the applicant's monthly income, `b` could be their years of employment, and `target` could indicate whether the applicant is likely to default on a loan. In a real dataset, there could be many more variables that the model can use to improve its predictions. Before being used for training, these values would go through several data transformation and preprocessing stages, such as scaling, encoding, or handling missing values, to prepare them for the ML model.

To help you better understand the relationship between the features and the `target` variable, you will generate a scatterplot in which each point is colored according to its class label, similar to what is shown in *Figure 3.4*.

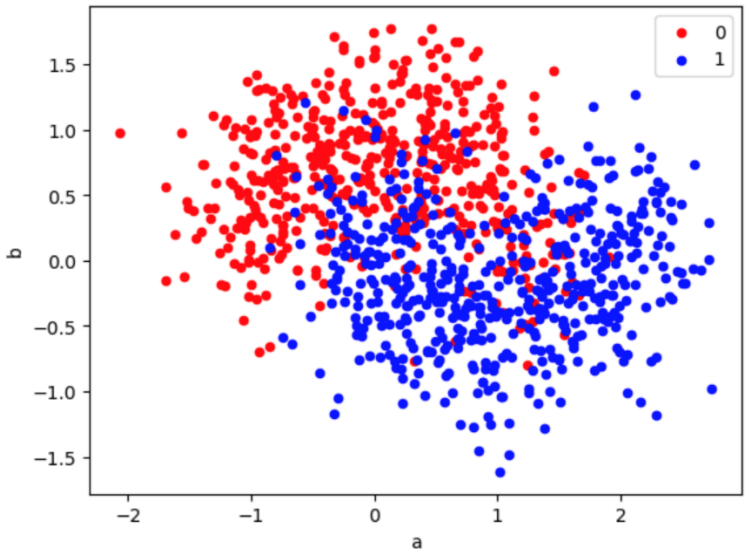



*Figure 3.4 — Scatterplot of the synthetic binary classification dataset*

In this visualization, red indicates `target = 0` (*LABEL 0*), while blue indicates `target = 1` (*LABEL 1*). This helps you visually assess how well the two classes are separated based on features `a` and `b`. At the same time, the plot provides a quick sanity check before model training.

Before we prepare this dataset, let's define a few key terms, concepts, and services relevant to what you'll work with in this section:

- **Synthetic dataset**: An artificially generated dataset that mimics real data patterns created for testing or experimentation.
- **Binary classification**: An ML task where a model learns to classify data into one of two possible classes. For example, the model may predict whether an email is spam or not spam, or whether a transaction is fraudulent or legitimate. This type of problem is common in applications such as fraud detection, medical diagnosis, and customer behavior prediction.
- **Train-test split**: A data preparation technique where a dataset is divided into training data used for model training and testing data used for model evaluation. This helps you measure how well the model performs on data it has not seen during training. By

evaluating the model on test data, you can better estimate how the model will perform on new, unseen data in real-world scenarios.

- **Amazon S3 bucket**: A storage resource in the AWS cloud that allows you to manage files (objects) over the internet. In this chapter, you will use a general-purpose S3 bucket to store and manage files such as datasets, model artifacts, and logs.

> **Note**
>
> This section assumes you are running the code inside the `mle_sagemaker.ipynb` notebook.

Now that you have a good understanding of these key concepts, let's go through the steps to prepare the synthetic dataset:

1. Initialize an empty dictionary called `var` to store datasets and other items for later use:

```
var = {}
```

2. Import the necessary libraries (`pandas` and `make_moons` from `sklearn.datasets`), then define the `generate_dataset()` function for preparing a synthetic dataset:

```
import pandas as pd
from sklearn.datasets import make_moons

def generate_dataset(
    samples=1000,
    noise=0.4,
    random_state=200
):
    params = {
        'n_samples': samples,
        'noise': noise,
        'random_state': random_state
    }
    X, y = make_moons(**params)

    return pd.DataFrame(
        dict(target=y, a=X[:,0], b=X[:,1])
    )
```

Here, `generate_dataset()` creates a synthetic two-dimensional dataset with two interleaving half circles, returning a `pandas` DataFrame with columns `a` and `b` for features and `target` for the class label. The `samples`, `noise`, and `random_state` parameters allow you to control the dataset size, amount of noise, and reproducibility.

**Note**

If the same `random_state` value is used when generating a dataset, the output will be deterministic—that is, you will get the same dataset every time. This is useful for reproducibility, which allows experiments and visualizations to be consistent across runs. Keep in mind that changing or omitting `random_state` will produce a different dataset each time.

3. Generate a synthetic dataset using the `generate_dataset()` function and store it in the `var` dictionary under the key `'dataset'`. Then, display the dataset:

```
var['dataset'] = generate_dataset()
var['dataset']
```

You should get the following DataFrame of values:

| | target | a | b |
|---|---|---|---|
| **0** | 1 | -0.147454 | -0.565003 |
| **1** | 0 | 0.223171 | 1.424979 |
| **2** | 1 | 1.513686 | -1.075581 |
| **3** | 1 | 1.548403 | -0.537688 |
| **4** | 1 | 0.179867 | 0.169598 |
| **...** | ... | ... | ... |
| **995** | 1 | 1.337359 | -0.237273 |
| **996** | 1 | 1.124726 | -0.555634 |
| **997** | 1 | 1.560981 | -0.196432 |
| **998** | 1 | 0.225870 | -1.090455 |
| **999** | 1 | 1.324885 | -1.090451 |

1000 rows × 3 columns

*Figure 3.5 — Generated synthetic dataset*

You should get a DataFrame with 1,000 rows and 3 columns: `target` (what the model will try to predict), and `a` and `b` (inputs used to predict the `target` value). Each row

represents a single observation, and the values in `a` and `b` are used by the model to learn patterns that allow it to predict the `target` class (*LABEL 0* or *LABEL 1*).

4. Define the `plot_dataset()` function for visualizing the synthetic dataset, which colors points based on their `target` value:

```
from matplotlib import pyplot

def plot_dataset(
    dataset,
    colors = {0:'red', 1:'blue'}
):
    fig, ax = pyplot.subplots()
    grouped = dataset.groupby('target')

    for key, group in grouped:
        params = {
            'ax': ax,
            'kind': 'scatter',
            'x': 'a',
            'y': 'b',
            'label': key,
            'color': colors[key]
        }

        group.plot(**params)

    pyplot.show()
```

When this function is called, it creates a scatter plot of the dataset, coloring points red for the `0` class (*LABEL 0*) and blue for the `1` class (*LABEL 1*).

5. Call the `plot_dataset()` function to visualize the synthetic dataset stored in `var['dataset']`:

```
plot_dataset(var['dataset'])
```

Calling the function produces the following visualization:

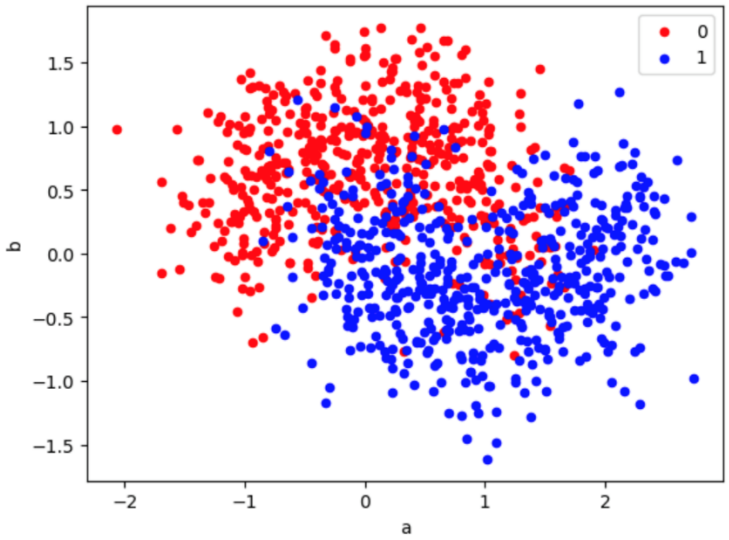



*Figure 3.6 — Scatterplot showing the synthetic two-class dataset after running plot_dataset()*

Scatterplots provide a visual way to compare two variables at once, where each point represents an observation. If you are new to scatterplots, this visual format helps reveal patterns, trends, or correlations. Colored scatterplots further distinguish groups, making it easier to see separations or overlaps. For instance, red indicates `target = 0` (*LABEL 0*) and blue indicates `target = 1` (*LABEL 1*) in this dataset.

**Note**

Continuing the example from earlier, if `a` is the applicant's monthly income and `b` is their years of employment, you can use these features to explore patterns in the dataset. A scatterplot of these two variables allows you to visually see patterns. For example, it may show that applicants with lower income and fewer years of employment are more likely to default on a loan. Of course, you could try various other visualizations, such as histograms, box plots, or pair plots, to explore the relationships between these variables and gain deeper insights into the data.

6. Define the `split_data()` function for splitting the dataset into training, validation, and test sets:

```
from sklearn.model_selection import train_test_split

def split_data(
    dataset,
    test_size=0.2,
    random_state=0
):

    AB, C = train_test_split(
        dataset,
        test_size=test_size,
        random_state=random_state
    )

    A, B = train_test_split(
        AB,
        test_size=0.25,
        random_state=random_state
    )

    return A, B, C
```

Here, since `test_size=0.2`, 20% of the dataset is allocated to the test set, while the remaining 80% is split further into training and validation sets. The validation set consists of 25% of the remaining 80%, which results in 60% of the data being used for training.

**Note**

Since the generated dataset has 1,000 records, this means 600 records are used for training, 200 for validation, and 200 for testing.

7. Use the `split_data()` function defined in the previous step to split the dataset stored in `var['dataset']` into training, validation, and test sets. Then, store each set in the `var` dictionary for later use:

```
training, validation, test = split_data(
    var['dataset']
)

var['training'] = training
var['validation'] = validation
var['test'] = test
```

8. Let's inspect the contents of the training set:

```
var['training']
```

This should give you the following DataFrame of values:

| | target | a | b |
|---|---|---|---|
| **787** | 0 | -0.094509 | 1.117692 |
| **683** | 0 | 0.250788 | -0.065240 |
| **616** | 1 | 1.390854 | -0.172775 |
| **691** | 0 | -0.247788 | 0.969354 |
| **972** | 0 | -1.085892 | 0.314411 |
| **...** | ... | ... | ... |
| **431** | 1 | 1.616753 | -0.671053 |
| **243** | 1 | 1.361652 | 0.485315 |
| **444** | 0 | 0.392299 | 0.786540 |
| **866** | 0 | 1.702440 | 0.650115 |
| **943** | 1 | 0.942595 | -1.246762 |

600 rows × 3 columns

*Figure 3.7 — The training set*

The model will be trained using this data, specifically, using the independent variables (`a` and `b`) to learn patterns that predict the `target` value. This training process helps the model develop the ability to make accurate predictions on new, unseen data.

9. Create a new directory called `data` using the `mkdir` command:

```
!mkdir -p data
```

You'll store the CSV files containing the training, validation, and test sets in this directory shortly.

10. Save the training set to a CSV file (`train.csv`):

```
var['training'].to_csv("./data/train.csv",
                       index=False,
                       header=False)
```

11. Save the validation set to a CSV file (`validation.csv`):

```
var['validation'].to_csv("./data/validation.csv",
                         index=False,
                         header=False)
```

12. Save the test set to a CSV file (`test.csv`):

```
var['test'].to_csv("./data/test.csv",
                   index=False,
                   header=False)
```

13. Store the test set without the `target` column in a new variable, `test_data`:

```
test_data = var['test'].drop("target", axis=1)
test_data
```

This should give you the following DataFrame of values:

| | a | b |
|---|---|---|
| **993** | 1.912737 | -0.427197 |
| **859** | 2.117991 | 0.685974 |
| **298** | 0.034832 | -0.019911 |
| **553** | 1.380312 | 0.442379 |
| **672** | 1.128483 | -0.012440 |
| **...** | ... | ... |
| **679** | 0.709688 | -0.531570 |
| **722** | 0.533799 | 0.699297 |
| **215** | 2.289943 | 0.443640 |
| **653** | 1.798953 | -0.075756 |
| **150** | 0.048546 | 0.935311 |

200 rows × 2 columns

*Figure 3.8 — Test set without the target column*

Here, you have a version of the test set that excludes the `target` variable. This is typically done to separate the features (input variables) from the target (predicted outcome or class) when preparing data for inference or making predictions.

14. Save the DataFrame of values inside `test_data` (without the `target` column) to a new CSV file (`test_no_target.csv`):

```
test_data.to_csv("./data/test_no_target.csv",
                 index=False,
                 header=False)
```

15. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length:

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase, k=length)
    )
```

16. Let's test the function we defined in the previous step. Generate a random string using the `generate_string()` function and store it in the `prefix` variable. Then, print the value of `prefix`:

```
prefix = generate_string()
print(prefix)
```

This should give you a random string of six lowercase letters, such as `jxdzyu`, which will change each time you run the function.

17. Initialize a `Session` instance and retrieve the execution role, region, and default S3 bucket associated with your SageMaker session:

```
session = Session()
role = get_execution_role()
bucket = session.default_bucket()
```

18. Define constants for the filenames (`train.csv`, `validation.csv`, and `test.csv`) and the directory (`data`) where the files are stored:

```
TRAIN_DATA = "train.csv"
VALIDATION_DATA = "validation.csv"
TEST_DATA = "test.csv"
DATA_DIRECTORY = "data"
```

19. Upload the CSV files inside the `data` directory to an S3 bucket using the `upload_data()` method, with the specified bucket and key prefix:

```
train_input = session.upload_data(
    DATA_DIRECTORY,
    bucket=bucket,
    key_prefix="{}/{}".format(prefix, DATA_DIRECTORY)
)
```

20. Use the AWS CLI to list the contents of the S3 path where the files were uploaded:

```
!aws s3 ls {train_input} --recursive
```

This should yield the following output:

```
2026-01-01 00:00:00   8192 .../data/test.csv
2026-01-01 00:00:00   7792 .../data/test_no_target.csv
2026-01-01 00:00:00  24633 .../data/train.csv
2026-01-01 00:00:00   8211 .../data/validation.csv
```

21. Set the `s3_train_input_path`, `s3_val_input_path`, `s3_test_path`, and `s3_output_path` variables to point to the corresponding dataset splits and the output location in S3:

```
s3_train_input_path = "s3://{}/{}/data/{}".format(
    bucket,
    prefix,
    TRAIN_DATA
)

s3_val_input_path = "s3://{}/{}/data/{}".format(
    bucket,
    prefix,
    VALIDATION_DATA
)

s3_output_path = "s3://{}/{}/output".format(
    bucket,
    prefix
)

s3_test_path = "s3://{}/{}/data/{}".format(
    bucket,
    prefix,
    TEST_DATA
)
```

**Note**

In the next section, we'll use these variables to configure the SageMaker training job.

At this stage, you have prepared a synthetic dataset and divided it into training, validation, and test sets. You also saved the dataset splits as CSV files and uploaded them to an Amazon S3

general-purpose bucket for subsequent use in model training, evaluation, and ML tasks. With the dataset stored in Amazon S3, we can now proceed with training a binary classifier in the next section.

# Training an XGBoost binary classifier

In this section, you will train an XGBoost model using a training job in Amazon SageMaker AI. As you dive deep into the model training process, you will retrieve the training container image, configure hyperparameters for binary classification, define training and validation data channels, and initialize a model trainer instance using various configuration parameters. After that, you will start the training job and monitor training progress through logs.

Before we proceed with training the XGBoost model, let's define a few key terms and concepts relevant to what you'll work with in this section:

- **Training input mode**: The method used to make training data available to a training job in Amazon SageMaker, such as downloading data from an Amazon S3 bucket to local storage using **file mode** or streaming data directly to the training instance using **pipe mode**. File mode is generally suitable for smaller datasets or workloads that require full data availability on local storage, while pipe mode is often preferred for very large datasets because it enables streaming data directly from the S3 bucket to the training instance and can reduce training startup time. In this chapter, you will use file mode for the hands-on examples.
- **S3 data distribution type**: The configuration that determines how training data stored in Amazon S3 is distributed across multiple training instances, such as fully replicated or sharded data distribution.
- **Train error**: The error metric computed on the training dataset that measures how well the model fits the training data during the learning process. Lower training error generally indicates that the model is learning patterns from the training data effectively. However, a very low training error alone does not guarantee good generalization performance.
- **Validation error**: The error metric computed on a separate validation dataset used to evaluate model performance and help detect overfitting during training. Validation error provides an estimate of how well the model is expected to perform on unseen data and is commonly used to monitor model generalization during training. You can think of training error and validation error like preparing for a mock exam before the actual test. The training error is similar to how well you perform while studying and practicing using your review materials, while the validation error is like your score on a mock exam that contains questions you have not previously encountered. A good

model should perform well on both the training data and validation data, just as you would want to do well in both practice exercises and mock examinations.

- **Container image**: A blueprint that contains the application code, libraries, and dependencies required to run a containerized application or workload. Container images are commonly used in Amazon SageMaker AI to provide consistent runtime environments for training, evaluating, and deploying ML models. By packaging all required components and prerequisites together, container images help ensure that scripts and applications run reliably across different environments.
- **Container**: A running instance of a container image. It is an isolated, lightweight execution environment that uses the host operating system's kernel while maintaining its own filesystem, processes, and network interfaces. You can think of a container as the actual dish prepared from a recipe, where the container image is the recipe that describes how the application environment should be constructed.
- **SageMaker training job**: A managed ML workload in Amazon SageMaker where the model training code is executed inside provisioned ML instance(s) using specified datasets, hyperparameters, and container image. While you can run training scripts inside a notebook environment, using a dedicated managed ML instance helps process larger datasets and supports the training requirements of different algorithms more efficiently. Training jobs can be configured with a specific instance type according to workload and model complexity. For example, smaller workloads may be suitable for instances such as `ml.m5.large` or `ml.m5.xlarge`, while more computationally intensive training tasks may require larger instance types such as `ml.m5.12xlarge` or GPU-enabled instances such as `p3.2xlarge`, depending on training performance requirements.
- **Hyperparameters**: Configuration values that control the learning process of an ML algorithm, such as learning rate, maximum tree depth, or number of epochs when training a model. The specific hyperparameters you configure depend on the algorithm being used, as different algorithms expose different tunable hyperparameters. Selecting the appropriate hyperparameter values is important because they directly influence model performance and training behavior. Hyperparameters are like the number of sets, repetitions, and rest intervals in a workout routine. The exercise itself (algorithm) stays the same, but adjusting these settings changes the results you achieve. For example, doing 3 sets of 20 repetitions produces a different outcome than doing 5 sets of 8 repetitions, even if the exercise remains the same. In the same way, changing the number of training epochs or learning rate can significantly affect how much the machine learning model learns and how well it performs.

Keep in mind that hyperparameters are not the same as model parameters. Model parameters are internal values that are learned automatically by the model during training, such as weights and biases. In the workout analogy, hyperparameters are like choosing the number of sets and repetitions in your training routine, while model parameters are like the physical improvements your body develops as a result of following that routine.

**Note**

This section assumes you are running the code inside the `mle_sagemaker.ipynb` notebook.

Now that you have a better understanding of these key terms and concepts, you can proceed with the steps to train the XGBoost binary classifier:

1. Import the necessary classes from `sagemaker.core.shapes` for defining data channels, data sources, output configurations, and stopping conditions, as well as `image_uris` for working with container images:

```
from sagemaker.core.shapes import (
    Channel,
    DataSource,
    S3DataSource,
    OutputDataConfig,
    StoppingCondition,
)

from sagemaker.core import image_uris
```

2. Retrieve the container image URI for training XGBoost models in the specified AWS region, and print the image URI:

```
image = image_uris.retrieve(
    framework="xgboost",
    region=region,
    image_scope="training",
    version="1.7-1"
)

print(image)
```

This should give you the following Docker image URI:

```
https://683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-xgboost:
1.7-1
```

3. Define and customize the hyperparameters for training the XGBoost model, including the objective function, number of boosting rounds, and evaluation metric:

```
hyperparameters = {
    "objective": "binary:logistic",
    "num_round": "50",
    "eval_metric": "error",
}
```

Here, we specify `binary:logistic` as the objective function, so the model knows it is solving a binary classification problem.

> **Note**
>
> XGBoost supports a variety of objective functions depending on the type of problem you are solving. For example, `binary:logistic` is used for binary classification (output probability), `multi:softmax` or `multi:softprob` for multiclass classification, and `reg:squarederror` for regression tasks. Choosing the appropriate objective function ensures that the model optimizes correctly for your specific problem. Feel free to check the following resource for more information about this topic: `https://github.com/dmlc/xgboost/blob/master/doc/parameter.rst`.

4. Set the base job name for the SageMaker training job to the previously generated prefix:

```
base_job_name = "xgb-" + prefix
```

5. Initialize a `ModelTrainer` instance with the specified job name, hyperparameters, training image, training input mode, IAM role, output data configuration, and stopping condition:

```
model_trainer = ModelTrainer(
    base_job_name=base_job_name,
    hyperparameters=hyperparameters,
```

```
    training_image=image,
    training_input_mode="File",
    role=role,
    output_data_config=OutputDataConfig(
        s3_output_path=s3_output_path
    ),
    stopping_condition=StoppingCondition(
        max_runtime_in_seconds=600
    ),
)
```

6. Define the data sources for the training and validation sets, specifying the S3 URI and data distribution type for each:

```
train_data_source=DataSource(
    s3_data_source=S3DataSource(
        s3_data_type="S3Prefix",
        s3_uri=s3_train_input_path,
        s3_data_distribution_type="FullyReplicated",
    )
)

val_data_source=DataSource(
    s3_data_source=S3DataSource(
        s3_data_type="S3Prefix",
        s3_uri=s3_val_input_path,
        s3_data_distribution_type="FullyReplicated",
    )
)
```

7. Start the training job by calling the `train()` method on the `ModelTrainer` instance (`model_trainer`), specifying the input data configuration for both training and validation sets, along with their respective data sources and content types:

```
model_trainer.train(
    input_data_config=[
        Channel(
            channel_name="train",
            content_type="csv",
            compression_type="None",
```

```
            record_wrapper_type="None",
            data_source=train_data_source,
        ),
        Channel(
            channel_name="validation",
            content_type="csv",
            compression_type="None",
            record_wrapper_type="None",
            data_source=val_data_source,
        )
    ],
)
```

Running the block of code yields the following set of logs:

```
...
[47]#011train-error:0.04333#011validation-error:0.19000
...
[48]#011train-error:0.04500#011validation-error:0.19000
...
[49]#011train-error:0.04000#011validation-error:0.18500

Final Resource Status: Completed
```

Calling the `train()` method on the `ModelTrainer` instance launches a SageMaker training job. SageMaker provisions a managed ML instance, streams or copies the input data file(s) from S3 to the training execution environment (inside the ML instance), and runs the training script using the specified hyperparameters. Training and validation metrics are logged in real time, and when the job completes, the resulting model artifacts are automatically saved to the designated S3 output location, ready for deployment or further evaluation. After the training job finishes, the managed ML instance is automatically terminated, so you are not billed for idle compute time.

**Note**

This step may take 2 to 15 minutes to complete. If it takes longer than 15 minutes, you can check the **Training & tuning jobs** page in the Amazon SageMaker AI console (within AWS Management Console) to view your job's status and history. Keep in mind that the training job may remain in the **Pending** state for some time if SageMaker AI has not yet been able to allocate the requested compute resources (displayed as **Training job waiting for capacity**). If you experience issues running this example, you

> should stop the training job and consider using a different instance type before retrying.

You've successfully trained an XGBoost binary classifier and completed the core steps involved in building a supervised learning model. This includes preparing the dataset, configuring the training job, and generating a model artifact that can be used for inference. As an ML engineer, you should evaluate the model and plan the next steps for deployment or optimization. It's also important to document your findings, including hyperparameter configuration as well as observed performance, so you can reproduce or improve the model in future iterations. With these in mind, we can now deploy the model to an inference endpoint and evaluate its performance in the next section.

# Deploying an XGBoost model to a real-time inference endpoint

Deploying a trained model is a critical step in turning ML experiments into production-ready systems that can handle real user requests. By making your model available through an inference endpoint, you enable applications to generate predictions dynamically and integrate AI capabilities into operational workflows. In this section, you will deploy the XGBoost model to a real-time inference endpoint to make the trained model accessible for on-demand predictions. After deployment, you will send sample inference requests to the endpoint to verify that the model is functioning correctly and producing valid prediction outputs.

Before we deploy the XGBoost model, let's define a few key terms, concepts, and services relevant to what you'll work with in this section:

- **Inference endpoint**: A managed endpoint in Amazon SageMaker that hosts a deployed ML model and accepts prediction requests from client applications. Conceptually, it is similar to a virtual server hosting a containerized application that exposes an API for real-time inference.

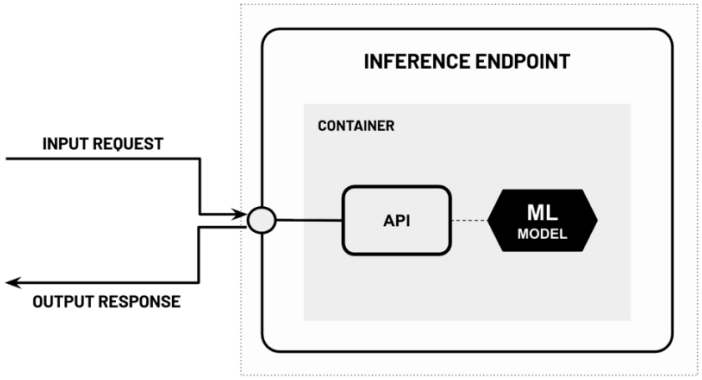



*Figure 3.9 — SageMaker inference endpoint*

For example, you can deploy a trained XGBoost model inside an inference endpoint to serve prediction requests. Here, client applications or components can send input data, such as transaction feature vectors, to obtain prediction outputs that determine whether a transaction is classified as fraudulent or legitimate.

- **Real-time inference**: A deployment mode where a trained model is hosted on a persistent endpoint and serves prediction requests immediately when they are received. In Amazon SageMaker AI, other deployment options are also available, such as **serverless inference**, **asynchronous inference**, and **batch transform**, depending on your workload requirements, traffic patterns, and cost optimization goals. Real-time inference is typically used when low-latency, on-demand prediction responses are needed.

- **Model evaluation**: The process of assessing model performance using metrics such as accuracy, precision, and recall to determine how well the model generalizes to unseen data. Model evaluation is important because it helps verify that the model meets performance requirements before it is deployed into production. At the same time, it allows you to compare different models and select the one that performs best for your use case. For example, let's say you have an XGBoost model that is used to automatically detect fraudulent transactions. If the model fails to correctly identify fraudulent activities or generates too many false alerts, it may cause financial losses or operational issues. Model evaluation helps you verify that the model meets the required detection performance before it is deployed in production.

**Note**

This section assumes you are running the code inside the `mle_sagemaker.ipynb` notebook.

Let's now proceed with deploying the XGBoost model to a real-time inference endpoint:

1. Import the necessary libraries, including `numpy` for numerical operations, `pandas` for data manipulation, and `XGBClassifier` from `xgboost`:

```
import numpy as np
import pandas as pd
from xgboost import XGBClassifier
```

2. Inspect the DataFrame of values stored in `test_data`:

```
test_data
```

This shows the test set DataFrame with 200 rows and 2 columns.

| | a | b |
|---|---|---|
| **993** | 1.912737 | -0.427197 |
| **859** | 2.117991 | 0.685974 |
| **298** | 0.034832 | -0.019911 |
| **553** | 1.380312 | 0.442379 |
| **672** | 1.128483 | -0.012440 |
| **...** | ... | ... |
| **679** | 0.709688 | -0.531570 |
| **722** | 0.533799 | 0.699297 |
| **215** | 2.289943 | 0.443640 |
| **653** | 1.798953 | -0.075756 |
| **150** | 0.048546 | 0.935311 |

200 rows × 2 columns

*Figure 3.10 — Test set with the target column removed*

Here, the test set stored in `test_data` does not include the `target` column and contains only the independent variables, `a` and `b`.

**Note**

Alternatively, you can download the CSV file containing the test data directly from the S3 bucket if you prefer to perform model evaluation in another environment.

3. Extract the first row from `test_data` and store it in the `sample_input` variable. Then, inspect the `sample_input` array:

```
sample_input = test_data.iloc[0].to_numpy()
sample_input
```

You should get something like this:

```
  array([ 1.9127373 , -0.42719723])
```

4. Store a sample output value (`[1.0]`) in a variable named `sample_output`:

```
sample_output = np.array([1.0])
```

5. Using the `sample_input` and `sample_output` variables defined in the previous steps, initialize a `SchemaBuilder` instance to define the schema for the model's input and output data:

```
schema_builder = SchemaBuilder(
    sample_input=sample_input,
    sample_output=sample_output
)
```

6. Define a custom `XGBoostModelSpec` class, with the `load()` method to load the XGBoost model and the `invoke()` method to make predictions based on the input data:

```
class XGBoostModelSpec(InferenceSpec):
    def load(self, model_dir: str):
        m = XGBClassifier()
        m.load_model(model_dir + "/xgboost-model")
        return m

    def invoke(
        self,
        input_object: object,
        model: object
    ):
        pred_proba = model.predict_proba(input_object)
        pred = np.argmax(pred_proba, axis=1)
        return pred
```

7. Create an instance of the `XGBoostModelSpec` class and store it in `inference_spec`. This instance will be used to load the model and make predictions:

```
inference_spec = XGBoostModelSpec()
```

8. Set the `instance_type` variable to specify the type of instance to be used for model deployment. In this case, we're using an `ml.m5.xlarge` instance for inference:

```
instance_type = "ml.m5.xlarge"
```

9. Initialize a `ModelBuilder` instance, configuring it with the `ModelTrainer` instance, IAM role, Docker image URI, `XGBoostModelSpec` instance, `SchemaBuilder` instance, and instance type for deployment on SageMaker:

```
model_builder = ModelBuilder(
    model=model_trainer,
    role_arn=role,
    image_uri=image,
    inference_spec=inference_spec,
    schema_builder=schema_builder,
    instance_type=instance_type,
)
```

10. Call the `build()` method on the `ModelBuilder` instance (`model_builder`) to initiate the process of building and deploying the model on SageMaker:

```
model_builder.build()
```

This yields the following set of logs:

```
INFO     Creating model with name: model-...
INFO     ✅ Model has been created: 'model-...' using server TRITON in
SAGEMAKER_ENDPOINT mode

Model(model_name='model-...', ...)
```

11. Deploy the model to a SageMaker inference endpoint:

```
endpoint_name = f"xgb-{prefix}-ep"

predictor = model_builder.deploy(
    endpoint_name=endpoint_name
)
```

This yields the following set of logs:

```
✅ Deployment successful: Endpoint 'xgboost-...' using model_builder.py:
1963 TRITON in SAGEMAKER_ENDPOINT mode (ARN: arn:aws:sagemaker:us-
east-1:...:endpoint/xgboost-ep-...)
```

**Note**

If you encounter an `AccessDeniedException` error (for example, `Failed to enable live logging: An error occurred ... when calling the FilterLogEvents operation`), update the SageMaker execution role (`AmazonSageMaker-ExecutionRole-...`) permissions to include the `CloudWatchFullAccessV2` policy or a more granular CloudWatch policy that allows access to log events. If there are already 10 permission policies attached to the role, you may need to remove a less critical policy, such as `AmazonSageMakerCanvasSMDataScienceAssistantAccess`, to make room for the required permissions.

12. Update the predictor's serializer and deserializer so that inference requests are sent to the model, and predictions are returned to the client (calling code) in the expected input (CSV) and output (JSON) formats:

```
from sagemaker.core.deserializers import (
    JSONDeserializer
)

from sagemaker.core.serializers import (
    CSVSerializer,
)

predictor.serializer = CSVSerializer()
predictor.deserializer = JSONDeserializer()
```

13. Use the `predictor.invoke()` method to send a sample input (`[-1.0, 1.5]`) to the model and retrieve the prediction result:

```
import json

result = predictor.invoke(
```

```
    body=[-1.0, 1.5],
    content_type="text/csv"
)

result.body
```

This should yield an output value as follows:

```
0.01789107546210289
```

This predicted value represents the model's estimated probability that the input belongs to the positive class (*LABEL 1*) in the binary classification problem. In other words, the deployed XGBoost model predicts that there is approximately a 1.79% likelihood that the `[-1.0, 1.5]` input belongs to class 1. Because this probability is very close to 0, the model would classify the input as *LABEL 0* when using a standard 0.5 decision threshold.

**Note**

The predicted value is generated by the inference endpoint, which applies the patterns and relationships learned during the training phase. The inference endpoint runs on a separate server outside of the Jupyter notebook, so requests are sent over the network to the endpoint, and the results are returned to your code.

14. Use the `predictor.invoke()` method to send a sample input (`[1.0, -1.0]`) to the model and retrieve the prediction result:

```
import json

result = predictor.invoke(
    body=[1.0, -1.0],
    content_type="text/csv"
)

result.body
```

This should give you the following output value:

```
0.9479150176048279
```

This time, the model outputs a predicted probability of approximately 0.9479, which represents a 94.79% likelihood that the input of `[1.0, -1.0]` belongs to the positive class (*LABEL 1*). Since this probability value is well above the standard 0.5 decision threshold, the model classifies this input as *LABEL 1*.

> **Note**
>
> If you want to invoke the inference endpoint from outside the notebook, such as inside an AWS Lambda function, you can use the `boto3` client for SageMaker runtime. This allows your Lambda function to send requests to the endpoint over the network and receive predictions, just like the `predictor.invoke()` method in the notebook.

15. Since the model returns a probability and we expect a binary output (`0` or `1`), let's define an `invoke_endpoint()` function that accepts two input features (`a` and `b`), sends them to the deployed endpoint, and returns the corresponding class label (that is, *LABEL 1* or *LABEL 0*):

```
import json

def invoke_endpoint(a, b, predictor=predictor):
    payload = f"{a},{b}"

    result = predictor.invoke(
        body=payload,
        content_type="text/csv"
    )

    probability = float(result.body)

    prediction = int(probability > 0.5)

    return prediction
```

16. Let's test the `invoke_endpoint()` function. Use the input values of `-1.0` and `1.5` and call the function to retrieve a sample prediction result:

```
invoke_endpoint(-1.0, 1.5)
```

You should get a predicted value of `0` (*LABEL 0*).

17. Let's test the `invoke_endpoint()` function again. Use the input values of `1.0` and `-1.0` and call the function to retrieve a sample prediction result:

```
invoke_endpoint(1.0, -1.0)
```

You should get a predicted value of `1` (*LABEL 1*).

> **Note**
>
> You can refer back to the scatterplot created earlier in this chapter to visually validate whether the predicted `target` value matches the input values.

18. Iterate through the rows of the test data, invoking the `invoke_endpoint()` function with each pair of input values (`a` and `b`), and store the predictions in a new column (`predicted_value`):

```
predictions = []

for _, row in var['test'].iterrows():
    pred = invoke_endpoint(row['a'], row['b'])
    predictions.append(pred)


var['test'].insert(0, 'predicted_value', predictions)
```

19. Let's examine the test set and review the `predicted_value` and `target` column values:

```
var['test']
```

This should display a DataFrame of values similar to the following:

| | predicted_value | target | a | b |
|---|---|---|---|---|
| **993** | 1 | 1 | 1.912737 | -0.427197 |
| **859** | 1 | 1 | 2.117991 | 0.685974 |
| **298** | 1 | 1 | 0.034832 | -0.019911 |
| **553** | 1 | 1 | 1.380312 | 0.442379 |
| **672** | 1 | 0 | 1.128483 | -0.012440 |
| **...** | ... | ... | ... | ... |
| **679** | 1 | 1 | 0.709688 | -0.531570 |
| **722** | 0 | 0 | 0.533799 | 0.699297 |
| **215** | 1 | 1 | 2.289943 | 0.443640 |
| **653** | 1 | 1 | 1.798953 | -0.075756 |
| **150** | 0 | 0 | 0.048546 | 0.935311 |

200 rows × 4 columns

*Figure 3.11 — DataFrame that includes the predicted_value column*

Here, the DataFrame now includes both the `predicted_value` column (generated by the model) and the original `target` column, allowing you to directly compare predictions against the true labels. This side-by-side view makes it easier to assess model performance and identify any misclassifications.

> **Note**
>
> Specifically, you want to check which rows have matching labels and which rows differ, as this highlights correct predictions versus errors.

20. Using scikit-learn's `metrics` module, compute the accuracy, precision, recall, F1 score, and confusion matrix:

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
```

```
)

target = var['test']['target']
predicted = var['test']['predicted_value']

accuracy = accuracy_score(target, predicted)
precision = precision_score(target, predicted)
recall = recall_score(target, predicted)
f1 = f1_score(target, predicted)
conf_matrix = confusion_matrix(target, predicted)
```

21. Print the computed evaluation metrics:

```
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score: {f1}")
print(f"Confusion Matrix:\n{conf_matrix}")
```

You should get the following set of values:

```
Accuracy: 0.84
Precision: 0.8557692307692307
Recall: 0.839622641509434
F1 Score: 0.8476190476190476
Confusion Matrix:
[[79 15]
 [17 89]]
```

**Note**

For more information about model evaluation metrics, feel free to check the following: `https://docs.aws.amazon.com/sagemaker/latest/dg/autopilot-metrics-validation.html`.

22. Finally, delete the SageMaker real-time inference endpoint:

```
predictor.delete()
```

This removes the deployed endpoint and terminates the managed ML instance hosting the model (`ml.m5.xlarge`), ensuring that you are no longer billed for the associated compute resources.

With these steps completed, you should have a good understanding of how to train and deploy ML models using Amazon SageMaker AI. You have learned the key steps involved in preparing data, configuring training jobs, evaluating model performance, and deploying models to inference endpoints. As an ML engineer, you should be able to apply these concepts and techniques to real-world problems, adapt workflows to different use cases, and build scalable, production-ready ML solutions. In practice, this means translating business requirements into ML solutions and ensuring that models deliver measurable impact. Building on what you've learned so far, we'll explore how to fine-tune a pretrained BERT model using SageMaker JumpStart in the next section.

# Setting up BERT fine-tuning with SageMaker JumpStart

The field of ML engineering has shifted from training models from scratch to leveraging large pre-trained models that can be adapted to specific tasks. Modern workflows focus on fine-tuning and task-specific alignment, which significantly reduces training time and data requirements. You can think of fine-tuning as professional onboarding training. A pretrained model is like a graduate who already has broad foundational knowledge. Once hired, the company provides focused training to help the employee perform specific job responsibilities. Similarly, fine-tuning adapts general model knowledge to better suit a particular application. Compared to training someone from the beginning of their education, onboarding a graduate usually requires less time and effort because they already understand the fundamentals. In the same way, fine-tuning a pretrained model requires significantly less data and compute resources than training a model from scratch. This makes fine-tuning a more practical approach when you want to quickly adapt a model to a new task while maintaining strong performance.

In the next few sections, we will explore how to fine-tune and deploy a BERT model using SageMaker JumpStart. We'll fine-tune a pretrained BERT model on the **Question Natural Language Inference** (**QNLI**) dataset so that it's able to better determine whether a passage logically supports/answers (*Entailment*) or does not support/answer (*Not Entailment*) a given question. This is especially useful when you're building question-answering systems, search engines, or **retrieval-augmented generation** (**RAG**) pipelines, where you need to verify whether the retrieved context truly answers a user's query. By fine-tuning on the QNLI dataset, you train the model to better recognize when a passage truly answers a question. This helps

your AI-powered applications give more accurate results and makes their responses more dependable.

**Note**

**Entailment** means that the context passage provides information that directly supports or answers the question. For example, if the question asks about the location of a landmark and the passage explicitly states where it is located, the pair is considered entailment. **Not Entailment**, on the other hand, means that the passage does not contain sufficient information to answer the question or is unrelated to the query, such as when the passage discusses a different topic or lacks the requested detail. For instance, if the question is `What time does the coffee shop open?` and the passage states, `The coffee shop opens at 7:00 AM every weekday`, this is Entailment because the passage directly answers the question. However, if the passage instead says, `The coffee shop is painted blue and has comfortable chairs`, this is clearly Not Entailment because it provides no information about opening hours. Similarly, if the question asks, `Does the gym offer yoga classes?` but the passage states, `The gym has a large parking lot and recently upgraded its sound system`, the pair is also Not Entailment since the passage does not mention yoga classes at all.

In this section, we'll prepare the notebook and initialize the BERT fine-tuning setup with Amazon SageMaker JumpStart. More specifically, you'll create a new JupyterLab notebook and ensure that the correct SageMaker Python SDK version is installed. After that, you'll specify the JumpStart model and configure relevant hyperparameters for fine-tuning. Finally, you will inspect the training configuration as well as the source code for fine-tuning the model. Since you are using SageMaker JumpStart, most of the prerequisites and settings have already been prepared and configured, allowing you to focus primarily on task-specific customization and training.

Let's define a few key terms, concepts, and services relevant to what you'll work with in this section:

- **Bidirectional Encoder Representations from Transformers (BERT)**: A Transformer-based deep learning model widely used for natural language understanding tasks such as text classification, question answering, and sentiment analysis. Unlike the original Transformer architecture, which uses an encoder-decoder structure, BERT utilizes only the encoder stack. Through **multi-head self-attention**, each token can attend to all other tokens in the sequence, enabling the model to capture deep bidirectional

contextual relationships. BERT is pretrained using two objectives: **masked language modeling** (**MLM**), which helps the model learn word-level context, and **next sentence prediction** (**NSP**), which helps it learn sentence-level relationships. These pretraining tasks enable BERT to learn rich contextual representations that can be fine-tuned for downstream tasks such as sentiment analysis, question answering, and **named entity recognition** (**NER**).

- **SageMaker JumpStart**: An ML hub that provides pretrained ML models, algorithms, and solution templates that you can immediately use with minimal configuration. SageMaker JumpStart allows you to quickly deploy foundation models or fine-tune pre-trained models, which helps accelerate experimentation by leveraging existing examples and reference implementations.
- **Natural language inference** (**NLI**): A natural language processing task that determines the logical relationship between two text statements, such as whether one statement entails, contradicts, or is neutral relative to another.
- **EULA**: An **end-user license agreement** that defines the legal terms and usage restrictions associated with software or ML models distributed through platforms such as model hubs.
- **Network isolation**: A security configuration that restricts a resource from accessing external network resources and services during execution. When network isolation is enabled, your SageMaker AI training and inference containers can't make any outbound network calls to any service, helping reduce exposure to external threats and unauthorized data access.

> **Note**
>
> This section assumes that you are running the hands-on examples inside a space in Amazon SageMaker Studio. You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1*.

With these in mind, let's go through the steps to prepare the JupyterLab notebook and initialize the BERT fine-tuning setup with SageMaker JumpStart:

1. In the left sidebar of JupyterLab, first navigate one directory higher to ensure you're in the right location. Then, right-click and select **New Folder**. Name the new folder `part2`, and double-click on it to enter the directory.

> **Note**
>
> Make sure that the `part1` and `part2` folders are placed next to each other within the same parent directory.

2. Click the **+** button to open the Launcher, then select **Python 3** (under **Notebook**). This will create a new `Untitled.ipynb` notebook inside the `part2` directory.
3. Let's rename the notebook to `sagemaker_jumpstart.ipynb`. Right-click on the **Notebook** tab and select **Rename notebook** from the context menu. Then, enter `sagemaker_jumpstart.ipynb` as the new name and click **Rename**.
4. Uninstall the current versions of `sagemaker` and `sagemaker-serve`, then install specific versions (`sagemaker` version 3.2.0 and `sagemaker-serve` version 1.2.0) to ensure that the setup works seamlessly with the hands-on code in this chapter:

```
%pip uninstall -y sagemaker sagemaker-serve
%pip install "sagemaker==3.2.0"
%pip install "sagemaker-serve==1.2.0"
```

5. After successfully installing the specified package versions, restart the kernel by selecting **Kernel** | **Restart Kernel** from the menu.

> **Note**
>
> This will ensure that the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

6. Suppress specific warnings related to namespace conflicts by filtering out warnings that match a given message pattern:

```
import warnings

m = "Field .* has conflict with protected namespace"

warnings.filterwarnings(
    "ignore",
    message=m
)
```

7. Import the necessary libraries, including key SageMaker Python SDK classes such as `ModelTrainer`, `JumpStartConfig`, and `Session` for model fine-tuning:

```
import json
import uuid

from sagemaker.train.model_trainer import ModelTrainer
from sagemaker.core.jumpstart import JumpStartConfig
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)
```

8. Initialize a SageMaker `Session` instance, retrieve the execution role (associated with the SageMaker AI domain), and retrieve the region and S3 bucket for model fine-tuning and deployment:

```
session = Session()
role = get_execution_role()
region = session.boto_region_name
bucket = session.default_bucket()
```

9. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length:

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase, k=length)
    )
```

You'll use this `generate_string()` function in later steps to generate identifiers for various AWS resources, such as SageMaker AI models and endpoints, to ensure that resource identifiers and names are unique.

10. Using the `generate_string()` function defined in the previous step, generate a random string and assign it to the `prefix` variable. Then, print the generated prefix to check its value:

```
prefix = generate_string()
print(prefix)
```

This should give you a random string of six lowercase letters, such as `jxdzyu`, which will change each time you run the function.

11. Initialize a `JumpStartConfig` instance with the specified model ID (`huggingface-spc-bert-base-cased`):

```
model_id = "huggingface-spc-bert-base-cased"

jumpstart_config = JumpStartConfig(
    model_id=model_id,
    accept_eula=False
)
```

Here, `huggingface-spc-bert-base-cased` refers to a BERT-base model designed for classifying relationships between two input sentences. This is commonly used for tasks such as NLI, question-answer relevance detection, and semantic similarity classification. For example, you can send a sentence pair such as `Where do the watersheds divide at?` and `The watersheds divide at Triple Divide Peak in Glacier National Park`. When you invoke the model, the BERT sentence pair classification model analyzes the relationship between the question and the candidate answer. The model then returns how likely the second sentence is a correct answer (or related) to the question.

**Note**

Given that this particular JumpStart model does not require explicit acceptance of additional licensing terms, we set `accept_eula=False`. In some cases, you may need to set `accept_eula=True` before you can train or deploy certain proprietary or third-party models. However, for this Hugging Face BERT model, you do not need to explicitly accept an EULA before proceeding.

12. Customize the hyperparameters, including the number of epochs, learning rate, and batch size, to configure the model training process:

```
hyperparameters = {
    "epochs": 2,
    "learning_rate": 6e-5,
    "train_batch_size": 32
}
```

13. Create a `ModelTrainer` instance to initialize the model trainer for training:

```
trainer = ModelTrainer.from_jumpstart_config(
    jumpstart_config=jumpstart_config,
    hyperparameters=hyperparameters,
    sagemaker_session=session
)
```

14. Inspect the source directory for the model's training script. This should return an S3 URI pointing to the source code directory in the S3 bucket:

```
trainer.source_code.source_dir
```

This should return an S3 URI pointing to the source code directory.

```
  's3://jumpstart-cache-.../source-directory-tarballs/huggingface/
  transfer_learning/spc/prepack/v1.1.0/sourcedir.tar.gz'
```

15. Store the training script's source directory in a variable named `source_dir`:

```
source_dir = trainer.source_code.source_dir
```

16. Download the source directory from the S3 URI specified in `source_dir` and save it as `sourcedir.tar.gz` in your local environment:

```
!aws s3 cp {source_dir} sourcedir.tar.gz
```

17. Create a new directory named `sourcedir` in your local environment to store the extracted source code files:

```
!mkdir -p sourcedir
```

18. Extract the contents of the `sourcedir.tar.gz` file into the newly created `sourcedir` directory using the `tar` command:

```
!tar -xzf sourcedir.tar.gz -C sourcedir
```

19. Inspect the extracted files and directories stored inside the `sourcedir` directory:

```
!ls -R sourcedir
```

This should give you the following output:

```
sourcedir:
__init__.py           js_inference_code  script_requirements.txt
__script_info__.json  lib                transfer_learning.py
constants             requirements.txt   version

sourcedir/constants:
__init__.py  constants.py

sourcedir/js_inference_code:
__init__.py  __script_info__.json  constants  inference.py  version

sourcedir/js_inference_code/constants:
__init__.py  constants.py

sourcedir/lib:
sagemaker_jumpstart_prepack_script_utilities

sourcedir/lib/sagemaker_jumpstart_prepack_script_utilities:
sagemaker_jumpstart_prepack_script_utilities-1.0.0-py2.py3-none-any.whl
```

*Figure 3.12 — Extracted files inside sourcedir*

Here, you'll find the training script and all supporting files used for fine-tuning the BERT model. This includes the main Python training script (`transfer_learning.py`), utility modules, a `constants.py` file defining configuration values, and any additional helper scripts required for data preprocessing, model loading, and evaluation. Together, these files form a self-contained package that SageMaker uses to run the training job.

20. Using **File Browser** (left sidebar), open and inspect the `transfer_learning.py` file located inside the `souredir` directory.

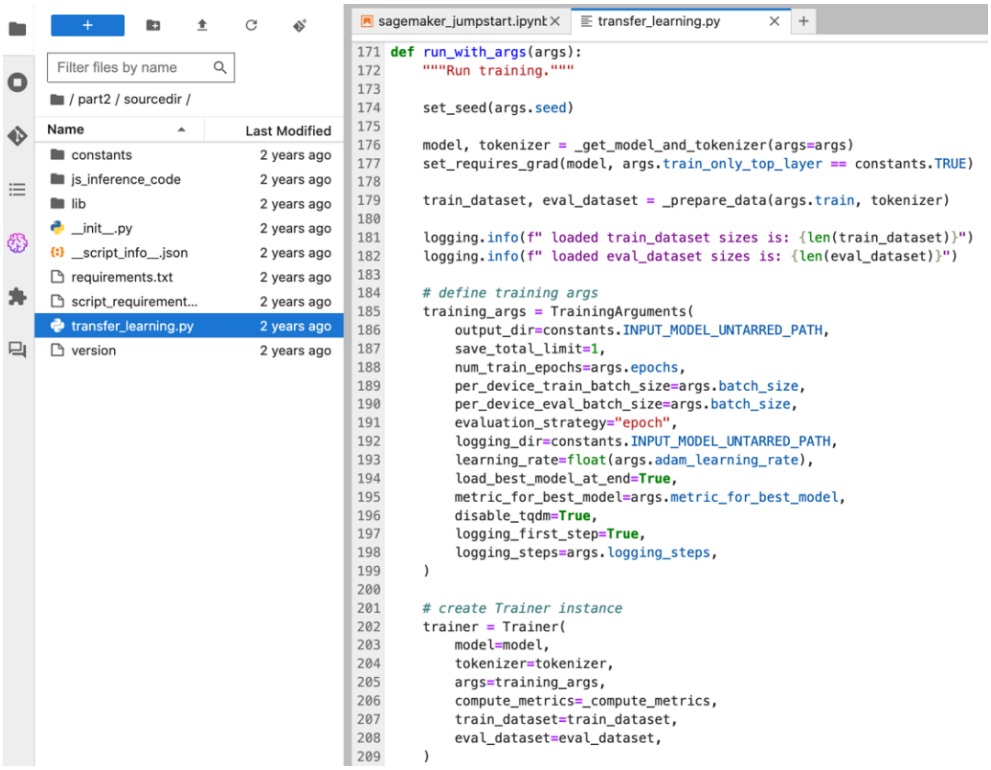



*Figure 3.13 — Inspecting the transfer_learning.py file*

The `transfer_learning.py` file is a fine-tuning script (with 226 lines of code) for adapting a pretrained BERT model to a specific NLP task. It handles argument parsing, dataset preparation, model loading, and tokenizer setup, and allows optional reinitialization or partial training of the top layer. The script preprocesses CSV data into PyTorch datasets, defines training parameters, and uses the Hugging Face Trainer API to train the model while computing metrics such as accuracy, precision, recall, and F1 score. After training, it saves the fine-tuned model and tokenizer to the specified output directory, updates the model metadata to indicate it has been fine-tuned, and copies inference code for deployment.

21. Next, using **File Browser**, open the `inference.py` file located inside the `js_inference_code` directory.

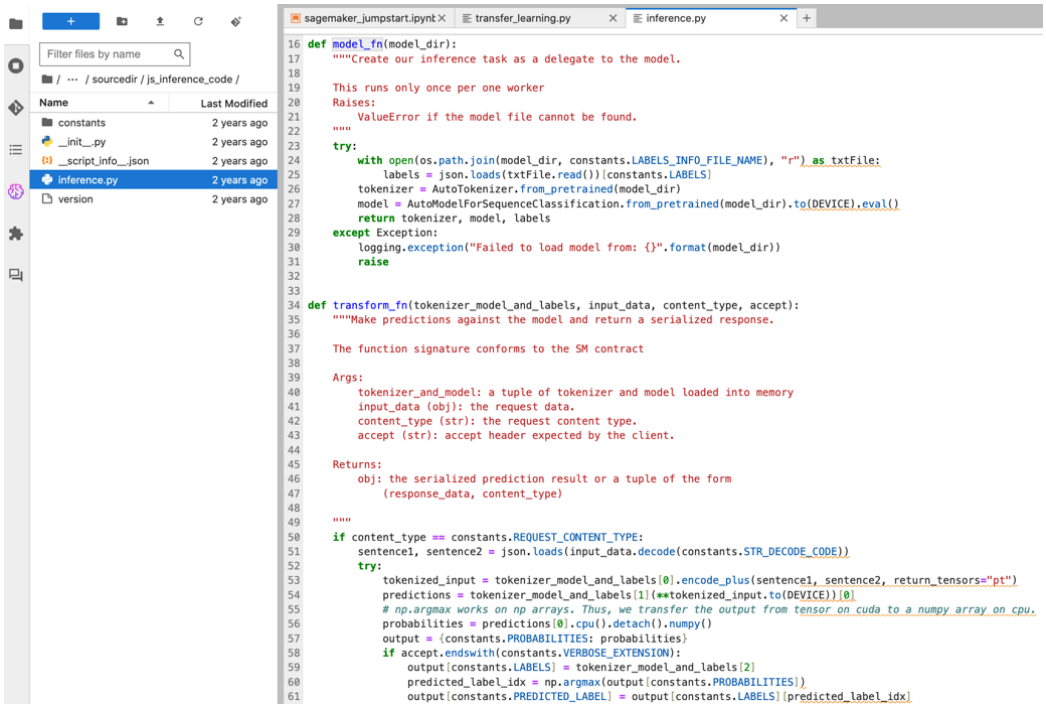



*Figure 3.14 — Inspecting the inference.py file*

The `inference.py` file is a Python script (with 68 lines of code) that defines how to load a fine-tuned BERT model, process input data, and generate predictions. It loads the trained model and tokenizer from the specified directory, sets up the device (CPU or GPU), and defines a `transform_fn()` function that takes input text pairs, tokenizes them, and computes prediction probabilities.

> **Note**
>
> The `inference.py` file is designed to work with SageMaker inference endpoints. It also includes logging and error checks to help you spot and troubleshoot any problems during prediction.

22. Switch back to the `sagemaker_jumpstart.ipynb` notebook and run the following to inspect the compute instance type that will be used for fine-tuning the model:

```
trainer.compute.instance_type
```

This returns the following output:

```
'ml.p3.2xlarge'
```

> **Note**
>
> Take note that it is possible to replace the instance type. For example, you can run `trainer.compute.instance_type = 'ml.p3.8xlarge'` to use a different instance type for your training job. However, make sure that the applied account-level service quota for the selected instance type is set to at least `1`; otherwise, SageMaker AI will not be able to provision the required compute resources, and the training job will fail.

23. Inspect the network isolation configuration for the model trainer to check whether it is enabled or disabled during model training:

```
trainer.networking.enable_network_isolation
```

This returns `True`, which means your SageMaker AI training container can't make any outbound network calls to any service, helping reduce exposure to external threats and unauthorized data access.

24. Inspect the hyperparameters that have been set for the model training by checking the `hyperparameters` attribute of the `trainer` instance:

```
trainer.hyperparameters
```

You should get the following dictionary of values:

```
{'epochs': 2,
 'adam-learning-rate': 2e-05,
 'batch-size': 8,
 'reinitialize-top-layer': 'Auto',
 'train-only-top-layer': 'False',
 'learning_rate': 6e-05,
 'train_batch_size': 32}
```

These hyperparameters impact how the model learns during training by defining important settings such as the learning rate, batch size, and the number of training epochs. For example, `learning_rate` determines how much the model updates its

weights during each optimization step, while `batch-size` affects how many training samples are processed together.

**Note**

As an ML engineer, you should experiment with different hyperparameter configurations to find a good balance between training efficiency and model performance.

At this point, you are ready to start the fine-tuning process. However, as you will see in the next section, you will be preparing and using a smaller dataset to validate the training workflow instead of using the full training dataset. This approach also speeds up the training job, so you won't have to wait as long for it to complete while testing the setup.

## Using a smaller dataset for fine-tuning

Using a smaller dataset allows you to iterate more quickly during experimentation, reduce computational cost, and validate your training workflow and code before running full-scale training and fine-tuning jobs. In this section, you will prepare a smaller version of the dataset for BERT fine-tuning and update the trainer configuration to use the new dataset. More specifically, you will inspect the training data configuration, download the dataset (which contains over 100,000 records) from Amazon S3, and prepare a subset consisting of 10,000 records for experimentation. You will then upload the new dataset to S3 and update the trainer configuration to ensure that the fine-tuning job uses the new training data.

**Note**

This section assumes you are running the code inside the `sagemaker_jumpstart.ipynb` notebook.

With this in mind, let's go through the steps to prepare and use a smaller dataset for fine-tuning:

1. Inspect the `input_data_config` attribute of the `ModelTrainer` instance, which contains the data channels and their corresponding configurations used for model training:

```
trainer.input_data_config
```

This returns the following:

```
[
    InputData(
        channel_name='training',
        data_source=S3DataSource(...),
        content_type=None),
    Channel(
        channel_name='model',
        ...
    )
]
```

2. Inspect the S3 URI of the training set:

```
trainer.input_data_config[0].data_source.s3_uri
```

This returns the following:

```
's3://jumpstart-cache-.../training-datasets/QNLI/'
```

3. Use the `mkdir` command to create a directory (`data`) for storing a copy of the dataset:

```
!mkdir -p data
```

4. Store the S3 URI of the dataset stored in an S3 bucket to a variable (`input_dataset`):

```
data_config = trainer.input_data_config[0]
input_dataset = data_config.data_source.s3_uri
```

5. List the contents of the S3 bucket where the training set is stored:

```
!aws s3 ls {input_dataset}
```

This returns the following output:

```
2020-11-28 01:53:34   26893414 data.csv
```

6. Download a copy of the dataset from the S3 bucket to the local `data` directory for further processing:

```
!aws s3 cp {input_dataset}data.csv data
```

7. Count the number of lines in the downloaded dataset to verify its size:

```
!wc -l data/data.csv
```

This gives you the following output:

```
114168 data/data.csv
```

8. Preview the first few lines of the downloaded dataset to check its structure using the `head` command:

```
!head data/data.csv
```

This returns the following:

```
0,What is the Grotto at Notre Dame?,"Immediately behind the basilica is the
Grotto, a Marian place of prayer and reflection."
1,What is the Grotto at Notre Dame?,"It is a replica of the grotto at
Lourdes, France where the Virgin Mary reputedly appeared to Saint
Bernadette Soubirous in 1858."
0,What sits on top of the Main Building at Notre Dame?,Atop the Main
Building's gold dome is a golden statue of the Virgin Mary.
...
```

9. Using **File Browser**, open and inspect the `data.csv` file (inside the `data` directory):

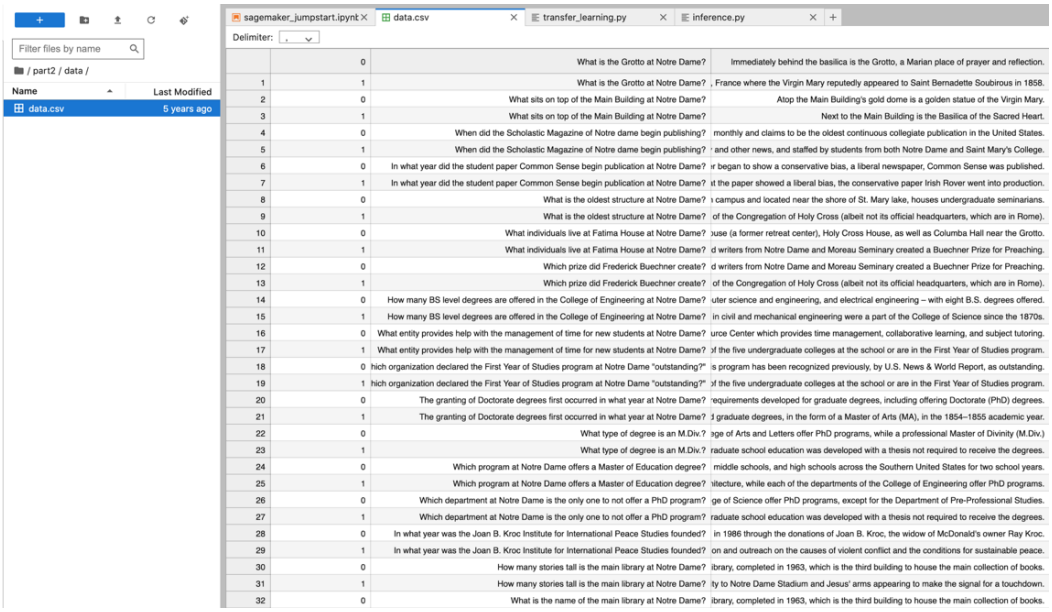



*Figure 3.15 — The data.csv file containing over 100,000 records*

The `data.csv` file contains three columns: the first column represents the class label (`0` for *Entailment* and `1` for *Not Entailment* based on the QNLI dataset), while the second and third columns contain the question and the corresponding context passage.

10. Use the `head` command to copy the first 10,000 lines from `data.csv` and save them to a new file (`data_10k.csv`):

```
!head -n 10000 data/data.csv > data/data_10k.csv
```

11. Verify the number of lines in the newly created file (`data_10k.csv`):

```
!wc -l data/data_10k.csv
```

This yields the following output:

```
  10000 data/data_10k.csv
```

12. Replace the original dataset with the newly created subset by renaming `data_10k.csv` to `data.csv`:

```
!mv data/data_10k.csv data/data.csv
```

13. Upload the training data to an S3 bucket:

```
DATA_DIRECTORY = "data"

train_input = session.upload_data(
    DATA_DIRECTORY + "/" + "data.csv",
    bucket=bucket,
    key_prefix=prefix
)
```

14. Let's check the S3 URI of the uploaded training data:

```
train_input
```

This should give you an S3 URI similar to the following:

```
  's3://sagemaker-.../akbcji/data.csv'
```

15. List the contents of the uploaded dataset in S3:

```
!aws s3 ls {train_input}
```

This should give you an output similar to the following:

```
  2026-01-01 00:00:00    2222778 data.csv
```

Here, you can see that the `data.csv` file has been uploaded successfully to the specified S3 location.

16. Update the input configuration to point to the newly uploaded file in S3:

```
conf = trainer.input_data_config[0]
conf.data_source.s3_uri = train_input
```

17. Verify the updated input data configuration for the trainer:

```
trainer.input_data_config[0]
```

This yields the following output:

```
InputData(
  channel_name='training',
  data_source=S3DataSource(
    s3_data_type='S3Prefix',
    s3_uri='s3://sagemaker-ap-.../akbcji/data.csv',
    ...
  ),
  content_type=None
)
```

At this point, your trainer is configured to use a subset of the dataset. You can now proceed with running the fine-tuning job using the reduced dataset to verify that the training pipeline is working as expected.

**Note**

Once you've completed the hands-on examples in this chapter, you can later scale the fine-tuning step using the full dataset if you want to further improve model performance.

## Running the BERT model fine-tuning job

In this section, you will fine-tune the BERT model with a SageMaker training job. The training process uses the QNLI dataset, which is an NLI dataset designed to help models learn semantic relationships between questions and corresponding context passages. With this dataset, the model learns to determine whether a passage logically supports/answers (*Entailment*) or does not support/answer (*Not Entailment*) a given question. Since you are using SageMaker JumpStart, the fine-tuning script as well as the training container image are already provided and configured for you. This simplifies the training setup because you do not need to manually

build or manage the training execution environment. Instead, you can focus on configuring the dataset, hyperparameters, and training job execution.

Once the fine-tuning step finishes, the pretrained BERT model is expected to better capture semantic and inference patterns specific to the QNLI dataset, allowing it to perform more accurately on question-context relationship classification tasks.

**Note**

This section assumes you are running the code inside the `sagemaker_jumpstart.ipynb` notebook.

Let's go through the steps to fine-tune the BERT model:

1. Retrieve the S3 URI for the pretrained BERT model, which will be used for fine-tuning:

```
conf = trainer.input_data_config[1]
model_tar_gz = conf.data_source.s3_data_source.s3_uri
model_tar_gz
```

   This yields the following output:

```
's3://jumpstart-cache-.../huggingface-training/train-huggingface-spc-bert-base-cased.tar.gz'
```

2. Copy the pretrained model from S3 to the local environment:

```
!aws s3 cp {model_tar_gz} model.tar.gz
```

3. Extract the contents of the model archive into a local directory (`model_files`):

```
!mkdir -p model_files
!tar -xzf model.tar.gz -C model_files
```

4. Use the `ls` command to list the extracted files in the `model_files` directory:

```
ls -lahF model_files
```

   This command lists the contents of the `model_files` directory in a detailed, human-readable format. The `-l` flag shows file permissions, ownership, size, and modification

date, `-a` includes hidden files, `-h` displays sizes in a human-readable format, and `-F` appends a symbol to indicate the file type.

Running the command yields the following output:

```
total 414M
...
-rw-r--r--. 1 sagemaker-user users  617 May 25  2021 config.json
-rw-r--r--. 1 sagemaker-user users 414M May 25  2021 pytorch_model.bin
-rw-r--r--. 1 sagemaker-user users  112 May 25  2021
special_tokens_map.json
-rw-r--r--. 1 sagemaker-user users  284 May 25  2021 tokenizer_config.json
-rw-r--r--. 1 sagemaker-user users 209K May 25  2021 vocab.txt
```

Here, you have the key model files such as `config.json`, `pytorch_model.bin`, `special_tokens_map.json`, `tokenizer_config.json`, and `vocab.txt`, which are all essential for loading and using the pretrained model.

5. Inspect the training image URI for the Hugging Face model:

```
trainer.training_image
```

This returns the URI of the Docker container image, which will be used for fine-tuning the model, which, in this case, would be the following:

```
'763104351884.dkr.ecr.....amazonaws.com/huggingface-pytorch-training:1.6.0-
transformers4.4.2-gpu-py36-cu110-ubuntu18.04'
```

When you start a training job, SageMaker AI launches a container based on this image. The training (or fine-tuning) script runs inside that container, which already includes the necessary frameworks, libraries, and dependencies required for the training process, allowing it to run in a controlled and predictable environment.

**Note**

SageMaker AI allows you to install additional dependencies while the training job is running. Custom container images are also supported if you want more control over the dependencies and execution environment.

6. With everything ready, start the model training process:

```
%%time
trainer.train()
```

This yields the following output:

```
INFO     Creating training_job resource.                                    resources.py:35539

INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ echo /opt/ml/input/config/resourceconfig.json:
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ cat /opt/ml/input/config/resourceconfig.json
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ echo
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ echo /opt/ml/input/config/inputdataconfig.json:
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ cat /opt/ml/input/config/inputdataconfig.json
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         {"current_host":"algo-1","current_instance_type":"ml.p3.2xlarge","c
         urrent_group_name":"homogeneousCluster","hosts":["algo-1"],"instanc
         e_groups":[{"instance_group_name":"homogeneousCluster","instance_ty
         pe":"ml.p3.2xlarge","hosts":["algo-1"]}],"network_interface_name":"
         eth0","topology":null}
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         /opt/ml/input/config/inputdataconfig.json:
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ echo
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ echo 'Setting up environment variables'
INFO     huggingface-spc-bert-base-cased-job-20260101000000/algo-1-176947203 resources.py:35869
         1:
         ++ /opt/conda/bin/python3
         /opt/ml/input/data/sm_drivers/scripts/environment.py
```

*Figure 3.16 — Logs generated while running the training job*

Calling the `train()` method launches a SageMaker training job. SageMaker provisions a managed ML instance (instance type: `ml.p3.2xlarge`), copies the input data file(s) from S3 to the training execution environment (inside the ML instance), and runs the training script using the specified hyperparameters. After the training job completes, the managed ML instance is automatically terminated.

**Note**

This step may take 10 to 15 minutes to complete. If it takes longer than 15 minutes, you can check the **Training & tuning jobs** page in the Amazon SageMaker AI console (within the AWS Management Console) to view your job's status and history. Keep in mind that the training job may remain in the

> **Pending** state for some time if SageMaker AI has not yet been able to allocate the requested compute resources (displayed as **Training job waiting for capacity**). If you experience issues running this example, you should stop the training job and consider using a different instance type (for example, `ml.p3.8xlarge`) before retrying.

7. Inspect the output data configuration for the `ModelTrainer` instance to retrieve the S3 path where the model output is stored:

```
data_config = trainer.output_data_config
output_path = data_config.s3_output_path
output_path
```

This yields an S3 URI similar to the following:

```
's3://sagemaker-.../huggingface-spc-bert-base-cased-job'
```

8. List the contents of the S3 output directory and filter for output files:

```
!aws s3 ls {output_path} --recursive | grep output
```

This lists the output file(s) in the S3 directory, which includes `model.tar.gz`:

```
...  402286194 huggingface-spc-bert-base-cased-job/huggingface-spc-bert-
base-cased-job-.../output/model.tar.gz
```

At this stage, you have completed the fine-tuning step. As you've just experienced in this section, SageMaker AI helps abstract the underlying infrastructure management so you can focus more on model development, training, and optimization instead of worrying about provisioning and maintaining compute resources. This allows you to concentrate on improving model performance and experimenting with different fine-tuning configurations as you work through the workflow. In the next section, you will see how the SageMaker Python SDK makes it easy to deploy the fine-tuned model to a real-time inference endpoint.

# Deploying the fine-tuned model to a real-time inference endpoint

In this section, you will deploy the fine-tuned BERT model to a SageMaker real-time inference endpoint. The endpoint is expected to accept a pair of statements and return output values that help determine whether a passage logically supports/answers (*Entailment*) or does not support/answer (*Not Entailment*) a given question. We'll go through the steps of deploying the model to a real-time inference endpoint and sending test inference requests to check the output of the deployed model. Once you have verified that the endpoint is working as expected, we'll delete the endpoint to avoid incurring additional costs.

Keep in mind that the overall deployment process is very similar to what you did with the XGBoost model earlier in this chapter. You will build the BERT model and generate artifact(s) for deployment using `ModelBuilder`, and deploy it to a real-time inference endpoint. The main difference lies in the model artifact(s), ML instance type, and container image being used, but the deployment workflow within SageMaker AI remains largely the same.

> **Note**
>
> This section assumes you are running the code inside the `sagemaker_jumpstart.ipynb` notebook.

With this in mind, let's proceed with deploying the fine-tuned model:

1. Initialize a `ModelBuilder` instance by passing the `ModelTrainer` instance (`trainer`) as an argument:

```
from sagemaker.serve.model_builder import ModelBuilder

model_builder = ModelBuilder(
    model=trainer,
    dependencies={"auto": False}
)
```

2. Generate a unique model name using the `generate_string()` function:

```
model_name = "jumpstart-" + generate_string()
```

3. Call the `build()` method of the `ModelBuilder` instance using the model name generated in the previous step:

```
core_model = model_builder.build(
    model_name=model_name
)
```

4. Deploy the model to a real-time inference endpoint (instance type: `ml.m5.large`), using a unique, randomly generated name:

```
%%time
endpoint_name = "jumpstart-ep-" + generate_string()
endpoint = model_builder.deploy(
    endpoint_name=endpoint_name
)
```

> **Note**
>
> This step may take 3 to 10 minutes to complete. You can take a short break while it completes.

5. Send a test request to the deployed model's endpoint for inference:

```
test_data = [
    "Where do the watersheds divide at?",
    "The watersheds divide at Triple Divide Peak in
     Glacier National Park."
]

body = json.dumps(test_data)

result = endpoint.invoke(
    body=body,
    content_type="application/list-text"
)

prediction = json.loads(
    result.body.read().decode('utf-8')
)
print(prediction)
```

This yields the following output:

```
{
    'probabilities': [
        2.7255473136901855,
        -2.737187385559082
    ]
}
```

Here, the two values shown are raw logits produced by the fine-tuned BERT sequence classification model. Because the first value (`2.73`) is significantly larger than the second (`-2.74`), the model predicts *CLASS 0* with high confidence. Based on the fine-tuning data used for training, *CLASS 0* corresponds to *Entailment*, meaning the context passage supports or answers the question. This prediction indicates that the model believes the *input pair is logically related*.

**Note**

Logits are the raw output scores produced by a classification model before they are converted into probabilities

6. Send another test request to the deployed model:

```
test_data = [
    "Where do the watersheds divide at?",
    "Montana is one of few geographic areas in the world
     whose rivers form parts of three major watersheds
     (i.e. where two continental divides intersect)."
]

body = json.dumps(test_data)

result = endpoint.invoke(
    body=body,
    content_type="application/list-text"
)

prediction = json.loads(
    result.body.read().decode('utf-8')
```

```
)
print(prediction)
```

This returns the following output:

```
{
    'probabilities': [
        -2.8714895248413086,
        2.6521551609039307
    ]
}
```

Because the second value (`2.65`) is significantly larger than the first (`-2.87`), the model predicts *CLASS 1* with high confidence. Based on the fine-tuning data used during training, *CLASS 1* corresponds to *Not Entailment*, meaning the context passage does not provide sufficient information to support or answer the question. This prediction indicates that the model believes *the input pair is not logically related*.

**Note**

You can define a helper function that maps the model's raw output values into human-readable labels such as `"Entailment"` or `"Not Entailment"`. This allows your application to directly return meaningful prediction results instead of exposing numerical scores. For example, you can simply implement a function that interprets the prediction output and returns the corresponding classification label based on your chosen decision rule.

7. Finally, delete the deployed model endpoint:

```
endpoint.delete()
```

This deletes the deployed endpoint and makes it no longer accessible for subsequent inference requests.

At this point, you should have a good idea of how to deploy fine-tuned models to a real-time inference endpoint in Amazon SageMaker AI. While there are various deployment strategies for ML models, the real-time inference deployment option presented in this chapter is one of the most common approaches used for serving models in production environments.

As an ML engineer, you should be familiar with other deployment options available in Amazon SageMaker AI and understand when each option is appropriate. Choosing the best deployment

strategy requires evaluating factors such as latency requirements, traffic patterns, cost constraints, and application architecture. Understanding the trade-offs between real-time inference, batch transform, asynchronous inference, and serverless inference will help you design and implement production-ready ML engineering solutions. You should also experiment with different instance types to find the optimal balance between performance, resource utilization, and operational cost.

> **Note**
>
> Now that you have completed the hands-on exercises in this chapter, make sure to stop or delete any unused resources, such as inference endpoints or spaces, in SageMaker AI to avoid unnecessary costs.

# Summary

In this chapter, you explored how to use SageMaker AI to implement end-to-end machine learning engineering workflows. You started by training and deploying an XGBoost model and then proceeded to fine-tuning a BERT model. As you worked through the examples, you learned how SageMaker AI simplifies the complexity of building and deploying various types of machine learning models by providing managed infrastructure for training, inference, and model lifecycle management.

In the next chapter, you will shift your focus to data management on AWS and explore how to modernize analytics with a managed transactional data lake. You will work with Amazon S3 Tables backed by the Apache Iceberg table format, which enables ACID transactions, schema evolution, and time-travel queries on data stored in Amazon S3, enabling you to run reliable and consistent analytics directly on your data lake.

# Further reading

To expand your practical knowledge of the concepts presented in this chapter, refer to the following resources:

- *SageMaker Python SDK Official GitHub repository* (`https://github.com/aws/sagemaker-python-sdk`)
- *SageMaker Python SDK V3 documentation* (`https://sagemaker.readthedocs.io/en/stable/`)

- *Amazon SageMaker AI Developer Guide | Programming Model for Amazon SageMaker AI* (`https://docs.aws.amazon.com/sagemaker/latest/dg/how-it-works-prog-model.html`)
- *AWS Blogs | Artificial Intelligence | Category: Amazon SageMaker* (`https://aws.amazon.com/blogs/machine-learning/category/artificial-intelligence/sagemaker/`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Keep your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 4

# Modernizing Analytics with a Managed Transactional Data Lake

Before training **machine learning** (**ML**) models, organizations may store, organize, and aggregate raw data from applications, databases, logs, and streaming systems into a centralized storage solution, such as a data warehouse or data lake. Nowadays, teams have the option to get the best of both worlds by building transactional data lake architectures using **Amazon S3 Tables**, which leverages open table formats such as Apache Iceberg to allow you to organize data into structured logical tables that behave similarly to traditional databases while preserving the scalability and flexibility of object storage. When working with large-scale data processing and transformation workloads, you can combine S3 Tables with distributed processing solutions such as **Amazon Elastic MapReduce** (**EMR**) and Apache Spark to clean, filter, aggregate, and engineer features from raw and semi-structured datasets before they are used for model training and inference for analytics and ML workflows.

In this chapter, you will focus on building and working with a transactional data lake using S3 Tables. You will explore how ML engineers can set up and query S3 table buckets, and process datasets stored in these table buckets using an EMR cluster with Iceberg support.

That said, we will cover the following topics in this chapter:

- Preparing and processing the synthetic data
- Creating an Amazon S3 table bucket
- Launching an Amazon EMR cluster with Apache Iceberg installed
- Performing Apache Iceberg queries on S3 tables with Apache Spark
- Performing time travel queries on S3 tables

By the end of this chapter, you will have a good understanding of how S3 tables can serve as the foundation of a transactional data lake for analytics and ML workloads.

# Technical requirements

Before working through the hands-on examples in this chapter and the next, make sure that all required technical prerequisites are properly set up and configured. Make sure you have the following prerequisites ready:

- **A text file where you can store important resource identifiers and configuration values**: You will use this file to keep track of key details such as your Amazon S3 general-purpose bucket name (`TAG:DATASET_BUCKET`), your S3 table bucket name (`TAG:TABLE_BUCKET_NAME`), the EC2 instance profile associated with your Amazon EMR cluster, and the primary node public DNS address of your EMR cluster. For instance, after uploading your CSV and Parquet datasets to your S3 general-purpose bucket, you'll be recording the bucket name in this file so you can easily reference it later when performing Apache Iceberg queries on S3 tables with Apache Spark.
- **A Virtual Private Cloud (VPC) network environment with at least two public subnets**: A VPC is a logically isolated virtual network in AWS that allows you to define and control your network settings. Inside a VPC, you can deploy and run AWS resources such as EC2 instances, databases, and other resources while controlling access and security. You can set up and configure this VPC network in the AWS Management Console by typing `vpc` in the search bar and selecting **VPC** from the search results. In the VPC console, click the **Create VPC** button, then select the **VPC and more** option. Specify the number of public and private subnets you require (that is, **2** public subnets and **0** private subnets), and proceed through the configuration steps. You will use this VPC later in the chapter when setting up an Amazon EMR cluster.

Before proceeding, confirm that your currently selected AWS Region matches the region where your VPC has been set up. You can change the region at any time by using the region selector in the upper-right corner of the AWS Management Console and choosing the correct region (for example, **us-east-1**) from the drop-down list.

> **Note**
>
> All the code, commands, and files used in this chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

## Preparing and processing the synthetic data

Imagine an e-commerce platform where customers, after purchasing a product, can leave comments or feedback about their experience. When this feedback is collected for training a sentiment analysis model, an internal team manually reviews each comment, assigns a sentiment score, labels it as `POSITIVE`, `NEUTRAL`, or `NEGATIVE`, and flags any spam, while recording the date of submission. This data, previously stored across multiple databases, is then centralized within a transactional data lake on AWS, which enables reliable and query-optimized access for analytics and ML workloads. Over time, the data stored in the data lake is used to train a ML model that can automatically score and tag sentiment for future customer comments. In this section, we will prepare synthetic data that reflects this scenario. We will download, clean, and process the synthetic data to prepare it for use in the hands-on examples that follow.

Before we jump into the hands-on portion, let's take a moment to cover some important terms and ideas:

- **Synthetic data**: Artificially generated data that has a structure and characteristics similar to real-world data, often used for testing, experimentation, or training ML models. Using synthetic data allows us to safely experiment, test workflows, and explore tools and techniques with datasets that resemble real-world data, without the privacy, security, or compliance concerns of actual customer data. It also keeps examples simpler and more focused, making it easier to understand key concepts.
- **Comma-Separated Values (CSV) file**: A text file format used to store tabular data. In this format, each line represents a row, and values are separated by commas. For example, a CSV file might store a list of user information with columns for `Name`, `Email`, and `Age`, where each row represents a different user.
- **Parquet file**: A columnar storage file format optimized for analytical workloads, which allows for efficient storage and retrieval of large datasets. Compared to CSV files, Parquet files store data by columns rather than rows, making queries on specific fields faster and reducing storage space for large datasets. For example, a Parquet file might store millions of rows of sales data with columns for `Date`, `Product`, and `Revenue`, to enable fast aggregation on just the `Revenue` column.
- **Amazon Simple Storage Service (S3)**: A scalable, durable, and secure object storage service from AWS used for general-purpose storage, backups, media assets, and other applications. It is also ideal for storing and retrieving datasets, training data, and other files used in data engineering, ML, and analytics workflows.

- **Amazon S3 general-purpose bucket**: A bucket type recommended for most use cases and access patterns. It stores objects (files) without specialized table or query features. In this chapter, we'll utilize several general-purpose S3 buckets to store and pass input and output files between different AWS services and resources.
- **AWS CloudShell**: A browser-based shell environment provided by AWS that allows you to run terminal commands, scripts, and manage various types of resources.

Being familiar with these terms will make it easier to work through the steps for preparing and processing the synthetic data in this section. Once the synthetic data is ready, we will store the files in an Amazon S3 general-purpose bucket, allowing them to be used as source data for ingestion into the transactional data lake.

## Downloading and preparing the synthetic tabular data

In this section, we will download two sample sentiment analysis datasets containing fields such as `statement`, `score`, `tag`, `is_spam`, and `date`. We will assign a `unique_id` value to each record to identify them individually. To simulate a real-world scenario where data comes from two different datastores, we will export one dataset as a CSV file and the other as a Parquet file. This will show the differences in data formats that data and ML engineers often work with. Later in the chapter, we will merge these datasets into a single table to give you hands-on experience in combining and managing data from multiple sources and formats.

> **Note**
>
> In this section, you will use the IAM root user account (that is, the original AWS account root user) to perform the initial setup steps. Keep in mind that for production use cases, using the root user is not recommended.

Let's now go through the process of downloading and preparing the synthetic tabular datasets:

1. Navigate to the CloudShell console through the search bar by typing `cloudshell` and then choosing **CloudShell** from the list of search results. Alternatively, you can click the CloudShell icon (**>_**) located in the upper-right corner of the AWS Management Console to open the CloudShell console. Inside CloudShell, you can run AWS CLI commands, manage resources, execute scripts, and interact with your AWS environment directly from a browser-based shell.

   > **Note**
   >
   > AWS CloudShell is not available in all AWS Regions. If it is unavailable in your preferred region, you can switch to a supported region such as `us-east-1` to continue with the steps. When working on the hands-on examples of this

> chapter, make sure that your VPC and all other resources you plan to use are set up within the same region.

2. Now that our CloudShell environment is ready, run the following command (after the $ sign) to install **IPython**, **pandas**, and **fastparquet**:

```
pip install ipython pandas fastparquet
```

Let's quickly discuss what these are for. IPython lets you interactively run Python code and explore your datasets efficiently. With `pandas`, you can manipulate and analyze your data, including creating, filtering, and transforming data frames. Lastly, `fastparquet` allows you to read and write Parquet files quickly, which is especially helpful when you are working with large datasets stored in columnar format.

> **Note**
>
> The following versions are used in this chapter: IPython 8.18.1, `pandas` 2.3.2, and `fastparquet` 2024.11.0. Versions close to these should also work without any issues.

3. Open the IPython **Read-Eval-Print-Loop** (**REPL**) interactive shell by running the following in the CloudShell terminal:

```
ipython
```

This should launch the IPython interactive environment.

4. Import the `pandas` library and prepare the `url_directory` string, which points to the GitHub folder containing the chapter's data files:

```
import pandas as pd

url_directory = ''.join([
    "https://raw.githubusercontent.com/",
    "PacktPublishing/",
    "Machine-Learning-Engineering-on-AWS",
    "-Second-Edition/",
    "refs/heads/main/chapter05/"
])
```

> **Note**
>
> When copying code directly from the GitHub repository into your IPython or Jupyter environment, you can use the `%cpaste` magic command. This allows you to paste multi-line code blocks safely without indentation or formatting issues. To use it, type `%cpaste` and press *Enter*, then paste your code. When you're done, type `--` on a new line and press *Enter* to execute the pasted code.

5. Define the `get_url()` function, which builds a complete URL by appending a given `filename` value to the `base url_directory`:

```
def get_url(filename, url_directory=url_directory):
    return url_directory + filename
```

6. Now, using the `get_url()` function you defined in the previous step, run the following block of code to load two CSV files (`data_up_to_2024.csv` and `data_2025_onwards.csv`) from the GitHub repository:

```
data_up_to_2024 = get_url("data_up_to_2024.csv")
df_up_to_2024 = pd.read_csv(data_up_to_2024)

data_2025_onwards = get_url("data_2025_onwards.csv")
df_2025_onwards = pd.read_csv(data_2025_onwards)
```

7. Let's quickly check what the first `DataFrame` looks like:

```
df_up_to_2024.head(20)
```

This should give us the following table of values:

| | statement | score | tag | is_spam | date |
|---|---|---|---|---|---|
| 0 | I love this! | 0.60 | POSITIVE | False | 2023-01-14 |
| 1 | The product is okay. | -0.08 | NEUTRAL | False | 2023-02-04 |
| 2 | It works as expected. | -0.10 | NEUTRAL | False | 2023-02-15 |
| 3 | This is awful. | -0.89 | NEGATIVE | False | 2023-03-09 |
| 4 | This is the best purchase I've made. | 0.97 | POSITIVE | False | 2023-03-17 |
| 5 | The product is okay. | -0.01 | NEUTRAL | False | 2023-04-13 |
| 6 | I would not recommend this to anyone. | -0.72 | NEGATIVE | False | 2023-04-21 |
| 7 | It's fine, neither good nor bad. | -0.15 | NEUTRAL | False | 2023-05-19 |
| 8 | Great service! | 0.86 | POSITIVE | False | 2023-05-22 |
| 9 | Highly recommend this product! | 0.75 | POSITIVE | False | 2023-06-04 |
| 10 | Anyone interested? | NaN | NaN | True | 2023-06-14 |
| 11 | Highly recommend this product! | 0.50 | POSITIVE | False | 2023-06-19 |
| 12 | It's fine, neither good nor bad. | 0.07 | NEUTRAL | False | 2023-07-17 |
| 13 | This is awful. | -0.80 | NEGATIVE | False | 2023-07-30 |
| 14 | Great service! | 0.72 | POSITIVE | False | 2023-08-11 |
| 15 | It's fine, neither good nor bad. | 0.08 | NEUTRAL | False | 2023-08-14 |
| 16 | Just checking in | NaN | NaN | True | 2023-08-27 |
| 17 | This product is amazing! | 0.87 | POSITIVE | False | 2023-09-11 |
| 18 | I hate this product. | -0.52 | NEGATIVE | False | 2023-09-21 |
| 19 | Nothing special about it. | -0.11 | NEUTRAL | False | 2023-09-25 |

*Figure 4.1 — The content of data_up_to_2024.csv*

Here, you have a DataFrame with five columns: `statement`, `score`, `tag`, `is_spam`, and `date`. Each row represents an individual record, where the statement contains a customer's comment or feedback, and the score quantifies the sentiment expressed, indicating how positive or negative the statement is on a defined scale. The `tag` column is used to categorize the sentiment of each entry (`POSITIVE`, `NEUTRAL`, or `NEGATIVE`), while `is_spam` indicates whether the record has been identified and flagged as spam. Finally, the `date` column records when the customer submitted the comment or feedback.

8. Now, let's check how the second `DataFrame` (`df_2025_onwards`) looks:

```
df_2025_onwards.head(20)
```

This should give us the following table of values:

| | statement | score | tag | is_spam | date |
|---|---|---|---|---|---|
| 0 | It's fine, neither good nor bad. | -0.09 | NEUTRAL | False | 2025-01-01 |
| 1 | I hate this product. | -0.73 | NEGATIVE | False | 2025-01-09 |
| 2 | The product is okay. | -0.10 | NEUTRAL | False | 2025-01-17 |
| 3 | This is the best purchase I've made. | 0.59 | POSITIVE | False | 2025-02-05 |
| 4 | Terrible experience. | -0.94 | NEGATIVE | False | 2025-02-09 |
| 5 | Very disappointed with this. | -0.96 | NEGATIVE | False | 2025-02-14 |
| 6 | It works as expected. | -0.08 | NEUTRAL | False | 2025-02-24 |
| 7 | This is the best purchase I've made. | 0.88 | POSITIVE | False | 2025-03-14 |
| 8 | Highly recommend this product! | 0.68 | POSITIVE | False | 2025-03-20 |
| 9 | This is awful. | -0.83 | NEGATIVE | False | 2025-03-27 |
| 10 | This product is amazing! | 0.89 | POSITIVE | False | 2025-04-03 |
| 11 | Anyone interested? | NaN | NaN | True | 2025-04-11 |
| 12 | Terrible experience. | -0.73 | NEGATIVE | False | 2025-05-07 |
| 13 | It works as expected. | 0.10 | NEUTRAL | False | 2025-05-15 |
| 14 | BUMP | NaN | NaN | True | 2025-05-26 |
| 15 | It's fine, neither good nor bad. | 0.11 | NEUTRAL | False | 2025-06-28 |
| 16 | I would not recommend this to anyone. | -0.74 | NEGATIVE | False | 2025-07-09 |
| 17 | I hate this product. | -0.64 | NEGATIVE | False | 2025-07-12 |
| 18 | The product is okay. | 0.12 | NEUTRAL | False | 2025-08-04 |
| 19 | Terrible experience. | -0.85 | NEGATIVE | False | 2025-08-17 |

*Figure 4.2 — The content of data_2025_onwards.csv*

The columns in `df_2025_onwards` match those in `df_up_to_2024`. The difference is that it includes only the more recent records, starting from January 1, 2025 (`2025-01-01`).

9. Define the `generate_id()` function, which returns a randomly generated **Universally Unique Identifier** (**UUID**) by calling `uuid.uuid4()` and converting it to a string:

```
import uuid

def generate_id():
    return str(uuid.uuid4())
```

You think of the UUID as a unique serial number for each record that is guaranteed to be different from every other record, even across systems.

10. Using the `generate_id()` function we defined in the previous step, generate a unique ID for each of the rows of the table and store these in a `unique_id` column using the following block of code:

```
df_up_to_2024['unique_id'] = df_up_to_2024.apply(
    lambda _: generate_id(), axis=1
)
```

```
df_2025_onwards['unique_id'] = df_2025_onwards.apply(
    lambda _: generate_id(), axis=1
)
```

11. Rearrange the columns so that we have the `unique_id` column as the first column:

```
unique_column = df_up_to_2024.pop('unique_id')
df_up_to_2024.insert(0, 'unique_id', unique_column)

unique_column = df_2025_onwards.pop('unique_id')
df_2025_onwards.insert(0, 'unique_id', unique_column)
```

12. Preview the first 10 rows of each dataset:

```
df_up_to_2024.head(10), df_2025_onwards.head(10)
```

This returns the following:

```
(                              unique_id                          statement  score       tag  is_spam        date
 0  704accd0-2635-4652-8a45-6aacddaf6159                        I love this!   0.60  POSITIVE    False  2023-01-14
 1  50410171-c31b-441e-a6a5-897120cba804                The product is okay.  -0.08   NEUTRAL    False  2023-02-04
 2  499a0e19-4b40-4fd4-a02e-1425946bdc1a               It works as expected.  -0.10   NEUTRAL    False  2023-02-15
 3  723b3c90-a1b0-4e53-a396-10084ef3dfec                       This is awful.  -0.89  NEGATIVE    False  2023-03-09
 4  8d51c5a8-cc53-47b8-a3a5-0dbdae201aee  This is the best purchase I've made.   0.97  POSITIVE    False  2023-03-17
 5  4e81375f-8ef6-4e63-9d25-18b60e74a090                The product is okay.  -0.01   NEUTRAL    False  2023-04-13
 6  f7745c97-6934-49cc-9102-9c667b0b1f32  I would not recommend this to anyone.  -0.72  NEGATIVE    False  2023-04-21
 7  af8ee513-466e-4c58-a120-e1b672be74c4      It's fine, neither good nor bad.  -0.15   NEUTRAL    False  2023-05-19
 8  c791e192-2fb2-456d-87be-5583a13a80d4                     Great service!   0.86  POSITIVE    False  2023-05-22
 9  081d0ef6-5e45-4500-99be-aa322eb60625        Highly recommend this product!   0.75  POSITIVE    False  2023-06-04,
                              unique_id                          statement  score       tag  is_spam        date
 0  d049037e-9c71-406d-939d-8d596143520f      It's fine, neither good nor bad.  -0.09   NEUTRAL    False  2025-01-01
 1  eec8df50-6882-493c-984b-6b9074933408                 I hate this product.  -0.73  NEGATIVE    False  2025-01-09
 2  9ad86d0c-b29f-4558-bf6f-b8070a89dc1e                The product is okay.  -0.10   NEUTRAL    False  2025-01-17
 3  c39d4c33-2ad9-4482-afef-fcd213cf5b63  This is the best purchase I've made.   0.59  POSITIVE    False  2025-02-05
 4  119a4165-467a-4ddc-8126-029029ee8ebe                 Terrible experience.  -0.94  NEGATIVE    False  2025-02-09
 5  6b6cd8c3-6ec9-4627-bcc6-d6db6d7df1b9           Very disappointed with this.  -0.96  NEGATIVE    False  2025-02-14
 6  62656615-3e2a-42ed-a821-6179c37b23b8               It works as expected.  -0.08   NEUTRAL    False  2025-02-24
 7  57057e78-1f08-4d15-bebe-6636bdbd373b  This is the best purchase I've made.   0.88  POSITIVE    False  2025-03-14
 8  96a74f5c-0ecd-4521-bb21-b65c3c14e785        Highly recommend this product!   0.68  POSITIVE    False  2025-03-20
 9  6391548c-91a1-4bad-95dd-d98435bae9c7                       This is awful.  -0.83  NEGATIVE    False  2025-03-27)
```

*Figure 4.3 — First 10 rows of df_up_to_2024 and df_2025_onwards*

Both `df_up_to_2024` and `df_2025_onwards` now include a `unique_id` column. This guarantees that each row can be uniquely identified, even if other columns have identical values. It also simplifies tracking, referencing, and merging records across datasets, reducing the risk of duplication and ensuring data integrity during further processing.

> **Note**
>
> As you'll see later in this chapter, `unique_id` will be especially helpful when working with online and offline feature stores. It lets us track and manage multiple versions of the same data reliably and ensures that each record can be retrieved or updated without confusion.

13. Let's export and save the values of the first DataFrame to a CSV:

```
df_up_to_2024.to_csv(
    'data_up_to_2024.csv',
    index=False
)
```

14. Then, let's save the values of the second DataFrame to a Parquet file:

```
df_2025_onwards.to_parquet(
    "data_2025_onwards.parquet",
    engine="fastparquet",
    index=False
)
```

15. Finally, exit the REPL environment:

```
exit
```

At this point, you have `data_up_to_2024.csv` and `data_2025_onwards.parquet`, with each record assigned a unique ID for easy tracking. These datasets are now ready to be merged and ingested into the transactional data lake later in the chapter.

## Inspecting the CSV and Parquet files

With the CSV and Parquet files ready, we now have two very different ways to store tabular data. CSV is simple and easy to read, but can be slow and large for big datasets, while Parquet is a columnar format optimized for performance and storage. In real-world data engineering and ML projects, you may also work with formats such as JSON, Avro, ORC, or Feather.

With this in mind, let's now proceed with inspecting the files in the next set of steps:

1. In the CloudShell terminal, run the following command to see what the stored data in our CSV file looks like:

```
head -n 20 data_up_to_2024.csv
```

This gives us an output similar to the following:

```
unique_id,statement,score,tag,is_spam,date
35ce3303-f520-4dbe-9759-ef87f261c3f3,I love this!,0.6,POSITIVE,False,2023-01-14
0e84ff9b-2845-48b7-9674-d4da4f3f3fee,The product is okay.,-0.08,NEUTRAL,False,2023-02-04
d36d5a11-138d-49a5-bcb8-acd2a889493a,It works as expected.,-0.1,NEUTRAL,False,2023-02-15
05fde437-4630-43ac-821a-77d86ead113a,This is awful.,-0.89,NEGATIVE,False,2023-03-09
bd2aabd2-e388-4616-acf9-90358091e76b,This is the best purchase I've made.,0.97,POSITIVE,False,2023-03-17
9c47757f-35be-420e-92bd-6c7eebd5a2b1,The product is okay.,-0.01,NEUTRAL,False,2023-04-13
e0bfe69b-ff8a-43f5-babc-8e9e857d3c6a,I would not recommend this to anyone.,-0.72,NEGATIVE,False,2023-04-21
b647c226-acb1-4d1e-ae43-25a5bcc08aa9,"It's fine, neither good nor bad.",-0.15,NEUTRAL,False,2023-05-19
a668404e-6524-453f-9587-94dd981224be,Great service!,0.86,POSITIVE,False,2023-05-22
d8935957-3be8-4823-8875-548bdb1e663d,Highly recommend this product!,0.75,POSITIVE,False,2023-06-04
23916b0c-f3aa-4c74-97b6-3cc9b8f47eab,Anyone interested?,,,True,2023-06-14
ea407d66-bd3b-4ca9-b0a8-2208e71c7593,Highly recommend this product!,0.5,POSITIVE,False,2023-06-19
8f1caa71-9914-40d9-b3c7-b9c4ea306e4e,"It's fine, neither good nor bad.",0.07,NEUTRAL,False,2023-07-17
b0c27eef-3a61-4985-b63c-30bb9bd1b0e0,This is awful.,-0.8,NEGATIVE,False,2023-07-30
4e7a6855-1bd6-4286-b031-a36d42ea6229,Great service!,0.72,POSITIVE,False,2023-08-11
f7c93fe9-7d53-49be-b19f-f0f274b4fc90,"It's fine, neither good nor bad.",0.08,NEUTRAL,False,2023-08-14
5bc30253-788e-4cbd-8ae1-8614c40be96d,Just checking in,,,True,2023-08-27
7a212bd4-bbe9-4374-b68b-f1682c24f1cd,This product is amazing!,0.87,POSITIVE,False,2023-09-11
f7047d8b-2242-420c-8eba-09032406878c,I hate this product.,-0.52,NEGATIVE,False,2023-09-21
```

*Figure 4.4 — Result of the head command*

The command you just ran shows the first 20 lines of the CSV file, so we can see the column headers and the first few rows. This helps us check the file's format and get a quick idea of the data it contains.

2. Install `parquet-tools` using the following:

```
pip install parquet-tools
```

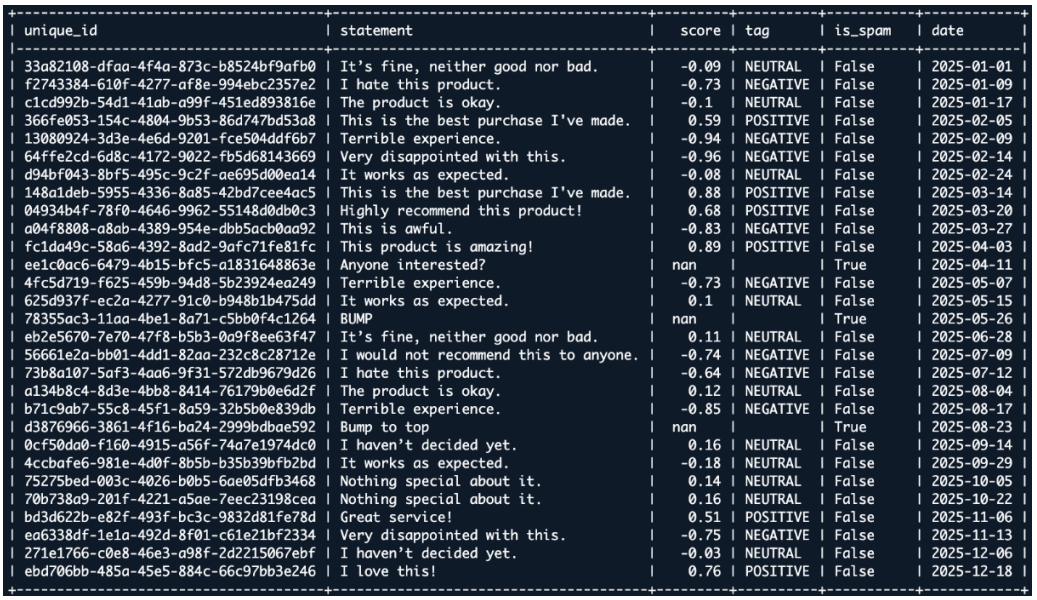
**parquet-tools** is a command-line utility that lets you inspect and interact with Parquet files. You can use it to view the schema, metadata, and contents of Parquet files, as well as perform basic file validation. This is especially helpful when you want to quickly check the structure or contents of large columnar datasets without loading them into Python.

> **Note**
>
> The following version is used in this chapter: `parquet-tools` 0.2.16. Versions close to this should also work without any issues.

3. Now, let's see what's stored inside the Parquet file with the `parquet-tools show` command:

```
parquet-tools show data_2025_onwards.parquet
```

This should yield the following:



| unique_id | statement | score | tag | is_spam | date |
|---|---|---|---|---|---|
| 33a82108-dfaa-4f4a-873c-b8524bf9afb0 | It's fine, neither good nor bad. | -0.09 | NEUTRAL | False | 2025-01-01 |
| f2743384-610f-4277-af8e-994ebc2357e2 | I hate this product. | -0.73 | NEGATIVE | False | 2025-01-09 |
| c1cd992b-54d1-41ab-a99f-451ed893816e | The product is okay. | -0.1 | NEUTRAL | False | 2025-01-17 |
| 366fe053-154c-4804-9b53-86d747bd53a8 | This is the best purchase I've made. | 0.59 | POSITIVE | False | 2025-02-05 |
| 13080924-3d3e-4e6d-9201-fce504ddf6b7 | Terrible experience. | -0.94 | NEGATIVE | False | 2025-02-09 |
| 64ffe2cd-6d8c-4172-9022-fb5d68143669 | Very disappointed with this. | -0.96 | NEGATIVE | False | 2025-02-14 |
| d94bf043-8bf5-495c-9c2f-ae695d00ea14 | It works as expected. | -0.08 | NEUTRAL | False | 2025-02-24 |
| 148a1deb-5955-4336-8a85-42bd7cee4ac5 | This is the best purchase I've made. | 0.88 | POSITIVE | False | 2025-03-14 |
| 04934b4f-78f0-4646-9962-55148d0db0c3 | Highly recommend this product! | 0.68 | POSITIVE | False | 2025-03-20 |
| a04f8808-a8ab-4389-954e-dbb5acb0aa92 | This is awful. | -0.83 | NEGATIVE | False | 2025-03-27 |
| fc1da49c-58a6-4392-8ad2-9afc71fe81fc | This product is amazing! | 0.89 | POSITIVE | False | 2025-04-03 |
| ee1c0ac6-6479-4b15-bfc5-a1831648863e | Anyone interested? | nan | | True | 2025-04-11 |
| 4fc5d719-f625-459b-94d8-5b23924ea249 | Terrible experience. | -0.73 | NEGATIVE | False | 2025-05-07 |
| 625d937f-ec2a-4277-91c0-b948b1b475dd | It works as expected. | 0.1 | NEUTRAL | False | 2025-05-15 |
| 78355ac3-11aa-4be1-8a71-c5bb0f4c1264 | BUMP | nan | | True | 2025-05-26 |
| eb2e5670-7e70-47f8-b5b3-0a9f8ee63f47 | It's fine, neither good nor bad. | 0.11 | NEUTRAL | False | 2025-06-28 |
| 56661e2a-bb01-4dd1-82aa-232c8c28712e | I would not recommend this to anyone. | -0.74 | NEGATIVE | False | 2025-07-09 |
| 73b8a107-5af3-4aa6-9f31-572db9679d26 | I hate this product. | -0.64 | NEGATIVE | False | 2025-07-12 |
| a134b8c4-8d3e-4bb8-8414-76179b0e6d2f | The product is okay. | 0.12 | NEUTRAL | False | 2025-08-04 |
| b71c9ab7-55c8-45f1-8a59-32b5b0e839db | Terrible experience. | -0.85 | NEGATIVE | False | 2025-08-17 |
| d3876966-3861-4f16-ba24-2999bdbae592 | Bump to top | nan | | True | 2025-08-23 |
| 0cf50da0-f160-4915-a56f-74a7e1974dc0 | I haven't decided yet. | 0.16 | NEUTRAL | False | 2025-09-14 |
| 4ccbafe6-981e-4d0f-8b5b-b35b39bfb2bd | It works as expected. | -0.18 | NEUTRAL | False | 2025-09-29 |
| 75275bed-003c-4026-b0b5-6ae05dfb3468 | Nothing special about it. | 0.14 | NEUTRAL | False | 2025-10-05 |
| 70b738a9-201f-4221-a5ae-7eec23198cea | Nothing special about it. | 0.16 | NEUTRAL | False | 2025-10-22 |
| bd3d622b-e82f-493f-bc3c-9832d81fe78d | Great service! | 0.51 | POSITIVE | False | 2025-11-06 |
| ea6338df-1e1a-492d-8f01-c61e21bf2334 | Very disappointed with this. | -0.75 | NEGATIVE | False | 2025-11-13 |
| 271e1766-c0e8-46e3-a98f-2d2215067ebf | I haven't decided yet. | -0.03 | NEUTRAL | False | 2025-12-06 |
| ebd706bb-485a-45e5-884c-66c97bb3e246 | I love this! | 0.76 | POSITIVE | False | 2025-12-18 |

*Figure 4.5 — Result after running the parquet-tools show command*

By running this command, we can see the file's schema along with a sample of its data. This helps us quickly understand what kind of data it contains.

4. Next, inspect the Parquet file using the `parquet-tools inspect` command:

```
parquet-tools inspect data_2025_onwards.parquet
```

This should give us the following:

```
...
############ Column(statement) ############
name: statement
path: statement
max_definition_level: 1
max_repetition_level: 0
```

```
    physical_type: BYTE_ARRAY
    logical_type: String
    converted_type (legacy): UTF8
    compression: SNAPPY (space_saved: 42%)
    ...
```

You can use `parquet-tools inspect` to examine the detailed schema and metadata of a Parquet file. You can see each column's name, type, nesting, and compression, which helps verify the structure and storage efficiency.

**Note**

As you probably know by now, CSV files are simple, human-readable text files that make them ideal for raw data ingestion, quick exports from external systems, and lightweight data sharing between teams. However, because they store data row by row without built-in schema enforcement or compression, they are less efficient for large-scale analytics workloads. Parquet files, on the other hand, use a columnar storage format that significantly improves query performance and reduces storage costs by compressing and scanning only the required columns. For this reason, CSV is commonly used at the raw ingestion stage of a data lake, while Parquet is preferred for structured, analytics-ready, and ML datasets within a transactional data lake architecture.

Although this example is simplified, it shows how we can inspect CSV and Parquet files in practice. Using commands such as `head` for CSV and `parquet-tools show` or `parquet-tools inspect` for Parquet lets us quickly check the structure, schema, and sample data before working with larger datasets. Keep in mind that inspecting the files is important to catch formatting issues, missing values, or unexpected data types early, which helps prevent errors later in analysis or processing.

Now that we've inspected the files, the next step is to upload them to an S3 bucket for storage and further processing.

## Uploading the CSV and Parquet files to an S3 bucket

Once the files are exported, the next step is to upload them to an S3 bucket. S3 serves as a durable and scalable storage layer that other services can easily access for analytics, processing, or ML tasks.

Let's upload the CSV and Parquet files to an S3 general-purpose bucket:

1. Create a new bucket where we will store a copy of the CSV file and Parquet file. Make sure to replace `<S3 BUCKET NAME>` with a bucket name (for a new S3 bucket) before running the commands:

```
S3_BUCKET_NAME="<S3 BUCKET NAME>"
aws s3 mb s3://$S3_BUCKET_NAME
```

2. Once the S3 bucket has been created, run the following commands to upload the CSV file as well as the Parquet file to the new S3 bucket:

```
CSV_FILE="data_up_to_2024.csv"
PARQUET_FILE="data_2025_onwards.parquet"
aws s3 cp $CSV_FILE s3://$S3_BUCKET_NAME/
aws s3 cp $PARQUET_FILE s3://$S3_BUCKET_NAME/
```

**Note**

At this point, both the CSV and Parquet datasets have been successfully uploaded to the new S3 general-purpose bucket. Make sure to note the bucket name, as you'll need it later in *Performing Apache Iceberg queries on S3 tables with Apache Spark*. You should store the bucket name (`TAG:DATASET_BUCKET`) in a text file using a code editor on your local machine so you can easily reference it in later steps.

In this section, we downloaded and prepared two synthetic tabular datasets—one exported as a CSV file and the other as a Parquet file—and uploaded these files to an S3 bucket. These steps ensure that our data is ready and accessible for use in later sections of this chapter. With this, we can now proceed with setting up the S3 table bucket in the next section.

# Creating an Amazon S3 table bucket

Managing large datasets on AWS can be challenging, especially for analytics, ML, and other data-intensive workloads. While it is possible to store datasets in general-purpose buckets in Amazon S3, these buckets operate at the object level, meaning that updates typically require rewriting entire files rather than modifying individual records. As datasets grow, this approach becomes increasingly inefficient and costly, particularly for workloads that require frequent updates or schema changes. For example, when an engineer needs to update a dataset with new columns, they must download the entire dataset, make the necessary changes, and then

upload the modified file back to S3, which is a time-consuming and generally impractical process.

**Amazon S3 Tables**, a cloud object store with built-in Iceberg support, simplifies data management by providing transactional capabilities such as ACID operations, schema evolution, and time travel, eliminating the need for full data rewrites. With S3 Tables, you can efficiently update and query your datasets, track changes over time, and maintain consistent historical versions, all without manually managing the underlying infrastructure. This makes it ideal for analytics and ML workloads that require frequent updates or evolving schemas.

In this section, I'll guide you through the creation of an Amazon S3 table bucket. You'll first enable integration with AWS analytics services, and then create a dedicated table bucket that will act as the storage foundation for your transactional data lake. You'll use this table bucket to store, manage, and query a sample dataset throughout the rest of the chapter.

With this in mind, let's walk through the process of creating a new table bucket:

1. In the AWS Management Console, navigate to the S3 console by typing `s3` in the search bar and then choosing **Amazon S3** from the list of search results.
2. In the navigation pane (left sidebar), click **Table buckets** under **Buckets**. This should open a page that shows all existing S3 table buckets in the selected region.
3. Locate and click on the **Enable integration** button (under **Integration with AWS analytics services**).
4. On the **Table buckets | Enable integration** page, click **Enable integration**:

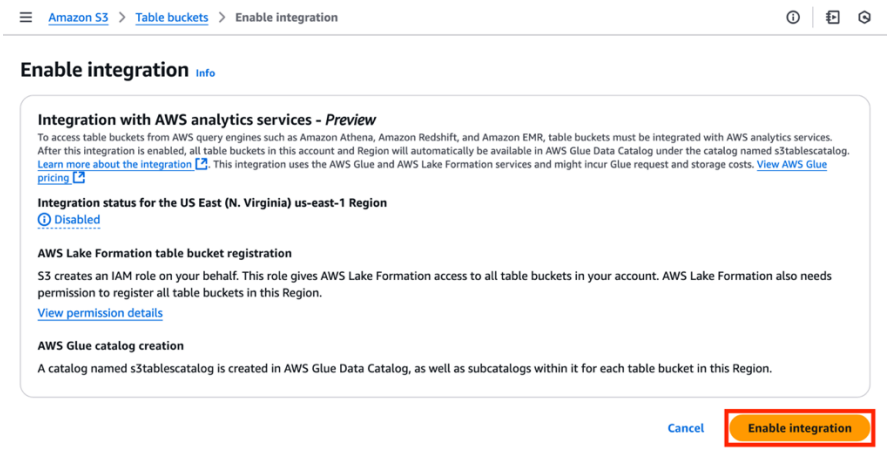



*Figure 4.6 — Enabling the integration with AWS analytics services*

This will enable your table(s) to be accessed by AWS analytics services, including Amazon Athena, Amazon Redshift, Amazon EMR, Quick Suite, and Amazon Data Firehose.

> **Note**
>
> For more information on enabling integration with AWS analytics services, refer to the following link: `https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables-integration-overview.html`.

5. On the **Table buckets** page, click the **Create table bucket** button.
6. On the **Create table bucket** page, specify a table bucket name in the text field highlighted in *Figure 4.7*.

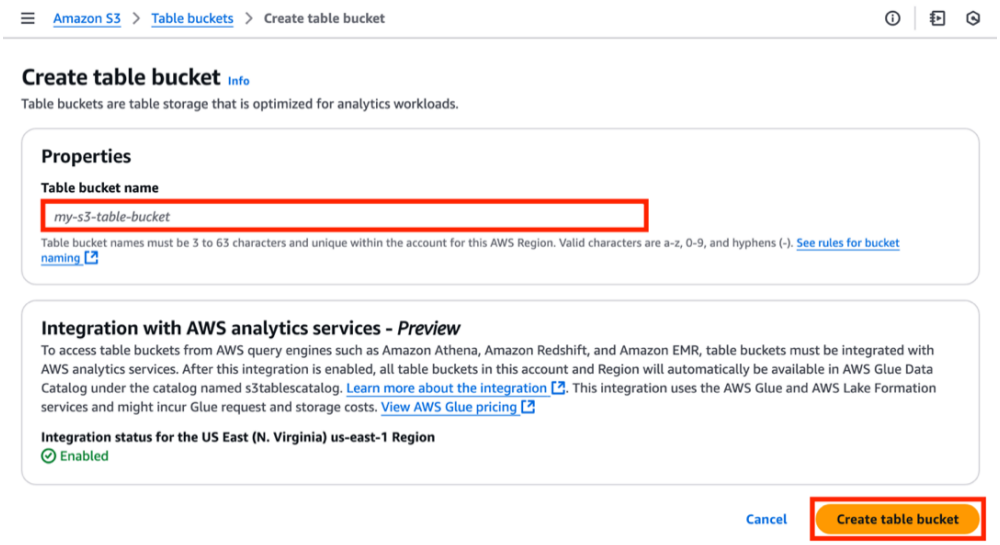



*Figure 4.7 — Creating a table bucket*

The table bucket name must be 3 to 63 characters long, use only lowercase letters, numbers, and hyphens (–), and start and end with a letter or number. It must not contain underscores (_) or periods (.), must not start with reserved prefixes, and must not end with reserved suffixes. Table bucket names only need to be unique within your current AWS Region.

> **Note**
>
> For more details on naming rules for Amazon S3 table buckets, tables, and namespaces, refer to the official documentation: `https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables-buckets-naming.html`.

7. Click on the **Create table bucket** button to proceed with the creation of the resource.

> **Note**
>
> Be sure to take note of the table bucket name, as you'll need it later in this chapter when we copy the data stored in the CSV and Parquet files to the table bucket. You should store the table bucket name (`TAG:TABLE_BUCKET_NAME`) in a text file using a code editor on your local machine, so you can easily reference it in later steps.

That's it! Your S3 table bucket is set up. Wasn't that easy? In this section, you enabled integration between S3 tables and AWS analytics services and set up an S3 table bucket. Inside this table bucket, you can create tables and store, modify, and query your data.

In the next section, you'll launch an Amazon EMR cluster with Apache Iceberg installed, which will allow you to upload the data to the table and perform analytical queries.

## Launching an Amazon EMR cluster with Apache Iceberg installed

In the previous section, you created an S3 table bucket, but it is still empty and doesn't contain any tables or data yet. To start adding tables and data, you can use an Amazon EMR cluster with Apache Iceberg installed. If you're unfamiliar with **Amazon EMR**, it is a fully managed big data platform from AWS that allows you to efficiently process and analyze large datasets at scale using a variety of open source frameworks. You can think of an Amazon EMR cluster as a set of interconnected computers that work together to run complex data processing and analytics tasks, while the S3 table bucket serves as the storage that holds your data and results.

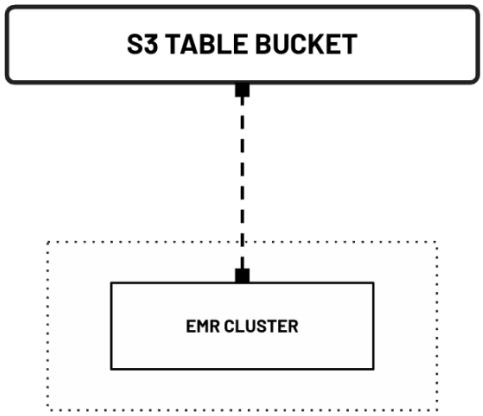



*Figure 4.8 — An EMR cluster writing data to an S3 table bucket*

In this section, you'll launch an EMR cluster with Iceberg installed, and update the EC2 instance profile for one of the cluster's instances to add the `AmazonS3TablesFullAccess` policy. After setting everything up, you'll then connect to the cluster via SSH. In the later sections of this chapter, you'll work with the S3 table bucket directly from within the cluster to load data, create tables, and run queries.

To prepare for the hands-on portion, it's essential to familiarize yourself with a few key services, terms, and concepts you'll work with throughout this section:

- **Amazon Elastic Compute Cloud (EC2)**: A scalable, on-demand compute service from AWS that allows you to run virtual machines (EC2 instances) in the cloud. EC2 provides flexible, cost-effective compute capacity and allows you to quickly scale up or down based on your needs.
- **EC2 key pair**: A pair of cryptographic keys used to securely connect to your EC2 instances. The public key is placed on the instance, while you keep the private key, which is used for authentication when connecting to the instance from a client machine.
- **EC2 instance profile**: An IAM role assigned to an EC2 instance. Depending on the permission policies attached to the role, it grants the instance permissions to interact with other AWS services, such as reading from Amazon S3, writing to DynamoDB, or performing actions on other AWS resources.

- **Amazon EMR cluster**: A collection of EC2 instances managed by Amazon EMR to run big data processing workloads. The cluster typically consists of one primary node, one or more core nodes for data storage and processing, and optionally, task nodes to handle additional processing tasks.

Familiarity with these services and terms will simplify the process of setting up our EMR cluster in this section, which includes installing Apache Iceberg on the cluster, updating the EC2 instance profile permissions, and connecting to the EMR cluster.

## Setting up an Amazon EMR cluster with Apache Iceberg installed

Provisioning an Amazon EMR cluster follows a similar process to configuring a virtual machine or server, where you choose the instance type, configure security, and install required software. In this section, I'll guide you through the process of launching an EMR cluster with Apache Iceberg installed.

> **Note**
>
> Ensure that each step in the cluster configuration process is completed carefully and that all required settings are properly reviewed before launching the cluster. Taking a few moments to verify the configuration of your EMR cluster helps avoid unnecessary troubleshooting and additional provisioning costs.

Let's go through the steps in detail:

1. Navigate to the EMR console through the search bar by typing `emr` and then choosing **EMR** from the list of search results.
2. Click the **Create cluster** button to start the cluster creation and configuration steps.
3. Under **Name and applications**, specify a cluster name (`cluster-for-s3-table-buckets`):

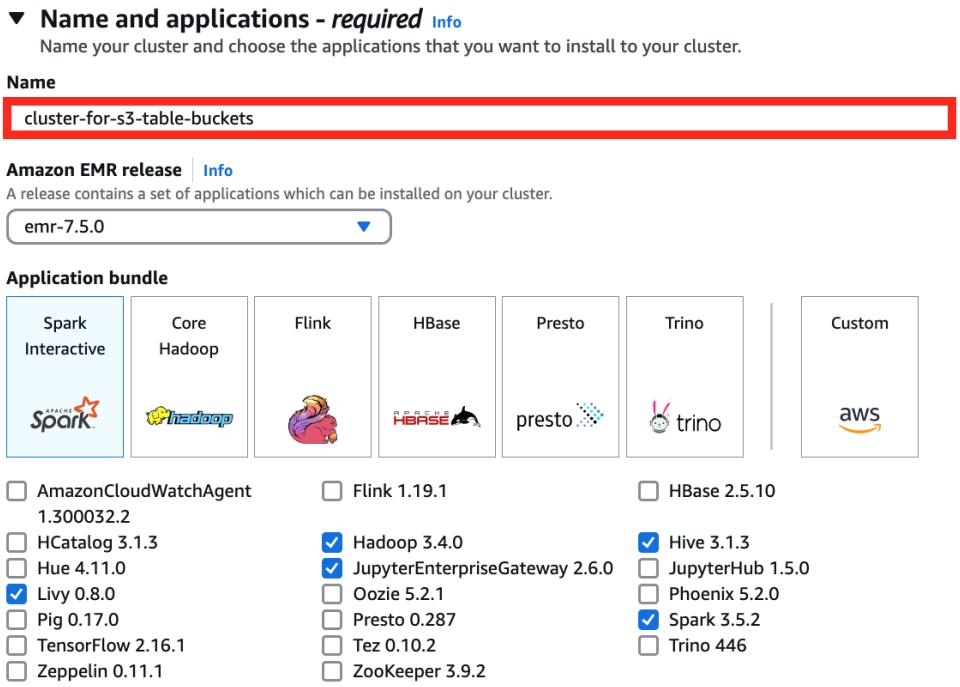



*Figure 4.9 — Configuring the EMR cluster name as well as the installed applications*

Before proceeding to the next steps, set the **Amazon EMR release** drop-down value to **emr-7.5.0**. Under **Application bundle**, choose **Spark Interactive**.

4. Under **Cluster configuration – required**, choose **Flexible instance fleets**. Set the **Choose EC2 instance type** drop-down values for both the **Primary** and **Core** nodes to **m5.xlarge**.

> **Note**
>
> Ensure that the **Use high availability** checkbox is unchecked.

5. For the **Task** node(s), click the **Remove instance fleet** button since it's not required for this setup.
6. Under **Networking**, choose the VPC with two public subnets that you prepared at the beginning of this chapter (*Technical requirements*). Then, select one of the subnets available under **Subnets**.

7. Under **Cluster termination and node replacement**, select **Automatically terminate cluster after idle time (Recommended)** for **Termination option**. Specify the **Idle time** field values to `0 days` and `04:00:00`, respectively. This means the cluster will automatically terminate after being idle for 4 hours.

> **Note**
>
> Make sure that the **Use termination protection** checkbox is unchecked

8. Under **Software settings**, select **Enter configuration**. Specify the following configuration in the text area:

```
[
  {
    "Classification":"iceberg-defaults",
    "Properties":{"iceberg.enabled":"true"}
  }
]
```

9. In the **Security configuration and EC2 key pair** section, click **Create key pair**, which will open a new tab for creating a new key pair. When creating the new key pair, specify a name for the key pair (`emr-cluster-key-pair`). For **Key pair type**, select **RSA**. For **Private key file format**, choose **.pem**. Click **Create key pair** to complete the key pair creation process and download the `.pem` file to your local machine.

10. Once you have created the new key pair, switch back to the browser tab where the **Create cluster** page is open. Make sure that the key pair (`emr-cluster-key-pair`) you just created is the EC2 key pair specified under **Amazon EC2 key pair for SSH to the cluster**.

> **Note**
>
> If you do not see the `emr-cluster-key-pair` you just created in the **Choose Amazon EC2 key pair for SSH to cluster** pop-up window, click the refresh button within the popup to reload the available key pairs. This ensures that the newly created key pair appears in the list and can be selected.

11. In the **Identity and Access Management (IAM) roles** section, under **Amazon EMR service role**, select **Create a service role**.

12. Under **EC2 instance profile for Amazon EMR**, select **Create an instance profile**. Under **S3 bucket access**, select **All S3 buckets in this account with read and write access**.

> **Note**
>
> Selecting **All S3 buckets in this account with read and write access** grants the EMR cluster overly permissive permissions to every Amazon S3 bucket in your AWS account. In production environments, you should restrict access to only the specific buckets and actions required by your Amazon EMR workloads.

13. Review the configuration details in the **Summary** pane (located on the right side of the page). Your EMR cluster should be configured as follows:
    - **Name**: `cluster-for-s3-table-buckets`
    - **Amazon EMR release**: **emr-7.5.0**
    - **Application bundle**: **Spark Interactive** (Hadoop 3.4.0, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, Livy 0.8.0, Spark 3.5....)
    - **Cluster configuration** | **Flexible instance fleets**: **Primary** (**m5.xlarge**), **Core** (**m5.xlarge**)
    - **Networking**: The VPC with two public subnets that you prepared at the start of this chapter (*Technical requirements*)
    - **Cluster termination**: **Automatically terminate cluster after idle time** | **Idle time**: 4 hours
    - **Software settings** | **Configuration**: `classification=iceberg-defaults`...
    - **Security configuration and EC2 key pair** | **Amazon EC2 key pair**: `emr-cluster-key-pair.pem`
    - **Identity and Access Management (IAM) roles** | **Service role**: New service role
    - **Identity and Access Management (IAM) roles** | **Instance profile**: New instance profile
14. Click the **Create cluster** button to start the actual creation and provisioning of the EMR cluster.

> **Note**
>
> You should see the following success notification: **Your cluster "cluster-for-s3-table-buckets" has been successfully created.**

At this stage, your EMR cluster is being provisioned and should be available for use shortly. Note that your cluster has been configured with broad S3 bucket access. Granting read and write access to all S3 buckets in the account is overly permissive and should only be used in a development or testing environment. In a production setting, it's important to restrict access to specific S3 buckets to minimize security risks. Since we are just working on an example in this chapter, this setup should be sufficient for now.

## Updating the EC2 instance profile permissions

To ensure that your Amazon EMR cluster can interact seamlessly with the necessary S3 buckets, it's important to update the EC2 instance profile permissions. Specifically, you'll need to attach the `AmazonS3TablesFullAccess` policy to the EC2 instance profile associated with one of the instances in the cluster. This policy grants the necessary permissions for the EMR cluster instances to read from and write to S3, which is essential for managing Apache Iceberg tables and performing other data operations within the cluster.

Let's now go through the process of configuring the EC2 instance profile with the required permissions:

1. On the **cluster details** page in the AWS Management Console, scroll down and locate the **Properties** tab. Under **Properties**, you should find the following sections: **Operating system**, **Cluster logs**, **Cluster termination and node replacement**, and **Network and security**.
2. Under **Network and security | Permissions**, locate and copy the EC2 instance profile of the EMR cluster. You can store this in a text file using a code editor on your local machine for easy reference.

   **Note**

   The EC2 instance profile typically starts with `AmazonEMR-InstanceProfile`.

3. Open the AWS Management Console in a new browser tab. Navigate to the **IAM** console through the search bar by typing `iam` and then choosing **IAM** from the list of search results.
4. In the **Search IAM** field in the navigation pane (left sidebar), paste the copied EC2 instance profile. Select the result that exactly matches the name of the EC2 instance profile.
5. Click **Add permissions**, then select **Attach policies** from the drop-down menu. Under **Other permission policies**, locate and select the `AmazonS3TablesFullAccess` permission policy. Click **Add permissions** afterward.

At this point, the EC2 instance profile has been updated with the necessary permissions, and your cluster is now fully configured to access and interact with Amazon S3 tables directly from within its nodes.

## Connecting to the EMR cluster

With the cluster configuration complete, you're now ready to connect to it via SSH. Each node in your EMR cluster is essentially an EC2 instance, so connecting to a node is similar to connecting to any EC2 instance via SSH, using the appropriate key pair and public IP address of the node.

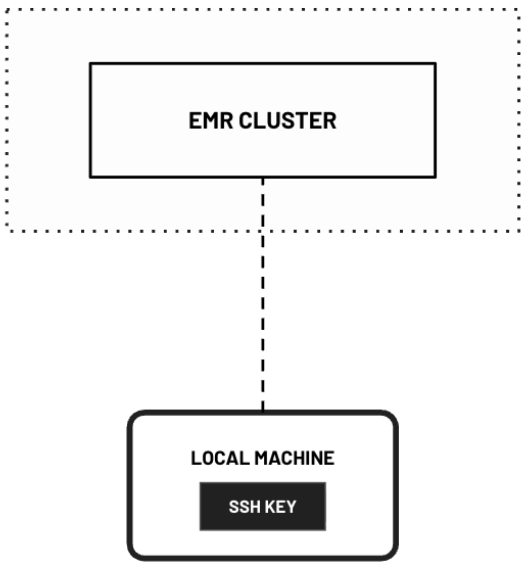



*Figure 4.10 — Connecting to the EMR cluster*

Before we can connect to the cluster, we need to add an inbound rule that permits SSH connections from our local machine's IP address. Once this rule is in place, we can proceed to connect to the cluster securely using our key pair.

**Note**

The commands in this section assume you are using a macOS or Linux terminal. Windows users can run the same commands using WSL, Git Bash, or a Linux virtual machine.

Now, let's walk through the steps to securely connect to your EMR cluster:

1. Switch back to the browser tab where the EMR cluster details page for **cluster-for-s3-table-buckets** is open.
2. Scroll down and locate the **Properties** tab. Locate the **Network and security** pane and expand **EC2 security groups (firewall)**. Click the link under **Primary node EMR managed security group**, which should open a new tab showing the security group configuration (**Details**, **Inbound rules**, **Outbound rules**, **Sharing**, **VPC associations**, and **Tags**).
3. Let's add an inbound rule to allow SSH connections from the IP address of the laptop or desktop machine you are using. Under **Inbound rules**, click **Edit inbound rules**. Click **Add rule**. Choose **SSH** from the list of drop-down options (under **Type**) and then select **My IP** (under **Source**). Click **Save rules**.

   **Note**

   Adding an inbound SSH rule to your Amazon EC2 security group allows you to establish a **secure shell** (**SSH**) connection to the instance from your current laptop or desktop machine. This configuration enables you to remotely access and manage the cluster node for administrative or troubleshooting tasks. For security reasons, the rule is restricted to your IP address only. In production environments, you should avoid opening SSH access to broad IP ranges and limit access to trusted hosts only.

4. Switch back to the browser tab where the EMR cluster details page for **cluster-for-s3-table-buckets** is open. Under **Summary** | **Cluster management** | **Primary node public DNS**, take note and copy the address of the primary node (`https://ec2-XXX-XXX-XXX-XXX.compute-1.amazonaws.com/`). You may paste and save the primary node public DNS address in a local text file for easy reference in the subsequent steps of this chapter. This helps you avoid repeatedly navigating back to the cluster details page.

   **Note**

   You may click **Connect to the Primary node using SSH** (under **Cluster management** | **Primary node public DNS**) to view the SSH command as well

> as the address of the primary node. Keep in mind that the SSH command may differ slightly, depending on the location of the `.pem` file on your local machine.

5. In the terminal of your local machine, navigate to the directory where the `.pem` file was downloaded. Then, run the following commands to connect to the EMR cluster. Make sure to replace `<EMR NODE>` with the address of the EMR cluster primary node before running the commands:

```
KEY_FILE="emr-cluster-key-pair.pem"
chmod 400 $KEY_FILE
ssh -i $KEY_FILE hadoop@<EMR NODE>
```

Once connected, you should see the following:

```
A newer release of "Amazon Linux" is available.
  Version 2023.6.20241212:
Run "/usr/bin/dnf check-release-update" for full release and version update info
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Dec 18 18:56:11 2024 from 180.190.37.249

EEEEEEEEEEEEEEEEEEE MMMMMMMM           MMMMMMMM RRRRRRRRRRRRRRR
E::::::::::::::::::E M::::::M         M::::::M R::::::::::::::R
EE:::::EEEEEEEEE:::E M:::::::M       M:::::::M R:::::RRRRRR:::::R
  E::::E       EEEEE M::::::::M     M::::::::M RR::::R      R::::R
  E::::E             M::::::M::M   M::M::::::M   R::R      R::::R
  E:::::EEEEEEEEEE   M::::::M M::M M::M M::::::M   R::RRRRRR:::::R
  E::::::::::::::E   M::::::M  M::M::M  M::::::M   R:::::::::::RR
  E:::::EEEEEEEEEE   M::::::M   M:::::M   M::::::M   R::RRRRRR::::R
  E::::E             M::::::M    M:::M    M::::::M   R::::R      R::::R
  E::::E       EEEEE M::::::M     MMM     M::::::M   R::::R      R::::R
EE:::::EEEEEEEE:::E M::::::M             M::::::M   R::::R      R::::R
E::::::::::::::::::E M::::::M             M::::::M RR::::R      R::::R
EEEEEEEEEEEEEEEEEEE MMMMMMMM             MMMMMMMM RRRRRRR      RRRRRR

[hadoop@ip-172-31-3-176 ~]$
```

*Figure 4.11 — Successful SSH connection to the EMR cluster primary node*

After running the SSH command, you should see a terminal prompt indicating a successful connection to the EMR cluster's primary node, typically displaying the

hostname and user (`hadoop@<EMR NODE>`), followed by a command prompt where you can start executing commands within the cluster.

With a successful SSH connection to the primary node, you're now ready to begin working with your EMR cluster and interacting with the S3 table bucket. In the next section, I'll show you how to use Apache Spark to run queries on fully-managed Iceberg tables stored in your S3 table bucket.

# Performing Apache Iceberg queries on S3 tables with Apache Spark

In this section, we will explore how to perform queries on our Iceberg-backed S3 table bucket using Apache Spark. Spark is the engine that runs your queries across the nodes of your EMR cluster, using Iceberg's metadata and the data stored in S3 to efficiently process only the relevant partitions and filter out unnecessary rows, making queries faster and more scalable.

To guide you through the steps of running queries and working with Iceberg tables on S3, this section is divided as follows:

- Accessing the Spark shell
- Creating a namespace
- Loading data from a CSV file to a table
- Loading data from a Parquet file to a table

By the end of this section, you will have a good understanding of how to query S3 tables using Spark from within an EMR cluster.

## Accessing the Spark shell

Let's continue where we left off in the previous section and run the following commands from within your EMR cluster to access the Spark shell:

1. Run the following to get the account ID of the AWS account:

```
ACCOUNT_ID=`aws sts get-caller-identity --query "Account" --output text`
```

2. Specify the name of the S3 table bucket:

```
TABLE_BUCKET="<SPECIFY S3 TABLE BUCKET NAME>"
```

Make sure to replace `<SPECIFY S3 TABLE BUCKET NAME>` with the name of the S3 table bucket you created in *Creating an Amazon S3 table bucket* before running the command.

> **Note**
>
> You can use the S3 table bucket name (`TAG:TABLE_BUCKET_NAME`) from your local code editor if you saved it there earlier. You can also retrieve the table bucket name from the Amazon S3 console.

3. Prepare the configuration settings by running the following:

```
PACKAGES=software.amazon.s3tables:s3-tables-catalog-for-iceberg-runtime:
0.1.3
CONF1=spark.sql.catalog.s3tablesbucket=org.apache.iceberg.spark.SparkCatalog
CONF2=spark.sql.catalog.s3tablesbucket.catalog-
impl=software.amazon.s3tables.iceberg.S3TablesCatalog
CONF3=spark.sql.catalog.s3tablesbucket.warehouse=arn:aws:s3tables:us-
east-1:$ACCOUNT_ID:bucket/$TABLE_BUCKET
CONF4=spark.sql.extensions=org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions
```

4. With everything ready, run the following command to initialize PySpark with the specified Iceberg catalog and configuration:

```
pyspark \
--packages $PACKAGES \
--conf $CONF1 \
--conf $CONF2 \
--conf $CONF3 \
--conf $CONF4 \
--conf spark.ui.showConsoleProgress=false \
--conf spark.log.level=ERROR
```

At this point, you should have the PySpark REPL before proceeding to the next part.

## Creating a namespace

Now, let's create a namespace, which serves as a database-like grouping to help structure and manage tables in your data lake:

1. Define a function for running SQL statements using PySpark:

```
def SQL(statement):
    spark.sql(statement).explain(True)
    spark.sql(statement).show(40, truncate=False)
```

2. Specify the `namespace` name:

```
namespace = "sentiments_namespace"
```

3. Create the namespace if it does not exist:

```
SQL(f"""
CREATE NAMESPACE IF NOT EXISTS s3tablesbucket.{namespace}
""")
```

4. Verify that the namespace created in the previous step exists:

```
SQL("SHOW NAMESPACES IN s3tablesbucket")
```

   This should return `sentiments_namespace`.

With the namespace ready, we can now proceed with loading data from a CSV file to a table (within this namespace) in the next part.

## Loading data from a CSV file to a table

Let's proceed with loading data from the CSV file (`data_up_to_2024.csv`) we prepared in the *Preparing and processing the synthetic data* section:

1. Specify the name of the S3 general-purpose bucket (`TAG:DATASET_BUCKET`) where the CSV and Parquet files have been uploaded (earlier in *Preparing and processing the synthetic data*):

```
s3_bucket = "<S3 BUCKET NAME>"
```

> **Note**
>
> You can use the S3 bucket name (`TAG:DATASET_BUCKET`) from your local code editor if you saved it there earlier. You can also retrieve the bucket name from the Amazon S3 console. Make sure the bucket contains the uploaded CSV (`data_up_to_2024.csv`) and Parquet (`data_2025_onwards.parquet`) files.

2. Run the following code to load the CSV file into a DataFrame:

```
df_up_to_2024 = spark.read.csv(
    f"s3://{s3_bucket}/data_up_to_2024.csv",
```

```
    header=True,
    inferSchema=True
)
```

3. Let's quickly check how the first 20 rows of our DataFrame look:

```
df_up_to_2024.show(20)
```

This should give us the following table of values:

```
+--------------------+--------------------+-----+--------+-------+----------+
|           unique_id|           statement|score|     tag|is_spam|      date|
+--------------------+--------------------+-----+--------+-------+----------+
|35ce3303-f520-4db...|        I love this!|  0.6|POSITIVE|  false|2023-01-14|
|0e84ff9b-2845-48b...|The product is okay.|-0.08| NEUTRAL|  false|2023-02-04|
|d36d5a11-138d-49a...|It works as expec...| -0.1| NEUTRAL|  false|2023-02-15|
|05fde437-4630-43a...|      This is awful.|-0.89|NEGATIVE|  false|2023-03-09|
|bd2aabd2-e388-461...|This is the best ...| 0.97|POSITIVE|  false|2023-03-17|
|9c47757f-35be-420...|The product is okay.|-0.01| NEUTRAL|  false|2023-04-13|
|e0bfe69b-ff8a-43f...|I would not recom...|-0.72|NEGATIVE|  false|2023-04-21|
|b647c226-acb1-4d1...|It's fine, neithe...|-0.15| NEUTRAL|  false|2023-05-19|
|a668404e-6524-453...|      Great service!| 0.86|POSITIVE|  false|2023-05-22|
|d8935957-3be8-482...|Highly recommend ...| 0.75|POSITIVE|  false|2023-06-04|
|23916b0c-f3aa-4c7...|  Anyone interested?| NULL|    NULL|   true|2023-06-14|
|ea407d66-bd3b-4ca...|Highly recommend ...|  0.5|POSITIVE|  false|2023-06-19|
|8f1caa71-9914-40d...|It's fine, neithe...| 0.07| NEUTRAL|  false|2023-07-17|
|b0c27eef-3a61-498...|      This is awful.| -0.8|NEGATIVE|  false|2023-07-30|
|4e7a6855-1bd6-428...|      Great service!| 0.72|POSITIVE|  false|2023-08-11|
|f7c93fe9-7d53-49b...|It's fine, neithe...| 0.08| NEUTRAL|  false|2023-08-14|
|5bc30253-788e-4cb...|    Just checking in| NULL|    NULL|   true|2023-08-27|
|7a212bd4-bbe9-437...|This product is a...| 0.87|POSITIVE|  false|2023-09-11|
|f7047d8b-2242-420...|I hate this product.|-0.52|NEGATIVE|  false|2023-09-21|
|3f71dbf2-97f4-4e2...|Nothing special a...|-0.11| NEUTRAL|  false|2023-09-25|
+--------------------+--------------------+-----+--------+-------+----------+
only showing top 20 rows
```

*Figure 4.12 — First 20 rows of our DataFrame*

Here, the DataFrame contains the `unique_id`, `statement`, `score`, `tag`, `is_spam`, and `date` columns, which represent the identifier, original text content, sentiment score, classification tag, spam indicator flag, and timestamp associated with each record.

4. Let's specify the table name and store it in the `table_name` variable:

```
name="sentiments_v1"
table_name = f"s3tablesbucket.{namespace}.{name}"
```

5. Now, let's create the table using the following `CREATE TABLE` statement:

```
SQL(f"""
CREATE OR REPLACE TABLE {table_name}(
```

```
  unique_id STRING,
  statement STRING,
  score DOUBLE,
  tag STRING,
  is_spam BOOLEAN,
  date DATE
)
USING iceberg
TBLPROPERTIES ('format-version'='2')
""")
```

Here, we are creating a table with the same set of columns as the DataFrame to be stored.

6. Save the `df_up_to_2024` DataFrame of values into the table:

```
df_up_to_2024.writeTo(table_name) \
    .using("iceberg") \
    .tableProperty("format-version","2") \
    .append()
```

7. Run the following `SELECT` statement to verify that the data has been successfully appended to our table:

```
SQL(f"SELECT * FROM { table_name }")
```

This should give us the following:

| unique_id | statement | score | tag | is_spam | date |
|---|---|---|---|---|---|
| 35ce3303-f520-4dbe-9759-ef87f261c3f3 | I love this! | 0.6 | POSITIVE | false | 2023-01-14 |
| 0e84ff9b-2845-48b7-9674-d4da4f3f3fee | The product is okay. | -0.08 | NEUTRAL | false | 2023-02-04 |
| d36d5a11-138d-49a5-bcb8-acd2a889493a | It works as expected. | -0.1 | NEUTRAL | false | 2023-02-15 |
| 05fde437-4630-43ac-821a-77d86ead113a | This is awful. | -0.89 | NEGATIVE | false | 2023-03-09 |
| bd2aabd2-e388-4616-acf9-90358091e76b | This is the best purchase I've made. | 0.97 | POSITIVE | false | 2023-03-17 |
| 9c47757f-35be-420e-92bd-6c7eebd5a2b1 | The product is okay. | -0.01 | NEUTRAL | false | 2023-04-13 |
| e0bfe69b-ff8a-43f5-babc-8e9e857d3c6a | I would not recommend this to anyone. | -0.72 | NEGATIVE | false | 2023-04-21 |
| b647c226-acb1-4d1e-ae43-25a5bcc08aa9 | It's fine, neither good nor bad. | -0.15 | NEUTRAL | false | 2023-05-19 |
| a668404e-6524-453f-9587-94dd981224be | Great service! | 0.86 | POSITIVE | false | 2023-05-22 |
| d8935957-3be8-4823-8875-548bdb1e663d | Highly recommend this product! | 0.75 | POSITIVE | false | 2023-06-04 |
| 23916b0c-f3aa-4c74-97b6-3cc9b8f47eab | Anyone interested? | NULL | NULL | true | 2023-06-14 |
| ea407d66-bd3b-4ca9-b0a8-2208e71c7593 | Highly recommend this product! | 0.5 | POSITIVE | false | 2023-06-19 |
| 8f1caa71-9914-40d9-b3c7-b9c4ea306e4e | It's fine, neither good nor bad. | 0.07 | NEUTRAL | false | 2023-07-17 |
| b0c27eef-3a61-4985-b63c-30bb9bd1b0e0 | This is awful. | -0.8 | NEGATIVE | false | 2023-07-30 |
| 4e7a6855-1bd6-4286-b031-a36d42ea6229 | Great service! | 0.72 | POSITIVE | false | 2023-08-11 |
| f7c93fe9-7d53-49be-b19f-f0f274b4fc90 | It's fine, neither good nor bad. | 0.08 | NEUTRAL | false | 2023-08-14 |
| 5bc30253-788e-4cbd-8ae1-8614c40be96d | Just checking in | NULL | NULL | true | 2023-08-27 |
| 7a212bd4-bbe9-4374-b68b-f1682c24f1cd | This product is amazing! | 0.87 | POSITIVE | false | 2023-09-11 |
| f7047d8b-2242-420c-8eba-09032406878c | I hate this product. | -0.52 | NEGATIVE | false | 2023-09-21 |
| 3f71dbf2-97f4-4e28-85a4-b2be09cbd2b4 | Nothing special about it. | -0.11 | NEUTRAL | false | 2023-09-25 |
| ea3fb396-d583-422e-8c29-2a58ff7a581c | Nothing special about it. | 0.05 | NEUTRAL | false | 2023-10-01 |
| 1800ed58-4f77-4af9-948a-d925258f1165 | This is the best purchase I've made. | 0.87 | POSITIVE | false | 2023-10-08 |
| 38be50a2-b3b3-4d87-9d29-2252787e2051 | This product is amazing! | 0.76 | POSITIVE | false | 2023-10-17 |
| 666eb59c-b3f3-4629-afe9-42a4efaca715 | I hate this product. | -0.52 | NEGATIVE | false | 2023-10-24 |
| fec7045f-b55e-459d-bc6a-5cb1b7c0cfbd | Great service! | 0.93 | POSITIVE | false | 2023-11-10 |
| a43d8141-11ba-4324-994b-084c1a9c3634 | Bump to top | NULL | NULL | true | 2023-11-16 |
| 2b2bb5d2-b289-4650-ace9-7f0dc765db98 | I love this! | 0.73 | POSITIVE | false | 2023-11-21 |
| 13be5b71-8018-49c9-9958-780e1179f031 | This product is amazing! | 0.8 | POSITIVE | false | 2023-12-02 |
| cac2e72f-7ae9-49d6-bc12-0bd38847b805 | Terrible experience. | -0.59 | NEGATIVE | false | 2023-12-15 |
| c22491e7-c701-4a21-84a9-a4a942d781ff | I hate this product. | -0.88 | NEGATIVE | false | 2023-12-30 |
| a32d0194-ff3b-4f26-a2c3-6f2e0d2f13ac | It's fine, neither good nor bad. | -0.06 | NEUTRAL | false | 2024-01-02 |
| c7e11b11-6b09-421b-b675-baeccd5c7d06 | Anyone interested? | NULL | NULL | true | 2024-01-11 |
| 72866349-6806-4560-892f-ee120a17a0d0 | This is the best purchase I've made. | 0.54 | POSITIVE | false | 2024-01-19 |
| c94769c9-0e01-4e3c-af52-1fa553b241eb | It works as expected. | 0.15 | NEUTRAL | false | 2024-01-22 |
| 6b94eac6-1aa0-4dcb-b3f1-c5884f196882 | I would not recommend this to anyone. | -0.79 | NEGATIVE | false | 2024-02-10 |
| b0081f56-85b0-4e89-aaba-1ea950cff9a3 | I haven't decided yet. | -0.07 | NEUTRAL | false | 2024-02-23 |
| 375e56a2-1c4a-4f89-ae2a-d7029a6b5f67 | I love this! | 0.92 | POSITIVE | false | 2024-02-28 |
| a8b04b94-fbef-4f57-a7e4-c97173f4c52f | Great service! | 0.65 | POSITIVE | false | 2024-03-01 |
| 751944b0-6e2f-42a6-831f-14452d5209f3 | BUMP | NULL | NULL | true | 2024-03-04 |
| 4d49c2e1-2df2-460b-8fe3-adaba441db43 | Very disappointed with this. | -0.95 | NEGATIVE | false | 2024-03-18 |

only showing top 40 rows

*Figure 4.13 — Result after running the SELECT statement*

*Figure 4.13* shows the result after running the `SELECT` statement, where the table contains the `unique_id`, `statement`, `score`, `tag`, `is_spam`, and `date`. The `date` values in the table range from `2023` to `2024`.

8. Run the following query to count the total number of records in the table:

```
SQL(f"SELECT COUNT(*) FROM { table_name }")
```

This should give us a total count of `71`.

9. Run the following query to count the number of records where `is_spam` is set to `true`:

```
SQL(f"""
SELECT COUNT(*) FROM { table_name } WHERE is_spam=true
""")
```

This should give us 12 records with `is_spam=true`.

If you've used SQL queries to interact with a traditional database before, you'll notice that working with Iceberg tables feels very similar. The familiar `CREATE TABLE`, `SELECT`, and `COUNT` statements work the same way, but now your data is stored in S3 table buckets and can be queried from various services.

## Loading data from a Parquet file to a table

In the previous part, we loaded data from a CSV file into an Iceberg table. Now, let's continue by loading data from a Parquet file (`data_2025_onwards.parquet`), which is just as easy and straightforward to work with in Spark and Iceberg:

1. Load the Parquet file into a PySpark DataFrame:

```
df_2025_onwards = spark.read.parquet(
    f"s3://{s3_bucket}/data_2025_onwards.parquet"
)
```

2. Let's quickly check the first 20 records of `df_2025_onwards`:

```
df_2025_onwards.show(20)
```

This should give us the following:

| unique_id | statement | score | tag | is_spam | date |
|---|---|---|---|---|---|
| 33a82108-dfaa-4f4... | It's fine, neithe... | -0.09 | NEUTRAL | false | 2025-01-01 |
| f2743384-610f-427... | I hate this product. | -0.73 | NEGATIVE | false | 2025-01-09 |
| c1cd992b-54d1-41a... | The product is okay. | -0.1 | NEUTRAL | false | 2025-01-17 |
| 366fe053-154c-480... | This is the best ... | 0.59 | POSITIVE | false | 2025-02-05 |
| 13080924-3d3e-4e6... | Terrible experience. | -0.94 | NEGATIVE | false | 2025-02-09 |
| 64ffe2cd-6d8c-417... | Very disappointed... | -0.96 | NEGATIVE | false | 2025-02-14 |
| d94bf043-8bf5-495... | It works as expec... | -0.08 | NEUTRAL | false | 2025-02-24 |
| 148a1deb-5955-433... | This is the best ... | 0.88 | POSITIVE | false | 2025-03-14 |
| 04934b4f-78f0-464... | Highly recommend ... | 0.68 | POSITIVE | false | 2025-03-20 |
| a04f8808-a8ab-438... | This is awful. | -0.83 | NEGATIVE | false | 2025-03-27 |
| fc1da49c-58a6-439... | This product is a... | 0.89 | POSITIVE | false | 2025-04-03 |
| ee1c0ac6-6479-4b1... | Anyone interested? | NULL | NULL | true | 2025-04-11 |
| 4fc5d719-f625-459... | Terrible experience. | -0.73 | NEGATIVE | false | 2025-05-07 |
| 625d937f-ec2a-427... | It works as expec... | 0.1 | NEUTRAL | false | 2025-05-15 |
| 78355ac3-11aa-4be... | BUMP | NULL | NULL | true | 2025-05-26 |
| eb2e5670-7e70-47f... | It's fine, neithe... | 0.11 | NEUTRAL | false | 2025-06-28 |
| 56661e2a-bb01-4dd... | I would not recom... | -0.74 | NEGATIVE | false | 2025-07-09 |
| 73b8a107-5af3-4aa... | I hate this product. | -0.64 | NEGATIVE | false | 2025-07-12 |
| a134b8c4-8d3e-4bb... | The product is okay. | 0.12 | NEUTRAL | false | 2025-08-04 |
| b71c9ab7-55c8-45f... | Terrible experience. | -0.85 | NEGATIVE | false | 2025-08-17 |

only showing top 20 rows

*Figure 4.14 — First 20 records of the df_2025_onwards DataFrame*

The `date` values in this DataFrame span from 2025 and beyond.

3. Append the Parquet file data to the table using the following block of code:

```
from pyspark.sql.functions import to_date

df_2025_onwards \
    .withColumn("date",
                to_date("date", "yyyy-MM-dd")) \
    .writeTo(table_name) \
    .using("iceberg") \
    .tableProperty("format-version","2") \
    .append()
```

4. Run the following query to check how many records there are in the table:

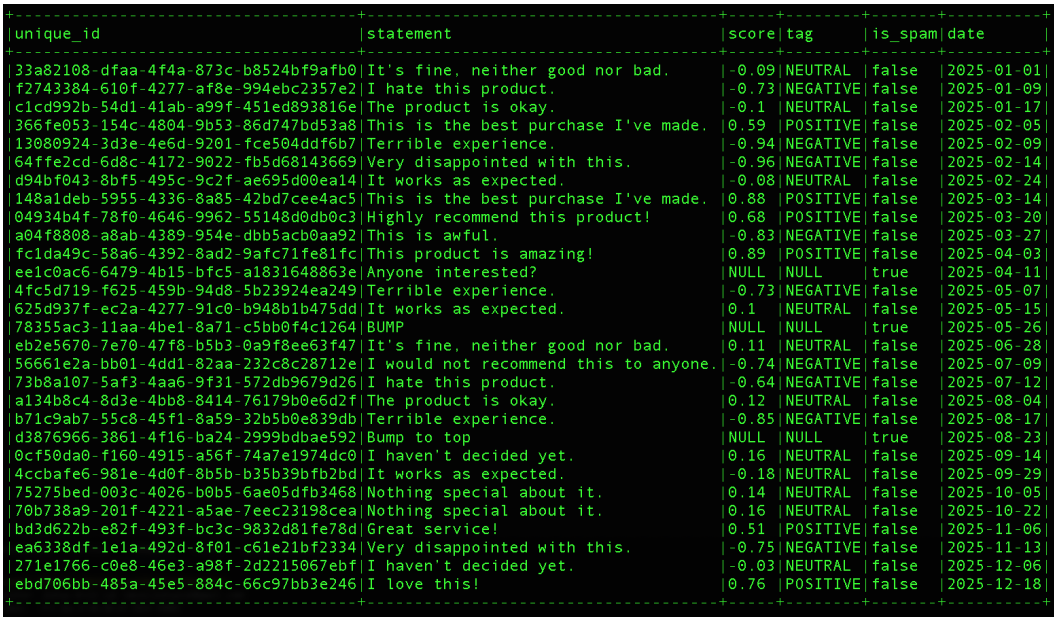
```
SQL(f"SELECT COUNT(*) FROM { table_name }")
```

This should return a count of `100`, as the table contains data from both the `df_up_to_2024` and `df_2025_onwards` DataFrames.

5. Run the following query to check whether the records have been added successfully:

```
SQL(f"""
SELECT * FROM { table_name } WHERE year(date) >= 2025
""")
```

We should get the following:



| unique_id | statement | score | tag | is_spam | date |
|---|---|---|---|---|---|
| 33a82108-dfaa-4f4a-873c-b8524bf9afb0 | It's fine, neither good nor bad. | -0.09 | NEUTRAL | false | 2025-01-01 |
| f2743384-610f-4277-af8e-994ebc2357e2 | I hate this product. | -0.73 | NEGATIVE | false | 2025-01-09 |
| c1cd992b-54d1-41ab-a99f-451ed893816e | The product is okay. | -0.1 | NEUTRAL | false | 2025-01-17 |
| 366fe053-154c-4804-9b53-86d747bd53a8 | This is the best purchase I've made. | 0.59 | POSITIVE | false | 2025-02-05 |
| 13080924-3d3e-4e6d-9201-fce504ddf6b7 | Terrible experience. | -0.94 | NEGATIVE | false | 2025-02-09 |
| 64ffe2cd-6d8c-4172-9022-fb5d68143669 | Very disappointed with this. | -0.96 | NEGATIVE | false | 2025-02-14 |
| d94bf043-8bf5-495c-9c2f-ae695d00ea14 | It works as expected. | -0.08 | NEUTRAL | false | 2025-02-24 |
| 148a1deb-5955-4336-8a85-42bd7cee4ac5 | This is the best purchase I've made. | 0.88 | POSITIVE | false | 2025-03-14 |
| 04934b4f-78f0-4646-9962-55148d0db0c3 | Highly recommend this product! | 0.68 | POSITIVE | false | 2025-03-20 |
| a04f8808-a8ab-4389-954e-dbb5acb0aa92 | This is awful. | -0.83 | NEGATIVE | false | 2025-03-27 |
| fc1da49c-58a6-4392-8ad2-9afc71fe81fc | This product is amazing! | 0.89 | POSITIVE | false | 2025-04-03 |
| ee1c0ac6-6479-4b15-bfc5-a1831648863e | Anyone interested? | NULL | NULL | true | 2025-04-11 |
| 4fc5d719-f625-459b-94d8-5b23924ea249 | Terrible experience. | -0.73 | NEGATIVE | false | 2025-05-07 |
| 625d937f-ec2a-4277-91c0-b948b1b475dd | It works as expected. | 0.1 | NEUTRAL | false | 2025-05-15 |
| 78355ac3-11aa-4be1-8a71-c5bb0f4c1264 | BUMP | NULL | NULL | true | 2025-05-26 |
| eb2e5670-7e70-47f8-b5b3-0a9f8ee63f47 | It's fine, neither good nor bad. | 0.11 | NEUTRAL | false | 2025-06-28 |
| 56661e2a-bb01-4dd1-82aa-232c8c28712e | I would not recommend this to anyone. | -0.74 | NEGATIVE | false | 2025-07-09 |
| 73b8a107-5af3-4aa6-9f31-572db9679d26 | I hate this product. | -0.64 | NEGATIVE | false | 2025-07-12 |
| a134b8c4-8d3e-4bb8-8414-76179b0e6d2f | The product is okay. | 0.12 | NEUTRAL | false | 2025-08-04 |
| b71c9ab7-55c8-45f1-8a59-32b5b0e839db | Terrible experience. | -0.85 | NEGATIVE | false | 2025-08-17 |
| d3876966-3861-4f16-ba24-2999bdbae592 | Bump to top | NULL | NULL | true | 2025-08-23 |
| 0cf50da0-f160-4915-a56f-74a7e1974dc0 | I haven't decided yet. | 0.16 | NEUTRAL | false | 2025-09-14 |
| 4ccbafe6-981e-4d0f-8b5b-b35b39bfb2bd | It works as expected. | -0.18 | NEUTRAL | false | 2025-09-29 |
| 75275bed-003c-4026-b0b5-6ae05dfb3468 | Nothing special about it. | 0.14 | NEUTRAL | false | 2025-10-05 |
| 70b738a9-201f-4221-a5ae-7eec23198cea | Nothing special about it. | 0.16 | NEUTRAL | false | 2025-10-22 |
| bd3d622b-e82f-493f-bc3c-9832d81fe78d | Great service! | 0.51 | POSITIVE | false | 2025-11-06 |
| ea6338df-1e1a-492d-8f01-c61e21bf2334 | Very disappointed with this. | -0.75 | NEGATIVE | false | 2025-11-13 |
| 271e1766-c0e8-46e3-a98f-2d2215067ebf | I haven't decided yet. | -0.03 | NEUTRAL | false | 2025-12-06 |
| ebd706bb-485a-45e5-884c-66c97bb3e246 | I love this! | 0.76 | POSITIVE | false | 2025-12-18 |

*Figure 4.15 — Result after running the SELECT statement*

In this query, we use the `WHERE` clause to filter records with a `date` value from 2025 onward. This allows us to verify that the data from the `df_2025_onwards` DataFrame has been correctly added to the table.

At this point, you should have a good idea of how to use Apache Spark on an EMR cluster to query the data stored in S3 tables. Now that you've explored how to load data from both CSV and Parquet files into Iceberg tables, create tables, and run basic SQL queries, you're ready to explore how to use time travel queries to access previous versions of your S3 tables.

**Note**

While we've focused on using Spark to query Iceberg tables in this section, Spark is a general-purpose distributed compute engine. It can also be used for a wide range of tasks, including data transformation, ML, streaming analytics, and processing data from other sources beyond Iceberg tables.

## Performing time travel queries on S3 tables

When using table formats such as Apache Iceberg with tables stored in Amazon S3 table buckets, every change to a table creates a new snapshot tracked in Iceberg's metadata. This allows you to query previous versions of the table without restoring backups or creating duplicate copies of the data. This capability is particularly useful for auditing changes, reproducing historical reports, troubleshooting data issues, and comparing how datasets have evolved over time.

Let's dive deeper into how we can query an older version of the data stored in S3 tables:

1. Run the following query to retrieve the list of snapshot IDs from the table's history:

```
SQL(f"SELECT * FROM {table_name}.history LIMIT 10")
```

   This should give us the list of `snapshot_id` values similar to what we have in *Figure 4.16*:

```
== Analyzed Logical Plan ==
made_current_at: timestamp, snapshot_id: bigint, parent_id: bigint, is_current_ancestor: boolean
GlobalLimit 10
+- LocalLimit 10
   +- Project [made_current_at#722, snapshot_id#723L, parent_id#724L, is_current_ancestor#725]
      +- SubqueryAlias s3tablesbucket.sentiments_namespace.sentiments_v1.history
         +- RelationV2[made_current_at#722, snapshot_id#723L, parent_id#724L, is_current_ancesto
r#725] s3tablesbucket.sentiments_namespace.sentiments_v1.history s3tablesbucket.sentiments_names
pace.sentiments_v1.history

== Optimized Logical Plan ==
GlobalLimit 10
+- LocalLimit 10
   +- RelationV2[made_current_at#722, snapshot_id#723L, parent_id#724L, is_current_ancestor#725]
 s3tablesbucket.sentiments_namespace.sentiments_v1.history

== Physical Plan ==
CollectLimit 10
+- *(1) Project [made_current_at#722, snapshot_id#723L, parent_id#724L, is_current_ancestor#725]
   +- BatchScan s3tablesbucket.sentiments_namespace.sentiments_v1.history[made_current_at#722, s
napshot_id#723L, parent_id#724L, is_current_ancestor#725] s3tablesbucket.sentiments_namespace.se
ntiments_v1.history (branch=null) [filters=, groupedBy=, pushedLimit=10] RuntimeFilters: []

+-----------------------+-------------------+-------------------+-------------------+
|made_current_at        |snapshot_id        |parent_id          |is_current_ancestor|
+-----------------------+-------------------+-------------------+-------------------+
|2024-12-21 17:20:51.5  |2100412406219901021|NULL               |true               |
|2024-12-21 17:37:02.266|1999046672719253849|2100412406219901021|true               |
+-----------------------+-------------------+-------------------+-------------------+
```

*Figure 4.16 — List of snapshots*

Here, we have two snapshots of the `sentiments_v1` table stored in the S3 table bucket. Each snapshot represents a specific point-in-time version of the table, capturing the state of the data at that moment.

> **Note**
>
> Snapshots are crucial for tracking changes and enabling time travel queries. These snapshots allow you to query historical versions of the data and perform incremental data processing across large datasets stored on Amazon S3.

2. Copy the snapshot ID from the row where `parent_id = NULL` (from the output of the previous step), and use this snapshot ID to replace `<SNAPSHOT ID>` before running the following command:

```
snapshot_id = "<SNAPSHOT ID>"
```

3. Run the following query to retrieve the version of the data from a specific snapshot:

```
SQL(f"""
SELECT COUNT(*) FROM {table_name}
```

```
for system_version as of {snapshot_id}
""")
```

This should return a count of 71.

4. Run the following query to retrieve the version of the data from a specific snapshot (using `snapshot_id`):

```
SQL(f"""
SELECT * FROM {table_name}
FOR system_version AS OF {snapshot_id}
ORDER BY date DESC
LIMIT 30
""")
```

This should give us the following:

```
+------------------------------------+------------------------------------+-----+--------+-------+----------+
|unique_id                           |statement                           |score|tag     |is_spam|date      |
+------------------------------------+------------------------------------+-----+--------+-------+----------+
|8cf58319-09e8-4317-9824-cdba6fc57932|Nothing special about it.           |-0.03|NEUTRAL |false  |2024-12-25|
|8f7e80f6-35a3-49c4-bbef-cc49fb124d25|I hate this product.                |-0.57|NEGATIVE|false  |2024-12-12|
|bdb3c9c0-da86-457d-a052-5b527951a747|Terrible experience.                |-0.72|NEGATIVE|false  |2024-12-08|
|d6afac51-b2cd-4618-9e33-33b12ff35b6c|Great service!                      |0.81 |POSITIVE|false  |2024-11-25|
|e9ce4c5d-9084-43c2-9a97-129a1e985eb5|UP                                  |NULL |NULL    |true   |2024-11-09|
|4aa3d530-4a3c-4a1b-84ed-f36e2d4775c5|Great service!                      |0.63 |POSITIVE|false  |2024-11-03|
|559f472a-e1f9-4f6c-8a69-54463b1fdfd3|Terrible experience.                |-0.52|NEGATIVE|false  |2024-10-19|
|328b0c79-40aa-43d9-b698-88e5be52b5f9|The product is okay.                |0.15 |NEUTRAL |false  |2024-10-15|
|ff8af294-21cf-402a-8514-448daf12e9e2|Nothing special about it.           |0.13 |NEUTRAL |false  |2024-10-12|
|b354a7a0-dfa4-4f58-ad1e-16f01a997b64|It works as expected.               |0.07 |NEUTRAL |false  |2024-10-07|
|0c6f949c-7fbf-462c-8b28-7d0729d4ff71|This is awful.                      |-0.83|NEGATIVE|false  |2024-09-28|
|b034d21d-c87f-4b30-8542-627a2a348446|I would not recommend this to anyone.|-0.73|NEGATIVE|false  |2024-09-15|
|db53dd24-a2b7-41e5-9ea6-be63918f535a|Terrible experience.                |-0.61|NEGATIVE|false  |2024-08-25|
|0bdfdfe2-848c-4f71-add7-f7705fdf4aa4|Just checking in                    |NULL |NULL    |true   |2024-08-19|
|86db4ae5-00a9-456b-9e3a-47cf292eed39|Terrible experience.                |-0.81|NEGATIVE|false  |2024-08-01|
|554d8424-ca04-4289-817f-14b7f0ad7812|It works as expected.               |-0.03|NEUTRAL |false  |2024-07-29|
|bb819ca3-a8ea-48f2-80e4-f2e54473e58d|This product is amazing!            |0.79 |POSITIVE|false  |2024-07-23|
|c1c304e7-b25d-4666-83eb-d04e0531ce62|This is the best purchase I've made.|0.59 |POSITIVE|false  |2024-07-04|
|691d5f54-f053-430d-9c59-76eb86a9909c|Very disappointed with this.        |-0.99|NEGATIVE|false  |2024-07-01|
|6ab01aae-ee8a-4baf-b6e2-5112506a17e6|It's fine, neither good nor bad.    |-0.05|NEUTRAL |false  |2024-06-30|
|c7d65866-45fb-49b8-9632-25d6ae554a32|Anyone interested?                  |NULL |NULL    |true   |2024-06-21|
|cec37c84-4683-4f84-b6ed-ea245d664892|This product is amazing!            |0.59 |POSITIVE|false  |2024-06-13|
|7572623e-baf7-4f09-8690-f84b61a166ef|BUMP                                |NULL |NULL    |true   |2024-06-03|
|18d89e43-28a4-49ef-81f3-6d55977c50c5|UP                                  |NULL |NULL    |true   |2024-05-23|
|b63f614f-dbd2-4353-94c1-862bc2619929|Bump to top                         |NULL |NULL    |true   |2024-05-17|
|104eb7a8-b864-4a3f-addc-f02085510533|Anyone interested?                  |NULL |NULL    |true   |2024-05-06|
|cd789107-35f9-4de3-8f44-5b8a18b9a6af|I hate this product.                |-0.8 |NEGATIVE|false  |2024-04-28|
|029e22b7-9fe2-4de6-9194-1eeb99e76570|Great service!                      |0.68 |POSITIVE|false  |2024-04-15|
|fac4523e-9934-4537-b38b-19acef89ddd7|I haven't decided yet.              |0.2  |NEUTRAL |false  |2024-04-12|
|5dad8e29-4810-4883-8bb0-09ae909eb009|I would not recommend this to anyone.|-0.98|NEGATIVE|false  |2024-04-03|
+------------------------------------+------------------------------------+-----+--------+-------+----------+
```

*Figure 4.17 — Query results*

Take note that this version of the table still contains the `score` column, which indicates that the dataset structure and values reflect the state of the table at the selected snapshot.

5. Run the following to drop the `score` column:

```
spark.sql(f"ALTER TABLE {table_name} DROP COLUMN score")
```

6. Then, run the following to verify that the `score` column has been dropped:

```
SQL(f"SELECT * FROM {table_name} LIMIT 30")
```

This should give us the following:

| unique_id | statement | tag | is_spam | date |
|---|---|---|---|---|
| 35ce3303-f520-4dbe-9759-ef87f261c3f3 | I love this! | POSITIVE | false | 2023-01-14 |
| 0e84ff9b-2845-48b7-9674-d4da4f3f3fee | The product is okay. | NEUTRAL | false | 2023-02-04 |
| d36d5a11-138d-49a5-bcb8-acd2a889493a | It works as expected. | NEUTRAL | false | 2023-02-15 |
| 05fde437-4630-43ac-821a-77d86ead113a | This is awful. | NEGATIVE | false | 2023-03-09 |
| bd2aabd2-e388-4616-acf9-90358091e76b | This is the best purchase I've made. | POSITIVE | false | 2023-03-17 |
| 9c47757f-35be-420e-92bd-6c7eebd5a2b1 | The product is okay. | NEUTRAL | false | 2023-04-13 |
| e0bfe69b-ff8a-43f5-babc-8e9e857d3c6a | I would not recommend this to anyone. | NEGATIVE | false | 2023-04-21 |
| b647c226-acb1-4d1e-ae43-25a5bcc08aa9 | It's fine, neither good nor bad. | NEUTRAL | false | 2023-05-19 |
| a668404e-6524-453f-9587-94dd981224be | Great service! | POSITIVE | false | 2023-05-22 |
| d8935957-3be8-4823-8875-548bdb1e663d | Highly recommend this product! | POSITIVE | false | 2023-06-04 |
| 23916b0c-f3aa-4c74-97b6-3cc9b8f47eab | Anyone interested? | NULL | true | 2023-06-14 |
| ea407d66-bd3b-4ca9-b0a8-2208e71c7593 | Highly recommend this product! | POSITIVE | false | 2023-06-19 |
| 8f1caa71-9914-40d9-b3c7-b9c4ea306e4e | It's fine, neither good nor bad. | NEUTRAL | false | 2023-07-17 |
| b0c27eef-3a61-4985-b63c-30bb9bd1b0e0 | This is awful. | NEGATIVE | false | 2023-07-30 |
| 4e7a6855-1bd6-4286-b031-a36d42ea6229 | Great service! | POSITIVE | false | 2023-08-11 |
| f7c93fe9-7d53-49be-b19f-f0f274b4fc90 | It's fine, neither good nor bad. | NEUTRAL | false | 2023-08-14 |
| 5bc30253-788e-4cbd-8ae1-8614c40be96d | Just checking in | NULL | true | 2023-08-27 |
| 7a212bd4-bbe9-4374-b68b-f1682c24f1cd | This product is amazing! | POSITIVE | false | 2023-09-11 |
| f7047d8b-2242-420c-8eba-09032406878c | I hate this product. | NEGATIVE | false | 2023-09-21 |
| 3f71dbf2-97f4-4e28-85a4-b2be09cbd2b4 | Nothing special about it. | NEUTRAL | false | 2023-09-25 |
| ea3fb396-d583-422e-8c29-2a58ff7a581c | Nothing special about it. | NEUTRAL | false | 2023-10-01 |
| 1800ed58-4f77-4af9-948a-d925258f1165 | This is the best purchase I've made. | POSITIVE | false | 2023-10-08 |
| 38be50a2-b3b3-4d87-9d29-2252787e2051 | This product is amazing! | POSITIVE | false | 2023-10-17 |
| 666eb59c-b3f3-4629-afe9-42a4efaca715 | I hate this product. | NEGATIVE | false | 2023-10-24 |
| fec7045f-b55e-459d-bc6a-5cb1b7c0cfbd | Great service! | POSITIVE | false | 2023-11-10 |
| a43d8141-11ba-4324-994b-084c1a9c3634 | Bump to top | NULL | true | 2023-11-16 |
| 2b2bb5d2-b289-4650-ace9-7f0dc765db98 | I love this! | POSITIVE | false | 2023-11-21 |
| 13be5b71-8018-49c9-9958-780e1179f031 | This product is amazing! | POSITIVE | false | 2023-12-02 |
| cac2e72f-7ae9-49d6-bc12-0bd38847b805 | Terrible experience. | NEGATIVE | false | 2023-12-15 |
| c22491e7-c701-4a21-84a9-a4a942d781ff | I hate this product. | NEGATIVE | false | 2023-12-30 |

*Figure 4.18 — Current version of the table*

Here, we can see that the `score` column is no longer present as it has been deleted in one of the queries in a previous step.

7. Now, let's utilize Iceberg's time travel capability to explore an earlier version. Run the following to query an old snapshot version of the table:

```
SQL(f"""
SELECT * FROM {table_name}
FOR system_version AS OF {snapshot_id}
ORDER BY date DESC
```

```
LIMIT 30
""")
```

This should give us the following:

```
+------------------------------------+-----------------------------------+-----+--------+-------+----------+
|unique_id                           |statement                          |score|tag     |is_spam|date      |
+------------------------------------+-----------------------------------+-----+--------+-------+----------+
|8cf58319-09e8-4317-9824-cdba6fc57932|Nothing special about it.          |-0.03|NEUTRAL |false  |2024-12-25|
|8f7e80f6-35a3-49c4-bbef-cc49fb124d25|I hate this product.               |-0.57|NEGATIVE|false  |2024-12-12|
|bdb3c9c0-da86-457d-a052-5b527951a747|Terrible experience.               |-0.72|NEGATIVE|false  |2024-12-08|
|d6afac51-b2cd-4618-9e33-33b12ff35b6c|Great service!                     |0.81 |POSITIVE|false  |2024-11-25|
|e9ce4c5d-9084-43c2-9a97-129a1e985eb5|UP                                 |NULL |NULL    |true   |2024-11-09|
|4aa3d530-4a3c-4a1b-84ed-f36e2d4775c5|Great service!                     |0.63 |POSITIVE|false  |2024-11-03|
|559f472a-e1f9-4f6c-8a69-54463b1fdfd3|Terrible experience.               |-0.52|NEGATIVE|false  |2024-10-19|
|328b0c79-40aa-43d9-b698-88e5be52b5f9|The product is okay.               |0.15 |NEUTRAL |false  |2024-10-15|
|ff8af294-21cf-402a-8514-448daf12e9e2|Nothing special about it.          |0.13 |NEUTRAL |false  |2024-10-12|
|b354a7a0-dfa4-4f58-ad1e-16f01a997b64|It works as expected.              |0.07 |NEUTRAL |false  |2024-10-07|
|0c6f949c-7fbf-462c-8b28-7d0729d4ff71|This is awful.                     |-0.83|NEGATIVE|false  |2024-09-28|
|b034d21d-c87f-4b30-8542-627a2a348446|I would not recommend this to anyone.|-0.73|NEGATIVE|false  |2024-09-15|
|db53dd24-a2b7-41e5-9ea6-be63918f535a|Terrible experience.               |-0.61|NEGATIVE|false  |2024-08-25|
|0bdfdfe2-848c-4f71-add7-f7705fdf4aa4|Just checking in                   |NULL |NULL    |true   |2024-08-19|
|86db4ae5-00a9-456b-9e3a-47cf292eed39|Terrible experience.               |-0.81|NEGATIVE|false  |2024-08-01|
|554d8424-ca04-4289-817f-14b7f0ad7812|It works as expected.              |-0.03|NEUTRAL |false  |2024-07-29|
|bb819ca3-a8ea-48f2-80e4-f2e54473e58d|This product is amazing!           |0.79 |POSITIVE|false  |2024-07-23|
|c1c304e7-b25d-4666-83eb-d04e0531ce62|This is the best purchase I've made.|0.59 |POSITIVE|false  |2024-07-04|
|691d5f54-f053-430d-9c59-76eb86a9909c|Very disappointed with this.       |-0.99|NEGATIVE|false  |2024-07-01|
|6ab01aae-ee8a-4baf-b6e2-5112506a17e6|It's fine, neither good nor bad.   |-0.05|NEUTRAL |false  |2024-06-30|
|c7d65866-45fb-49b8-9632-25d6ae554a32|Anyone interested?                 |NULL |NULL    |true   |2024-06-21|
|cec37c84-4683-4f84-b6ed-ea245d664892|This product is amazing!           |0.59 |POSITIVE|false  |2024-06-13|
|7572623e-baf7-4f09-8690-f84b61a166ef|BUMP                               |NULL |NULL    |true   |2024-06-03|
|18d89e43-28a4-49ef-81f3-6d55977c50c5|UP                                 |NULL |NULL    |true   |2024-05-23|
|b63f614f-dbd2-4353-94c1-862bc2619929|Bump to top                        |NULL |NULL    |true   |2024-05-17|
|104eb7a8-b864-4a3f-addc-f02085510533|Anyone interested?                 |NULL |NULL    |true   |2024-05-06|
|cd789107-35f9-4de3-8f44-5b8a18b9a6af|I hate this product.               |-0.8 |NEGATIVE|false  |2024-04-28|
|029e22b7-9fe2-4de6-9194-1eeb99e76570|Great service!                     |0.68 |POSITIVE|false  |2024-04-15|
|fac4523e-9934-4537-b38b-19acef89ddd7|I haven't decided yet.             |0.2  |NEUTRAL |false  |2024-04-12|
|5dad8e29-4810-4883-8bb0-09ae909eb009|I would not recommend this to anyone.|-0.98|NEGATIVE|false  |2024-04-03|
+------------------------------------+-----------------------------------+-----+--------+-------+----------+
```

*Figure 4.19 — Previous version of the table*

As we have in *Figure 4.19*, we can see that the `score` column is present in an older version of the table. Using `FOR system_version AS OF <SNAPSHOT ID>` enables you to access a previous snapshot of the table, letting you query its state as it existed at that specific point in time.

**Note**

Now that you have completed all the sections of this chapter, we can perform the cleanup steps to help us reduce and manage costs. Make sure to terminate the EMR cluster as well as IAM resources created in the *Launching an Amazon EMR cluster with Apache Iceberg installed* section. Since the data is securely stored in the S3 table bucket, deleting the EMR cluster will not result in any data loss.

At this stage, you should have a good idea of how the time travel capability in an Iceberg-backed S3 table bucket works when querying versioned datasets. By exploring earlier

snapshots, you can see how the table state changed over time without modifying or rewriting historical data. This feature is valuable for understanding dataset evolution, supporting audit requirements, and validating data modifications.

As an ML engineer, you should focus on optimizing queries, analyzing data access patterns, and adopting best practices for managing large datasets and maintaining cost-effective query performance. There are definitely more advanced techniques and nuances we did not cover in this chapter, but the hands-on examples you worked through should give you a solid foundation for working with a transactional data lake on S3.

# Summary

In this chapter, you explored how to build and work with a transactional data lake using S3 tables. More specifically, you learned how to create an Amazon S3 table bucket, launch an Amazon EMR cluster with Apache Iceberg installed, and perform queries using Apache Spark. You also practiced running time travel queries on S3 tables to retrieve previous dataset versions while enabling exploration of data changes across time.

In the next chapter, you will build on what you set up in this chapter and dive deeper into working with AWS Lake Formation permissions, running SQL queries in Amazon Athena, ingesting data into an Amazon SageMaker AI Feature Store, adding searchable metadata to features, and retrieving data from both online and offline feature store repositories.

# Further reading

To expand your practical knowledge of the concepts presented in this chapter, refer to the following resources:

- *Amazon S3 User Guide* | *Working with Amazon S3 Tables and table buckets*: `https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables.html`
- *Amazon S3 User Guide* | *Accessing Amazon S3 tables with Amazon EMR*: `https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables-integrating-emr.html`
- *Amazon S3 User Guide* | *Amazon S3 Tables integration with AWS analytics services overview*: `https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables-integration-overview.html`

- *Amazon S3 User Guide | S3 Tables maintenance*: `https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables-maintenance-overview.html`
- *AWS re:Invent 2025 - Accelerate analytics and AI w/ an open and secure lakehouse architecture-ANT309*: `https://www.youtube.com/watch?v=qIsxBWfVgsM`.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 5

# Practical Data Management on AWS

In the previous chapter, you learned how to build and work with a transactional data lake for data analytics and machine learning workloads. In this chapter, we will build on that foundation by exploring practical data management techniques for modern cloud-based ML environments, while leveraging AWS's managed services so you don't have to build your own data management solutions from scratch. You will learn how to work with AWS Lake Formation permissions, use Amazon Athena to query and process data stored in S3 table buckets, and ingest data into Amazon SageMaker Feature Store. You will also learn how to add searchable metadata to features and retrieve features from both the online and offline feature stores.

To help you build practical data management skills for modern cloud-based ML workflows, we will cover the following topics in this chapter:

- Working with AWS Lake Formation permissions
- Running SQL queries in Amazon Athena
- Ingesting data into a SageMaker feature store
- Adding searchable metadata to the features
- Retrieving data from the online and offline feature stores

By the end of this chapter, you will be better equipped to manage and organize datasets for machine learning workflows on AWS and apply these practical data management techniques in real-world projects.

# Technical requirements

We must have the following ready before we jump into the hands-on examples of this chapter:

- **A new IAM role named SageMakerAdminRole with the AmazonSageMakerAccessFullAccess and AmazonS3FullAccess permission policies attached**: You can create this IAM role by first opening the AWS Management Console, typing `IAM` in the search bar, and then selecting **IAM** from the list of results. Once in the IAM console, select **Roles** in the left-hand menu and then click the **Create role** button. From there, select **Custom trust policy** from the list of options under **Trusted entity type**. Replace the custom trust policy with the following trust policy that allows SageMaker to assume the role:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "",
            "Effect": "Allow",
            "Principal": {
                "Service": "sagemaker.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

  Continue to the permissions page and attach the `AmazonSageMakerFullAccess` and `AmazonS3FullAccess` policies. After completing the steps and saving the role (using the name `SageMakerAdminRole`), it will be ready for use by SageMaker resources that require these permissions. This role essentially grants SageMaker the administrative-level permissions required to operate fully within your AWS account.

- **A new IAM user with AdministratorAccess permissions**: You can create this IAM user through the IAM Console, by typing `IAM` in the search bar, and then selecting **IAM** from the list of results. Once in the IAM console, select **Users** from the left-hand menu and then click the **Create user** button. Enter the desired user name (`superadmin`), ensure that the **Provide user access to the AWS Management Console** checkbox is checked, choose **I want to create an IAM user**, and click **Next**. On the **Set permissions**

page, choose **Attach policies directly**, then search for and select the `AdministratorAccess` policy. Continue through the remaining steps, and finally click the **Create user** button. After completion, you'll be able to use the new IAM user to log in to the AWS Management Console with full administrative permissions.

**Note**

An IAM user with administrative-level permissions (for example, one attached with the `AdministratorAccess` policy) can perform most actions across AWS services required in this chapter. While it is generally recommended that IAM users follow the principle of least privilege and have more restrictive permissions, attaching the `AdministratorAccess` policy to your IAM user will simplify setup and should allow you to complete the hands-on examples in this chapter without permissions-related issues.

Before proceeding, confirm that your currently selected AWS Region matches the region where your VPC has been set up. You can change the region at any time by using the region selector in the upper-right corner of the AWS Management Console and choosing the correct region (for example, **us-east-1**) from the drop-down list.

**Note**

All the code, commands, and files used in this chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

# Working with AWS Lake Formation permissions

Working with large-scale data lakes often introduces governance challenges, especially when multiple teams, analytics tools, and services need controlled access to shared datasets. AWS Lake Formation helps address these challenges by providing a centralized way to manage data lake permissions and enforce consistent access controls. It integrates with various services to help you define fine-grained permissions, control who can access specific datasets, and apply governance policies across your data environment. This unified approach simplifies permission management while still supporting the flexibility required for analytics, machine learning, and other data-driven workloads.

For example, let's say you manage a data lake that contains customer transactions, product data, and sentiment analysis results from multiple sources. Data scientists and ML engineers may need access to the sentiment analysis tables for training machine learning models, while

BI teams may only need read access to specific tables used for building reports and visualizations. With Lake Formation, you can define these permissions in a central location to ensure that each team can access only the datasets relevant to their work.

In this section, you will configure Lake Formation permissions to allow a principal to access the data resources created in the previous chapter. More specifically, you'll grant the `superadmin` IAM user access to the database and tables stored in the S3 table bucket created in *Chapter 4*.

Let's quickly go over some important concepts and terminology before we dive into the hands-on portion:

- **AWS Lake Formation**: A service that helps you centrally manage, secure, and govern data lakes on AWS. It simplifies defining fine-grained data access permissions and enforcing consistent access controls for datasets stored in services such as Amazon S3.
- **Principal**: An entity that can be granted permissions to access AWS resources. In Lake Formation, this typically refers to an IAM user, IAM role, or AWS account that is allowed to access or manage data resources.
- **Permission**: An authorization granted to a principal that allows specific actions on data resources. In Lake Formation, permissions control actions such as reading tables, querying data, or managing access to databases and tables.
- **Catalog**: A centralized metadata repository that stores information about datasets, such as databases, tables, schemas, and storage locations. In AWS, this is typically implemented using the AWS Glue Data Catalog, which enables analytics services to discover and query data.

**Note**

In this section, you will use the IAM root user account (that is, the original AWS account root user) when granting data access permission in AWS Lake Formation. Keep in mind that for production use cases, using the root user is not recommended.

With these in mind, let's proceed with granting data access permissions in AWS Lake Formation.

1. Type `lake formation` in the search bar of the AWS Management Console and select **AWS Lake Formation** from the list of search results (*Figure 5.1*).

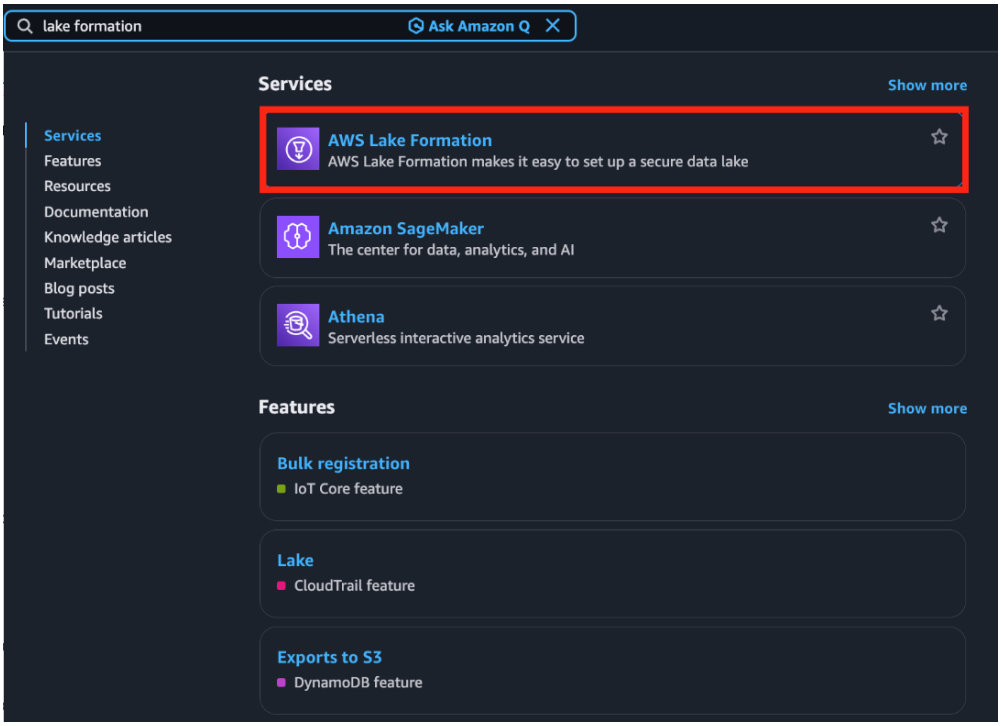



*Figure 5.1 — Navigating to the AWS Lake Formation console*

This will redirect you to the AWS Lake Formation Console, where you can manage data lake permissions, catalogs, and data access policies.

2. In the navigation pane (left sidebar), click **Data permissions** under **Permissions**. This should open a page that shows all existing data permissions for your account, including which principals (users, groups, or roles) have access to which data resources.
3. On the **Data permissions** page, click the **Grant** button to start the process of granting permissions. This will take you to the **Grant permissions** page, where you can specify the principals, resources, and permissions to grant (*Figure 5.2*).

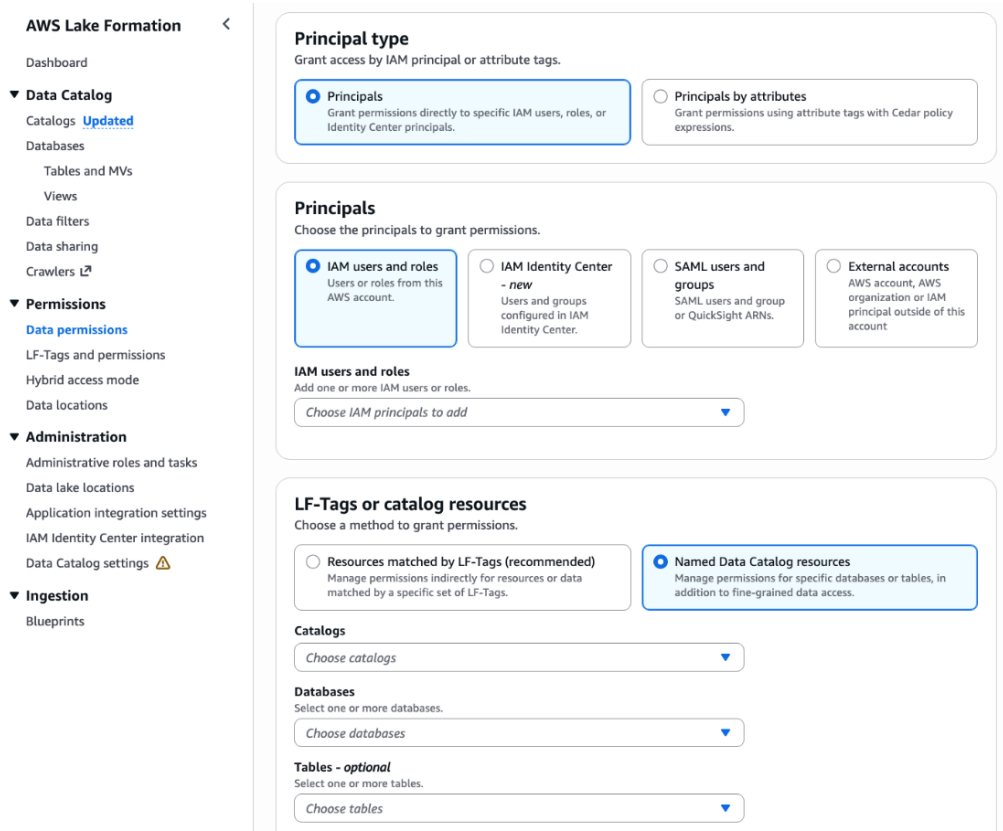



*Figure 5.2 — Grant permissions page*

Here, you can define the principal that will receive the permissions, select the catalog resource, and specify the actions that the principal is allowed to perform.

4. For **Principal type**, choose **Principals**.
5. Under **Principals**, select **IAM users and roles**.
6. Under **IAM users and roles**, select `superadmin` from the list of available IAM users or roles. This will grant the `superadmin` role the specified permissions.
7. Under **LF-Tags or catalog resources**, choose **Named Data Catalog resources**.
8. From the **Catalogs** drop-down menu, select the catalog resource that corresponds to your S3 table bucket, with the format `<ACCOUNT ID>:s3tablescatalog/<NAME OF TABLE BUCKET>`.

> **Note**
>
> Make sure `<NAME OF TABLE BUCKET>` matches the name of the S3 table bucket you created in *Creating an Amazon S3 table bucket* in *Chapter 4*. Refer to the S3 table bucket name (`TAG:TABLE_BUCKET_NAME`) from your local code editor if you saved it there earlier. You can also retrieve the table bucket name from the Amazon S3 console.

9. In the **Databases** field, select `sentiments_namespace` from the list of options available from the drop-down menu.
10. Under **Tables**, choose `All tables` to grant permissions on all tables within the database.
11. Under **Table permissions**, choose **Super** for both **Table permissions** and **Grantable permissions**.

> **Note**
>
> Selecting **Super** for both **Table permissions** and **Grantable permissions** grants the principal full administrative control over the tables within the database in AWS Lake Formation. This permission allows the principal to read, modify, manage, and delete table resources. Keep in mind that this level of access is powerful and should be assigned only when absolutely necessary in production environments.

At this stage, you've successfully configured Lake Formation permissions to grant the `superadmin` IAM user access to the database and tables stored in the S3 table bucket created in *Chapter 4*. In the next section, you will sign in to the AWS Management Console using the `superadmin` IAM user to run SQL queries in Amazon Athena.

## Running SQL queries in Amazon Athena

In many analytics workflows, teams may spend days or even weeks provisioning, configuring, and validating the infrastructure required to run data queries and processing jobs. When you want to query data stored in Amazon S3 using SQL, Amazon Athena provides a serverless way to analyze your data without having to provision or manage infrastructure yourself. If you haven't used Athena before, it is a service that lets you run SQL queries directly on your data. SQL is a popular choice for data analysis because it allows you to describe what you want from the data, while Athena handles the details of query execution behind the scenes.

In this section, you will learn how to run SQL queries in Amazon Athena to analyze the data stored in the Amazon S3 table bucket you worked with in the previous chapter. You'll explore how to execute queries to explore, filter, and modify data in much the same way you would when working with a traditional database.

> **Note**
>
> While Athena allows you to query data much like a traditional database, there are some important differences to be aware of. For example, Athena does not support stored procedures and has a few statements that are not available. To dive deeper into this topic, you may check the official Amazon Athena documentation: `https://docs.aws.amazon.com/athena/latest/ug/other-notable-limitations.html`.

We'll cover in this section two approaches for running SQL queries in Amazon Athena: using the AWS CLI and using Boto3 to run queries programmatically.

## Using the AWS CLI to run SQL queries with Amazon Athena

You'll now explore how to execute SQL queries in Athena directly from the command line using the AWS CLI. This method is a quick and convenient way to run ad hoc queries, check data, or automate simple workflows without writing code. You'll see how to submit queries, check their status, and retrieve results, all from the terminal.

Let's proceed with running SQL queries using the AWS CLI, which comes preinstalled in AWS CloudShell:

1. Open a new private browsing window and sign in to the AWS Management Console using the `superadmin` IAM user. If you are prompted to reset the password during sign-in, complete the password reset process before proceeding.

   > **Note**
   >
   > When signing in as an IAM user, you will be prompted to enter the AWS account ID or account alias, as well as the IAM username and password, which differs from the root account sign-in experience. If you have not yet created and configured this IAM user, you can refer to the *Technical requirements* section at the start of this chapter for more details.

2. Navigate to the same region (for example, **N. Virginia** | **us-east-1**) where the resources were created in *Chapter 4*, to ensure you are working within the correct region.

3. In the search box of the AWS Management Console, type `cloudshell` and then choose **CloudShell** from the search results. We'll run the commands in the following steps in the CloudShell environment.

> **Note**
>
> It may take approximately a minute for the CloudShell environment to load. You should be able to proceed to the next step and run Terminal commands once you see `[cloudshell-user@ip-XX-XX-XX-XX ~]$`.

4. Create a new S3 general-purpose bucket for the output location of the Athena queries:

```
ATHENA_OUTPUT_LOCATION="<NAME OF NEW S3 BUCKET>"
aws s3 mb s3://$ATHENA_OUTPUT_LOCATION
```

Keep in mind that this general-purpose bucket does not exist prior to running the commands.

> **Note**
>
> Make sure to note the bucket name, as you'll reference it in the subsequent steps. You should store the bucket name (`TAG:ATHENA_OUTPUT_BUCKET`) in a text file using a code editor on your local machine so you can easily reference it in later steps.

5. Specify the S3 table bucket name (`TAG:TABLE_BUCKET_NAME`) by assigning it to a variable:

```
TABLE_BUCKET_NAME="<TABLE BUCKET NAME>"
```

Make sure to replace `<SPECIFY S3 TABLE BUCKET NAME>` with the name of the S3 table bucket you created in *Creating an Amazon S3 table bucket* of *Chapter 4*, before running the command.

> **Note**
>
> You can use the S3 table bucket name (`TAG:TABLE_BUCKET_NAME`) from your local code editor if you saved it there while working on the hands-on examples of the previous chapter. Alternatively, you can retrieve the table bucket name from the Amazon S3 console by typing `S3` in the search bar of the AWS

> Management Console, selecting **S3** from the results, and navigating to the **Table buckets** page by clicking **Table buckets** under **Buckets** in the navigation pane (left sidebar).

6. Prepare a query string for retrieving a sample set of records:

```
CATALOG_BUCKET="s3tablescatalog/$TABLE_BUCKET_NAME"
NAMESPACE="sentiments_namespace"
TABLE="sentiments_v1"
QUERY_STRING="SELECT * FROM \"$CATALOG_BUCKET\".$NAMESPACE.$TABLE LIMIT 10"
echo $QUERY_STRING
```

This query string simply executes a SQL statement that retrieves a small sample of 10 records from the `sentiments_v1` table to verify that the table and data can be queried successfully.

7. With everything ready, let's use `aws athena start-query-execution` to run the SQL query:

```
OUTPUT_LOC="$ATHENA_OUTPUT_LOCATION"
RESULT_CONF="OutputLocation=s3://$OUTPUT_LOC/"

aws athena start-query-execution \
  --query-string "$QUERY_STRING" \
  --work-group "primary" \
  --result-configuration $RESULT_CONF
```

Here, we configure Amazon Athena to save the query output to the specified Amazon S3 location. Running the command should return the query execution ID, similar to what we have in the following block:

```
{
    "QueryExecutionId": "<QUERY EXECUTION ID>"
}
```

You will use this query execution ID to monitor or retrieve the results of the query execution in subsequent steps.

8. Copy the query execution ID from the output of the previous step and replace `<QUERY_EXECUTION_ID>` before running the following `aws athena get-query-execution` command:

```
aws athena get-query-execution \
  --query-execution-id <QUERY_EXECUTION_ID>
```

This should return an output with a structure similar to the following:

```
{
  "QueryExecution": {
    "QueryExecutionId": "...",
    "Query": "SELECT ...",
    "StatementType": "DML",
    "ResultConfiguration": {
      "OutputLocation": "s3://....csv"
    },
    ...
  }
}
```

The output contains metadata about the executed query in Amazon Athena. This includes details such as the `QueryExecutionId`, the SQL query that was executed, the execution status, timestamps for submission and completion, and performance statistics such as execution time and the amount of data scanned.

> **Note**
>
> The output would also include the `OutputLocation` value, which indicates where Amazon Athena stored the query results in Amazon S3. Note that the `OutputLocation` value should resemble a path such as `"s3://....csv"`.

9. Copy the `OutputLocation` value from the results of the previous step, which can be found under the `"ResultConfiguration"` section of the query output.

10. Use the `aws s3 cp` command to download a copy of the results from the S3 bucket. Make sure to replace `<SOURCE S3 PATH>` with the `OutputLocation` value (`"s3://....csv"`) you copied earlier before running the command.

```
aws s3 cp <SOURCE S3 PATH> output.csv
```

11. Inspect the contents of the downloaded file by running the following command:

```
cat output.csv
```

This will produce an output similar to the following:

```
"...","It's fine, neither good nor bad.","NEUTRAL","false","2025-01-01"
"...","I hate this product.","NEGATIVE","false","2025-01-09"
"...","The product is okay.","NEUTRAL","false","2025-01-17"
"...","This is the best purchase I've
made.","POSITIVE","false","2025-03-14"
"...","Highly recommend this product!","POSITIVE","false","2025-03-20"
```

The `output.csv` file contains the query results generated by Amazon Athena. Each row represents a record retrieved from the dataset and includes fields such as `unique_id`, `statement`, `tag`, `is_spam`, and `date`, which correspond to the metadata, sentiment classification, spam indicator, and timestamp information associated with each entry.

At this point, you should have a good idea of how to use the AWS CLI when running Athena queries. Unlike Amazon EMR, which requires provisioning and managing a cluster to process large datasets, Athena is serverless and lets you run queries directly on data without managing any infrastructure. While EMR provides more control over the computing environment and supports complex distributed processing, Athena is ideal for ad hoc queries and quick analysis using standard SQL. In addition to these, with Athena's pay-per-query pricing model, you only pay for the data scanned by the queries you run, so you aren't charged when you're not using it.

## Using Boto3 to run SQL queries with Amazon Athena

While the AWS CLI is useful for running ad hoc queries from the command line, there are times when you want to execute Athena queries programmatically as part of an application or workflow. In this section, you'll learn how to use Boto3, the AWS SDK for Python, to submit SQL queries, monitor their execution, and retrieve the results in code. This approach is especially useful for automation, data pipelines, and integrating Athena queries into analytics or machine learning workflows.

> **Note**
>
> While the examples in this section use IPython in AWS CloudShell, you can also run them from an EC2 instance (for example, an instance running Ubuntu) with an IAM role that has the necessary permissions for Amazon Athena and S3 access. This setup allows you to execute the same code and queries directly from the instance without relying on CloudShell. Be sure to stop or terminate the EC2 instance after completing the examples in this chapter to avoid unnecessary charges.

Let's go through the steps to run SQL queries in Amazon Athena using Boto3:

1. Use the `pip install` command to install `ipython`, `pandas`, `fsspec`, and `s3fs`:

   ```
   pip install ipython pandas fsspec s3fs
   ```

   > **Note**
   >
   > The following versions are used in this chapter: IPython `8.18.1`, pandas `2.3.2`, fsspec `2025.10.0`, and s3fs `2025.10.0`. Versions close to these should also work without any issues.

2. Now, open an IPython **Read-Eval-Print-Loop** (**REPL**) interactive shell:

   ```
   ipython
   ```

3. Run the following to specify the Amazon S3 output location that Amazon Athena will use to store query results:

   ```
   athena_output_s3_bucket_name = "<BUCKET NAME>"
   output_bucket = "s3://" + athena_output_s3_bucket_name
   ```

   Make sure to replace `<BUCKET NAME>` with the name of the general-purpose bucket you created earlier in this section (*Using the AWS CLI to run SQL queries with Amazon Athena*) before running the command. The bucket name is stored in the `ATHENA_OUTPUT_LOCATION` bash variable for easy reference.

> **Note**
>
> You can use the S3 bucket name (`TAG:ATHENA_OUTPUT_BUCKET`) from your local code editor if you saved it there earlier. You can also retrieve the bucket name from the Amazon S3 console.

4. Initialize the `boto3` client to interact with the Athena API:

```
import boto3
region_name = 'us-east-1'
athena = boto3.client('athena',
                      region_name=region_name)
```

You may change the `region_name` value if you created and configured your resources in a different AWS region. Make sure that the region you specify matches the region where your table bucket is located.

5. Specify the table bucket name (`TAG:TABLE_BUCKET_NAME`) by assigning it to a variable:

```
table_bucket = '<TABLE BUCKET NAME>'
```

Be sure to replace `<TABLE BUCKET NAME>` with the name of the S3 table bucket you created in *Creating an Amazon S3 table bucket* before running the command.

> **Note**
>
> You can use the S3 table bucket name (`TAG:TABLE_BUCKET_NAME`) from your local code editor if you saved it there earlier. You can also retrieve the table bucket name from the Amazon S3 console.

6. Prepare the query, database, and the query configuration using the following block of code:

```
t_a = f'"s3tablescatalog/{table_bucket}"'
t_b = 'sentiments_namespace'
t_c = 'sentiments_v1'
table = f'{t_a}.{t_b}.{t_c}'
query = f"SELECT * FROM { table }"
database = f"s3tablescatalog/{table_bucket}"
```

```
query_config = {
    'database': database,
    'output_bucket': output_bucket
}
```

7. Define the `execute_query()` function to programmatically submit SQL queries to Amazon Athena and return the query execution ID:

```
def execute_query(query, query_config=query_config):
    location = query_config['output_bucket']
    query_params = {
        'QueryString': query,
        'QueryExecutionContext': {
            'Database': query_config['database']
        },
        'ResultConfiguration': {
            'OutputLocation': location
        }
    }
    response = athena.start_query_execution(
        **query_params
    )
    print(response)
    return response['QueryExecutionId']
```

**Note**

When copying code directly from the GitHub repository into your IPython or Jupyter environment, you can use the `%cpaste` magic command. This allows you to paste multi-line code blocks safely without indentation or formatting issues. To use it, type `%cpaste` and press *Enter*, then paste your code. When you're done, type `--` on a new line and press *Enter* to execute the pasted code. For the succeeding code blocks in this chapter, you may continue using `%cpaste` to avoid formatting or indentation errors when pasting multi-line snippets.

8. Define the `get_s3_output_path()` function to retrieve the Amazon S3 location where Amazon Athena stored the query results:

```
def get_s3_output_path(athena_execution_id):
    details = athena.get_query_execution(
        QueryExecutionId = athena_execution_id
    )
    print(details)
    execution = details['QueryExecution']
    configuration = execution['ResultConfiguration']
    return configuration['OutputLocation']
```

9. With the `execute_query()` and `get_s3_output_path()` functions defined in the previous steps, we can now run the following block of code to execute the SQL query and retrieve the S3 output path where the output data is stored:

```
athena_execution_id = execute_query(query)
output_path = get_s3_output_path(athena_execution_id)
output_path
```

10. Download a copy of the `output.csv` file from the S3 bucket:

```
!aws s3 cp {output_path} tmp/output.csv
```

11. Load and check the contents of the CSV file:

```
import pandas as pd
pd.read_csv("tmp/output.csv")
```

This should return a `DataFrame` with the following columns: `unique_id`, `statement`, `tag`, `is_spam`, and `date`.

| | unique_id | statement | tag | is_spam | date |
|---|---|---|---|---|---|
| 0 | 33d817a8-3524-49f2-ade7-7e77e0222bff | I love this! | POSITIVE | False | 2023-01-14 |
| 1 | 9dc21229-6503-4731-960c-355d3c71c4b4 | The product is okay. | NEUTRAL | False | 2023-02-04 |
| 2 | 81e97e8b-d08a-45ea-8437-2ecb912cccd7 | It works as expected. | NEUTRAL | False | 2023-02-15 |
| 3 | 0a3fce7a-cced-4e11-bb85-7df2b58850ab | This is awful. | NEGATIVE | False | 2023-03-09 |
| 4 | 69260043-190f-429e-b100-aa4c5fcdaa18 | This is the best purchase I've made. | POSITIVE | False | 2023-03-17 |
| .. | ... | ... | ... | ... | ... |
| 80 | 389b9e1c-c71a-4e5b-ba6a-1f23c54ff284 | Nothing special about it. | NEUTRAL | False | 2025-10-22 |
| 81 | d46e37b9-b298-4683-b4ef-440a62e75d51 | Great service! | POSITIVE | False | 2025-11-06 |
| 82 | 10d097db-e4b8-4ae0-a5cb-84027bf8aefb | Very disappointed with this. | NEGATIVE | False | 2025-11-13 |
| 83 | f76cc1f7-04c2-4177-849c-6250b7657793 | I haven't decided yet. | NEUTRAL | False | 2025-12-06 |
| 84 | a0299935-3c99-42b3-a029-338a213c7241 | I love this! | POSITIVE | False | 2025-12-18 |

[85 rows x 5 columns]

*Figure 5.3 — DataFrame containing the values returned by the SQL query*

Here, the `DataFrame` returned contains the values returned by the SQL query (`SELECT * FROM <TABLE>`) executed in Amazon Athena and stored in the specified Amazon S3 output location.

12. At this stage, you may realize that executing a query involves too many steps, which can make the process feel slow and repetitive. With this, let's define the `SQL()` function, which will help us run SQL queries easily without calling separate functions to execute the query or retrieve results from S3:

```
from time import sleep

def SQL(statement):
    print(statement)
    execution_id = execute_query(statement)
    print('Sleeping for 15 seconds')
    sleep(15)

    if 'DELETE' in statement:
        return None
    else:
        output_path = get_s3_output_path(execution_id)
        return pd.read_csv(output_path)
```

The `SQL()` function runs a SQL statement on Amazon Athena and takes care of getting the results after execution. It prints the statement, executes it, and waits 15 seconds for

the query to finish. If the statement is not a `DELETE` statement, it retrieves the results from S3 and returns them as a pandas DataFrame. Otherwise, it returns `None`.

> **Note**
>
> The `SQL()` function shown here is a simplified implementation intended for demonstration purposes. It assumes that the Amazon Athena query will complete and that the corresponding Amazon S3 output file will be available within 15 seconds. However, this is not guaranteed, as query execution time depends on factors such as data size, query complexity, and service load. In production environments, you should implement proper polling logic to check the query before attempting to retrieve the results from S3.

13. Using the `SQL()` function you defined in the previous step, run the following query to get all records with `is_spam = True`:

```
SQL(f"SELECT * FROM { table } WHERE is_spam = True")
```

After a few seconds, you'll get the following `DataFrame` of values:

| | unique_id | statement | tag | is_spam | date |
|---|---|---|---|---|---|
| 0 | 9a87329f-cc7e-4f2d-8b77-7e7d882d053a | Anyone interested? | NaN | True | 2025-04-11 |
| 1 | 9d26b019-66e1-4812-a84e-2cfc6bdd6ed2 | BUMP | NaN | True | 2025-05-26 |
| 2 | 8be3e617-bc90-4f0b-87c3-f711d5f9bd2a | Bump to top | NaN | True | 2025-08-23 |
| 3 | bfe68b20-84d6-4f29-aa76-bdae4477d8f3 | Anyone interested? | NaN | True | 2023-06-14 |
| 4 | fbb99021-48c8-4441-9864-5b5a526b0c10 | Just checking in | NaN | True | 2023-08-27 |
| 5 | c8e5be80-d999-4860-a0dc-d91949aace86 | Bump to top | NaN | True | 2023-11-16 |
| 6 | 9642c4e9-6ff6-4d1f-8176-fa361c9716cf | Anyone interested? | NaN | True | 2024-01-11 |
| 7 | 03dd6dd8-01ee-4ed0-ba14-8859822bfecf | BUMP | NaN | True | 2024-03-04 |
| 8 | 0c47bdf6-c336-4a29-9d50-25acbb217173 | Anyone interested? | NaN | True | 2024-05-06 |
| 9 | 7ccc67aa-5690-49e0-a1d6-05b92960ff26 | Bump to top | NaN | True | 2024-05-17 |
| 10 | 522d7110-f4ba-4950-bed4-7fe487e94413 | UP | NaN | True | 2024-05-23 |
| 11 | 8e0e7749-593c-4953-ac75-f1d6cc52545c | BUMP | NaN | True | 2024-06-03 |
| 12 | 3cd2a397-75ef-4ba2-b125-ca65a55e2f09 | Anyone interested? | NaN | True | 2024-06-21 |
| 13 | dd7acbbb-9fd4-4ce9-9988-f4291088d142 | Just checking in | NaN | True | 2024-08-19 |
| 14 | 5897f8d0-d450-486e-86b4-2032950bb977 | UP | NaN | True | 2024-11-09 |

*Figure 5.4 — Result of the query selecting all spam records*

Behind the scenes, when you call the `SQL()` function, it submits the `SELECT * FROM <TABLE NAME> WHERE is_spam = True` query to Amazon Athena using the defined `execute_query()` function. Athena executes the query against the specified dataset and stores the results in the configured S3 output location. After waiting briefly for the

query to complete, the function retrieves the result file from S3 and loads it into a pandas `DataFrame`, which is then returned to you.

14. In case you're wondering whether it's possible to modify or even delete table records, the answer is yes—depending on the table format and the permissions configured through AWS Lake Formation. With this, let's delete all records with `is_spam = True`:

```
SQL(f"DELETE FROM { table } WHERE is_spam = True")
```

> **Note**
>
> For more information on supported operations, considerations, and limitations when querying Apache Iceberg tables in Athena, refer to the following resource: `https://docs.aws.amazon.com/athena/latest/ug/querying-iceberg.html`.

15. Run the following query to validate whether the spam entries have been successfully deleted:

```
SQL(f"""SELECT COUNT(*) AS spam_count FROM { table } WHERE is_spam =
True""")
```

You should see `spam_count` of `0`, confirming that no spam records remain in the table.

16. Let's retrieve all records from the table as well:

```
df = SQL(f"SELECT * FROM { table }")
print(df)
```

The result should look similar to the following output:

```
                              unique_id                                statement       tag  is_spam        date
0   fe9d38cc-ee40-4260-bb94-7e2c93200391                              I love this!  POSITIVE    False  2023-01-14
1   e9918b18-ecf9-41cd-b35c-efadc64d09bc                       The product is okay.   NEUTRAL    False  2023-02-04
2   9d9840af-2136-49c4-9159-45ecd73f31df                     It works as expected.   NEUTRAL    False  2023-02-15
3   0d53cee8-1836-4240-8f32-4b40b66bf027                             This is awful.  NEGATIVE    False  2023-03-09
4   a163aec4-b977-47bc-9b13-fb0f2f2fa8e3  This is the best purchase I've made.  POSITIVE    False  2023-03-17
..                                   ...                                      ...       ...      ...         ...
80  6e1656f9-4b0d-4417-ac1a-7940a54c7b43                    Nothing special about it.   NEUTRAL    False  2025-10-22
81  fd62901c-4c1c-4aab-995b-3938c9c493bd                                Great service!  POSITIVE    False  2025-11-06
82  b61caf5b-eb42-497c-bad0-84e626f41f5e                Very disappointed with this.  NEGATIVE    False  2025-11-13
83  6a1d2c41-adb6-4fc4-ab99-f5960cc5657e                      I haven't decided yet.   NEUTRAL    False  2025-12-06
84  e82ede36-13a9-4df4-aeed-756dae398862                              I love this!  POSITIVE    False  2025-12-18

[85 rows x 5 columns]
```

*Figure 5.5 — DataFrame containing all records from the table*

With the spam entries removed in the previous step, the `DataFrame` (`df`) now contains only the non-spam records.

> **Note**
>
> Removing spam entries is important because they can negatively impact analysis or make models less accurate. For example, if you are looking at customer comments on e-commerce product pages, spam messages can distract you from real comments. By removing them, your data stays clean, which improves any analysis or model you build.

17. Save the retrieved records to a CSV file (`sentiments_data.csv`) for later use:

```
df.to_csv("tmp/sentiments_data.csv", index=False)
```

18. Exit the REPL environment:

```
exit
```

19. Create a new S3 general-purpose bucket, which will be used to store the generated CSV file:

```
UPLOAD_BUCKET="<NAME OF NEW S3 BUCKET>"
aws s3 mb s3://$UPLOAD_BUCKET
```

> **Note**
>
> You may reuse the S3 bucket defined in the `ATHENA_OUTPUT_LOCATION` bash variable instead of creating a new one. You can use the S3 bucket name (`TAG:ATHENA_OUTPUT_BUCKET`) from your local code editor if you saved it there earlier.

20. Finally, upload the generated CSV file (`sentiments_data.csv`) to the S3 bucket:

```
aws s3 cp tmp/sentiments_data.csv s3://$UPLOAD_BUCKET/
```

Be sure to note down the bucket name, as you'll need this when we ingest data into a SageMaker feature store.

**Note**

You should store the bucket name (`TAG:UPLOAD_BUCKET`) in a text file using a code editor on your local machine so you can easily reference it in the next section.

At this point, you should have a good understanding of how to use Amazon Athena to query your data. You've seen how to run SQL queries using both the AWS CLI and Boto3, giving you the flexibility depending on whether you're working interactively or programmatically. With these approaches, you can quickly explore, validate, and analyze data stored in Amazon S3 without managing any infrastructure. While the examples in this chapter use a relatively small dataset, Amazon Athena and Amazon S3 table buckets are designed to scale and can perform efficiently even when working with much larger datasets. This makes Athena a convenient and powerful tool for ad hoc analysis and for integrating SQL queries into automated workflows.

## Ingesting data into a SageMaker feature store

A feature store helps you store, manage, and reuse machine learning features so the same feature definitions and transformations are used consistently during both training and inference. It fits into your ML workflow after feature engineering pipelines generate features and before models are trained or deployed for inference, allowing you and other team members (such as data scientists, data engineers, and ML engineers) to easily discover, share, and reuse trusted features.

Let's review a few additional key concepts that will be used throughout this section and the rest of this chapter:

- **Feature**: A feature is an individual attribute or variable that represents a measurable property of the data used by machine learning models. Examples include values such as a user ID, timestamp, purchase amount, or sentiment score.
- **Feature metadata**: Feature metadata is descriptive information associated with a feature that provides context about the feature's meaning and usage. This may include properties such as the feature description, data type, source dataset, or other custom attributes that help teams discover and manage features.
- **Feature store**: A feature store is a centralized system that stores, manages, and serves machine learning features for both training and inference. It enables consistent feature usage across models while supporting capabilities such as feature discovery, metadata management, and controlled access to features.

Feature stores can include offline stores for training datasets and online stores for low-latency inference, and they are particularly useful when features are shared across multiple models or

must remain consistent between training and production environments. SageMaker Feature Store simplifies this process by providing a fully managed service that supports feature ingestion, enables low-latency access for inference, and integrates easily with other AWS analytics and machine learning services.

**Note**

Offline feature stores are typically used for batch workloads, such as training or validating machine learning models, where completeness, historical accuracy, and the ability to recompute features over long time ranges are critical. After ingesting several months of user activity into the offline store, a data scientist can generate features from this historical data in a data lake to train a recommendation model. Online feature stores, in contrast, support real-time inference, where low-latency access to the most recent feature values is essential. After ingesting a user's latest transactions into the online store, a fraud detection system can query these features in milliseconds to evaluate risk and approve or block a payment.

Using a feature store provides several advantages for managing and operationalizing machine learning features:

- **Consistency between training and inference**: Ensures that the same feature definitions and transformations are used in both model training and production predictions
- **Feature reuse across teams and models**: Allows teams to discover and reuse existing features instead of recreating them in multiple pipelines
- **Faster model development**: Reduces the time spent rebuilding feature engineering workflows for each new model
- **Improved collaboration**: Provides a centralized location where teams can find, share, and manage trusted features
- **Operational reliability**: Supports feature versioning and standardized access patterns, helping maintain stable and reproducible ML pipelines

In this section, you will ingest data into a SageMaker feature store to centralize and manage machine learning features for consistent use across training and inference. More specifically, you'll set up and configure both an online feature store and an offline feature store and ingest data, with a later section near the end of the chapter dedicated to retrieving data from both stores. While this chapter does not cover all the steps required for complete end-to-end training and inference workflows, it illustrates how the feature store fits into the broader machine learning lifecycle.

## Setting up a SageMaker notebook instance

To interact with SageMaker and the feature store, you'll use a SageMaker notebook instance, which provides a fully managed Jupyter environment in the cloud. This notebook allows you to run code, explore data, and test your machine learning workflows without managing infrastructure. Using the notebook, you can easily connect to feature stores, ingest data, and later retrieve data from the feature stores.

Let's proceed with setting up the SageMaker notebook instance:

1. Type `sagemaker ai` in the search bar and choose **Amazon SageMaker AI** from the list of results.

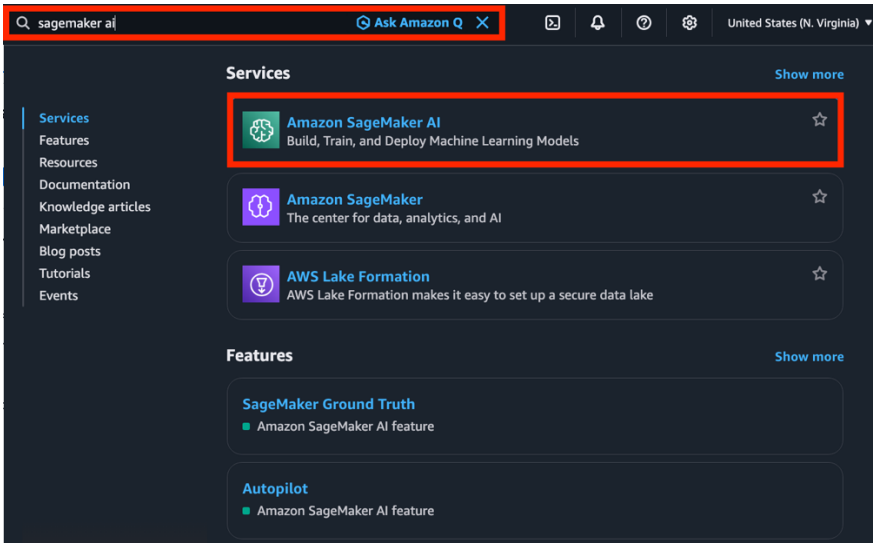



*Figure 5.6 — Navigating to the Amazon SageMaker AI console*

   This will redirect you to the Amazon SageMaker AI console, where you can manage machine learning workflows, including building, training, and deploying models.

2. In the navigation pane (left sidebar), click **Notebooks** under **Applications and IDEs**. This should open a page that shows all created notebook instances (if any).
3. On the **Notebooks and Git repos** page, click **Create notebook instance**.
4. Under **Notebook instance name**, specify a notebook instance name (for example, `feature-store-notebook-instance`).
5. Under **Notebook instance type**, choose `ml.t3.large` from the list of instance types in the drop-down menu.

6. For **Platform identifer**, choose `Amazon Linux 2023, Jupyter Lab 4`.
7. Under **Permissions and encryption**, choose `SageMakerAdminRole` for the **IAM role** value.
8. For **Root access**, choose `Enable`.
9. Click on the **Create notebook instance** button after the last section of the form to proceed with the notebook creation process.

> **Note**
>
> This may take 5 to 10 minutes to complete. Feel free to grab a cup of coffee or tea while waiting.

10. Once the status changes to **InService**, click **Open JupyterLab**. Clicking **Open JupyterLab** will open a new tab displaying a page similar to the one shown in *Figure 5.7*.

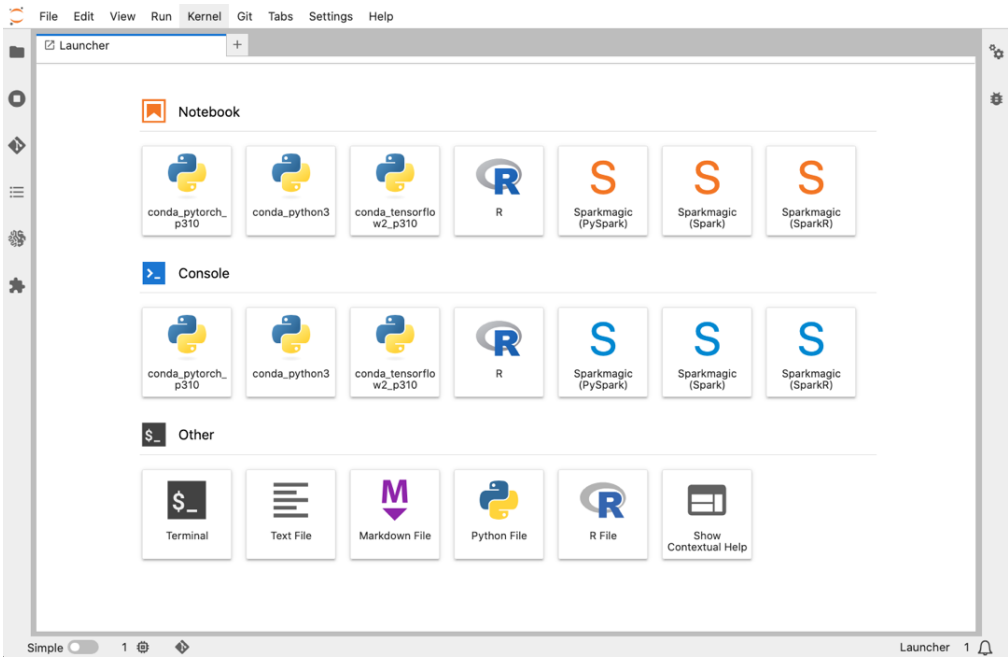



*Figure 5.7 — JupyterLab App Launcher*

Here, you can launch notebooks, open interactive consoles, and use other tools available in the JupyterLab workspace.

11. In the Launcher, start a new terminal session by choosing **Terminal** (under **Other**). Alternatively, you can start a new terminal session through the **File** menu by selecting **File** | **New**, then choosing **Terminal**.

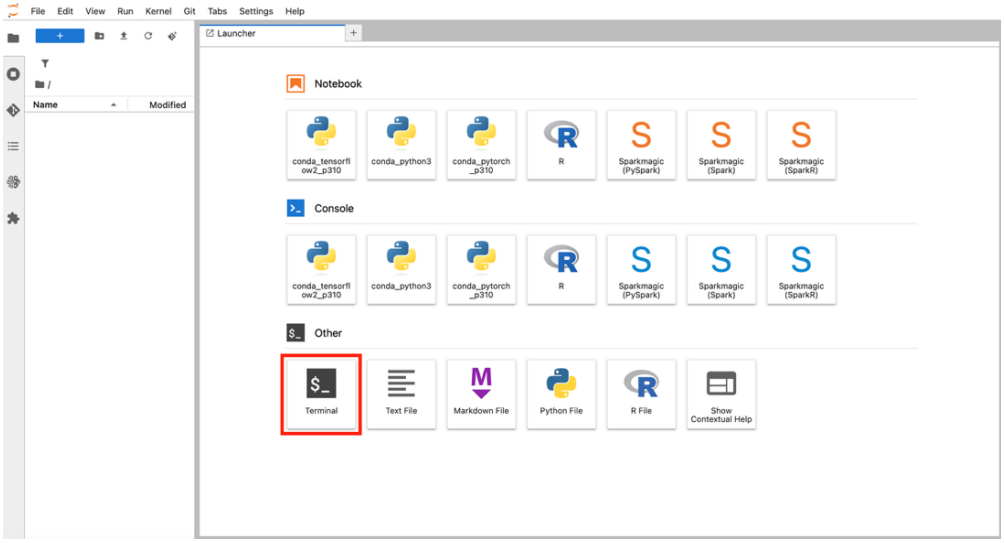



*Figure 5.8 — Launching a new terminal*

Inside this terminal, you will run the commands provided in the subsequent steps to download the dataset, which will be used for feature store ingestion.

12. Set the bucket name by replacing `<NAME OF EXISTING S3 BUCKET>` with the name of the S3 bucket where `sentiments_data.csv` was uploaded. Then run the following command to download `sentiments_data.csv` from the bucket:

```
UPLOAD_BUCKET="<NAME OF EXISTING S3 BUCKET>"
aws s3 cp s3://$UPLOAD_BUCKET/sentiments_data.csv .
```

Make sure to replace `<NAME OF EXISTING S3 BUCKET>` with the name of the general-purpose bucket you created in *Running SQL queries in Amazon Athena* before running the command.

**Note**

You can use the S3 bucket name (`TAG:UPLOAD_BUCKET`) from your local code editor if you saved it there earlier. You can also retrieve the bucket name from

> the Amazon S3 console. Note that the general-purpose bucket contains the uploaded CSV file (`sentiments_data.csv`).

13. Copy the downloaded `sentiments_data.csv` file to the `SageMaker` directory:

```
cp sentiments_data.csv SageMaker/
```

14. Open a new Launcher by selecting **File | Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **conda_pytorch_p310** (under **Notebook**).

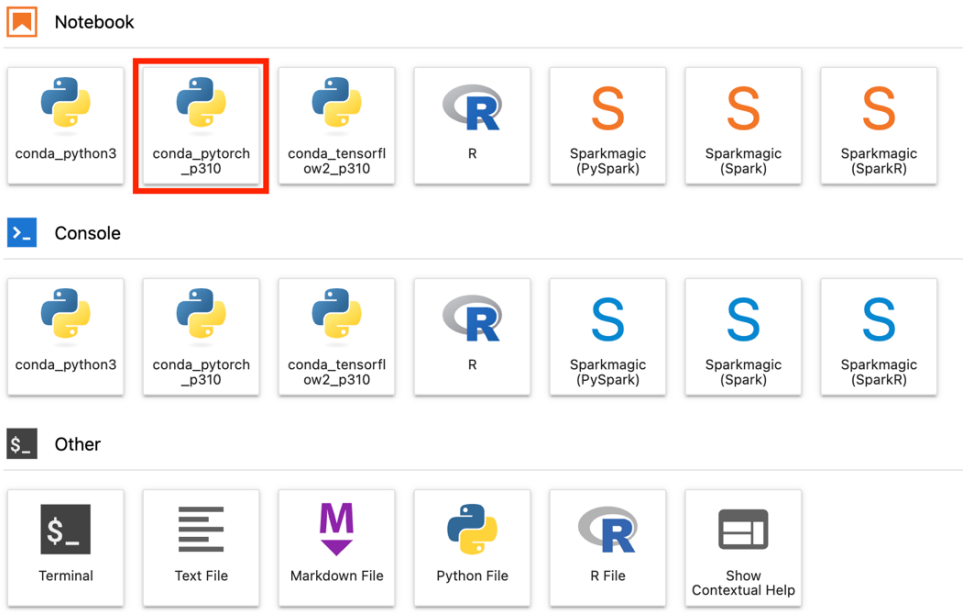



*Figure 5.9 — Launching a new notebook (using conda_pytorch_p310)*

Be sure to rename the notebook to `feature-store.ipynb` before proceeding. We will run the code in the following steps within this notebook.

15. To ensure compatibility with the hands-on examples in this chapter, uninstall any existing version of the SageMaker Python SDK and install version `3.5.0`.

```
%pip uninstall -y sagemaker
%pip install "sagemaker==3.5.0"
```

Since the installation process generates a lengthy output in the notebook cell, you may right-click the output area and select **Enable Scrolling for Outputs** to make the log output easier to review.

After the installation completes, you should see a message similar to the following in the generated log output:

```
...
Successfully installed sagemaker-3.5.0
Note: you may need to restart the kernel to use updated packages.
```

After successfully installing the specified package versions, restart the kernel by selecting **Kernel | Restart Kernel** from the menu.

> **Note**
>
> This will ensure the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

16. Suppress specific warnings related to namespace conflicts by filtering out warnings that match a given message pattern.

```
import warnings

m = "Field .* has conflict with protected namespace"

warnings.filterwarnings(
    "ignore",
    message=m
)
```

17. Now, import the required classes and helper functions from the SageMaker Python SDK.

```
from sagemaker.mlops.feature_store import (
    FeatureGroup,
    FeatureMetadata,
    OnlineStoreConfig,
    OfflineStoreConfig,
    S3StorageConfig,
```

```
    load_feature_definitions_from_dataframe,
    ingest_dataframe,
    create_athena_query,
)
```

These will be used later in this chapter to define the feature group, configure the online and offline stores, load feature definitions from a dataframe, ingest records, and run queries against the offline store.

At this stage, your SageMaker notebook instance is ready, and the specific version of the SageMaker Python SDK has been installed to support the hands-on exercises in this chapter.

## Preparing the data to be ingested into the feature store

In this section, we will clean and preprocess the dataset by dropping the `is_spam` column, adding an `event_time` feature, and simulating dataset updates by changing `NEUTRAL` sentiment tags to `POSITIVE`.

Let's now proceed with preparing and processing the data before ingesting it into the feature store:

1. In a new code cell of the Jupyter notebook, run the following to load the `sentiments_data.csv` data into a `DataFrame`:

```
import pandas as pd
df = pd.read_csv("sentiments_data.csv")
```

2. Drop the `is_spam` column:

```
df = df.drop('is_spam', axis=1)
```

3. Define the `get_event_time()` function, which converts a date string into a timestamp:

```
from datetime import datetime

def get_event_time(date_string):
    obj = datetime.strptime(date_string, '%Y-%m-%d')
    output = float(obj.timestamp())
    return output
```

4. Using the `get_event_time()` function defined in the previous step, retrieve the event time for the given date (`2024-01-01`) and store it in the `event_time` column of the `DataFrame`:

```
first_event_time = get_event_time('2024-01-01')
df['event_time'] = first_event_time
```

   Here, `event_time` helps determine the most recent version of a feature by identifying the record with the latest timestamp. If there are two records with the same identifier, the one with the most recent `event_time` value will be selected as the latest version.

5. Save the `DataFrame` data to a CSV file named `feature_store_01.csv`:

```
df.to_csv("feature_store_01.csv", index=False)
```

6. Now, let's simulate a real-world scenario where data processing and cleaning were applied to the first version of the dataset to enhance the tagging system and ensure more accurate sentiment labels. Run the following block of code to replace all `NEUTRAL` tag values with `POSITIVE`:

```
second_event_time = get_event_time('2025-01-01')

for index, row in df.iterrows():
    if row['tag'] == 'NEUTRAL':
        new_tag = 'POSITIVE'
        df.at[index, 'tag'] = new_tag
        df.at[index, 'event_time'] = second_event_time
```

   Note that for the modified rows, the `event_time` value is also updated accordingly.

7. Save the `DataFrame` data to a CSV file named `feature_store_02.csv`:

```
df.to_csv("feature_store_02.csv", index=False)
```

8. Open the `feature_store_01.csv` file from the file browser.

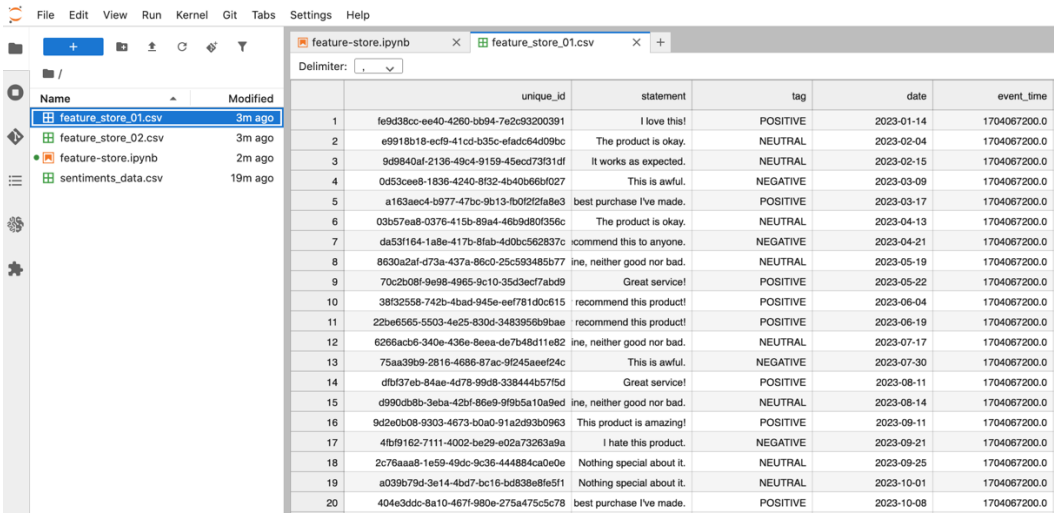



*Figure 5.10 — Opening the feature_store_01.csv file from the file browser*

Here, `feature_store_01.csv` contains the original data, still supporting `NEUTRAL` tag values.

9. Finally, open the `feature_store_02.csv` file from the file browser.

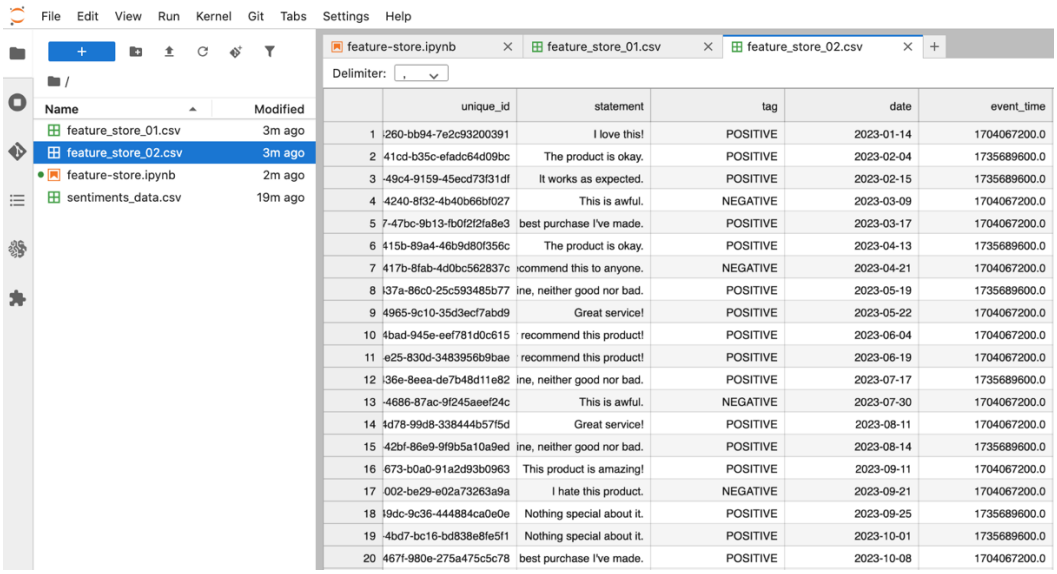



*Figure 5.11 — Opening the feature_store_02.csv file from the file browser*

As shown in *Figure 5.11*, `feature_store_02.csv` contains the modified data, with records having `NEUTRAL` tag column values updated to `POSITIVE`.

Keep in mind that this is an oversimplified example, and real-world data processing may involve more complex transformations. In a more realistic scenario, for example, the `NEUTRAL` tag value would be updated to either `POSITIVE` or `NEGATIVE` based on an additional score column that indicates the sentiment polarity or sentiment score, which determines whether the sentiment is positive or negative. For the purposes of this chapter, however, this simplified transformation is sufficient as we proceed with ingesting the data into the feature store.

## Ingesting data into a feature store

In this section, we'll ingest the processed data into a feature store. Before you can ingest data into SageMaker Feature Store, you'll first need to define the features and then create both the online and offline feature stores. This involves setting up the schema for your features, determining which features will reside in each store, and ensuring the data is properly structured for ingestion. The SageMaker Python SDK provides methods that simplify these steps to make it easy for you to define features, create stores, and ingest data quickly. Once the feature stores are created and the feature definitions are loaded, you can begin ingesting data into both the online and offline stores for use in training and inference.

Let's walk through the steps required to ingest the data into a feature store:

1. Switch back to the `feature-store.ipynb` Jupyter notebook.
2. In a new code cell of the Jupyter notebook, run the following code to initialize a few prerequisites, including the Boto3 clients for interacting with the relevant SageMaker APIs:

```
from time import sleep
from boto3 import Session as BotoSession

boto_session = BotoSession()
region = boto_session.region_name
client = boto_session.client

c1 = client(service_name='sagemaker',
            region_name=region)

service_name='sagemaker-featurestore-runtime'
c2 = client(service_name=service_name,
            region_name=region)
```

More specifically, this code initializes the `SageMaker` client (`c1`) for general machine learning operations and the `SageMakerFeatureStoreRuntime` client (`c2`) for accessing real-time feature data from the online feature store.

3. Load the CSV files into their respective `DataFrames`:

```
import pandas as pd
df_01 = pd.read_csv("feature_store_01.csv")
df_02 = pd.read_csv("feature_store_02.csv")
```

4. Load the feature definitions using the `load_feature_definitions_from_dataframe()` function:

```
defs = load_feature_definitions_from_dataframe(df_01)
```

Here, you load the feature definitions from the provided `DataFrame` (`df_01`) into the `FeatureGroup` object, which sets up the feature group schema and prepares it for data ingestion into SageMaker Feature Store.

5. Define the Amazon S3 bucket paths used for storing feature store input and output data:

```
import boto3
sts = boto3.client("sts")
account_id = sts.get_caller_identity()["Account"]

s3_bucket_name = f"sagemaker-{region}-{account_id}"
s3_input = f"s3://{s3_bucket_name}/store/input"
s3_output = f"s3://{s3_bucket_name}/store/output"
```

6. Retrieve the IAM role ARN using the `get_execution_role()` function, which is associated with the SageMaker notebook instance.

```
from sagemaker.core.helper.session_helper import (
    get_execution_role
)

role_arn = get_execution_role()
```

7. Create the feature group (`sentiments-feature-group`) by executing the following code block:

```
group_name = "sentiments-feature-group"

FeatureGroup.create(
    feature_group_name=group_name,
    feature_definitions=defs,
    record_identifier_feature_name="unique_id",
    event_time_feature_name="event_time",
    role_arn=role_arn,
    online_store_config=OnlineStoreConfig(
        enable_online_store=True
    ),
    offline_store_config=OfflineStoreConfig(
        s3_storage_config=S3StorageConfig(
            s3_uri=s3_input
        )
    ),
)
```

Here, the feature group is created by specifying the feature definitions, record identifier feature name, and event time feature name. The feature definitions describe the schema of the feature store, while the record identifier feature name uniquely identifies each record in the feature group. The offline store is configured to persist feature data in the S3 location specified by the `s3_uri` parameter. Enabling the online store by setting `enable_online_store=True` allows features to be retrieved with low-latency access, which is useful for real-time inference scenarios.

**Note**

Although this chapter does not cover full inference endpoint deployment, you will later simulate record retrieval from the online feature store to demonstrate how online feature access works.

8. Define the `get_status()` function to retrieve the current status of the feature group, which should help you automate status checks and efficiently manage the lifecycle of feature groups:

```
def get_status(group_name="sentiments-feature-group"):
    fg = FeatureGroup.get(
        feature_group_name=group_name
    )
    return fg.feature_group_status

print(get_status())
```

The `get_status()` function retrieves the status of a feature group by calling the `FeatureGroup.get()` method on the specified feature group name. It then returns the value of the `feature_group_status` attribute, which indicates the current lifecycle state of the feature group. The status value may be one of the following: `Creating`, `Created`, `CreateFailed`, `Deleting`, or `DeleteFailed`.

9. Using the `get_status()` function you defined in the previous step, continuously check and wait for the feature group to finish being created:

```
while get_status() == "Creating":
    print("Waiting...")
    sleep(10)

print(get_status())
```

You should get an output similar to the following:

```
Waiting...
Waiting...
Created
```

10. Use the `ingest_dataframe()` method twice to load the records stored in `df_01` and `df_02` to the feature store.

```
ingest_dataframe(
    feature_group_name="sentiments-feature-group",
    data_frame=df_01,
    max_workers=3,
```

```
    wait=True,
)

ingest_dataframe(
    feature_group_name="sentiments-feature-group",
    data_frame=df_02,
    max_workers=3,
    wait=True,
)
```

In this step, the `ingest_dataframe()` method is used to load the data from the pandas `DataFrames` (`df_01` and `df_02`) into the feature store. By setting `max_workers=3`, we allow up to three parallel threads to handle the ingestion, which helps speed up the process.

Note that it may take approximately 10-15 minutes for the ingested data to be reflected in the offline store. While waiting for this process to complete, we'll explore other features of SageMaker Feature Store in the next section, such as adding searchable metadata to features. We will also look at how to search for features in an existing feature group using the metadata to quickly find relevant information.

# Adding searchable metadata to features

When working with feature stores, metadata plays an important role in helping teams organize and discover features efficiently. By attaching descriptive metadata, features become easier to search, interpret, and reuse across multiple machine learning workflows. As the number of features, datasets, and feature groups grows, locating the right features can become increasingly challenging without proper metadata.

Metadata provides important context about each feature, such as its description, data type, source system, or creation time. This information allows teams to quickly understand what a feature represents and how it can be used. For example, an ML engineer or data scientist might search for features originating from a CRM system and use the metadata to identify customer interaction features, such as `last_contacted` or `account_status`, without manually inspecting each dataset. By adding searchable metadata, you can more easily filter and locate features with specific characteristics across a large feature store. This also improves overall feature management by making it easier to organize, maintain, and update feature groups over time. In this section, you will add searchable metadata to the features and verify that they can be discovered using the metadata.

**Note**

You'll be running the code in this section directly within the Jupyter notebook. Be sure to run the code cells in sequence to ensure proper execution and that everything works as expected.

With this in mind, let's proceed with updating the feature metadata:

1. In a new code cell of the Jupyter notebook, run the following code to update the description of the `unique_id` feature:

```
metadata = FeatureMetadata.get(
    feature_group_name="sentiments-feature-group",
    feature_name="unique_id"
)
metadata.update(
    description="UUID of the record"
)
```

2. Let's try that again, but this time with the `parameter_additions` parameter configuration specified:

```
from sagemaker.mlops.feature_store import (
    FeatureParameter
)

metadata.update(
    description="UUID of the record",
    parameter_additions=[
        FeatureParameter(
            key="generated",
            value="true"
        ),
        FeatureParameter(
            key="another-key",
            value="another-value"
        ),
    ]
)
```

Here, we are updating the metadata for the `unique_id` feature by specifying additional parameters using the `parameter_additions` argument. This argument allows us to attach custom key-value pairs (such as `"generated": "true"` and `"another-key": "another-value"`) to the feature's metadata.

> **Note**
>
> These parameters can be used to add context or other relevant information that can be searched and accessed later. For example, you might use them to track the source of a feature, such as whether it was generated by a model or an external system. This can be particularly useful for auditing or understanding data lineage, as it helps you trace where each feature comes from.

3. Prepare a dictionary with the descriptions of each field and update the feature metadata using the `descriptions` dictionary from the previous step:

```
descriptions = {
    "unique_id": "UUID of the record",
    "statement": "Statement or comment of a user",
    "tag": "Sentiment tag based on score",
    "date": "When the comment was shared",
    "event_time": "Feature store event time value",
}

for feature_name, desc in descriptions.items():
    meta = FeatureMetadata.get(
        feature_group_name="sentiments-feature-group",
        feature_name=feature_name
    )
    meta.update(description=desc)
```

4. Perform your first feature search query using the `search()` method of the SageMaker client instance (`c1`):

```
c1.search(Resource="FeatureMetadata")
```

This query will search for feature metadata across your feature groups. The following output will be returned, showing five features that match the search criteria:

```
 {'Results': [
   {'FeatureMetadata': {'FeatureGroupArn': '... ',
     'FeatureGroupName': 'sentiments-feature-group',
     'FeatureName': 'event_time',
     'FeatureType': 'Fractional',
      ...
     'Description': 'Feature store event time value',
     'Parameters': []}},
   {'FeatureMetadata': {'FeatureGroupArn': '...',
     'FeatureGroupName': 'sentiments-feature-group',
     'FeatureName': 'statement',
     'FeatureType': 'String',
       ...
     'Description': 'Statement or comment of a user',
     'Parameters': []}},
    ...
   ],
  'TotalHits': {'Value': 5, 'Relation': 'EqualTo'},
  'ResponseMetadata': {...}
 }
```

5. Now, let's search for features where `FeatureType` is `Fractional`:

```
c1.search(
    Resource="FeatureMetadata",
    SearchExpression={'Filters': [
        {'Name': 'FeatureType',
         'Operator': 'Equals',
         'Value': 'Fractional'}
    ]}
)
```

This returns the following output, showing that only one feature matches the search criteria:

```
 {'Results': [
   {'FeatureMetadata': {'FeatureGroupArn': '...',
     'FeatureGroupName': 'sentiments-feature-group',
```

```
      'FeatureName': 'event_time',
      'FeatureType': 'Fractional',
       ...
      'Description': 'Feature store event time value',
      'Parameters': []}},
   'TotalHits': {'Value': 1, 'Relation': 'EqualTo'},
   'ResponseMetadata': {...}
  }
```

The `TotalHits` value of `1` indicates that only one feature, `'event_time'`, matches the specified `FeatureType` of `'Fractional'`.

6. Let's search for features with a `Description` value that contains the word `'comment'`:

```
c1.search(
    Resource="FeatureMetadata",
    SearchExpression={'Filters': [
        {'Name': 'Description',
         'Operator': 'Contains',
         'Value': 'comment'}
    ]}
)
```

This returns the following output:

```
  {'Results': [
    {'FeatureMetadata': {'FeatureGroupArn': '...',
      'FeatureGroupName': 'sentiments-feature-group',
      'FeatureName': 'statement',
      'FeatureType': 'String',
       ...
      'Description': 'Statement or comment of a user',
      'Parameters': []}},
    {'FeatureMetadata': {'FeatureGroupArn': '...',
      'FeatureGroupName': 'sentiments-feature-group',
      'FeatureName': 'date',
      'FeatureType': 'String',
       ...
      'Description': 'When the comment was shared',
      'Parameters': []}}],
   'TotalHits': {'Value': 2, 'Relation': 'EqualTo'},
```

```
  'ResponseMetadata': {...}
 }
```

The `TotalHits` value of 2 indicates that two features, `'statement'` and `'date'`, match the specified `Description` containing the word `'comment'`. Note that the search uses the `'Contains'` operator, meaning it will return any feature where `Description` includes the word `'comment'`, regardless of its position in the text.

7. Let's search for features based on specific parameters:

```
c1.search(
    Resource="FeatureMetadata",
    SearchExpression={'Filters': [
        {'Name': 'Parameters.generated',
         'Operator': 'Equals',
         'Value': 'true'}
    ]}
)
```

This query searches for features where `Parameters` includes a key-value pair with `'generated': 'true'`. The following output will be returned, showing 1 feature that matches the search criteria:

```
 {'Results': [
   {'FeatureMetadata': {'FeatureGroupArn': '...',
     'FeatureGroupName': 'sentiments-feature-group',
     'FeatureName': 'unique_id',
     'FeatureType': 'String',
      ...
     'Description': 'UUID of the record',
     'Parameters': [
       {'Key': 'generated',
        'Value': 'true'},
       {'Key': 'another-key',
        'Value': 'another-value'}
     ]}
   }],
  'TotalHits': {'Value': 1, 'Relation': 'EqualTo'},
  'ResponseMetadata': {...}
 }
```

*Wasn't that easy?* Searching features in the feature store is straightforward and generally requires just a few lines of code. When working with feature stores, it helps to keep things organized by giving features clear names, grouping related ones, and adding useful metadata such as descriptions, data types, and source systems. If your feature store metadata clearly explains what each feature represents, where it originates, how frequently it is updated, and any important nuances, you can easily locate and use the right features without manually inspecting datasets.

## Retrieving data from the online and offline feature stores

Feature stores play a critical role in ensuring consistent and reliable access to features across both training and inference workflows. However, retrieving features for offline analysis and real-time serving involves different requirements and design considerations:

- Offline feature retrieval is typically optimized for batch access, where throughput, completeness, and historical accuracy are more important than low latency. These workloads often require capabilities such as point-in-time correctness, historical backfills, and the ability to recompute features over extended time ranges for model training, validation, and experimentation. For this reason, offline feature stores are commonly built on data warehouses or data lakes that prioritize scalability, cost efficiency, and reproducibility.
- Online feature retrieval, on the other hand, supports real-time inference scenarios where features must be available with minimal latency and high reliability. Online stores are designed for fast key-based lookups, predictable performance, and consistent access to the most recent feature values. Achieving this typically requires specialized low-latency storage systems, caching mechanisms, and coordination with the offline store to keep feature values synchronized.

These differences introduce trade-offs related to system complexity, data duplication, and operational overhead. A well-designed feature store helps manage these trade-offs by providing a unified layer for feature definitions and access, reducing training-serving skew and enabling safe feature reuse across both offline and online workflows. As you probably know by now, Amazon SageMaker Feature Store helps simplify this architecture by allowing you to store and retrieve features for both training and real-time inference using a managed service. In this section, you'll see how the SageMaker Python SDK makes it easier to retrieve features from both the online and offline stores of Amazon SageMaker Feature Store.

**Note**

You'll be running the code in this section directly within the same notebook used in the previous sections. Be sure to run the code cells in sequence to ensure proper execution and that everything works as expected.

Now, let's proceed with retrieving data from the online and offline feature stores:

1. In a new code cell of the Jupyter notebook (`feature-store.ipynb`), run the following to get the `unique_id` column value of the first row of `df_01`:

```
first_id = df_01.iloc[0].unique_id
```

2. Inspect the unique identifier value stored in `first_id`:

```
first_id
```

This should return a UUID value similar to the following:

```
'33d817a8-3524-49f2-ade7-7e77e0222bff'
```

3. Let's retrieve the latest value of the record from the online feature store:

```
feature_group = FeatureGroup.get(
    feature_group_name="sentiments-feature-group"
)

response = feature_group.get_record(
    record_identifier_value_as_string=str(first_id)
)
```

Here, we used `get_record()` to retrieve the most recent version of a record from the online feature store, using the `unique_id` value of the record.

4. Use `pprint()` to inspect the feature store record:

```
from pprint import pprint

clean = {
    "expires_at": response.expires_at,
```

```
    "record": [
        { "feature": f.feature_name,
          "value": f.value_as_string }
        for f in response.record
    ]
}

pprint(
    clean,
    indent=2,
    width=60,
    compact=True,
    sort_dicts=False
)
```

This returns the following output:

```
{ 'expires_at': Unassigned(),
  'record': [ { 'feature': 'unique_id',
                'value': '...'},
              { 'feature': 'statement',
                'value': 'I love this!'},
              {'feature': 'tag', 'value': 'POSITIVE'},
              {'feature': 'date', 'value': '...'},
              { 'feature': 'event_time',
                'value': '...'}]}
```

Using this formatted output, you can quickly inspect the features of the record, such as `statement`, `tag`, `date`, and `event_time`.

**Note**

From this point, the record retrieved from the online feature store can be used as input for machine learning inference. In practice, the retrieved features would typically be sent to a deployed model endpoint or prediction pipeline to generate real-time predictions, since the online feature store is designed for low-latency feature retrieval during inference workflows.

5. Before running queries against the offline feature store, let's examine the Parquet files that contain the stored feature data.

```
offline_config = feature_group.offline_store_config
storage_config = offline_config.s3_storage_config
config_s3_uri = storage_config.resolved_output_s3_uri
!aws s3 ls { config_s3_uri } --recursive
```

This block of code lists the Parquet files stored in the S3 bucket, along with additional information such as the file size and timestamp of each file. You will see output similar to the following:

```
  ...
  2026-01-01 00:00:00       2190 store/input/...parquet
  2026-01-01 00:00:00       2161 store/input/...parquet
```

**Note**

If you do not get any results after running the previous query, please wait for about 10-15 minutes and then try again.

6. Create an `AthenaQuery` instance using the `create_athena_query()` function.

```
from sagemaker.core.helper.session_helper import (
    Session
)

session = Session()
group_name = "sentiments-feature-group"

query = create_athena_query(
    group_name,
    session=session
)
```

7. Inspect the internal state of the `AthenaQuery` instance (`query`) using `vars()` to view its attributes:

```
pprint(vars(query))
```

This should display key internal attributes of the `AthenaQuery` instance, which include the catalog name, database name, session, and the generated offline store table name associated with the feature group.

```
{'catalog': 'AwsDataCatalog',
 'database': 'sagemaker_featurestore',
 'sagemaker_session': ...,
 'table_name': 'sentiments_feature_group_1772694083'}
```

8. Retrieve the table name from the `AthenaQuery` instance:

```
table = query.table_name
```

9. Define the `SQL` function that executes the specified Athena query and returns the results as a pandas `DataFrame`:

```
def SQL(
    statement: str,
    feature_group_name="sentiments-feature-group",
    session=session,
    output_location=s3_output,
    no_df=False
):
    print("---" * 20)
    print(f"QUERY: {statement}")
    print("---" * 20)

    query = create_athena_query(
        feature_group_name,
        session=session
    )

    query.run(
        query_string=statement,
        output_location=output_location
    )

    query.wait()

    if not no_df:
```

```
            return query.as_dataframe()

        return None
```

This function simplifies querying the offline feature store by abstracting the Athena query execution workflow. It allows you to run SQL statements, monitor query progress, and retrieve results in `DataFrame` format for analysis. Instead of writing multiple lines of code, you can perform the entire workflow by calling the `SQL()` function once.

10. Using the `SQL()` function defined in the previous step, run the following query to check if the offline store data is ready:

```
SQL(f"SELECT * FROM { table }")
```

This returns the following `DataFrame`:

| | unique_id | statement | tag | date | event_time | write_time | api_invocation_time | is_deleted |
|---|---|---|---|---|---|---|---|---|
| 0 | 69d4c9d5-f3bd-4f03-b764-8ed89932e501 | It works as expected. | NEUTRAL | 2025-05-15 | 1.704067e+09 | 2026-01-03 05:05:47.598 | 2026-01-03 04:59:34.000 | False |
| 1 | c1835285-69a9-47d0-86bc-c09e5c749047 | Great service! | POSITIVE | 2024-11-25 | 1.704067e+09 | 2026-01-03 05:05:47.598 | 2026-01-03 04:59:34.000 | False |
| 2 | c1835285-69a9-47d0-86bc-c09e5c749047 | Great service! | POSITIVE | 2024-11-25 | 1.704067e+09 | 2026-01-03 05:05:47.598 | 2026-01-03 04:59:36.000 | False |
| 3 | 6e1656f9-4b0d-4417-ac1a-7940a54c7b43 | Nothing special about it. | POSITIVE | 2025-10-22 | 1.735690e+09 | 2026-01-03 05:05:47.605 | 2026-01-03 04:59:35.000 | False |
| 4 | 22be6565-5503-4e25-830d-3483956b9bae | Highly recommend this product! | POSITIVE | 2023-06-19 | 1.704067e+09 | 2026-01-03 05:05:47.566 | 2026-01-03 04:59:34.000 | False |
| ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 165 | ace0e078-df04-4f3b-9abb-f516fd764f0b | Terrible experience. | NEGATIVE | 2024-10-19 | 1.704067e+09 | 2026-01-03 05:05:47.603 | 2026-01-03 04:59:34.000 | False |
| 166 | 70c2b08f-9e98-4965-9c10-35d3ecf7abd9 | Great service! | POSITIVE | 2023-05-22 | 1.704067e+09 | 2026-01-03 05:05:47.603 | 2026-01-03 04:59:35.000 | False |
| 167 | 61ecc99f-b665-48c6-b7cb-8e033b99be34 | I hate this product. | NEGATIVE | 2025-07-12 | 1.704067e+09 | 2026-01-03 05:05:47.603 | 2026-01-03 04:59:35.000 | False |
| 168 | 0e5b0f1c-f82b-4380-8a7d-f4db3b07fb77 | I hate this product. | NEGATIVE | 2023-10-24 | 1.704067e+09 | 2026-01-03 05:05:47.603 | 2026-01-03 04:59:35.000 | False |
| 169 | ace0e078-df04-4f3b-9abb-f516fd764f0b | Terrible experience. | NEGATIVE | 2024-10-19 | 1.704067e+09 | 2026-01-03 05:05:47.603 | 2026-01-03 04:59:35.000 | False |

170 rows × 8 columns

*Figure 5.12 — Resulting DataFrame after querying the Athena table*

The offline feature store keeps all versions of the data for retraining and auditing, ensuring past data is available for comparison and reproducibility in model training. As a result, the `DataFrame` includes both versions of each record: version 1 from `feature_store_01.csv`, which uses the `POSITIVE`, `NEUTRAL`, and `NEGATIVE` tag values, and version 2 from `feature_store_02.csv`, which uses only the `POSITIVE` and `NEGATIVE` tag column values. In contrast, the online feature store stores only the latest

version of each record (using only `POSITIVE` and `NEGATIVE` tag column values) to provide real-time, low-latency access for production inference.

> **Note**
>
> If you do not get any results after running the previous query, please wait for about 10-15 minutes and then try again.

11. Run the following query to count the number of records in the table:

```
SQL(f"SELECT COUNT(*) FROM { table }")
```

This should return `170`, indicating a total of 170 records. Note that there are duplicates in the table since the offline store retains all versions of the data. This allows for tracking changes over time, but can result in multiple records with the same identifier. With the right query, you can filter out duplicates and retrieve only the unique records.

12. Run the following query to retrieve the latest version of each record from the offline store:

```
SQL(f"""
WITH latest_versions AS (
    SELECT *,ROW_NUMBER()
    OVER (
        PARTITION BY unique_id
        ORDER BY event_time DESC
   ) AS rn
    FROM {table}
)
SELECT * FROM latest_versions WHERE rn = 1;
""")
```

This query makes use of the `WITH` clause, which creates a temporary result set that assigns row numbers to each record based on `unique_id` and `event_time`. The `SELECT` statement filters out only the rows where the row number (`rn`) is `1`, representing the latest version.

Running this query will return a `DataFrame` containing the most recent version of each record:

| | unique_id | statement | tag | date | event_time | write_time | api_invocation_time | is_deleted | rn |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 5dad4fe6-6839-4990-b41b-c41ed092ad15 | Nothing special about it. | POSITIVE | 2024-12-25 | 1.735690e+09 | 2026-01-03 05:05:47.574 | 2026-01-03 04:59:35.000 | False | 1 |
| 1 | 18490b47-9572-4854-8680-614438f381b4 | The product is okay. | POSITIVE | 2025-08-04 | 1.735690e+09 | 2026-01-03 05:05:47.574 | 2026-01-03 04:59:35.000 | False | 1 |
| 2 | dcf86779-8ea6-4f8c-ac50-980d2d9287fd | Great service! | POSITIVE | 2024-11-03 | 1.704067e+09 | 2026-01-03 05:05:47.582 | 2026-01-03 04:59:34.000 | False | 1 |
| 3 | 03b57ea8-0376-415b-89a4-46b9d80f356c | The product is okay. | POSITIVE | 2023-04-13 | 1.735690e+09 | 2026-01-03 05:05:47.587 | 2026-01-03 04:59:35.000 | False | 1 |
| 4 | 76546e60-22b7-47ac-9208-e240b55dc7d9 | I would not recommend this to anyone. | NEGATIVE | 2025-07-09 | 1.704067e+09 | 2026-01-03 05:05:47.617 | 2026-01-03 04:59:34.000 | False | 1 |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 80 | c024c484-156c-4f70-9511-19476cb0bf1e | This is awful. | NEGATIVE | 2024-09-28 | 1.704067e+09 | 2026-01-03 05:05:47.613 | 2026-01-03 04:59:34.000 | False | 1 |
| 81 | 1885bdfc-79f5-4a11-983c-9b220895fc71 | Great service! | POSITIVE | 2024-04-15 | 1.704067e+09 | 2026-01-03 05:05:47.570 | 2026-01-03 04:59:34.000 | False | 1 |
| 82 | db6aaa6f-f73e-4ab4-a236-f634f3aa52e3 | This product is amazing! | POSITIVE | 2025-04-03 | 1.704067e+09 | 2026-01-03 05:05:47.570 | 2026-01-03 04:59:34.000 | False | 1 |
| 83 | 487736c6-f5e1-46b9-8729-d39ad0e43651 | Terrible experience. | NEGATIVE | 2025-02-09 | 1.704067e+09 | 2026-01-03 05:05:47.592 | 2026-01-03 04:59:34.000 | False | 1 |
| 84 | fd62901c-4c1c-4aab-995b-3938c9c493bd | Great service! | POSITIVE | 2025-11-06 | 1.704067e+09 | 2026-01-03 05:05:47.592 | 2026-01-03 04:59:34.000 | False | 1 |

85 rows × 9 columns

*Figure 5.13 — Resulting DataFrame containing the latest version of each record*

This time, only 85 records are returned, as duplicates have been filtered out, leaving only the most recent version of each record based on `unique_id` and `event_time`.

13. Alternatively, we can create a view that would allow us to retrieve the latest version of each record. If you are wondering what a view is, it's a virtual table in SQL that stores a query to retrieve data, simplifying complex queries and allowing access to data without directly querying the underlying tables. Let's create the view as follows:

```
SQL(f"""
CREATE OR REPLACE VIEW latest_versions_view AS
WITH latest_versions AS (
    SELECT *,ROW_NUMBER()
       OVER (
           PARTITION BY unique_id
           ORDER BY event_time DESC
       ) AS rn
    FROM { table }
)
SELECT * FROM latest_versions WHERE rn = 1;
""", no_df=True)
```

This query creates a view called `latest_versions_view` that stores the result of the query to retrieve the most recent version of each record.

14. Once the view has been created, we can check if we would get the same exact set of results by running the following query:

```
SQL("SELECT * FROM latest_versions_view")
```

This should return the following `DataFrame`:

| | unique_id | statement | tag | date | event_time | write_time | api_invocation_time | is_deleted | rn |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 38f32558-742b-4bad-945e-eef781d0c615 | Highly recommend this product! | POSITIVE | 2023-06-04 | 1.704067e+09 | 2026-01-03 05:05:47.588 | 2026-01-03 04:59:34.000 | False | 1 |
| 1 | 03b57ea8-0376-415b-89a4-46b9d80f356c | The product is okay. | POSITIVE | 2023-04-13 | 1.735690e+09 | 2026-01-03 05:05:47.587 | 2026-01-03 04:59:35.000 | False | 1 |
| 2 | fe9d38cc-ee40-4260-bb94-7e2c93200391 | I love this! | POSITIVE | 2023-01-14 | 1.704067e+09 | 2026-01-03 05:05:47.635 | 2026-01-03 04:59:34.000 | False | 1 |
| 3 | 5dad4fe6-6839-4990-b41b-c41ed092ad15 | Nothing special about it. | POSITIVE | 2024-12-25 | 1.735690e+09 | 2026-01-03 05:05:47.574 | 2026-01-03 04:59:35.000 | False | 1 |
| 4 | 18490b47-9572-4854-8680-614438f381b4 | The product is okay. | POSITIVE | 2025-08-04 | 1.735690e+09 | 2026-01-03 05:05:47.574 | 2026-01-03 04:59:35.000 | False | 1 |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 80 | 0e3cb463-e7ce-4403-b417-67f8ba5c0724 | This is the best purchase I've made. | POSITIVE | 2025-02-05 | 1.704067e+09 | 2026-01-03 05:05:47.573 | 2026-01-03 04:59:34.000 | False | 1 |
| 81 | a0c0ce4a-872f-4d77-81c2-24ec758ee28a | The product is okay. | POSITIVE | 2025-01-17 | 1.735690e+09 | 2026-01-03 05:05:47.603 | 2026-01-03 04:59:35.000 | False | 1 |
| 82 | f3331914-30df-4b52-9489-dd97101f4a4c | I haven't decided yet. | POSITIVE | 2024-02-23 | 1.735690e+09 | 2026-01-03 05:05:47.597 | 2026-01-03 04:59:35.000 | False | 1 |
| 83 | e849627d-26cd-4a27-a07c-4e64572fc8af | The product is okay. | POSITIVE | 2024-10-15 | 1.735690e+09 | 2026-01-03 05:05:47.682 | 2026-01-03 04:59:35.000 | False | 1 |
| 84 | 8d727b77-d276-4c4b-8825-f71f3a68396d | I hate this product. | NEGATIVE | 2024-04-28 | 1.704067e+09 | 2026-01-03 05:05:47.562 | 2026-01-03 04:59:34.000 | False | 1 |

85 rows × 9 columns

*Figure 5.14 — Resulting DataFrame after querying the view*

By creating the view, we'll retrieve the same set of results, where the `tag` column only uses the `POSITIVE` and `NEGATIVE` values, excluding any records with `NEUTRAL` tags that might have been present in earlier versions.

> **Note**
>
> Note that when using Amazon Athena, the results may be sorted differently with each run unless an explicit `ORDER BY` clause is applied.

15. Finally, delete the feature group after completing the exercises in this chapter.

```
feature_group.delete()
```

This ensures that unused resources are cleaned up, helping you avoid unnecessary storage usage and additional costs.

At this point, you should have a good understanding of how to retrieve data from both the online and offline feature stores using the SageMaker Python SDK. By working on the hands-

on examples in this chapter, you learned how SageMaker Feature Store simplifies data ingestion, metadata management, and feature retrieval through a fully managed feature store service. With this foundation, you are better equipped to manage datasets efficiently and reliably when building real-world solutions on AWS.

**Note**

Now that you have completed all the sections in this chapter, you can perform the cleanup steps to help manage resource usage and control costs. Do not forget to delete the S3 table bucket that was created and used in this chapter and the previous chapter, along with any table resources stored inside it. You should also stop the running SageMaker notebook instance once you have finished working with the hands-on examples in this chapter.

# Summary

In this chapter, you explored key AWS services and capabilities for supporting data management in modern data analytics and machine learning workloads. You learned how to work with AWS Lake Formation permissions, use Amazon Athena to query and process data stored in S3 table buckets, and ingest data into Amazon SageMaker Feature Store. You also learned how to add searchable metadata to features and retrieve data from both the online and offline feature stores.

In the next chapter, we will shift our focus and dive deep into various data processing services and capabilities on AWS. By the end of the next chapter, you should have a strong grasp of the AWS services that can be used for various data processing and analysis requirements.

# Further reading

To expand your practical knowledge of the concepts presented in this chapter, refer to the following resources:

- *Amazon Athena User Guide | Understanding tables, databases, and data catalogs in Athena* (`https://docs.aws.amazon.com/athena/latest/ug/understanding-tables-databases-and-the-data-catalog.html`)
- *Amazon S3 User Guide | Querying Amazon S3 tables with Athena* (`https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables-integrating-athena.html`)

- *Amazon SageMaker AI User Guide | Create, store, and share features with Feature Store* (`https://docs.aws.amazon.com/sagemaker/latest/dg/feature-store.html`)
- *Amazon SageMaker AI Developer Guide | Adding searchable metadata to your features* (`https://docs.aws.amazon.com/sagemaker/latest/dg/feature-store-add-metadata.html`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 6

# Pragmatic Data Processing on AWS

Imagine that you need to process and augment thousands of text entries to fine-tune a **machine learning** (**ML**) model. You've tried running your data-processing script on your local machine, only to realize that it would take days or even weeks to finish. Given the limited compute power and memory of your local setup, you have considered spinning up an EC2 instance, installing all the necessary packages, and configuring the environment manually. While this approach can work, it can quickly become overwhelming and error-prone as your datasets and processing requirements grow. Of course, you might be tempted to write scripts that automatically start and stop EC2 instances to run your custom processing workflows and reduce compute costs. However, this is easier said than done, given the many details involved in configuring instances, installing packages, and handling errors or failures during processing. The good news is that you don't have to build and manage all of this yourself, as **SageMaker Processing** handles the heavy lifting and lets you run your custom scripts in a fully managed environment.

In this chapter, we'll take a closer look at SageMaker Processing jobs and how they enable you to run data-processing tasks without managing infrastructure. We'll begin by discussing the core concepts, terminology, and common use cases to help you understand where processing jobs fit in an ML workflow. We'll then work with a basic example to familiarize you with the steps to run a processing job, and explore more complex scenarios, including using processing jobs to automate back translation for data augmentation.

We will cover the following topics in this chapter:

- Getting started with SageMaker Processing jobs
- Running your first SageMaker Processing job

- Preparing the input data and script for the back translation job
- Automating back translation workflows with SageMaker Processing jobs

By the end of the chapter, you'll have a solid understanding of how to leverage SageMaker Processing jobs for real-world data-processing workflows. You'll also have seen how you can run almost any script or code inside a container on a dedicated ML instance, enabling a wide range of custom data-processing or automation tasks within your ML workflows.

# Technical requirements

Before we start, you must have the following ready:

- The SageMaker notebook instance set up from *Chapter 5*, configured with the required IAM role (`SageMakerAdminRole`). Make sure to start the notebook instance before working on the hands-on portion of this chapter.
- **Applied account-level quota value** under **ml.p3.2xlarge for processing job usage** set to 2 (or above), similar to what we have in *Figure 6.1*:

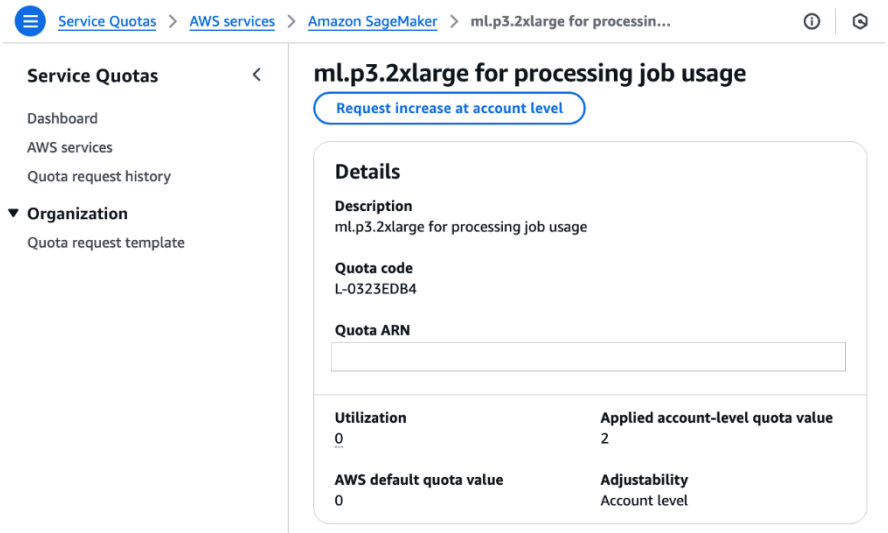



*Figure 6.1 — Required Applied account-level quota value setting for processing job usage*

You can review and request quota increases through the **Service Quotas** console (within the AWS Management Console). To view the applied quota values and perform the required quota increase request for the **ml.p3.2xlarge for processing job usage** applied account-level quota value, simply navigate to `https://us-`

`east-1.console.aws.amazon.com/servicequotas/home/services/sagemaker/quotas` and locate the quota value to be increased.

- **Applied account-level quota value** under **ml.m5.xlarge for processing job usage** set to 2 (or above).

> **Note**
>
> When requesting or verifying the quota, make sure that you select **ml.m5.xlarge for processing job usage** (or **ml.p3.2xlarge for processing job usage** if you plan to use GPU-based instances). SageMaker provides several instance-level quota categories, such as cluster usage, cluster spot instance usage, endpoint usage, notebook instance usage, processing job usage, spot training usage, and training job usage. Since this chapter focuses on SageMaker Processing jobs, ensure that you modify the quota specifically for the processing job usage category for the appropriate instance type rather than the other categories.

The source code and other files used in each chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

# Getting started with SageMaker Processing jobs

Running resource-intensive data-processing scripts on a local machine is often impractical, as they can require more CPU, memory, or GPU resources than a personal computer can provide. In the past, data scientists and ML engineers had to launch an EC2 instance to handle the workload and then shut it down once the job finished to save costs. With SageMaker Processing jobs, a dedicated ML instance is automatically provisioned to execute your script in a fully managed environment, allowing you to focus on developing your custom data-processing code while SageMaker AI manages the infrastructure for you.

Before we dive into the hands-on portion of this chapter, let's quickly go over some important concepts and terminology:

- **CloudWatch**: A managed monitoring and logging service from AWS that collects metrics, logs, and events from your AWS resources and applications. Since SageMaker AI is fully integrated with CloudWatch, processing, training, and inference jobs automatically send logs and performance metrics for monitoring and troubleshooting.

- **CloudWatch log group**: A collection of log streams in Amazon CloudWatch Logs. In SageMaker AI, each type of job typically writes its logs to a dedicated log group. For example, training job logs are stored under `/aws/sagemaker/TrainingJobs`, while processing job logs appear in `/aws/sagemaker/ProcessingJobs`.
- **CloudWatch log stream**: A sequence of log events that come from a single source, such as one execution of a SageMaker Processing job. Each log stream belongs to a log group.
- **Processing job**: A fully managed SageMaker AI job that runs data preprocessing, feature engineering, or other custom scripts in a managed environment. The job executes code inside a container on ML instances.
- **ML instance**: An Amazon EC2 instance type that SageMaker AI provisions to run your processing, training, or inference workloads. When working with SageMaker AI, you can choose the specific ML instance type to match the compute, memory, or GPU requirements of your workload. For example, `ml.m5.xlarge` is a general-purpose CPU instance suitable for most processing and training tasks, while `ml.p3.2xlarge` and `ml.g4dn.xlarge` provide GPU acceleration. The `ml.p3.2xlarge` instance type is ideal for high-performance deep learning training, whereas `ml.g4dn.xlarge` is commonly used for inference or smaller-scale GPU workloads.
- **Container**: A lightweight, self-contained environment that ensures your application code or script runs consistently across different environments and platforms. SageMaker AI uses containers to execute your scripts in a managed and consistent environment. While you have the ability to provide a custom container image for your job, SageMaker AI also offers a variety of preconfigured container images for common frameworks and tasks to make it easier for you to get started quickly.

When a SageMaker Processing job runs, a dedicated ML instance (or multiple instances) is provisioned to execute your script in a fully managed environment. The script and its dependencies are run inside a container, which helps ensure consistent execution across the ML instances.

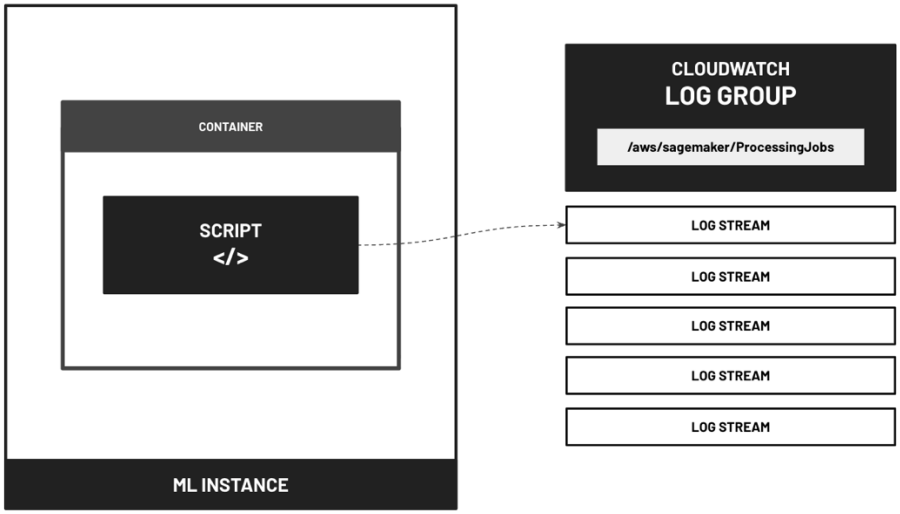



*Figure 6.2 — SageMaker Processing job sending execution logs to CloudWatch Logs*

In *Figure 6.2*, we can see that during the execution of the processing job, detailed logs and metrics are sent to a CloudWatch log stream within the appropriate log group (`/aws/sagemaker/ProcessingJobs`), which allows you to monitor progress and troubleshoot as needed. After the job completes, the ML instances are automatically terminated, so you only incur costs while the job is running.

**Note**

In practice, smaller EC2 instances running Jupyter notebooks are often sufficient for lightweight data exploration and preprocessing. For more resource-intensive workloads, however, you'll need more powerful compute resources. SageMaker Processing enables you to run heavier data-processing scripts on dedicated ML instances that are automatically provisioned for the job and terminated afterward. This approach allows you to scale your data-processing workflows without needing to manually start or stop instances.

SageMaker Processing jobs are commonly used for a variety of data preparation and ML workflow tasks, including the following:

- **Data preprocessing**: Cleaning, transforming, and normalizing raw datasets before training an ML model. For example, you might have a dataset with missing values, inconsistent formatting, or mixed data types that need standardization. Using a

SageMaker Processing job, you can automatically apply these transformations at scale to ensure your dataset is ready for model training.

- **Feature engineering**: Creating new features or transforming existing ones to improve model performance. Suppose you are building a recommendation system and want to combine user behavior logs into meaningful engagement scores. A processing job can generate these features across millions of records quickly, without manual intervention.
- **Data validation and analysis**: Running scripts that inspect datasets for missing values, anomalies, or schema issues. Imagine you receive multiple CSV files from different sources, each with slightly different column names or data formats. Processing jobs can standardize the data and flag inconsistencies automatically before feeding it into your ML pipeline.
- **Batch data processing**: Performing large-scale data transformations or aggregations on datasets stored in services such as Amazon S3. For instance, you may need to convert all text data into tokenized vectors for NLP tasks. A SageMaker Processing job can efficiently process large amounts of data in S3 without requiring you to manage servers manually.
- **Model evaluation**: Running evaluation scripts that compute performance metrics for trained models. Suppose you have trained multiple models on a dataset and want to compare accuracy, F1 score, or other metrics. Processing jobs can automatically run evaluation scripts on large test datasets, producing standardized reports to help you select the best model.

**Note**

There are other capabilities and solutions, such as **SageMaker Clarify**, that build on top of SageMaker Processing, enabling you to run bias detection, explainability, and data quality workflows at scale using the same managed processing infrastructure. SageMaker Clarify Processing jobs allow you to run these analyses on datasets and models, automatically generating reports that highlight potential biases, feature importances, and overall data quality. While you can prepare your own custom code to perform these tasks, it is generally recommended to leverage the built-in capabilities of SageMaker Clarify to save time and ensure best practices are followed.

As you go through the hands-on portion of this chapter to set up and run your processing job, you will notice that the job execution environment is not limited to a specific type of task. In practice, you can run almost any script or code inside the container within the ML instance, as

long as the runtime and the required dependencies are properly configured in the container environment. This means that processing jobs are not restricted to the example use cases listed previously and can be used for a wide range of custom data-processing or automation tasks within your ML workflows.

# Running your first SageMaker Processing job

In this section, we will start with a simple processing job to help you understand the key steps involved in setting up and running a SageMaker Processing job. We will set up the required directories, create a Python script that prints `Running!`, and verify that it runs correctly before using it in a SageMaker Processing job. Once the job completes, we will locate and review the execution logs in CloudWatch to ensure that everything executed successfully.

With these in mind, let's proceed with running our first processing job:

1. Type `sagemaker ai` in the search bar of the AWS Management Console and choose **Amazon SageMaker AI** from the list of results.

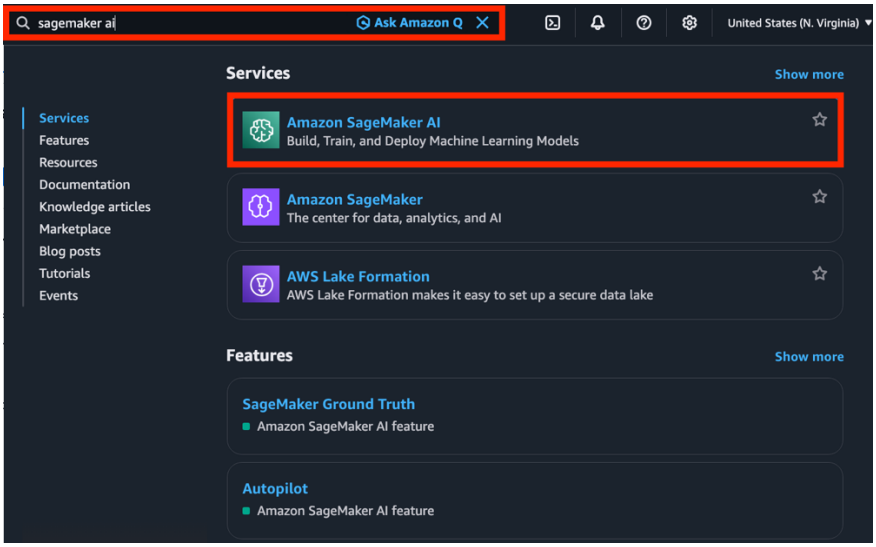



*Figure 6.3 — Navigating to the Amazon SageMaker AI console*

   This will redirect you to the Amazon SageMaker AI console, where you can manage ML workflows, including building, training, and deploying models.

2. In the navigation pane (left sidebar), click **Notebooks** under **Applications and IDEs**. This should open a page that shows all created notebook instances (if any).

3. On the **Notebooks and Git repos** page, locate the notebook instance you created in *Chapter 5*, and click **Start**.
4. Wait for **Status** to change to **InService**, then click **Open JupyterLab**.
5. In the left sidebar of JupyterLab (the file browser), right-click and select **New Folder**. Name the folder `processing`, then double-click it to navigate into the directory.
6. Click the **+** button to open **Launcher**, then select **Terminal** (under **Other**).

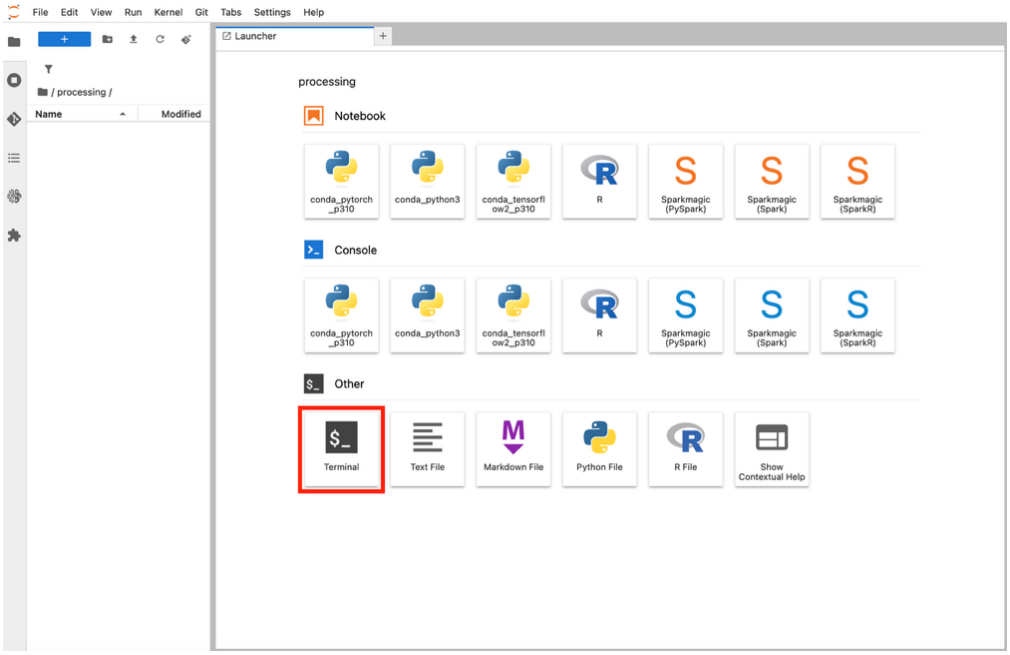



*Figure 6.4 — Launching a terminal*

Inside this terminal, you will run the commands provided in the subsequent steps to create the processing script, which will be used by the SageMaker Processing job.

7. In the terminal, confirm that you are inside the `processing` directory by running the following command:

```
pwd
```

This should return the following:

```
/home/ec2-user/SageMaker/processing
```

8. Next, prepare the directory structure for the scripts and navigate into it:

```
mkdir -p scripts/000
cd scripts/000
```

9. Finally, create an empty Python file named `processing.py` inside the `000` directory:

```
touch processing.py
```

10. In the file browser, locate the `processing.py` file inside **processing** | **scripts** | **000**.

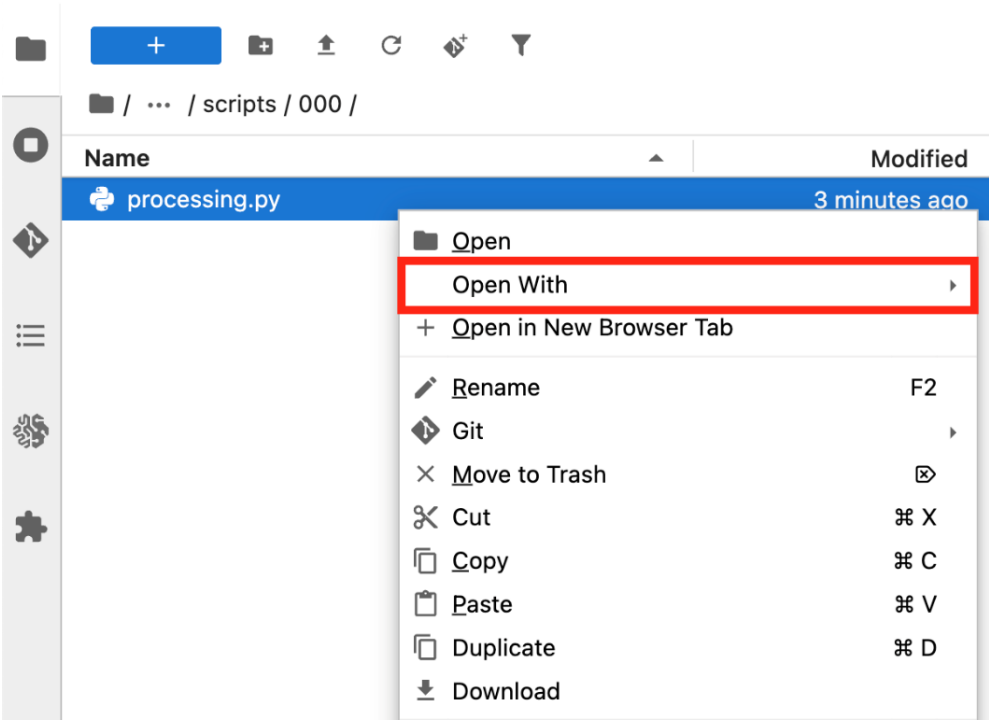



*Figure 6.5 — Opening processing.py in the editor interface*

Right-click the file (**processing.py**) and select **Open With** | **Editor** to open it in the editor interface. Since you just created the file in the previous step, it should open as a blank file in the editor.

11. Add the following code to the `processing.py` file to define the script's entry point:

```
if __name__=='__main__':
    print('Running!')
```

Here, the `if __name__ == '__main__':` block ensures that the code inside it runs only when the script is executed directly.

> **Note**
>
> Don't forget to save `processing.py` after adding the code to ensure your updates are included when you run the processing job.

12. Switch back to the terminal, then run the Python script using the following command:

```
python processing.py
```

This should yield the following output, confirming that the Python script executed successfully and is ready for the next steps:

```
Running!
```

13. In the left sidebar of JupyterLab (the file browser), navigate back to the `processing` directory.
14. Open a new **Launcher** window by selecting **File** | **New Launcher** from the menu. In **Launcher**, create a new Jupyter notebook by choosing **conda_pytorch_p310** (under **Notebook**).

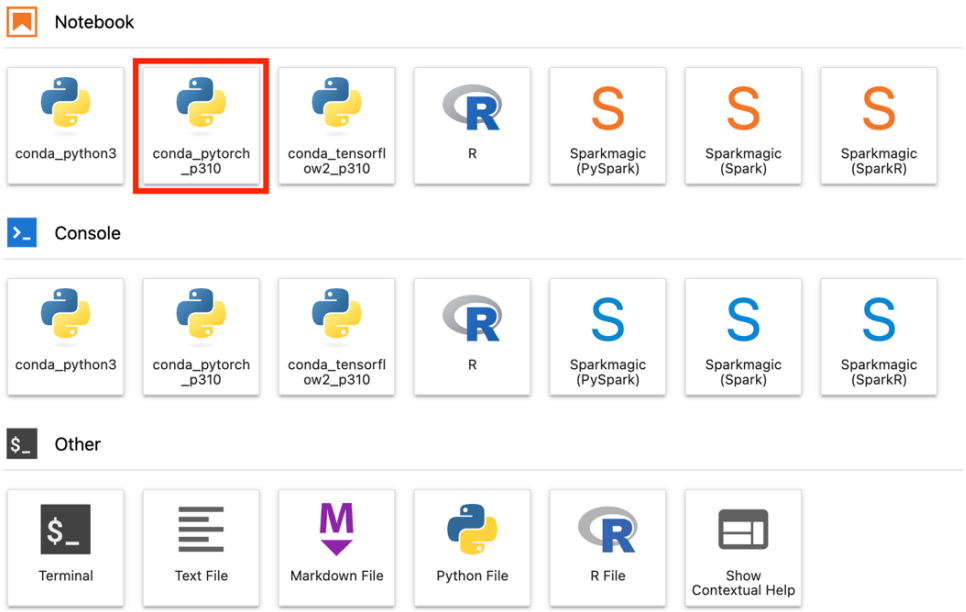



*Figure 6.6 — Launching a new notebook (using conda_pytorch_p310)*

Be sure to rename the notebook to `000.ipynb` before proceeding. We will run the code in the following steps within this notebook.

> **Note**
>
> Make sure that `000.ipynb` is located in the same directory as the `scripts` directory so that all paths in your processing job resolve correctly.

15. To ensure compatibility with the hands-on examples in this chapter, uninstall any existing version of the SageMaker Python SDK and install version `3.5.0`:

```
%pip uninstall -y sagemaker
%pip install "sagemaker==3.5.0"
```

Since the installation process generates a lengthy output in the notebook cell, you may right-click the output area and select **Enable Scrolling for Outputs** to make the log output easier to review.

After the installation completes, you should see a message similar to the following in the generated log output:

```
...
Successfully installed sagemaker-3.5.0
Note: you may need to restart the kernel to use updated packages.
```

After successfully installing the specified package versions, restart the kernel by selecting **Kernel** | **Restart Kernel** from the menu.

> **Note**
>
> This will ensure that the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 3 to 5 minutes for the kernel to fully restart.

16. Suppress specific warnings related to namespace conflicts by filtering out warnings that match a given message pattern:

```
import warnings

m = "Field .* has conflict with protected namespace"

warnings.filterwarnings(
    "ignore",
    message=m
)
```

17. Run the following code to import the required SageMaker utilities and retrieve the current session, execution role, and AWS Region:

```
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)

session = Session()
execution_role = get_execution_role()
region = session.boto_region_name
```

18. Next, define the instance configuration that will be used for the processing job:

```
instance_type = "ml.m5.xlarge"
instance_count = 1
```

19. Import the necessary components from the SageMaker Python SDK and initialize a `FrameworkProcessor` instance, which will be used to run your processing job in a later step:

```
from sagemaker.core.image_uris import (
    get_training_image_uri
)
from sagemaker.core.processing import (
    FrameworkProcessor
)

image_uri = get_training_image_uri(
    region=region,
    framework="pytorch",
    framework_version="1.13",
    py_version="py39",
    instance_type=instance_type,
)

processor = FrameworkProcessor(
    image_uri=image_uri,
    role=execution_role,
    instance_type=instance_type,
    instance_count=instance_count,
)
```

20. Define the `random_string()` function that generates a random uppercase string:

```
import random
import string

def random_string(length=6):
    return ''.join(random.choices(
        string.ascii_uppercase,
```

```
        k=length
    ))
```

21. Use the `random_string()` function defined in the previous step to generate a unique processing job name:

```
job_name = f"processing-job-{random_string()}"
job_name
```

This should give you a unique job name similar to the following:

```
'processing-job-YIUQAT'
```

**Note**

Keep in mind that since `random_string()` produces a random value, you'll get a job name that is different from the example output value.

22. Define the source directory and the Python script that will be used for the processing job:

```
import os

source_dir = os.path.abspath("scripts/000")
code = "processing.py"
```

23. Run the processing, using the specified script source directory and job name:

```
processor.run(
    code=code,
    source_dir=source_dir,
    job_name=job_name,
    wait=True,
)
```

Once the job finishes, you should see log output similar to the following:

```
...
Final Resource Status: Completed
```

24. Switch to the browser tab with the SageMaker AI console.
25. From the SageMaker AI console's navigation pane (left sidebar), choose **Processing jobs** under **Data preparation**.
26. On the **Processing jobs** page, click the name of the most recently run processing job to open its details page.
27. Scroll down to the **Monitoring** section and click **View logs**. This will open the CloudWatch log group (**/aws/sagemaker/ProcessingJobs**) details page (*Figure 6.7*).

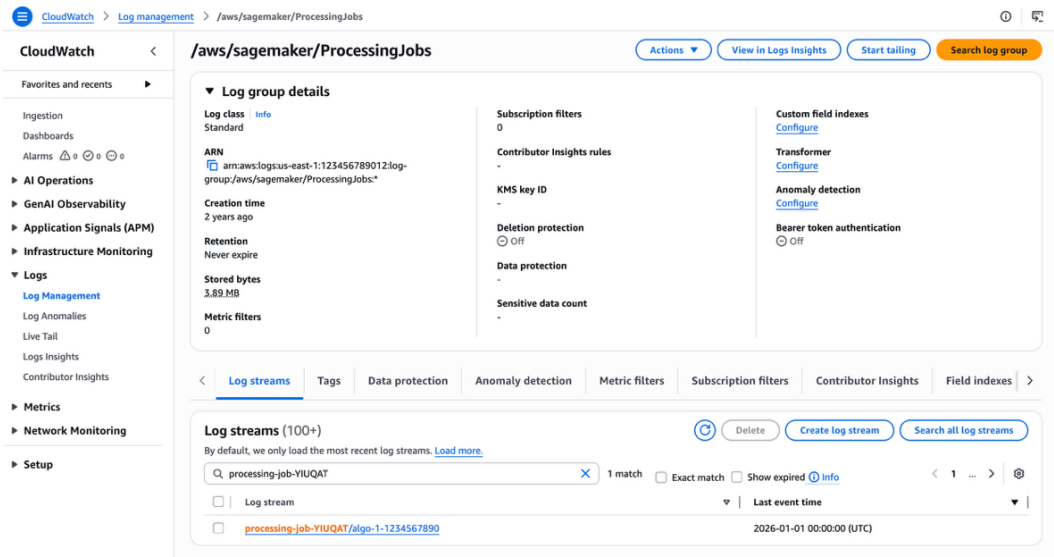



*Figure 6.7 — CloudWatch log group details page*

Here, you can choose the relevant log stream to view detailed execution logs for your SageMaker Processing job.

28. Click on the log stream name to open the page that displays the detailed execution logs for the processing job you just ran. You should find log entries similar to the following to confirm that your job executed successfully:

```
Files after extraction:
total 24
drwxr-xr-x 3 root root 4096 Jan 01 00:00 .
drwxr-xr-x 4 root root 4096 Jan 01 00:00 ..
drwxrwxr-x 2 1000 1000 4096 Jan 01 00:00 .ipynb_checkpoints
-rw-rw-r-- 1 1000 1000 47 Jan 01 00:00 processing.py
-rw-r--r-- 1 root root 634 Jan 01 00:00 runproc.sh
```

```
-rw-r--r-- 1 root root 329 Jan 01 00:00 sourcedir.tar.gz
Running!
No newer events at this moment. Auto retry paused. Resume
```

Here, the `Running!` log message indicates that the script you prepared earlier in this chapter (`processing.py`) ran successfully inside the processing job.

Congratulations on successfully running your first SageMaker Processing job! While it may be tempting to jump straight into a complex example, it's best to start with a simple script to ensure that your environment, file structure, and execution flow are all set up correctly. Beginning with a small, manageable example allows you to identify issues quickly and avoid unnecessary confusion during debugging. Once the basic script runs successfully, you can gradually expand it to include more sophisticated logic or larger data-processing tasks. In the next few sections, we'll build on this example by running a more complex processing job that reflects a real-world scenario.

# Preparing the input data and processing script for the back translation job

As an ML engineer, you'll often encounter challenges when working with languages that have limited training data. For example, while there will be plenty of data in English for natural language tasks, there is usually much less in other languages, such as Tagalog. Back translation helps overcome this obstacle by generating new training data from existing sentences. By translating sentences through intermediate languages and back again, you'll have new sentence variants with slightly different phrasing. With back translation, sentences in Tagalog such as `"Kumain ako ng mansanas"` might be translated to English as `"I ate an apple"` and then back to Tagalog as `"Ako ay kumain ng isang mansanas"`. Given that subtle shifts in sentence structure and word choice happen during translation, back translation can be used for data augmentation to generate paraphrased versions of the original sentences.

In this section, you'll prepare the prerequisites for back translation. This includes preparing a small CSV file containing sample sentences in Tagalog that will serve as input for the back translation job. This smaller file will be extracted from a larger CSV dataset to reduce processing time and allow you to quickly validate the job later in this chapter.

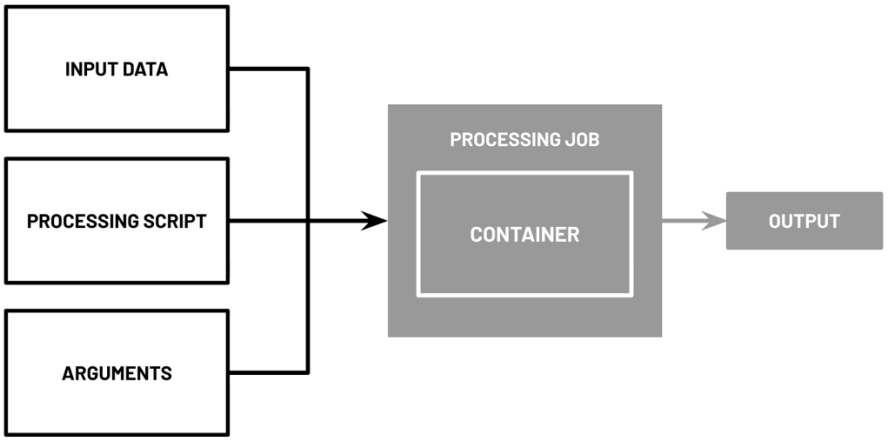



*Figure 6.8 — Preparing the prerequisites of the back translation job*

You'll also download a prebuilt `processing.py` script that processes the sentences in batches, translating them into one or more intermediate languages and then back to the original source language. In addition, you'll identify the intermediate languages that will be passed to the script as arguments.

Let's go through the steps of preparing the prerequisites of the back translation job:

1. In the left sidebar of JupyterLab (the file browser), navigate back to the `processing` directory.
2. Click the **+** button to open **Launcher**, then select **Terminal** (under **Other**).
3. In the terminal, confirm that you are inside the `processing` directory by running the following command:

   ```
   pwd
   ```

   This should return the following:

   ```
   /home/ec2-user/SageMaker/processing
   ```

4. Use the `mkdir` command to create the `input`, `output`, and `scripts` directories:

   ```
   mkdir -p input output scripts/001
   ```

As we go through all of the instructions in this chapter, we will be storing the processing job input data files in the `input` directory, the processing job output data files in the `output` directory, and the job processing scripts in the `scripts` directory.

5. With the directories ready, let's download the CSV file containing 250 sample positive sentiments from the GitHub repository:

```
DIR_PATH=https://raw.githubusercontent.com/PacktPublishing/Machine-
Learning-Engineering-on-AWS-Second-Edition/refs/heads/main/chapter06
CSV_PATH=$DIR_PATH/positive_sentiments_tagalog_250.csv

wget $CSV_PATH -O input/input_tagalog_250.csv
```

Here, we use the capital-letter `O` flag to specify the output filename that will be used for the saved file. In this case, the CSV file stored in the GitHub repository (`positive_sentiments_tagalog_250.csv`) will be downloaded into the CloudShell environment as `input_tagalog_250.csv` inside the `input` directory.

6. At this point, you might be wondering what data our downloaded file contains! Let's check what the downloaded CSV file looks like using the `head` command:

```
head -n 6 input/input_tagalog_250.csv
```

This should return the first few lines of the `input_tagalog_250.csv` file:

```
Talagang nakakatuwang kainin ang pagkain ng Some Brand.
Talagang swak sa panlasa ang pagkain ng Some Brand.
"Ang sarap ng Some Brand, mapapa-order ka ulit talaga."
"Ang sarap ng Some Brand, masarap na masarap talaga."
"Para sa pagkain na swak sa panlasa, subukan ang Some Brand."
"Hinding-hindi ka magsisisi sa Some Brand, ang bango at sarap!"
```

Note that these statements are written in Tagalog and describe food experiences, often praising the flavor or quality of a specific brand. For example, "`Talagang nakakatuwang kainin ang pagkain ng Some Brand`" roughly translates to "`The food from Some Brand is truly a pleasure to eat`". A portion of this list will be used as input for back translation to generate paraphrased versions.

7. Run the following commands to copy the first 10 lines of the `input_tagalog_250.csv` file and save these lines in a new file (`input/input.csv`):

```
ORIG_FILE="input/input_tagalog_250.csv"
NEW_FILE="input/input.csv"
head -n 10 $ORIG_FILE > $NEW_FILE
```

8. Now, let's download the `requirements.txt` file and save it inside the `scripts` directory:

```
REQ_PATH=$DIR_PATH/scripts/001/requirements.txt
wget $REQ_PATH -O scripts/001/requirements.txt
```

9. Let's quickly check what our `requirements.txt` looks like:

```
cat scripts/001/requirements.txt
```

This should give us the following:

```
transformers>=4.30.0
pandas>=2.0.0
numpy>=1.25.0
sentencepiece>=0.1.98
```

SageMaker AI automatically installs all the packages listed in `requirements.txt` in the processing job's container environment. This ensures that your script has all the dependencies it needs before it runs.

10. Now, let's download the `processing.py` script and save it inside the `scripts` directory:

```
SCRIPT_PATH=$DIR_PATH/scripts/001/processing.py
wget $SCRIPT_PATH -O scripts/001/processing.py
```

11. Let's quickly check what our script looks like:

```
less -N scripts/001/processing.py
```

This should give us the following:

```
 1 import torch
 2 import pandas as pd
 3 import argparse
 4 import os
 5 import importlib
 6 import ast
 7 import logging
 8 import sys
 9 import numpy as np
10 import math
11
12 from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
13
14
15 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
16
17 logging_level = logging.getLevelName('INFO')
18 logging_stream_handler = logging.StreamHandler(sys.stdout)
19 logging_format = "%(asctime)s - %(message)s"
20 logging.basicConfig(
21     level=logging_level,
22     handlers=[logging_stream_handler],
23     format=logging_format,
24 )
25 logger = logging.getLogger(__name__)
26
27
28 def log(log_string, logger=logger):
29     logger.info(log_string)
30
31
32 def log_line(logger=logger):
33     logger.info("=" * 50)
34
scripts/001/processing.py
```

*Figure 6.9 — A few lines from the scripts/001/processing.py file*

This script performs text data augmentation using back translation to generate paraphrased versions of input sentences. It begins by loading a list of sentences from a CSV file and splitting them into batches. For each batch, the script translates the sentences from the source language to one or more intermediate languages using Hugging Face translation models. It then translates the results back to the original language, which often produces sentences with slightly different wording while preserving the original meaning. The generated sentences are then deduplicated to remove repeated entries. Finally, the resulting set of augmented sentences is written to a new CSV file, which can be used for tasks such as training or improving NLP models.

The core of the data augmentation process happens in the `back_translate_list()` function, which performs the two-step translation used for back translation:

```
def back_translate_list(...):
    log("back_translate_list()")
    translated_text_list = translate_list(
        statement_list=statement_list,
        tokenizer=first_tokenizer,
        model=first_model,
        device=device
    )
    output_list = translate_list(
```

```
        statement_list=translated_text_list,
        tokenizer=second_tokenizer,
        model=second_model,
        device=device
    )
    return output_list
```

The `back_translate_list()` function generates paraphrased sentences using back translation. It first translates the input sentences into an intermediate language with one tokenizer and model, then translates them back to the original language using another tokenizer and model. The function then returns the list of back-translated sentences.

**Note**

While viewing the `processing.py` code in `less`, you can press *Q* on your keyboard to exit and return to the command prompt.

With your script and sample data ready, you will use SageMaker Processing in the next section to automate the back translation process and generate additional training examples.

# Automating back translation workflows with SageMaker Processing jobs

In this section, you'll use a SageMaker Processing job to run the back translation script in a managed environment that allows you to process large datasets efficiently using dedicated ML instances. You'll start by setting up a new Jupyter notebook, installing the required version of the SageMaker Python SDK, and restarting the kernel to ensure the newly installed package is properly loaded and available for use. After that, you'll initialize a `FrameworkProcessor` instance for handling SageMaker Processing tasks, and configure it with the PyTorch container image, the desired instance type (`ml.p3.2xlarge`), and the count (`1` instance), along with the execution role, which will allow your processing job to run in a dedicated ML instance separate from your notebook.

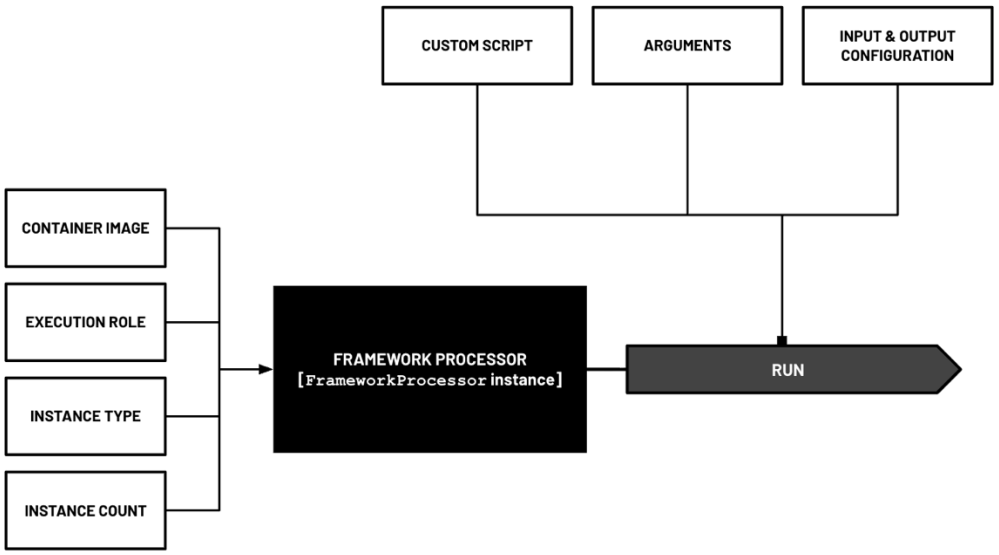



*Figure 6.10 — Initializing a FrameworkProcessor instance*

You will also configure the processing job with additional arguments (the source and intermediate languages for back translation) along with the input configuration (`ProcessingS3Input`) and output configuration (`ProcessingS3Output`) that tells SageMaker where to read the input data from S3, where to store the output inside the container, and when to upload the results back to S3. Finally, you'll run the processing job with the `run()` method of the `FrameworkProcessor` instance, using the custom script, arguments, and input and output configuration that you prepared in the previous steps. This will provision a dedicated ML instance where your custom script executes inside a container and performs the back translation on the statements from your CSV file (`input.csv`). After that, SageMaker AI would automatically upload the resulting output file(s) to the specified S3 location and mark the processing job as completed.

With this in mind, let's proceed with configuring and running the SageMaker Processing job:

1. Open a new **Launcher** window by selecting **File | New Launcher** from the menu. In **Launcher**, create a new Jupyter notebook by choosing **conda_pytorch_p310** (under **Notebook**).
2. Be sure to rename the notebook to `001.ipynb` before proceeding. We will run the code in the following steps within this notebook.

> **Note**
>
> Make sure that `001.ipynb` is located in the same directory as the `scripts`, `input`, and `output` directories so that all paths in your processing job resolve correctly.

3. To ensure compatibility with the hands-on examples in this chapter, uninstall any existing version of the SageMaker Python SDK and install version `3.5.0`:

```
%pip uninstall -y sagemaker
%pip install "sagemaker==3.5.0"
```

Since the installation process generates a lengthy output in the notebook cell, you may right-click the output area and select **Enable Scrolling for Outputs** to make the log output easier to review.

After the installation completes, you should see a message similar to the following in the generated log output:

```
...
Successfully installed sagemaker-3.5.0
Note: you may need to restart the kernel to use updated packages.
```

After successfully installing the specified package versions, restart the kernel by selecting **Kernel** | **Restart Kernel** from the menu.

> **Note**
>
> This will ensure the newly installed packages are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 3 to 5 minutes for the kernel to fully restart.

4. Run the following code to suppress specific warnings related to namespace conflicts by filtering out warnings that match a given message pattern:

```
import warnings

m = "Field .* has conflict with protected namespace"
```

```
warnings.filterwarnings(
    "ignore",
    message=m
)
```

5. Import the required SageMaker Python SDK components and retrieve the current session, execution role, and AWS Region:

```
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)

session = Session()
execution_role = get_execution_role()
region = session.boto_region_name
```

6. Next, define the instance configuration that will be used for the processing job:

```
instance_type = "ml.p3.2xlarge"
instance_count = 1
```

7. Import the necessary components from the SageMaker Python SDK and initialize a `FrameworkProcessor` instance, which will be used to run your processing job in a later step:

```
from sagemaker.core.image_uris import (
    get_training_image_uri
)
from sagemaker.core.processing import (
    FrameworkProcessor
)

image_uri = get_training_image_uri(
    region=region,
    framework="pytorch",
    framework_version="2.0",
    py_version="py310",
    instance_type=instance_type,
)
```

```
processor = FrameworkProcessor(
    image_uri=image_uri,
    role=execution_role,
    instance_type=instance_type,
    instance_count=instance_count,
)
```

Here, we specify the framework (`pytorch`), version (`2.0`), Python version (`py310`), and instance type (`ml.p3.2xlarge`) for initializing the `FrameworkProcessor` instance. By using `ml.p3.2xlarge`, your back translation processing will run efficiently on the GPU during the processing job.

8. Define the `random_string()` function, which generates a random uppercase string:

```
import random
import string

def random_string(length=6):
    return ''.join(random.choices(
        string.ascii_uppercase,
        k=length
    ))
```

9. Use the `random_string()` function defined in the previous step to generate a unique processing job name:

```
random_id = random_string()
job_name = f"processing-job-{ random_id }"
job_name
```

This should give you a unique job name similar to the following:

```
'processing-job-SDQJSC'
```

10. Define the source directory and the Python script that will be used for the processing job:

```
import os
```

```
source_dir = os.path.abspath("scripts/001")
code = "processing.py"
```

11. Import the necessary classes to handle the input and output configuration of our processing job:

```
from sagemaker.core.shapes import (
    ProcessingInput,
    ProcessingS3Input,
    ProcessingOutput,
    ProcessingS3Output
)
```

12. Define the S3 paths for storing the input and output data files:

```
s3_data_path = ('').join([
    "s3://",
    session.default_bucket(),
    "/" + random_id,
])

input_data_path = s3_data_path + "/input"
output_data_path = s3_data_path + "/output"
```

Here, we define a unique S3 base path using the default SageMaker bucket and a random ID, along with separate folders for input and output, so that the job's data is stored in a unique location that avoids conflicts with previous jobs.

13. Let's verify the S3 paths prepared in the previous step:

```
print(s3_data_path)
print(input_data_path)
print(output_data_path)
```

This should give you output similar to the following:

```
s3://sagemaker-us-east-1-.../SDQJSC
s3://sagemaker-us-east-1-.../SDQJSC/input
s3://sagemaker-us-east-1-.../SDQJSC/output
```

14. Upload the input CSV file from the local machine to the S3 input path so that the processing job has access to the data it needs:

```
!aws s3 cp input/input.csv {input_data_path}/input.csv
```

15. List the contents of the S3 input folder to confirm that the file was successfully uploaded and is ready for the processing job:

```
!aws s3 ls {input_data_path} --recursive
```

This should return the following:

```
   2026-01-01 00:00:00        572 SDQJSC/input/input.csv
```

16. Define the arguments that the processing script will use, including the source language and target languages for back translation, so that the script can run dynamically based on these parameters:

```
languages = str(['en','pt','de'])

arguments = [
    "--source_language", "tl",
    "--languages_for_back_translation", languages,
]
```

Here, we're specifying Tagalog (`tl`) as the source language and English (`en`), Portuguese (`pt`), and German (`de`) as the intermediate languages for back translation. This means a Tagalog sentence will be translated into English and then back into Tagalog as part of the back translation process. The script applies the same process to Portuguese and German as intermediate languages. Since the script allows input arguments to specify the source and intermediate languages, it can be reused with minimal to no code changes. This means you can back-translate using English, Tagalog, or any supported language simply by passing different parameters to the processing job.

**Note**

Behind the scenes, the back translation script uses pre-trained translation models to convert sentences into an intermediate language and then back to the original language. For example, it first translates Tagalog (`tl`) sentences

> into English (en) using the Hugging Face `Helsinki-NLP/opus-mt-tl-en` model, and then translates them back to Tagalog using `Helsinki-NLP/opus-mt-en-tl` to produce paraphrased sentences. Feel free to check out `https://huggingface.co/Helsinki-NLP/opus-mt-tl-en` for more information.

17. Create a `ProcessingS3Input` instance pointing to the S3 input path and specifying how the data should be read inside the container, and then wrap it in a `ProcessingInput` so SageMaker knows where to make the input data available:

```
processing_s3_input = ProcessingS3Input(
    s3_uri=input_data_path,
    local_path="/opt/ml/processing/input",
    s3_data_type="S3Prefix",
    s3_input_mode="File",
    s3_data_distribution_type="ShardedByS3Key",
)

inputs = [
    ProcessingInput(
        input_name="input-00",
        s3_input=processing_s3_input
    )
]
```

18. Create a `ProcessingS3Output` instance that points to the S3 output path and specifies that output files should be uploaded at the end of the job, and then wrap it in a `ProcessingOutput` so SageMaker knows where to store the processing output file(s):

```
processing_s3_output = ProcessingS3Output(
    s3_uri=output_data_path,
    local_path="/opt/ml/processing/output",
    s3_upload_mode="EndOfJob",
)

outputs = [
    ProcessingOutput(
            output_name="output-00",
```

```
            s3_output=processing_s3_output
        ),
]
```

19. With all of the prerequisites ready, let's run the `processor.run()` method by providing the code, source directory, job name, arguments, inputs, and outputs, which triggers SageMaker AI to provision a managed ML instance, execute the processing script inside a container within the ML instance, and store the results in S3:

```
processor.run(
    code=code,
    source_dir=source_dir,
    job_name=job_name,
    arguments=arguments,
    inputs=inputs,
    outputs=outputs,
    wait=True,
)
```

When you call `processor.run()`, SageMaker AI provisions a dedicated ML instance (1 × `ml.p3.2xlarge`), separate from your notebook, to execute the processing job. Your custom processing script (`processing.py`) runs inside a container on this instance, using the PyTorch container image retrieved via `get_training_image_uri()`.

Once the processing job finishes, you should get the following log output to confirm that the job ran successfully:

```
...
Final Resource Status: Completed
```

Since the dedicated ML instance is deleted after the job completes, you should be able to access the output files in S3, as SageMaker AI uploads the output files from `/opt/ml/processing/output` inside the container wthin the ML instance to the specified S3 location (e.g., `s3://sagemaker-us-east-1-.../.../output`) and then marks the job status as **Completed**.

> **Note**
>
> This step typically takes 6 to 20 minutes to complete. Feel free to grab a cup of coffee or tea while waiting.

20. Run the following block of code to retrieve the `ProcessingS3Output` configuration from the most recent processing job associated with the `FrameworkProcessor` instance:

```
job = processor.jobs[0]
output_config = job.processing_output_config
s3_output = output_config.outputs[0].s3_output
s3_output
```

You should get an output similar to the following:

```
ProcessingS3Output(
  s3_uri='s3://sagemaker-us-east-1-.../SDQJSC/output',
  s3_upload_mode='EndOfJob',
  local_path='/opt/ml/processing/output'
)
```

Here, you can find the local directory inside the container within the ML instance where the script writes the output (`local_path`) when the output is uploaded from the local directory to the specified S3 location (`s3_upload_mode`), as well as the S3 path where the output files are stored (`s3_uri`).

21. List all files in the S3 output location obtained from the previous step to confirm that the processing job successfully uploaded the output files to S3:

```
!aws s3 ls {s3_output.s3_uri} --recursive
```

This gives you an output similar to the following:

```
2026-01-01 00:00:00    1625  SDQJSC/output/output.csv
```

22. Define a variable (`dl_path`) that references the S3 path of the `output.csv` file:

```
dl_path = s3_output.s3_uri + "/output.csv"
```

23. Using the variable defined in the previous step (`dl_path`), download the CSV file from the specified S3 output path to the `output` directory in your notebook instance:

```
!aws s3 cp {dl_path} output/output.csv
```

This should yield the following log output:

```
download: s3://sagemaker-us-east-1-.../SDQJSC/output/output.csv to output/
output.csv
```

From here, you can inspect the downloaded file locally within your notebook instance.

24. Use the `cat` command to inspect the downloaded `output.csv` file:

```
!cat output/output.csv
```

This should yield an output similar to the following:

```
Hinding - hindi mo pagsisisihan ang gayong apoy! Hindi ka kailanman
manghihimagod!
Hindi ko inisip na ang pagkain ni Ilan Brand ay ang ganitong uri ng
panlasa.
Tuwang - tuwa akong kainin ang pagkain ng Ilang Brandiks.
...
Ang masarap na pagkaing tinatawag na Some Brand.
Hindi ka kailanman matatangay ng gayong apoy!
```

From an input of 10 statements, our processing job using back translation produced about 30 statements, increasing the dataset size and introducing variation for data augmentation. If this process were applied to the full dataset instead of just 10 sample statements, it would generate significantly more resulting sentences, since the number of statements produced is generally proportional to the number of intermediate languages used. For example, with an input of 250 statements and 3 intermediate languages, the back translation process would produce roughly 750 statements. In practice, however, the final number of statements will likely be lower after duplicate outputs are removed.

You may notice that back translation occasionally produces sentences that are grammatically incorrect or slightly nonsensical; however, the technique is still valuable for data augmentation. Even imperfect outputs introduce variation in wording and phrasing, helping models learn more robust representations. Overall, the benefits of generating diverse paraphrased sentences outweigh the occasional errors.

Now that you've completed the hands-on examples in this chapter, you could experiment by running another SageMaker Processing job using the full version of your Tagalog dataset to generate a larger set of paraphrased sentences. Alternatively, you could try processing a

different dataset, such as English text, and explore using a different combination of intermediate languages for back translation. I'll leave this to you as an exercise.

**Note**

Do not forget to stop the running SageMaker notebook instance once you have finished working with the hands-on examples in this chapter.

At this stage, you should have a solid understanding of how SageMaker Processing jobs work. In this section, you successfully set up a data-processing job, ran your back translation script on a dedicated ML instance, and examined the resulting output files in S3. From a small set of sample input statements, the processing job automatically generated multiple paraphrased sentences, showing how back translation can be used for data augmentation. Before we end this chapter, let's review some best practices you should apply when working with SageMaker Processing jobs:

- **Choose the right instance type**: Use smaller instances for lightweight data-processing workloads and more powerful ML instances for resource-intensive tasks. For example, exploratory data analysis or small-scale preprocessing can run on `ml.m5.xlarge`, while large datasets or deep learning feature engineering may require a GPU instance such as `ml.p3.2xlarge`. Selecting the right instance ensures efficiency and cost-effectiveness.
- **Prepare and define dependencies carefully**: Ensure all required Python packages are listed in `requirements.txt` so SageMaker AI can install them automatically within the container. This prevents runtime errors caused by missing libraries and ensures your script runs consistently across different jobs. For instance, including `transformers`, `pandas`, and `numpy` (and specifying exact versions) guarantees that your processing job executes without interruptions.
- **Batch processing for scalability**: Split large datasets into batches to prevent memory issues and parallelize processing where possible. For example, if you have millions of rows in an S3 dataset, processing in batches of 10,000 rows each reduces the risk of memory overload and allows SageMaker to scale processing efficiently.
- **Make your scripts flexible with arguments**: Design your scripts to accept input parameters for file paths, batch size, or processing options so they can be reused with different datasets or configurations. Make sure to clearly document all available arguments and their expected values so that you can easily reuse the script in the future.
- **Security and access control**: Use appropriate IAM roles and S3 bucket permissions to restrict access to sensitive data and ensure secure processing. For example, create a

dedicated IAM role for your processing job with read-only access to input bucket(s) and write access to output bucket(s), to minimize the risk of accidental data exposure.

- **Optimal file formats**: Consider using other storage formats to balance performance and cost. For example, Parquet with compression can improve I/O speed while saving on S3 storage.
- **Documentation and logging**: Document your scripts with clear comments and implement logging for key steps and events to make your data-processing scripts easier to maintain and troubleshoot. For example, log the start and end times of each batch, record the number of processed records, and add descriptive comments explaining non-obvious transformations, so anyone revisiting the script can quickly understand its behavior.
- **Version control and code quality**: Track and manage your code with Git, ensure reliability through testing and code reviews, and use static code analyzers to detect potential security issues and enforce coding standards before executing scripts in SageMaker Processing jobs.
- **Local development and testing**: Before running a SageMaker Processing job, develop and debug your scripts on your local machine or in a Jupyter notebook. This allows you to quickly iterate and identify issues, since provisioning a dedicated ML instance in SageMaker can take additional time and incur costs.

Following these best practices helps you get the most out of SageMaker Processing jobs. As you incorporate one or more of these guidelines, you'll see improvements in efficiency, reliability, and maintainability across your data-processing workflows. They also make it easier to collaborate with other team members and troubleshoot issues when they arise. Over time, consistently applying these practices will help you scale your workflows to larger datasets and more complex processing tasks without running into common pitfalls. Ultimately, this approach lets you concentrate on your data workflows and model development, leaving the infrastructure management to SageMaker AI.

# Summary

In this chapter, you explored how to use SageMaker Processing jobs to run resource-intensive data-processing scripts. You started by running a simplified example to see how a SageMaker Processing job works. You then downloaded and prepared the input data and the `processing.py` script for the back translation job to ensure that all prerequisites are in place.

You automated the back translation process using SageMaker Processing by running your script in an ML instance managed by SageMaker AI. Along the way, you also learned best practices for designing, managing, and scaling your data-processing workflows efficiently and securely.

In the next chapter, we will explore a variety of model-training and fine-tuning strategies with SageMaker AI. Building on what you've learned so far, you'll see how SageMaker allows you to focus on your training requirements while it manages the underlying infrastructure for you. See you in the next chapter!

# Further reading

To expand your practical knowledge of the concepts presented in this chapter, refer to the following resources:

- *Amazon SageMaker AI Developer Guide* | *Data transformation workloads with SageMaker Processing* (`https://docs.aws.amazon.com/sagemaker/latest/dg/processing-job.html`)
- *Amazon SageMaker AI Developer Guide* | *How to Build Your Own Processing Container (Advanced Scenario)* (`https://docs.aws.amazon.com/sagemaker/latest/dg/build-your-own-processing-container.html`)
- *Amazon SageMaker AI Developer Guide* | *Run SageMaker Clarify Processing Jobs for Bias Analysis and Explainability* (`https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-processing-job-run.html`)
- *Hugging Face* | *Helsinki-NLP Research Group* (`https://huggingface.co/Helsinki-NLP`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 7

# SageMaker AI Model Training and Tuning Capabilities

In the past, if you wanted to fine-tune a **large language model** (**LLM**), you typically had to set up and manage everything yourself. You were responsible for configuring the training environment, installing the correct dependencies, and provisioning compute resources capable of handling large-scale workloads. You also needed to manage how model artifacts were stored and organized and build your own deployment pipeline before the model could be used in production. All of this added significant operational overhead and often slowed you down when your main goal was to improve the model itself. Today, you can use Amazon SageMaker AI to handle much of that heavy lifting. It provides managed capabilities for training, hyperparameter tuning, and deployment, allowing you to focus more on building and improving your model instead of managing infrastructure.

In this chapter, you will explore how to fine-tune a LLM using SageMaker AI as part of an end-to-end machine learning workflow. You will begin by setting up a serverless MLflow App to track and manage training experiments, followed by executing a supervised fine-tuning job. You will then see how SageMaker AI automatically provisions and orchestrates the required compute resources for training. After that, you will perform hyperparameter tuning to systematically evaluate multiple configurations in parallel and identify the best-performing model based on a defined objective metric. Finally, you will deploy the selected model to a real-time inference endpoint for serving predictions.

We will cover the following topics in this chapter:

- Setting up a serverless MLflow App
- Fine-tuning an LLM on Amazon SageMaker AI
- Deploying the Fine-Tuned Model

- Performing Hyperparameter Tuning with Amazon SageMaker AI
- Deploying the Best-Performing Model from Hyperparameter Tuning

By the end of this chapter, you will see how SageMaker AI makes it easier to train models and tune hyperparameters, so you can focus on improving model performance instead of managing training infrastructure.

# Technical requirements

Before proceeding with the hands-on examples in this chapter, ensure that the following prerequisites and setup requirements are in place:

- **An existing SageMaker Studio space**: You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1*. Ensure that the space is configured with sufficient storage capacity (for example, at least 100 GB) to accommodate datasets, model artifacts, logs, and intermediate training outputs generated throughout the examples in this chapter.
- **Sufficient account-level quota for selected ML instance types**: Ensure that your AWS account has the required applied account-level quota values for the SageMaker AI ML instance types used in this chapter. You should have at least 1× `ml.g5.4xlarge` for the real-time inference endpoint (**ml.g5.4xlarge for endpoint usage**), at least 3× `ml.m5.xlarge` training instances to support hyperparameter tuning (**ml.m5.xlarge for spot training job usage**), and at least 1× `ml.m5.xlarge` for the real-time inference endpoint (**ml.m5.xlarge for endpoint usage**). If any of these quotas are set to 0 or below the required level, SageMaker AI will not be able to provision the necessary compute resources, causing training, tuning, or endpoint creation to fail. You can review and adjust these limits in the Service Quotas console in the AWS Management Console.
- **A code editor installed on your local machine** (such as Visual Studio Code or Sublime Text): You'll need this when working with the code and configuration files used throughout the hands-on exercises and examples in this book.

The Jupyter notebooks, source code, and other files used for each chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

# Setting up a serverless MLflow app

When you are building machine learning models, tracking your experiments is just as important as training them. Without proper tracking, it becomes difficult to understand what

worked, what didn't, and how to reproduce your results. As an ML engineer, managing experiments can quickly become overwhelming, especially when you are running multiple training jobs and trying different configurations. Tracking metrics, parameters, logs, and model artifacts across local files or unstructured storage can lead to confusion and inconsistency. To address these challenges, you need a centralized way to organize and manage all experiment-related information.

In this section, you will set up a serverless MLflow App to track and manage your experiments in a more organized and scalable way. Once it is set up, MLflow provides a single interface where all experiment data is logged and easily accessible. This allows you to compare, analyze, and review different runs without manually managing files or organizing outputs. In SageMaker AI, the MLflow App provides a fully managed experience where AWS handles provisioning, scaling, and maintenance of the MLflow tracking infrastructure. This means you do not need to deploy or manage a tracking server yourself, while still using standard MLflow APIs to log experiments, parameters, metrics, and artifacts. As a result, you can focus on experiment tracking and model development rather than infrastructure setup and operational overhead.

*Figure 7.1 — Using MLflow to track experiments and manage model artifacts*

Let's say you used supervised fine-tuning for model customization on a domain-specific dataset. MLflow will automatically log the relevant training metrics for that run. You should

see the model metrics recorded under the selected MLflow experiment run, including values such as **epoch**, **gpt_loss**, **grad_norm**, **lr**, **total_loss**, and **total_tokens_in_step**, along with other training metrics captured during training. This allows you to easily analyze and review experiment results, compare different runs, and understand how changes in configuration impact model performance.

> **Note**
>
> This section assumes that you are running the hands-on examples inside a space in Amazon SageMaker Studio. You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1*.

Let's go through the steps to set up an MLflow App:

1. In the left sidebar of JupyterLab, right-click and select **New Folder**. Name the folder `sft`, then double-click it to navigate into the directory. Alternatively, you can create the folder by clicking the **New Folder** button in the File Browser toolbar, then rename it to `sft` and open it to navigate into the directory.

   > **Note**
   >
   > All notebooks and associated files for this section and the next two sections should be created and stored in this directory.

2. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `mlflow_app_setup.ipynb`. We will run the code in the following steps within this notebook.
3. In a new notebook cell, upgrade boto3 by running the following command:

   ```
   %pip install --upgrade boto3
   ```

   This ensures that your notebook is using the latest version of `boto3`, which gives you access to the most recent AWS service features and API updates.

   > **Note**
   >
   > The examples in this chapter have been tested using boto3 version `1.42.94`. Versions close to this release or newer should work without issues, as they

> include the required APIs and features used throughout the exercises. After restarting the kernel in the next step, you can verify the installed version by running `%pip show boto3`.

4. Now that we have installed the latest version of `boto3`, restart the kernel by selecting **Kernel | Restart Kernel** from the menu. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.
5. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length. We'll use this to give our endpoints and other resources a unique name so they don't conflict with existing ones in your account.

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase,
                       k=length))
```

6. Let's generate a random string using the `generate_string()` function and store it in the variable `unique`. Print the value of `unique` afterward.

```
unique = generate_string()
```

7. Generate a unique name for the Amazon S3 bucket that will serve as the MLflow App artifact store:

```
mlflow_s3_bucket = f"mlflow-s3-bucket-{unique}"
```

8. Create the S3 bucket using the AWS CLI:

```
!aws s3 mb s3://{mlflow_s3_bucket}
```

9. Generate a unique name for the MLflow App:

```
mlflow_app_name = f"mlflow-app-{unique}"
```

10. Import `boto3` and initialize a SageMaker AI client:

```
import boto3
client = boto3.client("sagemaker")
```

11. Use the `help()` function to inspect the `create_mlflow_app()` method and understand its required and optional parameters:

```
help(client.create_mlflow_app)
```

You should see output similar to the following, helping you review the request syntax and identify the required parameters such as `Name`, `ArtifactStoreUri`, `RoleArn`, and `ModelRegistrationMode` for the MLflow App creation steps that follow.

```
Help on method create_mlflow_app in module botocore.client:

create_mlflow_app(*args, **kwargs) ...
    Creates an MLflow Tracking Server using a general purpose Amazon S3
bucket as the artifact store.

    See also: ...

    **Request Syntax**
    ::

      response = client.create_mlflow_app(
          Name='string',
          ArtifactStoreUri='string',
          RoleArn='string',
          ModelRegistrationMode='...',
          WeeklyMaintenanceWindowStart='string',
          AccountDefaultStatus='ENABLED'|'DISABLED',
          ...
      )
     ...
```

Let's go through some of the parameters you'll provide when creating the MLflow App. The `Name` parameter refers to a unique identifier you will provide for the MLflow App. The `ArtifactStoreUri` corresponds to the Amazon S3 bucket location where MLflow will store experiment artifacts, including models, metrics, and logs. The `RoleArn`

corresponds to the IAM role that SageMaker AI will assume to access the S3 artifact store, and it should include permissions that allow read and write access to the specified S3 bucket. The `ModelRegistrationMode` maps to whether models created in MLflow will be automatically registered in the SageMaker Model Registry, allowing you to enable or disable this behavior depending on your workflow.

> **Note**
>
> For more information on `create_mlflow_app()`, feel free to check: `https://docs.aws.amazon.com/boto3/latest/reference/services/sagemaker/client/create_mlflow_app.html`.

12. Retrieve the execution role that SageMaker AI will use when configuring the MLflow App:

```
from sagemaker.core.helper.session_helper import (
    get_execution_role
)

role = get_execution_role()
```

13. Create the MLflow App by specifying the required configuration, including the app name, S3 bucket for storing artifacts, and the IAM execution role ARN:

```
registration_mode = "AutoModelRegistrationEnabled"

response = client.create_mlflow_app(
    Name=mlflow_app_name,
    ArtifactStoreUri="s3://" + mlflow_s3_bucket,
    RoleArn=role,
    ModelRegistrationMode=registration_mode
)
```

14. List the available MLflow Apps and retrieve the ARN of the newly created app:

```
apps = client.list_mlflow_apps()

newly_created_app = max(
    apps["Summaries"],
    key=lambda x: x["CreationTime"]
```

```
)

app_arn = newly_created_app["Arn"]
app_arn
```

15. Use `describe_mlflow_app()` to inspect the current configuration and status of the MLflow App you just created.

```
client.describe_mlflow_app(Arn=app_arn)
```

This returns a response similar to the following, which includes key metadata such as the app name, S3 artifact location, execution role, and current provisioning status:

```
{'Arn': 'arn:aws:.../app-CLPGKFL4SEH7',
 'Name': 'mlflow-app-...',
 'ArtifactStoreUri': 's3://...',
 'RoleArn': '...',
 'Status': 'Creating',
  ...}
```

16. Define a `wait_for_mlflow_app()` function that polls the MLflow App status until provisioning is complete:

```
from time import sleep

def wait_for_mlflow_app(arn):
    while True:
        response = client.describe_mlflow_app(
            Arn=app_arn
        )
        status = response['Status']
        print(f"STATUS: {status}")

        if status != 'Creating':
            break

        sleep(15)
```

17. Call the function to wait until the MLflow App is ready:

```
wait_for_mlflow_app(app_arn)
```

> **Note**
>
> The MLflow App provisioning process may take around 5 to 15 minutes to complete. Feel free to grab a cup of coffee or tea while you wait.

18. Once provisioning is complete, print the MLflow App ARN:

```
app_arn
```

This outputs the unique Amazon Resource Name associated with the MLflow App:

```
'arn:aws:sagemaker:us-east-1:...:mlflow-app/app-...'
```

Make sure to copy this ARN to a text editor on your local machine for later reference (**TAG**: **MLFLOW APP ARN**), as you will need it in subsequent steps.

> **Note**
>
> Before proceeding with the next steps, make sure to double-check that the `app_arn` value corresponds to the MLflow App you just created in this chapter. This is especially important if multiple MLflow Apps already exist in your account, as `list_mlflow_apps()` may return several entries. Verifying the ARN ensures that you are working with the correct MLflow App resource throughout the rest of the chapter.

At this stage, you have successfully provisioned a serverless MLflow App. This setup provides a centralized and convenient way to track experiments, store model artifacts, and manage model lifecycle metadata without requiring you to manage any underlying servers. In the next section, you will use this tracking environment to log and monitor a supervised fine-tuning job as you train a LLM on SageMaker AI.

# Fine-tuning an LLM on Amazon SageMaker AI

Fine-tuning is a key step in adapting LLMs to real-world applications beyond their general-purpose pretraining. It allows you to specialize a model for a specific domain, incorporate industry-specific terminology, and teach it new task behaviors that are not well represented in the original training data. In many cases, it can also help reduce hallucinations by grounding the model in curated, domain-relevant examples. Compared to retrieval-augmented generation, fine-tuning adjusts the model's parameters to influence behavior and task performance, while retrieval-augmented generation relies on external data sources at inference time to supply relevant context.

In this section, you will perform supervised fine-tuning to customize a LLM on a domain-specific dataset using SageMaker AI. You will use the SageMaker Python SDK V3 and its built-in `SFTTrainer`, one of the SDK's specialized fine-tuning Trainers, to run the training job. You will track experiments using the MLflow App you set up in the previous section and inspect the resulting outputs and artifacts generated during training.

There are multiple ways to perform supervised fine-tuning in SageMaker AI. One approach is to run a training job, where SageMaker AI provisions compute instances for the duration of the training and automatically terminates them once the job completes. Another approach is to use **SageMaker HyperPod**, which is designed for large-scale, long-running training workloads that require persistent and highly optimized compute clusters. In this section, you will focus on using a **SageMaker training job** to fine-tune a LLM, where SageMaker AI handles infrastructure provisioning, scaling, and cleanup of compute resources, allowing you to focus on configuring and executing the training workflow. The training job will be launched from a Jupyter notebook running in a SageMaker Studio environment, similar to what is shown in *Figure 7.2*.

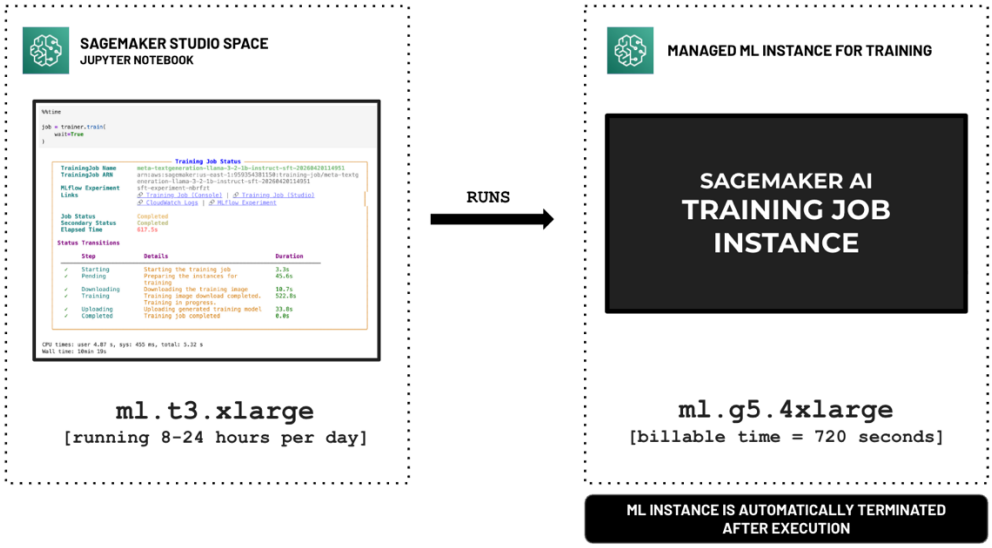



*Figure 7.2 — Separation of development environment and managed training infrastructure*

In this setup, a Jupyter notebook running in a SageMaker Studio space, such as on an `ml.t3.xlarge` instance, is used to configure and start training jobs. The actual model training step runs on a separate managed compute instance, such as `ml.g5.4xlarge`, which is provisioned on demand and automatically deleted after the completion of the training job. With this setup, you can optimize costs since high-end compute instances are provisioned only when required for training workloads.

If you are wondering what happens to the data stored on the training instance after it is deleted when the training job completes, the training artifacts such as model checkpoints, logs, and outputs are not lost. Before the instance is terminated, these artifacts are saved to a specified Amazon S3 bucket as part of the training job configuration, ensuring that the results of the training job remain available for later use, evaluation, and deployment.

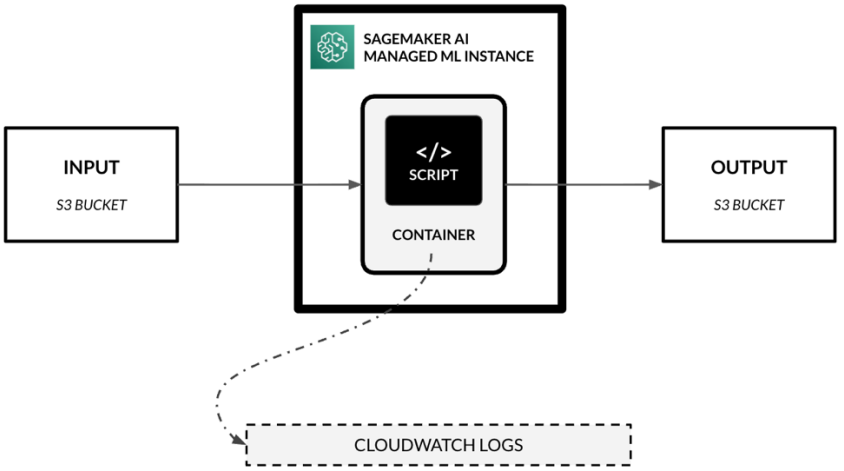



*Figure 7.3 — Data flow between Amazon S3 and SageMaker AI training instances*

Without a managed service, running a training workflow requires you to handle multiple operational tasks, including setting up the development environment, provisioning and configuring compute resources, managing training execution, and handling data movement between storage and training infrastructure. SageMaker AI reduces this complexity by decoupling the development environment from the training infrastructure and automatically managing the full lifecycle of training jobs.

SageMaker AI training jobs follow a managed execution lifecycle in which the underlying infrastructure is provisioned on demand. When a training job starts, SageMaker AI downloads the input data from the specified S3 bucket along with the specified training image from a container registry. It then launches a containerized training environment on a managed compute instance and runs the training step inside that container.

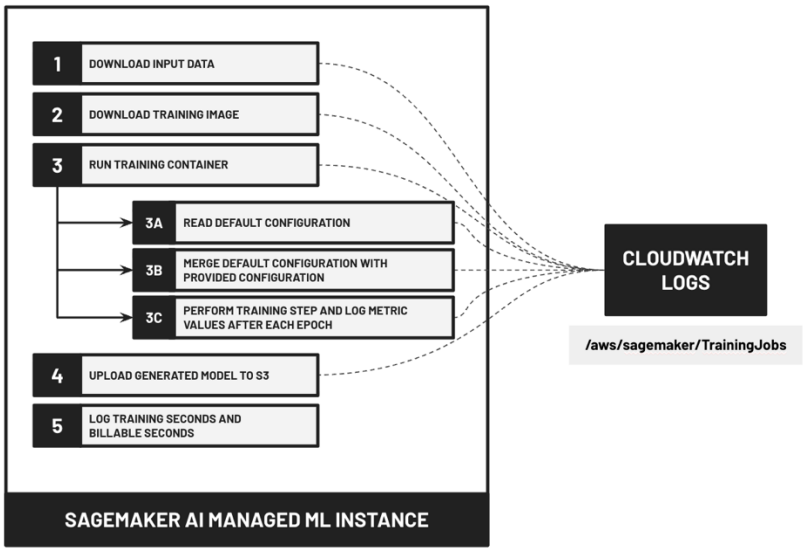



*Figure 7.4 — Managed execution lifecycle of a SageMaker AI training job*

During execution, the training script inside the container reads input data, performs model optimization, and periodically logs metrics such as loss and other performance indicators. As the training job completes, the resulting model artifacts, logs, and checkpoints are saved to the specified S3 bucket. The compute instance is then automatically terminated. Throughout the process, SageMaker AI also records runtime information such as training duration and billable compute time to support monitoring and cost tracking.

Last year, specialized model fine-tuning capabilities were introduced in the **SageMaker Python SDK V3**, making it easy for practitioners to apply advanced model alignment techniques through a simplified interface. More specifically, the SDK introduced four specialized fine-tuning Trainers for different fine-tuning techniques:

- `SFTTrainer` for **Supervised Fine-Tuning** which enables you to adapt LLMs using labeled instruction-response pairs. This approach is typically used to specialize a general-purpose model for downstream tasks such as question answering, summarization, and instruction following.
- `DPOTrainer` for **Direct Preference Optimization** which provides a way to align model behavior with human preference data without introducing a separate reward model. By directly optimizing based on preference data, it simplifies the alignment process while supporting use cases such as improving response quality, safety alignment, and style adaptation.

- `RLAIFTrainer` for **Reinforcement Learning from AI Feedback** which allows you to use Amazon Bedrock models as automated evaluators to generate reward signals. This reduces dependence on human annotation while supporting scalable alignment workflows for improving helpfulness, coherence, and response quality.
- `RLVRTrainer` for **Reinforcement Learning from Verifiable Rewards** which is intended for cases where reward signals are programmatically defined and can be validated through code or rules. It is well suited for tasks with deterministic evaluation criteria such as correctness checks, structured output validation, or domain-specific constraints.

These four Trainers all follow a consistent and familiar structure, so you can apply different fine-tuning techniques without changing how you interact with the SDK and without having to worry about managing the underlying infrastructure. In addition, they have built-in MLflow integration that handles experiment tracking, metrics logging, and artifact management for you.

**Note**

You will see the `SFTTrainer` in action in the hands-on portion of this section as you set up and run a supervised fine-tuning job using SageMaker Python SDK V3.

With these in mind, let's proceed with fine-tuning an LLM on SageMaker AI:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `training_001.ipynb`. We will run the code in the following steps within this notebook.
2. Uninstall any existing versions of `sagemaker` and `sagemaker-serve`, then install the required versions (`sagemaker==3.5.0` and `sagemaker-serve==1.2.0`):

```
%pip uninstall -y sagemaker sagemaker-serve
%pip install "sagemaker==3.5.0"
%pip install "sagemaker-serve==1.2.0"
```

3. Let's install the `conda-pack` package as well:

```
%pip install conda-pack
```

4. Now that we've installed the required packages and package versions, restart the kernel by selecting **Kernel | Restart Kernel** from the menu. This will help ensure the newly installed package versions are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

> **Note**
>
> After installing the required package versions, verify that the correct versions are in place by running `%pip show sagemaker` and `%pip show sagemaker-serve`. You should see `sagemaker 3.5.0` and `sagemaker-serve 1.2.0`, which confirms that your environment is ready for the examples in this chapter.

5. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length. We'll use this to give our endpoints and other resources a unique name so they don't conflict with existing ones in your account.

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase,
                       k=length))
```

6. Let's generate a random string using the `generate_string()` function and store it in the variable `unique`. Print the value of `unique` afterward.

```
unique = generate_string()
print(unique)
```

This should print a randomly generated string, similar to `fgmgfi`. We'll use this randomly generated string to avoid naming conflicts when creating new resources.

7. Using the identifier generated in the previous step, construct a unique S3 bucket name by appending it to a base name.

```
s3_bucket = f"mle-on-aws-2nd-ed-{unique}"
```

8. Let's check the bucket name before creating it:

```
s3_bucket
```

9. Now, create the S3 bucket using the `aws s3 mb` command. This bucket will be used to store datasets, model artifacts, and other resources.

```
!aws s3 mb s3://{s3_bucket}
```

10. Run the following block of code to download the `data.jsonl` file from the official GitHub repository:

```
import urllib.request
from pathlib import Path

URL_PARTS = [
    "https://raw.githubusercontent.com/",
    "PacktPublishing/",
    "Machine-Learning-Engineering-on-",
    "AWS-Second-Edition/",
    "refs/heads/main/",
    "chapter07/data.jsonl",
]

url = "".join(URL_PARTS)
output_file = Path("data.jsonl")
urllib.request.urlretrieve(url, output_file)
```

This file contains the training dataset in JSON Lines format, where each line represents a prompt-completion pair used for supervised fine-tuning.

11. After downloading the dataset, inspect the first few records to understand its structure and content.

```
!head data.jsonl
```

Each entry contains a prompt and a corresponding completion, which will be used during training.

```
{
  "prompt": "User: Recommend a movie to ...",
```

```
    "completion": "I would recommend the movie, ..."
  }
  {
    "prompt": "User: Determine the result obtained ...",
    "completion": "5338245 - 50629795848152 = ..."
  }
  {
    "prompt": "User: Considering that a film ...",
    "completion": "Step 1: Convert the duration ..."
  }
  ...
```

12. Next, use the `aws s3 cp` command to upload the dataset to Amazon S3 so it can be accessed by the SageMaker training job.

```
!aws s3 cp data.jsonl s3://{s3_bucket}/data/data.jsonl
```

13. Create a model package group to organize and manage different versions of your trained models.

```
from sagemaker.core.resources import (
    ModelPackageGroup
)

group_name = f"model-package-group-{unique}"
model_package_group=ModelPackageGroup.create(
    model_package_group_name=group_name
)
```

This allows you to track, version, and register models as part of a structured deployment workflow.

14. Specify the Amazon S3 path to the training dataset and store it in the `dataset` variable.

```
dataset = f"s3://{s3_bucket}/data/data.jsonl"
```

This path tells SageMaker AI where the training dataset is stored in Amazon S3. During the training job, SageMaker AI automatically retrieves the data from this location and makes it available to the training container.

15. Define the S3 output path where the training artifacts, including the fine-tuned model, logs, and intermediate outputs, will be stored.

```
output_path = f"s3://{s3_bucket}/output/"
```

16. Configure MLflow to use the tracking server associated with your serverless MLflow App. Replace `<TRACKING ARN>` with the MLflow App ARN (**TAG: MLFLOW APP ARN**) obtained from the MLflow App created earlier in *Setting up a serverless MLflow App*.

```
import mlflow
tracking_server = "<TRACKING ARN>"
mlflow.set_tracking_uri(tracking_server)
```

This configures MLflow to use the serverless MLflow App you set up in the previous section, which enables experiment tracking, logging, and visualization of training runs through the MLflow App.

> **Note**
>
> You can run `help(mlflow)` to view MLflow's modules, APIs, and usage information.

17. Set the MLflow experiment and run names.

```
mlflow_experiment_name = f"sft-experiment-{unique}"
mlflow_run_name = f"run-{unique}"
```

These identifiers help you organize and distinguish different training runs within the MLflow App.

18. Specify the base model to fine-tune.

```
model = "meta-textgeneration-llama-3-2-1b-instruct"
```

In this section, you will use a pre-trained LLM, specifically a Llama 3.2 Instruct variant. The model has been pre-trained on a broad corpus of text and instruction fine-tuned during post-training to improve its ability to follow natural language instructions, making it suitable for supervised fine-tuning on task-specific datasets.

19. Initialize an `SFTTrainer` object, which you will use to set up and run supervised fine-tuning for the model.

```
from sagemaker.train.sft_trainer import SFTTrainer
from sagemaker.train.common import TrainingType

trainer = SFTTrainer(
    model=model,
    training_type=TrainingType.LORA,
    model_package_group=model_package_group,
    training_dataset=dataset,
    s3_output_path=output_path,
    accept_eula=True,
    mlflow_experiment_name=mlflow_experiment_name,
    mlflow_run_name=mlflow_run_name
)
```

Here, you configure the model, apply LoRA-based parameter-efficient fine-tuning, and define the training dataset and output S3 path. You also provide MLflow experiment and run names to enable tracking and comparison of training runs.

20. Let's inspect the `SFTTrainer` object (`trainer`) configuration to verify that all parameters have been set correctly before starting the training job.

```
trainer.__dict__
```

This should yield the current configuration of the SFTTrainer object as a Python dictionary, containing all initialized parameters such as the base model, training type, dataset paths, output configuration, and MLflow settings:

```
{
 'sagemaker_session': ...,
 'role': ...,
 'base_job_name': ...,
 'tags': ...,
 'hyperparameters': ...,
 'output_data_config': ...,
 'input_data_config': ...,
  ...
 'stopping_condition': ...,
 '_model_arn': ...,
```

```
  'accept_eula': ...
}
```

21. Next, inspect the hyperparameters that will be used during training.

```
from pprint import pprint
pprint(trainer.hyperparameters.to_dict())
```

You should see the hyperparameter configuration resolved by the `SFTTrainer`, including training settings and MLflow-related fields:

```
{'global_batch_size': '8',
 'learning_rate': '0.0001',
 'lr_warmup_ratio': '0.1',
 'max_epochs': '5',
 'mlflow_run_id': '',
  ...
 'train_val_split_ratio': '0.9'}
```

These hyperparameters define key aspects of the training process, including batch size, learning rate, number of training epochs, warmup strategy, and the train-validation split ratio used during fine-tuning.

22. Start the training job using the configured SFTTrainer.

```
%%time

job = trainer.train(
    wait=True
)
```

Here, we set `wait=True` to wait for the training job to finish before continuing. You should see the **Training Job Status** box (*Figure 7.5*), which includes key metadata such as the **TrainingJob Name**, **TrainingJob ARN**, **MLflow Experiment**, **Job Status**, **Secondary Status**, and **Elapsed Time**, which help you track the progress and lifecycle of the training run in real time.

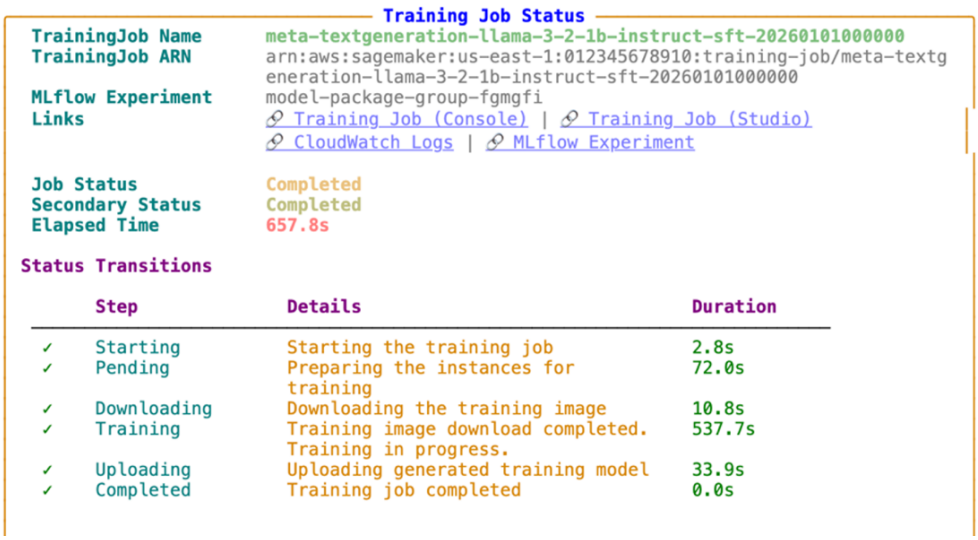



*Figure 7.5 — Training Job Status*

The **Training Job Status** box also provides links or references to the corresponding details page (AWS Management Console or SageMaker Studio) for the SageMaker AI training job, as well as the associated MLflow Experiment details page for further inspection.

> **Note**
>
> If you encounter the error `ClientError: Could not load HF dataset from disk. Also failed to load jsonl dataset from directory '/opt/ml/input/data/train'`, it may be caused by either a transient issue (such as temporary I/O delays or container initialization timing) or a problem with the dataset configuration. As a first step, you can retry the training job, as transient issues may resolve on a subsequent attempt. However, if the error persists, verify that the dataset is correctly uploaded, properly formatted, and accessible at the expected path.

23. Under **Training Job Status**, click **MLflow Experiment**. This should open the **Experiments** page showing the list of experiments available in the MLflow tracking server. Wait until the experiment with the specified experiment name (`sft-experiment-...`) appears in the list. Click the experiment name under the **Name**

column associated with the experiment created for the training job to open the experiment detail page, similar to what is shown in *Figure 7.6*.

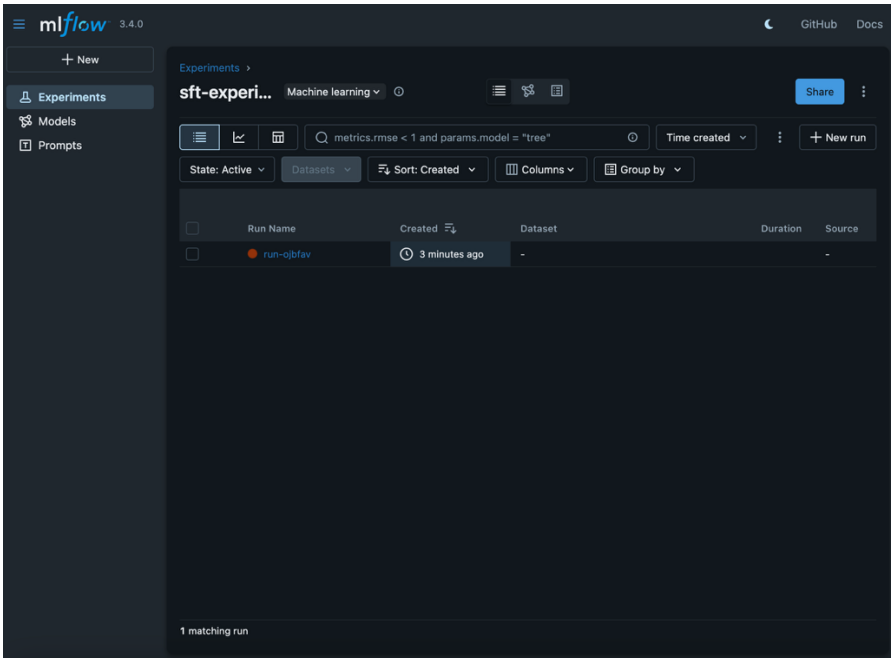



*Figure 7.6 — Experiment detail page*

Here, you can view the experiment run(s) along with details associated with your SageMaker AI training job(s), including **Run Name**, **Created**, **Dataset**, **Duration**, **Source**, and **Models**.

> **Note**
>
> If you encounter an error when opening the MLflow **Experiments** page from **Training Job Status**, switch to the browser tab with SageMaker Studio open, navigate to **MLflow** under **Applications**, locate the MLflow App you created (under **MLflow Apps**), and click **Open MLflow** to access the user interface for the MLflow App.

24. Click the run name (under **Run Name**) to open the detail page, similar to what is shown in *Figure 7.7*.

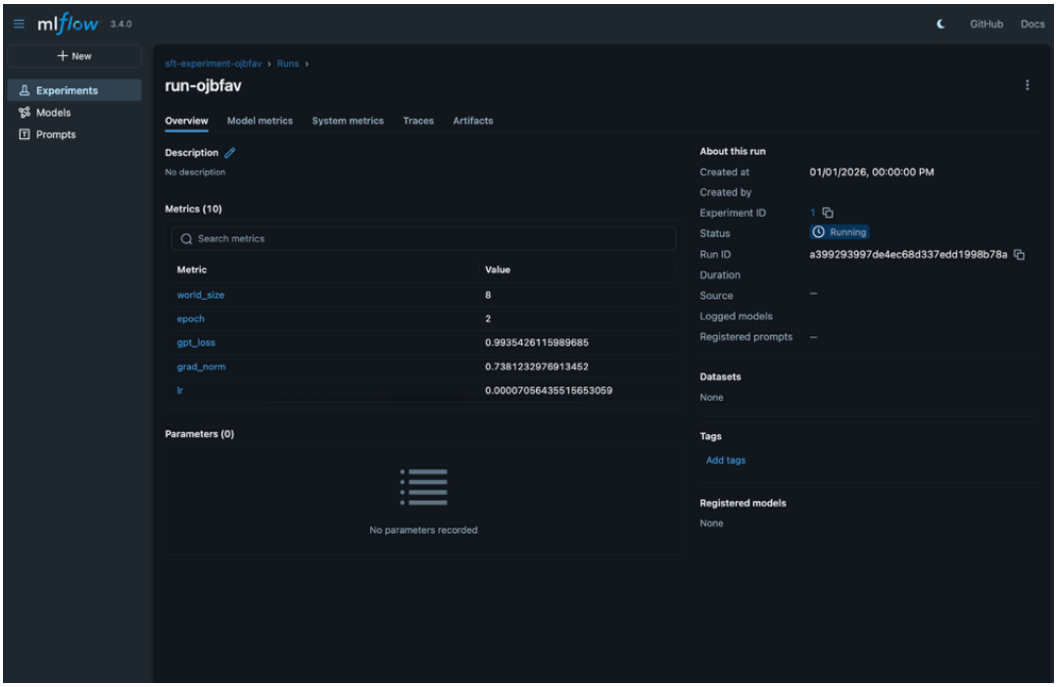



*Figure 7.7 — MLflow experiment run detail page*

Here you can inspect the full experiment run information, including parameters, model metrics, system metrics, traces, artifacts, and other details.

25. Navigate to the **Model metrics** tab to view the evaluation metrics associated with the selected experiment run.

*Figure 7.8 — Viewing the model metrics for the selected MLflow experiment run*

You should see metrics such as **epoch**, **gpt_loss**, **grad_norm**, **lr**, **total_loss**, **total_tokens_in_step**, and other metrics recorded during the run.

26. Switch back to the notebook (*training_001.ipynb*) and click **Training Job (Studio)** under **Training Job Status**. This should open the corresponding SageMaker Studio details page associated with the training job (*Figure 7.9*).

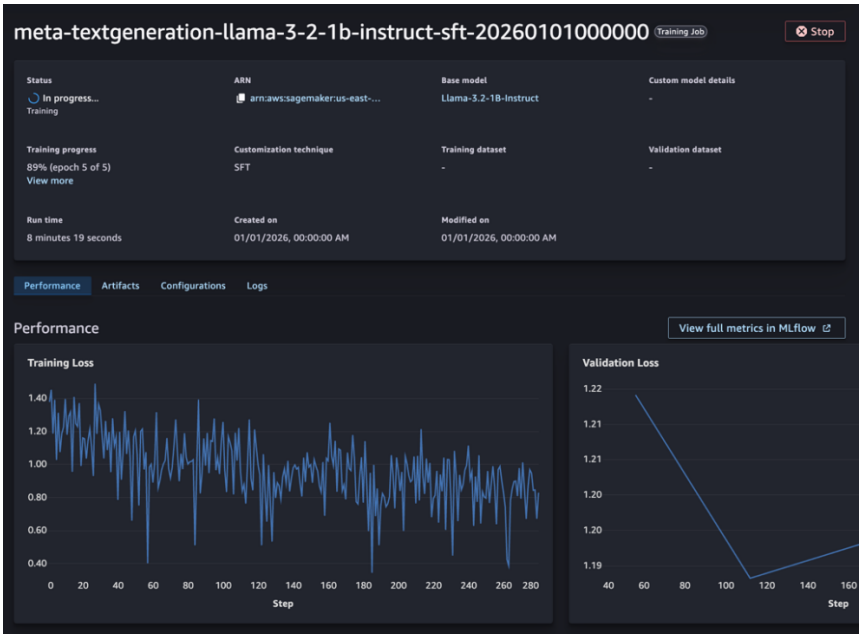



*Figure 7.9 — Training job details page (SageMaker Studio)*

Here, you can view the training job status, ARN, base model, training progress, customization technique, run time, performance metrics, artifacts, and other related metadata associated with the training job.

27. Switch back to the notebook (*training_001.ipynb*) and wait for the SageMaker AI training job to complete before proceeding with the next steps.

> **Note**
>
> Since only a fraction of the fine-tuning dataset (`trl-lib/Capybara`) was used during training, the model may not achieve its optimal performance. As a result, responses may be less accurate, less consistent, or less context-aware compared to a fully trained model.

28. Run the following to inspect the training job object (`job`).

```
pprint(job.__dict__)
```

This returns a dictionary containing detailed training job metadata, similar to the following:

```
{ ...
 'hyper_parameters': {...},
  ...
 'input_data_config': [...],
  ...,
 'output_data_config': OutputDataConfig(...),
 'output_model_package_arn': '...',
  ...
 'progress_info': TrainingProgressInfo(...),
  ...
 'role_arn': '...',
  ...
 'training_job_status': 'Completed',
  ...
}
```

Here, you can inspect the training job configuration and metadata, including hyperparameters, input data configuration, output data configuration, model artifact information, progress information, IAM role ARN, and the training job status.

29. Build a model from the training job output using `ModelBuilder`:

```
from sagemaker.serve import ModelBuilder

model_builder = ModelBuilder(
    model=job,
    instance_type="ml.g5.4xlarge"
)

model = model_builder.build()
```

30. Inspect the model object to view its metadata:

```
pprint(model.__dict__)
```

This returns the model's configuration and metadata, including fields such as the container definition, model name, and model ARN:

```
{'containers': [ContainerDefinition(...)],
  ...
 'model_arn': '...:model/model-ab2885ae',
 'model_name': 'model-ab2885ae',
  ...}
```

31. Retrieve the location of the trained model artifacts from the training job object:

```
artifacts = job.__dict__['model_artifacts']
```

32. List the contents of the S3 bucket containing the trained model artifacts:

```
ad = artifacts.__dict__
s3_model_artifacts = ad['s3_model_artifacts']

!aws s3 ls {s3_model_artifacts} --recursive
```

This displays model checkpoint files, tokenizer files, configuration files, and training state artifacts.

```
...
2026-01-01 00:00:00        348 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
chat_template.jinja
2026-01-01 00:00:00        865 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/config.json
...
2026-01-01 00:00:00      50525 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
tokenizer_config.json
2026-01-01 00:00:00        884 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/step_0056/
client_state.pt
...
```

33. Use `aws s3 ls` to inspect the contents of the `hf_merged` model checkpoint directory:

```
merged = f"{s3_model_artifacts}/checkpoints/hf_merged"

!aws s3 ls {merged} --recursive
```

This lists the merged model artifacts stored in Amazon S3 after fine-tuning and model merging (for example, after LoRA merging into the base model). This should produce output similar to:

```
2026-01-01 00:00:00        348 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
chat_template.jinja
2026-01-01 00:00:00        865 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/config.json
2026-01-01 00:00:00        184 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
generation_config.json
2026-01-01 00:00:00 2471645608 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
model.safetensors
2026-01-01 00:00:00        296 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
special_tokens_map.json
2026-01-01 00:00:00   17208712 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
tokenizer.json
2026-01-01 00:00:00      50525 output/meta-textgeneration-llama-3-2-1b-
instruct-sft-20260101000000/output/model/checkpoints/hf_merged/
tokenizer_config.json
```

> **Note**
>
> **Low-Rank Adaptation (LoRA)** is a parameter-efficient fine-tuning technique where only a small set of additional trainable weights is learned while the base model weights remain frozen, making training more efficient in terms of compute and memory. Model merging refers to the process of combining these learned LoRA adapter weights back into the base model weights to produce a single merged checkpoint. After merging, the model can be used directly for inference without separate adapter weights, which simplifies deployment and production usage.

34. Use the `mkdir` command to create a local directory for storing model artifacts:

```
model_files_directory = f"model_files_{unique}"
!mkdir -p {model_files_directory}
```

35. Copy the merged model artifacts from Amazon S3 to the local directory created in the previous step:

```
%%time
!aws s3 cp {s3_model_artifacts}/checkpoints/hf_merged
{model_files_directory}/ --recursive
```

36. Use the `tar` command to create a compressed archive of the model files:

```
%%time
!tar -czvf model.tar.gz -C {model_files_directory} .
```

> **Note**
>
> This step may take approximately 5 to 15 minutes to complete. Feel free to grab a cup of coffee or tea while you wait.

37. Next, use the `aws s3 cp` command to upload the compressed model artifact to Amazon S3:

```
%%time
s3_model_path = f"s3://{s3_bucket}/model/model.tar.gz"
!aws s3 cp model.tar.gz {s3_model_path}
```

38. Store the S3 model path as well as the S3 bucket name in the notebook state (IPython store) for reuse in later steps:

```
%store s3_model_path
%store s3_bucket
```

39. Retrieve the container image URI associated with the model and store it for later use:

```
containers = model.__dict__['containers']
container_image = containers[0].__dict__['image']
%store container_image
```

40. Store the local model files directory name as well:

```
%store model_files_directory
```

41. Retrieve the SageMaker execution role and store the role ARN in the IPython store for reuse in later steps.

```
from sagemaker.core.helper.session_helper import (
    get_execution_role
)

role = get_execution_role()
%store role
```

42. Finally, store the `unique` identifier used for naming resources:

```
%store unique
```

Great job! You have just completed an end-to-end supervised fine-tuning workflow using SageMaker AI, from dataset preparation and training configuration to running a managed training job and inspecting the resulting model artifacts. You also integrated MLflow for

experiment tracking and observed how training outputs are structured for analysis, versioning, and reuse. In the next section, you will deploy the fine-tuned model and see how it can be used for real-time inference.

# Deploying the Fine-Tuned Model

In this section, you will configure and deploy the fine-tuned Llama 3.2 Instruct model for real-time inference. You will use `boto3` to create the SageMaker AI model, define the endpoint configuration, and launch a real-time inference endpoint. After deployment, you will assess the model's responses using different prompts.

In case you're wondering about the difference between the SageMaker Python SDK and `boto3`, the SageMaker Python SDK provides a higher-level interface that simplifies common machine learning workflows such as training, hyperparameter tuning, and deployment by abstracting much of the underlying infrastructure details. In contrast, `boto3` provides lower-level access to AWS service APIs, giving you more direct and explicit control over resources like models, endpoint configurations, and endpoints, which is useful when you want to understand or customize each step of the deployment process.

**Note**

In this chapter, you will use `boto3` to perform a straightforward deployment of the fine-tuned model to a SageMaker AI real-time inference endpoint. In the next chapter, *SageMaker AI Model Deployment Options and Strategies*, you will explore more advanced deployment patterns and management strategies for production-grade inference workflows.

Let's proceed with deploying the fine-tuned model to a real-time inference endpoint:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `training_002.ipynb`. We will run the code in the following steps within this notebook.
2. Load the previously stored variables so you can reuse the configuration values required for model deployment, including the container image, model artifacts, and execution role.

```
%store -r unique
%store -r role
%store -r container_image
```

```
%store -r model_files_directory
%store -r s3_model_path
%store -r s3_bucket
```

3. Create a SageMaker AI model by specifying the inference container image and the Amazon S3 location of the trained model artifacts.

```
import boto3
sm = boto3.client("sagemaker")
model_name = f"model-{unique}"

sm.create_model(
    ModelName=model_name,
    PrimaryContainer={
        "Image": container_image,
        "ModelDataUrl": s3_model_path,
        "Environment": {
            "SHM_SIZE": "1g",
            "HF_MODEL_ID": "/opt/ml/model",
            "MAX_INPUT_LENGTH": "2048",
            "MAX_TOTAL_TOKENS": "4096",
            "NUM_SHARD": "1"
        }
    },
    ExecutionRoleArn=role
)
```

Here, we also configure environment variables to manage runtime inference settings such as token limits, memory usage, and model loading configuration.

4. Create an endpoint configuration that defines how the model will be deployed.

```
endpoint_config_name = f"endpoint-config-{unique}"

sm.create_endpoint_config(
    EndpointConfigName=endpoint_config_name,
    ProductionVariants=[
        {
            "VariantName": "AllTraffic",
            "ModelName": model_name,
            "InitialInstanceCount": 1,
```

```
            "InstanceType": "ml.g5.4xlarge",
            "InitialVariantWeight": 1
        }
    ]
)
```

In this case, a GPU-based instance (`ml.g5.4xlarge`) is used to provide the compute capacity required for serving high-performance, real-time inference requests.

5. Create a real-time inference endpoint using the endpoint configuration to deploy the model and make it available for live prediction requests.

```
endpoint_name = f"endpoint-{unique}"

sm.create_endpoint(
    EndpointName=endpoint_name,
    EndpointConfigName=endpoint_config_name
)
```

6. Define a `wait_for_endpoint()` function that continuously checks the endpoint status until it becomes available. If the endpoint creation fails, the function raises an error.

```
from time import sleep

def wait_for_endpoint(endpoint_name,
                      sagemaker_client=sm):
    while True:
        response = sagemaker_client.describe_endpoint(
            EndpointName=endpoint_name
        )
        status = response['EndpointStatus']
        print(f"ENDPOINT STATUS: {status}")

        if status == 'InService':
            break
        elif status == 'Failed':
            error = "ENDPOINT CREATION FAILED"
            raise RuntimeError(error)

        sleep(15)
```

7. Run the following block of code to wait for the endpoint to finish provisioning and reach the `InService` state before proceeding.

```
%%time
wait_for_endpoint(endpoint_name)
```

This should show the endpoint provisioning progress, with status messages similar to:

```
ENDPOINT STATUS: Creating
ENDPOINT STATUS: Creating
...
ENDPOINT STATUS: Creating
ENDPOINT STATUS: InService
CPU times: user 157 ms, sys: 35.4 ms, total: 192 ms
Wall time: 8min 33s
```

**Note**

This step may take around 8 to 15 minutes to finish. Feel free to pause here and enjoy a quick coffee or tea while the process completes.

8. Store the endpoint name so you can reuse it later for inference or testing.

```
%store endpoint_name
```

9. Retrieve and inspect the model configuration to verify details such as the container image, environment variables, and model artifact location.

```
from pprint import pprint

response = sm.describe_model(
    ModelName=model_name
)

pprint(response)
```

The response includes metadata about the model, including deployment recommendations, container configuration, and the S3 location of the model artifacts.

```
{
  'CreationTime': ...,
  'DeploymentRecommendation': ...
  'EnableNetworkIsolation': False,
  'ExecutionRoleArn': '...',
  'ModelArn': '...:model/model-gfifgm',
  'ModelName': 'model-gfifgm',
  'PrimaryContainer': {
    'Environment': {...},
    'Image': '.../djl-inference:...',
    'Mode': 'SingleModel',
    'ModelDataSource': {...},
    'ModelDataUrl': 's3://.../model/model.tar.gz'
  },
  'ResponseMetadata': {...}
}
```

10. Retrieve and inspect the endpoint configuration to confirm the instance type, scaling configuration, and associated model.

```
response = sm.describe_endpoint_config(
    EndpointConfigName=endpoint_config_name
)

pprint(response)
```

The output shows the production variant configuration, including the instance type (`ml.g5.4xlarge`) and traffic routing settings.

```
{
  'CreationTime': ...,
  'EnableNetworkIsolation': False,
  'EndpointConfigArn': '...',
  'EndpointConfigName': '...',
  'ProductionVariants': [
    {
      'InitialInstanceCount': 1,
      'InitialVariantWeight': 1.0,
```

```
        'InstanceType': 'ml.g5.4xlarge',
        'ModelName': 'model-fgmgfi',
        'VariantName': 'AllTraffic'
      }
    ],
    'ResponseMetadata': {...}
  }
```

11. Retrieve and inspect the endpoint details to verify that it is successfully deployed and in the `InService` state.

```
response = sm.describe_endpoint(
    EndpointName=endpoint_name
)

pprint(response)
```

The response provides information about the endpoint status, deployed instances, and the container image used for inference.

```
  {
    'CreationTime': ...,
    'EndpointArn': 'arn:...:endpoint/endpoint-fgmgfi',
    'EndpointConfigName': 'endpoint-config-fgmgfi',
    'EndpointName': 'endpoint-fgmgfi',
    'EndpointStatus': 'InService',
     ...
    'ProductionVariants': ...
    'ResponseMetadata': {...}
  }
```

12. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `training_003.ipynb`. We will run the code in the following steps within this notebook.
13. Load the previously stored endpoint name so you can use the deployed endpoint for inference.

```
%store -r endpoint_name
```

14. Initialize the SageMaker AI Runtime client (with `boto3`) and define the `invoke()` function that sends prompts to the deployed endpoint. This function formats the request payload and prints the model's generated response.

```
import boto3
import json

client = boto3.client("sagemaker-runtime")

def invoke(prompt, client=client):
    response = client.invoke_endpoint(
        EndpointName=endpoint_name,
        ContentType="application/json",
        Body=json.dumps({
            "inputs": prompt,
            "parameters": {"max_new_tokens": 500}
        })
    )

    print(response["Body"].read().decode())
```

15. Let's invoke the endpoint with a sample prompt to generate a response.

```
prompt = """
### Instruction:
Improve the given scientific sentence and provide detailed explanations to
the related questions.

### Input:
- Task 1: Improve the following sentence for clarity and scientific tone:
  "It is notable to mention that various studies have shown that water
serves a key role in the synthesis of vital organic compounds."

- Task 2: Explain the process through which water contributes to the
synthesis of essential organic compounds.

- Task 3: Discuss how the scarcity or abundance of water in an environment
impacts the diversity and functionality of life forms, particularly in
terms of metabolic processes and survival strategies.

### Response:
```

```
"""

output = invoke(prompt)
output
```

This should yield the following output:

```
{"generated_text": "1. Improved sentence:\n\"It is notable to mention that
various studies have demonstrated that water plays a crucial role in the
synthesis of vital organic compounds, which are essential for life..."}
```

16. Next, invoke the deployed model again with a prompt that instructs the model to generate a job description using the given role, industry, location, and requirements.

```
prompt = """
### Instruction:
Write a job description.

### Input:
Role: Part-time Finance Manager
Industry: Charity
Location: UK
Requirements: Previous charity experience is essential

### Response:
"""

output = invoke(prompt)
output
```

This should yield an output similar to the following:

```
{"generated_text": "The role of Part-time Finance Manager is a part-time
position within a UK-based charity organization. The successful candidate
will be responsible for managing the financial aspects of the charity,
ensuring the organization's financial stability and growth. Key
responsibilities include overseeing budgeting, financial planning, and
financial reporting..."}
```

17. Finally, test the endpoint with a prompt that helps evaluate the model's ability to generate a comprehensive blog post while addressing multiple subtopics.

```
prompt = """
### Instruction:
Write a comprehensive blog post and address the related topics in detail.

### Input:
- Topic: The importance of maintaining physical fitness during remote work
- Requirements:
  - Include tips and exercises that can be done at home without any special
equipment
- Additional Questions:
  1. Discuss the potential benefits and drawbacks of high-intensity
interval training (HIIT) for remote workers.
  2. Elaborate on the recovery process after a HIIT workout for remote
workers and how they can manage it without affecting productivity.

### Response:
"""

output = invoke(prompt)
output
```

You should get an output similar to the following:

```
{"generated_text": "Maintaining physical fitness during remote work is
crucial for both physical and mental health. It not only helps in reducing
the risk of chronic diseases but also improves overall well-being. Here are
some tips and exercises that can be done at home without any special
equipment..."}
```

18. Now, delete the endpoint using the following block of code.

```
import boto3
sm = boto3.client("sagemaker")

sm.delete_endpoint(
    EndpointName=endpoint_name
)
```

*Wasn't that easy?!* In this section, you deployed the fine-tuned Llama 3.2 Instruct model to a SageMaker AI real-time inference endpoint and then invoked the endpoint using different prompts to evaluate the model's ability to generate structured, context-aware responses across multiple tasks.

> **Note**
>
> Keep in mind that because only a small subset of the original fine-tuning dataset (`trl-lib/Capybara`) was used during training, the deployed model's performance may not fully reflect its potential. As a result, you may observe that the responses are less accurate, less consistent, or less context-aware compared to a model trained on the complete dataset.

As you continue using the SageMaker Python SDK and boto3, you will start to notice a consistent workflow pattern that involves defining resources, configuring them, deploying them, and then interacting with them programmatically. This pattern applies across training, tuning, and inference workflows, making it easier for you to build, automate, and scale your machine learning pipelines. Once you become familiar with this flow, you can confidently extend it to more advanced use cases and production-ready architectures.

# Performing Hyperparameter Tuning with Amazon SageMaker AI

In this section, you will learn how to perform hyperparameter tuning using SageMaker AI to automatically search for the best model configuration. You will define a hyperparameter search space, run multiple training jobs in parallel, and use SageMaker AI to evaluate each job against a specified objective metric to identify the best-performing model.

Before running a hyperparameter tuning job in SageMaker AI, it is important to understand what hyperparameters are and what hyperparameter tuning actually means. **Hyperparameters** are configuration values that are set before the training process begins and are not learned from the data. Examples include parameters such as `max_depth` in XGBoost, `learning_rate` in gradient boosting models, or batch size in deep learning models. These values directly affect how the model learns from the data and how well it generalizes to unseen data, but there is no direct way to know the best values in advance. You can think of hyperparameters like the settings on a cooking recipe, such as oven temperature or cooking time. Small changes in these settings can significantly change the final result, even when you use the same ingredients. Continuing with this analogy, hyperparameter tuning is like trying out different versions of the same recipe with slightly different settings each time. Instead of

guessing everything manually, you systematically test multiple combinations, taste each result, and keep track of which settings produce the best outcome.

To better understand hyperparameter tuning, it is helpful to step back and review the machine learning process, particularly how datasets are structured and used during model development. In machine learning, datasets are typically split into training and test sets, or into training, validation, and test sets (*Figure 7.10*).

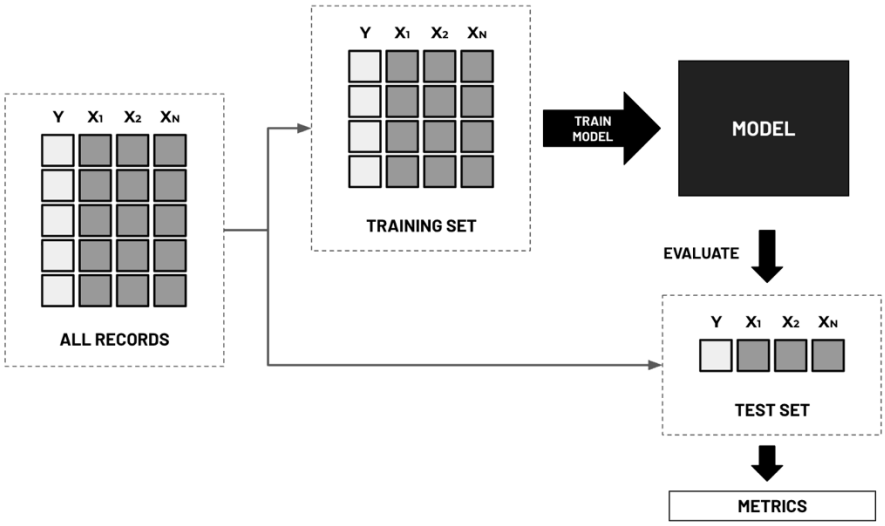



*Figure 7.10 — How datasets are structured and used during model development*

The training set is used to fit the model by allowing it to learn patterns from the data, while the test set is used to evaluate how well the trained model generalizes to unseen data. When a validation set is included, it is used during development to tune hyperparameters and support model selection before performing the final evaluation on the test set.

While you might initially think that training a single model is sufficient to complete a task, in practice you will run multiple experiments to improve performance. You will try different hyperparameter configurations and launch multiple training jobs to observe how each setting affects model performance. Each run produces a different model, and you compare the results to determine which configuration performs best. This iterative experimentation process is a standard part of building production-grade machine learning systems. This is where hyperparameter tuning becomes important. Instead of manually testing each configuration one by one, you define a search space and let SageMaker AI automatically explore different combinations of hyperparameters (*Figure 7.11*).

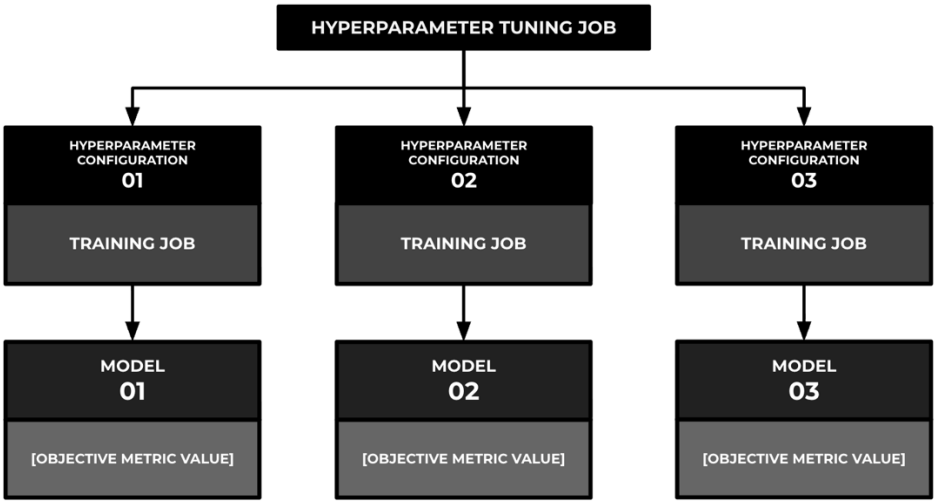



*Figure 7.11 — Hyperparameter tuning job*

With a **hyperparameter tuning job**, SageMaker AI launches multiple training jobs, evaluates each run using a specified objective metric, and selects the best-performing configuration. If you were to do this manually, you would repeatedly train the model with different hyperparameter combinations, evaluate each model on a validation dataset, and compare results to identify the best setup. As the number of hyperparameters increases, the number of possible combinations grows quickly, making manual experimentation inefficient and difficult to manage. SageMaker AI automates this process by sampling hyperparameter combinations from a defined search space and running each configuration as a separate training job. It then evaluates the results using metrics emitted during training, such as validation loss or F1 score, and selects the configuration that optimizes the chosen objective. Once the tuning job is complete, you can deploy the best-performing model directly to a SageMaker AI endpoint for inference.

**Note**

In this section, you will use the same dataset introduced in *Chapter 3*. The dataset contains two input features, `a` and `b`, and a binary target variable, `target`, which takes values of `0` (**LABEL 0**) or `1` (**LABEL 1**). The model learns the relationship between the input features and the target during training by identifying patterns in the (a, b) feature space that separate the two classes. This same dataset will be used as the basis for a hyperparameter tuning job that trains multiple XGBoost models with different configurations. Each model learns the mapping from a and b to target slightly

differently depending on its hyperparameter settings, such as tree depth. This allows you to compare how different configurations affect the model's ability to separate the two classes.

Now that we have a good understanding of how hyperparameter tuning works and why it is needed, let's go through the steps to perform hyperparameter tuning with SageMaker AI:

1. In the left sidebar of JupyterLab, right-click and select **New Folder**. Name the folder `hpo`, then double-click it to navigate into the directory. Alternatively, you can create the folder by clicking the **New Folder** button in the File Browser toolbar, then rename it to `hpo` and open it to navigate into the directory.

   **Note**

   All notebooks and associated files for this section, up to the end of the section, should be created and stored in this directory.

2. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `training_004.ipynb`. We will run the code in the following steps within this notebook.
3. Uninstall any existing versions of `sagemaker` and `sagemaker-serve`, then install the required versions (`sagemaker==3.5.0` and `sagemaker-serve==1.2.0`). In addition to these, install the `conda-pack` package as well.

   ```
   %pip uninstall -y sagemaker sagemaker-serve
   %pip install "sagemaker==3.5.0"
   %pip install "sagemaker-serve==1.2.0"
   %pip install conda-pack
   ```

4. Now that we've installed the required packages and package versions, restart the kernel by selecting **Kernel | Restart Kernel** from the menu. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

   **Note**

   After installing the required package versions, confirm that the correct versions are in place by running `%pip show sagemaker` and `%pip show`

> `sagemaker-serve`. Ensure that the installed versions are `sagemaker 3.5.0` and `sagemaker-serve 1.2.0`, respectively, before proceeding.

5. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length.

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase,
                       k=length))
```

6. Let's use the `generate_string()` function we defined in the previous step to generate a random string:

```
unique = generate_string()
print(unique)
```

   Here, we store the generated string in the `unique` variable for later use. We'll use this to give our resources a unique name so they don't conflict with existing ones in your account.

7. Define constants for your dataset filenames and the local directory where the data is stored.

```
TRAIN_DATA = "train.csv"
VALIDATION_DATA = "validation.csv"
TEST_DATA = "test.csv"
DATA_DIRECTORY = "data"
```

8. Next, define the `download_file()` function for downloading a file from the book's official GitHub repository and saving it to a local directory inside your SageMaker Studio space.

```
import urllib.request
from pathlib import Path
from typing import Union
```

```
BASE_URL_PARTS = [
    "https://raw.githubusercontent.com/",
    "PacktPublishing/",
    "Machine-Learning-Engineering-on-",
    "AWS-Second-Edition/",
    "refs/heads/main/",
    "chapter07/",
]

def download_file(
        filename: str,
        directory: Union[str, Path] = DATA_DIRECTORY
    ) -> Path:
    url = "".join(BASE_URL_PARTS) + filename

    directory_path = Path(directory)
    directory_path.mkdir(parents=True, exist_ok=True)

    output_file = directory_path / filename
    urllib.request.urlretrieve(url, output_file)

    return output_file
```

This function builds the full file URL by combining the base GitHub path with the provided filename. It then creates the target local directory if it does not already exist, downloads the file from the repository, and saves it to that directory. Finally, it returns the local path of the downloaded file so it can be used in later steps.

9. Using the `download_file()` function we defined in the previous step, download the training, validation, and test datasets to your local directory.

```
download_file(TRAIN_DATA)
download_file(VALIDATION_DATA)
download_file(TEST_DATA)
```

**Note**

You can use the File Browser to verify that the CSV files have been successfully downloaded and are present inside the *data* directory.

10. Import and prepare the prerequisites for the hyperparameter tuning job, including a `Session` object, the execution role ARN, as well as the S3 bucket that will be used to store data and model artifacts.

```
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)

session = Session()
role = get_execution_role()
bucket = session.default_bucket()
```

11. Upload the contents of the *data* directory to the specified Amazon S3 bucket under a unique prefix.

```
train_input = session.upload_data(
    DATA_DIRECTORY,
    bucket=bucket,
    key_prefix="{}/{}".format(unique, DATA_DIRECTORY)
)
```

12. Next, verify that the data was uploaded successfully by listing the contents of the S3 bucket.

```
!aws s3 ls {train_input} --recursive
```

13. Construct the full Amazon S3 URIs for the training, validation, and test datasets, as well as the output location for storing training artifacts.

```
s3_train_input_path = "s3://{}/{}/data/{}".format(
    bucket,
    unique,
    TRAIN_DATA
)

s3_val_input_path = "s3://{}/{}/data/{}".format(
    bucket,
    unique,
    VALIDATION_DATA
```

```
)

s3_output_path = "s3://{}/{}/output".format(
    bucket,
    unique
)

s3_test_path = "s3://{}/{}/data/{}".format(
    bucket,
    unique,
    TEST_DATA
)
```

If you are wondering why these S3 paths need to be defined, they are used by SageMaker AI during the training process. When the training job runs, the input datasets are automatically downloaded from the specified S3 input paths.

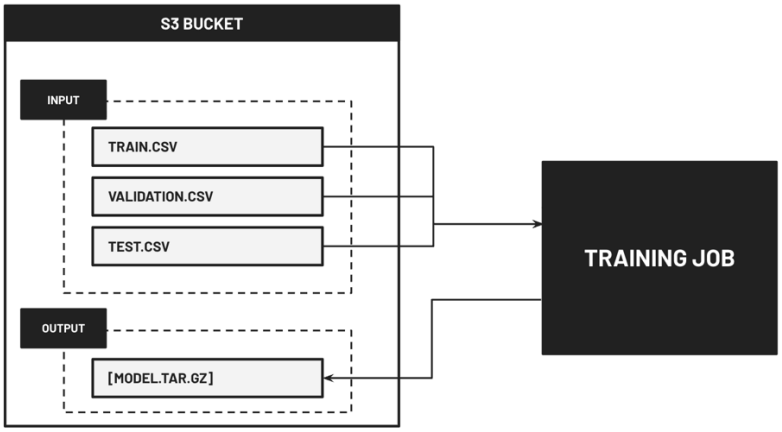



*Figure 7.12 — How SageMaker AI reads from and writes to S3 buckets during training*

Once training is completed, the resulting model artifacts and outputs are uploaded and stored in the specified S3 output path.

> **Note**
>
> Since we are simulating a hyperparameter tuning job in this section, multiple *model.tar.gz* files will be generated, one for each training job executed during the tuning process. Each file contains the trained model artifacts corresponding to a specific hyperparameter configuration and is stored in the specified S3 output path.

14. Define the `copy_model_server_file()` function which copies the `model.py` file from the *triton* folder in the SageMaker `serve` directory to the root of the `serve` directory:

```
import os
import shutil
import sagemaker

def copy_model_server_file():
    sagemaker_path = os.path.dirname(
        sagemaker.__file__)

    source_path = os.path.join(
        sagemaker_path, 'serve',
        'model_server', 'triton', 'model.py')

    destination_path = os.path.join(
        sagemaker_path, 'serve', 'model.py')

    if os.path.exists(source_path):
        shutil.copy(source_path, destination_path)
        print(f"Successfully copied "
              f"model.py from {source_path} "
              f"to {destination_path}")
    else:
        print(f"Source file not found "
              f"at {source_path}")
```

15. Use the `copy_model_server_file()` function defined in the previous step to copy the `model.py` file to the `serve` directory.

```
copy_model_server_file()
```

> **Note**
>
> This should address a possible issue that may have existed in earlier versions of the SageMaker Python SDK when building and deploying machine learning models.

16. Import the necessary classes for defining input channels, data sources, output configuration, and stopping conditions for the training job.

```
from sagemaker.core.shapes import (
    Channel,
    DataSource,
    S3DataSource,
    OutputDataConfig,
    StoppingCondition,
)

from sagemaker.core import image_uris
```

We'll use these as we go through configuring the hyperparameter tuning job inputs, outputs and runtime behavior.

17. Retrieve the current AWS region, which will be used to select the appropriate container image for the training job.

```
import boto3

region = boto3.Session().region_name
print(region)
```

If your SageMaker Studio space, where the Jupyter notebook is running, is located in the N. Virginia region, the returned value should be `us-east-1`.

18. Retrieve the XGBoost training container image URI for the specified region and version.

```
image = image_uris.retrieve(
    framework="xgboost",
    region=region,
    image_scope="training",
    version="1.7-1"
```

```
)

print(image)
```

This should return the container image URI for the XGBoost training image in the specified region and version, similar to:

```
....amazonaws.com/sagemaker-xgboost:1.7-1
```

Later, when you run the hyperparameter tuning job, SageMaker AI uses this container image to launch training containers across multiple managed ML instances, ensuring that the correct XGBoost version and runtime dependencies are available during model training.

SageMaker AI provides a wide range of prebuilt container images for machine learning libraries, algorithms, and frameworks, including XGBoost, TensorFlow, and PyTorch, which are optimized and maintained by AWS for training and inference workloads. In addition to using these prebuilt images, you can also bring your own custom container image if you need a specific runtime environment, dependencies, or configuration that is not available in the managed options. You can also provide a custom training script that defines the training logic executed within the container during the training job. In this chapter, we primarily use a prebuilt container image to simplify the training setup and focus on the core machine learning workflow.

19. Define the base set of hyperparameters for the XGBoost training job.

```
hyperparameters = {
    "objective": "binary:logistic",
    "num_round": "50"
}
```

20. Define the hyperparameter search space for tuning. In this case, the `max_depth` parameter will be explored within a specified range.

```
from sagemaker.core.parameter import IntegerParameter

hyperparameter_ranges = {
    "max_depth": IntegerParameter(4, 10),
}
```

The hyperparameter search space defines the range of values that SageMaker AI will explore during hyperparameter tuning. In this case, the `max_depth` parameter is allowed to vary between `4` and `10`. During the tuning process, SageMaker AI launches multiple training jobs, each with a different value selected from this range. For example, one training job might be configured with `max_depth = 4`, while another might use `max_depth = 7`, and another might use `max_depth = 10`. Each training job trains a separate model using its assigned configuration, and the results are evaluated against a defined objective metric. Based on these results, SageMaker AI identifies the hyperparameter value that produces the best-performing model.

21. Define a base name for the training jobs that will be launched during hyperparameter tuning.

```
base_job_name = "xgb-hpo-" + unique
```

22. Initialize and configure the `ModelTrainer` object by specifying the training image, compute configuration, hyperparameters, and output settings.

```
from sagemaker.train import ModelTrainer
from sagemaker.train.configs import Compute

compute = Compute(
    enable_managed_spot_training=True
)

model_trainer = ModelTrainer(
    base_job_name=base_job_name,
    hyperparameters=hyperparameters,
    training_image=image,
    training_input_mode="File",
    role=role,
    compute=compute,
    output_data_config=OutputDataConfig(
        s3_output_path=s3_output_path
    ),
    stopping_condition=StoppingCondition(
        max_runtime_in_seconds=1800,
        max_wait_time_in_seconds=1800
    ),
)
```

Here, you are enabling **managed spot training**, which allows SageMaker AI to use spare EC2 capacity at a significantly reduced cost compared to on-demand instances.

> **Note**
>
> Keep in mind that spot instances can be interrupted when AWS needs the capacity back. SageMaker AI supports checkpointing and resuming training, but this depends on the training script properly saving checkpoints to durable storage such as Amazon S3. This makes spot instances a cost-efficient option for longer training jobs where occasional interruptions are acceptable.

23. Define the metrics that will be extracted from the training logs and used during hyperparameter tuning and selection of the best-performing model based on the objective metric, and initialize a `HyperparameterTuner` object to manage the tuning process by launching multiple training jobs with different hyperparameter configurations and evaluating them against the specified metric definitions.

```
from sagemaker.train.tuner import HyperparameterTuner

metric_definitions = [
    {
        "Name": "validation-logloss",
        "Regex": "validation-logloss:([0-9\\.]+)"
    },
    {
        "Name": "train-logloss",
        "Regex": "train-logloss:([0-9\\.]+)"
    }
]

tuner = HyperparameterTuner(
    model_trainer=model_trainer,
    objective_metric_name="validation:f1",
    hyperparameter_ranges=hyperparameter_ranges,
    metric_definitions=metric_definitions,
    max_jobs=6,
    max_parallel_jobs=3,
)
```

Here, `max_jobs=6` specifies the maximum number of training jobs to run during the tuning process, and `max_parallel_jobs=3` limits the number of jobs executed in parallel.

24. Define the training and validation data sources using the S3 paths you constructed earlier.

```
train_data_source=DataSource(
    s3_data_source=S3DataSource(
        s3_data_type="S3Prefix",
        s3_uri=s3_train_input_path,
        s3_data_distribution_type="FullyReplicated",
    )
)

val_data_source=DataSource(
    s3_data_source=S3DataSource(
        s3_data_type="S3Prefix",
        s3_uri=s3_val_input_path,
        s3_data_distribution_type="FullyReplicated",
    )
)
```

25. With everything ready, run the following block of code to start the hyperparameter tuning job. This will launch multiple training jobs with different parameter combinations and wait for completion.

```
%%time

tuner.tune(
    inputs=[
        Channel(
            channel_name="train",
            content_type="csv",
            compression_type="None",
            record_wrapper_type="None",
            data_source=train_data_source,
        ),
        Channel(
            channel_name="validation",
            content_type="csv",
```

```
            compression_type="None",
            record_wrapper_type="None",
            data_source=val_data_source,
        )
    ],
    wait=True
)
```

This step may take approximately 5 to 15 minutes to complete. Feel free to take a short break while it runs in the background.

**Note**

While waiting, you can switch to the browser tab where the AWS Management Console is open and locate the hyperparameter tuning job to monitor its progress. To do this, navigate to the SageMaker AI console, then in the navigation pane select **Training & tuning jobs** under **Model training & customization**. This will take you to a page where you can view both training jobs and hyperparameter tuning jobs in your AWS account, along with their status and configuration details.

26. Retrieve metadata about the tuning job, including its configuration and status.

```
tuner.describe()
```

This should yield an output similar to the following:

```
HyperParameterTuningJob(
    hyper_parameter_tuning_job_name='...',
    hyper_parameter_tuning_job_arn='...',
    hyper_parameter_tuning_job_config=...,
    training_job_definition=...,
    hyper_parameter_tuning_job_status='Completed',
    creation_time='...',
    hyper_parameter_tuning_end_time='...',
    training_job_status_counters=...,
    objective_status_counters=...,
    best_training_job=...,
    tuning_job_completion_details=...,
```

```
        consumed_resources=...
    )
```

27. Retrieve the hyperparameter tuning job analytics and convert the results into a Pandas DataFrame for analysis.

```
analytics = tuner.analytics()
df = analytics.dataframe()
```

28. Display the `DataFrame` containing the results of all training jobs.

```
df
```

This should display a DataFrame of values, similar to:

| | max_depth | TrainingJobName | TrainingJobStatus | FinalObjectiveValue | TrainingStartTime | TrainingEndTime | TrainingElapsedTimeSeconds |
|---|---|---|---|---|---|---|---|
| 0 | 4.0 | sagemaker-xgboost-260101-0210-006-a497b1d2 | Completed | 0.82660 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| 1 | 7.0 | sagemaker-xgboost-260101-0210-005-1f835764 | Completed | 0.81586 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| 2 | 6.0 | sagemaker-xgboost-260101-0210-004-32c6c004 | Completed | 0.81206 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| 3 | 9.0 | sagemaker-xgboost-260101-0210-003-b008ff71 | Completed | 0.82175 | 2026-01-01 00:00:01 | 2026-01-01 00:01:51 | 110.0 |
| 4 | 8.0 | sagemaker-xgboost-260101-0210-002-e342abda | Completed | 0.81045 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| 5 | 5.0 | sagemaker-xgboost-260101-0210-001-b2604ddd | Completed | 0.82752 | 2026-01-01 00:00:01 | 2026-01-01 00:01:40 | 99.0 |

*Figure 7.13 — Hyperparameter tuning job results DataFrame*

Here, you have a `DataFrame` that summarizes the results of all training jobs executed during the hyperparameter tuning process. Each row represents a single training job, with columns such as `max_depth`, `TrainingJobName`, and `TrainingJobStatus` describing the hyperparameter configuration and job status. Additional columns, including `FinalObjectiveValue`, `TrainingStartTime`, `TrainingEndTime`, and `TrainingElapsedTimeSeconds`, capture the model's performance on the objective metric as well as the timing details, which allow you to compare outcomes across different hyperparameter configurations.

> **Note**
>
> You may observe a different set of `max_depth` values and `FinalObjectiveValue` results in your output, depending on how the hyperparameter tuning job sampled configurations within the defined search space and the specific training runs executed.

29. Sort the results to identify the best-performing training jobs based on the objective metric.

```
df.sort_values(
    by="FinalObjectiveValue",
    ascending=False
)
```

This should return a `DataFrame` sorted in descending order by `FinalObjectiveValue`, with the highest-performing training jobs listed first:

| | max_depth | TrainingJobName | TrainingJobStatus | FinalObjectiveValue | TrainingStartTime | TrainingEndTime | TrainingElapsedTimeSeconds |
|---|---|---|---|---|---|---|---|
| **5** | 5.0 | sagemaker-xgboost-260101-0210-001-b2604ddd | Completed | 0.82752 | 2026-01-01 00:00:01 | 2026-01-01 00:01:40 | 99.0 |
| **0** | 4.0 | sagemaker-xgboost-260101-0210-006-a497b1d2 | Completed | 0.82660 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| **3** | 9.0 | sagemaker-xgboost-260101-0210-003-b008ff71 | Completed | 0.82175 | 2026-01-01 00:00:01 | 2026-01-01 00:01:51 | 110.0 |
| **1** | 7.0 | sagemaker-xgboost-260101-0210-005-1f835764 | Completed | 0.81586 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| **2** | 6.0 | sagemaker-xgboost-260101-0210-004-32c6c004 | Completed | 0.81206 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |
| **4** | 8.0 | sagemaker-xgboost-260101-0210-002-e342abda | Completed | 0.81045 | 2026-01-01 00:00:01 | 2026-01-01 00:01:41 | 100.0 |

*Figure 7.14 — DataFrame sorted by FinalObjectiveValue in descending order*

As an ML engineer, you should examine the hyperparameter configuration associated with each training job as it defines the exact configuration used to train that specific model instance. This enables you to analyze how differences in hyperparameter choices made during tuning are associated with variations in model performance across training jobs.

30. Retrieve the name of the best-performing SageMaker AI training job identified during the hyperparameter tuning process.

```
best_training_job = tuner.best_training_job()
best_training_job
```

Here, SageMaker AI selects and returns the name of the training job with the best performance based on the objective metric defined in the tuner. The value returned by `tuner.best_training_job()` should match the `TrainingJobName` of the training job that achieved the best value of the objective metric from the previous step's results.

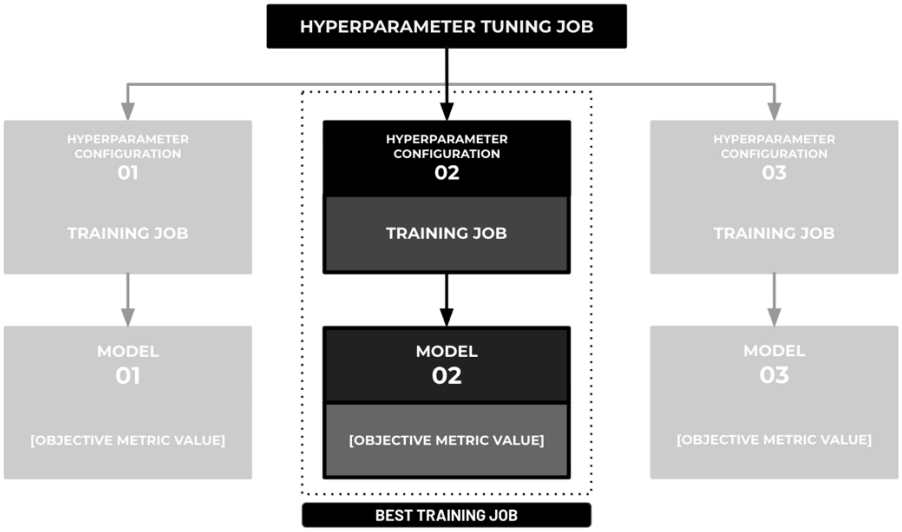



*Figure 7.15 — Identifying the best training job*

Since the objective metric is set to `validation:f1` and the tuning job is configured with a `Maximize` objective, SageMaker AI selects the training job with the highest observed `validation:f1` value. In the analytics output, this corresponds to the training job with the highest `FinalObjectiveValue` for the objective metric.

31. Store the name of the best training job, the unique identifier, and the execution role for reuse in the next section.

```
%store best_training_job
%store unique
%store role
```

At this stage, you have completed the configuration and execution of a hyperparameter tuning job in SageMaker AI. Instead of manually testing different hyperparameter combinations, you defined a search space and allowed SageMaker AI to automatically explore multiple configurations in parallel. This approach reduces the time required for experimentation while providing a systematic way to identify the best-performing model based on a defined objective metric. In the next section, you will deploy the best-performing model to a managed ML instance for real-time inference.

# Deploying the Best-Performing Model from Hyperparameter Tuning

In this section, you will take the best-performing model selected by the hyperparameter tuning job and deploy it to a real-time inference endpoint in SageMaker AI. You will use the artifacts generated during the tuning process, inspect the training job metadata, and prepare the components required for deployment. You will then create a SageMaker AI model, define an endpoint configuration, and launch a managed real-time inference endpoint that serves predictions.

Let's proceed with deploying the best-performing model:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `training_005.ipynb`. We will run the code in the following steps within this notebook.

   **Note**

   Make sure to create this notebook inside the *hpo* directory so that all files and outputs for this section are organized in the same location.

2. Load the previously stored variables (`best_training_job`, `unique`, and `role`) using the following block of code.

```
%store -r best_training_job
%store -r unique
%store -r role
```

3. Use the training job name stored in `best_training_job` to load a `TrainingJob` object:

```
from sagemaker.core.resources import TrainingJob

job = TrainingJob.get(best_training_job)
job
```

Keep in mind that the training job has already completed at this point, and `TrainingJob.get()` simply returns an object that maps to the existing SageMaker AI training job.

> **Note**
>
> You may use `help(TrainingJob)` to view the class documentation, including available methods and attributes for interacting with SageMaker AI training jobs.

4. Inspect the attributes of the `TrainingJob` object to view its configuration and metadata.

```
job.__dict__
```

This should yield an output with a structure similar to the following:

```
{'training_job_name': 'sagemaker-xgboost-...',
 'training_job_arn': 'arn:aws:sagemaker:...',
  ...
 'tuning_job_arn': 'arn:aws:sagemaker:...',
  ...
 'model_artifacts': ModelArtifacts(...),
  ...
 'hyper_parameters': {...},
 'algorithm_specification': ...,
 'role_arn': 'arn:aws:iam::.../service-role/...',
 'input_data_config': [...],
 'output_data_config': OutputDataConfig(...),
 'resource_config': ResourceConfig(...),
  ...}
```

Here, the output shows the underlying attributes and configuration stored within the `TrainingJob` object. These include identifiers, configuration settings, and execution details associated with the SageMaker AI training job.

5. Run the following block of code to retrieve the container image used during training.

```
algo_spec = job.__dict__['algorithm_specification']
container_image = algo_spec.training_image
print(container_image)
```

This should output the container image URI similar to the following:

```
...us-east-1.amazonaws.com/sagemaker-xgboost:1.7-1
```

This output corresponds to the XGBoost container image (version 1.7-1) used in the training job. The same image will also be used when deploying the model in later steps.

6. Next, let's retrieve the S3 path where the model artifacts generated by the training job are stored.

```
model_artifacts = job.__dict__['model_artifacts']
s3_model_path = model_artifacts.s3_model_artifacts
print(s3_model_path)
```

You should get an S3 URI that points to the bucket where the `model.tar.gz` file is stored, similar to `s3://.../output/model.tar.gz`.

**Note**

The *model.tar.gz* file contains the output of the SageMaker AI training job, which includes the trained model and associated artifacts. There are different ways to deploy this model depending on the use case and infrastructure requirements, and it can be deployed either within SageMaker AI or outside SageMaker AI using your own deployment setup. Within SageMaker AI, a common approach is to deploy it within a managed real-time inference endpoint, where AWS handles provisioning, scaling, and hosting of the inference infrastructure. In the next set of steps, we will use this approach by first creating the model, then defining an endpoint configuration, and finally deploying the endpoint.

7. Use the `create_model()` method to create a SageMaker AI model by specifying the container image, model artifacts, and environment variables used to configure the inference container at runtime.

```
import boto3
sm = boto3.client("sagemaker")
model_name = f"model-{unique}"

sm.create_model(
    ModelName=model_name,
    PrimaryContainer={
        "Image": container_image,
        "ModelDataUrl": s3_model_path,
        "Environment": {
            "SHM_SIZE": "1g",
             "HF_MODEL_ID": "/opt/ml/model",
            "MAX_INPUT_LENGTH": "2048",
            "MAX_TOTAL_TOKENS": "4096",
            "NUM_SHARD": "1"
        }
    },
    ExecutionRoleArn=role
)
```

8. Use the `create_endpoint_config()` method to define the deployment configuration for the model, including the production variant, instance type, initial instance count, and traffic weight distribution.

```
endpoint_config_name = f"endpoint-config-{unique}"

sm.create_endpoint_config(
    EndpointConfigName=endpoint_config_name,
    ProductionVariants=[
        {
            "VariantName": "AllTraffic",
            "ModelName": model_name,
            "InitialInstanceCount": 1,
            "InstanceType": "ml.m5.xlarge",
            "InitialVariantWeight": 1
        }
```

```
    ]
)
```

Here, the model is configured to run on an `ml.m5.xlarge` instance, which defines the compute capacity that will be used when the endpoint is deployed.

**Note**

Creating an endpoint configuration does not provision any managed infrastructure. It only defines how the model should be deployed, including the instance type and scaling configuration. The actual provisioning of resources and deployment of the model occurs when you call `create_endpoint()` in the next step. At this point, you might be wondering what the advantage of having an endpoint configuration is. With an endpoint configuration, you can create or re-create an endpoint later using the same deployment settings after deleting an existing endpoint, without redefining the configuration. It enables reproducible deployments by allowing the same endpoint configuration to be reused when creating endpoints. It can also be used when updating an existing endpoint, where a new configuration is applied to change the deployment settings.

9. Use the `create_endpoint()` method to deploy a real-time inference endpoint using the specified endpoint configuration, making the trained model available for inference requests.

```
endpoint_name = f"endpoint-{unique}"

sm.create_endpoint(
    EndpointName=endpoint_name,
    EndpointConfigName=endpoint_config_name
)
```

During this process, SageMaker AI provisions a managed compute instance based on the specified instance type and deploys the model to a real-time inference endpoint. It retrieves the container image from the container registry and starts the inference container using the container image on the provisioned instance. It then downloads the *model.tar.gz file* (which contains the trained model artifacts) from the specified Amazon S3 location and loads it into the container. Once deployment is complete, the endpoint is ready to accept inference requests and return predictions.

> **Note**
>
> Do not forget to delete the SageMaker AI real-time inference endpoint after completing the remaining steps in this chapter. This helps prevent unnecessary charges, as the deployed managed compute instance continues to run and incur costs while the endpoint is active.

10. Define the `wait_for_endpoint()` function that polls the endpoint status until it's in the `InService` state. This lets you wait for the deployment of the SageMaker AI real-time inference endpoint to complete before sending requests.

```
from time import sleep

def wait_for_endpoint(endpoint_name,
                      sagemaker_client=sm):
    while True:
        response = sagemaker_client.describe_endpoint(
            EndpointName=endpoint_name
        )
        status = response['EndpointStatus']
        print(f"ENDPOINT STATUS: {status}")

        if status == 'InService':
            break
        elif status == 'Failed':
            error = "ENDPOINT CREATION FAILED"
            raise RuntimeError(error)

        sleep(15)
```

11. Use the `wait_for_endpoint()` function defined in the previous step to wait for the endpoint to become available.

```
%%time
wait_for_endpoint(endpoint_name)
```

> **Note**
>
> This step may take 2 to 10 minutes to complete. Feel free to grab a cup of coffee or tea while you wait.

12. Use the endpoint name to load an `Endpoint` object and configure its serializer and deserializer for request and response handling.

```
from sagemaker.core.resources import Endpoint
predictor = Endpoint.get(endpoint_name)

from sagemaker.core.deserializers import (
    JSONDeserializer
)

from sagemaker.core.serializers import (
    CSVSerializer,
)

predictor.serializer = CSVSerializer()
predictor.deserializer = JSONDeserializer()
```

By configuring the serializer and deserializer of the endpoint, your input data is transformed into the format expected by the model, and the model's response is transformed into a format that your code can easily use.

> **Note**
>
> When you send data to the endpoint, the input is first formatted using `CSVSerializer` so the model can process it correctly. After the model returns a response in JSON format, `JSONDeserializer` is used to parse the output into a Python data structure such as a dictionary or list.

13. Use the `predictor.invoke()` method to send a sample input (`[1.0, -1.0]`) to the model and retrieve the prediction result.

```
import json

result = predictor.invoke(
```

```
    body=[1.0, -1.0],
    content_type="text/csv"
)

result.body
```

This should yield an output value of:

```
0.9383734464645386
```

This predicted value represents the model's estimated probability that the input belongs to the positive class (**LABEL 1**) in the binary classification task. In other words, the deployed XGBoost model predicts that there is approximately a 93.84% likelihood that the input `[1.0, -1.0]` corresponds to class 1 (**LABEL 1**). Since this value is greater than the standard 0.5 decision threshold, the model would classify the input as **LABEL 1**. Keep in mind that you may observe a slightly different value depending on the exact model trained during hyperparameter tuning and its resulting parameters.

> **Note**
>
> Since the deployed model is one of the candidates produced by the hyperparameter tuning job, its predictions reflect the specific hyperparameter configuration associated with that selected model. If a different candidate model from the same hyperparameter tuning job is deployed instead, it may produce different inference results for the same input due to differences in the learned parameters. For example, the current model may return **LABEL 1** for a given input, while another candidate model from the same tuning job may return **LABEL 0** for the same input.

14. Use the `predictor.invoke()` method to send a sample input (`[-1.0, 1.5]`) to the model and retrieve the prediction result.

```
import json

result = predictor.invoke(
    body=[-1.0, 1.5],
    content_type="text/csv"
)
```

```
result.body
```

This should give you the following output value:

```
0.013632148504257202
```

This time, the model outputs a predicted probability of approximately `0.0136`, which represents a 1.36% likelihood that the input `[-1.0, 1.5]` belongs to the positive class (**LABEL 1**). Since this probability value is well below the standard 0.5 decision threshold, the model classifies this input as **LABEL 0**.

15. Since the model returns a probability and we expect a binary output (`0` or `1`), define an `invoke_endpoint()` function that takes two input features (`a` and `b`), sends them to the deployed endpoint, and returns the corresponding class label (**LABEL 1** or **LABEL 0**).

```
import json

def invoke_endpoint(a, b, predictor=predictor):
    payload = f"{a},{b}"

    result = predictor.invoke(
        body=payload,
        content_type="text/csv"
    )

    probability = float(result.body)

    prediction = int(probability > 0.5)

    return prediction
```

Here, you have a function that sends the input values for features `a` and `b` to the inference endpoint as a CSV-formatted request body. The endpoint returns a model prediction, which is interpreted, and converted into an integer class label.

16. Let's test the inference endpoint using the `invoke_endpoint()` function we defined in the previous step. Use the input values `-1.0` and `1.5` and call the function to retrieve a sample prediction result.

```
invoke_endpoint(-1.0, 1.5)
```

You should get a predicted value of `0` (**LABEL 0**).

> **Note**
>
> Behind the scenes, the `invoke_endpoint()` function formats the input values as a CSV string and sends the request to the inference endpoint. The endpoint returns a prediction score, which is treated as a probability and then compared against a decision threshold of `0.5`. Based on this comparison, the function returns `0` (**LABEL 0**) or `1` (**LABEL 1**) as the final binary class label. In this specific set of input values, the prediction score (`0.0136...`) falls below the decision threshold, resulting in a predicted class label of 0 (**LABEL 0**).

17. Let's test the inference endpoint again, this time using the input values `1.0` and `-1.0`:

```
invoke_endpoint(1.0, -1.0)
```

Since the prediction score (`0.93837...`) is above the decision threshold of `0.5`, the model returns a predicted class label of `1` (**LABEL 1**).

18. Now that we have finished testing the deployed model, delete the SageMaker AI real-time inference endpoint using the following:

```
predictor.delete()
```

This removes the deployed endpoint and terminates the managed ML instance hosting the model, which helps ensure that you are no longer billed for the associated compute resources.

Congratulations! You have successfully deployed the best-performing model selected by the hyperparameter tuning job and verified that it generates predictions using sample inputs. You used training job metadata, model artifacts, and inference container configuration to create and deploy a managed real-time endpoint in SageMaker AI. At this stage, you should have a clear understanding of how to move from automated experimentation to a deployed, real-time inference endpoint.

**Note**

In this chapter, you have only scratched the surface of hyperparameter tuning in SageMaker AI. In practice, hyperparameter tuning can be extended beyond a single model type or algorithm. You can apply it across different model families, such as tree-based models like XGBoost as well as deep learning models built with frameworks like TensorFlow or PyTorch, allowing you to compare performance across fundamentally different approaches. This makes hyperparameter tuning a powerful and flexible tool for systematically improving model performance at scale.

# Summary

In this chapter, you explored how to fine-tune a LLM using Amazon SageMaker AI as part of an end-to-end machine learning workflow. You set up a serverless MLflow App to track and manage training experiments, executed a supervised fine-tuning job, and observed how SageMaker AI abstracts the underlying infrastructure while automatically orchestrating the required compute resources. You then performed hyperparameter tuning to systematically evaluate multiple configurations in parallel and identify the best-performing model based on a defined objective metric. Finally, you deployed the best-performing model to a real-time inference endpoint for serving predictions.

In the next chapter, we will explore different options and strategies for deploying models in SageMaker AI. You will work with a pre-trained model and deploy it using a real-time endpoint, a serverless endpoint, an asynchronous inference endpoint, and a batch transform job. You will also practice advanced techniques such as shadow testing to validate a new model without impacting live traffic, and canary traffic shifting to gradually route a portion of production requests to a new model. Along the way, you will learn how to configure data capture to record inference requests and responses for monitoring and evaluation.

# Further reading

To expand your practical knowledge of the concepts presented in this chapter, refer to the following resources:

- *Amazon SageMaker AI | Developer Guide | Accelerate generative AI development using managed MLflow on Amazon SageMaker AI* (`https://docs.aws.amazon.com/sagemaker/latest/dg/mlflow.html`)
- *Amazon SageMaker AI | Developer Guide | Model training* (`https://docs.aws.amazon.com/sagemaker/latest/dg/train-model.html`)

- *Amazon SageMaker AI | Developer Guide | Built-in algorithms and pretrained models in Amazon SageMaker* (`https://docs.aws.amazon.com/sagemaker/latest/dg/algos.html`)
- *AWS Blogs | Advanced fine-tuning methods on Amazon SageMaker AI* (`https://aws.amazon.com/blogs/machine-learning/advanced-fine-tuning-methods-on-amazon-sagemaker-ai/`)
- *SageMaker Python SDK V3 | Customizing Models* (`https://sagemaker.readthedocs.io/en/stable/model_customization/index.html`)
- *Amazon SageMaker AI | Developer Guide | Automatic model tuning with SageMaker AI* (`https://docs.aws.amazon.com/sagemaker/latest/dg/automatic-model-tuning.html`)
- *Amazon SageMaker AI | Developer Guide | Understand the hyperparameter tuning strategies available in Amazon SageMaker AI* (`https://docs.aws.amazon.com/sagemaker/latest/dg/automatic-model-tuning-how-it-works.html`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 8

# SageMaker AI Model Deployment Options and Strategies

Imagine you've spent days or weeks training an ML model that can solve a real problem. As you probably know already, if it just sits in your experimentation environment, nobody else can use it. To make it truly useful, you need to deploy your model so it can handle requests, process data, and deliver predictions in real time. In the past, you would have had to write custom code to serve your model and set up the entire serving infrastructure. You would also need to manage scaling, load balancing, and reliability manually, which could take days or even weeks. This made deploying models time-consuming and error-prone. SageMaker AI simplifies this process by managing the infrastructure and offering a range of deployment options, so you can focus on putting your model into action and realizing its value.

In this chapter, we'll explore various options and strategies for deploying models in SageMaker AI. You will work with an existing pretrained model and deploy it using a variety of SageMaker AI deployment options, including **real-time endpoints**, **serverless endpoints**, **asynchronous inference endpoints**, and **batch transform jobs**. You will also practice advanced deployment techniques, such as **shadow testing**, which lets you validate a new model variant without impacting live traffic, and **canary traffic shifting**, which gradually directs a small portion of production requests to a new model before a full rollout. Along the way, you'll also learn how to configure **data capture** to capture inference requests and responses for monitoring and evaluation purposes.

This chapter covers the following topics:

- Preparing your JupyterLab Notebook for model deployment
- Deploying your model to a real-time inference endpoint
- Deploying your model to a serverless inference endpoint
- Running batch inference with batch transform
- Deploying your model to an asynchronous inference endpoint
- Setting up a shadow test with a SageMaker inference endpoint
- Using canary traffic shifting when performing blue/green deployments

As you work through the hands-on examples in this chapter, you will gain a practical understanding of each deployment approach, the scenarios where it fits best, and how to perform inference with your deployed model. Now, let's get started!

# Technical requirements

Before proceeding with the hands-on examples in this chapter, confirm that the following prerequisites and setup requirements have been addressed:

- **Sufficient account-level quota for selected ML instance types**: When deploying models, ensure that the applied account-level quota value for the selected ML instance types (for example, **ml.m5.xlarge for endpoint usage** , **ml.m5.xlarge for transform job usage**, and **ml.m5.large for endpoint usage**) is set to at least 2. If the quota value is `0`, SageMaker AI will not be able to provision the required compute resources, and the job or endpoint creation will fail. You can review and request quota increases through the **Service Quotas** console (within the AWS Management Console).
- **An existing SageMaker Studio space**: You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1* to follow along with the examples in this book.
- **A code editor installed on your local machine** (such as Visual Studio Code or Sublime Text): You'll need this when working with the code and configuration files used throughout the hands-on exercises and examples in this book.

The Jupyter notebooks, source code, and other files used for each chapter are available in this book's GitHub repository. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

**Note**

It is recommended to use an **IAM** user with limited permissions instead of the root account when running the examples in this book. We will discuss this, along with other security best practices, in detail toward the end of this book. If you are just starting to use AWS, you may proceed with using the root account in the meantime.

## Preparing your JupyterLab notebook for model deployment

In this section, you will prepare and configure your JupyterLab notebook, which you will use throughout this chapter to deploy a pretrained XGBoost model. You'll start by creating a new notebook and installing the specific version of the SageMaker Python SDK needed for the examples in this chapter. After that, you'll download a prebuilt model file (*model.tar.gz*) from the GitHub repository and upload it to Amazon S3 so it's ready for deployment. Once the model is in S3, you'll retrieve the required container image and configuration parameters needed to deploy the model to different SageMaker AI endpoints. In the following sections, this model will be deployed across the various SageMaker AI deployment options, allowing you to see how the same model can run in different inference setups.

The model we'll work with over the next few sections is the XGBoost model trained in *Chapter 3*. It takes input values for `a` and `b` and predicts the corresponding `target` value automatically. To refresh your memory of how the features relate to the target variable, here is the same scatterplot from *Chapter 3*, with points colored by their class label.

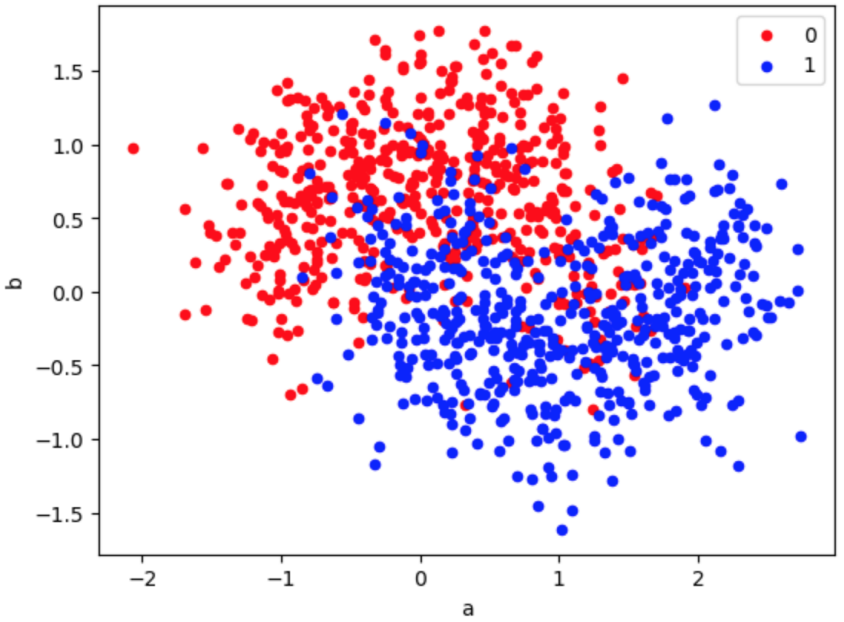



*Figure 8.1 — Scatterplot of the binary classification dataset used to train the XGBoost model*

Red points indicate `target = 0` (**LABEL 0**) and blue points indicate `target = 1` (**LABEL 1**). This visualization helps you check the separation between the two classes based on `a` and `b`, and provides a quick sanity check prior to running inference with the model. While we won't go through the steps to train the model again, it's important to have a clear idea of what the inputs and expected outputs look like. This way, you can quickly verify that everything is working correctly when you run inference. For instance, when the model receives `a = 1.9127`… and `b = -0.4271`…, it should yield an output of `1` (**LABEL 1**).

**Note**

This section assumes that you are running the hands-on examples inside a space in Amazon SageMaker Studio. You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1*. Make sure the space is running before moving forward with the examples in this chapter.

Let's proceed to set up the notebook and the necessary steps for deploying the model.

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under

**Notebook**). Before continuing with the next steps, rename the notebook to `deployment_options.ipynb`. We will run the code in the following steps within this notebook.

2. Uninstall any existing versions of `sagemaker` and `sagemaker-serve`, then install the required versions (`sagemaker==3.5.0` and `sagemaker-serve==1.2.0`):

```
%pip uninstall -y sagemaker sagemaker-serve
%pip install "sagemaker==3.5.0"
%pip install "sagemaker-serve==1.2.0"
```

3. Let's install the `conda-pack` package as well:

```
%pip install conda-pack
```

4. Now that we've installed the required packages and package versions, restart the kernel by selecting **Kernel** | **Restart Kernel** from the menu.

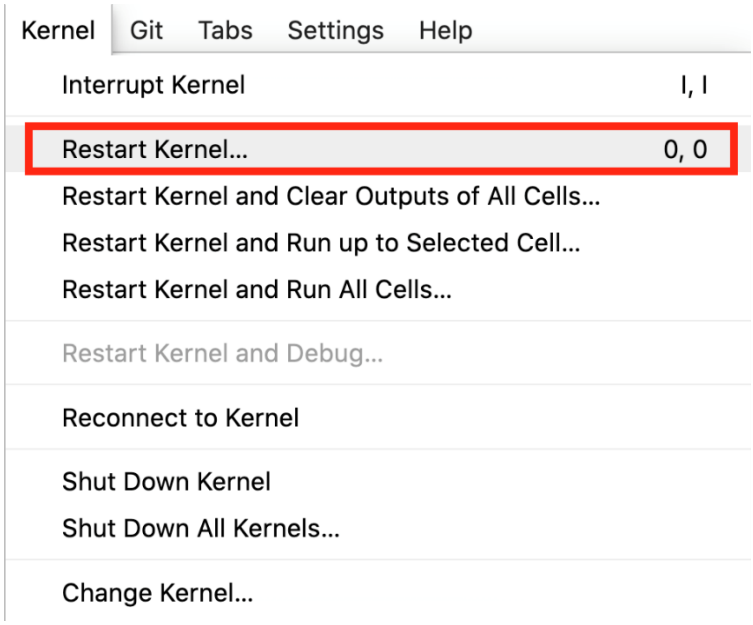



*Figure 8.2 — Restarting the kernel*

This will help ensure the newly installed package versions are properly loaded and available for use. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

> **Note**
>
> After installing the required package versions, verify that the correct versions are in place by running `%pip show sagemaker` and `%pip show sagemaker-serve`. You should see `sagemaker 3.5.0` and `sagemaker-serve 1.2.0`, which confirms that your environment is ready for the examples in this chapter.

5. After restarting the kernel, run the following in a new cell to suppress specific namespace conflict warnings using Python's `warnings` module, to help you focus on the relevant output.

```
import warnings

m="Field .* has conflict with protected namespace"
warnings.filterwarnings(
    "ignore",
    message=m
)
```

6. Import the SageMaker Python SDK in your notebook and check the location of the installed SageMaker package by running the following:

```
import sagemaker
sagemaker.__file__
```

This should return a path similar to:

```
'/opt/conda/lib/python3.12/site-packages/sagemaker/__init__.py'
```

7. Define the `copy_model_server_file()` function which copies the `model.py` file from the triton folder in the SageMaker `serve` directory to the root of the `serve` directory:

```
import os
import shutil

def copy_model_server_file():
    sagemaker_path = os.path.dirname(
        sagemaker.__file__)
```

```
    source_path = os.path.join(
        sagemaker_path, 'serve',
        'model_server', 'triton', 'model.py')

    destination_path = os.path.join(
        sagemaker_path, 'serve', 'model.py')

    if os.path.exists(source_path):
        shutil.copy(source_path, destination_path)
        print(f"Successfully copied "
              f"model.py from {source_path} "
              f"to {destination_path}")
    else:
        print(f"Source file not found "
              f"at {source_path}")
```

8. Use the `copy_model_server_file()` function defined in the previous step to copy the `model.py` file to the `serve` directory.

```
copy_model_server_file()
```

> **Note**
>
> This should address a possible issue that may have existed in earlier versions of the SageMaker Python SDK when building and deploying XGBoost and other similar ML models.

9. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length. We'll use this to give our endpoints and other resources a unique name so they don't conflict with existing ones in your account.

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase,
                       k=length))
```

10. Let's generate a random string using the `generate_string()` function and store it in the variable `unique`. Print the value of `unique` afterward.

```
unique = generate_string()
print(unique)
```

This should give you a random string, such as `boelab` or `ijtbxo`, which we'll use to give endpoints and other resources unique names throughout the chapter. For example, the real-time inference endpoint will be named `xgb-ep-{unique}`, the serverless endpoint `xgb-serverless-ep-{unique}`, the batch transform model `xgb-transform-model-{unique}`, and the asynchronous endpoint `xgb-async-ep-{unique}`. This helps ensure all your resources have distinct names and avoids conflicts.

11. Next, retrieve the current AWS region associated with your session:

```
import boto3
region = boto3.Session().region_name
print(region)
```

You should get `us-east-1` as the region name value. If you are running this notebook in a different region, or if `boto3` was configured with a specific region in your environment, you may get a different region value.

12. Run the following block of code to define the function `download_file()` that can download any file from the `chapter08` folder of the official GitHub repository of this book.

```
import urllib.request
from pathlib import Path

BASE_URL_PARTS = [
    "https://raw.githubusercontent.com/",
    "PacktPublishing/",
    "Machine-Learning-Engineering-on-",
    "AWS-Second-Edition/",
    "refs/heads/main/",
    "chapter08/",
]
```

```
def download_file(filename: str):
    url = "".join(BASE_URL_PARTS) + filename

    output_file = Path(filename)
    urllib.request.urlretrieve(url, output_file)

    return output_file
```

You can later call this function with the desired filename to save it locally (in the SageMaker Studio space).

13. Now, use the function you defined to download the model file (`model.tar.gz`) from GitHub.

```
download_file("model.tar.gz")
```

14. Initialize a `Session` instance and retrieve the execution role and default S3 bucket associated with your SageMaker session:

```
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)

session = Session()
role = get_execution_role()
bucket = session.default_bucket()
```

**Note**

The execution role used here is the same role used by your SageMaker Studio space, which allows the notebook to interact with AWS services such as Amazon S3 and SageMaker on your behalf. In production environments, it is recommended to use a separate IAM role for the inference endpoint so that permissions can be scoped specifically to what the deployed model needs. This helps follow the principle of least privilege and improves overall security.

15. Upload the model file to an S3 bucket using the `upload_data()` method, with the specified bucket and key prefix:

```
MODEL_DIRECTORY = "model"

s3_model = session.upload_data(
    "model.tar.gz",
    bucket=bucket,
    key_prefix="{}/{}".format(unique, MODEL_DIRECTORY)
)
```

16. After uploading, inspect the returned value to see where the `model.tar.gz` file was stored:

```
s3_model
```

This should return an S3 path similar to:

```
s3://sagemaker-us-east-1-.../.../model/model.tar.gz'
```

17. Retrieve the SageMaker AI container image URI for XGBoost inference:

```
from sagemaker.core import image_uris

image = image_uris.retrieve(
    framework="xgboost",
    region="us-east-1",
    image_scope="inference",
    version="1.7-1"
)

print(image)
```

This should give you the container image URI, which will be used in the following sections to deploy the XGBoost model.

```
http://....dkr.ecr.us-east-1.amazonaws.com/sagemaker-xgboost:1.7-1
```

In this example, the container image version is `1.7-1`. Keep in mind that SageMaker AI provides multiple versions of the XGBoost container image, so you can select the version that matches the XGBoost library required by your model.

At this point your notebook is all set up and ready for the next steps. With the specific package versions installed, the pretrained model uploaded to S3, and the container image URI ready, you can now jump into deploying the model in the next section. Keep in mind that these setup steps would be similar for other models, with the main differences being the container image and the model artifact used.

## Deploying your model to a real-time inference endpoint

Real-time inference endpoints in SageMaker AI let you deploy ML models so they can respond to prediction requests through an API. The model runs on a fully managed instance that continuously handles requests as they arrive. This deployment option works well when you need predictions returned quickly for individual inputs with low latency. You can also deploy models trained outside of SageMaker AI using this method, so they can run on a managed instance and handle real-time inference. In this section, you'll deploy a pretrained XGBoost model to a SageMaker AI real-time inference endpoint. Once the endpoint is up and running, you'll send sample requests to it and see how it generates predictions in real time.

Suppose you want to make your model available outside of your notebook or development environment so that other applications can invoke it. You would typically need to build a backend API from scratch that loads the model, processes incoming requests, and returns the inference output. Normally, this API would run inside a container on an EC2 instance that you have to manage (*Figure 8.3*).

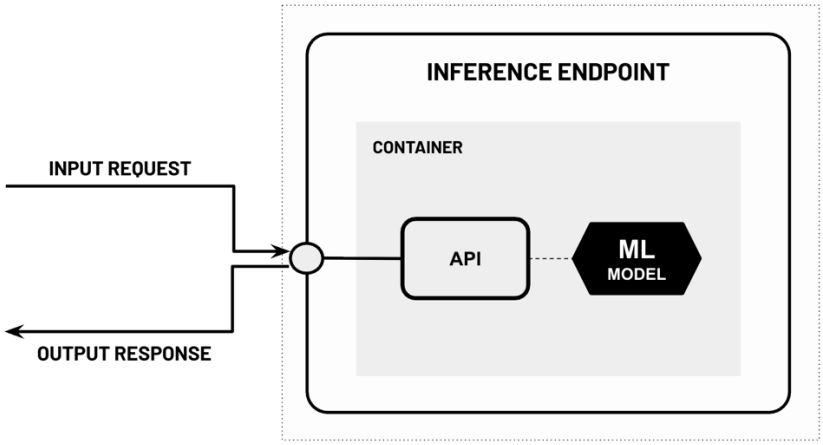



*Figure 8.3 — Deploying your model to a SageMaker AI inference endpoint*

If you were doing this manually, every time you wanted to deploy a different model or change settings, you would have to spin up a new container and EC2 instance. You would also need to configure the inference endpoint to transfer logs to a centralized logging service, set up monitoring and alerting for errors or latency issues, and manage scaling to handle varying traffic loads. On top of that, you would be responsible for handling updates, security patches, and maintaining high availability. SageMaker AI takes care of all that for you, letting you just specify the model, container image, instance type, and other parameters when creating the endpoint using the SageMaker Python SDK. That way, you can deploy and update models quickly without worrying about the underlying infrastructure. Over the next few sections, you'll see how SageMaker AI has automated and abstracted much of this while providing deployment options like serverless inference, async inference, and batch transform.

**Note**

In this section, you'll continue using the `deployment_options.ipynb` notebook. The upcoming steps and code will all be run in this notebook.

With this in mind, let's go through the steps to deploy the XGBoost model to a SageMaker AI real-time inference endpoint:

1. Import the necessary classes and helper functions for building and deploying your model, including `SchemaBuilder`, `InferenceSpec`, and `ModelBuilder`.

```
from sagemaker.serve.builder.schema_builder import (
    SchemaBuilder
)
from sagemaker.serve.spec.inference_spec import (
    InferenceSpec
)
from sagemaker.serve import ModelBuilder
```

2. Define a custom inference specification for your XGBoost model (`XGBoostModelSpec`), with the `load()` method to load the XGBoost model and the `invoke()` method to make predictions based on the input data.

```
import numpy as np
import pandas as pd
import xgboost
from xgboost import XGBClassifier


class XGBoostModelSpec(InferenceSpec):
    def load(self, model_dir: str):
        m = XGBClassifier()
        m.load_model(model_dir + "/xgboost-model")
        return m

    def invoke(self,
               input_object: object,
               model: object):
        pred_proba = model.predict_proba(input_object)
        pred = np.argmax(pred_proba, axis=1)
        return pred
```

3. Next, initialize an instance of your custom inference specification class to define how the model will be loaded and invoked during deployment.

```
inference_spec = XGBoostModelSpec()
```

4. Define a sample input and output, and initialize a `SchemaBuilder` instance. This helps SageMaker AI understand the expected input and output shapes and types for your model.

```
import numpy as np

sample_input = np.array([ 1.9127373 , -0.42719723])
sample_output = np.array([1.0])

schema_builder = SchemaBuilder(
    sample_input=sample_input,
    sample_output=sample_output
)
```

5. Initialize a `ModelBuilder` instance to prepare the SageMaker AI model for deployment.

```
model_builder = ModelBuilder(
    model_path=s3_model,
    role_arn=role,
    image_uri=image,
    inference_spec=inference_spec,
    schema_builder=schema_builder
)
```

   Here, we configure it with the model artifact stored in Amazon S3, IAM role, Docker image URI, `XGBoostModelSpec` instance, and `SchemaBuilder` instance. Together, these configuration settings tell SageMaker AI where to retrieve the model, which execution role to use, which container image will be used to host the model, and how inference inputs and outputs are structured.

6. Call the `build()` method on the `ModelBuilder` instance (`model_builder`) to initiate the process of building and deploying the model on SageMaker.

```
model_builder.build()
```

This step packages the model artifact, the container image, and the inference specification into a deployable SageMaker AI model.

7. Define a unique name for the SageMaker AI inference endpoint where the model will be deployed.

```
endpoint_name = f"xgb-ep-{unique}"
```

Here, you construct the endpoint name dynamically using the value in `unique`, which helps avoid naming conflicts with other inference endpoints and endpoint configurations.

8. Deploy the model to a SageMaker AI inference endpoint by calling the `deploy()` method on your `ModelBuilder` instance (`model_builder`).

```
predictor = model_builder.deploy(
    endpoint_name=endpoint_name
)
```

This sets up a real-time inference endpoint with the specified name and runs your model inside a managed ML instance. After a few minutes, you should see deployment logs confirming that the real-time inference endpoint has been created successfully.

```
...
Deployment successful: Endpoint 'xgb-ep-...' using TRITON in
SAGEMAKER_ENDPOINT mode
```

The returned `predictor` object can then be used to invoke predictions on new data.

> **Note**
>
> This step may take approximately 5–15 minutes to complete. Feel free to grab a cup of coffee or tea while waiting.

9. Configure how the `predictor` sends input data to the endpoint and how it interprets the responses by setting the serializer and deserializer.

```
from sagemaker.core.deserializers import (
    JSONDeserializer
)
```

```
from sagemaker.core.serializers import (
    CSVSerializer,
)

predictor.serializer = CSVSerializer()
predictor.deserializer = JSONDeserializer()
```

Here, the predictor converts your input data into a format the model expects (CSV) and then converts the model's response into something your code can easily use.

**Note**

When you send data to the endpoint, the predictor first formats the input as CSV so the model can read it properly. After the model processes the request and returns a result in JSON format, the predictor uses the deserializer to parse that JSON response into a Python data structure such as a dictionary or list.

10. Define the function `invoke_endpoint()` to invoke the inference endpoint with input values and return the predicted result.

```
import json

def invoke_endpoint(a, b, predictor=predictor):
    payload = f"{a},{b}"

    result = predictor.invoke(
        body=payload,
        content_type="text/csv"
    )

    return int(result.body)
```

This function uses the `invoke()` method of the specified `predictor` to send input data to the deployed inference endpoint. The values for `a` and `b` are formatted as a CSV payload and passed to the endpoint as the request body. The endpoint processes the input and returns the predicted result, which the function then converts to an integer.

11. Let's test the inference endpoint using the `invoke_endpoint()` function we defined in the previous step. Use the input values `-1.0` and `1.5` and call the function to retrieve a sample prediction result.

```
invoke_endpoint(-1.0, 1.5)
```

You should get a predicted value of `0` (**LABEL 0**).

12. Let's test the inference endpoint again, this time using the input values `1.0` and `-1.0`:

```
invoke_endpoint(1.0, -1.0)
```

You should get a predicted value of `1` (**LABEL 1**).

13. Finally, delete the SageMaker AI real-time inference endpoint.

```
predictor.delete()
```

This removes the deployed endpoint and terminates the managed ML instance hosting the model, to help ensure that you are no longer billed for the associated compute resources.

*Wasn't that easy?!* At this point, you've seen how quickly you can go from a model stored in S3 to a fully running real-time endpoint. While building these types of inference endpoints yourself is possible, SageMaker AI makes deploying ML models quick and effortless. Using the SageMaker Python SDK, you were able to deploy the model to a managed ML instance by specifying the container image, model artifact, IAM execution role, and other configuration settings. As you'll see over the next few sections, by adding just a few lines of code using the SageMaker Python SDK, you'll be able to use other deployment options such as serverless inference, asynchronous inference, and batch transform. These options build on what you learned here, letting you handle different workloads and use cases with minimal extra setup.

# Deploying your model to a serverless inference endpoint

In this section, you'll learn how to deploy a model to a SageMaker AI serverless inference endpoint. These endpoints let you run inference without managing or provisioning servers, which makes them ideal for applications with intermittent or unpredictable traffic. SageMaker AI automatically scales the compute resources in or out depending on the traffic, and you only pay for the requests you make (and not for idle time), making this a cost-effective deployment

option. This allows you to focus on delivering value through your models instead of managing infrastructure or capacity.

What makes this different from real-time endpoints is the concept of cold starts. Since serverless endpoints do not keep compute resources running continuously, there may be a short delay when a new request triggers the provisioning of resources, especially if the endpoint has not received traffic for some time. This startup time, known as a **cold start**, can introduce additional latency compared to always-on real-time endpoints. However, for workloads with intermittent traffic, this trade-off is often acceptable given the cost savings and automatic scaling benefits.

> **Note**
>
> You can mitigate cold starts by using **Provisioned Concurrency**, which keeps endpoints warm and ready to handle requests. While the hands-on example in this section does not include Provisioned Concurrency, it is an important option to consider, especially when you have predictable bursts in your traffic and want to balance latency with cost efficiency.

Let's proceed with deploying your model to a serverless inference endpoint:

1. Using the same `deployment_options.ipynb` notebook from the previous section, define a unique name for the serverless endpoint you'll set up in this section.

   ```
   endpoint_name = f"xgb-serverless-ep-{unique}"
   ```

   Here, we use a unique identifier (`unique`) to ensure that the endpoint name doesn't conflict with any existing endpoints or endpoint configurations in your account. In addition, the endpoint name starts with `xgb-serverless-` while in the previous section it started with `xgb-ep-`. This naming difference also helps prevent any conflicts between endpoints.

2. Configure and deploy the model to a serverless endpoint using `ServerlessInferenceConfig`:

   ```
   from sagemaker.core.inference_config import (
       ServerlessInferenceConfig
   )

   inference_config = ServerlessInferenceConfig(
       memory_size_in_mb=2048
   ```

```
)

serverless_predictor = model_builder.deploy(
    endpoint_name=endpoint_name,
    inference_config=inference_config,
)
```

Here, we use `ServerlessInferenceConfig` to specify the memory allocated to the serverless container. Then, we call `deploy()` to create the serverless endpoint and make the model available for inference.

> **Note**
>
> This step may take approximately 5–15 minutes to complete. Feel free to grab a cup of coffee or tea while waiting.

3. Configure the input serializer and output deserializer for the serverless inference endpoint.

```
serverless_predictor.serializer = CSVSerializer()
serverless_predictor.deserializer = JSONDeserializer()
```

Here, the predictor serializes the request payload as CSV and deserializes the endpoint response from JSON.

4. Let's test the inference endpoint using the `invoke_endpoint()` function. Use the input values `-1.0` and `1.5` and call the function to retrieve a sample prediction result.

```
invoke_endpoint(
    -1.0, 1.5,
    predictor=serverless_predictor
)
```

You should get a predicted value of `0` (**LABEL 0**).

5. Let's test the inference endpoint again, this time using the input values `1.0` and `-1.0`:

```
invoke_endpoint(
    1.0, -1.0,
    predictor=serverless_predictor
)
```

You should get a predicted value of `1` (**LABEL 1**).

6. Finally, clean up by deleting the serverless inference endpoint.

```
serverless_predictor.delete()
```

At this point, you should be comfortable with deploying a model to a serverless inference endpoint. In this section, you saw that it only takes a few additional lines of code with `ServerlessInferenceConfig` to configure the endpoint. SageMaker AI automatically provisions resources, scales them as needed, and routes requests efficiently. This allows you to run inference reliably while keeping costs low and avoiding infrastructure management.

# Running batch inference with batch transform

In the previous two sections, you explored real-time and serverless inference endpoints for handling individual requests or unpredictable traffic patterns. In this section, you'll use **batch transform** to perform inference on large datasets without deploying a real-time endpoint, making it ideal for asynchronous or bulk data processing. When running batch inference with batch transform, SageMaker AI launches a managed ML instance, loads the specified model and input data, performs inference, writes the results to S3, and then terminates the instance automatically, providing a fully managed, scalable workflow similar to SageMaker Processing.

**Note**

In this section, you'll continue using the `deployment_options.ipynb` notebook. The upcoming steps and code will all be run in this notebook.

Let's walk through the steps to run batch inference using batch transform:

1. Use the `download_file()` function to download the CSV file containing the data we'll use in this section.

```
download_file("batch_input.csv")
```

This CSV file contains multiple rows of input values for the model, for example, `1.9127..., -0.4271...`. Each row represents a separate record that will be processed by the model, allowing you to run predictions on multiple inputs in a single batch transform job.

2. Upload the model file to the S3 bucket using the following block of code.

```
INPUT_DIRECTORY = "input"

batch_input = session.upload_data(
    "batch_input.csv",
    bucket=bucket,
    key_prefix="{}/{}".format(unique, INPUT_DIRECTORY)
)
```

3. Build a SageMaker AI model to use for the batch transform job.

```
model_name = f"xgb-transform-model-{unique}"
model_builder.build(model_name=model_name)
```

This step prepares the model using the model artifact, container image, and inference configuration defined earlier in the *Preparing your JupyterLab Notebook for Model Deployment* section, making it ready to run inference with batch transform.

4. Initialize the `Transformer` instance for the batch transform job.

```
output_path = f"s3://{bucket}/transform-output/"

from sagemaker.core.transformer import Transformer

transformer = Transformer(
    model_name=model_name,
    instance_count=1,
    instance_type="ml.m5.xlarge",
    accept="text/csv",
    assemble_with="Line",
    output_path=output_path,
    sagemaker_session=session,
)
```

Here, you specify how SageMaker AI should run the batch transform job, including the compute resources, input and output formats, and where to store the results in S3.

5. Run the batch transform job on the uploaded CSV input.

```
transformer.transform(
    batch_input,
```

```
    content_type="text/csv",
    split_type="Line",
    input_filter="$[1:]",
)
```

After a few minutes, you should see a log message similar to:

```
Final Resource Status: Completed
```

This confirms that the batch inference job finished successfully and that the results are now stored in S3.

> **Note**
>
> This step may take around 5 to 15 minutes to complete. Feel free to grab a cup of coffee or tea while you wait.

6. Verify the output file(s) in S3 using the `aws s3 ls` command.

```
!aws s3 ls {output_path}
```

The output should include a file similar to:

```
2026-01-01 00:00:00        800 batch_input.csv.out
```

7. Copy the batch transform output file (`batch_output.csv.out`) from the S3 bucket to your local environment (that is, your Studio space) so you can inspect the results.

```
output_file_path = output_path + "batch_input.csv.out"
!aws s3 cp {output_file_path} batch_output.csv
```

Here, we also rename the file to `batch_output.csv` for easier reference.

8. *So, how does your output CSV file look after the batch transform?* Let's inspect the results of your batch transform job by running the following:

```
!cat batch_output.csv
```

You will find that the `batch_output.csv` file contains one prediction per line, and each line corresponds to a row in the input CSV. For example, you might get the following output:

```
1.0
1.0
...
0.0
0.0
1.0
...
1.0
1.0
```

Each line contains a predicted class for the corresponding input row. For example, if one of your input rows contains the values `1.9127...` and `-0.4271...`, you will see a corresponding output of `1.0`. Each prediction will be either `0.0` (**LABEL 0**) or `1.0` (**LABEL 1**), which you can use to analyze, review, or post-process your results.

At this point, you should be comfortable running inference with batch transform and understand how it can be used to perform high-throughput predictions on multiple records in a single batch job. While the example in this section involved a small dataset, the same approach scales seamlessly to much larger datasets in production.

# Deploying your model to an asynchronous inference endpoint

Over the last few sections, we explored different ways to run inference with SageMaker AI, including real-time endpoints for immediate predictions, serverless endpoints for intermittent traffic, and batch transform for large-scale batch processing. Each approach has its own strengths and is suited to different workload requirements. In this section, you'll learn how to deploy a model to a SageMaker AI asynchronous inference endpoint. These endpoints allow you to handle longer-running requests without blocking real-time traffic, making them ideal for large inputs or computationally intensive models. You send a request, the endpoint processes it in the background, and stores the output in S3 for later retrieval. To keep things simple, we'll use a small, illustrative example that demonstrates the workflow and key concepts of asynchronous inference. As we go through the hands-on portion of this section, you'll see how this simplified example can be extended and adapted to more complex, real-world scenarios.

**Note**

In this section, you'll continue using the `deployment_options.ipynb` notebook. The upcoming steps and code will all be run in this notebook.

With this in mind, let's proceed with deploying our ML model to an asynchronous inference endpoint:

1. First, define the asynchronous endpoint name (`endpoint_name`) and the Amazon S3 output location (`output_path`), where the inference results will be stored.

```
endpoint_name = f"xgb-async-ep-{unique}"
output_path = f"s3://{bucket}/async-output/"
```

2. Run the following code to prepare the asynchronous inference configuration and build the model that will be deployed to the endpoint.

```
from sagemaker.serve.async_inference import (
    async_inference_config
)

config = async_inference_config.AsyncInferenceConfig(
    output_path=output_path
)

model_builder = ModelBuilder(
    model_path=s3_model,
    role_arn=role,
    image_uri=image,
    inference_spec=inference_spec,
    schema_builder=schema_builder
)

model_builder.build()
```

This should yield a set of logs similar to the following:

```
...
Creating model with name: model-...
```

```
Model has been created: 'model-...' using server TRITON in
SAGEMAKER_ENDPOINT mode
```

3. Next, using the asynchronous inference configuration (`config`) prepared in the previous step, deploy the model and create the asynchronous inference endpoint.

```
async_predictor = model_builder.deploy(
    endpoint_name=endpoint_name,
    inference_config=config
)
```

After a few minutes, you should see deployment logs confirming that the endpoint has been created successfully.

```
...
Deployment successful: Endpoint 'xgb-async-ep-...' using TRITON in
SAGEMAKER_ENDPOINT mode
```

> **Note**
>
> This step may take approximately 5–15 minutes to complete. Feel free to grab a cup of coffee or tea while waiting.

4. Download the input file (`async_input.csv`) that you will use for the asynchronous inference request.

```
download_file("async_input.csv")
```

5. After downloading the `async_input.csv` file, upload it to Amazon S3 where SageMaker AI will read the input during inference.

```
INPUT_DIRECTORY = "input"

async_input = session.upload_data(
    "async_input.csv",
    bucket=bucket,
    key_prefix="{}/{}".format(unique, INPUT_DIRECTORY)
)
```

6. Now, invoke the asynchronous inference endpoint using the input file (`async_input.csv`) uploaded to S3 in the previous step.

```
import json

result = async_predictor.invoke_async(
    input_location=async_input,
    content_type="text/csv"
)
```

Here, instead of invoking the endpoint directly and waiting for an immediate response as with a real-time endpoint, the request is processed in the background, and the results are written to Amazon S3 once the job is complete.

7. Because the request runs asynchronously, the results and logs don't appear right away as they would with a real-time endpoint. You can run the following code to inspect what happened during the request.

```
import boto3

client = boto3.client("logs")

ep_name = endpoint_name
log_group = f"/aws/sagemaker/Endpoints/{ep_name}"

response = client.describe_log_streams(
    logGroupName=log_group,
    orderBy="LastEventTime",
    descending=True,
    limit=1
)

latest_stream = response["logStreams"][0]

events = client.get_log_events(
    logGroupName=log_group,
    logStreamName=latest_stream["logStreamName"],
    limit=50,
    startFromHead=False
)
```

```
for e in events["events"]:
    print(e["message"])
```

Here, we retrieve the latest log stream from the endpoint's CloudWatch Logs to review the asynchronous inference request. For example, you might see a log like this:

```
...:[sagemaker logs] [...] Inference request succeeded.
ModelLatency: 0 us, RequestDownloadLatency: 251690 us,
ResponseUploadLatency: 102232 us, TimeInBacklog: 37 ms,
TotalProcessingTime: 462 ms
```

Let's dive a bit deeper into what these metrics mean. `ModelLatency` reflects the interval of time taken by the deployed model to respond, `RequestDownloadLatency` measures how long it took to download the input from S3, `ResponseUploadLatency` indicates the time to upload the results back to S3, `TimeInBacklog` reflects how long the request waited in the queue, and `TotalProcessingTime` represents the total elapsed time from when the inference request was received until the request finished processing.

**Note**

In case you don't see a similar set of logs, you may need to rerun the code block after a minute or so, as CloudWatch can take a moment to propagate the logs from the asynchronous request. For more information on this topic, feel free to check: `https://docs.aws.amazon.com/sagemaker/latest/dg/async-inference-monitor.html`.

8. Let's check the S3 path where the inference output was stored.

```
output_location = result.output_location
output_location
```

This should return a path similar to:

```
's3://sagemaker-us-east-1-.../async-output/....out'
```

9. Now, download the output file from S3 to your Studio space so you can inspect it:

```
!aws s3 cp {output_location} async_output.csv
```

10. Inspect the results stored in the output file (`async_output.csv`):

    ```
    !cat async_output.csv
    ```

    You should get a predicted value of `0.0` (**LABEL 0**).

11. Finally, delete the asynchronous inference endpoint.

    ```
    async_predictor.delete()
    ```

    This removes the deployed endpoint and terminates the managed ML instance hosting the model (`ml.m5.large`), which helps ensure that you are no longer billed for the associated compute resources.

By now, you should have a good sense of how to deploy your model to an asynchronous inference endpoint. You may have wondered whether this was really necessary for a small CSV file, since for small workloads, a serverless or real-time endpoint would have worked just fine. Asynchronous endpoints shine when you're dealing with larger datasets, longer processing times, or models that aren't suited for immediate responses, such as big CSVs, video files, or image collections. In this section, we kept things simple so you could focus on the core workflow. As you worked through the hands-on portion, you saw how this setup could scale to handle more complex, real-world scenarios.

# Setting up a shadow test with a SageMaker inference endpoint

As an ML engineer, you'll often encounter situations where you need to evaluate a new model in a production-like environment without risking disruption to live users. There are various ways that help you solve this requirement, and **shadow testing** is one effective approach that let's you compare the outputs of a new model against your existing production model in real time. In this section, we'll set up shadow testing on a SageMaker AI inference endpoint, which allows you to safely evaluate and compare model outputs while keeping live traffic unaffected. With this approach, all requests sent to an inference endpoint are processed by the primary production model, while a mirrored copy of the requests is also sent to the shadow model (*Figure 8.4*).

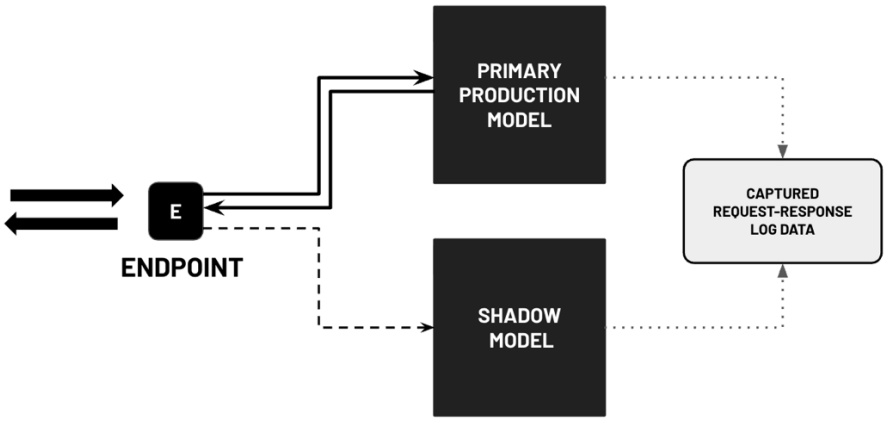



*Figure 8.4 — Setting up shadow testing*

The shadow model's responses are captured and stored (for example, in a storage bucket) for evaluation but not returned to the client. With shadow testing, you can monitor performance, compare accuracy, and verify that the new model behaves as expected before fully shifting production traffic.

We will be working with two pretrained question-answering models, designed to extract relevant answers from a given text context in response to user questions:

- `distilbert-base-uncased-distilled-squad` (**MODEL A**) — a smaller, faster model optimized for low-latency inference,
- `roberta-base-squad2` (**MODEL B**) — a larger, more expressive model that can capture nuanced answers.

Using these models, we'll simulate a shadow test in which **MODEL A** handles live requests while **MODEL B** receives mirrored traffic for evaluation. SageMaker AI manages this setup by creating separate production variants for each model, each running in its own container on dedicated ML instances. This allows the endpoint to route live traffic to the primary model while simultaneously sending copies of the requests to the shadow model without affecting users. All request-response pairs from both variants are then captured and stored in an S3 bucket for offline analysis and evaluation.

## Creating a shadow test

In the next set of steps, you'll retrieve the SageMaker AI inference container images for two pretrained models, register the models, and create an endpoint configuration that includes both a production variant (**MODEL A**) and a shadow variant (**MODEL B**). You'll also configure data capture to log all requests and responses. Finally, you'll deploy a SageMaker AI real-time inference endpoint using the endpoint configuration you created, which routes live traffic to the production model while mirroring requests to the shadow model for evaluation and offline analysis without impacting live users.

Let's proceed with setting up shadow testing:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Rename the notebook to `deployment_strategies.ipynb`.
2. Uninstall any existing versions of `sagemaker` and `sagemaker-serve`, then install the required versions (`sagemaker==3.5.0` and `sagemaker-serve==1.2.0`):

```
%pip uninstall -y sagemaker sagemaker-serve
%pip install "sagemaker==3.5.0"
%pip install "sagemaker-serve==1.2.0"
```

3. Now that we've installed the required packages and package versions, restart the kernel by selecting **Kernel | Restart Kernel** from the menu. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

> **Note**
>
> After installing the required package versions, verify that the correct versions are in place by running `%pip show sagemaker` and `%pip show sagemaker-serve`. You should see `sagemaker 3.5.0` and `sagemaker-serve 1.2.0`, which confirms that your environment is ready for the examples in this chapter.

4. Use the following block of code to retrieve the URI of the inference container image that SageMaker AI will use to serve your model (**MODEL A**).

```
from sagemaker.core.image_uris import (
    retrieve as retrieve_image
)
```

```
region = 'us-east-1'
image01_params = {
    "framework": 'huggingface',
    "region": region,
    "version": '4.12.3',
    "image_scope": 'inference',
    "base_framework_version": 'pytorch1.9.1',
    "py_version": 'py38',
    "container_version": 'ubuntu20.04',
    "instance_type": 'ml.m5.xlarge'
}

image_01_uri = retrieve_image(**image01_params)
image_01_uri
```

This should give you an inference container image URI similar to the following:

```
'763104351884.dkr.ecr.us-east-1.amazonaws.com/huggingface-pytorch-
inference:1.9.1-transformers4.12.3-cpu-py38-ubuntu20.04'
```

Here, the image configuration values in `image01_params` determine the inference container image URI returned by `retrieve_image()`, which is an alias of `retrieve()` in `sagemaker.core.image_uris`.

5. Retrieve the inference container URI for a second model (**MODEL B**), specifying a different framework version for shadow deployment.

```
image02_params = {
    "framework": 'huggingface',
    "region": region,
    "version": '4.17.0',
    "image_scope": 'inference',
    "base_framework_version": 'pytorch1.10.2',
    "py_version": 'py38',
    "container_version": 'ubuntu20.04',
    "instance_type": 'ml.m5.xlarge'
}

image_02_uri = retrieve_image(**image02_params)
image_02_uri
```

This should give you an inference container image URI similar to the following:

```
'763104351884.dkr.ecr.us-east-1.amazonaws.com/huggingface-pytorch-
inference:1.10.2-transformers4.17.0-cpu-py38-ubuntu20.04'
```

Here, the parameters in `image02_params` specify a different Hugging Face Transformers version (`4.17.0`) as well as a newer PyTorch version (`1.10.2`) for the shadow deployment. This helps ensure that your second model (**MODEL B**) runs with the intended framework versions, which may differ from those used by the first model (**MODEL A**).

6. Define the configuration for the first model (**MODEL A**) using `distilbert-base-uncased-distilled-squad` for inference.

```
model_01_id = 'distilbert-base-uncased-distilled-squad'

first_model = {
    'Image': image_01_uri,
    'ContainerHostname': 'firstModel',
    'Environment': {
        'HF_MODEL_ID': model_01_id,
        'HF_TASK':'question-answering'
    }
}
```

Here, we use the container image URI stored in `image_01_uri`, retrieved in the previous step, which corresponds to the `huggingface-pytorch-inference:1.9.1-transformers4.12.3-cpu-py38-ubuntu20.04` image.

7. Define the configuration for the second model (**MODEL B**), using a different pretrained model (`roberta-base-squad2`) for shadow testing.

```
model_02_id = 'deepset/roberta-base-squad2'

second_model = {
    'Image': image_02_uri,
    'ContainerHostname': 'secondModel',
    'Environment': {
        'HF_MODEL_ID': model_02_id,
        'HF_TASK': 'question-answering'
```

```
    }
}
```

Here, in addition to using a different pretrained model, we're also using a different container image URI stored in `image_02_uri` (`huggingface-pytorch-inference:1.10.2-transformers4.17.0-cpu-py38-ubuntu20.04`).

8. Retrieve the execution role for your current environment and initialize a SageMaker client using `boto3`.

```
import boto3

from sagemaker.core.helper.session_helper import (
    get_execution_role
)

execution_role = get_execution_role()
sagemaker_client = boto3.client('sagemaker')
```

9. Register the first model `shadow-testing-001` (**MODEL A**) with `create_model()`, using the container image configuration that points to `huggingface-pytorch-inference:1.9.1-transformers4.12.3-cpu-py38-ubuntu20.04` and uses the pretrained model `distilbert-base-uncased-distilled-squad`.

```
model_name = "shadow-testing-001"

model_config = {
    "ModelName": model_name,
    "ExecutionRoleArn": execution_role,
    "Containers": [first_model]
}

sagemaker_client.create_model(**model_config)
```

10. Register the second model `shadow-testing-002` (**MODEL B**) as well, this time using the container image `huggingface-pytorch-inference:`

`1.10.2-transformers4.17.0-cpu-py38-ubuntu20.04` and the pretrained model `deepset/roberta-base-squad2`.

```
model_name = "shadow-testing-002"

model_config = {
    "ModelName": model_name,
    "ExecutionRoleArn": execution_role,
    "Containers": [second_model]
}

sagemaker_client.create_model(**model_config)
```

11. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length. We'll use this to give our endpoints and other resources a unique name so they don't conflict with existing ones in your account.

```
import random
import string

def generate_string(length=6):
    return ''.join(
        random.choices(string.ascii_lowercase,
                       k=length))
```

12. Let's generate a random string using the `generate_string()` function and store it in the variable `unique`. Print the value of `unique` afterward.

```
unique = generate_string(length=12)
print(unique)
```

This should give you a random string which we'll use to give endpoints and other resources unique names throughout the chapter.

13. Generate a unique S3 bucket name to store your data capture files:

```
unique_s3_bucket_name = f"data-capture-{unique}"
```

14. Create the S3 bucket using the `aws s3 mb` command:

```
!aws s3 mb s3://{ unique_s3_bucket_name }
```

15. Define a data capture configuration to store all request payloads and model responses in the specified S3 data capture bucket (`s3_capture_path`).

```
s3_capture_path = f"s3://{unique_s3_bucket_name}/"

data_capture_config = {
    "EnableCapture": True,
    "InitialSamplingPercentage": 100,
    "DestinationS3Uri": s3_capture_path,
    "CaptureOptions": [
        {"CaptureMode": "Input"},
        {"CaptureMode": "Output"}
    ],
    "CaptureContentTypeHeader": {
        "CsvContentTypes": ["text/csv"],
        "JsonContentTypes": ["application/json"]
    }
}
```

Here we configure SageMaker AI to record every request sent to the endpoint and every corresponding model response, rather than only a subset, which is useful for monitoring, auditing, or performance evaluation.

16. Define an endpoint configuration that includes both a production variant (using the model `shadow-testing-001` | **MODEL A**) which handles all live traffic, and a shadow variant (using the model `shadow-testing-002` | **MODEL B**) that receives mirrored requests for testing.

```
config_name = "shadow-testing-endpoint-config-001"

production_variants = [
    {
        "VariantName": "shadow-testing-variant-001",
        "ModelName": "shadow-testing-001",
        "InstanceType": "ml.m5.xlarge",
        "InitialInstanceCount": 1,
```

```
        "InitialVariantWeight": 1,
    }
]

shadow_variants = [
    {
        "VariantName": "shadow-testing-variant-002",
        "ModelName": "shadow-testing-002",
        "InstanceType": "ml.m5.xlarge",
        "InitialInstanceCount": 1,
        "InitialVariantWeight": 1,
    }
]
```

17. Use the `create_endpoint_config()` method to create an endpoint configuration that makes use of your production and shadow variants, with traffic mirroring enabled.

```
sagemaker_client.create_endpoint_config(
    EndpointConfigName=config_name,
    ProductionVariants=production_variants,
    ShadowProductionVariants=shadow_variants,
    DataCaptureConfig=data_capture_config
)
```

Here, we configure data capture so that all requests and responses to the inference endpoint are automatically logged. *How does data capture work?* Data capture works by automatically recording a copy of all requests sent to the production variant, as well as the corresponding responses. When shadow variants are configured, a mirrored copy of each request is also sent to them, but their responses are not returned to the client. These request-response pairs are stored in the S3 bucket specified in `DataCaptureConfig`, allowing you to monitor model performance, compare outputs between variants, and perform offline analysis without impacting live traffic.

**Note**

Data capture can be configured with or without shadow testing. Even for an endpoint running a single production model, data capture will record the requests and responses, storing them in the specified S3 bucket for monitoring and analysis.

18. Use the endpoint configuration prepared in the previous step to launch a managed SageMaker AI real-time inference endpoint that serves live traffic while mirroring requests to the shadow model.

```
sagemaker_client.create_endpoint(
    EndpointName="shadow-testing-endpoint-001",
    EndpointConfigName=config_name,
)
```

19. Define the `wait_for_endpoint()` function that polls the endpoint status until it's in the `InService` state, letting you wait for the deployment to complete before sending requests.

```
from time import sleep

def wait_for_endpoint(endpoint_name):
    while True:
        response = sagemaker_client.describe_endpoint(
            EndpointName=endpoint_name
        )
        status = response['EndpointStatus']
        print(f"ENDPOINT STATUS: {status}")

        if status == 'InService':
            break
        elif status == 'Failed':
            error = "ENDPOINT CREATION FAILED"
            raise RuntimeError(error)

        sleep(15)
```

20. Use the `wait_for_endpoint()` function defined in the previous step to wait for the endpoint to become available.

```
%%time
wait_for_endpoint("shadow-testing-endpoint-001")
```

You should see the endpoint status change from `Creating` to `InService` once deployment is complete, similar to the following:

```
ENDPOINT STATUS: Creating
ENDPOINT STATUS: Creating
...
ENDPOINT STATUS: Creating
ENDPOINT STATUS: Creating
ENDPOINT STATUS: InService
CPU times: user 64.1 ms, sys: 341 µs, total: 64.5 ms
Wall time: 3min 2s
```

21. Use the `describe_endpoint()` method to retrieve and review the endpoint configuration:

```
sagemaker_client.describe_endpoint(
    EndpointName="shadow-testing-endpoint-001"
)
```

You should get the following output:

```
{'EndpointName': 'shadow-testing-endpoint-001',
 'EndpointArn': '...',
 'EndpointConfigName': 'shadow-testing-endpoint-config-001',
 'ProductionVariants': [{'VariantName': 'shadow-testing-variant-001', ...],
 ...
 'ShadowProductionVariants': [{'VariantName': 'shadow-testing-
variant-002', ...],
 'ResponseMetadata': {...}}
```

Here, the presence of the production variant `shadow-testing-variant-001` (variant for **MODEL A**) and the shadow variant `shadow-testing-variant-002` (variant for **MODEL B**) confirms that shadow testing is configured for this inference endpoint.

22. Define the `invoke_endpoint()` function for invoking the inference endpoint.

```
from json import loads as json_loads
from json import dumps as json_dumps

runtime_client = boto3.client('sagemaker-runtime')
```

```
def invoke_endpoint(input_payload, endpoint_name):
    response = runtime_client.invoke_endpoint(
        EndpointName=endpoint_name,
        ContentType="application/json",
        Accept="application/json",
        Body=json_dumps(input_payload),
    )

    response_body = response['Body']

    output = json_loads(response_body.read().decode())
    return output
```

Here, you define a function that sends inference requests to the deployed SageMaker AI real-time endpoint using the SageMaker AI runtime client (`boto3`) and retrieves the model's predictions. The function formats the input payload as JSON, sends it to the endpoint, and then parses the response returned by the model.

23. Use the `invoke_endpoint()` function defined in the previous step to send a sample inference request to the endpoint.

```
input_payload_01 = {"inputs": {
    "context": "Python is a widely-used programming language known for its
readability and ease of use. It supports multiple programming paradigms,
including procedural, object-oriented, and functional programming. Python
is popular in data science, web development, automation, and more.",
    "question": "What are the uses of Python?",
}}

invoke_endpoint(
    input_payload=input_payload_01,
    endpoint_name="shadow-testing-endpoint-001"
)
```

Here, you construct an input payload with the context and question, then send it to the SageMaker AI endpoint using `invoke_endpoint()`. You should then receive a response similar to the following:

```
{'score': 0.5491470098495483,
 'start': 59,
```

```
 'end': 86,
 'answer': 'readability and ease of use'}
```

When the input payload is sent to the SageMaker AI endpoint, the model `shadow-testing-001` (**MODEL A**), using `distilbert-base-uncased-distilled-squad`, analyzes the context and determines the most likely answer to the provided question. The response includes the extracted answer, a confidence score, and the character positions (start and end) indicating where the answer appears in the original context.

> **Note**
>
> Behind the scenes, SageMaker AI routes a copy of the request to the shadow model `shadow-testing-002` (**MODEL B**), using `roberta-base-squad2`. This enables you to observe and compare its responses without impacting production traffic.

24. Send another inference request, this time asking about the Moon.

```
input_payload_02 = {"inputs": {
    "context": "The Moon is Earth's only natural satellite and the fifth-
largest satellite in the Solar
System. It is about one-quarter the diameter of Earth, making it the
largest natural satellite relative to the size of its planet.",
    "question": "How large is the Moon?",
}}

invoke_endpoint(
    input_payload=input_payload_02,
    endpoint_name="shadow-testing-endpoint-001"
)
```

You should get the following output:

```
{'score': 0.5532084107398987,
 'start': 108,
 'end': 132,
 'answer': 'one-quarter the diameter'}
```

Here, the production model `shadow-testing-001` (**MODEL A**), processes the context and returns the answer `"one-quarter the diameter"`. Similar to the first inference request, SageMaker AI also routes a copy of this request to the shadow model `shadow-testing-002` (**MODEL B**). Later, you'll see that the shadow model returns a slightly different response compared to the production model.

In this shadow testing setup, your endpoint continues to serve live traffic as usual, with users receiving only the primary model's answers. Behind the scenes, each request is mirrored to the shadow model, and SageMaker AI captures all request-response pairs in the configured S3 bucket.

**Note**

You may need to wait 10 to 20 minutes for SageMaker AI to fully capture all request–response pairs in the configured S3 bucket. During this time, feel free to grab a cup of coffee or tea. Once the data is available, you can explore both the production and shadow model outputs in the next section.

## Inspecting the captured data

In this section, we'll explore the captured data stored in S3. As we go through the request-response pairs, you'll see how the shadow model's outputs differ from the production model.

Let's proceed with inspecting the captured data in S3:

1. List all data capture files in your S3 bucket:

```
!aws s3 ls { s3_capture_path } --recursive
```

You should get the following output:

```
...       1537 shadow-testing-endpoint-001/shadow-testing-variant-001/.../
39-22-491-cc7a9e82-9bd8-459e-a93e-36b3e1613e71.jsonl
...       1650 shadow-testing-endpoint-001/shadow-testing-variant-002/.../
39-22-486-2dcaf9c8-f500-46fa-9373-4a23349dc04e.jsonl
```

Here, you can find all the captured request and response files generated by both the production model `shadow-testing-001` (**MODEL A**), using `distilbert-base-uncased-distilled-squad`, and the shadow model `shadow-testing-002` (**MODEL B**), using `roberta-base-squad2`. Each *.jsonl* file contains

the inference events that were recorded during the shadow testing setup, organized by endpoint variant and timestamped folders.

2. Copy all captured files from the S3 bucket to examine the endpoint requests and responses locally.

```
!aws s3 cp { s3_capture_path } . --recursive
```

3. Use the File Browser to open the `shadow-testing-endpoint-001/shadow-testing-variant-001` directory, navigate through the subdirectories, locate the *.jsonl* file within the nested folders, and open it in the editor to inspect the captured requests and responses (*Figure 8.5*).

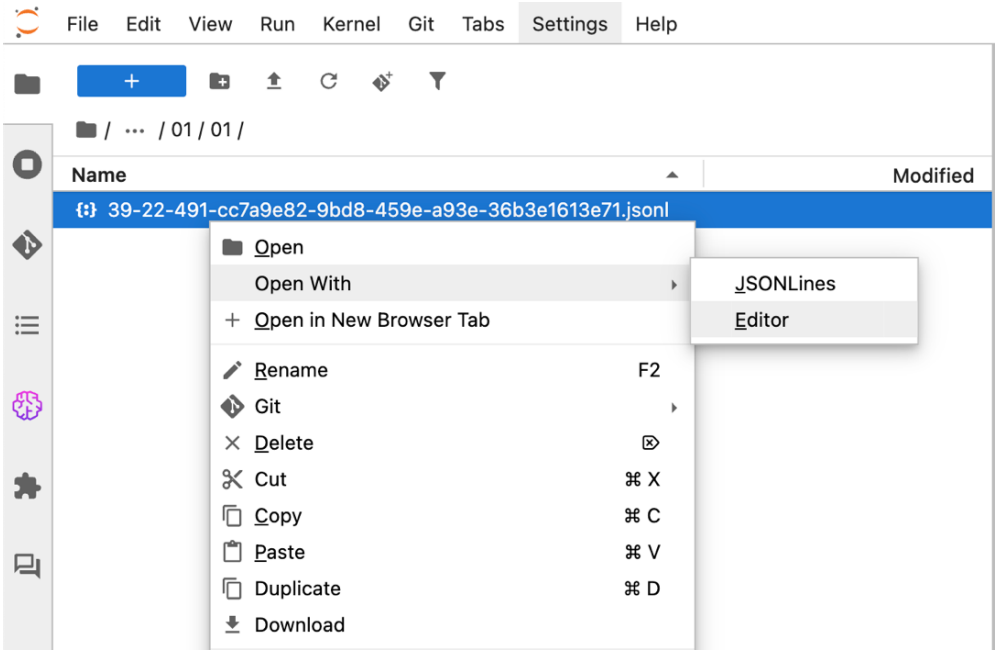



*Figure 8.5 — Inspecting the .jsonl file using the Editor*

You should see JSON Lines records, where each entry captures an input and its corresponding model output, for the model `shadow-testing-001` (**MODEL A**). In the first record, you should see the following input–output pair for the question "*What are the uses of Python?*":

```
{
  "captureData": {
```

```
    "endpointInput": { ... },
    "endpointOutput": {
      ...
      "data": {
        ...,
        "answer": "readability and ease of use"
      },
      ...
    }
  },
  ...
}
```

Here, the model `shadow-testing-001` (**MODEL A**) returned the answer "`readability and ease of use.`".

For the second record, you should see another input-output pair, this time for the question "*How large is the Moon?*", similar to:

```
{
  "captureData": {
    "endpointInput": { ... },
    "endpointOutput": {
      ...,
      "data": {
        ...,
        "answer": "one-quarter the diameter"
      },
      ...
    }
  },
  ...
}
```

Here, the same model `shadow-testing-001` (**MODEL A**) returns the answer "`one-quarter the diameter`" based on the provided context about the Moon.

4. Use the File Browser to open the `shadow-testing-endpoint-001/shadow-testing-variant-002` directory, navigate through the subdirectories, locate the *.jsonl* file within the nested folders, and open it in the editor to inspect the captured

requests and responses. In the first record, you should see the response of the model `shadow-testing-002` (**MODEL B**) for the question "*What are the uses of Python?*":

```
{
  "captureData": {
    "endpointInput": { ... },
    "endpointOutput": {
      ...,
      "data": {
        ...,
        "answer": "data science, web development, automation, and more"
      },
      ...
    }
  },
  ...
}
```

Here, the model `shadow-testing-002` (**MODEL B**) returned "`data science, web development, automation, and more`" which contrasts with `shadow-testing-001`'s output of "`readability and ease of use`".

For the second record, you should see the response of the model `shadow-testing-002` for the question"*How large is the Moon?*", similar to:

```
{
  "captureData": {
    "endpointInput": { ... },
    "endpointOutput": {
      ...,
      "data": {
        ...,
        "answer": "one-quarter the diameter of Earth"
      },
      ...
    }
  },
  ...
}
```

Here, the model `shadow-testing-002` (**MODEL B**) returns a slightly different answer, "`one-quarter the diameter of Earth`" compared to the output of model `shadow-testing-001`, which was "`one-quarter the diameter`".

Shadow testing allows you to safely evaluate new models without impacting live traffic. By capturing all requests and responses, you can compare the shadow model's outputs against the production model to identify differences, edge cases, or unexpected behavior.

## Promoting a shadow variant

After running your shadow testing setup for a while, you may decide to promote one variant to handle all traffic exclusively. This involves updating the inference endpoint so that only the selected model is active. In this section, we'll have the model `shadow-testing-002` (**MODEL B**) be the sole active variant.

Let's see how we can configure the endpoint to run a single production variant in the next set of steps:

1. Navigate back to your Jupyter notebook *deployment_strategies.ipynb*, so you can continue running the code blocks in the cells and proceed with the next steps.
2. Let's create a new endpoint configuration that references the model (**MODEL B**) you want to deploy as a single variant. This configuration includes the instance type (`ml.m5.xlarge`), initial instance count (`1`), variant weight, as well as the data capture configuration.

```
endpoint_config_name = "shadow-testing-endpoint-config-002"

single_variant = {
    "VariantName": "shadow-testing-variant-002",
    "ModelName": "shadow-testing-002",
    "InstanceType": "ml.m5.xlarge",
    "InitialInstanceCount": 1,
    "InitialVariantWeight": 1,
}

sagemaker_client.create_endpoint_config(
    EndpointConfigName=endpoint_config_name,
    ProductionVariants=[
        single_variant
    ],
   DataCaptureConfig=data_capture_config
)
```

Here, you have a new endpoint configuration for a single variant, that specifies the model, instance type, instance count, variant weight, and data capture setting.

> **Note**
>
> Note that this configuration is not yet active as the endpoint is still using the previous shadow testing configuration, so no traffic has been switched to this single variant yet.

3. Use the `update_endpoint()` method to apply the new endpoint configuration.

```
sagemaker_client.update_endpoint(
    EndpointName="shadow-testing-endpoint-001",
    EndpointConfigName=endpoint_config_name,
)
```

This updates the existing endpoint `shadow-testing-endpoint-001` to use the specified configuration, which triggers SageMaker AI to deploy the new configuration and transition the endpoint while keeping the same endpoint name.

4. Wait until the endpoint `shadow-testing-endpoint-001` is fully updated and in the `InService` state by calling the `wait_for_endpoint()` function.

```
%%time
wait_for_endpoint("shadow-testing-endpoint-001")
```

You should get the following output:

```
ENDPOINT STATUS: Updating
ENDPOINT STATUS: Updating
...
ENDPOINT STATUS: Updating
ENDPOINT STATUS: Updating
ENDPOINT STATUS: InService
CPU times: user 59.1 ms, sys: 485 µs, total: 59.6 ms
Wall time: 3min 2s
```

Here, you should see multiple `Updating` log messages while SageMaker AI applies the new configuration. After a few minutes, you should see the status change to `InService`, indicating that the update is complete.

> **Note**
>
> This process typically takes around 3 to 10 minutes to complete. Feel free to take a short break and grab a cup of coffee or tea while you wait.

5. Use `describe_endpoint()` to check that the new variant (using **MODEL B**) has been deployed and the endpoint is in service.

```
sagemaker_client.describe_endpoint(
    EndpointName="shadow-testing-endpoint-001"
)
```

You should get the following output:

```
{
  'EndpointName': 'shadow-testing-endpoint-001',
  'EndpointArn': '.../shadow-testing-endpoint-001',
  'EndpointConfigName': 'shadow-testing-endpoint-config-002',
  'ProductionVariants': [
    {'VariantName': 'shadow-testing-variant-002',...}
  ],
  'DataCaptureConfig': { ... },
  'EndpointStatus': 'InService',
   ...
}
```

Here, you can see that only the `shadow-testing-variant-002` variant (using **MODEL B**) is now active, confirming that the endpoint is running a single model.

Congratulations! You have successfully promoted the shadow model (**MODEL B**) to become the sole production variant. With this, your endpoint is now running a single model to handle all incoming requests.

> **Note**
>
> If you decide not to promote the shadow model, you can instead update the endpoint configuration to remove the shadow variant and retain only the original production model (**MODEL A**). This involves creating a new endpoint configuration with a single production variant referencing **MODEL A**, then applying it using `update_endpoint()`.

> Once the update completes, the endpoint will route all traffic solely to the original production model (**MODEL A**).

## Cleaning up

Before we wrap up this section, let's clean up the endpoints, endpoint configurations, and models you created:

1. Let's delete the real-time inference endpoint using `delete_endpoint()`:

```
sagemaker_client.delete_endpoint(
    EndpointName="shadow-testing-endpoint-001"
)
```

2. Delete the endpoint configurations `shadow-testing-endpoint-config-001` and `shadow-testing-endpoint-config-002` using `delete_endpoint_config()`.

```
endpoint_configurations = [
    "shadow-testing-endpoint-config-001",
    "shadow-testing-endpoint-config-002"
]

for config in endpoint_configurations:
    sagemaker_client.delete_endpoint_config(
        EndpointConfigName=config
    )
```

3. Finally, delete the models `shadow-testing-001` and `shadow-testing-002` using `delete_model()`.

```
models = [
    "shadow-testing-001",
    "shadow-testing-002"
]

for model in models:
    sagemaker_client.delete_model(
        ModelName=model
    )
```

As you've worked through the hands-on examples in this section, you've seen how SageMaker AI simplifies setting up shadow testing by allowing you to define both production and shadow variants within a single endpoint configuration. In addition, built-in data capture automatically records requests and responses, making it easier to analyze model performance and compare outputs. *Imagine having to set all of this up yourself without a managed service like SageMaker AI!* You would need to manage separate infrastructure for each model, implement request mirroring logic, build logging pipelines, and ensure everything scales reliably under production traffic. SageMaker AI abstracts these complexities, allowing you to save time and focus on evaluating and improving your models rather than building and maintaining the underlying systems.

## Using canary traffic shifting when performing Blue/Green deployments

When you deploy new versions of your ML models, you want to minimize downtime and keep the user experience seamless. **Blue/Green deployments** help by letting you switch traffic between two environments, with one running the current version (**BLUE**) and the other running the new version (**GREEN**). You can think of it like preparing a redesigned website behind the scenes. Users continue using the current version (**BLUE**) while the new one (**GREEN**) is fully prepared, then traffic is switched instantly with no visible downtime. If the new version does not perform as expected, you can quickly route traffic back to the old version (**BLUE**) environment to restore the previous stable experience. This strategy is especially useful for ML systems, where model performance can vary in production. It allows you to safely transition to newer model versions while maintaining a reliable fallback option at all times.

In this section, you'll see how canary traffic shifting takes this further by gradually sending a small percentage of live traffic from the old model (**BLUE**) to the new model (**GREEN**). Instead of directing all traffic at once, you get a controlled rollout that can be reversed if needed. SageMaker AI simplifies the process by letting you implement canary traffic shifting with just a few lines of code while it handles the underlying Blue/Green deployment setup automatically. This allows you to catch issues early and validate the new release before fully rolling it out.

**Note**

Canary traffic shifting is just one of the traffic shifting modes available in SageMaker AI. Depending on your deployment needs, you can also use linear or all-at-once traffic shifting modes to gradually or immediately route traffic to the new version. Each approach offers different trade-offs between speed of rollout and risk mitigation.

## Configuring a canary deployment in SageMaker AI

In this section, we'll configure a canary deployment in SageMaker AI with **MODEL A** as the current version (**BLUE**) and **MODEL B** as the new version (**GREEN**). We begin by setting up an inference endpoint running **MODEL A** and applying a deployment configuration that specifies how traffic will be shifted to **MODEL B**. We then go through the process of updating the endpoint configuration from **MODEL A** to **MODEL B** while gradually shifting requests to observe the canary deployment in action.

> **Note**
>
> As you follow along, we will primarily refer to **MODEL A** (**BLUE**) and **MODEL B** (**GREEN**) to guide you through the canary deployment workflow.

With this in mind, let's proceed with setting up the inference endpoint and simulating the canary deployment:

1. Inspect the configuration stored in `first_model` to ensure it references the correct inference image and pre-trained model.

   ```
   print(first_model)
   ```

   You should see output similar to the following:

   ```
   {
     'Image': '.../huggingface-pytorch-inference:1.9.1-transformers4.12.3-cpu-
   py38-ubuntu20.04',
     'ContainerHostname': 'firstModel',
     'Environment': {
       'HF_MODEL_ID': 'distilbert-base-uncased-distilled-squad',
       'HF_TASK': 'question-answering'
     }
   }
   ```

2. Register the model `traffic-shifting-001` (**MODEL A**) with `create_model()`, using the container image configuration that points to `huggingface-pytorch-inference`:

`1.9.1-transformers4.12.3-cpu-py38-ubuntu20.04` and uses the pretrained model `distilbert-base-uncased-distilled-squad`.

```
model_config = {
    "ModelName": "traffic-shifting-001",
    "ExecutionRoleArn": execution_role,
    "Containers": [first_model]
}
sagemaker_client.create_model(**model_config)
```

3. Create a new endpoint configuration with a single production variant that uses the `traffic-shifting-001` model (**MODEL A**), using the pretrained model `distilbert-base-uncased-distilled-squad`, running on two `ml.m5.xlarge` instances, and data capture enabled.

```
config_name = "traffic-shifting-endpoint-config-001"
only_variant = {
    "VariantName": "traffic-shifting-only-variant",
    "ModelName": "traffic-shifting-001",
    "InstanceType": "ml.m5.xlarge",
    "InitialInstanceCount": 2,
    "InitialVariantWeight": 1,
}

sagemaker_client.create_endpoint_config(
    EndpointConfigName=config_name,
    ProductionVariants=[only_variant],
    DataCaptureConfig=data_capture_config
)
```

4. Define a blue/green deployment configuration using a canary traffic routing strategy, where a portion of production traffic is initially routed to the new fleet, followed by a wait interval to monitor the deployment before completing the update, and store this configuration in `deployment_config`.

```
deployment_config = {
    "BlueGreenUpdatePolicy": {
        "TrafficRoutingConfiguration": {
            "Type": "CANARY",
            "CanarySize": {
```

```
                "Type": "CAPACITY_PERCENT",
                "Value": 40
            },
            "WaitIntervalInSeconds": 1200
        },
        "TerminationWaitInSeconds": 1200,
        "MaximumExecutionTimeoutInSeconds": 3600
    },
}
```

5. Using the configuration settings defined in the previous steps, create the endpoint using `create_endpoint()`:

```
sagemaker_client.create_endpoint(
    EndpointName="traffic-shifting-endpoint-001",
    EndpointConfigName=config_name,
    DeploymentConfig=deployment_config
)
```

6. Wait until the SageMaker AI real-time inference endpoint status changes to `InService`.

```
%%time
wait_for_endpoint("traffic-shifting-endpoint-001")
```

You should get the following output:

```
ENDPOINT STATUS: Creating
ENDPOINT STATUS: Creating
ENDPOINT STATUS: Creating
...
ENDPOINT STATUS: Creating
ENDPOINT STATUS: Creating
ENDPOINT STATUS: InService
CPU times: user 64.3 ms, sys: 750 µs, total: 65 ms
Wall time: 3min 2s
```

**Note**

This step may take approximately 3 to 10 minutes to complete.

7. Inspect the inference endpoint details using `describe_endpoint()`:

```
sagemaker_client.describe_endpoint(
    EndpointName="traffic-shifting-endpoint-001"
)
```

You should get the following output:

```
{
  'EndpointName': 'traffic-shifting-endpoint-001',
  'EndpointArn': '.../traffic-shifting-endpoint-001',
  'EndpointConfigName': 'traffic-shifting-endpoint-config-001',
  'ProductionVariants': [{
    'VariantName': 'traffic-shifting-only-variant',
    ...
  }],
  ...
}
```

**Note**

At this point, the endpoint is set up with only the `traffic-shifting-001` model (**MODEL A**), using the pretrained model `distilbert-base-uncased-distilled-squad`, as the active production variant handling all requests.

8. Inspect the payload stored in `input_payload_01` before using it to invoke the endpoint:

```
print(input_payload_01)
```

You should get an output matching the following structure:

```
{
  'inputs': {
    'context': 'Python is a widely-used programming language known for its
readability and ease of use. It supports multiple programming paradigms,
including procedural, object-oriented, and functional programming. Python
is popular in data science, web development, automation, and more.',
    'question': 'What are the uses of Python?'
```

```
    }
  }
```

9. Invoke the real-time inference endpoint using the payload stored in `input_payload_01`:

```
invoke_endpoint(
    input_payload=input_payload_01,
    endpoint_name="traffic-shifting-endpoint-001"
)
```

You should get the following output:

```
{'score': 0.5491470098495483,
 'start': 59,
 'end': 86,
 'answer': 'readability and ease of use'}
```

Here, the `traffic-shifting-001` model (**MODEL A**) performs question answering on the provided context and outputs "`readability and ease of use`" as the predicted answer.

10. Inspect the payload stored in `input_payload_02` before using it to invoke the endpoint:

```
print(input_payload_02)
```

You should get an output matching the following structure:

```
{
  'inputs': {
    'context': "The Moon is Earth's only natural satellite and the fifth-
largest satellite in the Solar System. It is about one-quarter the diameter
of Earth, making it the largest natural satellite relative to the size of
its planet.",
    'question': 'How large is the Moon?'
  }
}
```

11. Invoke the real-time inference endpoint using the payload stored in `input_payload_02`:

```
invoke_endpoint(
    input_payload=input_payload_02,
    endpoint_name="traffic-shifting-endpoint-001"
)
```

You should get the following output:

```
{'score': 0.5532084107398987,
 'start': 108,
 'end': 132,
 'answer': 'one-quarter the diameter'}
```

Similarly, the only deployed model (**MODEL A**) processes the input and returns the most relevant answer, "`one-quarter the diameter`", along with its confidence score and character positions.

12. Inspect the configuration stored in `second_model` to ensure it references the correct inference image and pretrained model.

```
print(second_model)
```

You should get an output similar to the following:

```
{
  'Image': .../huggingface-pytorch-inference:1.10.2-transformers4.17.0-cpu-
py38-ubuntu20.04',
  'ContainerHostname': 'secondModel',
  'Environment': {
    'HF_MODEL_ID': 'deepset/roberta-base-squad2',
    'HF_TASK': 'question-answering'
  }
}
```

13. Register the model `traffic-shifting-002` (**MODEL B**) with `create_model()`, using the container image configuration that points to `huggingface-pytorch-inference`:

`1.10.2-transformers4.17.0-cpu-py38-ubuntu20.04` and uses the pretrained model `roberta-base-squad2`.

```
model_config = {
    "ModelName": "traffic-shifting-002",
    "ExecutionRoleArn": execution_role,
    "Containers": [second_model]
}
sagemaker_client.create_model(**model_config)
```

14. Create a new endpoint configuration for the second model `traffic-shifting-002` (**MODEL B**):

```
config_name = "traffic-shifting-endpoint-config-002"

new_variant = {
    "VariantName": "traffic-shifting-only-variant",
    "ModelName": "traffic-shifting-002",
    "InstanceType": "ml.m5.xlarge",
    "InitialInstanceCount": 2,
    "InitialVariantWeight": 1,
}

sagemaker_client.create_endpoint_config(
    EndpointConfigName=config_name,
    ProductionVariants=[new_variant],
    DataCaptureConfig=data_capture_config
)
```

In this new endpoint configuration, a single production variant is defined, with the `traffic-shifting-002` model (**MODEL B**) set as the active variant.

**Note**

Although this configuration defines a single production variant, the deployment uses a canary strategy, so traffic will be gradually shifted to the new model (**MODEL B**) during the update process rather than all at once.

15. Update the endpoint with the new configuration using `update_endpoint()`:

```
sagemaker_client.update_endpoint(
    EndpointName="traffic-shifting-endpoint-001",
    EndpointConfigName=config_name,
    RetainDeploymentConfig=True
)
```

16. Define the `wait_and_describe_endpoint()` function, which repeatedly checks the status of a SageMaker AI inference endpoint and pretty-prints its properties and configuration until the endpoint reaches the `InService` state.

```
from pprint import pprint

def wait_and_describe_endpoint(endpoint_name):
    while True:
        response = sagemaker_client.describe_endpoint(
            EndpointName=endpoint_name
        )

        status = response['EndpointStatus']

        print("-"*100)
        print(f"ENDPOINT STATUS: {status}")
        print("-"*100)
        pprint(response, indent=2)
        print("-"*100)

        if status == 'InService':
            break
        elif status == 'Failed':
            error = "ENDPOINT CREATION FAILED"
            raise RuntimeError(error)

        sleep(120)
```

17. Use the `wait_and_describe_endpoint()` function defined in the previous step to check the status of your endpoint while the canary traffic shifting deployment is in progress.

```
%%time
endpoint_name = "traffic-shifting-endpoint-001"
wait_and_describe_endpoint(endpoint_name)
```

You should get a similar set of logs:

```
...
---------------------------------------------
ENDPOINT STATUS: Updating
---------------------------------------------
{ ...,
  'LastDeploymentConfig': {
    'BlueGreenUpdatePolicy': {
      ...
      'TrafficRoutingConfiguration': {
        'CanarySize': {
          'Type': 'CAPACITY_PERCENT',
          'Value': 40
        },
        'Type': 'CANARY',
        'WaitIntervalInSeconds': 1200
      }
    }
  },
  ...
  'PendingDeploymentSummary': {
    'EndpointConfigName': '...-endpoint-config-002',
    'ProductionVariants': [ {
      ...,
      'VariantStatus': [ {
        'StartTime': ...,
        'Status': 'Baking',
        'StatusMessage': ...
      }],
     ...
    } ]
  },
  'ProductionVariants': [...],
```

```
    ...
  }
  ...
```

Here, the `Baking` status in the `PendingDeploymentSummary` dictionary indicates that SageMaker AI is monitoring the new variant (**GREEN | MODEL B**) for a predefined period before proceeding with the full deployment. If any configured alarms are triggered during this phase, traffic automatically rolls back to the existing variant (**BLUE | MODEL A**). This period allows you to validate the new variant's stability and performance before all production traffic is routed to it.

> **Note**
>
> Once the endpoint reaches the `InService` state, all production traffic is being served exclusively by the new variant, with the model `traffic-shifting-002` (**GREEN | MODEL B**).

18. Inspect the payload stored in `input_payload_01` before using it to invoke the endpoint:

```
print(input_payload_01)
```

You should get an output matching the following structure:

```
{
  'inputs': {
    'context': 'Python is a widely-used programming language known for its
readability and ease of use. It supports multiple programming paradigms,
including procedural, object-oriented, and functional programming. Python
is popular in data science, web development, automation, and more.',
    'question': 'What are the uses of Python?'
  }
}
```

19. Invoke the real-time inference endpoint using the payload stored in `input_payload_01`:

```
invoke_endpoint(
    input_payload=input_payload_01,
    endpoint_name="traffic-shifting-endpoint-001"
)
```

You should get the following output:

```
{'score': 0.30364930629730225,
 'start': 220,
 'end': 271,
 'answer': 'data science, web development, automation, and more'}
```

Here, the model `traffic-shifting-002` (**GREEN | MODEL B**), using `roberta-base-squad2`, returns a different answer, "`data science, web development, automation, and more`", compared to the other model `traffic-shifting-001` (**BLUE | MODEL A**), using `distilbert-base-uncased-distilled-squad`, which was "`readability and ease of use`".

20. Inspect the payload stored in `input_payload_02` before using it to invoke the endpoint:

```
print(input_payload_02)
```

You should get an output matching the following structure:

```
{
  'inputs': {
    'context': "The Moon is Earth's only natural satellite and the fifth-
largest satellite in the Solar System. It is about one-quarter the diameter
of Earth, making it the largest natural satellite relative to the size of
its planet.",
    'question': 'How large is the Moon?'
  }
}
```

21. Invoke the real-time inference endpoint using the payload stored in `input_payload_02`:

```
invoke_endpoint(
    input_payload=input_payload_02,
    endpoint_name="traffic-shifting-endpoint-001"
)
```

You should get the following output:

```
{'score': 0.14790363609790802,
 'start': 108,
 'end': 141,
 'answer': 'one-quarter the diameter of Earth'}
```

Here, the model `traffic-shifting-002` (**GREEN | MODEL B**) returns a different answer, "`one-quarter the diameter of Earth`", compared to the other model `traffic-shifting-001` (**BLUE | MODEL A**) which was "`one-quarter the diameter `".

At this stage, you should have a good understanding of how canary deployments work and how they're configured in SageMaker AI. Before we wrap up this section, it's important to clean up the resources you created to avoid unnecessary charges.

## Cleaning up

It's time to delete the SageMaker AI real-time inference endpoint along with the associated endpoint configurations and the deployed models used throughout the traffic shifting setup.

Let's proceed with the clean up steps:

1. Use `delete_endpoint()` to delete the SageMaker endpoint `traffic-shifting-endpoint-001`.

```
sagemaker_client.delete_endpoint(
    EndpointName="traffic-shifting-endpoint-001"
)
```

2. Use `delete_endpoint_config()` to delete the endpoint configurations `traffic-shifting-endpoint-config-001` and `traffic-shifting-endpoint-config-002`.

```
endpoint_configurations = [
    "traffic-shifting-endpoint-config-001",
    "traffic-shifting-endpoint-config-002",
]

for config in endpoint_configurations:
    sagemaker_client.delete_endpoint_config(
```

```
        EndpointConfigName=config
    )
```

3. Finally, use `delete_model()` to delete the models `traffic-shifting-001` (**MODEL A**) and `traffic-shifting-002` (**MODEL B**):

```
models = [
    "traffic-shifting-001",
    "traffic-shifting-002"
]

for model in models:
    sagemaker_client.delete_model(
        ModelName=model
    )
```

**Note**

Feel free to stop the SageMaker Studio space (`mle-on-aws-space`) as well when you are finished using it.

At this point, you should have a solid understanding of how canary traffic shifting works and why it is a valuable technique for reducing deployment risk. SageMaker AI makes it easy to implement this traffic shifting mode, so you can focus on evaluating and improving your models rather than managing the underlying infrastructure. As an ML engineer, it is important to be familiar not only with canary deployments but also with other traffic shifting modes in SageMaker AI. All-at-once traffic shifting is best for low-risk updates or smaller user bases. Linear traffic shifting gradually ramps up traffic to the new version, while canary traffic shifting lets you test carefully with a small portion of live traffic before fully rolling out the update. Choosing the right approach helps you minimize risk, maintain a smooth user experience, and make the most of SageMaker AI's deployment capabilities.

## Summary

In this chapter, we explored different options and strategies for deploying models in SageMaker AI. You worked with a pretrained model and deployed it using a real-time endpoint, a serverless endpoint, an asynchronous inference endpoint, and a batch transform job. You also practiced advanced techniques like shadow testing to validate a new model without affecting live traffic, and canary traffic shifting to gradually route a portion of production requests to a new model. Along the way, you learned how to configure data capture to record inference requests and responses for monitoring and evaluation.

In the next chapter, you'll dive into building end-to-end ML pipelines and applying MLOps principles to automate workflows, track model performance, and keep your ML systems running smoothly. You'll also explore other relevant ML Engineering topics, including integrating security best practices and managing costs, giving you a well-rounded view of designing, deploying, and managing production-ready machine learning systems.

## Further reading

To deepen your understanding of the concepts discussed in this chapter, explore the following resources:

- *Amazon SageMaker AI | Developer Guide | Understand the options for deploying models and getting inferences in Amazon SageMaker AI* (`https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model-get-started.html`)
- *Amazon SageMaker AI | Developer Guide | Shadow tests* (`https://docs.aws.amazon.com/sagemaker/latest/dg/shadow-tests.html`)
- *Amazon SageMaker AI | Developer Guide | Deployment guardrails for updating models in production* (`https://docs.aws.amazon.com/sagemaker/latest/dg/deployment-guardrails.html`)
- *Amazon SageMaker AI | Developer Guide | Best practices* (`https://docs.aws.amazon.com/sagemaker/latest/dg/best-practices.html`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 9

# Automating LLMOps Workflows with SageMaker Pipelines

An automated pipeline is simply a sequence of connected steps that runs tasks in a defined order with minimal manual intervention. When you execute a pipeline, each step runs according to the dependencies and workflow logic defined within it. Before teams implement automated pipelines, workflows are often run manually using individual steps such as training, evaluation, and deployment. This works well for early experimentation, but it becomes harder to manage as the process grows and is repeated over time. You need to track dependencies between steps, manage inputs and outputs, and ensure consistent execution. Over time, this manual approach becomes less reliable and more difficult to maintain.

In this chapter, you will use **SageMaker Pipelines** to incrementally address these challenges by applying core pipeline design principles. You will start with simple, single-step pipelines focused on fine-tuning and evaluation. From there, you will evolve them into multi-step pipelines that connect individual stages into a structured, automated workflow. You will also add conditional steps that control execution flow based on pipeline outputs. As you go through the hands-on examples, you will explore how simple workflows are gradually evolved into fully automated pipelines, similar to how production-grade **machine learning** (**ML**) systems are built in practice. The chapter ends with key best practices and design considerations for building scalable, production-ready ML pipelines.

This chapter covers the following topics:

- Setting up the project environment and dependencies
- Building and Running the Single-Step Fine-Tuning Pipeline
- Building and Running the Single-Step Evaluation Pipeline
- Configuring and Running a Two-Step Fine-Tuning and Evaluation Pipeline

- Preparing the Lambda functions for deploying a model to an endpoint
- Completing the LLMOps pipeline
- Best Practices and Key Considerations for Building Automated ML Workflows

By the end of this chapter, you will understand how to design, build, and evolve production-ready ML pipelines using SageMaker Pipelines. You will learn how manual workflows can be gradually converted into structured, automated pipelines that improve manageability, scalability, and maintainability in real-world systems.

# Technical requirements

Before proceeding with the hands-on examples in this chapter, confirm that the following prerequisites and setup requirements have been addressed:

- **Sufficient account-level quota for selected ML instance types**: Ensure that your AWS account has the required applied account-level quota values for the SageMaker AI ML instance types used in this chapter. You should have at least 2× `ml.g5.4xlarge` for the real-time inference endpoint (**ml.g5.4xlarge for endpoint usage**), and at least 2× `ml.m5.xlarge` for processing job usage and pipeline execution workloads (**ml.m5.xlarge for processing job usage**). You can review and adjust these limits in the Service Quotas console in the AWS Management Console.
- **An existing SageMaker Studio space**: You can use the SageMaker Studio space (`mle-on-aws-space`) that you set up in *Chapter 1* to follow along with the examples in this book. You may upgrade the space to an `ml.m5.xlarge` instance type and allocate a minimum of 100 GB of storage to properly support the compute and data requirements of the hands-on exercises.
- **A code editor installed on your local machine** (such as Visual Studio Code or Sublime Text): You'll need this when working with the code and configuration files used throughout the hands-on exercises and examples in this book.
- **Updated space execution role trust relationship configuration**: Ensure that the execution role attached to your SageMaker Studio space allows both SageMaker AI and Amazon Bedrock to assume the role. This is required because the workflows and evaluation jobs used throughout this chapter rely on SageMaker AI processing jobs and Bedrock-powered LLM-as-a-Judge evaluation capabilities. The trust policy for the execution role should include the following configuration:

```
{
    "Version": "2012-10-17",
    "Statement": [
```

```
        {
            "Effect": "Allow",
            "Principal": {
                "Service": [
                    "sagemaker.amazonaws.com",
                    "bedrock.amazonaws.com"
                ]
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

If the execution role does not include these trusted service principals, processing jobs, pipeline executions, or evaluation tasks may fail when attempting to access SageMaker AI or Amazon Bedrock resources. You can review and update the trust relationship configuration from the **Identity and Access Management (IAM)** console in the AWS Management Console.

- **Updated execution role permissions for SageMaker Studio space**: Ensure that the execution role attached to your SageMaker Studio space includes the required service permissions for running end-to-end LLMOps workflows. In addition to the correct trust relationship for SageMaker AI and Amazon Bedrock, the role must also include the following managed policies: `AmazonBedrockFullAccess`, `AmazonS3FullAccess`, `AmazonSageMakerFullAccess`, `AWSLambda_FullAccess`, and `CloudWatchFullAccessV2`. You can review and update the IAM permissions attached to the execution role from the IAM console in the AWS Management Console.

> **Note**
>
> While working on the examples of this chapter, you may optionally use `AdministratorAccess` to reduce permission-related errors and simplify setup. Keep in mind that the IAM roles configuration used across the chapter is overly permissive and should not be used in production environments without further refinement and restriction of permissions.

The Jupyter notebooks, source code, and other files used for each chapter are available in this book's GitHub repository: `https://github.com/PacktPublishing/Machine-Learning-Engineering-on-AWS-Second-Edition/tree/main/chapter09`.

# Setting up the project environment and dependencies

ML workflows are generally implemented as automated pipelines made up of sequential stages that execute in a defined order. This enables you to reliably orchestrate training, evaluation, and deployment tasks without manual intervention, reducing operational overhead and allowing ML engineers and data scientists to focus on higher-value work such as model refinement and experimentation.

In this chapter, you will build an automated pipeline that starts with fine-tuning and evaluation stages and then expands into a deployment workflow that includes a conditional step that controls how deployment is handled based on the pipeline's outputs (*Figure 9.1*).

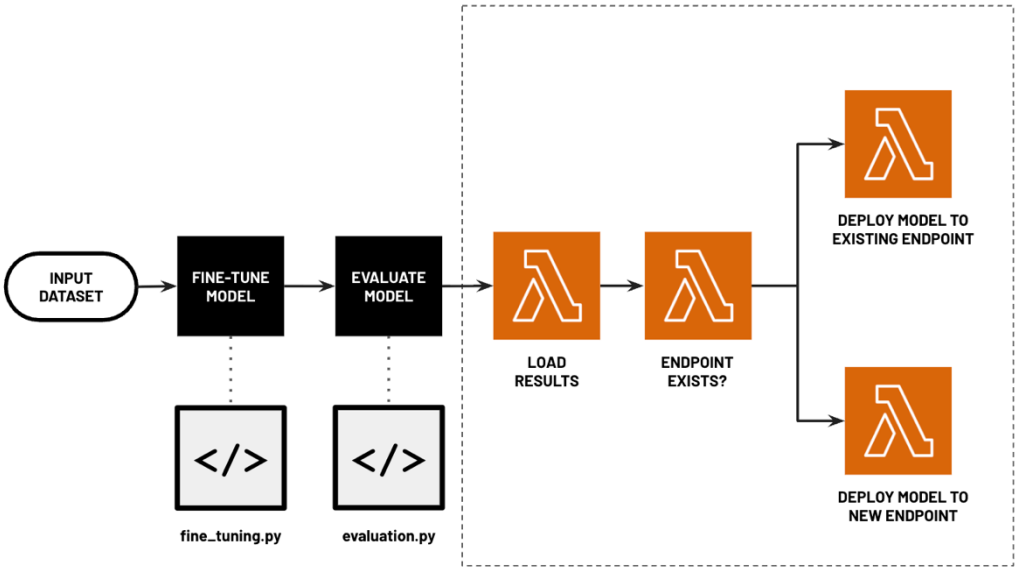



*Figure 9.1 — The complete end-to-end LLMOps pipeline you'll build in this chapter*

You will use Lambda-powered steps to retrieve pipeline outputs and evaluate the current endpoint state. The workflow then routes execution to either create a new SageMaker AI endpoint or update an existing one accordingly.

Before you begin building this pipeline, you will need to ensure that several prerequisites are in place, including the required datasets, supporting scripts, and configuration files. In this section, you will download the following required project files:

- **fine_tuning.py** — This script implements the full supervised fine-tuning workflow, including training configuration, model initialization, and artifact generation. You will use this script to define and execute the model training process.
- **evaluation.py** — This script runs the LLM-as-a-judge evaluation workflow, including model loading, inference execution, evaluation metric computation, and result aggregation. You will use this script to evaluate model outputs using a structured scoring process.
- **data.jsonl** — This dataset contains prompt-completion pairs used for supervised fine-tuning. You will use this data to train the model on labeled examples.
- **custom_metric.json** — This configuration file defines the evaluation rubric and scoring criteria used by the LLM-as-a-judge evaluator. You will use this file to control how model outputs are scored during evaluation.
- **gen_qa.jsonl** — This dataset is used to evaluate model responses on generative question-answering tasks. You will use it to benchmark model performance on unseen prompts.
- **pipeline_wrapper.py** — This helper utility simplifies inspection and debugging of SageMaker pipeline executions, including access to step inputs, outputs, and logs. You will use it to explore and debug pipeline runs more effectively.

In the sections that follow, you will use these prerequisites throughout multiple stages of the pipeline to implement and execute the full end-to-end workflow. These files and scripts will serve as the foundation for different components of the system, including model fine-tuning, evaluation, and deployment, ensuring that each stage of the pipeline is properly configured and connected as you progress through the implementation.

**Note**

This section assumes that you are running the hands-on examples in a SageMaker Studio space. As mentioned in the *Technical requirements* section, you may upgrade the SageMaker Studio space to an `ml.m5.xlarge` instance type and allocate a minimum of 100 GB of storage before proceeding. Remember to stop the space when it is not in use to avoid unnecessary charges.

With this in mind, let's proceed with setting up the project environment:

1. In the left sidebar of JupyterLab, right-click and select **New Folder**. Name the folder `pipeline`, then double-click it to navigate into the directory. Alternatively, you can create the folder by clicking the **New Folder** button in the File Browser toolbar, then rename it to `pipeline` and open it to navigate into the directory.

   **Note**

   All notebooks, directories, and associated files for this chapter should be created and stored in this directory.

2. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `setup.ipynb`. We will run the code in the following steps within this notebook.

3. Uninstall any existing versions of `sagemaker` and `sagemaker-serve`, then install the pinned versions required for this chapter:

   ```
   %pip uninstall -y sagemaker sagemaker-serve
   %pip install "sagemaker==3.5.0"
   %pip install "sagemaker-serve==1.2.0"
   %pip install "rich==14.2.0"
   ```

   **Note**

   To better manage long outputs in Jupyter notebooks, right-click the output area and choose **Enable Scrolling for Outputs**. This keeps results contained in a scrollable box instead of expanding the entire cell.

4. Now that we've installed the required packages and package versions, restart the kernel by selecting **Kernel | Restart Kernel** from the menu. If prompted to restart the kernel, click **Restart**. Then, wait for approximately 2 to 3 minutes for the kernel to fully restart.

   **Note**

   After installing the required package versions, confirm that the correct versions are in place by running `%pip show sagemaker` and `%pip show`

> `sagemaker-serve`. Ensure that the installed versions are `sagemaker 3.5.0` and `sagemaker-serve 1.2.0`, respectively, before proceeding.

5. Run the following code to create the directory (`code`) for storing code files used in the pipeline:

```
CODE_DIRECTORY = "code"
!mkdir -p {CODE_DIRECTORY}
```

6. Next, create the directory (`data`) for storing data files used in the pipeline:

```
DATA_DIRECTORY = "data"
!mkdir -p {DATA_DIRECTORY}
```

7. Next, define the `download_file()` function for downloading a file from the book's official GitHub repository and saving it to a local directory inside your SageMaker Studio space.

```
import urllib.request
from pathlib import Path
from typing import Union

BASE_URL_PARTS = [
    "https://raw.githubusercontent.com/",
    "PacktPublishing/",
    "Machine-Learning-Engineering-on-",
    "AWS-Second-Edition/",
    "refs/heads/main/",
    "chapter09/",
]

def download_file(
        filename: str,
        directory: Union[str, Path] = CODE_DIRECTORY
    ) -> Path:
    url = "".join(BASE_URL_PARTS) + filename

    directory_path = Path(directory)
    directory_path.mkdir(parents=True, exist_ok=True)
```

```
    output_file = directory_path / filename
    urllib.request.urlretrieve(url, output_file)

    return output_file
```

This function builds the full file URL by combining the base GitHub path with the provided filename. It then creates the target local directory if it does not already exist, downloads the file from the repository, and saves it to that directory. Finally, it returns the local path of the downloaded file so it can be used in later steps.

8. Using the `download_file()` function we defined in the previous step, download the `fine_tuning.py` and `evaluation.py` script files.

```
download_file("fine_tuning.py")
download_file("evaluation.py")
```

> **Note**
>
> You can use the File Browser to verify that the script files have been successfully downloaded and are present inside the *code* directory.

9. Inspect the fine-tuning script to review its structure and confirm the training workflow implementation.

```
!cat code/fine_tuning.py
```

This should print the full script inside *code/fine_tuning.py*, similar to the following:

```
...

@contextmanager
def block(label):
    print(f"[{label}]: START")
    yield
    print(f"[{label}]: END")

...

def perform_fine_tuning(
    dataset,
```

```
        output_path,
        mlflow_experiment_name,
        mlflow_run_name,
        region,
        role_arn
    ):

        ...

        model = "..."

        with block("MODEL_PACKAGE_GROUP"):
            ...

        with block("TRAINING"):
            trainer = SFTTrainer(
                ...
            )
            ...

        ...
```

This script implements an end-to-end supervised fine-tuning workflow for a **large language model** (**LLM**). It installs required dependencies, configures the AWS region, and orchestrates a multi-stage execution that includes model training with `SFTTrainer`, model building with `ModelBuilder`, artifact processing, and model registration. Each stage is wrapped in structured logging blocks to improve traceability, making it easier to monitor progress and debug execution issues. At the end of the run, it generates a *results.json* file that captures key metadata such as the training job, model artifacts, S3 locations, and MLflow tracking information for later use in evaluation and deployment workflows.

**Note**

At the time of writing, `SFTTrainer` is not yet supported natively within SageMaker Pipelines in the SageMaker Python SDK. As a result, it is executed within a processing job rather than as a dedicated pipeline step type.

10. Inspect the fine-tuning script to review its structure and confirm the training workflow implementation.

```
!cat code/evaluation.py
```

This should print the full script inside *code/evaluation.py*, similar to the following:

```
...


def load_json(path):
    ...


def run_evaluation(
    eval_dataset,
    custom_metric_path,
    output_path,
    evaluator_model,
    region
):

    ...

    with block("MODEL_PACKAGE_LOOKUP"):
        ...

    with block("EVALUATOR_INIT"):
        ...

        evaluator = LLMAsJudgeEvaluator(
            ...
        )

    ...

    with block("EXECUTION"):
        ...

    ...
```

11. Using the `download_file()` function, download the input dataset (`data.jsonl`) to your local directory (*data*).

```
download_file(
    "data.jsonl",
    directory=DATA_DIRECTORY
)
```

> **Note**
>
> You can use the File Browser to verify that the CSV files have been successfully downloaded and are present inside the *data* directory.

12. After downloading the dataset, inspect the first few records to understand its structure and content.

```
!head data/data.jsonl
```

Each entry contains a prompt and a corresponding completion, which will be used during training.

```
{
  "prompt": "User: Recommend a movie to ...",
  "completion": "I would recommend the movie, ..."
}
{
  "prompt": "User: Determine the result obtained ...",
  "completion": "5338245 - 50629795848152 = ..."
}
{
  "prompt": "User: Considering that a film ...",
  "completion": "Step 1: Convert the duration ..."
}
...
```

13. Using the `download_file()` function, download the `custom_metric.json` file to your local directory.

```
download_file(
    "custom_metric.json",
```

```
    directory="."
)
```

14. Inspect the contents of the custom metric definition file to verify the evaluation configuration used by the LLM-as-a-Judge process.

```
!cat custom_metric.json
```

This should give you the following JSON output:

```
{
    "customMetricDefinition": {
        "name": "PositiveSentiment",
        "instructions": "...",
        "ratingScale": [
            {
                "definition": "Good",
                "value": {
                    "floatValue": 1
                }
            },
            {
                "definition": "Poor",
                "value": {
                    "floatValue": 0
                }
            }
        ]
    }
}
```

15. Using the `download_file()` function, download the `gen_qa.jsonl` file to your local directory.

```
download_file(
    "gen_qa.jsonl",
    directory=DATA_DIRECTORY
)
```

16. Using the `download_file()` function, download the `pipeline_wrapper.py` file to your local directory so it can be used for inspecting execution results in the next section.

```
download_file(
    "pipeline_wrapper.py",
    directory="."
)
```

17. Inspect the contents of the downloaded `pipeline_wrapper.py` file.

```
!cat pipeline_wrapper.py
```

This should print the full script inside *pipeline_wrapper.py*, similar to the following:

```
import boto3

class PipelineExecution:
    def __init__(self, execution):
        self.execution = execution
        self.sm = boto3.client("sagemaker")
        self.logs_client = boto3.client("logs")

        raw_steps = execution.list_steps()

        ...

    def number_of_steps(self):
        return len(self.steps_metadata)

    ...
```

Here, you have a wrapper class (`PipelineExecution`) that simplifies inspection and debugging of SageMaker AI pipeline executions.

**Note**

This `PipelineExecution` class is a custom wrapper defined in *pipeline_wrapper.py* and should not be confused with the similarly named class in the SageMaker Python SDK.

Before ending this section, let's take a moment to review how the `PipelineExecution` class simplifies execution monitoring and debugging. The `PipelineExecution` class (from *pipeline_wrapper.py*) provides a high-level interface to interact with the `PipelineExecution` object from the SageMaker Python SDK, including convenience methods such as `number_of_steps()` to return the total number of executed steps, `step()` to retrieve a specific step in the workflow, and `outputs()` to return the outputs of the final step.

Here is sample code showing how to create a `PipelineExecution` object and use it to inspect execution outputs and logs.

```
from pipeline_wrapper import PipelineExecution
pipeline_execution = PipelineExecution(
    execution=execution
)

# Inspect the final step output(s)
pipeline_execution.outputs()

# Retrieve the CloudWatch logs for the first step
pipeline_execution.step(1).logs()
```

The `PipelineStep` object returned by `step()` corresponds to an individual step in the execution and provides methods to inspect its inputs, outputs, and logs. The `logs()` method retrieves the corresponding CloudWatch logs for monitoring execution progress and debugging issues. The `inputs()` method retrieves input data sources, while the `outputs()` method returns generated artifacts such as model artifacts or processed data. Behind the scenes, these methods are lightweight wrappers over existing functionality in the SageMaker Python SDK and AWS services. For example, execution metadata is retrieved using SageMaker AI API calls, while inputs and outputs are derived from job metadata returned by `describe_processing_job()` and `describe_training_job()`.

**Note**

While you could achieve the same result using the raw SageMaker Python SDK and AWS API calls, the wrapper class from *pipeline_wrapper.py* abstracts these details and makes debugging more straightforward.

# Building and running the Single-Step Fine-Tuning pipeline

While you may be tempted to build the entire pipeline all in one go, it is often more practical to validate individual components first. This approach helps ensure that each stage behaves as expected before being integrated into a full workflow. In this section, you will build and execute a single-step SageMaker pipeline that performs supervised fine-tuning of a language model, similar to what is shown in *Figure 9.2*.

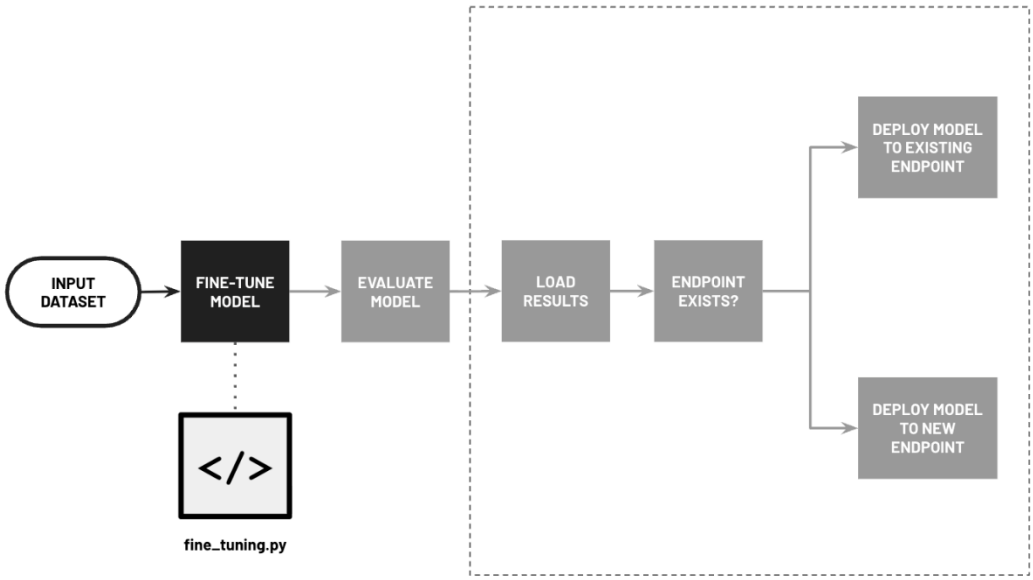



*Figure 9.2 — Building the single-step fine-tuning pipeline*

Since `SFTTrainer` is not yet natively supported in SageMaker Pipelines, the training logic is executed within a processing job using the previously downloaded *fine_tuning.py* as the core script for this workflow. As you build the pipeline, you will define components and configurations that will be reused across multiple pipelines of increasing complexity throughout this chapter.

## Building and running the pipeline

You will now construct the fine-tuning pipeline by defining parameters, configuring a processing job, and assembling the pipeline using SageMaker Pipelines primitives. You will

then deploy the pipeline definition, start an execution, and observe how the single-step workflow runs end-to-end.

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `fine_tuning_step.ipynb`. We will run the code in the following steps within this notebook.

   **Note**

   Make sure that *fine_tuning_step.ipynb* is created directly inside the pipeline directory and not inside the code directory. At this stage, the pipeline directory should contain both *setup.ipynb* and *fine_tuning_step.ipynb*.

2. Import the required Python libraries and SageMaker modules needed to build and run the pipeline, including workflow parameters, session utilities, processing components, and pipeline constructs.

```
import random
import string

from sagemaker.core.workflow.parameters import (
    ParameterString
)
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)
from sagemaker.core.processing import ScriptProcessor
from sagemaker.core.shapes import (
    ProcessingOutput,
    ProcessingS3Output
)
from sagemaker.mlops.workflow.pipeline import (
    Pipeline
)
from sagemaker.mlops.workflow.steps import (
    ProcessingStep
)
```

```
from sagemaker.core.workflow.pipeline_context import (
    PipelineSession
)
from sagemaker.core import image_uris
```

3. Define the `generate_string()` function for generating a random string of lowercase letters with a specified length.

```
def generate_string(length=6):
    return ''.join(
        random.choices(
            string.ascii_lowercase,
            k=length
        )
    )
```

4. Initialize the pipeline prerequisites, including the `Session` and `PipelineSession` objects, execution role, AWS Region, and dynamically generated S3 paths for the dataset and output artifacts.

```
unique = generate_string()
sagemaker_session = Session()
pipeline_session = PipelineSession()
role = get_execution_role()
region = sagemaker_session.boto_region_name

prefix = f"pipeline-{unique}"
s3 = sagemaker_session.default_bucket()
dataset_path = f"s3://{s3}/{prefix}/data/data.jsonl"
output_path = f"s3://{s3}/{prefix}/output/"
```

5. Use the `aws s3 cp` command to upload the prepared dataset to Amazon S3 so it can be used by the pipeline during execution in later steps.

```
!aws s3 cp data/data.jsonl {dataset_path}
```

6. Next, let's define pipeline parameters to make the workflow reusable and configurable.

```
role_arn_param = ParameterString(
    name="TrainingRoleArn",
```

```
    default_value=role
)

dataset_param = ParameterString(
    name="Dataset",
    default_value=dataset_path
)

output_path_param = ParameterString(
    name="OutputPath",
    default_value=output_path
)

mlflow_exp_param = ParameterString(
    name="MLflowExperimentName",
    default_value=f"sft-experiment-{unique}"
)

mlflow_run_param = ParameterString(
    name="MLflowRunName",
    default_value=f"run-{unique}"
)

region_param = ParameterString(
    name="Region",
    default_value=region
)
```

Here, we define pipeline parameters for the IAM role, dataset location, output path, MLflow experiment name, MLflow run name, and AWS Region.

7. Initialize a `ScriptProcessor` object that configures the compute environment, container image, and execution settings for the processing job that runs the SFT process.

```
processor = ScriptProcessor(
    image_uri=image_uris.retrieve(
        framework="sklearn",
        region=region,
        version="1.2-1",
        py_version="py3",
```

```
        instance_type="ml.m5.xlarge",
    ),
    instance_type="ml.m5.xlarge",
    instance_count=1,
    base_job_name=f"sfttrainer-job-{unique}",
    sagemaker_session=pipeline_session,
    role=role,
)
```

8. Define the processing job output location as well as the local container output path:

```
processing_output_path = f"s3://{s3}/{prefix}/output"
local_path = "/opt/ml/processing/output"
```

9. Configure the processing step by specifying output handling, providing runtime arguments to the fine-tuning script, and defining the corresponding `ProcessingStep` for execution.

```
step_args = processor.run(
    inputs=[],
    outputs=[
        ProcessingOutput(
            output_name="output",
            s3_output=ProcessingS3Output(
                s3_uri=processing_output_path,
                local_path=local_path,
                s3_upload_mode="EndOfJob"
            )
        ),
    ],
    code="code/fine_tuning.py",
    arguments=[
        "--dataset", dataset_param,
        "--output-path", output_path_param,
        "--mlflow-experiment-name", mlflow_exp_param,
        "--mlflow-run-name", mlflow_run_param,
        "--region", region_param,
        "--role-arn", role_arn_param,
    ],
)
```

```
step_process = ProcessingStep(
    name=f"ProcessingSFTTrainer-{unique}",
    step_args=step_args,
)
```

10. Construct the pipeline by specifying the execution steps (in this case a single step), input parameters, and session configuration required to run the workflow.

```
pipeline = Pipeline(
    name=f"FineTuningPipeline-{unique}",
    steps=[step_process],
    parameters=[
        dataset_param,
        output_path_param,
        mlflow_exp_param,
        mlflow_run_param,
        region_param,
        role_arn_param,
    ],
    sagemaker_session=pipeline_session
)
```

11. Create or update the pipeline definition and associate it with the specified IAM role.

```
pipeline.upsert(role_arn=role)
```

12. With everything ready, start the pipeline execution.

```
mlflow_exp_name = f"sft-exp-run-{unique}"

execution = pipeline.start(
    parameters={
        "Dataset": dataset_path,
        "OutputPath": output_path,
        "MLflowExperimentName": mlflow_exp_name,
        "MLflowRunName": f"run-{unique}",
        "Region": region,
        "TrainingRoleArn": role,
    }
)
```

As there is only a single step in this pipeline, it executes end to end without any branching or additional workflow stages.

13. Examine the pipeline's underlying dictionary representation to better understand its configuration and parameters.

```
from rich.pretty import pprint
pprint(pipeline.__dict__)
```

This should give you the following output:

```
{
    "name": "FineTuningPipeline-...",
    "parameters": [
        {
            "name": "Dataset",
            "parameter_type": "String",
            "default_value": "..."
        },
        ...
    ],
    "mlflow_config": None,
    "steps": [
        "<....ProcessingStep>"
    ],
    "sagemaker_session": "...",
    ...
}
```

14. Use the `%store` magic to save the ARN of the current pipeline execution and persist it for reuse in subsequent sections of this chapter.

```
execution_arn = execution.arn
%store execution_arn
```

15. Use the `%store` magic to persist additional variables so you can reuse them in subsequent sections of this chapter without redefining them.

```
%store unique
%store role
%store region
```

```
%store prefix
%store s3
```

16. Export key pipeline variables into a structured JSON file to create a reusable snapshot of the execution environment for future steps or debugging purposes.

```
import json

config = {
    "unique": unique,
    "role": role,
    "region": region,
    "prefix": prefix,
    "s3": s3
}

output_file = "context.json"

with open(output_file, "w") as f:
    json.dump(config, f, indent=4)
```

17. Use the `describe()` method on the execution object and pretty-print the returned dictionary to inspect detailed information about the pipeline execution, including its current status and associated metadata.

```
pprint(execution.describe())
```

The response should include key fields such as the pipeline and execution ARNs, display name, execution status, and experiment configuration, similar to the example below:

```
{
  "PipelineArn": "...",
  "PipelineExecutionArn": "...",
  "PipelineExecutionDisplayName": "...",
  "PipelineExecutionStatus": "Executing",
  "PipelineExperimentConfig": {
    "ExperimentName": "...",
    "TrialName": "..."
  },
```

```
    ...
}
```

18. Wait for the pipeline execution to finish.

```
%%time
import time

while True:
    details = execution.describe()
    status = details["PipelineExecutionStatus"]
    print("Status:", status)

    if status != "Executing":
        break

    time.sleep(60)
```

**Note**

While waiting for the pipeline execution to complete, you may proceed with the next steps to explore and monitor the execution through the user interface. This allows you to review the execution details, parameters, and logs in real time as the workflow progresses.

19. Switch to the browser tab where SageMaker Studio is open. In the left sidebar, choose **Pipelines**. On the **Pipelines** page, locate the pipeline you just created and click its name under the **Name** column to open the pipeline details page. From here, you can access **Executions**, **Graph**, **Parameters**, **Versions**, and **Information** for the pipeline.
20. Select the execution name under the **Executions** tab to open its details (*Figure 9.3*).

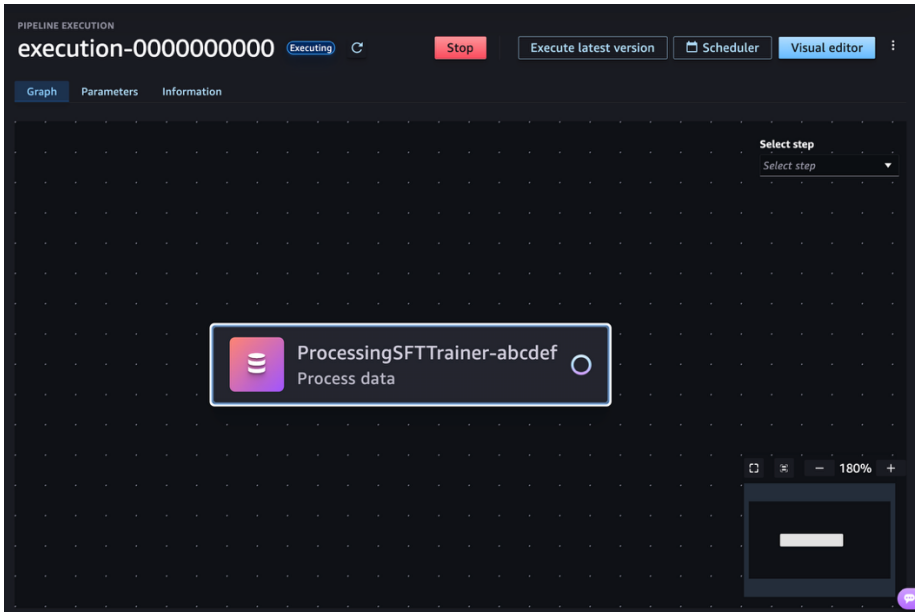



*Figure 9.3 — Pipeline execution graph*

Here, you'll be able to inspect the **Graph**, **Parameters**, and **Execution Information** related to the selected execution.

21. Double-click the rectangle corresponding to the step whose name begins with **ProcessingSFTTrainer** to view its details.

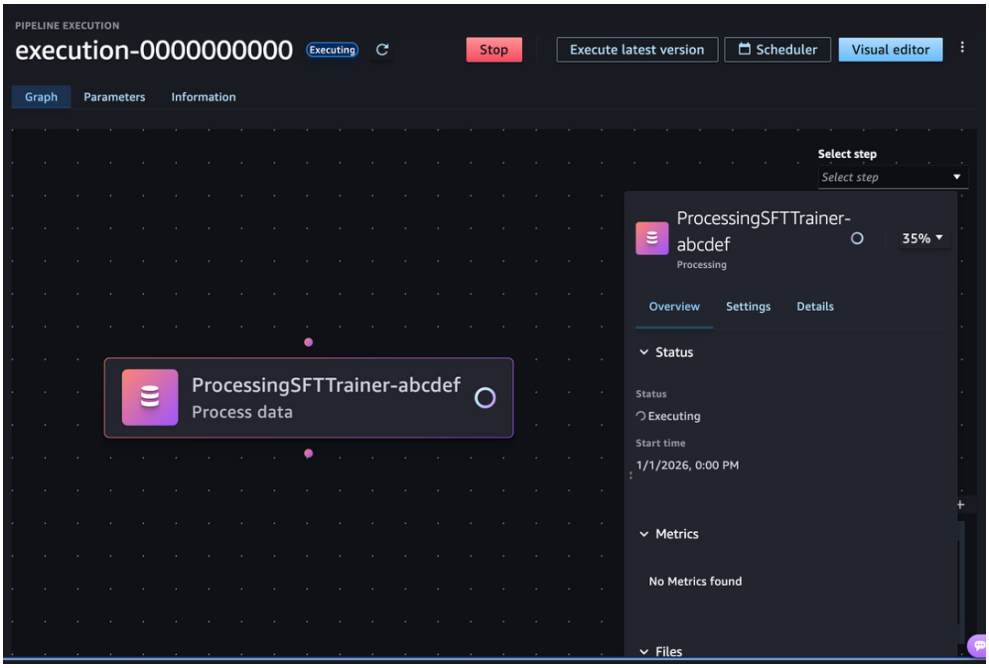



*Figure 9.4 — Viewing the details of the execution step*

Under **Overview**, you can view details such as the status, start time, metrics, and files. Under **Settings**, you can review key execution parameters, including the instance type, instance count, and container image URI. You will also find runtime configuration details such as the script arguments, code path, output path, dataset path, MLflow experiment name, and IAM role ARN, along with the associated input and output files used by the job. Under **Details**, you'll see a consolidated view of the pipeline execution metadata, including the job name, processing job ARN, start time, total run duration, and access to execution logs for troubleshooting and monitoring.

22. Switch back to the browser tab where the SageMaker Studio space with the Jupyter Notebook is open.

> **Note**
>
> Wait until the pipeline execution completes before moving forward. The execution usually takes about 15 to 25 minutes to finish.

*Wasn't that easy?!* You have now successfully built and executed a single-step fine-tuning pipeline using SageMaker Pipelines with the SageMaker Python SDK. In the next part, you will explore the processing logs to understand the runtime behavior of the training job and learn how to troubleshoot and debug pipeline executions in more detail.

## Inspecting pipeline execution logs

You'll now examine the completed pipeline execution logs. You will retrieve the execution object using its ARN, inspect execution metadata, and use a helper wrapper to explore logs, outputs, and generated artifacts such as model outputs and results files.

Let's now proceed with inspecting the pipeline execution logs:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `fine_tuning_step_02.ipynb`. We will run the code in the following steps within this notebook.

   **Note**

   Make sure that *fine_tuning_step_02.ipynb* is created directly inside the *pipeline* directory and not inside the *code* directory.

2. Restore the previously saved pipeline execution ARN from the IPython store using the `%store` magic command:

```
%store -r execution_arn
execution_arn
```

3. Restore the `PipelineExecution` object from the previously saved ARN.

```
from sagemaker.mlops.workflow.pipeline import (
    PipelineExecution
)
from sagemaker.core.helper.session_helper import (
    Session
)

sagemaker_session = Session()

execution = PipelineExecution(
```

```
    arn=execution_arn,
    sagemaker_session=sagemaker_session,
)
```

4. Run the following in a new cell to retrieve and inspect the execution metadata with `execution.describe()`:

```
from rich.pretty import pprint
pprint(execution.describe())
```

This should return a structured response, similar to the following:

```
{
  "PipelineArn": "...",
  "PipelineExecutionArn": "...",
  "PipelineExecutionDisplayName": "...",
  "PipelineExecutionStatus": "Succeeded",
  "PipelineExperimentConfig": {
    "ExperimentName": "...",
    "TrialName": "..."
  },
  ...
}
```

5. Use the `PipelineExecution` helper class (from *pipeline_wrapper.py*) to wrap the execution object and simplify inspection of logs, outputs, and artifacts generated during the workflow.

```
from pipeline_wrapper import PipelineExecution
pipeline_execution = PipelineExecution(
    execution=execution
)
```

6. Print the name of the first (and only) pipeline step:

```
pipeline_execution.step(1).name
```

This should return `'ProcessingSFTTrainer-...'`, which confirms that the first pipeline step corresponds to the fine-tuning stage.

7. Next, retrieve and inspect the processing job logs from the pipeline step:

```
pipeline_execution.step(1).logs()
```

This allows you to easily debug and analyze execution logs, which include installation, training, and artifact generation stages, similar to the following:

```
[INSTALL]: START
...
[INSTALL]: END
...
[FINE_TUNING_PIPELINE]: START
[MODEL_PACKAGE_GROUP]: START
...
[MODEL_PACKAGE_GROUP]: END
[TRAINING]: START
...
[MODEL_BUILD]: END
[ARTIFACT_PROCESSING]: START
...
[ARTIFACT_PROCESSING]: END
[MODEL_CREATION]: START
...
[MODEL_CREATION]: END
[FINE_TUNING_PIPELINE]: END
[WRITE_OUTPUT]: START
Saved results to .../results.json
[WRITE_OUTPUT]: END
```

Here, a `results.json` file is written to the output directory within the managed ML instance and is automatically uploaded to the specified Amazon S3 location.

8. Retrieve the pipeline output location, which contains all artifacts generated by the execution, including the results.json file stored in Amazon S3

```
pipeline_execution.outputs()
```

This should return an output similar to the following:

```
['s3://.../pipeline-.../output']
```

9. Use the `aws s3 ls` command to list the contents of the output S3 directory.

```
results = pipeline_execution.outputs()[0]

!aws s3 ls {results} --recursive
```

This should list all files stored in the S3 output location.

```
...
2026-01-01 00:00:00       1045 .../results.json
```

Scroll down to the bottom of the list to locate the `results.json` file, which contains the consolidated results of the execution, including references to the trained model artifacts, SageMaker AI resources, Amazon S3 output paths, and MLflow experiment metadata.

10. Use the `aws s3 cp` command to download the *results.json* file from Amazon S3:

```
!aws s3 cp {results}/results.json results.json
```

11. Examine the downloaded `results.json` file generated by the processing job.

```
!cat results.json
```

This should return JSON output similar to the following:

```
{
  "region": "us-east-1",
  "training_job_name": "...",
  "training_job_arn": "arn:aws:...",
  "model_package_group_name": "...",
  "model_package_group_arn": "arn:aws:...",
  "model_name": "model-...",
  "model_arn": "arn:aws:...",
  "s3_model_artifacts": "s3://.../output/model",
  "s3_model_path": "s3://.../model-....tar.gz",
  "dataset": "s3://.../data/data.jsonl",
  "output_path": "s3://.../output/",
  "mlflow_experiment_name": "sft-exp-run-...",
  "mlflow_run_name": "run-..."
}
```

Here, the *results.json* file provides a convenient reference to the resources created during fine-tuning. You can use these values to locate the generated model artifacts in Amazon S3, identify the associated SageMaker AI resources, and integrate the outputs into deployment or evaluation pipelines.

12. Load the `results.json` file into a Python variable

```
import json
results = json.loads(open("results.json").read())
```

> **Note**
>
> You may use `pprint(results)` to inspect the contents of the `results` variable in a more readable, structured format.

13. Store the results object for later use using IPython's `%store` magic.

```
%store results
```

From here, the *results.json* file can serve as the starting point for building and testing the subsequent steps in the pipeline. While it may be tempting to design the entire pipeline end-to-end in one pass, it is generally better to build and validate each step iteratively before combining them. This approach helps you isolate issues early, simplify debugging, and ensure each component behaves as expected before integrating it into a full workflow.

# Building and running the Single-Step evaluation pipeline

In this section, you will build and execute a single-step evaluation pipeline that uses LLM-as-a-Judge to assess the performance of a fine-tuned model. You will configure evaluation inputs, define custom metrics, retrieve the fine-tuned model, and run an automated evaluation workflow using SageMaker Pipelines.

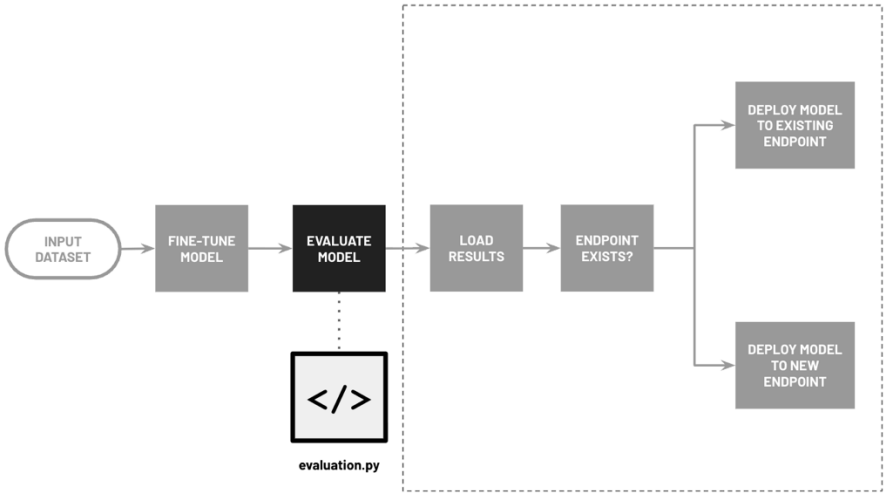



*Figure 9.5 — Building the single-step evaluation pipeline*

You will use the downloaded *evaluation.py* file as the core script powering the evaluation process. Since LLM-as-a-Judge evaluation is not yet natively supported in SageMaker Pipelines, the evaluation logic is executed within a SageMaker Processing job. The script runs the LLM-as-a-Judge evaluation process on the fine-tuned model artifacts. It applies the defined evaluation criteria, executes inference on the evaluation dataset, and computes the custom metrics used to assess model performance.

## Setting up and running LLM-as-a-Judge model evaluation

**LLM-as-a-Judge** is an evaluation approach where an LLM is used as an evaluator to assess the quality of another model's outputs. Instead of relying only on traditional automated metrics, it uses a separate LLM to score responses based on defined criteria such as correctness, completeness, and relevance. In this part, you will configure the evaluation inputs, initialize the evaluator, and run the LLM-as-a-Judge evaluation workflow.

**Note**

In this workflow, the evaluator model assesses the fine-tuned model's outputs using an evaluation dataset and assigns structured scores according to both built-in and custom metrics, which enable more flexible and human-like assessment of model performance.

Let's proceed with setting up and running an evaluation job:

1. Open a new Launcher by selecting **File** } **New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `evaluation.ipynb`. We will run the code in the following steps within this notebook.

   **Note**

   Make sure that *evaluation.ipynb* is created directly inside the *pipeline* directory and not inside the *code* directory.

2. Restore the results object as well as all pipeline-related variables from the previous sections for reuse in this notebook.

```
%store -r results
%store -r unique
%store -r role
%store -r region
%store -r prefix
%store -r s3

print("results:", results)
print("unique:", unique)
print("role:", role)
print("region:", region)
print("prefix:", prefix)
print("s3:", s3)

import json
results = json.loads(open("results.json").read())
```

   Alternatively, if you restarted the kernel, you can restore the required variables from the saved JSON files.

```
import json

results = json.loads(open("results.json").read())
config = json.loads(open("context.json").read())

unique = config["unique"]
```

```
role = config["role"]
region = config["region"]
prefix = config["prefix"]
s3 = config["s3"]
```

3. Extract MLflow experiment metadata from the stored results object and define a new run name for evaluation.

```
experiment_name = results['mlflow_experiment_name']
run_name = results['mlflow_run_name'] + "-eval"
```

4. Load the custom metric definition JSON file (`custom_metric.json`) from disk and inspect its content and structure.

```
import json
from rich.pretty import pprint

with open("custom_metric.json", "r") as f:
    custom_metric_dict = json.load(f)

pprint(custom_metric_dict)
```

You should get an output similar to the following, which defines the custom evaluation metric, including its name, instructions, and rating scale used for scoring model responses:

```
{
  "customMetricDefinition": {
    "name": "PositiveSentiment",
    "instructions": "You are a skilled evaluator...",
    "ratingScale": [
      {
        "definition": "Good",
        "value": { "floatValue": 1 }
      },
      {
        "definition": "Poor",
        "value": { "floatValue": 0 }
      }
    ]
```

```
    }
  }
```

**Note**

Here, you have a custom evaluation metric called `PositiveSentiment`, which is used to assess how positive or favorable a model's response is. It includes clear evaluation instructions and a simple rating scale that assigns a higher score to positive responses and a lower score to negative ones, allowing the evaluator to quantify sentiment in a structured way.

5. Convert the custom metric definition into a JSON string format required by the evaluator.

```
custom_metrics = json.dumps([custom_metric_dict])
```

6. Define the evaluation dataset location and output path in Amazon S3.

```
eval_ds_path = f"s3://{s3}/{prefix}/eval/gen_qa.jsonl"
eval_output_path = f"s3://{s3}/{prefix}/eval/output"
```

7. Upload the evaluation dataset to Amazon S3 so it can be accessed during evaluation.

```
!aws s3 cp data/gen_qa.jsonl {eval_ds_path}
```

8. Initialize a SageMaker AI `boto3` client (`sm_client`) to interact with model package metadata and registry.

```
import boto3

sm_client = boto3.client(
    "sagemaker",
    region_name=region
)
```

9. List the available model packages in the registry and retrieve the latest version.

```
group_name = results["model_package_group_name"]

response = sm_client.list_model_packages(
```

```
    ModelPackageGroupName=group_name,
    SortBy="CreationTime",
    SortOrder="Descending"
)

pprint(response)
```

This should give you the following output:

```
{
  "ModelPackageSummaryList": [
    {
      "ModelPackageGroupName": "...",
      "ModelPackageVersion": 1,
      "ModelPackageArn": "...",
      "CreationTime": "...",
      "ModelPackageStatus": "Completed",
      "ModelPackageRegistrationType": "Logged"
    }
  ],
  "ResponseMetadata": {...}
}
```

10. Extract the latest model package ARN for evaluation.

```
summary_list = response["ModelPackageSummaryList"]
model_package_arn = summary_list[0]['ModelPackageArn']
model_package_arn
```

11. Import the `LLMAsJudgeEvaluator` class for configuring and running LLM-as-a-Judge evaluation. Define the evaluator model and built-in metrics used for evaluation as well.

```
from sagemaker.train.evaluate import (
    LLMAsJudgeEvaluator
)

evaluator_model = 'amazon.nova-pro-v1:0'
builtin_metrics = ["Completeness", "Faithfulness"]
```

> **Note**
>
> Here, we define the evaluator model as **Amazon Nova Pro** (`amazon.nova-pro-v1:0`) for LLM-as-a-Judge evaluation and specify the built-in metrics that will be used to assess the model's responses, including `Completeness` and `Faithfulness`.

12. Initialize the LLM-as-a-Judge evaluator with both built-in and custom metrics.

```
evaluator = LLMAsJudgeEvaluator(
    model=model_package_arn,
    evaluator_model=evaluator_model,
    dataset=eval_ds_path,
    builtin_metrics=builtin_metrics,
    custom_metrics=custom_metrics,
    s3_output_path=eval_output_path,
    evaluate_base_model=False,
    mlflow_experiment_name=experiment_name,
    mlflow_run_name=run_name
)
```

13. Inspect the configured evaluator object to confirm settings before execution.

```
pprint(evaluator)
```

This should return output similar to the following, showing the evaluator configuration including the model, session settings, and experiment tracking details:

```
LLMAsJudgeEvaluator(
|   region=None,
|   role=None,
|   sagemaker_session=...,
|   model='...',
|   base_eval_name='...',
|   s3_output_path='...',
|   mlflow_resource_arn='...',
|   mlflow_experiment_name='...',
|   mlflow_run_name='...',
```

```
    ...
)
```

14. Execute the evaluation job and store the execution handle.

```
execution = evaluator.evaluate()
```

15. Inspect the evaluation execution object to confirm that the job has been successfully submitted and to review its initial metadata, including status, name, and configuration details.

```
pprint(execution)
```

You should see output similar to the following, which represents the initialized evaluation execution object and its associated metadata:

```
LLMAJEvaluationExecution(
|   arn='...',
|   name='...',
|   status=PipelineExecutionStatus(...),
|   last_modified_time=...,
|   eval_type=<EvalType.LLM_AS_JUDGE: 'llmasjudge'>,
|   s3_output_path='...',
|   steps=[]
)
```

16. Wait for the evaluation job to complete before retrieving results.

```
execution.wait()
```

This should give you the following output:

```
---------- Pipeline Execution Status ----------

 Evaluation Job      ...
 Links               Pipeline Execution (Studio) ...
 Overall Status      Executing
 Target Status       Succeeded
 Elapsed Time        163.4s

Pipeline Steps
```

```
    Step Name                    Status        ...
    CreateEvaluationAction       Succeeded     1.2s
    EvaluateCustomInferenceModel Executing.    ...

  ...
```

17. In the SageMaker Studio navigation pane (left sidebar), select **Pipelines**. On the Pipelines page, locate and open the pipeline that starts with `SagemakerEvaluation`. Under **Executions**, click the latest pipeline execution that starts with `eval-meta`. You should see a four-step pipeline similar to the one shown in *Figure 9.6*.

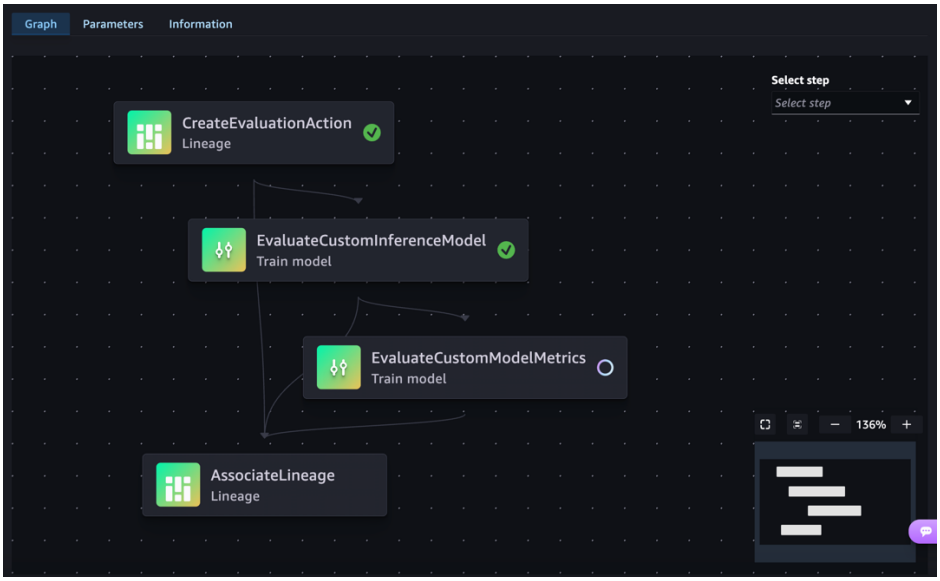



*Figure 9.6 — Four-step evaluation workflow*

Here, we have a four-step evaluation workflow that creates the evaluation action (**CreateEvaluationAction**), evaluates the inference model (**EvaluateCustomInferenceModel**), evaluates model metrics (**EvaluateCustomModelMetrics**), and associates lineage metadata (**AssociateLineage**).

> **Note**
>
> This may take 25 to 40 minutes to complete. Feel free to grab a cup of coffee or tea while you wait.

1. Switch back to the browser tab where the *evaluation.ipynb* notebook is already open and scroll to the bottom of the notebook to continue.
2. Run the following to display a sample of evaluation results for inspection.

```
execution.show_results(
    limit=10,
    offset=0,
    show_explanations=False
)
```

> **Note**
>
> Keep in mind that the evaluation metrics produced by the LLM-as-a-Judge process are influenced by the dataset used during evaluation. Variations in prompts, expected answers, and sample quality can affect the resulting scores. Given that the dataset used in this chapter is a simplified example dataset, the generated scores are expected to vary from real-world evaluation results. In practice, you should design or source datasets that closely match your domain, tasks, and expected user inputs for accurate evaluation results.

3. Print the final execution object for reference and debugging.

```
pprint(execution)
```

This should give you the following output:

```
LLMAJEvaluationExecution(
    arn='...',
    name='eval-meta-...',
    status=PipelineExecutionStatus(
        overall_status='Succeeded',
        step_details=[
            StepDetail(
                name='CreateEvaluationAction',
```

```
                ...
            ),
            StepDetail(
                name='EvaluateCustomInferenceModel',
                ...
            ),
            StepDetail(
                name='EvaluateCustomModelMetrics',
                ...
            ),
            StepDetail(
                name='AssociateLineage',
                ...
            )
        ],
        failure_reason=None
    ),
    ...
)
```

The `step_details` section provides a breakdown of each stage in the evaluation workflow, including the creation of the evaluation action (`CreateEvaluationAction`), model inference evaluation (`EvaluateCustomInferenceModel`), custom metric computation (`EvaluateCustomModelMetrics`), and lineage association (`AssociateLineage`).

> **Note**
>
> This helps you trace the full execution flow and verify that each step has completed successfully.

Now that you have a better understanding of how LLM-as-a-Judge works, you are ready to move on to the next stage, where you will operationalize this evaluation approach within a pipeline. In the next part, you will integrate the evaluation workflow into a repeatable pipeline component, enabling you to automate the evaluation of fine-tuned models. This will help you connect model training, evaluation, and pipeline orchestration into a unified LLMOps workflow.

## Building and running the pipeline

You will now build on what you learned in the previous part by constructing a pipeline that automates the evaluation workflow. You will define pipeline parameters, upload evaluation assets to Amazon S3, configure processing steps, and implement the evaluation step as a reusable pipeline component. Finally, you will start the pipeline execution and verify that all inputs and outputs are correctly connected.

Let's proceed with building and running the single-step evaluation pipeline:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `evaluation_step.ipynb`. We will run the code in the following steps within this notebook.

   **Note**

   Make sure that *evaluation_step.ipynb* is created directly inside the *pipeline* directory and not inside the *code* directory.

2. Restore the previously saved pipeline context variables so they can be reused in this notebook.

   ```
   %store -r unique
   %store -r role
   %store -r region
   %store -r prefix
   %store -r s3

   import json
   results = json.loads(open("results.json").read())
   ```

   You can also restore previously saved pipeline context using the following block of code:

   ```
   import json

   results = json.loads(open("results.json").read())
   config = json.loads(open("context.json").read())
   ```

```
unique = config["unique"]
role = config["role"]
region = config["region"]
prefix = config["prefix"]
s3 = config["s3"]
```

3. Import the relevant classes and functions needed to build and run the pipeline.

```
from sagemaker.core.workflow.parameters import (
    ParameterString
)
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)
from sagemaker.core.processing import ScriptProcessor
from sagemaker.core.shapes import (
    ProcessingInput,
    ProcessingOutput,
    ProcessingS3Output
)
from sagemaker.mlops.workflow.pipeline import Pipeline
from sagemaker.mlops.workflow.steps import (
    ProcessingStep
)
from sagemaker.core.workflow.pipeline_context import (
    PipelineSession
)
from sagemaker.core.workflow.functions import Join
from sagemaker.core import image_uris
```

4. Initialize the required pipeline prerequisites, including the SageMaker session, pipeline session, and execution role. These components provide the runtime context and permissions needed to create and execute the pipeline.

```
sagemaker_session = Session()
pipeline_session = PipelineSession()
role = get_execution_role()
```

5. Upload the datasets and configuration files to Amazon S3 using the following block of code.

```
sp = f"s3://{s3}/{prefix}"

results_s3 = f"{sp}/input/"
eval_dataset_s3 = f"{sp}/data/gen_qa.jsonl"
custom_metric_s3 = f"{sp}/input/custom_metric.json"
output_s3 = f"{sp}/output/"

!aws s3 cp data/gen_qa.jsonl {eval_dataset_s3}
!aws s3 cp custom_metric.json {custom_metric_s3}
!aws s3 cp results.json {results_s3}
```

6. Next, let's define pipeline parameters to make the workflow reusable and configurable.

```
results_param = ParameterString(
    name="ResultsPath",
    default_value=results_s3
)

dataset_param = ParameterString(
    name="EvalDataset",
    default_value=eval_dataset_s3
)

metric_param = ParameterString(
    name="CustomMetricPath",
    default_value=custom_metric_s3
)

output_param = ParameterString(
    name="OutputPath",
    default_value=output_s3
)

region_param = ParameterString(
    name="Region",
    default_value=region
)
```

7. Initialize a `ScriptProcessor` object that configures the compute environment, container image, and execution settings for the processing job that runs the evaluation job.

```
eval_processor = ScriptProcessor(
    image_uri=image_uris.retrieve(
        framework="sklearn",
        region=region,
        version="1.2-1",
        py_version="py3",
        instance_type="ml.m5.xlarge",
    ),
    instance_type="ml.m5.xlarge",
    instance_count=1,
    base_job_name=f"llm-eval-{unique}",
    sagemaker_session=pipeline_session,
    role=role,
)
```

8. Next, define the input configuration for the evaluation step:

```
from sagemaker.core.shapes import (
    ProcessingInput,
    ProcessingOutput,
    ProcessingS3Output,
    ProcessingS3Input
)

from sagemaker.core.workflow.functions import Join
results_s3_uri = Join(
    on="",
    values=[
        results_param,
        "results.json"
    ]
)

results_input = ProcessingInput(
    input_name="results",
    s3_input=ProcessingS3Input(
        s3_uri=results_s3_uri,
```

```
        local_path="/opt/ml/processing/input",
        s3_data_type="S3Prefix",
        s3_input_mode="File",
        s3_data_distribution_type="ShardedByS3Key",
    ),
)
```

9. Configure the processing step by specifying output handling, providing runtime arguments to the evaluation script, and defining the corresponding `ProcessingStep` for execution.

```
local_path = "/opt/ml/processing/output"

eval_step_args = eval_processor.run(
    inputs=[results_input],
    outputs=[
        ProcessingOutput(
            output_name="output",
            s3_output=ProcessingS3Output(
                s3_uri=output_param,
                local_path=local_path,
                s3_upload_mode="EndOfJob"
            )
        )
    ],
    code="code/evaluation.py",
    arguments=[
        "--eval-dataset", dataset_param,
        "--custom-metric-path", metric_param,
        "--output-path", output_param,
        "--region", region_param,
    ],
)

eval_step = ProcessingStep(
    name=f"LLM-Eval-{unique}",
    step_args=eval_step_args,
)
```

10. Construct the pipeline by specifying the execution steps (in this case, a single step), input parameters, and session configuration required to run the workflow.

```
pipeline = Pipeline(
    name=f"EvaluationPipeline-{unique}",
    steps=[eval_step],
    parameters=[
        results_param,
        dataset_param,
        metric_param,
        output_param,
        region_param,
    ],
    sagemaker_session=pipeline_session,
)
```

11. Create or update the pipeline definition using `upsert()` and associate it with the specified IAM role.

```
pipeline.upsert(role_arn=role)
```

12. With everything ready, start the pipeline execution.

```
execution = pipeline.start()
```

13. Retrieve the evaluation pipeline execution ARN from the execution object and store it for later use.

```
eval_pipeline_execution_arn = execution.arn
%store eval_pipeline_execution_arn
```

14. Run the following block of code to wait for the pipeline execution to finish.

```
%%time
import time

while True:
    details = execution.describe()
    status = details["PipelineExecutionStatus"]
    print("Status:", status)
```

```
    if status != "Executing":
        break

    time.sleep(60)
```

> **Note**
>
> This step may take around 25 to 45 minutes to complete. Feel free to grab a cup of coffee or tea while you wait.

With the pipeline now built and executed, you have successfully automated the evaluation workflow using SageMaker Pipelines. In the next part, you will inspect the completed pipeline execution in detail, including logs and evaluation results generated during the LLM-as-a-Judge process.

## Inspecting pipeline execution logs

You will now explore and inspect the logs generated by the completed pipeline execution. Here, you will retrieve the execution using its ARN and use a helper class to make inspection easier. You will then review logs, outputs, and evaluation results, including per-sample predictions, metrics, and artifacts generated by the LLM-as-a-Judge process.

Let's proceed with inspecting the pipeline execution logs:

1. Open a new Launcher by selecting **File** | **New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `evaluation_step_02.ipynb`. We will run the code in the following steps within this notebook.
2. Restore the evaluation pipeline execution ARN using the `%store` magic command.

```
%store -r eval_pipeline_execution_arn
eval_pipeline_execution_arn
```

3. Reconstruct the `PipelineExecution` object from the evaluation pipeline execution ARN.

```
from sagemaker.mlops.workflow.pipeline import (
    PipelineExecution
)
```

```
from sagemaker.core.helper.session_helper import (
    Session
)

sagemaker_session = Session()

execution = PipelineExecution(
    arn=eval_pipeline_execution_arn,
    sagemaker_session=sagemaker_session,
)
```

4. Run the following in a new cell to retrieve and inspect the execution metadata with `execution.describe()`:

```
from rich.pretty import pprint
pprint(execution.describe())
```

This should return a structured response, similar to the following:

```
{
  "PipelineArn": "...",
  "PipelineExecutionArn": "...",
  "PipelineExecutionDisplayName": "...",
  "PipelineExecutionStatus": "Succeeded",
  "PipelineExperimentConfig": {
    "ExperimentName": "...",
    "TrialName": "..."
  },
  ...
}
```

5. Use the `PipelineExecution` helper class (from *pipeline_wrapper.py*) to wrap the execution object and simplify inspection of logs, outputs, and artifacts generated during the workflow.

```
from pipeline_wrapper import PipelineExecution

pipeline_execution = PipelineExecution(
```

```
    execution=execution
)
```

6. Retrieve and inspect the processing job logs from the first (and only) pipeline step:

```
pipeline_execution.step(1).logs()
```

This should yield the following output:

```
...
=== Evaluation 10 of 20 ===
Prompt: User: What is a neural network?
Model Response: ['A neural network is a computer system inspired by the
structure and function of the human brain. It\'s a network of
interconnected nodes or "neurons" that process information and make
decisions b...

  Metric                                Score
 ──────────────────────────────────────────────

  Builtin.Completeness                    ...
  Builtin.Faithfulness                    ...
  PositiveSentiment                       ...


═══════════════════════════════════════════
Showing evaluations 1-10 of 20
To see more:
  job.show_results(limit=10, offset=10)  ...
  job.show_results(limit=None)  ...
═══════════════════════════════════════════

[RESULTS]: END
[EVALUATION_PIPELINE]: END
[WRITE_OUTPUT]: START
/opt/ml/processing/output/eval_results.json
/opt/ml/processing/output/training_results.json
[WRITE_OUTPUT]: END
```

With the evaluation pipeline successfully completed, you now have a reusable pipeline step that automates LLM-as-a-Judge evaluation using a `ProcessingStep`-based implementation. You also learned how to convert a standalone evaluation workflow into a structured and repeatable component that can be integrated into larger LLMOps pipelines. In the next section,

you will build on what you have created so far by combining fine-tuning and evaluation into a fully automated two-step pipeline.

## Configuring and running a Two-Step Fine-Tuning and evaluation pipeline

So far, you have been working with single-step pipelines to understand and validate individual stages in isolation. In this section, you will build, configure, and run a two-step pipeline that performs supervised fine-tuning followed by evaluation, using the concepts, and components covered in the previous sections.

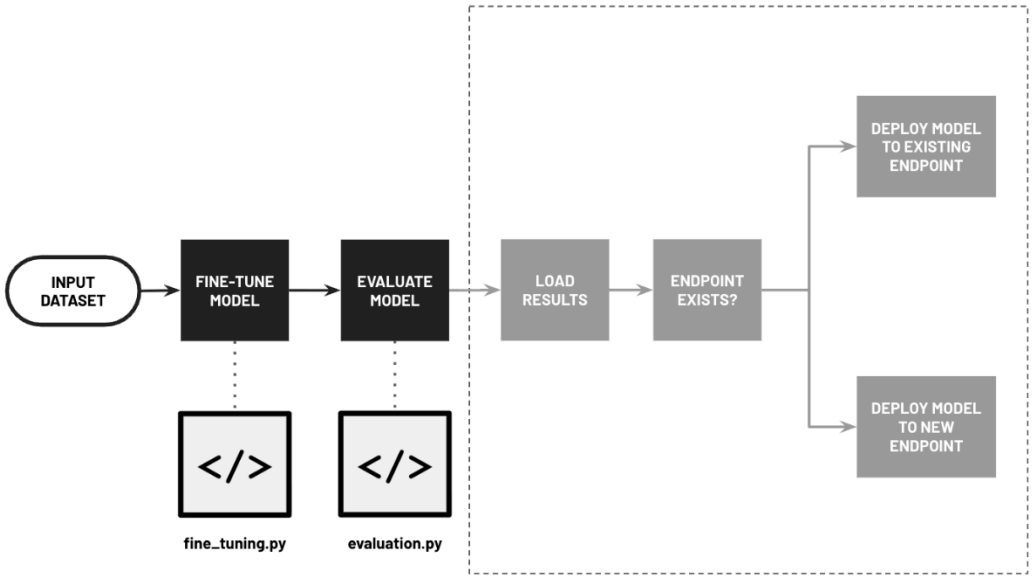



*Figure 9.7 — Building a two-step fine-tuning and evaluation pipeline*

To implement this automated workflow, you will define pipeline parameters, configure processing steps, and establish dependencies between steps to ensure the correct execution order. The output produced by the fine-tuning step is passed directly into the evaluation step, ensuring that evaluation is performed on the most recent model artifacts and generated results.

## Building and running the pipeline

You will start by creating a new Jupyter notebook and importing the required libraries and modules needed to construct the pipeline. From there, you will define helper functions, configure S3 paths, upload datasets, and progressively set up and configure pipeline parameters, processing steps, and execution logic until the full two-step workflow is ready to run. After completing the pipeline setup, you will run the pipeline and monitor its execution until it completes.

Let's walk through the steps to build and run the pipeline:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `complete_pipeline.ipynb`. We will run the code in the following steps within this notebook.

   **Note**

   Similar to the other *.ipynb* notebooks created throughout this chapter, make sure that *complete_pipeline.ipynb* is created directly inside the *pipeline* directory and not within any of its subdirectories.

2. Define the `UniqueString` helper class, which lazily generates and persists a reusable random identifier. The class first attempts to load the value from a local *unique.txt* file. If the file does not exist, a new random string is generated, written to disk, and reused for the remainder of the notebook session.

```
import random
import string
from pathlib import Path


class UniqueString:
    file_path = Path("unique.txt")

    @classmethod
    def generate_string(cls, length=12):
        return ''.join(
            random.choices(
                string.ascii_lowercase,
```

```
                k=length
            )
        )

    @classmethod
    @property
    def value(cls):
        if not hasattr(cls, "_value"):
            if cls.file_path.exists():
                cls._value = cls.file_path.read_text()
                cls._value = cls._value.strip()
            else:
                cls._value = cls.generate_string()
                cls.file_path.write_text(cls._value)

        return cls._value
```

3. Print the persisted unique identifier generated by the `UniqueString` helper class.

```
print(UniqueString.value)
```

If a value already exists in *unique.txt*, the same identifier will be reused; otherwise, a new value will be generated and stored automatically.

4. Import the required libraries, modules, and classes used to configure and construct the pipeline, including workflow parameters, session utilities, processing components, and pipeline step definitions.

```
from sagemaker.core.workflow.parameters import (
    ParameterString
)
from sagemaker.core.helper.session_helper import (
    Session,
    get_execution_role
)
from sagemaker.core.processing import (
    ScriptProcessor
)
from sagemaker.core.shapes import (
    ProcessingInput,
    ProcessingOutput,
```

```
    ProcessingS3Output,
    ProcessingS3Input
)
from sagemaker.mlops.workflow.pipeline import (
    Pipeline
)
from sagemaker.mlops.workflow.steps import (
    ProcessingStep
)
from sagemaker.core.workflow.pipeline_context import (
    PipelineSession
)
from sagemaker.core.workflow.functions import (
    Join
)
from sagemaker.core import (
    image_uris
)
```

5. Access the generated or persisted unique identifier for this notebook session.

```
unique = UniqueString.value
```

6. Initialize the pipeline prerequisites, including the `Session` and `PipelineSession` objects, execution role, AWS Region, and dynamically generated variables used for naming and organizing Amazon S3 resources and pipeline artifacts.

```
sagemaker_session = Session()
pipeline_session = PipelineSession()
role = get_execution_role()
region = sagemaker_session.boto_region_name

prefix = f"pipeline-{unique}"
s3 = sagemaker_session.default_bucket()
```

7. Next, define the Amazon S3 paths that will be used to store the training dataset, evaluation dataset, custom metric definition, and output artifacts generated by the fine-tuning and evaluation steps.

```
train_dataset_s3 = (
    f"s3://{s3}/{prefix}/data/data.jsonl"
)

eval_dataset_s3 = (
    f"s3://{s3}/{prefix}/data/gen_qa.jsonl"
)

custom_metric_s3 = (
    f"s3://{s3}/{prefix}/input/custom_metric.json"
)

train_output_s3 = (
    f"s3://{s3}/{prefix}/training-output/"
)

eval_output_s3 = (
    f"s3://{s3}/{prefix}/evaluation-output/"
)
```

8. Upload the training dataset, evaluation dataset, and custom metric definition file to Amazon S3 so they can be accessed by the pipeline processing steps during execution.

```
!aws s3 cp data/data.jsonl {train_dataset_s3}
!aws s3 cp data/gen_qa.jsonl {eval_dataset_s3}
!aws s3 cp custom_metric.json {custom_metric_s3}
```

9. Define the pipeline parameters for the fine-tuning workflow, including the IAM role used for execution, the location of the training dataset, and the Amazon S3 path where outputs will be stored.

```
role_arn_param = ParameterString(
    name="TrainingRoleArn",
    default_value=role
)
```

```
train_dataset_param = ParameterString(
    name="Dataset",
    default_value=train_dataset_s3
)

train_output_param = ParameterString(
    name="TrainingOutputPath",
    default_value=train_output_s3
)
```

These parameters allow the pipeline to be dynamically configured at runtime without modifying the underlying code.

10. Next, define pipeline parameters that manage experiment tracking and execution context, including the MLflow experiment and run names, as well as the AWS Region.

```
mlflow_exp_param = ParameterString(
    name="MLflowExperimentName",
    default_value=f"sft-experiment-{unique}"
)

mlflow_run_param = ParameterString(
    name="MLflowRunName",
    default_value=f"run-{unique}"
)

region_param = ParameterString(
    name="Region",
    default_value=region
)
```

11. Define additional pipeline parameters that configure the evaluation step, including the evaluation dataset location, custom metric definition path, and the S3 output path for evaluation results.

```
eval_dataset_param = ParameterString(
    name="EvalDataset",
    default_value=eval_dataset_s3
)

custom_metric_param = ParameterString(
```

```
    name="CustomMetricPath",
    default_value=custom_metric_s3
)

eval_output_param = ParameterString(
    name="EvaluationOutputPath",
    default_value=eval_output_s3
)
```

12. Initialize a `ScriptProcessor` object that configures the compute environment, container image, and execution settings for the processing job that runs the supervised fine-tuning process.

```
local_path = "/opt/ml/processing/output"

train_processor = ScriptProcessor(
    image_uri=image_uris.retrieve(
        framework="sklearn",
        region=region,
        version="1.2-1",
        py_version="py3",
        instance_type="ml.m5.xlarge",
    ),
    instance_type="ml.m5.xlarge",
    instance_count=1,
    base_job_name=f"sfttrainer-job-{unique}",
    sagemaker_session=pipeline_session,
    role=role,
)
```

**Note**

Here, we are configuring the processing job to use the same execution role associated with the SageMaker Studio space. Before deploying the pipeline to production, you should create a dedicated IAM role with more restrictive permissions, following the principle of least privilege to limit access to only the resources required by the pipeline. The permissions policy should be made more granular by explicitly scoping access to specific S3 buckets, prefixes, and SageMaker AI actions required for each step of the workflow.

13. Define the caching configuration for the pipeline step execution.

```
from sagemaker.mlops.workflow.steps import CacheConfig

cache_config = CacheConfig(
    enable_caching=True,
    expire_after="PT12H"
)
```

Here, you configure step-level caching for the pipeline using `CacheConfig`. When caching is enabled, SageMaker Pipelines can reuse the results of previous step executions if the inputs, code, and configuration remain unchanged. The `expire_after="PT12H"` setting specifies that cached results remain valid for 12 hours before the step is executed again.

14. Configure the corresponding `ProcessingStep` object using the following block of code.

```
train_step_args = train_processor.run(
    inputs=[],
    outputs=[
        ProcessingOutput(
            output_name="output",
            s3_output=ProcessingS3Output(
                s3_uri=train_output_s3,
                local_path=local_path,
                s3_upload_mode="EndOfJob"
            )
        )
    ],
    code="code/fine_tuning.py",
    arguments=[
        "--dataset", train_dataset_param,
        "--output-path", train_output_param,
        "--mlflow-experiment-name", mlflow_exp_param,
        "--mlflow-run-name", mlflow_run_param,
        "--region", region_param,
        "--role-arn", role_arn_param,
    ],
)

train_step = ProcessingStep(
```

```
    name=f"FineTuning-{unique}",
    step_args=train_step_args,
    cache_config=cache_config
)
```

15. Initialize a `ScriptProcessor` object that configures the compute environment, container image, and execution settings for the processing job that runs the evaluation process.

```
eval_processor = ScriptProcessor(
    image_uri=image_uris.retrieve(
        framework="sklearn",
        region=region,
        version="1.2-1",
        py_version="py3",
        instance_type="ml.m5.xlarge",
    ),
    instance_type="ml.m5.xlarge",
    instance_count=1,
    base_job_name=f"llm-eval-{unique}",
    sagemaker_session=pipeline_session,
    role=role,
)
```

**Note**

Similar to the processing job used for the fine-tuning step, the evaluation step also uses the same execution role associated with the SageMaker Studio space. In production environments, it is recommended to use dedicated IAM roles with least-privilege permissions, ideally with separate roles per pipeline step. *Why is this important?* If the fine-tuning step is compromised and its execution role has broader permissions such as access to Amazon Bedrock APIs that are not strictly required for training, an attacker could potentially invoke those services and cause unintended or malicious usage. By restricting permissions to only what is required for each step, you ensure that even if one component is compromised, the potential impact is significantly limited and does not extend to unrelated services or workflows.

16. Extract the fine-tuning output artifact location from the training step and construct the evaluation input configuration by referencing the expected *results.json* file that will be generated after the training process completes and is stored in Amazon S3.

```
tpoc = train_step.properties.ProcessingOutputConfig
ts3uri = tpoc.Outputs["output"].S3Output.S3Uri

results_input = ProcessingInput(
    input_name="results",
    s3_input=ProcessingS3Input(
        s3_uri=Join(
            on="",
            values=[ts3uri, "results.json"]
        ),
        local_path="/opt/ml/processing/input",
        s3_data_type="S3Prefix",
        s3_input_mode="File",
        s3_data_distribution_type="FullyReplicated",
    ),
)
```

17. Configure the corresponding `ProcessingStep` object using the following block of code.

```
local_path = "/opt/ml/processing/output"

eval_step_args = eval_processor.run(
    inputs=[
        results_input
    ],
    outputs=[
        ProcessingOutput(
            output_name="output",
            s3_output=ProcessingS3Output(
                s3_uri=eval_output_param,
                local_path=local_path,
                s3_upload_mode="EndOfJob"
            )
        )
    ],
    code="code/evaluation.py",
    arguments=[
```

```
        "--eval-dataset", eval_dataset_param,
        "--custom-metric-path", custom_metric_param,
        "--output-path", eval_output_param,
        "--region", region_param,
    ],
)

eval_step = ProcessingStep(
    name=f"Evaluation-{unique}",
    step_args=eval_step_args,
    depends_on=[train_step],
    cache_config=cache_config
)
```

Here, we use `depends_on=[train_step]` to ensure that the evaluation step only runs after the fine-tuning step has successfully completed. This guarantees the correct execution order in the pipeline and ensures that the evaluation step uses the outputs produced by the training step.

**Note**

For more complex pipelines with multiple steps, `depends_on` can be used to explicitly define dependencies between different stages, ensuring that each step only runs when all its required preceding steps have finished. This helps maintain deterministic execution order and avoids issues related to missing or incomplete intermediate artifacts.

18. Construct the pipeline by defining the ordered execution steps (fine-tuning followed by evaluation), specifying all input parameters required to make the workflow configurable, and providing the session configuration needed to run the pipeline.

```
pipeline = Pipeline(
    name=f"CompletePipeline-{unique}",
    steps=[
        train_step,
        eval_step
    ],
    parameters=[
        role_arn_param,
        train_dataset_param,
```

```
        train_output_param,
        mlflow_exp_param,
        mlflow_run_param,
        region_param,
        eval_dataset_param,
        custom_metric_param,
        eval_output_param,
    ],
    sagemaker_session=pipeline_session,
)
```

19. Use the `upsert()` method to create or update the pipeline definition and associate it with the specified IAM role.

```
pipeline.upsert(role_arn=role)
```

20. With everything ready, start the pipeline execution.

```
execution = pipeline.start(
    parameters={
        "Dataset": train_dataset_s3,
        "TrainingOutputPath": train_output_s3,
        "MLflowExperimentName": (
            f"sft-experiment-{unique}"
        ),
        "MLflowRunName": (
            f"run-{unique}"
        ),
        "Region": region,
        "TrainingRoleArn": role,
        "EvalDataset": eval_dataset_s3,
        "CustomMetricPath": custom_metric_s3,
        "EvaluationOutputPath": eval_output_s3,
    }
)
```

Here, we pass runtime parameter values that override the default pipeline parameters, including dataset locations, output paths, MLflow tracking configuration, AWS Region, IAM role, and evaluation inputs.

21. Retrieve the pipeline execution ARN from the started execution object and store it for later reference.

```
pipeline_execution_arn = execution.arn
%store pipeline_execution_arn
```

> **Note**
>
> Alternatively, you can retrieve the pipeline execution ARN directly from the SageMaker Studio interface by navigating to the **Pipelines** section, selecting the relevant pipeline, and opening the specific execution details page.

22. Run the following block of code to continuously monitor the pipeline execution status and wait until the execution completes.

```
%%time
import time

while True:
    details = execution.describe()
    status = details["PipelineExecutionStatus"]
    print("Status:", status)

    if status != "Executing":
        break

    time.sleep(60)
```

The loop checks the pipeline status every 60 seconds and exits automatically once the execution is no longer in the `Executing` state. The `%%time` magic command displays the total elapsed execution time for the monitoring process.

> **Note**
>
> Wait for the pipeline execution to complete before proceeding to the next steps. Depending on the workload and resources used, the execution may take approximately 45 to 90 minutes to finish.

*Wasn't that easy?* You just built and executed a fully functional two-step pipeline using SageMaker Pipelines. You made use of everything covered so far in a step-by-step, iterative

approach to ensure that each component worked correctly on its own before integrating them into a complete workflow. With this approach, each stage was validated independently, making the final pipeline easier to build, debug, and understand.

## Inspecting pipeline execution logs

We will now retrieve and inspect the logs generated by the two-step pipeline. This will help you validate that both the fine-tuning and evaluation steps ran successfully and produced the expected outputs.

Let's proceed with inspecting the pipeline execution logs:

1. Open a new Launcher by selecting **File | New Launcher** from the menu. In the Launcher, create a new Jupyter notebook by choosing **Python 3 (ipykernel)** (under **Notebook**). Before continuing with the next steps, rename the notebook to `complete_pipeline_02.ipynb`. We will run the code in the following steps within this notebook.
2. Retrieve the previously stored pipeline execution ARN and display its value to confirm you are referencing the correct pipeline run.

```
%store -r pipeline_execution_arn
pipeline_execution_arn
```

3. Restore the `PipelineExecution` object from the previously saved ARN, so you can programmatically access metadata of the pipeline execution.

```
from sagemaker.mlops.workflow.pipeline import (
    PipelineExecution
)
from sagemaker.core.helper.session_helper import (
    Session
)

sagemaker_session = Session()

execution = PipelineExecution(
    arn=pipeline_execution_arn,
    sagemaker_session=sagemaker_session,
)
```

4. Run the following in a new cell to retrieve and inspect the execution metadata with `execution.describe()`:

```
from rich.pretty import pprint
pprint(execution.describe())
```

This should return a structured response, similar to the following:

```
{
  "PipelineArn": "...",
  "PipelineExecutionArn": "...",
  "PipelineExecutionDisplayName": "...",
  "PipelineExecutionStatus": "Succeeded",
  "PipelineExperimentConfig": {
    "ExperimentName": "...",
    "TrialName": "..."
  },
  ...
}
```

5. Use the `PipelineExecution` helper class (from *pipeline_wrapper.py*) to wrap the execution object and simplify inspection of logs, outputs, and artifacts generated during the workflow.

```
from pipeline_wrapper import PipelineExecution

pipeline_execution = PipelineExecution(
    execution=execution
)
```

6. Display the total number of steps defined in the pipeline execution to confirm that both the fine-tuning and evaluation stages were included successfully.

```
pipeline_execution.number_of_steps()
```

This should return 2, indicating that the pipeline execution contains two steps: the fine-tuning stage and the evaluation stage.

> **Note**
>
> If the pipeline execution has not yet progressed to the second step when you run this command, the result may be 1 instead of 2. Wait for the pipeline execution to finish, then run the command again.

7. Display the name of the first pipeline step to verify that it corresponds to the fine-tuning stage.

```
pipeline_execution.step(1).name
```

8. Retrieve logs from the fine-tuning step of the pipeline.

```
pipeline_execution.step(1).logs()
```

This should return logs similar to the following:

```
...
[ARTIFACT_PROCESSING]: END
[MODEL_CREATION]: START
Created model: model-...
Model ARN: arn:aws:sagemaker:...:model/model-...
[MODEL_CREATION]: END
[FINE_TUNING_PIPELINE]: END
[WRITE_OUTPUT]: START
Saved results to /opt/ml/processing/output/results.json
[WRITE_OUTPUT]: END
```

9. Display the name of the second pipeline step to verify that it corresponds to the evaluation stage.

```
pipeline_execution.step(2).name
```

10. Retrieve logs from the evaluation step of the pipeline.

```
pipeline_execution.step(2).logs()
```

This should display logs similar to the following:

```
...
=== Evaluation 10 of 20 ===
Prompt: User: What is a neural network?
Model Response: ['A neural network is a computer system inspired by the
structure and function of the human brain. It\'s a network of
interconnected nodes or "neurons" that process information and make
decisions b...

  Metric                                Score
 ──────────────────────────────────────────────

  Builtin.Completeness                    ...
  Builtin.Faithfulness                    ...
  PositiveSentiment                       ...

═══════════════════════════════════════════════
Showing evaluations 1-10 of 20
To see more:
  job.show_results(limit=10, offset=10)  ...
  job.show_results(limit=None)  ...
═══════════════════════════════════════════════

[RESULTS]: END
[EVALUATION_PIPELINE]: END
[WRITE_OUTPUT]: START
/opt/ml/processing/output/eval_results.json
/opt/ml/processing/output/training_results.json
[WRITE_OUTPUT]: END
```

Here, you can see that the evaluation step completed successfully, generated evaluation metrics for the model responses, and saved the evaluation output artifacts to the processing output directory.

11. Use the `outputs()` method of the `PipelineExecution` object to retrieve the pipeline execution output(s).

```
pipeline_execution.outputs()
```

You should see output similar to the following:

```
['s3://.../evaluation-output/']
```

12. Retrieve the pipeline output path and use it to construct the S3 path for the training_results.json file.

```
output = pipeline_execution.outputs()[0]
s3_path = output + "training_results.json"
```

13. Use the `aws s3 ls` command to verify that the `training_results.json` file exists in Amazon S3.

```
!aws s3 ls {s3_path}
```

14. Print the full S3 path of the `training_results.json` file.

```
print(s3_path)
```

This should give you the following output:

```
s3://.../training_results.json
```

Copy this value to your text editor (`TAG: JSON_S3_PATH`).

At this point, you should have a solid understanding of how to build, execute, and troubleshoot a pipeline end to end. You have seen how individual steps are configured, how dependencies control execution order, and how outputs are passed between stages. You have also learned how to inspect execution metadata and review logs to confirm that each step completed successfully. Together, these capabilities give you a practical foundation for identifying issues and iterating on more complex pipeline workflows.

**Note**

As an ML engineer, you should also apply these practices when designing production-ready pipelines, ensuring that each step is modular, observable, and reproducible. You should think carefully about how data and artifacts move between stages, and verify that each component can be tested and validated independently before integration. In addition, you should prioritize clear logging, proper error handling, and well-defined

dependencies to make debugging and maintenance more straightforward as your workflows grow in complexity.

# Preparing the Lambda functions for deployment of a model to an endpoint

The rest of this chapter focuses on using Lambda-powered steps to automate and orchestrate the deployment of the fine-tuned model as part of the pipeline workflow. These steps enable you to introduce conditional logic and operational control into the deployment process. In this section, you will set up four Lambda functions that orchestrate model deployment within the pipeline, including reading S3-based artifacts, checking endpoint status, and managing deployment of the fine-tuned model to either new or existing endpoints.

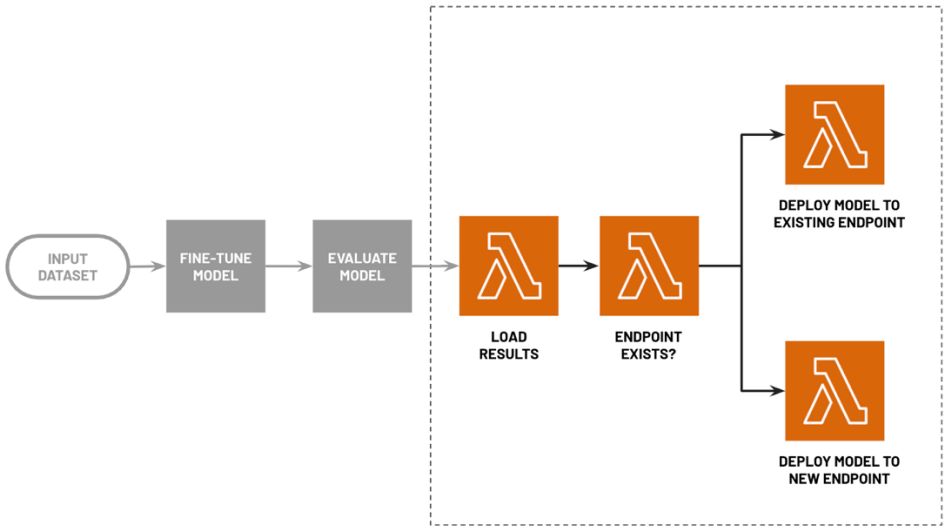



*Figure 9.8 — Lambda functions that orchestrate model deployment within the pipeline*

These Lambda functions will act as key integration components between the initial pipeline step outputs and deployment steps. Together, they enable you to complete the end-to-end workflow from model fine-tuning, evaluation, and deployment. You can think of these Lambda functions as a ride on a conveyor belt in a packaging plant, where each station performs a specific action before forwarding the item. While each function in this pipeline has a specific role, Lambda itself is flexible and can run virtually any type of code as long as it fits within the

event-driven execution model. This makes it suitable for a wide range of orchestration and automation tasks beyond the deployment logic shown here.

**Note**

If this is your first time working with **AWS Lambda**, it is a serverless compute service that lets you run code without provisioning or managing servers. When you create a Lambda function, you primarily define its inputs, outputs, and the function code that executes whenever it is invoked. You also specify how the function should process incoming events and what response it should return to downstream services. It automatically scales based on demand and only charges you for the compute time used during execution. You typically use Lambda to run small, event-driven functions that integrate with other AWS services, making it well suited for lightweight orchestration tasks in ML pipelines.

You will be setting up the following functions in this section:

- **s3-json-reader** — Reads a JSON file from Amazon S3 and makes the data available for later steps in the pipeline. You will use this function to load initial pipeline step output and configuration data stored in S3.
- **check-if-endpoint-exists** — Checks whether a SageMaker AI endpoint already exists so the pipeline knows what to do next.
- **deploy-model-to-new-endpoint** — Creates a new SageMaker AI endpoint and deploys the fine-tuned model when no endpoint is available. You will use this function when deploying the model for the first time in a given environment.
- **deploy-model-to-existing-endpoint** — Updates an existing SageMaker AI endpoint with the new model version. You will use this function to roll out updated models without creating new ML instances.

As you set up and configure these functions, you will realize they are reusable building blocks, each handling a specific part of the deployment process. When combined, they form the full deployment logic used in the pipeline, allowing you to extend it to support end-to-end deployment.

**Note**

While it is generally a good practice to include an additional pipeline step that checks evaluation scores and applies conditional logic based on defined thresholds, this step is intentionally omitted in this chapter for simplicity. In real-world workflows, you would

typically use such a step to gate deployment decisions or trigger automated rollback mechanisms based on evaluation results.

## Setting up a Lambda function that reads a JSON file in S3

We will now create and configure a Lambda function that retrieves a JSON file from Amazon S3 and makes its contents available for subsequent pipeline steps. You will use this function to load initial pipeline step output(s) along with configuration data stored in S3.

*Figure 9.9 — Lambda function that reads a JSON file in S3*

The Lambda function takes an Amazon S3 path as its input parameter and uses it to locate and download the target JSON file. It then reads and parses the contents of the file, returning the extracted JSON values as the output. This step runs between the evaluation step and the step that checks whether an endpoint exists, where it prepares evaluation outputs for downstream deployment logic. This allows subsequent pipeline steps to easily consume structured data stored in S3 without requiring direct file handling logic.

Let's proceed with setting up the Lambda function:

1. Switch to the browser tab where the AWS Management Console is open. Navigate to the Lambda console by typing `lambda` in the AWS Management Console search bar, and then selecting **Lambda** from the list of search results.

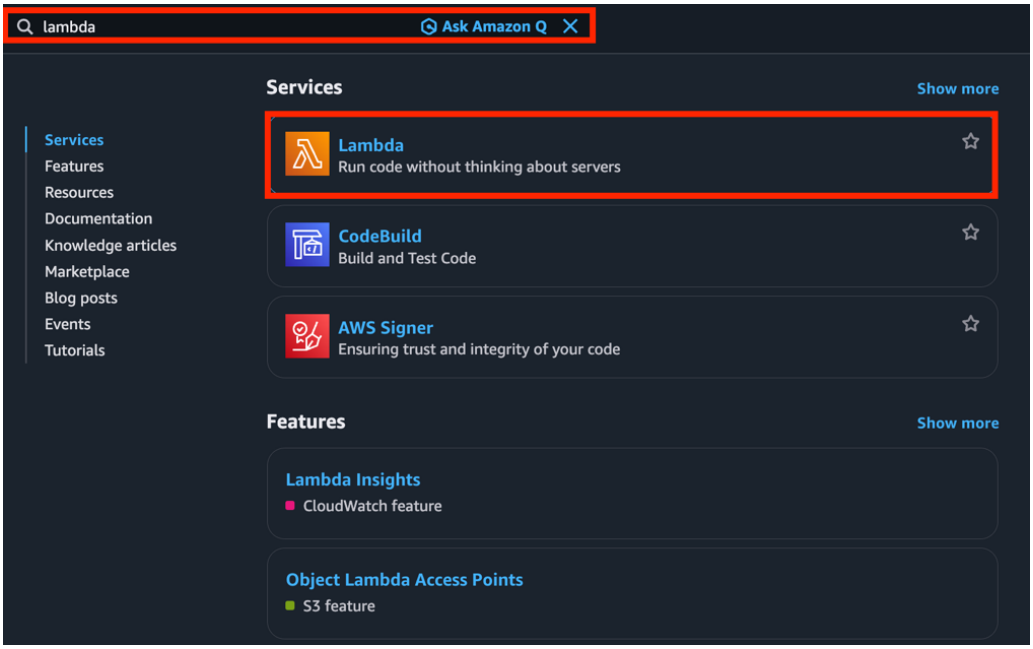



*Figure 9.10 — Navigating to the AWS Lambda console*

This will redirect you to the AWS Lambda console interface where you can create and manage functions that you'll incorporate into the pipeline.

2. Locate and click the **Create function** button (in the upper-right corner of the **Functions** page) to open the **Create function** page (*Figure 9.11*).

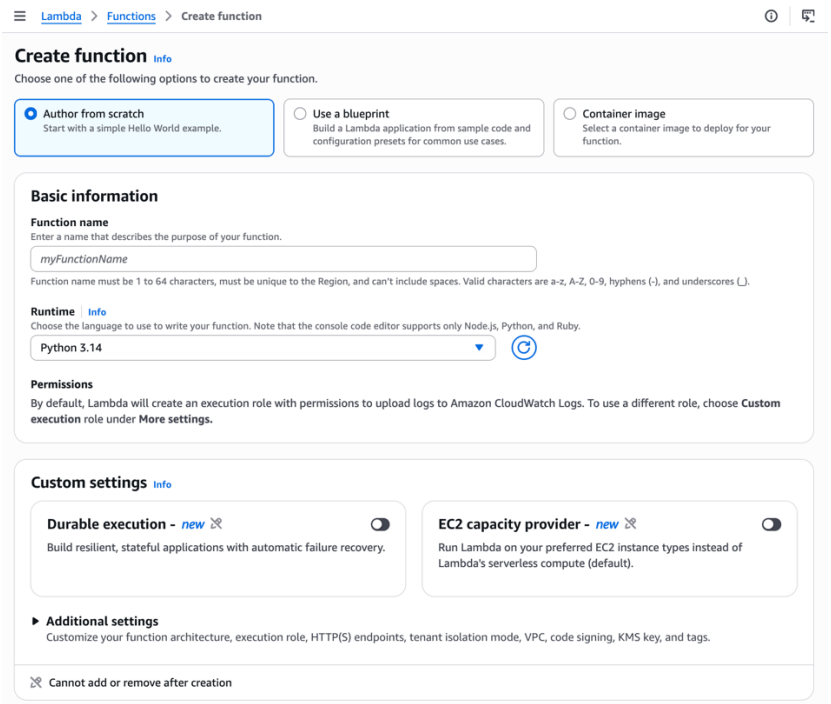



*Figure 9.11 — Create function*

On the **Create function** page, select **Author from scratch** as the creation option. Then, under **Basic information**, specify `s3-json-reader` as the function name and select `Python 3.14` (or higher) as the runtime.

3. Scroll to the bottom of the page and click **Create function** to create the Lambda function.
4. Navigate to the **Configuration** tab, and under **General configuration**, click **Edit**, which will open the **Edit basic settings** page. Specify the following configuration values: **Memory**: `512` MB and **Timeout**: `1` min `0` sec, then click **Save**.
5. Click the **Permissions** tab, and then click the **Role name** link to open the associated IAM role in a new browser tab. Attach the `AmazonS3ReadOnlyAccess` policy to the role before proceeding.

> **Note**
>
> You can do this by selecting **Add permissions**, choosing **Attach policies**, and then searching for and selecting `AmazonS3ReadOnlyAccess` before saving the changes.

6. Open a new browser tab and navigate to the following link: `https://github.com/PacktPublishing/Machine-Learning-Engineering-on-AWS-Second-Edition/tree/main/chapter09`. Locate the *s3-json-reader.py* file and open it to review the code. Click the **Raw** button to display the raw source code, then copy the entire Lambda function code.
7. Return to the Lambda console tab and navigate to the Code tab. Delete the boilerplate code currently in `lambda_function.py` and replace it with the copied code. Click the **Deploy** button.
8. Click the **Test** button. In the **Configure test event** pop-up window, enter `event-000` as the Event name, and provide the following JSON payload under Event JSON.

```
{
  "s3_path": "<S3 PATH>"
}
```

Replace `<S3 PATH>` with the S3 path of the *training_results.json* file that you copied into your text editor (`TAG: JSON_S3_PATH`).

> **Note**
>
> The format of `<S3_PATH>` should follow the structure `s3://<bucket-name>/<path>/training_results.json`.

9. Click **Save**. Click **Invoke** to invoke the Lambda function with the test payload. This will output logs and a response object, similar to the following.

```
Status: Succeeded
Test Event Name: event-000

Response:
{
```

```
    "region": "us-east-1",
    ...
    "model_arn": "arn:.../model-...",
    "s3_model_artifacts": "s3://.../output/model",
    "s3_model_path": "s3://.../model/model-....tar.gz",
    ...
}
```

**Note**

Copy the `model_arn` value from the response output and save it to a text editor on your local machine (`TAG: MODEL_ARN`). You will use this value in subsequent steps of the pipeline for deploying the fine-tuned model.

10. Scroll up to the **Function overview pane** and copy the **Function ARN** value to your text editor (`TAG: S3_JSON_READER_FUNCTION_ARN`).

Perfect, 1 down, 3 more to go! With the `s3-json-reader` Lambda function in place, you now have a reusable component that enables the pipeline to retrieve and parse structured data directly from Amazon S3. In the next part, you will continue preparing the deployment workflow prerequisites by setting up a Lambda function that determines whether a SageMaker AI endpoint already exists.

## Setting up the Lambda function for checking if an endpoint exists

We will now create and configure a Lambda function that checks whether a SageMaker AI endpoint already exists. This function will be used as part of the deployment decision logic within the pipeline to determine how the fine-tuned model should be deployed.

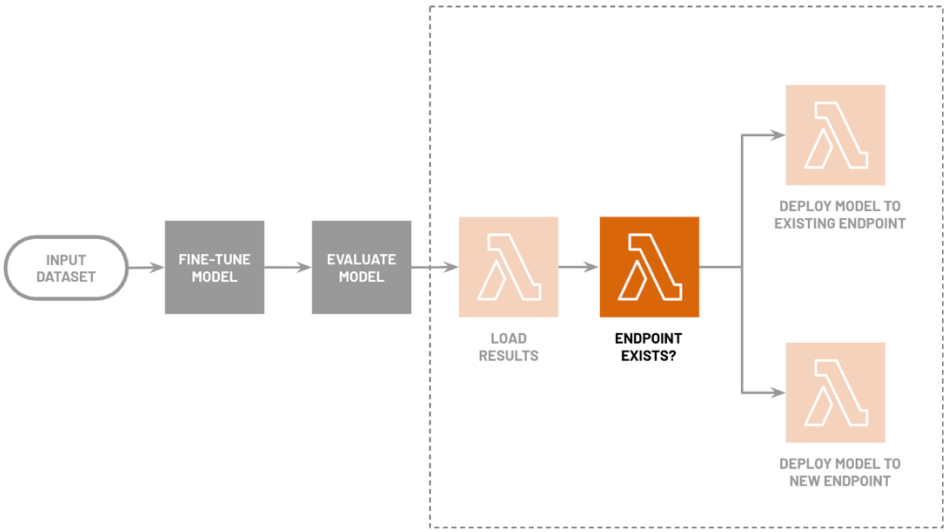



*Figure 9.12 — Lambda Function for checking whether an endpoint exists*

The Lambda function takes an endpoint name as one of its input parameters and checks whether an endpoint with that name is currently running. It returns a simple response indicating whether the endpoint exists, along with any relevant metadata needed for downstream decision-making. This step is expected to run after the evaluation step and after the JSON results file is loaded, and before the actual model deployment step(s). It helps guide the pipeline toward either creating a new endpoint or updating an existing one based on the evaluation outputs.

Let's proceed with setting up the Lambda function:

1. Return to the **Functions** page in the AWS Lambda console and click **Create function**.
2. On the **Create function** page, select **Author from scratch** as the creation option. Then, under **Basic information**, specify `check-if-endpoint-exists` as the function name and select `Python 3.14` (or higher) as the runtime. Scroll to the bottom of the page and click **Create function** to create the Lambda function.
3. Navigate to the **Configuration** tab, and under **General configuration**, click **Edit**, which will open the **Edit basic settings** page. Specify the following configuration values: **Memory**: `512` MB and **Timeout**: `1` min `0` sec, then click **Save**.

4. Click the **Permissions** tab, and then click the **Role name** link to open the associated IAM role page in a new browser tab. Attach the `AmazonSageMakerReadOnly` policy to the role before proceeding.

   > **Note**
   >
   > You can do this by selecting **Add permissions**, choosing **Attach policies**, and then searching for and selecting `AmazonSageMakerReadOnly` before saving the changes.

5. Open a new browser tab and navigate to the following link: `https://github.com/PacktPublishing/Machine-Learning-Engineering-on-AWS-Second-Edition/tree/main/chapter09`. Locate the *check-if-endpoint-exists.py* file and open it to review the code. Click the **Raw** button to display the raw source code, then copy the entire Lambda function code.
6. Return to the Lambda console tab and navigate to the **Code** tab. Delete the boilerplate code currently in `lambda_function.py` and replace it with the copied code. Click the **Deploy** button.
7. Click the **Test** button. In the **Configure test event** pop-up window, enter `event-000` as the Event name and provide the following JSON payload under Event JSON.

   ```
   {
     "endpoint_name": "endpoint-100"
   }
   ```

8. Click **Save**. Click **Invoke** to invoke the Lambda function with the test payload. This will output logs and a response object, similar to the following.

   ```
   Status: Succeeded
   Test Event Name: event-000

   Response:
   {
     "endpoint_exists": false
   }
   ```

9. Scroll up to the **Function overview pane** and copy the **Function ARN** value to your text editor (`TAG: ENDPOINT_EXISTS_FUNCTION_ARN`).

Great! 2 down, 2 to go! With the `check-if-endpoint-exists` Lambda function in place, you now have a reusable component that lets the pipeline check whether a SageMaker AI endpoint already exists. In the next part, you will continue working on the prerequisite resources by setting up the Lambda function that deploys the model to a new endpoint.

## Setting up the Lambda function for deploying a model to a new endpoint

We will now create and configure a Lambda function that deploys the fine-tuned model to a new SageMaker AI endpoint when no existing endpoint is available. This function is used as part of the pipeline's deployment logic to provision infrastructure for serving the model.

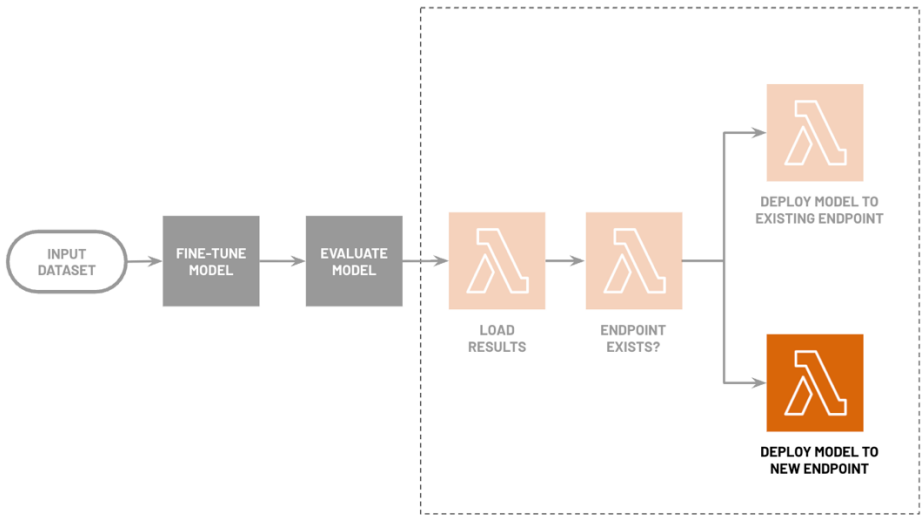



*Figure 9.13 — Lambda function for deploying a model to a new endpoint*

The Lambda function takes the model artifact location and deployment configuration as input parameters and creates a new SageMaker AI endpoint to host the fine-tuned model. This step is expected to run when the pipeline determines that no existing endpoint is available.

Let's proceed with setting up the Lambda function:

1. Return to the **Functions** page in the AWS Lambda console and click **Create function**.
2. On the **Create function** page, select **Author from scratch** as the creation option. Then, under **Basic information**, specify `deploy-model-to-new-endpoint` as the function name and select `Python 3.14` (or higher) as the runtime. Scroll to the bottom of the page and click **Create function** to create the Lambda function.

3. Navigate to the **Configuration** tab, and under **General configuration**, click **Edit**, which will open the **Edit basic settings** page. Specify the following configuration values: **Memory**: `1024` MB and **Timeout**: `15` min `0` sec, then click **Save**.
4. Click the **Permissions** tab, and then click the **Role name** link to open the associated IAM role page in a new browser tab. Attach the `AmazonSageMakerFullAccess` policy to the role before proceeding.

> **Note**
>
> You can do this by selecting **Add permissions**, choosing **Attach policies**, and then searching for and selecting `AmazonSageMakerFullAccess` before saving the changes. Note that this policy grants broad permissions to SageMaker resources and is generally considered overly permissive, so it should be used only for development or learning purposes, not in production environments.

5. Open a new browser tab and navigate to the following link: `https://github.com/PacktPublishing/Machine-Learning-Engineering-on-AWS-Second-Edition/tree/main/chapter09`. Locate the *deploy-model-to-new-endpoint.py* file and open it to review the code. Click the **Raw** button to display the raw source code, then copy the entire Lambda function code.
6. Return to the Lambda console tab and navigate to the **Code** tab. Delete the boilerplate code currently in `lambda_function.py` and replace it with the copied code. Click the **Deploy** button.
7. Click the **Test** button. In the **Configure test event** pop-up window, enter `event-000` as the Event name, and provide the following JSON payload under Event JSON.

```
{
  "endpoint_name": "endpoint-100",
  "model_arn": "<MODEL ARN>",
  "instance_type": "ml.g5.4xlarge"
}
```

> **Note**
>
> Replace `<MODEL ARN>` with the **Amazon Resource Name** (**ARN**) of the fine-tuned model you want to deploy. Use the value you previously copied and stored in your text editor (`TAG: MODEL_ARN`).

8. Click **Save**. Click **Invoke** to invoke the Lambda function with the test payload.
9. Scroll up to the **Function overview pane** and copy the **Function ARN** value to your text editor (`TAG: DEPLOY_NEW_FUNCTION_ARN`).

Awesome! 3 down, 1 to go! With the *deploy-model-to-new-endpoint* Lambda function in place, you now have a building block that allows the pipeline to create a new SageMaker AI endpoint and deploy the fine-tuned model when no endpoint is available. In the next part, you will set up the Lambda function that deploys a fine-tuned model to an existing endpoint.

**Note**

Before proceeding to the next steps, verify that the SageMaker AI endpoint has been created successfully. Navigate to **Endpoints** (under **Deployments & inference**) in the SageMaker AI console navigation pane (left sidebar) and ensure that the endpoint status is listed as **InService**.

## Setting up the Lambda function for deploying a model to an existing endpoint

We will now create and configure a Lambda function that deploys the fine-tuned model to an existing SageMaker AI endpoint. This function is used as part of the pipeline's deployment logic when an endpoint with the specified name already exists and needs to be updated with a new model version.

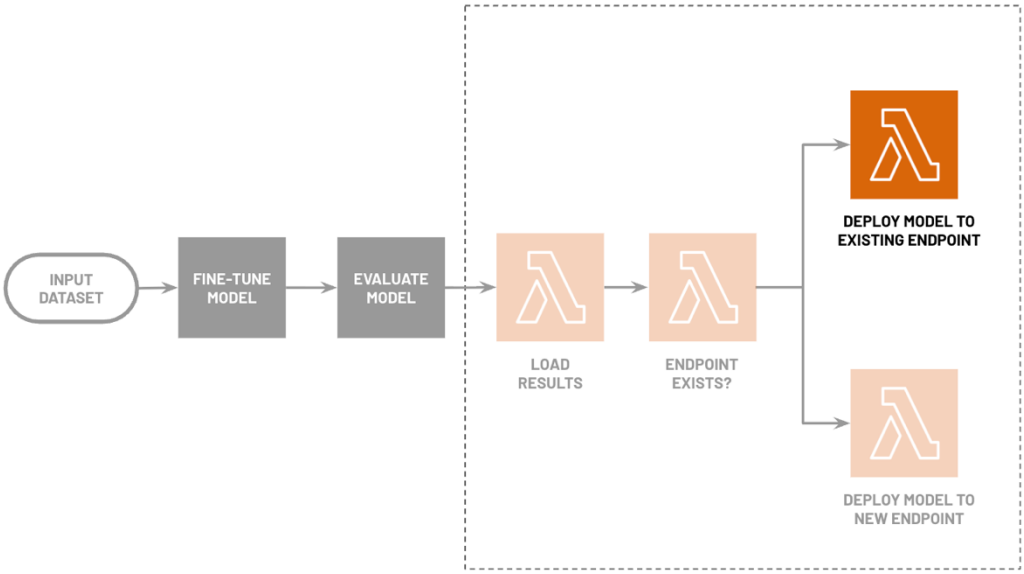



*Figure 9.14 — Lambda function for deploying a model to an existing endpoint*

The Lambda function takes the model artifact location and the target endpoint name as input parameters, and updates the existing SageMaker AI endpoint with the new model version. This step is expected to run when the pipeline detects that an active endpoint already exists and ensures that there is only one active endpoint even across multiple pipeline runs.

Let's proceed with setting up the Lambda function:

1. Return to the **Functions** page in the AWS Lambda console and click **Create function**.
2. On the **Create function** page, select **Author from scratch** as the creation option. Then, under **Basic information**, specify `deploy-model-to-existing-endpoint` as the function name and select `Python 3.14` (or higher) as the runtime. Scroll to the bottom of the page and click **Create function** to create the Lambda function.
3. Navigate to the **Configuration** tab, and under **General configuration**, click **Edit**, which will open the **Edit basic settings** page. Specify the following configuration values: **Memory**: `1024` MB and **Timeout**: `15` min `0` sec, then click **Save**.
4. Click the **Permissions** tab, and then click the **Role name** link to open the associated IAM role page in a new browser tab. Attach the `AmazonSageMakerFullAccess` policy to the role before proceeding.

> **Note**
>
> You can do this by selecting **Add permissions**, choosing **Attach policies**, and then searching for and selecting `AmazonSageMakerFullAccess` before saving the changes. Note that this policy grants broad permissions to SageMaker resources and is generally considered overly permissive, so it should be used only for development or learning purposes, not in production environments.

5. Open a new browser tab and navigate to the following link: `https://github.com/PacktPublishing/Machine-Learning-Engineering-on-AWS-Second-Edition/tree/main/chapter09`. Locate the *deploy-model-to-existing-endpoint.py* file and open it to review the code. Click the **Raw** button to display the raw source code, then copy the entire Lambda function code.
6. Return to the Lambda console tab and navigate to the Code tab. Delete the boilerplate code currently in `lambda_function.py` and replace it with the copied code. Click the **Deploy** button
7. Click the **Test** button. In the **Configure test event** pop-up window, enter `event-000` as the Event name, and provide the following JSON payload under Event JSON.

```
{
  "endpoint_name": "endpoint-100",
  "model_arn": "<MODEL ARN>",
  "instance_type": "ml.g5.4xlarge"
}
```

> **Note**
>
> Replace `<MODEL ARN>` with the ARN of the fine-tuned model you want to deploy. Use the value you previously copied and stored in your text editor (`TAG: MODEL_ARN`).

8. Click **Save**. Click **Invoke** to invoke the Lambda function with the test payload.
9. Scroll up to the **Function overview pane** and copy the **Function ARN** value to your text editor (`TAG: DEPLOY_EXISTING_FUNCTION_ARN`).

With the *deploy-model-to-existing-endpoint* Lambda function in place, you now have a building block that allows the pipeline to update an existing SageMaker AI endpoint with the latest

fine-tuned model version. With this final deployment function configured, you can now proceed with constructing the end-to-end pipeline that ties together fine-tuning, evaluation, and deployment.

> **Note**
>
> Make sure to delete the SageMaker AI real-time inference endpoint created in this section before moving on to the next section.

## Completing the LLMOps pipeline

So far, the last pipeline you created in this chapter was a two-step pipeline that performed supervised fine-tuning followed by evaluation. This helped you understand how to connect training and evaluation stages into a single automated workflow. In this section, you will complete the end-to-end LLMOps workflow by building on the two-step pipeline you defined earlier and extending it with additional deployment steps using the Lambda functions you created and configured in the previous section.

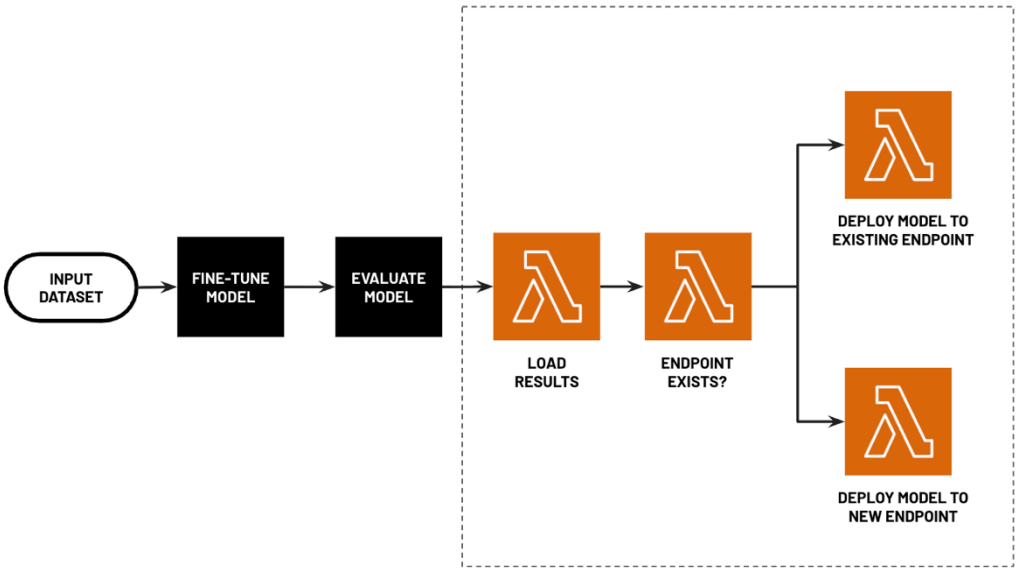



*Figure 9.15 — The complete pipeline flow for the end-to-end LLMOps workflow*

Similar to what is shown in *Figure 9.15*, you will add the **ReadS3JSON**, **CheckIfEndpointExists**, **DeployToExistingEndpoint**, and **DeployToNewEndpoint** steps to the existing pipeline. These Lambda-based steps retrieve model artifacts from Amazon S3, check whether an endpoint exists or not, and use it to decide whether to update an existing endpoint or create a new one. Together, they extend the pipeline into a fully automated end-to-end LLMOps workflow.

> **Note**
>
> The code in this section will be run inside the *complete_pipeline.ipynb* notebook. You will build on top of the existing pipeline defined in the previous sections by adding additional deployment steps and orchestration logic.

Let's proceed with completing the pipeline:

1. Switch back to the browser tab where the *complete_pipeline.ipynb* notebook is already open and scroll to the bottom of the notebook to continue.
2. Import the relevant classes required to integrate AWS Lambda functions into SageMaker Pipelines and manage conditional deployment logic.

```
from sagemaker.mlops.workflow.lambda_step import (
    LambdaStep,
    LambdaOutput,
    LambdaOutputTypeEnum
)

from sagemaker.core.workflow.conditions import (
    ConditionEquals
)

from sagemaker.mlops.workflow import (
    ConditionStep
)

from sagemaker.core.workflow.functions import (
    JsonGet
)

from sagemaker.core.lambda_helper import (
```

```
    Lambda
)
```

3. Define a pipeline parameter for the SageMaker endpoint name to make the endpoint name configurable across different pipeline executions.

```
ENDPOINT_NAME = "endpoint-100"

endpoint_name_param = ParameterString(
    name="EndpointName",
    default_value=ENDPOINT_NAME
)
```

In this section, the first pipeline run creates the endpoint, while all subsequent runs update the existing endpoint.

4. Define a `LambdaStep` that loads the pipeline results artifact from Amazon S3 and makes the `model_arn` value available to subsequent deployment steps.

```
json_reader_lambda = LambdaStep(
    name="ReadS3JSON",
    lambda_func=Lambda(
        function_arn="<SPECIFY LAMBDA ARN>"
    ),
    inputs={
        "s3_path": Join(
            on="",
            values=[ts3uri, "results.json"]
        ),
    },
    outputs=[
        LambdaOutput(
            output_name="model_arn",
            output_type=LambdaOutputTypeEnum.String
        )
    ],
    depends_on=[eval_step]
)
```

> **Note**
>
> Make sure to replace `<SPECIFY LAMBDA ARN>` with the actual ARN of the **s3-json-reader** Lambda function deployed in your AWS account (`TAG: S3_JSON_READER_FUNCTION_ARN`). This ensures the pipeline step correctly invokes the function that reads the results artifact from Amazon S3.

5. Create a Lambda-based workflow step that checks whether the specified SageMaker AI endpoint already exists prior to deployment.

```
endpoint_exists_lambda = LambdaStep(
    name="CheckIfEndpointExists",
    lambda_func=Lambda(
        function_arn="<SPECIFY LAMBDA ARN>"
    ),
    inputs={
        "endpoint_name": endpoint_name_param
    },
    outputs=[
        LambdaOutput(
            output_name="endpoint_exists",
            output_type=LambdaOutputTypeEnum.Boolean
        )
    ],
    depends_on=[json_reader_lambda]
)
```

> **Note**
>
> Make sure to replace `<SPECIFY LAMBDA ARN>` with the actual ARN of the **check-if-endpoint-exists** Lambda function deployed in your AWS account (`TAG: ENDPOINT_EXISTS_FUNCTION_ARN`). This ensures the pipeline step correctly invokes the function that checks whether a SageMaker AI endpoint exists before proceeding with deployment.

6. Extract the `model_arn` output from the previous `LambdaStep` and pass it as input to a new deployment step.

```
properties_1 = json_reader_lambda.properties
json_model_arn_1 = properties_1.Outputs['model_arn']

step_lambda_deploy_to_existing_endpoint = LambdaStep(
    name="DeployToExistingEndpoint",
    lambda_func=Lambda(
        function_arn="<SPECIFY LAMBDA ARN>"
    ),
    inputs={
        "endpoint_name": endpoint_name_param,
        "model_arn": json_model_arn_1,
        "instance_type": "ml.g5.4xlarge"
    },
    depends_on=[endpoint_exists_lambda]
)
```

This step invokes a Lambda function that updates an existing SageMaker AI endpoint with the fine-tuned model produced in a previous pipeline step.

**Note**

Make sure to replace `<SPECIFY LAMBDA ARN>` with the actual ARN of the **deploy-model-to-existing-endpoint** Lambda function deployed in your AWS account (`TAG: DEPLOY_EXISTING_FUNCTION_ARN`). This ensures the pipeline step correctly invokes the function that updates an existing SageMaker AI endpoint with the fine-tuned model.

7. Configure a `LambdaStep` that deploys the trained model by creating a new endpoint when no prior endpoint is detected.

```
properties_2 = json_reader_lambda.properties
json_model_arn_2 = properties_2.Outputs['model_arn']

step_lambda_deploy_to_new_endpoint = LambdaStep(
    name="DeployToNewEndpoint",
    lambda_func=Lambda(
        function_arn="<SPECIFY LAMBDA ARN>"
```

```
    ),
    inputs={
        "endpoint_name": endpoint_name_param,
        "model_arn": json_model_arn_2,
        "instance_type": "ml.g5.4xlarge"
    },
    depends_on=[endpoint_exists_lambda]
)
```

**Note**

Make sure to replace `<SPECIFY LAMBDA ARN>` with the actual ARN of the **deploy-model-to-new-endpoint** Lambda function deployed in your AWS account (`TAG: DEPLOY_NEW_FUNCTION_ARN`). This ensures the pipeline step correctly invokes the function that creates a new SageMaker AI endpoint and deploys the fine-tuned model.

8. Set up a conditional step that evaluates endpoint existence and routes the pipeline to either deploy to an existing endpoint or provision a new one accordingly.

```
Outputs = endpoint_exists_lambda.properties.Outputs
left = Outputs['endpoint_exists']

cond_equals = ConditionEquals(
    left=left,
    right=True
)

if_steps = [step_lambda_deploy_to_existing_endpoint]
else_steps = [step_lambda_deploy_to_new_endpoint]

step_endpoint_exists_condition = ConditionStep(
    name="EndpointExists",
    conditions=[cond_equals],
    if_steps=if_steps,
    else_steps=else_steps
)
```

9. Construct the full pipeline definition by combining training, evaluation, and Lambda-based orchestration steps into a single end-to-end workflow.

```
pipeline = Pipeline(
    name=f"CompletePipeline-{unique}",
    steps=[
        train_step,
        eval_step,
        json_reader_lambda,
        endpoint_exists_lambda,
        step_endpoint_exists_condition
    ],
    parameters=[
        role_arn_param,
        train_dataset_param,
        train_output_param,
        mlflow_exp_param,
        mlflow_run_param,
        region_param,
        eval_dataset_param,
        custom_metric_param,
        eval_output_param,
        endpoint_name_param
    ],
    sagemaker_session=pipeline_session,
)
```

10. Use the `upsert()` method to create or update the SageMaker Pipeline definition in the service under the specified IAM role.

```
pipeline.upsert(role_arn=role)
```

**Note**

Since the pipeline already exists, SageMaker AI performs an update rather than creating a new resource.

11. With everything ready, start the pipeline execution using the `start()` method.

```
execution = pipeline.start(
    parameters={
        "Dataset": train_dataset_s3,
        "TrainingOutputPath": train_output_s3,
        "MLflowExperimentName": (
            f"sft-experiment-{unique}"
        ),
        "MLflowRunName": (
            f"run-{unique}"
        ),
        "Region": region,
        "TrainingRoleArn": role,
        "EvalDataset": eval_dataset_s3,
        "CustomMetricPath": custom_metric_s3,
        "EvaluationOutputPath": eval_output_s3,
        "EndpointName": ENDPOINT_NAME
    }
)
```

12. Run the following block of code to continuously monitor the pipeline execution status and wait until the workflow completes.

```
%%time
import time

while True:
    details = execution.describe()
    status = details["PipelineExecutionStatus"]
    print("Status:", status)

    if status != "Executing":
        break

    time.sleep(60)
```

13. Switch to the browser tab where SageMaker Studio is open. In the left sidebar, choose **Pipelines**. On the **Pipelines** page, locate the pipeline that starts with

`CompletePipeline` and click its name under the **Name** column to open the pipeline details page.

14. Under **Executions**, select the most recent pipeline execution to open its details. Wait until the execution has completed before proceeding.

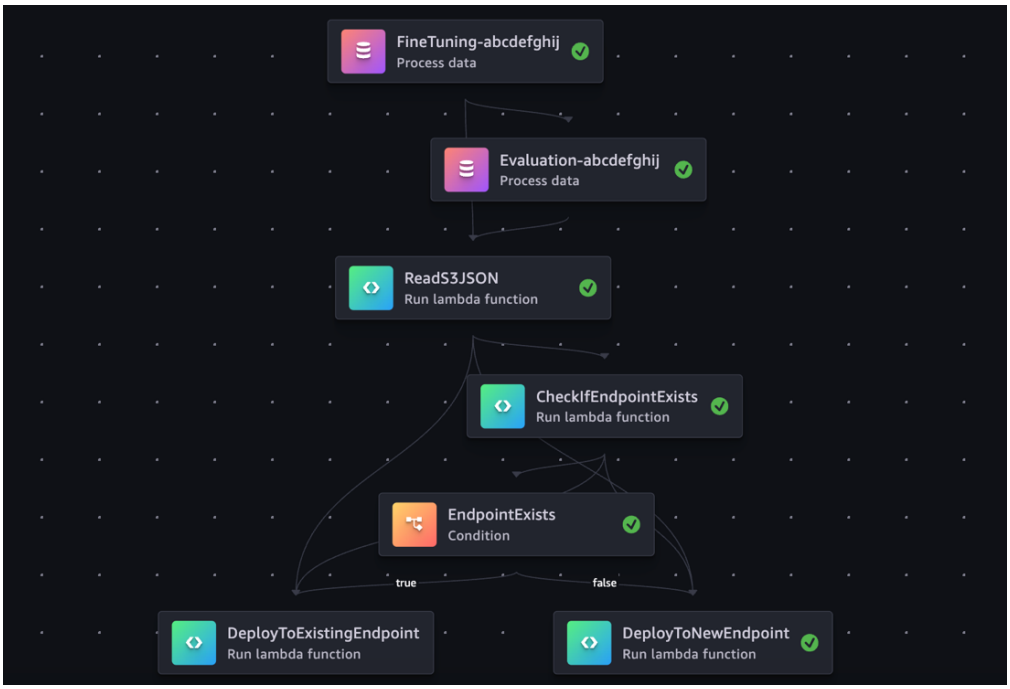



*Figure 9.16 — A pipeline execution that created a new SageMaker AI endpoint and deployed the latest trained model for inference*

Since the endpoint (named `endpoint-100`) does not yet exist, the pipeline ran the **DeployToNewEndpoint** step, which created a new SageMaker AI endpoint and deployed the fine-tuned model.

15. Switch back to the browser tab where the *complete_pipeline.ipynb* notebook is already open and scroll to the bottom of the notebook to continue.

16. Define an `invoke()` function that sends the specified prompt to a deployed SageMaker AI endpoint and returns the model's generated response.

```
import boto3
import json

client = boto3.client("sagemaker-runtime")
```

```
def invoke(prompt, client=client):
    response = client.invoke_endpoint(
        EndpointName=ENDPOINT_NAME,
        ContentType="application/json",
        Body=json.dumps({
            "inputs": prompt,
            "parameters": {"max_new_tokens": 500}
        })
    )

    print(response["Body"].read().decode())
```

17. Let's invoke the endpoint with a sample prompt to generate a response.

```
prompt = """
### Instruction:
Improve the given scientific sentence and provide detailed explanations to
the related questions.

### Input:
- Task 1: Improve the following sentence for clarity and scientific tone:
  "It is notable to mention that various studies have shown that water
serves a key role in the synthesis of vital organic compounds."

- Task 2: Explain the process through which water contributes to the
synthesis of essential organic compounds.

- Task 3: Discuss how the scarcity or abundance of water in an environment
impacts the diversity and functionality of life forms, particularly in
terms of metabolic processes and survival strategies.

### Response:
"""

output = invoke(prompt)
output
```

This should yield the following output:

```
{"generated_text": "1. Improved sentence:\n\"It is notable to mention that
various studies have demonstrated that water plays a crucial role in the
synthesis of vital organic compounds, which are essential for life..."}
```

18. Next, invoke the deployed model again with a prompt that instructs the model to generate a job description using the given role, industry, location, and requirements.

```
prompt = """
### Instruction:
Write a job description.

### Input:
Role: Part-time Finance Manager
Industry: Charity
Location: UK
Requirements: Previous charity experience is essential

### Response:
"""

output = invoke(prompt)
output
```

This should yield an output similar to the following:

```
{"generated_text": "The role of Part-time Finance Manager is a part-time
position within a UK-based charity organization. The successful candidate
will be responsible for managing the financial aspects of the charity,
ensuring the organization's financial stability and growth. Key
responsibilities include overseeing budgeting, financial planning, and
financial reporting..."}
```

19. Scroll up and locate the notebook cell that initiates `pipeline.start()` and re-run it, along with all subsequent cells, to trigger a fresh pipeline execution and validate the updated workflow.

```
execution = pipeline.start(
    ...
)
```

20. Switch to the browser tab where SageMaker Studio is open. In the left sidebar, choose **Pipelines**. On the **Pipelines** page, locate the pipeline that starts with `CompletePipeline` and click its name under the **Name** column to open the pipeline details page.
21. Under **Executions**, select the most recent pipeline execution to open its details. Wait until the execution has completed before proceeding.

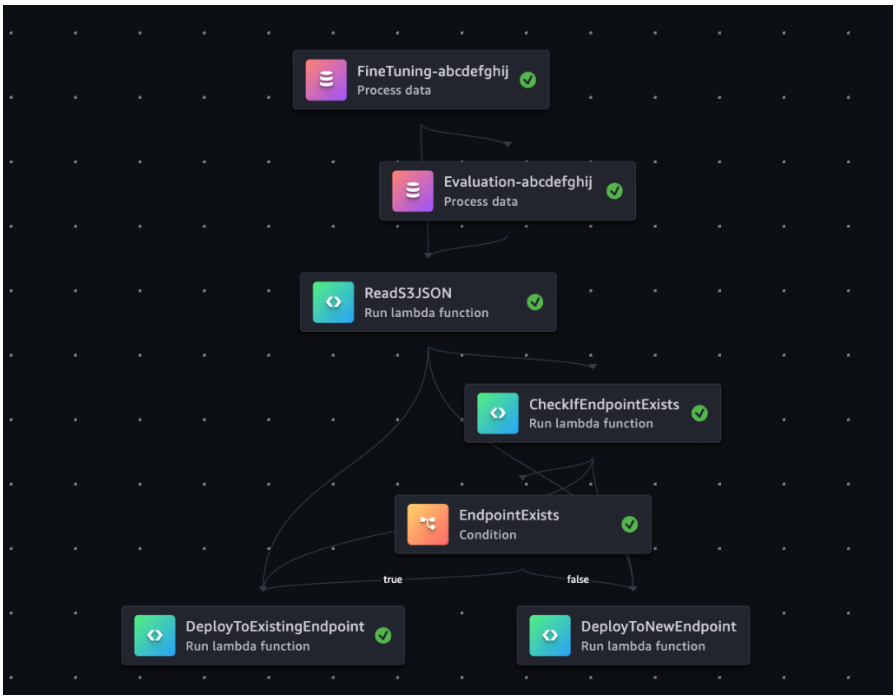



*Figure 9.17 — A pipeline execution that updated an existing SageMaker AI endpoint with the latest trained model*

Since the endpoint (named `endpoint-100`) already exists, the pipeline ran the **DeployToExistingEndpoint** step, which updated the existing SageMaker AI endpoint with the fine-tuned model.

22. Switch back to the browser tab where the *complete_pipeline.ipynb* notebook is already open and scroll to the bottom of the notebook to continue.
23. Finally, delete the endpoint using the following block of code.

```
import boto3

sm = boto3.client("sagemaker")

sm.delete_endpoint(
    EndpointName=ENDPOINT_NAME
)
```

**Note**

Now that you've completed all the hands-on examples in this book, it is important to clean up your environment by stopping and deleting all active AWS resources. Make sure that no endpoints are running and that all resources created in this chapter and the preceding chapter have been stopped or deleted.

Amazing! You have now completed a fully operational LLMOps pipeline that integrates model fine-tuning, evaluation, and deployment using SageMaker Pipelines and AWS Lambda. This workflow demonstrated how modular components can be chained into an end-to-end system that adapts to infrastructure state, such as whether an endpoint already exists. At this stage, you have gained practical knowledge of how to orchestrate ML workflows in production environments, from model training and evaluation through to deployment and inference.

# Best practices and key considerations for building automated ML workflows

When you design, build, and manage automated ML and deployment workflows, it is important to think beyond simply getting the pipeline to run successfully. You also need to consider how the workflow will scale over time, how easy it will be to maintain, and how securely it can operate across different environments. As your workflows become more complex, small design decisions can significantly affect reliability, cost, troubleshooting effort, and long-term operational overhead. In this section, you will focus on both operational and

security best practices to help you build workflows that are not only efficient and scalable, but also secure and well-architected across all stages of the pipeline.

## Operational best practices

Operational best practices focus on how you design and run automated ML workflows in a way that keeps them efficient, modular, scalable, and easy to operate over time. As you move from simple pipelines to more complex, production-grade systems, you need to think about how different components interact, how workloads are managed, and how the system can adapt as requirements evolve.

Here are some important best practices and design considerations you should keep in mind when building and operating automated ML workflows:

- **Use serverless functions for lightweight orchestration and decision logic**: Use serverless compute services such as AWS Lambda as part of your workflow orchestration layer for lightweight tasks such as branching logic, integration steps, post-processing, or conditional evaluation, similar to the pipeline you set up in this chapter.
- **Prepare and maintain custom container images for full runtime control**: When you require full control over dependencies, runtime configurations, security tooling, or specialized libraries across pipeline steps, you should build and maintain custom container images. For example, you might start with a standard base image and progressively extend it by adding the exact Python libraries, system dependencies, and security tools required by your training or inference workloads. This helps you avoid the common *"it works in development but not in production"* problem by ensuring a consistent runtime environment across training, evaluation, and deployment stages. You can also include security scanning tools, system packages, and domain-specific libraries that are not available in standard managed environments, giving you greater flexibility to meet compliance and operational requirements. In practice, these images are typically stored and versioned in **Amazon Elastic Container Registry**, making it easier to manage updates, track versions, and roll back changes when needed. By maintaining versioned container images, you ensure each pipeline step runs in a controlled and reproducible environment while still allowing your workflows to evolve safely.
- **Leverage existing SageMaker Python SDK classes to accelerate pipeline development**: Use the Amazon SageMaker Python SDK and its built-in pipeline components, framework abstractions, and managed integrations whenever possible instead of building custom implementations from scratch. This helps reduce maintenance effort, improve consistency, and accelerate development. For example,

SageMaker Pipeline components such as `ProcessingStep`, `TrainingStep`, and `ModelStep` allow you to define end-to-end workflows without reimplementing common training, evaluation, or deployment logic. This approach also reduces the risk of subtle implementation inconsistencies while ensuring alignment with AWS best practices. In addition, using standardized abstractions makes it easier to onboard new team members and maintain long-term portability across environments.

- **Centralize and version LLM prompt management for consistency and governance**: Treat prompts as managed assets rather than embedding them directly in code, and store them in a centralized, version-controlled system so they can be tracked, tested, and updated independently of pipeline logic. This enables you to monitor prompt evolution over time, compare performance across versions, and roll back when needed. For example, separating prompts used for training data generation, evaluation, and inference allows each to be optimized independently without affecting other workflow stages. This approach improves governance, collaboration, and reproducibility while reducing the risk of inconsistent model behavior caused by unmanaged prompt changes.
- **Implement caching strategies for pipeline steps to improve efficiency and reduce compute cost**: Introduce step-level caching in ML pipelines so that previously executed steps with unchanged inputs do not need to be recomputed. This reduces runtime and compute cost, particularly for expensive stages such as data preprocessing, feature engineering, model training, and evaluation. For example, when a pipeline run is triggered with identical inputs and parameters, cached outputs can be reused instead of re-executing the step, improving iteration speed during experimentation. This approach improves efficiency and ensures deterministic reuse of validated intermediate outputs across runs.
- **Continuously review and optimize pipeline design and execution for performance, cost, and maintainability**: Continuously evaluate pipeline steps, infrastructure usage, and deployment processes to balance performance, maintainability, security, and cost efficiency. As part of this process, identify opportunities to simplify complex workflows by breaking large, tightly coupled pipelines into smaller, modular pipelines with clearly defined responsibilities. This improves testability, observability, and reusability while enabling faster iteration on individual stages. For example, separating data preparation, training, and deployment into distinct pipelines allows each stage to evolve independently without impacting the full workflow. This approach also improves scalability and operational control since each pipeline can be independently optimized, scheduled, and monitored.

- **Optimize compute and infrastructure usage to reduce overall cost**: Optimize cost efficiency across ML pipelines by selecting appropriate compute options, right-sizing resources, and leveraging distributed processing where applicable. For example, you can use **SageMaker Managed Spot Training** for fault-tolerant training workloads, which helps reduce compute costs compared to On-Demand instances while accounting for potential interruptions. In addition, ensure that compute, memory, and storage resources are appropriately sized for each pipeline step to avoid over-provisioning or under-utilization. You can also improve efficiency by applying parallelization strategies for data processing and distributed training where supported, allowing workloads to complete faster while making better use of available infrastructure. This combination of cost-aware instance selection, proper resource allocation, and parallel execution helps maintain performance while minimizing unnecessary operational expenses.
- **Explore alternative approaches for building automated ML workflows beyond standard pipelines**: While pipeline-based workflows are common, you can also design ML systems using other architectural patterns depending on your requirements. For example, you can build event-driven workflows using **AWS Step Functions** or **Amazon EventBridge** to coordinate training, evaluation, and deployment steps without relying on a single linear pipeline. You can also use Kubernetes-based platforms such as Kubeflow or custom orchestration on Kubernetes clusters to manage end-to-end ML workflows with fine-grained control over scheduling, scaling, and resource isolation. In addition, hybrid architectures that combine pipelines with microservices-based components can help decouple inference, feature processing, and monitoring for independent scaling. Some use cases may also benefit from streaming or real-time architectures where model updates and predictions are continuously triggered by incoming data. In practice, many production systems combine multiple patterns depending on latency, complexity, and scalability requirements.

These operational best practices work together to help you build ML workflows that are efficient, maintainable, scalable, and cost-effective over time. By combining automation, modular design, reuse of managed components, caching, containerization, and cost-aware infrastructure choices, you can reduce unnecessary complexity while improving reliability, development speed, and operational efficiency. As your workflows evolve, applying these principles consistently helps you maintain control over performance, cost, and operational overhead without slowing down iteration.

## Security best practices

Security is a critical dimension of any automated ML workflow, alongside performance, cost, and maintainability. As your pipelines handle sensitive data, models, and infrastructure resources, you need to design security controls from the outset rather than treating them as an afterthought. This includes enforcing strong identity and access management, protecting sensitive data throughout the pipeline lifecycle, and clearly defining isolation boundaries between workloads.

The following best practices focus on how you can secure ML pipelines across identity, data, compute, and configuration layers:

- **Enforce fine-grained access control for pipeline resources using least privilege IAM design**: Apply the principle of least privilege by assigning separate IAM roles and permissions for different pipeline stages, including processing jobs, training jobs, deployment steps, and monitoring components, along with the specific AWS resources associated with each stage. Instead of sharing broad or overly permissive roles across unrelated components, define narrowly scoped permissions that allow each pipeline step to access only the resources it explicitly requires. For example, a training job should have access only to training datasets and model artifacts, while deployment roles should be limited to actions such as creating and updating endpoints, and monitoring roles should be restricted to accessing logs and metrics required for model performance tracking. This separation reduces the blast radius of misconfigurations and improves the overall security posture of your ML workflows.
- **Isolate sensitive pipeline operations in dedicated execution environments with controlled access and restricted outbound traffic**: Separate sensitive operations such as model deployment, data preprocessing, and model evaluation into dedicated execution environments with tightly scoped IAM permissions and clearly defined network boundaries, including controlled outbound (egress) access where required. By isolating workloads at the environment level and restricting both inbound and outbound connectivity, you reduce the risk of unintended interaction between pipeline stages and help prevent data exfiltration. Each environment should only have access to the data sources, compute resources, and external services it explicitly requires. This also improves operational clarity by making it easier to reason about data flow and service interactions within each stage, while strengthening compliance posture in regulated environments where data handling and network exposure must be tightly controlled.
- **Protect and manage personally identifiable information (PII) throughout ML pipelines with strict handling controls**: Identify and classify PII early in the pipeline

and apply appropriate safeguards across all stages, including processing, training, evaluation, and deployment. This includes minimizing PII exposure by collecting and processing only the data required for the task, and applying techniques such as masking, tokenization, or anonymization before data is used in downstream workflows. For example, PII should be removed or transformed during data preprocessing so that training and evaluation steps do not directly access raw sensitive attributes. In addition, access to datasets containing PII should be tightly controlled using fine-grained IAM permissions and isolated execution environments to reduce the risk of unauthorized access or leakage. This approach helps ensure compliance with data protection regulations, improves security posture, and reduces the likelihood of sensitive information being exposed during model development or inference.

- **Secure credentials and configuration using managed secret storage solutions with controlled runtime access**: Store secrets, credentials, API keys, and sensitive configuration values in managed secret management services such as AWS Secrets Manager or AWS Systems Manager Parameter Store instead of embedding them in scripts, notebooks, or container images. This ensures sensitive information is centrally managed, encrypted both at rest and in transit, and retrieved only at runtime by authorized components with explicitly defined permissions. For example, pipeline steps can securely retrieve database credentials or API tokens during execution without hardcoding them into application code or environment variables. This approach simplifies credential rotation, reduces operational overhead, and minimizes the risk of accidental exposure through source control, logs, or container artifacts.
- **Implement granular IAM controls, auditing, and governance for pipeline management actions**: Define fine-grained permissions for IAM users, roles, and groups that are responsible for creating, modifying, and executing ML pipelines, ensuring access is aligned with specific responsibilities. For example, separate permissions for pipeline creation, pipeline updates, and pipeline execution help prevent unauthorized or accidental changes to production workflows. In addition, enforce auditing and logging of all pipeline-related actions using services such as **AWS CloudTrail** to maintain traceability of who created or modified pipelines and when changes occurred. This improves governance, supports compliance requirements, and provides visibility into operational activity across the ML lifecycle.

By following these security best practices, you can build ML pipelines that are efficient, scalable, and resilient against misconfiguration, unauthorized access, and data exposure risks. These principles help you maintain a strong security posture across the ML lifecycle while keeping workflows operationally manageable and compliant with organizational and regulatory requirements.

As an ML Engineer, you should consistently apply these best practices across the ML lifecycle, from data preparation and model development through model evaluation, deployment, and monitoring. This means thinking beyond individual pipeline steps and focusing on how design decisions, tooling choices, and architectural patterns work together to create systems that are reliable, scalable, secure, and cost-efficient. By integrating these practices into your day-to-day engineering work, you can build ML workflows that are easier to maintain, easier to evolve, and better aligned with production-grade requirements over time.

# Summary

In this chapter, you learned how to design and operationalize LLMOps pipelines using SageMaker Pipelines. You began with single-step pipelines for fine-tuning and evaluation and then extended this foundation into a multi-step pipeline, which allowed you to explore how individual stages can be composed into structured, automated workflows. As the pipeline evolved, you introduced additional Lambda-based steps to load model artifacts, extract pipeline outputs, and orchestrate deployment logic. Finally, you reviewed key design principles and best practices for building production-grade ML workflows, with an emphasis on scalability, maintainability, security, and cost efficiency.

Congratulations! You've reached the end of the book. Completing all the chapters, including the hands-on examples and solutions, is a significant milestone. Throughout this journey, you explored how to design, build, and operationalize ML workflows from the ground up, moving from foundational concepts to more advanced, production-oriented practices. If you found this book useful, consider sharing it with others who may benefit from it as well.

# Further reading

If you would like to explore these topics further, refer to the *Further reading* sections in the earlier chapters. You can also supplement your understanding with the resources listed below:

- *Amazon SageMaker AI | Developer Guide | Pipelines* (`https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html`)
- *SageMaker Python SDK V3 | Implement MLOps* (`https://sagemaker.readthedocs.io/en/v3docs/ml_ops/`)
- *AWS Blogs | LLM-as-a-judge on Amazon Bedrock Model Evaluation* (`https://aws.amazon.com/blogs/machine-learning/llm-as-a-judge-on-amazon-bedrock-model-evaluation/`)

- *AWS Blogs | Amazon Bedrock Prompt Management is now available in GA* (`https://aws.amazon.com/blogs/machine-learning/amazon-bedrock-prompt-management-is-now-available-in-ga/`)
- *Amazon SageMaker AI | Developer Guide | Best practices* (`https://docs.aws.amazon.com/sagemaker/latest/dg/best-practices.html`)
- *AWS Identity and Access Management | User Guide | Security best practices in IAM* (`https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html`)

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

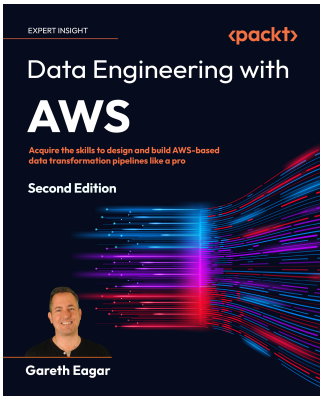
## Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

# Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

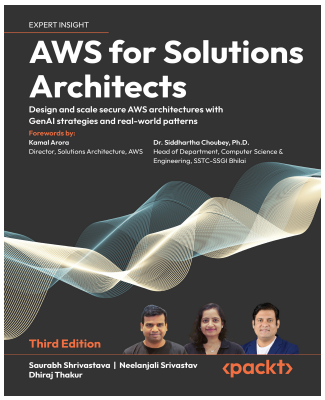



**Data Engineering with AWS, Second Edition**

Gareth Eagar

ISBN: 978-1-80461-442-6

- Seamlessly ingest streaming data with Amazon Kinesis Data Firehose
- Optimize, denormalize, and join datasets with AWS Glue Studio
- Use Amazon S3 events to trigger a Lambda process to transform a file
- Load data into a Redshift data warehouse and run queries with ease
- Visualize and explore data using Amazon QuickSight
- Extract sentiment data from a dataset using Amazon Comprehend
- Build transactional data lakes using Apache Iceberg with Amazon Athena
- Learn how a data mesh approach can be implemented on AWS




**AWS for Solutions Architects, Third Edition**

Saurabh Shrivastava, Neelanjali Srivastav, Dhiraj Thakur

ISBN: 978-1-83664-193-3

- Gain expertise in GenAI, MLOps, AIOps, and mobile edge computing
- Optimize performance and handle errors in cloud-native applications
- Prepare for new AWS certifications, including those focused on AI and data engineering
- Apply best practices for security, compliance, and AWS Learning Badges
- Explore the updated Well-Architected Framework with six pillars
- Get in-depth knowledge of new storage options and security group configurations

## Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packt.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

## Share your thoughts

Now you've finished *Machine Learning Engineering on AWS, Second Edition*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

`https://packt.link/r/1835881092`

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

# Index

## A


**AI agents** **1**
- building, with Amazon Bedrock AgentCore 67 – 69
- building, with Amazon SageMaker AI and Strands Agents 61 – 66
- customizing, to interact with SageMaker AI endpoint 73 – 79
- with Amazon Bedrock and Strands Agents 31 – 45

**AWS CLI**
- used, for running to SQL queries in Amazon Athena 242 – 246

**AWS CloudShell** **196**

**AWS CloudTrail** **553**

**AWS Lake Formation permissions**
- concepts and terminology 238
- working with 237 – 241

**AWS Step Functions** **551**

**Amazon Bedrock** **1, 7**
- AI agents with 31 – 45
- foundation models, exploring in 7 – 14

**Amazon Bedrock AgentCore**
- used, for building AI agents 67 – 69
- working with 69 – 73

**Amazon Bedrock Knowledge Base**
- setting up 86 – 100

**Amazon EMR cluster**
- connecting to 216 – 219
- EC2 instance profile permissions, updating 215, 216
- launching, with Apache Iceberg installed 209 – 211
- setting up, with Apache Iceberg installed 211 – 214

**Amazon Elastic Compute Cloud (EC2)** **210**

**Amazon Elastic Container Registry** **549**

**Amazon Elastic MapReduce (EMR)** **193**

**Amazon EventBridge** **551**

**Amazon Nova Pro** **490**

**Amazon S3 table bucket**
- creating 206 – 209

**Amazon SageMaker AI** **14**
- LLM, fine-tuning on 330 – 350
- reference link 51
- used, for building AI agents 61 – 66
- used, for performing hyperparameter tuning 360 – 378

**Amazon Simple Storage Service (S3)** **195**

**Amazon Virtual Private Cloud** **15**

**Apache Iceberg**
- queries, performing on S3 tables with Apache Spark 219

**Apache Iceberg installed**
- used, for launching Amazon EMR cluster 209 – 211
- used, for setting up Amazon EMR cluster 211 – 214

**Apache Iceberg queries, on S3 tables**
- data, loading from CSV files 221 – 225
- data, loading from Parquet file 225 – 228
- namespace, creating 220
- Spark shell, accessing 219, 220

**Apache Spark**
- used, for performing Apache Iceberg queries on S3 tables 219

**asynchronous inference endpoint**
- model, deploying to 413 – 418

**automated ML workflows**

best practices 548
building, considerations 548
operational best practices 549 – 551
security best practices 552 – 554

# B

**BERT fine-tuning**
setting up, with SageMaker JumpStart 165 – 175
**BERT model fine-tuning job**
running 181 – 185
**Bedrock AgentCore Runtime**
used, for cleaning up agents 85, 86
used, for deploying production-ready agents 80 – 85
**Bidirectional Encoder Representations from Transformers (BERT)** **3, 166**
**Blue/Green deployments**
canary deployment, configuring in SageMaker AI 440 – 451
cleaning up 451, 452
performing, with canary traffic shifting 439
**Boto3**
used, for running to SQL queries in Amazon Athena 246 – 255
**batch inference**
running, with batch transform 410 – 413
**batch transform**
used, for running batch inference 410 – 413

# C

**Claude** **3**
**Comma-Separated Values (CSV) file** **195**
data, loading to S3 tables 221 – 225
inspecting 202 – 205
uploading, to S3 bucket 205, 206
**canary deployment**
configuring, in SageMaker AI 440 – 451
**canary traffic shifting**
using, when performing Blue/Green deployments 439
**cold start** **408**
**custom_metric.json** **459**

# D

**data**
ingesting, into SageMaker feature store 255, 256, 265 – 269
preparing, to ingest into SageMaker feature store 262 – 265
retrieving, from online and offline SageMaker feature store 275 – 284
**data.jsonl** **459**

# E

**EC2 instance profile permissions**
updating 215, 216
**Endpoint Configuration**
Inference Endpoint, setting up from 111 – 113
**eXtreme Gradient Boosting (XGBoost)** **127**
**embeddings** **6**
**entailment** **166**
**evaluation.py** **459**

# F

**fastparquet** **197**
**few-shot learning** **3**
**file mode** **147**
**fine-tuned model**
deploying 351 – 360
deploying, to real-time inference endpoint 186 – 189
**fine-tuning**
smaller dataset, using for 176 – 181
techniques 333
**fine_tuning.py** **459**
**foundation models** **3**
exploring, in Amazon Bedrock 7 – 14

## G

**GPT** **3**

**Generative AI** **1**
- for modern machine learning engineer 3 – 7

**gen_qa.jsonl** **459**

## H

**hyperparameter tuning**
- best-performing model, deploying from 378 – 387
- performing, with Amazon SageMaker AI 360 – 378

## I

**IAM permissions**
- configuring, for SageMaker Studio space 26 – 30

**inference endpoint** **153**
- setting up, from Endpoint Configuration 111 – 113

## J

**JSON file, in S3**
- Lambda function, setting up that reads 524 – 528

**JupyterLab** **17, 127**

**JupyterLab App Launcher** **17**

**JupyterLab notebook** **18 – 20, 127**
- preparing, for model deployment 393 – 400
- setting up 125 – 135

## K

**kernel** **20, 127**

## L

**LLM-as-a-Judge model evaluation**
- evaluation approach, setting up and running 485 – 494

**LLMOps pipeline**
- completing 536 – 548

**Lambda functions**
- preparing, for deployment of model to endpoint 522, 523
- setting up, for checking if endpoint exists 528 – 531
- setting up, for deploying model to existing endpoint 533 – 536
- setting up, for deploying model to new endpoint 531 – 533
- setting up, that reads JSON file in S3 524 – 528

**Large Language Models (LLMs)** **4, 463**
- fine-tuning, on Amazon SageMaker AI 330 – 350

**Low-Rank Adaptation (LoRA)** **349**

## M

**ML workflows**
- project environment and dependencies, setting up 458 – 468

**machine learning (ML) model** **1, 193, 287**
- deploying, to Real-Time inference endpoints 401 – 407

**machine learning engineering (ML engineering)** **1**
- Generative AI, using for 3 – 7

**masked language modeling (MLM)** **166**

**model**
- deploying, to asynchronous inference endpoint 413 – 418
- deploying, to SageMaker AI serverless inference endpoint 407 – 410

**multi-head self-attention** **166**

## N

**Natural language inference (NLI)** **167**

**Not Entailment** **166**

**named entity recognition (NER)** **167**

**next sentence prediction (NSP)** **167**

# P

**Parquet file** **195**
data, loading to S3 tables 225 – 228
inspecting 202 – 205
uploading, to S3 bucket 205, 206
**Provisioned Concurrency**
using 408
**pipe mode** **147**
**pipeline_wrapper.py** **459**
**pretrained LLM**
deploying, with SageMaker AI 49 – 61
**private subnets** **15**
**production-ready agents**
capabilities 85
cleaning up, with Bedrock AgentCore Runtime 85, 86
deploying, with Bedrock AgentCore Runtime 80 – 85
**prompt engineering** **6**
**prompts** **4**
**public subnets** **15**

# Q

**Question Natural Language Inference (QNLI)** **165**

# R

**RAG-powered AI agent**
building 113 – 119
building, to interacts with SageMaker AI inference endpoint 109 – 111
building, with Strands Agents 100 – 109
cleaning up 119, 120
Inference Endpoint, setting up from Endpoint Configuration 111 – 113
**Read-Eval-Print-Loop (REPL)** **197, 247**
**Real-Time inference endpoints**
ML models, deploying to 401 – 407
**retrieval-augmented generation (RAG)** **6, 165**

# S

**S3 bucket**
CSV and Parquet files, uploading to 205, 206
**S3 tables**
Apache Spark, used for performing Apache Iceberg queries on 219
time travel queries, performing 228 – 233
**SQL queries, in Amazon Athena**
running 241
running, with AWS CLI 242 – 246
running, with Boto3 246 – 255
**SageMaker AI** **7**
canary deployment, configuring 440 – 451
used, for deploying pretrained LLM 49 – 61
**SageMaker AI Domain** **15, 125**
**SageMaker AI inference endpoint**
AI agents, customizing to interact with 73 – 79
RAG-powered AI agent, building to interact with 109 – 111
**SageMaker AI serverless inference endpoint**
model, deploying to 407 – 410
**SageMaker Clarify** **292**
**SageMaker Experiments** **15**
**SageMaker HyperPod** **15**
**SageMaker JumpStart**
used, for setting up BERT fine-tuning 165 – 175
**SageMaker Managed Spot Training** **551**
**SageMaker Model Monitor** **15**
**SageMaker Pipelines** **15**
**SageMaker Processing jobs** **289 – 292**
concepts and terminology 289, 290
data preparation and ML workflow tasks 291

input data and processing 302 – 307
script, preparing for back translation job
running 293 – 302
used, for automating back translation workflows 307 – 319

**SageMaker Python SDK 49, 126**
documentation link 133

**SageMaker Python SDK V3 333**

**SageMaker Studio 15, 125**
environment, setting and configuring 14 – 25
space 16

**SageMaker Studio space 126**
IAM permissions, configuring for 26 – 30

**SageMaker Training 15**

**SageMaker feature store**
advantages 256
data, ingesting into 255, 256, 265 – 269
data, preparing to ingest into 262 – 265
data, retrieving from online and offline 275 – 284
notebook instance, setting up 257 – 262
searchable metadata, adding to 269 – 275

**Single-Step evaluation pipeline**
execution logs, inspecting 501 – 503

**Stable Diffusion 3**

**Strands Agents 1**
AI agents with 31 – 45
used, for building AI agents 61 – 66
used, for building RAG-powered AI agent 100 – 109

**searchable metadata**
adding, to SageMaker feature store 269 – 275

**secure shell (SSH) 217**

**serverless MLflow app**
setting up 322 – 329

**shadow test, with SageMaker inference endpoint**
captured data, inspecting 431 – 435
cleaning up 438, 439
creating 420 – 431
setting up 418, 419
variant, promoting 435 – 437

**single-dtep fine-tuning pipeline**
execution logs, inspecting 480 – 484

**single-step evaluation pipeline**
building and running 484, 485, 495 – 501
LLM-as-a-Judge model evaluation, setting up and running 485 – 494

**single-step fine-tuning pipeline**
building and running 469 – 480

**synthetic data**
preparing and processing 195, 196
terms and ideas 195, 196

**synthetic dataset**
preparing, for binary classification 135 – 146

**synthetic tabular data**
downloading and preparing 196 – 202

**system prompt 4**

# T

**Tom's Obvious Minimal Language (TOML) 68**

**token 5**

**two-step fine-tuning pipeline**
building and running 505 – 517
configuring and running 504
execution logs, inspecting 517 – 521

# U

**Universally Unique Identifier (UUID) 200**

**uv 69**

# X

**XGBoost binary classifier**
training 147 – 153

**XGBoost model**
deploying, to real-time inference endpoint 153 – 165

JupyterLab notebook, preparing for deploying 393 – 400

## Z

**zero-shot 3**